

TestMAX™ ATPG and TestMAX Diagnosis Commands

Version P-2019.03-SP4, September 2019

SYNOPSYS®

Copyright Notice and Proprietary Information

Copyright © 2019 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <http://www.synopsys.com/Company/Pages/Trademarks.aspx>. All other product or company names may be trademarks of their respective owners. Inc.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
690 E. Middlefield Road
Mountain View, CA 94043
www.synopsys.com

Contents

Add Commands	12
add_atpg_primitives	13
add_atpg_constraints	15
add_capture_masks	18
add_cell_constraints	21
add_chain_masks	24
add_clocks	27
add_delay_paths	32
add_display_gates	33
add_display_proc	35
add_distributed_processors	36
add_equivalent_nofaults	38
add_faults	39
add_lbist_seeds	45
add_net_connections	47
add_nofaults	50
add_pi_constraints	52
add_pi_equivalences	54
add_po_masks	57
add_scan_chains	59
add_scan_enables	61
add_slow_bidis	63
add_slow_cells	64

add_waveform_signals	65
Analyze Commands	66
analyze_buses	67
analyze_compressors	70
analyze_faults	73
analyze_feedback_path	82
analyze_nonscan_loading	84
analyze_simulation_data	86
analyze_test_points	90
analyze_violation	93
analyze_wires	94
License Commands	95
get_licenses	96
remove_licenses	98
report_licenses	99
Read Commands	100
read_cell_model	101
read_drc	102
read_faults	103
read_image	106
read_layout	108
read_memory_files	111
read_netlist	113
read_nofaults	116
read_sdc	117
read_timing	118
Remove Commands	119
remove_atpg_constraints	120
remove_atpg_primitives	122
remove_capture_masks	123

remove_cell_constraints	125
remove_clocks	126
remove_compressors	128
remove_delay_paths	129
remove_display_gates	130
remove_distributed_processors	131
remove_faults	132
remove_lbist_seeds	138
remove_licenses	139
remove_net_connectionss	140
remove_nofaults	141
remove_physical	143
remove_pi_constraints	145
remove_pi_equivalences	146
remove_po_masks	147
remove_scan_chains	148
remove_scan_enables	149
remove_sdc	151
remove_slow_bidis	152
remove_slow_cells	153
remove_waveform_signals	154
Report Commands	155
report_atpg_constraints	157
report_atpg_primitives	159
report_buses	161
report_capture_masks	166
report_cell_constraintss	167
report_clocks	168
report_compressors	172
report_delay_paths	177

report_display_gates	179
report_distributed_processors	180
report_fanin	181
report_fanout	183
report_faults	184
report_feedback_paths	194
report_instances	196
report_layout	198
report_lbist_seeds	201
report_licenses	202
report_lockup_latches	203
report_memory	207
report_modules	210
report_net_connections	214
report_nets	215
report_nofaults	217
report_pi_constraints	219
report_nonscan_cells	220
report_patterns	223
report_physical	228
report_pi_equivalences	231
report_pin_data	232
report_pin_name	233
report_po_masks	234
report_primitives	235
report_power	240
report_rules	243
report_scan_ability	245
report_scan_cells	246
report_scan_chains	250

report_scan_enables	252
report_scan_path	253
report_sdc	256
report_serializers	258
report_settings	260
report_slow_bidis	267
report_slow_cells	268
report_summaries	269
report_timing	276
report_toggle_weights	278
report_version	279
report_violations	281
report_wires	283
Run Commands	285
run_atpg	286
run_build_model	294
run_diagnosis	296
run_drc	305
run_fault_sim	309
run_justification	312
run_observe_analysis	314
run_simulation	316
run_testpoint_analysis	322
Set Commands	324
set_atpg	325
set_build	342
set_buses	348
set_colors	350
set_commands	352
set_contention	354

set_delay	358
set_diagnosis	365
set_distributed	378
set_drc	380
set_environment	394
set_environment_gui	395
set_environment_info	397
set_environment_reports	398
set_environment_transcript	400
set_environment_viewer	401
set_faults	405
set_iddq	409
set_learning	411
set_match_names	414
set_messages	418
set_netlist	420
set_patterns	423
set_physical	430
set_physical_db	431
set_pindata	434
set_primitive_report	442
set_random_patterns	444
set_rules	445
set_scan_ability	448
set_sdc	450
set_simulation	452
set_toggle_weights	461
set_wgl	463
set_workspace_sizes	468
set_ydf	470

Shell Commands	472
cat	473
cd	474
clear	475
cp	476
ls	477
mkdir	479
mv	480
pwd	481
rm	482
system	483
Tcl API Commands	485
add_display_proc	486
all_clocks	487
all_inputs	488
all_outputs	489
all_registers	490
filter_collection	491
foreach_in_collection	494
get_attribute	496
get_candidates	498
get_cells	502
get_clocks	506
get_diag_files	509
get_fanin	512
get_fanouts	513
get_faults	514
get_instances	516
get_modules	517
get_patterns	519

get_pins	521
get_ports	524
get_primitive	526
get_scan_chains	530
get_settings	532
get_timing	533
list_attributes	535
remove_display_procs	537
Update Commands	538
update_clock	539
update_faults	541
update_scale	543
update_streaming_patterns	544
update_wft	547
Write Commands	549
write_drc_file	550
write_faults	556
write_image	559
write_netlist	562
write_patterns	564
write_physical_db	574
write_simtrace	576
write_testbench	578
write_ydf	581
Other Commands	588
alias	589
build	591
close_physical_db	592
define_proc_attributes	593
drc	597

exit	598
extract_nets	599
extract_pins	601
gsv_print	602
gui_start	603
gui_stop	604
help	605
man	607
match_names	609
open_physical_db	611
parse_proc_arguments	612
Predefined Aliases	614
quit	615
refresh_schematic	616
reorder_patterns	617
reset_state	619
source	621
test	622
unalias	623

Add Commands

This section describes the following commands:

- [add_atpg_constraints](#)
- [add_lbist_seeds](#)
- [add_atpg_primitives](#)
- [add_nofaults](#)
- [add_capture_masks](#)
- [add_net_connections](#)
- [add_cell_constraints](#)
- [add_pi_constraints](#)
- [add_chain_masks](#)
- [add_pi_equivalences](#)
- [add_clocks](#)
- [add_po_masks](#)
- [add_delay_paths](#)
- [add_scan_chains](#)
- [add_display_gates](#)
- [add_scan_enables](#)
- [add_display_proc](#)
- [add_slow_bidis](#)
- [add_distributed_processors](#)
- [add_slow_cells](#)
- [add_equivalent_nofaults](#)
- [add_waveform_signals](#)

add_atpg_primitives

Overview

Use this command to create logical functions that need to be constrained. Constraining these functions might be useful in avoiding undesirable conditions in patterns generated by the ATPG process.

Syntax

```
add_atpg_primitives
  name <and | or | sel1 | sel01 | equiv>
  [-module name] {input_connection_list}
```

Arguments

name

Indicates the [symbolic name](#) to be assigned to the ATPG primitive being added. This name is a reference, which can be used in future `add_atpg_constraints` commands. The primitive receives a [ATPG primitive ID](#) number, which can be used to reference it.

and | or | sel1 | sel01 | equiv

Indicates the logic function assigned to the ATPG primitive being added. The possible choices are:

`and` - The output is the logical AND of the inputs.

`or` - The output is the logical OR of the inputs.

`sel1` - The output is 1 if one input is 1 with all others 0, 0 if two or more inputs are 1 or if all inputs are 0's, and X for all other conditions.

`sel01` - The output is 1 either if one input is 1 with all others 0 or all inputs are 0. The output is 0 if two or more inputs are 1, and X for all other conditions.

`equiv` - The output is 1 if all inputs are either all zeros or all ones. The output is 0 if both 1 and 0 occur on inputs, and X for all other conditions.

-module name

Indicates that the scope of the ATPG primitive definition should be a specific module rather than the entire design. The ATPG primitive is added to the module definition and will therefore be present for each instantiation of that module in the design. When this occurs, multiple ATPG primitives result, one for each instance in the design. Each will have a unique ATPG primitive ID. The input connection list when `-module` is used is limited to the scope of the module. Only pin path names within the hierarchy of the module are accepted as input connections.

input_connection_list

Indicates the names of the points that are to be connected to the inputs of the ATPG primitive. These can be an internal [pin pathname](#), a primary input (port), or the output of a previously defined ATPG primitive (referenced by its [gate ID](#)). The input connections should not be clocks or clock nets and you cannot reference as input the gate ID of a previously defined ATPG primitive that uses the `sel1`, `sel01`, or `equiv` function. A tilde (~) character preceding an input indicates that the logical inversion of the connection is to be

used as an input. If the `-module` option is specified, then any port name or pin pathname is relative to the scope of the module specified. If the `-module` option is not used, then any pin pathname must be at the design level. These are the lowest-level pins corresponding to the pins of the library cells. If a fault cannot be placed on the pin pathname in question, then that pin pathname is probably not a design-level pin and is most likely an intermediate-level pin.

Allowed Command Modes

DRC, Test

Description

Use this command to create logical functions that need to be constrained. Constraining these functions might be useful in avoiding undesirable conditions in patterns generated by the ATPG process.

For example, if a design uses internal three-state drivers and the driver enable lines can all be off at the same time, which results in a floating net, you can define an ATPG primitive with each of the enable pins as inputs to a SEL1 function. The output of the gate can then be constrained to a 1 and the ATPG pattern generation then produces patterns for which there is always one internal driver turned on.

Internal storage is used to keep track of these "virtual" gates. No more than 200 ATPG primitives can be defined. You can use the command [set_environment_viewer](#) `-max_gate number` to adjust the size of this storage pool.

For more information on using the `add_atpg_primitives` command, see ["Using ATPG Constraints"](#) in the *TestMAX ATPG User Guide*.

Examples

```
DRC> add_atpg_primitives MY_ATPG_PRIM equiv \
  {BLASTER/MAIN/CPU/TP/CYCL/CDEC/U1936/in1 \
  BLASTER/MAIN/ALU_CORE/TP/CYCL/CDEC/U1936/in1 \
  BLASTER/MAIN/ALU_CORE/TP/CYCL/CDEC/U16/in2 \
  BLASTER/MAIN/ALU_CORE/TP/CYCL/CDEC/U13/in0 }
Gate with ID#=20201 has been added to the ATPG primitive list.
DRC> report_atpg_primitives -all
name          id#    type      inputs
-----
MY_ATPG_PRIM  20201  EQUIV      861 990 1232 723
```

See Also

[add_atpg_constraints](#)
[report_atpg_primitives](#)
[remove_atpg_primitives](#)
[set_environment_viewer -max_gate](#)

add_atpg_constraints

Overview

This command defines constraints that must be satisfied during ATPG pattern generation. You can also apply the constraints to the DRC process.

Syntax

```
add_atpg_constraints
  name <0 | 1 | z>
  [-module name]
  <atpg_primitive_name | id | module_pinname | pin_pathname>
  [-drc | -iddq]
  [-no_detection_credit file_name]
```

Arguments

name

Specifies the symbolic name assigned to the ATPG constraint that is being created.

0 | 1 | z

Indicates the value of the constraint.

-module name

Indicates that the ATPG constraint is added for the selected pins for every usage of the selected module.

atpg_primitive_name | id | module_pinname | pin_pathname

Indicates the site of the constraint. The site can be identified by its [ATPG primitive name](#), [primitive ID](#), [pin pathname](#), or [module pin name](#) (if `-module` is used).

-drc | -iddq

Specifies that the ATPG constraint is monitored during the DRC analysis or IDDQ analysis as well as during ATPG pattern generation. The `-drc` option specifies that the ATPG constraint must be satisfied during the simulation of the load_unload, shift, setup, and other DRC procedures in addition to being satisfied during ATPG pattern generation. All ATPG analyses (such as contention ability checking) must also satisfy these constraints. A DRC ATPG constraint can only be added or removed during DRC command mode. The `-iddq` option specifies that the ATPG constraint is to be satisfied only during an IDDQ measure strobe and only if the IDDQ fault model has been selected. An IDDQ measure strobe corresponds to the time in the tester cycle specified in the DRC file when outputs would be measured.

-no_detection_credit file_name

Specifies a file containing a list of cells that are not used for fault detection credit. These cells are still used for good machine values. This option is supported only in TestMAX ATPG.

Note the following:

- Test generation avoids using no detection credit cells as observe points, which can produce additional AU faults.
- Fault simulation avoids using these cells when giving detection credit for faults.
- This option is supported for all fault models, test generation engines, and compression.
- If you specify the `-no_detection_credit` option, any other options used in the same command line are ignored. You can specify other options using a separate command line.

For a complete description of using this option, see [Defining, Reporting, and Removing No Detection Credit Cells](#).

Allowed Command Modes

DRC, TEST

Description

This command defines constraints that must be satisfied during ATPG pattern generation. You can also specify the constraints to be considered during the DRC process.

During Basic-Scan and Fast-Sequential pattern generation, an ATPG constraint is considered to be met if the desired value is achieved during the pre-clock time. It is not required that the desired value be achieved for "clock on" or post-clock times. For purposes of algorithm speed and memory efficiency, very little effort is made to achieve the desired value during "clock on" or post-clock times. If you want to check for constraint violations during clock and post-clock times, use the command `set_contention -capture_constraint_check`.

During Full-Sequential pattern generation, an ATPG constraint is considered to be met only if the desired value is achieved during pre-clock, active-clock, and post-clock times.

For design inputs, you should use the `add_pi_constraints` command rather than ATPG constraints to achieve constant values. The effect of a PI constraint is more complete and more thoroughly propagated during the various DRC checks and simulation checks than an ATPG constraint.

For more information on using the `add_atpg_constraints` command, see ["Using ATPG Constraints"](#) in the *TestMAX ATPG User Guide*.

Examples

```
# by atpg\_primitive\_name
DRC> add_atpg_constraints my_constraint1 1 my_atpg_primitive1
# by primitive\_id
DRC> add_atpg_constraints my_constraint2 0 1911
# by pin\_pathname
DRC> add_atpg_constraints my_constraint3 1 CORE/ALU/DEMU/U266/H02
DRC> report_atpg_constraints -all -verbose
name val DRC site
-----
my_constraint1 1 no my_atpg_primitive1 (20217)
my_constraint2 0 no MAIN/CPU/CAL/TBL/U1842 (1911)
my_constraint3 1 no CORE/ALU/DEMU/U266/H02 (12683)
```


See Also[add_atpg_primitives](#)[add_pi_constraints](#)[remove_atpg_constraints](#)[report_atpg_constraints](#)[Primitive ID](#)

add_capture_masks

Overview

Use this command to mask any potential value captured into a scan or nonscan sequential cell during a capture clock.

Syntax

```
add_capture_masks
  {instance_path | gate_id | -scan {chain_name:index}}
  [-index number]
  [-load_only]
```

Arguments

instance_path | gate_id

Specifies the [instance pathname](#) or the [gate ID](#) of the scan or nonscan sequential element for the capture value you want to mask. This is equivalent to considering the data input of the state element connected to a TIEX. If the state element is a scan cell, it can still be loaded to a non-X value during a scan load. One or more instances can be specified on the same command line.

-scan {chain_name:index}

Specifies a capture mask using the scan chain name and the index reference. The scan chains are identified after the [run_drc](#) command is finished processing. Any information specified by the `-scan` option is saved and applied after the [run_drc](#) command completes scan chain tracing.

All errors related to incorrect references are printed after the [run_drc](#) command is processed. You can use the `-scan` option to specify multiple mask references in a list. There are no spaces on either side of the colon for each specified reference. For example:

```
DRC> add_capture_masks -scan {scan_chain_1:0 scan_chain_1:1}
```

You cannot specify the `-scan` option with other capture mask data. If you use this option, it must be the only option specified with the `add_capture_masks` command.

-index number

When the *instance_path* argument defines a reference that contains multiple sequential elements (a multibit cell), you can use the `-index` option to specify the specific bit to be masked. Without this option, all bits of this type of reference are masked. Bit order is a function of a model definition. If you use the `-index` option, you should also use the [report_capture_masks](#) command to review which specific bit (identified by its gate ID) was masked and verify that the bit is intended for masking.

The `-index` option specifies only a single value that is applied to all names associated with the specified *instance_path* argument. If you need to mask multiple specific bits of an instance, you should specify the `add_capture_masks` command multiple times with a different index value in each command. The `-index` option does not affect gate IDs, as there is no ambiguity in this context. The index value must be greater than or equal to 1; the first bit of a multibit cell is nearest to the scan-input of the cell and is referenced as

"1". If the index value exceeds the number of bits in the instance, no mask is defined and no errors are generated.

-load_only

Causes scan and nonscan cells to load unknown values (Xs). These cells continue to capture known values during the capture cycles. This functionality masks nonscan cells when the [set_simulation -shift_cycles n](#) command is specified. For scan cells, the -load_only option has the same effect as the `add_cell_constraints X` command.

The -load_only option modifies the output of the `report_capture_masks` command. When this option is specified, the report appears as follows:

```
TEST-T> report_capture_masks
gate_id type lo instance_name
-----
45 DFF -- s1
58 DFF LO n0and
```

The output report includes an "lo" column. If the -load_only switch applies to a cell, the "LO" characters appear in this column. When the -load_only option does not apply to a cell, two dash characters "--" appear in this column.

Any nonscan cells masked by the `add_capture_masks -load_only` command can be designated as load-0 (L0) or load-1 (L1) cells. But they do not behave as L0 or L1 cells because of the capture masks. The [report_capture_masks](#) command reports them correctly, and ATPG and simulation handle them correctly. However, the output of the [report_nonscan_cells](#) command gives them misleading designations.

Allowed Command Modes

DRC

Description

Use this command to mask any potential value captured into a scan or nonscan sequential cell during a capture clock.

If the state element is a scan cell, it can still be loaded to a non-X value during a scan chain shift. If no capture clock is applied, its value will remain non-X and be visible in a scan chain unload. If you want to affect the values loaded and observed during scan chain shifting, see the `add_cell_constraints` command.

There is a difference between [using add_capture_masks vs. add_cell_constraints](#). The method to mask a specific sequential cell instance depends on whether the cell is scan or nonscan, and whether the mask is to be applied during load, during capture, or both.

For information on specifying lists and using backslashes in Tcl mode, see "[Specifying Lists in Tcl Mode](#)."

Examples

```
DRC> add_capture_masks { MAIN/U111/K1 MAIN/U112/J99 MAIN/U113/R1 }
3 capture masks were added to capture mask list.

DRC> report_capture_masks
gate_id type instance_name
-----
19286 DFF MAIN/U111/K1
19287 DFF MAIN/U112/J99
19288 DFF MAIN/U113/R1
```

```
DRC> add_capture_masks -scan A:2  
  
DRC> add_capture_masks -scan {scan_chain_1:0 scan_chain_1:1}
```

See Also[add_cell_constraints](#)[report_capture_masks](#)[remove_capture_masks](#)

add_cell_constraints

Overview

This command adds cell constraints that are used to control the load and unload values allowed on scan cells.

Syntax

```
add_cell_constraints
<0 | 1 | x | 0x | 1x | ox | xx>
<[ chain_name | instance_name>
[-position {< cell_pos1 | sci> [ cell_pos2 | sci]>} | -all]>>
[-index number]
```

Arguments

0 | 1 | x | 0x | 1x | ox | xx

Indicates the type of constraint to be added. The choices 0, 1, or x indicate the constrained value that should be in place at the end of the scan chain load. When a 0, 1, or x constraint is applied, the scan cell is always loaded with this value for every pattern. A capture clock loads new data into the scan cell that might be different from the 0/1/X loaded by a scan load.

When a 0x constraint is specified, the scan cell is always loaded to zero (0) and will always observe X.

When a 1x constraint is specified, the scan cell is always loaded to one (1) and will always observe X.

A ox constraint indicates the observed value is always masked, or considered to be X. The value loaded by a scan load or captured by a capture clock is not affected, and the actual scan cell value continues to affect the fanout cone of the scan cell. A selection of ox affects only the expected value for the scan chain unload. This value is X (or masked). To affect the value loaded by a scan load, use the xx selection. To affect the value loaded by a capture clock, use the `add_capture_masks` command.

Some compression restrictions may affect the ability to apply cell constraints. In the case of chain tests in compression mode, [M177](#) messages might be reported when attempting to specify constraints, as shown in the following example:

```
add_cell_constraints 1 8897
add_cell_constraints ox 9299
add_cell_constraints 0 8706
add_cell_constraints 1 8597
add_cell_constraints 1 8610
add_cell_constraints 0 8853
```

In this case, the `-verbose` option of the `set_contention` command is set and the following messages are reported:

```
Warning: Cell constraint 11-15 set to 1 not satisfied for preclock chain
test pattern (pat1=0). (M177)
Warning: Cell constraint 25-14 set to 0 not satisfied for preclock chain
test pattern (pat1=3). (M177)
Warning: Cell constraint 16-10 set to 1 not satisfied for preclock chain
```

```
test pattern (pat1=0). (M177)
```

Warning: Cell constraint 17-16 set to 1 not satisfied for preclock chain

```
test pattern (pat1=0). (M177)
```

Warning: Cell constraint 11-19 set to 0 not satisfied for preclock chain

```
test pattern (pat1=2). (M177)
```

Note: An `ox` constraint is the letter "o" followed by the letter "x" and might look like a zero with some fonts.

An `xx` constraint causes the scan cell to always be loaded with X and to always observe X. It is equivalent to combining `x` and `ox`.

`chain_name -position < cell_pos1 | sci> [ cell_pos2 | sci]> | -all>`

Indicates the scan chain name and position within the scan chain where the cell constraint is applied. Position 0 is the cell closest to the scan chain output. A range of cells can be specified or the keyword `-all` which indicates all cells in the scan chain. The keyword `sci` is used for a cell position and indicates the numeric value equivalent for the scan cell closest to the scan chain input.

`instance_name`

Specifies the constraint location by the [instance_name](#) of the latch or flip-flop of the scan cell to be constrained.

`-index number`

If the `instance_name` argument defines a reference that contains multiple sequential elements (a multibit cell), you can use the `-index` option to specify the specific bit you want to constrain. Without this option, all bits of this type of reference are constrained. Bit order is a function of a model definition. If you use the `-index` option, you should also use the [report_cell_constraints](#) command to review and verify the specific bit (identified by its gate ID) that was constrained.

The `-index` option specifies only a single value that is applied to all names associated with the specified `instance_name` argument. If you need to constrain multiple specific bits of an instance, you should specify the `add_cell_constraints` command multiple times with a different index value in each command. The `-index` option does not affect gate IDs, as there is no ambiguity in this context. The index value must be greater than or equal to 1; the first bit of a cell is referenced as "1". If the index value exceeds the number of bits in the instance, no constraint is defined and no errors are generated.

Allowed Command Modes

DRC

Description

This command adds cell constraints that are used to control the load and unload values allowed on scan cells. The test generator creates only patterns that satisfy cell constraints. The fault simulator rejects patterns that fail to satisfy cell constraints and does not give any fault detection credit for those patterns.

By default, the DRC process does not consider the effect of cell constraints. However, you can change this by using the `-use_cell_constraints` or `-clockdrc_use_cell_constraints` options of the [set_drc](#) command.

Masking the scan cell closest to the scan chain output (cell 0) does not inhibit patterns that measure the primary output port associated with the scan chain. To do so, use the `add_po_mask` command.

Note that there is a difference between [using add_capture_masks and add_cell_constraints](#). The method to mask a specific sequential cell instance depends on whether the cell is scan or nonscan, and whether the mask is to be applied during load, during capture, or both.

For information on specifying lists and using backslashes in Tcl mode, see "[Specifying Lists in Tcl Mode](#)."

For more information on using the `add_cell_constraints` command, see "[Masking Scan Cell Inputs and Outputs](#)."

Examples

```
DRC-T> report_primitives 130
/reg2/lat1 (130) DLAT (P_LAT_RS)
!SB I (/TIE_1)
RB I 28-
CK I 17-
D I 105-/reg2/MX1/Q
Q O 131-/reg2/r/D
DRC-T> add_cell_constraints OX reg2/r/D

# by instance name
DRC-T> add_cell_constraints 1 MAIN/CPU/TP/FI/OFIFO/reg3
# by instance name
DRC-T> add_cell_constraints ox MAIN/PER/PRT_1/PORTIN/reg3
# by name & position
DRC-T> add_cell_constraints xx c34 -position 6
DRC-T> report_cell_constraints
type chain pos. site name
-----
1 ...    ... MAIN/CPU/TP/FI/OFIFO/reg3
OX ...    ... MAIN/PER/PRT_1/PORTIN/reg3
XX c34    6    ...\par# mask observe on all bits of a scan chain
DRC-T> add_cell_constraints OX CHAIN_1 -all
DRC-T> add_po_masks SDO

# load a range of a scan chain cells to 1's
DRC-T> add_cell_constraints 1 CHAIN_2 -position { 23 29 }

# 400 bit scan chain:
# you can load the first half but not observe
# and observe the last half but not load
DRC-T> add_cell_constraints OX CHAIN_2 -position { 399 200 }
DRC-T> add_cell_constraints X CHAIN_2 -position { 199 0 }
```

See Also

[add_capture_masks](#)
[remove_cell_constraints](#)
[pin_pathname](#)
[instance_name](#)
[report_cell_constraints](#)
[set_contention](#)

add_chain_masks

Overview

This command generates patterns for masking scan chains using DFTMAX Ultra compression.

Syntax

```
add_chain_masks
  [-name chain_name]
  [-do_not_mask chain_name]
  [-compressor compressor_name]
  [-filename filename]
  [-percent percentage]
  [-external]
  [-diagnosis]
  [-diagnosis_pattern_count value]
  [-force_single_fanout_mode]
  [-reset]
```

Arguments

-name *chain_name*

Specifies the name of the chain or chains for masking. If you specify more than one chain, the names must be provided in Tcl list format. The following example masks the ch1, ch2, and ch3 chains:

```
add_chain_masks -name {ch1 ch2 ch3}
```

-do_not_mask *chain_name*

Specifies the name of the chain or chains that should not be masked under any circumstance. If you specify more than one chain, the names must use Tcl list format. The following example specifies that the ch2, ch4, and ch7 chains are not masked in any unload configuration:

```
add_chain_masks -do_not_mask {ch2 ch4 ch7}
```

-compressor *compressor_name*

Specifies the name of the compressor containing chains for masking. This option applies to multi-codec designs. If you specify `-do_not_mask {ch2 ch4 ch6}` and `-compressor compressor1`, and ch2 and ch4 belong to `compressor1`, then all chains of `compressor1` are masked, except ch2 and ch4.

```
add_chain_masks -compressor my_comp
```

In the following example, `ch1_of_my_comp` and `ch10_of_my_comp` are chains from the `my_comp` compressor. In this case, all chains of the `my_comp` compressor are masked except for these two chains:

```
add_chain_masks -compressor my_comp -do_not_mask {ch1_of_my_comp \
ch4_of_different_comp ch10_of_my_comp}
```

-filename *filename*

Specifies the name of a file containing the chain names you want to mask. This file must contain only chain names separated by spaces, tabs, or new lines. The following example masks all chains specified in the abc.txt file:

```
add_chain_masks -filename abc.txt
```

-percent *percentage*

Specifies the total percentage of external patterns for masking. This option must be used with the `-external` option. The default value is 100.

-external

Masks only the external patterns. This option can be specified with the `-percent` option.

-diagnosis

Masks the external patterns for diagnosis. For more information on using this option, see [Chain Diagnosis for Multiple Chain Defects](#).

-diagnosis_pattern_count *value*

Specifies the pattern count for masking each chain when using the `-diagnosis` option. The default `value` is 50.

-force_single_fanout_mode

Forces TestMAX ATPG to unload the pattern only in single fanout mode. In this case, all multi-fanout modes are masked for all patterns. You can specify this option with the `-compressor` option, if needed. The following example forces TestMAX ATPG to unload patterns in a single fanout mode:

```
add_chain_masks -force_single_fanout_mode
```

The following example forces TestMAX ATPG to unload patterns from the my_comp compressor in a single fanout mode:

```
add_chain_masks -force_single_fanout_mode -compressor my_comp
```

-reset

Deletes all previously specified chain masks.

Allowed Command Modes

TEST

Description

This command generates patterns for masking scan chains using DFTMAX Ultra compression. If defective scan chains are present, TestMAX ATPG uses DFTMAX Ultra compression to generate patterns that mask internal scan chains to debug mismatches. In this case, TestMAX ATPG assumes that one or more of the internal scan chains load and unload X values. This masking can be applied to new ATPG runs. You can also mask scan chains in external patterns for diagnostics. Use the `-streaming_xtol_chain_test_type` option of the [set_atpg](#) command to control the generation of DFTMAX Ultra compression test patterns for diagnostics.

Examples

In the following example, the `-do_not_mask` option takes the higher priority for the ch4 chain. As a result, the ch1 and ch3 chains are masked, and the ch2, ch4, and ch5 chains are not masked for all unload configurations.

```
add_chain_masks -name {ch1 ch3 ch4} -do_not_mask {ch2 ch4 ch5}
```

The following example masks all chains in the abc.txt file and chains 12, 45, and 67:

```
add_chain_masks -filename abc.txt -name {12 45 67}
```

The following example masks chains 12, 45, and 67 on 30 percent of the external patterns:

```
add_chain_masks -name {12 45 67} -external -percent 30
```

The following example masks all chains specified in the abc.txt file and is applied to 20 percent of the external patterns for diagnosis:

```
add_chain_masks -filename abc.txt -external -percent 20 -diagnosis
```

See Also

[Using TestMAX ATPG and DFTMAX Ultra Compression](#)

add_clocks

Overview

Use this command to specify and define clocks.

Syntax

```
add_clocks
  <0 | 1> {pin_names_list}
  [-intclock]
  [-cycle { cycle_id <-always on | -always off> | <int_node_name <0 | 1>>}]
  [-master_observe]
  [-pllclock]
  [-pll_shift]
  [-refclock]
  [-ref_timing {period LE TE }]
  [-shift]
  [-timing {period LE TE measure_time}]
  [-unit <ps | ns>]
```

Arguments

0 | 1

Specifies the [off state](#) of the clock. The off state is the logic level (either 0 or 1) for which the clock port is inactive. It results in holding the state of sequential devices such as flip-flops, latches, or RAMs. An active-high clock has an off state of 0 and an active-low clock has an off state of 1.

pin_names_list

Specifies the [port name](#) that should be treated as a clock. You can specify multiple port names using a single command, but they must all share the same off state.

Any backslash that is embedded within a Tcl list is automatically stripped. The following two examples show how to overcome this situation:

- Use a double-backslash, for example:

```
add_clocks 0 {\A[0] \B[0]}
```
- Use two levels of braces, for example:

```
add_clocks 0 {{\A[0]} {\B[0]}}
```

As an alternative, you can remove backslashes entirely. In this case, TestMAX ATPG commands automatically match specified identifiers that have no backslashes to identifiers in the database that have backslashes.

```
-intclock [-cycle { cycle_id <-always on | -always off>| int_node_name <0 | 1 >>}] [-pll_source node_name]
```

Defines internal nets as clocks, controlled by the identified scan cell from the clock chain, when you use it with the [off state](#) option.

Before defining internal clocks, specify the `set_drc num_pll_cycles` option, which defines the number of PLL clock cycles supported per load.

This option defines zero or more control clauses; each clause associates a cycle (zero-based) with one or more conditions. For the clock to be turned on, the selected cycle must meet all conditions simultaneously; even if it does not meet one condition, the clock is turned off. A condition can be always on, always off, or a net name/pin path name followed by either 0 or 1. The conditions must be combinational justifiable independent of faults effects, so that internal clock control clauses can be pre-calculated, independent of pattern generation.

In Tcl mode, you specify only one `-cycle` for both 0 and 1 conditions, as shown in the following example:

```
add_clocks 0 intclk3 -intclock -pll_source pllclk3 \
  -cycle { 0 clock_chain/cell[3]/Q 1 \
    1 clock_chain/cell[4]/Q 1 }
```

Also note the following:

- You should not use logic added by [add_atpg_primitives](#) command for the `-cycle` argument or you will get unpredictable and probably erroneous results.
- `intclk3` as an internal clock with offstate 0; its PLL source is `pllclk3`; `intclk3` is pulsed in cycle 0 when cell 3 of chain `clock_chain` is 1, and is pulsed in cycle 1 when cell 4 of chain `clock_chain` is 1.

```
add_clocks 0 intclk1 -intclock -pll_source pllclk3 \
  -cycle { 0 clock_chain/cell[1]/Q 1 \
    0 clock_chain/cell[0]/Q 0 \
    1 clock_chain/cell[2]/Q 1 \
    1 clock_chain/cell[0]/Q 0 }
```

- `intclk1` as an internal clock with offstate 0; its PLL source is `pllclk3`; `intclk1` is pulsed in cycle 0 when cell 1 of chain `clock_chain` is 1 and cell 0 is 0, and is pulsed in cycle 1 when cell 2 of chain `clock_chain` is 1 and cell 0 is 0. Internal clocks are used for all capture operations, and normal clock rule checks are applied to these clocks. No external clocks are used for capture when internal clocks are defined. Additional rule checks are done to check the stated correspondence between an internal clock and scan cells from the clock chain.
- It is valid for a pattern to load the clock-controlling scan cells so that no internal clocks are pulsed, and still have one or more system pulses that pulse the defined external reference clocks. All internal clocks must have the same number of control clauses, corresponding to consecutive cycles, starting with 0. For example, it is an error to define control clauses for cycles 0, 1, and 2 for `intclk3`, and to define control clauses only for cycles 0 and 1 for `intclk2`. It is also an error to define control clauses for cycles 0 and 2, but not for cycle 1. The `-always on` condition can only be defined for cycle 0, otherwise scan ATPG cannot be used.
- The `-always on` argument cannot be used with the fast-sequential test generator.
- The `-always off` argument can be used in conjunction with a clock-chain cycle specification. However, `-always on` is incompatible with clock chains.

TestMAX ATPG supports a legacy controller containing two identical cycles, with each cycle having at least one clock specified as `-always on`. Individual clock conditioning cannot be used when the `-always on` argument is used. Because each pattern has two system cycles, chain test and single cycle basic-scan patterns cannot be created. Only optimized two-cycle, transition-fault ATPG can be used in this case.

The following examples show how to use the `-always on` and `-always off` arguments:

```
add_clocks 0 pll1/PLLCLK -pllclock
add_clocks 0 pll2/PLLCLK -pllclock
```

```

add_clocks 0 U1/U2/Z -intclock \
  -cycle { 0 -always on \
    1 -always on } \
  -pll_source pll1/PLLCLK
add_clocks 0 U2/U2/Z -intclock\
  cycle { 0 -always off \
    1 -always off } \
  -pll_source pll2/PLLCLK

```

-master_observe

Identifies the clock that is applied in a master_observe procedure. This option only affects the `write_drc_file` operation if a master_observe procedure was not defined; a template master_observe procedure is defined in the procedures generated from the `write_drc_file` operation. You should review this template for completeness and appropriateness before using it with the `run_drc` command

-pllclock

Defines internal nets as free-running clocks (outputs of PLLs or clock shapers) when you use it with the [off state](#) option. These clocks are considered asynchronous to each other and to all other clocks. TestMAX ATPG uses these clocks only during scan design rule checking and PLL-specific clock rule checking. Pattern generation does not consider these clocks. During scan rule checking after the simulation of each rule checking event (for example, an event in the `load_unload` procedure), TestMAX ATPG simulates the pulses of the free-running PLL clocks until the circuit stabilizes or a user-defined limit is reached.

-pllshift

Specifies that the clocks to be added are defined as PLL shift clocks. Note that PLL shift clocks are used only for initializing scan cells to X during DRC. The shift process must be driven from an external clock.

PLL shift clocks can also be defined in the STL procedure file in the `Clocks {}` block by using the `PLLShift` identifier. For example, the following specification defines internal node `des_unit_inst/my_clock_controller/U2/Z` as a PLL shift clock with off-state 0:

```

Clocks {
  "des_unit_inst/my_clock_controller/U2/Z" PLLShift { Offstate 0; }
  ...
}

```

The `-pll_shift` option can also be used to specify internal serializer shift clocks for serialized DFTMAX compression. The serialized DFTMAX architecture usually has a Serializer FSM whose output gates all the external shift clocks to create internal shift clocks used for scan shifting in serialized compression mode. Typically, the gated external shift clocks outputs are specified as internal serializer shift clocks for serialized DFTMAX. This specification is used to initialize scan cells to X during Serializer DRC. You still need to pulse external shift clocks in the STL procedure file to perform scan shifting in Serialized DFTMAX mode.

-refclock

Defines primary inputs (PIs) or bidirectional ports (PIOs) as PLL reference clocks when you use this option with the off state option. PLL reference clocks are external, free-running clocks necessary to keep the PLLs synchronized. For TestMAX ATPG, these are non-logical

clocks; not used in either DRC or ATPG. Each pulse instruction in every pattern generated pulses the PLL reference clocks. To keep PLLs synchronized, every vector in every procedure defined in the STL procedure file must pulse all PLL reference clocks. You need not define PLL reference clocks supplied by an onboard oscillator, which are not needed for DRC. You need not pulse these clocks in the STL procedure file.

-ref_timing {period LE TE}

Defines the test cycle period associated with the reference clock. *LE* is the time of the leading edge of the ref clock, while *TE* is the time of the trailing edge of the ref clock. For example:

```
add_clocks 0 clkref3 -shift -refclock -ref_timing {31286 5000 15000} \
-unit ps
```

Note that the `-unit` switch is optional and can be `ns` (the default) or `ps`.

-shift

Marks the clock as 1, which is used during the shifting of scan chains and causes the `write_drc_file` command to pulse the clock in the Shift procedure when a template DRC file is created. Asynchronous set/reset ports or RAM/ROM controls defined as clocks should generally be defined without the `-shift` option.

This option does not honor any PI equivalence relation, so if you declare two pins to be equivalent, and then one of them as a clock with the shift attribute, the other is declared a clock but *without* a shift attribute. For this reason, you should always declare all clocks first and PI equivalence relations on those clocks second. Use of the `-shift` option should explicitly list each clock.

-timing {period LE TE measure_time}

Defines the test cycle period associated with the clock. *LE* is the time of the leading edge of the clock, while *TE* is the time of the trailing edge of the clock. The measure time option specifies the time within the test cycle at which the design's output pins is measured. If *measure_time* is less than *LE*, then a preclock measure protocol is implied. If *measure_time* is greater than *TE*, then an end-of-cycle measure protocol is implied. The default for timing is: period 100, t1=50, t2=70, measure_time=40. If more than one `-timing` option is specified, then the last one encountered reading from left to right is used.

Setting the `-timing` option changes the default for future `add_clocks` commands for which the `-timing` option is omitted.

All periods, measure times, and units must be the same for all instances of this command.

If a DRC file is specified with the `run_drc` command, there is a potential for conflict between the clocks, timing, periods, or measures defined with the add clock command and those defined within the DRC file. See the `run_drc` command for rules of priority.

Note: You can use the `-timing` option only in the *first* `add_clocks` command. Attempting to use `-timing` after one or more `add_clocks` commands have been issued without the `-timing` option is reported as an error.

-unit <ps | ns>

Specifies the time units used with the `-timing` option and should only be specified if `-timing` is used. By default, the time unit is `ns`. All units must be the same for all add clock commands.

Allowed Command Modes

DRC

Description

Use this command to specify and define clocks. Top-level ports that can change the stored state of sequential devices are defined as [clocks](#). This includes not only the traditional active-high and active-low clocks, but also the asynchronous sets/resets and RAM write lines. For ATPG purposes, a "clock" is a pulsed input port that can change the state of a flip-flop, latch, or RAM.

If a DRC file is specified with the `run_drc` command, there is a potential for conflict between the clocks or timing defined with the `add_clocks` command and the timing that might exist in the DRC file. When the same clock is defined in multiple places, the results are unpredictable, so this situation should be avoided. An exception is when you define timing using the `add_clocks -timing` command. In this case, the Timing block in the DRC file takes precedence when a conflict occurs. See the `run_drc` command for a complete description of the rules of priority.

It is highly recommended that you always define clocks before defining the PI equivalence relationships for those clocks.

Note: Clocks and other pulsed ports can also be defined using a STIL procedure file.

For more information on using the `add_clocks` command, see ["Declaring Clocks."](#)

Examples

```
DRC> remove_clocks -all
DRC> add_clocks 0 {INC CLK }
DRC> add_clocks 1 RSTB
DRC> report_clocks
clock_name off usage
-----
/INC          0
/CLK          0   master shift
/RSTB         1
```

```
DRC> add_clocks 0 CLK1 -timing {200 50 80 40} -unit ns -shift
DRC> add_clocks 0 CLK2 -timing {200 50 80 40} -shift
DRC> add_clocks 1 RSTB -timing {200 50 80 40}
DRC> report_clocks
clock_name off period  t1_time t2_time meas_time usage
-----
CLK1        0   200ns   50      80      40      shift_clock
CLK2        0   200ns   50      80      40      shift_clock
RSTB        1   200ns   50      80      40
```

See Also

[remove_clocks](#)
[report_clocks](#)
[run_drc](#)
[write_drc_file](#)

add_delay_paths

Overview

This command reads in a delay path list from a specified file.

Syntax

```
add_delay_paths file_name
```

Arguments

file_name

Reads in a [delay path](#) list from this file.

Allowed Command Modes

TEST

Description

This command reads in a delay path list from a specified file. For more information on using the `add_delay_paths` command, see "[Generating Path Delay Tests](#)" in the *TestMAX ATPG User Guide*.

Examples

```
TEST> add_delay_paths stopwatch
Warning: Rule P2 (missing delay path name) was violated 4 times.
Warning: Rule P21 (path node values not satisfiable) was violated 2 times.
Delay path file results: #paths=4, #invalid=0, #untestable=2, CPU
time=0.04 seconds.
```

See Also

[report_delay_paths](#)

[remove_delay_paths](#)

add_display_gates

Overview

Use this command to selectively add gates to the GSV.

Syntax

```
add_display_gates
  [<pin_pathname | gate_id | -all> ]
  [-color <blue | green | orange | pink | purple | yellow | white |
  default>]
  [-path {begin_gate_idend_gate_id} ]
```

Arguments

<pin_pathname | gate_id | -all>

Adds gates associated with the [pin_pathname](#), [gate ID](#), or all gates to the GSV. Choices *pin_pathname*, *gate_id*, and *-all* are only accepted in primitive view.

-color <blue | green | orange | pink | purple | yellow | white | default>

Designates a particular color for displaying selected gates in the graphical schematic viewer (GSV).

-path {begin_gate_idend_gate_id}

This option traces the path between two sequential elements in the GSV. You must specify two end-points that are not separated by any other sequential elements. A path, in this case, is considered to be the combinational logic between sequential elements.

Allowed Command Modes

DRC and Test

Description

Use this command to selectively add gates to the GSV.

Examples

```
TEST> add_display_gates mux3/MUX2_UDP_1/Q
```

If the scan_chain report reports the following path:

```
UCL2C/UCL2C__512/uctlru_0/Xcrtz_0/\XTagWayGen[0]/Xetl1__UCL2C_TagWay_
TA0/XI320
```

Then the path is specified as follows:

```
TEST> add_display_gates "UCL2C/UCL2C__512/uctlru_0/Xcrtz_0/\\XTagWayGen
[0]/Xetl1__UCL2C_TagWay_TA0/XI320"
```

Note in this case that the “\” is escaped.

The following example displays all gates in the color blue that are associated with the pin pathname mux3/MUX2_UDP_1/Q:

```
TEST> add_display_gates -color blue mux3/MUX2_UDP_1/Q
```

See Also

[remove_display_gates](#)

[report_display_gates](#)

add_display_proc

Overview

This command registers a Tcl procedure name that is called when you select a context-sensitive menu item in the GSV.

Syntax

```
add_display_proc proc_name
```

Arguments

proc_name

The name of the procedure to be added.

Description

This command registers a Tcl procedure name that is called when you select a context-sensitive menu item in the GSV. After this command is specified, it determines if the procedure has been defined. If the procedure being added is not defined, the following error message is displayed:

```
Error: No command or proc named "%s" is found.
```

When you select a menu item, the corresponding Tcl procedure is called with an argument. If the GSV is at the design level (library cells shown), the instance name of the selected cell is sent as the argument to the proc call. If the GSV is at a primitive level, the gate id is used. You can use a Tcl form such as “The arg value is \$arg” to distinguish between the gate id or the instance name.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

[remove_display_proc](#)

add_distributed_processors

Overview

Use this command to add machines to the pool of distributed processors.

Syntax

```
add_distributed_processors
  {name_list}
  [-min_slaves number | -lsf bsub_exec | -nslaves d | -options {options_
list} | -grd exec> -nslaves d]
  [-options {options_list}]
```

Arguments

name_list

Specifies up to 50 slave machines by name. For each machine name given, a TestMAX ATPG process launches on that machine during distributed fault simulation. If you have a machine with several processors, you can instruct TestMAX ATPG to use those processors by specifying that machine name again for each subsequent processor.

Note: You can execute multiple processes on a uniprocessor machine, but the various processes would have to share processor time. Therefore, you will not be able to take full advantage of the parallelization, and in this case it would be better to use LSF.

-lsf bsub_exec -nslaves d -options {options_list}

Specifies the path to the LSF job submission executable (bsub), the number of slaves to be spawned, and additional options to LSF. For using LSF to launch the slaves, all of these options must be specified. If you do not have any additional options to pass to bsub, you can pass empty options using -options " ". See your IT support for the proper set of options (like queue, project, and so forth) you need for launching an LSF job.

-grd exec -nslaves d -options {options_list}

Specifies the path to the GRID job submission executable (qsub), the number of slaves to be spawned, and additional options to GRID. For using GRID to launch the slaves, all of these options must be specified as well as the -script option of the set_distributed command. If you do not have any additional options to pass to qsub, you can pass empty options using -options without an argument. See your IT support for the proper set of options (like project, resource specification, and so forth) you need for launching an GRID job.

You can use -options to pass the simple bsub or qsub options. For the more complex options bound by single or double quotation marks, see [Passing Complex Options to LSF/GRID](#).

-min_slaves number

Enables you to specify the minimum number of slave machines to start with distributed processing. After waiting for the timeout period and after the minimum number of slave machines is met, the distributed processor start with the number of available slaves. After

it starts, it does not use more slave machines when more CPUs are available. You can use this option when all of the requested CPUs are not available.

Allowed Command Modes

BUILD, DRC, TEST

Description

Use this command to add machines to the pool of distributed processors. For more information on `add_distributed_processors`, see "[Running Distributed ATPG](#)" in the *TestMAX ATPG User Guide*.

Note that TestMAX ATPG also supports Multicore ATPG. For details, see "[Running Multicore ATPG](#)."

See "[License Keys](#)" for licensing requirements.

Examples

```
TEST> add_distributed_processors {himalaya himalaya himalaya}
```

```
TEST> add_distributed_processors -lsf "/remote/tools/bsub" \  
-options "-q rush " -nslaves 15
```

See Also

[remove_distributed_processors](#)

[report_distributed_processors](#)

[set_distributed](#)

Getting Started for Distributed ATPG Users

add_equivalent_nofaults

Overview

This command propagates the existing list of nofaults to their equivalent faults and adds them to the nofault list.

Syntax

None

Arguments

None

Allowed Command Modes

Test

Description

This command propagates the existing list of nofaults to their equivalent faults and adds them to the nofault list. This includes user-defined nofaults as well as netlist modules which might have nofault attributes on module pins or internal gate pins.

Examples

```
TEST> add_equivalent_nofaults
49326 equivalent faults were added to nofault list.
```

See Also

[add_nofaults](#)

[remove_nofaults](#)

[read_nofaults](#)

[report_nofaults](#)

add_faults

Overview

Use this command to create a list of faults for fault simulation or test generation.

Syntax

```
add_faults [ instance_name | pin_pathname | -module name |
  -all | -clocks | -scan_enable | -bridge_location bridge_location1 bridge_
  location2 | -node_file name]
  [-launch launch_clock]
  [-capture capture_clock]
  [-cell_aware]
  [-exclusive] [-shared]
  [-shared_launch]
  [-shared_capture]
  [-inter_clock_domain]
  [-intra_clock_domain]
  [-stuck <0 | 1 | 01> | -slow <r | f | rf>]
  [-bridge <0|1|01>]
  [-aggressor_node <first | second | both>]
  [-dynamic_bridge <r|f|rf>]
  [-dominant_node <first | second | both>]
  [-max_invalid_report d]
```

Arguments

instance_name

Adds faults associated with the selected hierarchical [instance](#) to the fault list except for faults with a nofault attribute.

pin_pathname

Adds faults associated with the specified [pin_pathname](#) to the fault list. You can use the `-stuck` or the `-slow` options to specify which faults to add. The default is both stuck 0 and stuck 1. Faults with a nofault attribute are not placed in the fault list.

-module name

Adds faults to all instances of the named module; not that the *name* argument can be a fixed string or a [limited regular expression](#).

-all

Adds faults at all potential fault sites in the design to the fault list except for faults with a nofault attribute.

For transition faults, the `add_faults -all` command does not add any faults on clock or scan-enable logic by default. This is because it is not meaningful to test faults in such logic at functional speeds with scan-based patterns.

The `add_faults -clocks` and `add_faults -scan_enable` options do allow the addition of such faults to the transition fault list if required. This option could not be used when the fault model is set to bridging and dynamic_bridging.

-cell_aware

Enables TestMAX ATPG to use the cell-aware ATPG. In this case, all defects become faults in fault lists, and both static and dynamic defects are supported. The ATPG process uses both primary and secondary faults, and merges and simulates the faults. For more information on cell-aware ATPG, see "[Cell-Aware Test](#)."

-clocks

Adds clock, set, and reset faults to the fault list.

There are multiple reasons a fault is defined as a clock fault. In some cases, the fault is on an external or internal clock, or on a gate in the fanout cone of an external or internal clock. The `clock_flag` (displayed by the Display Gate Info command in the TestMAX ATPG Schematic Viewer) is set to 1 if the gate is in the fanout cone of an external or internal clock. If the `clock_flag` is set to 1, the faults on the gate are considered clock faults.

Another reason a fault is recognized as a clock fault is if it is on a pin TestMAX ATPG recognizes as a clock pin of a sequential device. These clock pins include the clock, set, and reset pins of flip-flops, latches, and memories. TestMAX ATPG also considers a fault as a clock fault if the fault is on a gate in the fanin cone of a clock pin of a sequential device. If a gate is in both the fanin cone of a clock pin and the fanin cone of the data input pin of a flip-flop or latch, the faults on that gate are recognized as clock faults.

-scan_enable

Adds scan enable faults to the fault list.

-bridge_location *bridge_location1 bridge_location2*

Specifies the nodes of the bridging or dynamic bridging fault being added. Both nodes must be specified and when specified, the `-node_file` option cannot be used. For the bridging or dynamic bridging fault model, if this option is not used, the `-node_file` must be used.

Either node can be selected to be an aggressor node, victim node, both using the `-aggressor` option for the bridging fault model and the `-dominant` option for the dynamic bridging fault model. The arguments for `add_faults -bridge_location` could be output pins. For example:

```
TEST> add_faults -bridge_location mic0/alu0/sub_307/U4/Z mic0/pc0/U33/Z
```

Input pins can also be used but only if [set_fault](#) `-bridge_input` has been specified. A limited set of net names are supported. A warning is given if an invalid bridge location is specified. In order for a bridging pair to be read successfully, the fault model must be set before the bridge type using the command [set_faults](#) `-model <bridging | dynamic_bridging>`.

-node_file *name*

Specifies the name of a file that contains node pairs for use in creating the bridging and dynamic bridging faults added to the fault list. When this option is used, the bridge location pair or the dynamic bridging pair cannot be specified. If this option is not used, the bridge location pair or the dynamic bridging pair must be specified. You can use the `-aggressor_node` option or the `-dominant_node` option to specify the victim and aggressor nodes. See [Node File Format for Bridging Faults](#) for the format of this file. The node file can be compressed in gzip format. In order for bridging pairs specified in a node file to be read successfully, the fault model must be set before the bridge type using the command [set_faults](#) `-model <bridging | dynamic_bridging>`.

-launch *launch_clock*

Specifies the launch clock of the faults to be added to the fault list. You can use this option with or without the `-capture` switch. The `-launch` option adds all faults that are launched exclusively by the specified clock. You can specify the `PI`, `NO_CLOCK`, or `MULTIPLE` keywords (in all upper or all lowercase) with this option to add faults that are launched by the `PI/PIO`, no clock domains, or multiple clock domains, respectively. For a complete description of how TestMAX ATPG distinguishes faults captured by a clock and launched by a clock, see [Using Clock Domain-Based Faults](#).

-capture *capture_clock*

Specifies the capture clock of the faults to be added to the fault list. This option adds all faults that are captured exclusively by the specified clock. You can specify the `PO`, `NO_CLOCK`, or `MULTIPLE` keywords (in all upper or all lowercase) with this option to add faults that are captured by the `PO/PIO`, no clock domains, or multiple clock domains, respectively. For a complete description of how TestMAX ATPG distinguishes faults captured by a clock and launched by a clock, see [Using Clock Domain-Based Faults](#).

-exclusive

Specifies that only the faults that are launched and captured exclusively (using a single launch and a single capture) are to be added to the fault list. Faults exclusively launched by `PI` or observed by `PO` are also added. For further details, see [Using Clock Domain-Based Faults](#).

-shared

Specifies that only the faults that are launched or captured by multiple clocks are added to the fault list. This excludes all `PI` and `PO` faults. When used with the `-launch` and `-capture` options, the `-shared` option changes the meaning of that switch so that faults launched or captured by the specified clock and other clocks are added; in this case, exclusive faults are not added. For further details, please see [Using Clock Domain-Based Faults](#).

-shared_launch

Specifies that faults launched by the specified clock and other clocks are added to the fault list. An error is reported if you specify this switch without also using the `-launch` option. For further details, see [Using Clock Domain-Based Faults](#).

-shared_capture

Specifies that faults captured by the specified clock and other clocks are added to the faults list. An error is reported if you use this switch without also using `-capture`. For further details, see [Using Clock Domain-Based Faults](#).

-inter_clock_domain

Adds only exclusive faults that are launched and captured by different clock domains. For further details, see [Using Clock Domain-Based Faults](#).

-intra_clock_domain

Adds only exclusive faults that are launched and captured by the same clock domains. For further details, see [Using Clock Domain-Based Faults](#).

-stuck <0 | 1 | 01> | -slow <r | f | rf>

Specifies faults to be added based upon their stuck or slow values. Note that these options do not work with instances names, but do work for pinpath names and module names.

-bridge <0|1|01>

Specifies the type of bridging fault being added. The type can be either

0 - ba0 (bridging fault at 0)

1 - ba1 (bridging fault at 1)

01 - both (bridging fault at 0 and bridging fault at 1)

The default is 01 (both).

-aggressor_node <first | second | both>

Specifies which node in a node pair is used as the aggressor node of a bridging fault: First, Second, or Both. The default is Both. How combinations of -bridge and -aggressor_node option settings interact are shown in the following table.

-bridge	-aggressor_node	bridge_location1	bridge_location2
0	First	aggressor	ba0
0	Second	ba0	aggressor
0	Both	aggressor	ba0
		ba0	aggressor
1	First	aggressor	ba1
1	Second	ba1	aggressor
1	Both	aggressor	ba1
		ba1	aggressor
01	First	aggressor	ba0
		aggressor	ba1
01	Second	ba0	aggressor
		ba1	aggressor
01	Both	<i>aggressor</i>	<i>ba0</i>
		<i>aggressor</i>	<i>ba1</i>
		<i>ba0</i>	<i>aggressor</i>
		<i>ba1</i>	<i>aggressor</i>

In the default case, four faults are added for each node pair defined (this condition is shown italicized in the table above.) Duplicate faults are not included in the fault list. A message is issued when the `add_faults` command executes, indicating how many new faults were added to the list.

-dynamic_bridge <r|f|rf>

Specifies the type of dynamic bridging fault to be added to the faults list. The type can be any of the following:

r - bsr (bridge slow to rise)

f - bsf (bridge slow to fall)

rf - both (bridge slow to rise and bridge slow to fall)

The default is rf (both).

-dominant_node <first | second | both>

Specifies which node in a node pair is used as the dominant node of a dynamic bridging fault: *first*, *second*, or *both*. The default is *both*. The following table shows how combinations of `-dynamic_bridge` and `-dominant_node` option settings interact:

-Dynamic_bridge	-Dominant_bridge	Dynamic_bridge_location1	Dynamic_bridge_location2
<i>r</i>	<i>First</i>	<i>Dominant</i>	<i>bsr</i>
<i>r</i>	<i>Second</i>	<i>Bsr</i>	<i>dominant</i>
<i>R</i>	<i>Both</i>	<i>Dominant</i>	<i>bsr</i>
		<i>Bsr</i>	<i>dominant</i>
<i>f</i>	<i>First</i>	<i>Dominant</i>	<i>bsf</i>
<i>F</i>	<i>Second</i>	<i>bsf</i>	<i>dominant</i>
<i>F</i>	<i>Both</i>	<i>Dominant</i>	<i>bsf</i>
		<i>Bsf</i>	<i>dominant</i>
<i>Rf</i>	<i>First</i>	<i>Dominant</i>	<i>bsr</i>
		<i>Dominant</i>	<i>bsf</i>
<i>Rf</i>	<i>Second</i>	<i>bsr</i>	<i>dominant</i>
		<i>bsf</i>	<i>dominant</i>
<i>Rf</i>	<i>Both</i>	<i>Dominant</i>	<i>bsr</i>
		<i>Dominant</i>	<i>bsf</i>
		<i>Bsr</i>	<i>dominant</i>
		<i>Bsf</i>	<i>dominant</i>

In the default case, four faults are added for each node pair defined (this condition is shown italicized in the table above.) Duplicate faults are not included in the fault list. A message is issued when the `add_faults` command executes that indicates how many new faults were added to the list.

-max_invalid_report *d*

Reports why a bridge pair was rejected. The *d* argument specifies maximum number of invalid bridging fault nodes for which to report the reason for rejection.

Allowed Command Modes

Test

Description

Use this command to create a list of faults for fault simulation or test generation. Allowed fault sites include top-level ports and input and output pins of cells, which have a netlist-defined pin name. The `nofault` attribute can be used to prevent selected faults from being placed in the fault list.

Note the following:

- When the Bridging fault model is used, only the following options of the `add_faults` command are supported:
`[-bridge_location bridge_location1bridge_location2] [-node_file name]> [-bridge <0|1|01>] [-aggressor_node <first | second | both>]`
- Faults defined in an external file can also be added to a fault list using the `read_faults` command.
- All the `add_faults` options, other than `-launch` and `-capture` are exclusive and cannot be issued at the same time. If you issue these options together, a UI error message will appear.
- When an argument for the `add_faults` command is in a Tcl script, it must be specified as a Tcl list. For example:

```
foreach_in_collection pin $mem_pins {add_faults [list [get_attribute
$pin pin_pathname] ]}
```

Examples

```
TEST> add_faults -all
TEST> report_faults -summary
      Uncollapsed Fault Summary Report
-----
fault class      code #faults
-----
Detected          DT   7325
Possibly detected PT    0
Undetectable      UD  1742
ATPG untestable   AU 18706
Not detected      ND 30447
-----
total faults 58220
test coverage 12.97%
-----
```

```
TEST> remove_faults -all
TEST> add_faults /reg0/MX1
TEST> report_faults -all
sa0 NC reg0/MX1/SE
sa1 NC reg0/MX1/SE
sa0 NC reg0/MX1/D
sa1 NC reg0/MX1/D
sa0 DI reg0/MX1/SDI
sa1 DI reg0/MX1/Q
sa0 DI reg0/MX1/Q
sa1 DI reg0/MX1/SDI
```

```
TEST> remove_faults -all
TEST> add_faults mux0/MUX2_UDP_1/Q
TEST> add_faults mux0/SL
TEST> add_faults mux0/A -stuck 1
TEST> report_faults -all
sa0 NC mux0/MUX2_UDP_1/Q
sa1 NC mux0/A
sa1 NC mux0/MUX2_UDP_1/Q
sa0 NC mux0/SL
sa1 NC mux0/SL
```

See Also

[add_nofaults](#)
[read_faults](#)
[remove_faults](#)
[report_faults](#)
[set_faults](#)
[set_netlist](#)
[Fault Classes](#)

add_lbist_seeds

Overview

This command specifies user-defined seed values that TestMAX ATPG uses to generate patterns for the DFTMAX LogicBIST tool instead of using randomly generated seeds (the default).

Syntax

```
add_lbist_seeds {list_of_seed_values}
```

Arguments

list_of_seed_values

Specifies a list of seed values as a string of 0 and 1 values. The length of each seed must be equal to the core bit pseudo-random pattern generation (PRPG) length in the design. Note that all-zeroes (000...) is a special seed value that causes the PRPG to remain in an all-zeroes state. In this case, there is no 1 value to start the sequence. You can use an all-zeroes specification to prevent the data generation process from entering a codec static state.

Allowed Command Modes

Test

Description

This command specifies user-defined seed values that TestMAX ATPG uses to generate patterns for the DFTMAX LogicBIST tool instead of using randomly generated seeds (the default). Based on the seed values, TestMAX ATPG also calculates the signature and resulting coverage.

You can specify multiple `add_lbist_seeds` commands, and you can specify multiple seed values for each command. Each call to the command appends additional seed values to the existing queued set of seed values. Each call to the [run_atpg](#) command consumes the next unused seed value in the queue.

To reset the seed operation to the default behavior, or to start a new set of seed definitions, use the `remove_lbist_seeds` command. For more information on using TestMAX ATPG with the DFTMAX LogicBIST tool, see the *DFTMAX LogicBIST User Guide*.

Examples

```
# single seed
add_lbist_seeds 1011010101001010101010101111011

# multiple seeds
add_lbist_seeds { 1011010101001010101010101111011
1101111010101010101001010101101 }

# incremental add of seeds
add_lbist_seeds 1011010101001010101010101111011
add_lbist_seeds { 1101111010101010101001010101101
1011010101001010101010101111011 }
add_lbist_seeds 1011010101001010101010101111011
```

See Also

[remove_lbist_seeds](#)

[report_lbist_seeds](#)

add_net_connections

Overview

Use this command to change the behavior of nets in a netlist.

Syntax

```
add_net_connections
< pi | po | pio | tie0 | tie1 | tiex | tiez >
<{net_names} | pin_pathname> [-port name]
[-module {names} | -all] [-disconnect | -remove]
[-buf]
```

Arguments

pi | po | pio | tie0 | tie1 | tiex | tiez

Indicates the type of net connection to be made. `pi` is primary input, `po` is primary output, `pio` is primary input/output. The "tie" values are for 0/1/x/z tied net connections.

net_name | pin_pathname

Specifies the nets for which the selected connection is to be added. The net can be identified by its [net name](#) or by an associated [pin pathname](#). A net pathname or pin pathname below the "design" level (often identified by the use of the `celldefine` directive in Verilog) cannot be the target of an `add_net_connections` command. For ideas on valid pin path names, review the fault list for the block in question.

Note: Net connections are a *deferred list* that is operated on only after `run_build_model` is executed. You should review the status messages during build for any [B29](#) violations. These are indications that the net connection target was not found.

`pin_pathname` identifies the net connected to the pin that needs a new connection. Therefore, all the sinks connected to the driving net will get the new connection.

-port name

Specifies the port name to which the connection is added. This can be only used for `pi`, `po`, or `pio` connections. If the port does not exist; it is added. If it does exist, it is reused. If a set of vectored (bussed) signals (VCS definitions with bracketed consecutive integers) are added with this command, the assumed order of the created VCS port vector matches the order in which the signals were declared. This option cannot be combined with `-remove`.

-module name

Indicates that a net connection is made for every instantiation of the selected module. The `report_modules` command identifies the valid net and pin names of the module. The `-module` option can be combined only with the `tie0`, `tie1`, `tiex`, or `tiez` connections. When the *same net name* is to be handled in multiple modules, then all net connections for this same net name need to be defined in a single `add net connections` command using the `-module` option multiple times. When modules are modified, the effect cannot be undone using the `remove_net_connections` command. To undo the changes, the original module must be re-read with the `read_netlist` command.

-all

Specifies that the `net_name` supplied is to be processed for all modules where it can be found. Without this option, only a `net_name` match at the top design level is processed. Changes made with this option cannot be undone using the `remove_net_connections` command. To undo the changes, all modules should be deleted from memory and re-read from their original source files.

-disconnect

Indicates that the net is disconnected from its original drivers. Without this option, the original driver remains. This option cannot be used if the target net is a top-level bidi (PIO) or output (PO).

-remove

Indicates that the `pin_pathname` is removed from the design. The pin pathname must specify an existing top-level port. The port is disconnected from its original driver. This option should be used together with `PI`, `PO`, or `PIO` option. It can also be used with the `tie0`, `tie1`, `tiex`, or `tiez` option if the port is an input or bidirectional port. The `-remove` option cannot be combined with any of the following: `-port`, `-module`, `-all`, or `-disconnect`. When a port is removed, it does not show up in any patterns created.

-buf

This option, when specified with the `-disconnect` option, disconnects the target input (note that it can only be used with the `tie0`, `tie1`, `tiez`, `tiex` and `pi` net connections, and has effect only when used on module input pins). When `-buf` is specified, a buffer is inserted in front of the selected pin, thus isolating it from the rest of the net. The net connection is then applied, which will disconnect the buffer and replace it with a `pi`. The buffer can be removed by regular flattening optimizations.

Allowed Command Modes

Build

Description

Use this command to change the behavior of nets in a netlist. These changes are considered the same as netlist data when performing fault simulation and test generation. By default, these changes affect the in-memory image of the design (flattened design) and are reflected in patterns that are created and written. Net connections using `-module` or `-all` also change the netlist data and affect the output of the `write_netlist` command. The `-nonet_connections_change_netlist` option of the `set_build` command can be used to change the behavior so that only the flattened design is affected, but not the modules produced using the `write_netlist` command.

This command builds a deferred list of netlist changes that are performed when the `run_build_model` command is executed. Net connections can be defined before library or design modules have been read in and are defined before the top level module is chosen. For this reason, it is not possible to completely check whether a net connection target specified is valid until the run build model is performed. When a net connection target cannot be found, a [B29](#) summary violation message is reported during the flattening process. Use the `report_violations B29` command in conjunction with the `report_net_connections` command to review net connections after the build process ends.

Note the following:

- Use of net connections that name specific net or pin path names inhibits the ability of TestMAX ATPG to use some default RAM optimization techniques during flattening. However, this is *not* the case if the net connections are limited to the `-module`, `-all`, or `-remove` options. Not being able to employ the

default RAM optimization strategy can lead to increased RAM needs for the design during flattening to 1.5X over the RAM needs without net connections. If your design is large, change the original netlist instead of using the net connections to accomplish the desired modifications.

- Any current flattened in-memory design is deleted from memory when the net connection list is changed by adding or deleting net connections.

Examples

```
BUILD> add_net_connections TIE1 GLOBAL_VCC -all
BUILD> add_net_connections TIE0 GLOBAL_GND -all
BUILD> add_net_connections TIE1 GND -module {ORC1 BEA47 IOB21 }
BUILD> add_net_connections PI level2/L1/p3/b -port new_pin
BUILD> add_net_connections PI level2/L1/out11 -disconnect
BUILD> add_net_connections PO level2/L1/out11
BUILD> add_net_connections TIE0 float1 -module nestedand
BUILD> add_net_connections PI pwr1 gnd1 -remove
BUILD> add_net_connections TIE0 enable1 -remove
BUILD> add_net_connections TIE1 enable2 -remove
BUILD> add_net_connections TIEZ enable3 -remove
```

See Also

[remove_net_connections](#)

[report_modules](#)

[report_net_connections](#)

[set_build](#)

add_nofaults

Overview

Use this command to inhibit the placement of faults at specific locations.

Syntax

```
add_nofaults
  <instance_name | pin_pathname | -module { module_name [optional_pin_
    names]} | -instance instance_name >
  [-stuck <0 | 1 | 01> | -slow <r | f | rf>]
```

Arguments

instance_name

Adds the nofault attribute to all faults associated with the hierarchical [instance pathname](#). The *instance_name* argument must be a fixed string.

pin_pathname

Adds the nofault attribute to faults on the [pin pathname](#).

-module {module_name [optional_pin_names]}

Adds the nofault attribute to one or more pins of every instantiation of a given module. The *module_name* argument can be a fixed string or a [limited regular expression](#). If an optional pin name or names is specified, then only the faults associated with that module and pin are added. If no pins are specified, then all pins of the module are used.

-instance instance_name

Adds the nofault attribute to all faults associated with the *instance_name* argument. The *instance_name* argument can be a fixed string or a [limited regular expression](#), however only one *instance_name* can be specified.

-stuck <0 | 1 | 01> | -slow <r | f | rf>

Specifies faults based upon their stuck or slow values. Note that these options do not work with instances names, but do work for pinpath names and module names.

Allowed Command Modes

DRC, Test

Description

Use this command to inhibit the placement of faults at specific locations. You must assign the nofaults attribute to faults before using the `add_faults` or `read_faults` command to inhibit adding a fault to selected locations. You can use the `add_equivalent_nofaults` command to set the nofault attribute for all faults that are equivalent to faults that currently have the nofault attribute.

Examples

```
# nofault stuck-at-1 fault of pin 'SL' of every instance
# of module MUX2H
TEST> add_nofaults -module MUX2H -stuck 1 SL
```

```
4 faults were added to nofault list.
```

```
# nofault a specific pin
```

```
TEST> add_nofaults mux3/MUX2_UDP_1/Q  
2 faults were added to nofault list.
```

```
# nofault an instance
```

```
TEST> add_nofaults mux0  
15 faults were added to nofault list.
```

```
TEST> add_nofault -instance uPOWERA/add_CTS_CNIVX8L_G6B1I1
```

```
Fast lookup enabled for 259001 instantiated modules and 385621 gates, CPU_  
time=1.00 sec, Memory=5MB
```

```
4 faults were added to nofault list.
```

```
TEST> add_nofault -instance uPOWERA/uRAM40K/M322RAM48K_8K6_NOBACKA0_b/U.*
```

```
1342 faults were added to nofault list in wildcard instance  
uPOWERA/uRAM40K/M322RAM48K_8K6_NOBACKA0_b/U.*.
```

See Also

[add_equivalent_nofaults](#)

[read_nofaults](#)

[report_nofaults](#)

[remove_nofaults](#)

add_pi_constraints

Overview

Use this command to define constant logic values on top-level ports.

Syntax

```
add_pi_constraints
  <0 | 1 | x | z> [-all_bidis | -all_bidis_except_clocks |
    {port_names} | -all | -all_except_clocks]
```

Arguments

0 | 1 | x | z

Indicates the constrained value for the selected primary input or bidirectional port. Only a port connecting to circuitry that can accept a Z state is allowed to be constrained to a Z value.

-all_bidis | -all_bidis_except_clocks

Selects the set of all BIDI pins, which is useful when setting all BIDs to Z (constraining BIDs to output mode).

{port_names} | -all | -all_except_clocks

Indicates the site of the PI (Primary Input) constraint. PI constraints can be placed only on primary input or bidirectional ports. Multiple [port names](#) can be defined on a single command, but they must all share the same constrained value.

Allowed Command Modes

DRC

Description

Use this command to define constant logic values on top-level ports. The ATPG algorithm generates only patterns that satisfy the defined list of PI constraints.

A PI constraint translates into a fixed port value during the application of the various capture procedures of the STIL procedure file.

By default, a PI constraint will also be placed on the defined pin during procedures such as the test_setup, load_unload, and shift — unless you explicitly override it using an assignment statement in the procedure. This is done to prevent unnecessary switching of PIs when entering the shift operation and to constrain inputs when last-shift is chosen for the launch method used for transition delay testing.

If you want the constrained value to always be placed on the PI constraint in a procedure, follow one of these guidelines:

- Set the values in a vector at the beginning of the procedure and leave the vector unchanged throughout the procedure.
- Use an F statement in the capture procedure to specify that the ports have constant values throughout the procedure; and do not change these values throughout the procedure.

See [Controlling Bidirectional Ports in STIL](#) for an example.

Note the following:

- A port previously defined as a clock can be constrained using the `add_pi_constraints` command, but only to its off value.
- An input mask is accomplished by defining a PI constraint of X on an input.
- To avoid measuring a primary output port, use the `add_po_masks -all` command.
- PI constraints can also be defined directly in a STIL procedure file.

Examples

```
DRC> add_pi_constraints 1 TEST_MODE
DRC> add_pi_constraints 0 TICK
DRC> report_pi_constraints
port_name  constrain_value
-----
/TEST_MODE 1
/TICK      0
```

Why would we mask output, is this only during shift or capture or both
how to choose (criteria) which OP to mask ,

```
DRC> add_pi_constraints 1 {TEST_MODE TICK CLK}
DRC> report_pi_constraints
port_name  constrain_value
-----
/TEST_MODE 1
/TICK      1
/CLK       1
```

Why constraining clock even if we constrain it to its OFF state should i constraint reset as well , what will happen

See Also

[report_pi_constraints](#)
[remove_pi_constraints](#)

add_pi_equivalences

Overview

Use this command to define constant logic relationships between top-level ports.

Syntax

```
add_pi_equivalences
  port_name
  [-invert | -differential | -same_polarity]
  {port_name_list}
```

Arguments

port_name

Specifies the initial [port](#) for which an equivalence relationship is defined. This port must be a primary input or bidirectional port.

-invert

Specifies that the following ports is constrained to be at an inverted value relative to the first port in the list.

-differential

This option is similar in function to the `-invert` option with two differences. The first is that only two pins are accepted. The second is that pins declared as having a `-differential` relationship that are also clocks retain that relationship when clock grouping is enabled. A differential clock relationship formed with the `-invert` option might be ignored by clock grouping. Pins declared as having a differential relationship is driven to opposite values by generated patterns.

-same_polarity

This option is similar in function to the `-differential` option. The `-same_polarity` option only accepts two pins, but these two signals must have the same clock_off state (unlike differential clocks). Pins that are declared to have a `same_polarity` relationship will always be active at the same time; this relationship is maintained even if clock-grouping options are applied (pins with `same_polarity` will not be ungrouped). You can use the `-same_polarity` option to restrict pattern generation for equivalent external clocks, however you should be aware that this is a restriction on the clocking opportunities in the design, and might impact coverage. Note that the `-same_polarity` option has been replaced by the `-dynamic_clock_equivalencing` option of the [set_drc](#) command. By default, pin equivalences are automatically deleted for clocks during DRC when dynamic clock grouping is enabled. You can use the `-dynamic_clock_equivalencing disturb` option of the [set_drc](#) command to allow the use of pin equivalences for clocks when dynamic clock grouping is enabled using normally calculated disturb cells. The `nodisturb` option allows the use of pin equivalences for clocks when dynamic clock grouping is enabled with no consideration for disturb cells.

port_name_list

Specifies the [ports](#) that is constrained to be at an equivalent (or complementary if the `-invert` option is selected) value relative to the initial port.

Allowed Command Modes

DRC

Description

Use this command to define constant logic relationships between top-level ports. Two or more ports can be declared to have an equivalence, which means they should be driven to the same value by ATPG patterns, or they can be declared to have an inverse equivalence, which means they should be driven to the binary complement of each other by ATPG patterns. The ATPG algorithm generates only patterns that satisfy the defined list of PI equivalence relations and will avoid patterns that do not.

When two ports are to be declared equivalent or differential clocks, first define them as clocks, then define the equivalence relationship. For example:

Recommended:

```
add_clocks 0 CLK1 -shift
add_clocks 0 CLK2 -shift
add_pi_equivalences {CLK1 CLK2}
```

Not Recommended:

```
add_pi_equivalences {CLK1 CLK2}
add_clocks 0 CLK1 -shift
add_clocks 0 CLK2 -shift
```

This command is commonly used to define the relationship of differential inputs. TestMAX ATPG supports differential inputs for clocks and data ports. However, special consideration might be required for [differential scan chain input modeling](#) for pattern creation.

DRC rule checking considers PI equivalence relationships only for clock rule checking. No other DRC checks consider potential effects of PI equivalence relationships.

A PI equivalence translates into a specific matched or complementary value during the application of the various `capture_XXX` procedures of the STL procedure file. A PI equivalence *does not* cause the defined pins to have any specific value in the `load_unload`, `shift`, `test_setup`, or other procedures. If you want specific values placed on pins during `test_setup` or scan chain loading and unloading, you need to edit the `test_setup`, `load_unload`, or `shift` procedures to explicitly set the desired port values.

An equivalence relationship between internal circuit points or a top-level port and an internal circuit point is created using a combination of `add_atpg_primitives` and `add_atpg_constraints` commands.

Note: PI equivalences can also be defined directly in a STIL procedure file within capture clock procedures. However, if you use an STIL procedure file that describes a clock as having a differential relationship, you still need to use the commands shown in the following example before running the STIL procedure file. If you don't, you will get a V12 warning, and the clocks will not be pulsed during ATPG:

```
add_clocks 0 CLK_216A_P -shift
add_clocks 1 CLK_216A_N -shift
add_pi_equivalences CLK_216A_P -differential CLK_216A_N
```

Examples

```
DRC> add_pi_equivalences {NL0 NL1 NL2}
DRC> add_pi_equivalences ENA_P -inv ENA_N
DRC> report_pi_equivalences
port_name equivalent pins
-----
NL0          NL1 NL2
ENA_P        ~ENA_N
```

See Also

[add_atpg_constraints](#)

[add_atpg_primitives](#)

[remove_pi_equivalences](#)

[report_pi_equivalences](#)

add_po_masks

Overview

This command defines masks on output and bidirectional ports. This might be useful when there are circuit outputs that might not be observable in a particular tester or system environment (such as boundary scan).

Syntax

```
add_po_masks  
  <port_name | -all>
```

Arguments

port_name

Specifies the port whose output is to be masked. The test generator and fault simulator do not give fault detection credit for these ports and the expected value is set to "X". The PO mask is used only for nonshift (system or broadside) cycles and does not mask data from scan chain outputs during scan-shift cycles.

-all

Indicates that all top-level primary outputs and bidirectional ports is masked. The use of `-all` is a convenient way to add a PO mask to all output pins.

Allowed Command Modes

DRC

Description

This command defines masks on output and bidirectional ports. This might be useful when there are circuit outputs that might not be observable in a particular tester or system environment (such as boundary scan).

Note: To avoid placing a value on an input port, define a PI constraint of "X" using the `add_pi_constraints` command.

Examples

```
DRC> report_po_masks
Warning: Requested report contained no entries. (M13)
DRC> add_po_masks {D0 D1 D2 D3 }
DRC> report_po_masks
output port_name
-----
/D0
/D1
/D2
/D3
```

```
DRC> remove_po_masks -all
DRC> add_po_masks -all
DRC> report_po_masks
-----
/D0
/SDO2
/COUT
/D0
/D1
/D2
/D3
```

See Also

[add_pi_constraints](#)

[report_po_masks](#)

[remove_po_masks](#)

[Pin Name](#)

add_scan_chains

Overview

This command is used to define scan chains. It is an alternative to defining scan chains in the STIL procedure file and is intended to be used before issuing a `write_drc_file` command to create a template STIL procedure file or before using the `run_drc` command without a DRC file argument.

Syntax

```
add_scan_chains  
  chain_name input_pin output_pin
```

Arguments

chain_name

The chain name is the symbolic name used to reference the scan chain. Select a name that has not already been used as the label of a scan chain. It is suggested you begin the chain name with an alphabetic character, as some pattern output formats have this restriction.

input_pin

This is the name of the top-level design port that operates as the scan chain input. When a differential scan chain input exists, use the `add_pi_equivalences` command to define the relationship between the two input pins, but define only one as the scan chain input.

output_pin

This is the name of the top-level design port that operates as the scan chain output corresponding to the specified input. The output pin must not have been specified as the scan input or output of another scan chain. Differential scan chain outputs are not supported.

Allowed Command Modes

DRC

Description

This command is used to define scan chains. It is an alternative to defining scan chains in the STIL procedure file and is intended to be used before issuing a `write_drc_file` command to create a template STIL procedure file or before using the `run_drc` command without a DRC file argument.

If a DRC file is specified with the `run_drc` command, any scan chains defined using the `add scan chains` command are ignored. See the `run_drc` command for this and other rules of priority.

Examples

```
DRC> add_scan_chains chain_1 si[1] so[1]
DRC> add_scan_chains chain_2 si[2] so[2]
DRC> add_scan_chains C3 si[3] so[3]
DRC> report_scan_chains
chain    group length input_pin output_pin
-----
chain_1  sg0     0      si[1]    so[1]
chain_2  sg0     0      si[2]    so[2]
C3       sg0     0      si[3]    so[3]
```

See Also

[report_scan_chains](#)
[remove_scan_chains](#)
[run_drc](#)
[write_drc_file](#)

add_scan_enables

Overview

This command is used to specify that a pin is to be initialized to a specific value before scan chain shifting. It is an alternative to defining a load_unload preamble vector in the STIL procedure file and is intended to be used before issuing a `write_drc_file` command to create a STIL procedure template or when a `run_drc` command is used without a procedure file.

Syntax

```
add_scan_enables
  <0 | 1 | z> pin_name
```

Arguments

0 | 1 | z

This defines the port's state during scan chain shifting. The port is initialized to this value in a load_unload preamble cycle before the shift procedure.

pin_name

This is the name of the top-level design port to be initialized during scan shifting. Multiple pins can be specified in the same command if they are all initialized to the same value.

Allowed Command Modes

DRC

Description

This command is used to specify that a pin is to be initialized to a specific value before scan chain shifting. It is an alternative to defining a load_unload preamble vector in the STIL procedure file and is intended to be used before issuing a `write_drc_file` command to create a STIL procedure template or when a `run_drc` command is used without a procedure file.

The `add_scan_enables` command does not consider the affect of any PI equivalences that might exist or might have been previously defined. If you define a scan enable for one pin in a PI equivalence pin group, you should also explicitly define the appropriate scan enable state for all other pins.

If a DRC file is specified with the `run_drc` command, there is a potential for conflict between the scan enables defined with the `add_scan_enables` command and the preamble vectors of the load_unload procedure defined in the DRC file. See the `run_drc` command for rules of priority.

Examples

```
DRC> add_scan_enables 1 SCAN_EN
DRC> add_scan_enables Z {bidi[0] bidi[1] bidi[2]}
DRC> report_scan_enables
value port
-----
1      SCAN_EN (6)
Z      bidi[0] (3210)
Z      bidi[1] (3211)
Z      bidi[2] (3212)
```

See Also[report_scan_enables](#)[remove_scan_enables](#)[run_drc](#)[write_drc_file](#)[Defining Scan Chain Load/Unload in STIL](#)

add_slow_bidis

Overview

This command is used to modify the behavior of BUS primitives associated with bidirectional ports such that when these ports are in output mode, the values driven by internal tristate gates only propagate to the bidirectional port's output path.

Syntax

```
add_slow_bidis  
  <port_name | -all>
```

Arguments

port_name

Specifies the output of the BUS primitive that is being modified to an X.

-all

Indicates that all BUS primitives are modified to an X.

Allowed Command Modes

DRC

TEST

Description

This command is used to modify the behavior of BUS primitives associated with bidirectional ports such that when these ports are in output mode, the values driven by internal tristate gates only propagate to the bidirectional port's output path. Any input mode gates that receive data from these ports is simulated with an X in output mode.

This command prevents TestMAX ATPG from generating "loopback" tests; for example, tests that would cause a value to come from the internal driver to a bidirectional port and then loop back to the internal logic normally driven when the bidirectional port is in input mode. Since this sort of test risks activating a path that is not used in a circuit's normal functionality, and which might be longer than a single clock cycle, it is frequently desirable to avoid such tests during at-speed testing, such as transition and path delay testing.

See Also

[remove_slow_bidis](#)

[report_slow_bidis](#)

add_slow_cells

Overview

This command is used for at-speed testing to modify the behavior of DFF or DLAT cells when these cells drive multi cycle or false paths.

Syntax

```
add_slow_cells  
  <instance_path | gate_id>
```

Arguments

instance_path | gate_id

Specifies the [instance pathname](#) or the [gate ID](#) of the DFF or DLAT element whose simulation behavior is being modified.

Allowed Command Modes

DRC

Description

This command is used for at-speed testing to modify the behavior of DFF or DLAT cells when these cells drive multi cycle or false paths. A transition launched from these cells is simulated as an X. Alternately the `add_capture_masks` command can be used to mask DFF or DLAT cells that capture multi cycle or false paths. However, the `add slow cells` command is usually preferable as it allows for steady-state values to be used in generating at-speed tests and for those steady-state values to propagate and be captured along slower paths (might be useful when combining at-speed tests with stuck-fault tests).

For Basic-Scan ATPG, the output is masked if there is a transition on the last-shift cycle. For Fast-Sequential or Full-Sequential ATPG, the output is masked if there is a transition on any ATPG clock cycle.

In multi clock domains, this command has the same affect in all ATPG modes and fault models. For example, because the paths between clock domains are thought of in terms of a timing exception, you could do one of two things for an at-speed test:

Ensure that the exception sources are held at a constant value during launch by using this command on flip-flops at the beginning of the paths

Ensure that the exception sinks are masked during capture by using the `add_capture_masks` command on the flip-flops at the end of the paths. This option is generally less desirable than the above option, because more faults will usually become untestable.

See Also

[remove_slow_cells](#)
[report_slow_cells](#)

add_waveform_signals

Overview

Adds waveforms associated with the *pin_pathname*, *gate ID*, or *net_name*, or all gates to the simulation waveform viewer.

Syntax

```
add_waveform_signals  
  < pin_pathname | gate_id | net_names | -all >
```

Arguments

pin_pathname | *gate_id* | *net_names* | **-all**

Specifies the [pin pathname](#) or the [gate ID](#) of the element whose simulation behavior is being monitored. If you specify the **-all** option, TestMAX ATPG adds all gates to the simulation waveform viewer.

Allowed Command Modes

DRC and Test

Description

Adds waveforms associated with the *pin_pathname*, *gate ID*, or *net_name*, or all gates to the simulation waveform viewer.

If you enter a net name on the command line, the simulation waveform viewer keeps the net name as the signal name. If you pick a net from the GSV and add it to the simulation waveform viewer, the net name is the signal name. The `remove_waveform_signals` command must have same options as add waveform signals.

See Also

[remove_waveform_signals](#)

Analyze Commands

This section describes the following commands:

- [analyze_buses](#)
- [analyze_compressors](#)
- [analyze_faults](#)
- [analyze_feedback_path](#)
- [analyze_nonscan_loading](#)
- [analyze_simulation_data](#)
- [analyze_test_points](#)
- [analyze_violation](#)
- [analyze_wires](#)

analyze_buses

Overview

Use this command to analyze potential problems associated with buses.

Syntax

```
analyze_buses  
  <gate_id... | -all>  
  [-exclusive [first | all] | -prevention | -zstate ]  
  [-patterns number]  
  [-random]  
  [-store]  
  [-update]
```

Arguments

gate_id

Specifying one or more numeric BUS [gate IDs](#) indicates that the BUS analysis should be performed for those [primitive IDs](#). Detailed information of the analysis is given. An error is reported if the primitive ID is not an ATPG BUS primitive.

-all

Specifies that the BUS analysis is to be performed for all BUS devices. When the analysis is completed, a summary message is displayed.

-exclusive [first | all]

Specifies that the BUS analysis is to check for mutually exclusive control circuitry among BUS driver sources. If a mutually exclusive condition is identified, the BUS device can never be in contention and can be moved to the PASS category. This exclusivity checking is the default BUS analysis.

You can select what happens if the analysis fails, in other words, if a pattern is found which proves that more than one BUS driver is turned on at the same time. Use of **First** causes the analysis to stop on the first BUS device that fails this check. Use of **all** causes analysis to continue on to the next BUS device until all BUS devices have been processed and reported. The default is **first**.

Combining **-exclusive all** with **-all** for the list of BUS devices to analyze and report is not advised as it might consume significant amounts of CPU time and produce a large report. For this reason, combining these two options is prohibited and attempting to use them will result in an error message. Use of **First** is recommended except for experienced debugging.

-prevention

Specifies that the BUS analysis should try to prevent contention on all BUS devices simultaneously.

-zstate

Specifies that the BUS analysis should check for the ability of a BUS to have no drivers turned on, causing a floating net or Z-state condition.

-patterns *number*

Specifies the number of random patterns to be used for identifying contention-free buses.

-random

Specifies that random patterns are to be used for identifying contention-free buses.

-store

Causes the random patterns used for identifying contention-free buses to be stored in the internal pattern buffer.

-update

Specifies that the information learned from the BUS analysis for contention or Z-state status should be used to update the results of the DRC BUS analysis. Without this option, the BUS analysis results are reported to the transcript but are not used to update the DRC results.

Allowed Command Modes

DRC, Test

Description

Use this command to analyze potential problems associated with buses. You can perform the following analyses:

- Individual buses can be analyzed to determine if they are capable of being placed at a contention condition. If a pattern can be created to satisfy a contention condition, the simulation values for that pattern are displayed by setting pindata reporting to pattern 0.
- Individual buses can be analyzed to determine if they are capable of being placed in a contention-free condition. If a pattern is found to create a contention-free condition, the simulation values for that pattern is displayed by setting pindata reporting to pattern 0.
- All buses can be analyzed to determine if they are capable of simultaneously being placed at a contention-free condition. If a pattern can be created to satisfy a contention-free condition, the simulation values for that pattern are displayed by setting pindata reporting to pattern 0.
- Individual buses can be analyzed to determine if they are capable of being placed at a "Z" state. If a pattern can be created to satisfy a Z-state condition, the simulation values for that pattern are displayed by setting pindata reporting to pattern 0.

Note: The bus analysis uses test generation parameters that are affected by the `set_atpg` and `set_contention` commands.

Examples

```
TEST> report_buses -contention fail
      contention    zstate    #drivers
gate_id status/capture status    strong/weak behavior_data
-----
  85346 fail unstable    bidi      5 0
  85359 fail unstable    bidi      5 0
TEST> analyze_buses 85359
Bus 85359 (47526,47461) failed contention check (values available in
pattern 0).
TEST> analyze_buses 85346
Bus 85346 (47537,47512) failed contention check (values available in
```

```
pattern 0).
```

```
TEST> analyze_buses -all  
Bus Contention results: #pass=257, #bidi=31, #fail=289, #abort=0, CPU  
time=500.12
```

```
TEST> analyze_buses -all -zstate -update  
Bus Zstate ability results: #pass=160, #bidi=128, #fail=231, #abort=58,  
CPU time=104.2
```

```
TEST> analyze_buses -all -prevention  
Busses passed contention prevention (values available in pattern 0).
```

See Also

[report_buses](#)

[set_atpg](#)

[set_contention](#)

[Understanding the analyze_buses_report](#)

analyze_compressors

Overview

This command enables you to run ATPG by creating a virtual compressor.

Syntax

```
analyze_compressors
  <-num_chains number>
  <-num_inputs number>
  <-num_scanouts number>
  [-num_clockchains number]
  [-nodiag]
  [-num_xchains number | auto]
  [-use_all_unload_nodes | -nouse_all_unload_nodes]
  [-verbose]
  [-xchain_analysis]
  [-xtolerance <default | high>]
```

Arguments

-num_chains *number*

Specifies the number of internal chains used for the analysis.

-num_inputs *number*

Specifies the number of scan inputs used for scan compression implementation.

-num_scanouts *number*

Specifies the number of scan outputs used for scan compression implementation.

-num_clockchains *number*

The default is 0 (no clock-chains). Indicates that *number* scan chains should be configured as clock chains when designing the load decompressor. Clock chains must have dedicated, load-mode independent scan inputs, and these inputs cannot be used to control X-tolerant mode selection for the unload compressor.

-num_xchains *number* | auto

The default is 0 (no X-chains). When a non-zero value is given, the first *number* chains are considered X-chains when creating the unload compressor. There is no effect on the load decompressor. When running `analyze_compressors` on a scan design, allocation of scan cells into chains during the analysis does nothing special for X-chains. If `auto` is selected, first the number of X-chains is determined based on X-cells analysis (see the `-xchain_analysis` option); user-defined capture masks are also considered. Then, internal scan chains are created so that all X-cells (and only X-cells) are placed on X-chains.

-nodiag

Disables the requirement to generate a compressor with high precision diagnosis. By default, the `analyze_compressors` command generates a compressor with failure diagnosis; this requires a specific number of scan outputs for a given number of chains.

-use_all_unload_nodes | -nouse_all_unload_nodes

The `-use_all_unload_nodes` option has the following effect when generating an unload compressor:

Adds a variety of different multiple-observability modes.

Fills up unused modes to avoid R21 violations

The `-nouse_all_unload_modes` option is provided only for backward-compatibility.

-xchain_analysis

Performs X-chain analysis during the TEST command mode of a TestMAX ATPG session.

This analysis calculates the probability of X for each gate in the circuit by simulating 1024 random patterns under the following conditions:

Constrained PIs are set to their constrained value.

Clocks are set to their off state.

Enable pins are set to their non-shift state, if identified. You can define enable pins using the `add_scan_enables` command; in this case, the state is assumed to be the shift mode value.

Bidirectional pins are set to Z, but are reset to a random binary value for patterns in which the internal driver is at Z. In this case, the `-bidi_fill` option of the [set_simulation](#) command must be specified.

All other PIs are set to random values.

Constrained scan cells are set to their constrained value.

Constant value cells are set to their constant value.

Load cells are set to their load values.

Transparent latches are initialized to X and updated during simulation.

Other scan cells are set to random values.

All other state elements are set to X.

A report is provided for each scan chain, which indicates the average X-capture probability for its scan cells. After it is calculated, the X-probability can be displayed for reported and displayed gates using the `set_pindata -gatedata` command.

This data is automatically deleted when leaving the TEST command mode.

After running the `-xchain_analysis` option, the X-probabilities of every gate in the design are stored and can be displayed by the `set_pindata -gatedata` command. The number shown for each gate is the percentage of simulated patterns for which the gate was at X value.

-xtolerance <default | high>

The `default` keyword specifies no extra X-tolerance gating, and only includes X-tolerance in an XOR compressor. The `high` keyword specifies full X-tolerance gating with shared load/unload controls. For multithreading, you must set the `-xtolerance` option to high. If you don't set this option, non-threaded processing is used for analysis instead of multithreading, and an [M729](#) message is issued.

Allowed Command Modes

DRC, Test

Description

This command creates a virtual compressor and enables you to perform what-if analysis to examine the benefits of using scan compression on your design. You can select parameters for the compressor and the decompressor before running ATPG.

For what-if analysis, you must separately run ATPG in scan mode and then compare the results to the output produced by the `analyze_compressors` command. This command deletes the internal patterns after it is finished running.

Note the following:

- You cannot run the `analyze_compressors` command when the [set_simulation](#) -shift_cycles command is set to a non-zero value.
- You must set the -xtolerance option to high.

The `analyze_compressors` command prints out estimated data volume and test volume reductions you can achieve. These estimates assume that you have exactly same coverage and same pattern count in regular scan ATPG. You also get an estimate of number of gates per chain required to create the compressors you have chosen.

The `analyze_compressors` command uses the following equations for the test volume reduction and data volume reduction estimates:

```
Cycle reduction per pattern = (Length of longest scan chain in scan mode)
-----
(Length of longest scan chain in scan compression mode)
```

```
Scan Test Data Volume per pattern = 3 x Length of longest scan chain x
number of scan chains
Scan Compression Test Data Volume per pattern = Length of longest scan chain
x
(Number of scan inputs +
2 x Number of Scan Outputs )
```

```
Data reduction per pattern = (Scan Test Data Volume per pattern)
-----
(Scan Compression Test Data Volume per pattern)
```

Examples

```
Data reduction per pattern: 11.81X
Cycle reduction per pattern: 11.81X
Compressors area overhead per internal chain: 9.0 equivalent two-input NAND
gates
```


analyze_faults

Overview

This command prints a series of reports that are used to analyze why faults are not detected.

Syntax

```
analyze_faults
  << pin_pathname -stuck <0 | 1> [-observe gate_id]>
    | delay_path_name -slow <r|f> [-observe gate_id]>
    | instance_name.. >>
  < bridge_location1 bridge_location2 <-bridge <0|1> | -dynamic_bridge <r|f>
  >>
  [-class fault_class]
  [-display]
  [-fast_sequential | -full_sequential]
  [-fault_simulation]
  [-max max_faults]
  [-source_points min_percent max_points]
  [-verbose]
```

Arguments

***pin_pathname* -stuck <0 | 1>**

The *pin_pathname* argument specifies the fault site to be analyzed by its [pin pathname](#). The analysis consists of a controllability check, and a detectability check. TestMAX ATPG does not use the fault simulation engine to validate fault detection unless the `-fault_simulation` option is also specified. This means that even though a fault is found to be controllable and detectable by ATPG, it might be rejected by the fault simulation engine if it is not also detected by that analysis.

The `-stuck` argument specifies the stuck value of the selected fault.

`-observe gate_id`

Indicates the fault is only considered detected if it propagates to the specified [primitive ID](#). This option is ignored for fault classes. The default behavior considers all potential observe points.

instance_name

Specifies the names of the instances for analysis.

`-class fault_class...`

Selects a specific [class of faults](#) for analysis. You can use this option multiple times in the command. This option is not supported for path delay or IDDQ faults.

***delay_path_name* -slow <r | f>**

The *delay_path_name* argument specifies the name of the path to be analyzed when the fault model is set to Delay_path. The analysis proceeds by attempting to satisfy all [off-path inputs](#), and stops when these cannot be satisfied. The `-slow` option specifies either

f) fault for analysis when analyzing [Delay path](#) faults. Note that you can use the `-slow` option with the pin path name.

`bridge_location1 bridge_location2 < -bridge <0|1> | -dynamic_bridge <r|f> >`

bridge_location1, 2 - Specifies the victim and aggressor nodes, respectively.

`-bridge` - Specifies the type of bridging fault. The type can be either:

0 - ba0 (bridging fault at 0)

1 - ba1 (bridging fault at 1)

`-dynamic_bridge` - Specifies the type of dynamic bridging fault. The type can be either:

r - bsr (bridge slow to rise)

f - bsf (bridge slow to fall)

`-fast_sequential`

Selects Fast-Sequential ATPG methods for the analysis. The default behavior is to perform the analysis using Basic-Scan (combinational) ATPG methods unless either this option or the `-full_sequential` option is selected.

`-full_sequential`

Selects Full-Sequential ATPG methods for the analysis. The default behavior is to do the analysis using Basic-Scan (combinational) ATPG methods unless either this option or the `-fast_sequential` option is selected.

`-fault_simulation`

Specifies that in addition to the normal fault analysis, a fault simulation should also be performed to ensure that any fault detection credit given during the fault analysis is validated by fault simulation. By default, this simulation is not performed.

During the ATPG process, fault classification comes from detection due to generated patterns but is also double-checked by performing a fault simulation on the potential pattern. So to get an equivalent analysis of the fault site to what would occur during the ATPG process, it is necessary to both analyze the fault from an ATPG pattern generation perspective and also check the resulting pattern with a fault simulation. It is sometimes the case that ATPG pattern generation succeeds and the fault would be considered class=DS, but fault simulation fails and the fault cannot be classified as detected.

`-max_max_faults`

Specifies the maximum number of faults to report for a verbose fault class analysis for each potential problem type. The default is to report all faults with the `-verbose` option.

`-source_points min_percent max_points`

Prints a list of blockage points, blockage value source points, and constraint value source points based on the order of their effect on explained faults. The parameters for this option control the length of the report. The *min_percent* parameter is the minimum percentage of explained faults for the point to be reported. The *max_points* parameter is the maximum number of points to list in each category; 0 specifies that there is no limit. This option must be used with the `-class` option. The fault classes are supported: Undetectable (UD) and ATPG Untestable (AU), and any of their subclasses.

`-verbose`

Selects detailed information reporting during the analysis.

Allowed Command Modes

Test

Description

This command prints a series of reports that are used to analyze why faults are not detected. Note that this command is not supported when using DFTMAX Ultra with threaded or non-threaded TestMAX ATPG.

When specified for one or more classes of faults, the `analyze_faults` command provides a summary of how many times certain potential detection problems were encountered. For example: "Connected to a TLA" (transparent latch) or "Constrained."

When used on a selected fault identified by a pin pathname, this command performs a detailed combinational analysis to determine if the fault can be detected. If the fault cannot be detected, it tries to identify the source of the failed detection.

Note the following:

- The fault analysis uses test generation; these parameters are controlled by the `set_atpg`, `set_contention`, and `set_delay` commands.
- Fault analysis is not supported for the hold time fault model.
- When using this command for path delay analysis, the test generator first tries to produce a test that is a candidate for a robust test and, if that fails, tries to produce a non-robust test. When it is trying to produce a test, it proceeds through a sequence of tasks and reports on the success of each one. It continues through this sequence until it fails at one of them. When it fails, it will stop the analysis and print a report. If the command goes through the entire sequence without failure, it will report success in the summary message at the end of the fault analysis. The sequence of tasks is as follows:
 1. Generate the values necessary to launch the transition at the head of the selected path.
 2. Generate the values necessary to capture the transition at the tail of the path.
 3. Generate the values necessary to propagate the transition through the first non-launch, non-capture node in the path description.
 4. Generate the values necessary to propagate the transition through the second non-launch, non-capture node in the path description.
 5. Generate the values necessary to propagate the transition through the last non-launch, non-capture node in the path description.

Basically, the test generator is taking care of the launch node first, the capture node second, and is then moving down the path one node at a time from the launch node to the capture node until it hits a point at which it cannot succeed. When it hits this failure node in the path, then it prints out a message indicating that it has failed.

If you run `analyze_faults` in default mode, without any other switches set, the printout for a successful test will look like this:

```
-----
---
Full Sequential Analysis performed for Path Delay fault : path0 slow to
fall
-----
---
Non-robust path sensitization successful ...
Test generation to ANY observe successful - observe is DFF 14 ...
The gate_report data is now set to "seq_sim_data"
```

For an unsuccessful test, the printout will look like this:

```
-----
---
Full Sequential Analysis performed for Path Delay fault : path0 slow to
fall
-----
---
Non-robust path sensitization unsuccessful ...
Test generation of LAUNCH DFF 15 @ frame 1 (cycle 0, frame 1) successful
...
Test generation of LAUNCH DFF 15 @ frame 1 (cycle 0, frame 1) and CAPTURE
DFF 14 @ @ frame 3 (cycle 1, frame 0) successful ...
Test generation to OR 10 ( OFF-PATH input AND 9) successful ...
Test generation to MUX 12 ( OFF-PATH input PI 2) successful ...
Test generation to AND 11 ( OFF-PATH input PI 1) unsuccessful ...
Analyzing conflict data ...
    3 conflicting requirements found :
        1. PI 1 required @ 1
        2. AND 9 required @ 0
        3. DFF 15 required @ 1
The gate_report data is now set to "seq_sim_data"
```

If you set the `-verbose` option of `analyze_faults`, the printout for a successful test is as follows:

```
-----
---
Full Sequential Analysis performed for Path Delay fault :
    path0 slow to fall
    path head : output of DFF 15 1->0
    .. path tail : input 1 of MUX 12 1->0
-----
---
Strong non-robust path sensitization successful ( 0 backtrack(s), time used
= 0.02 )
Test generation to ANY observe successful -
observe is DFF 13 ( 0 backtrack(s) , time used = 0.04 ) ...
    Test pattern data :
        Load big_chain-0 : DFF 13 = 1
        Load big_chain-1 : DFF 14 = 1
        Load big_chain-2 : DFF 15 = 1
        Set WFT : LAUNCH, time = 10
        Force : PI 0 = 0
            Force : PI 1 = 1
            Force : PI 2 = 0
        Pulse Clock : PI 4 = 010
        Set WFT : CAPTURE, time = 20
        Pulse Clock : PI 4 = 010
        Calling PROOFS ...
        Launch :
            DFF 15 1->0 @ frame 1 (cycle 0, frame 1)
```

```

Delay path :
    OR 9 1->0
    AND 11 1->0
    MUX 12 1->0
Capture :
    DFF 13 1->1 @ frame 3 (cycle 1, frame 0)
Test pattern detection path:
    DFF 13 0/1
The gate_report data is now set to "seq_sim_data"

```

There are several differences between this verbose printout and the default. One difference is that the nodes at the head and tail of the path are listed, along with their required transition values. A second difference is that the type of test that was generated is identified, in this case a Strong non-robust test, which is the type of test that is a candidate for a robust test. Since robustness is determined only by the fault simulator, all the test generator can do is to identify a candidate for such a test designation. A final difference is that the verbose printout contains the actual test vector generated: the scan chain values, the PI values, the clock pulses, and the waveform tables used. This is followed by a listing of the nodes along the path and their transition values. For a verbose printout of an unsuccessful test, you will see something like the following:

```

-----
---
Full Sequential Analysis performed for Path Delay fault :
path0 slow to fall
path head : output of DFF 15 1->0
path tail : input 1 of MUX 12 0->1
-----
---

Weak non-robust path sensitization unsuccessful ( 3 backtrack(s) , time
used = 0.01 )
Test generation of LAUNCH DFF 15 @ frame 1 (cycle 0, frame 1) successful
( 0 backtrack(s), time used = 0.01 ) ...
Test generation of LAUNCH DFF 15 @ frame 1 (cycle 0, frame 1) and CAPTURE
DFF 14 @ @ frame 3 (cycle 1, frame 0) successful
( 0 backtrack(s), time used = 0.01 ) ...
Test generation to OR 10 ( OFF-PATH input AND 9) successful
( 0 backtrack(s), time used = 0.02 ) ...
Test generation to MUX 12 ( OFF-PATH input PI 2) successful
( 0 backtrack(s), time used = 0.01 ) ...
Test generation to AND 11 ( OFF-PATH input PI 1) unsuccessful
( 2 backtrack(s), time used = 0.00 ) ...
Analyzing conflict data ...
... 3 conflicting requirements found :
    1. PI 1 required @ 1
    2. AND 9 required @ 0
    3. DFF 15 required @ 1
... Calling PROOFS ...
Launch :
    DFF 15 1->0 @ frame 1 (cycle 0, frame 1)
Delay path :
    OR 10 0->1
    AND 11 0->x

```

```

    MUX 12 0->x
Capture :
    DFF 14 x->0 @ frame 3 (cycle 1, frame 0)
The gate_report data is now set to "seq_sim_data"

```

As with the case of a successful test, the path head and tail are identified and so is the type of test attempted. In this case, the type of test attempted was a Weak non-robust test, which is not a candidate for a robust test designation. In cases where the Strong non-robust test has been attempted and failed, its failure is considered to be implied and there is no printout for it; instead all that is noted in the printout is the result of the attempt for the Weak non-robust test. Consider the verbose printout in this case in more detail.

```

Weak non-robust path sensitization unsuccessful ( 3 backtrack(s), time used
= 0.01 ) ...

```

The line above indicates only that the overall attempt at a Weak non-robust test was unsuccessful.

```

Test generation of LAUNCH DFF 15 @ frame 1 (cycle 0, frame 1) successful
( 0 backtrack(s) , time used = 0.01 ) ...

```

The line above indicates that the test generator was able to launch the transition successfully at DFF 15, the head of the path.

```

Test generation of LAUNCH DFF 15 @ frame 1 (cycle 0, frame 1) and CAPTURE
DFF 14 @ @ frame 3 (cycle 1, frame 0)
successful ( 0 backtrack(s), time used = 0.01 ) ...

```

The line above indicates that the test generator was able to both launch the transition at the head of the path and capture it at the tail of the path.

```

Test generation to OR 10 ( OFF-PATH input AND 9) successful
( 0 backtrack(s), time used = 0.02 ) ...

```

The line above indicates that the test generator was able to propagate the transition successfully through the first gate on the path after the launch node, the OR gate 10, and that the off-path input that it has set to accomplish this comes from the AND gate 9.

```

Test generation to MUX 12 ( OFF-PATH input PI 2)
successful ( 0 backtrack(s), time used = 0.01 ) ...

```

The line above indicates that the test generator was able to propagate the transition successfully through the second gate on the path after the launch node, the MUX gate 12, and that the off-path input that it has set to accomplish this comes from the PI 2.

```

Test generation to AND 11 ( OFF-PATH input PI 1) unsuccessful
( 2 backtrack(s), time used = 0.00 ) ...

```

The line above indicates that the test generator failed when it tried to propagate the transition through the third gate on the path after the launch node, the AND gate 11, and that the off-path input that it tried unsuccessfully to put a propagating value on was the PI 1.

```

Analyzing conflict data ...
... 3 conflicting requirements found :
    1. PI 1 required @ 1
    2. AND 9 required @ 0

```

3. DFF 15 required @ 1

When such a failure occurs, the test generator will take all of the requirements that exist up to that point in the sequence of tasks and will try to remove them one at a time to see if it can satisfy the remaining requirements. If it is still unable to satisfy the remaining requirements even with one requirement removed, it will simply discard that requirement on the assumption that it is not involved in the failure. If it is able to satisfy the remaining requirements with that one removed, however, it will save that as one of the requirements that is critically involved in the failure. The lines just above give all the requirements that were identified as being critically involved in the failure for this particular path, namely the PI 1 required at a value of 1, the AND gate 9 required at a value of 0, and the DFF 15 required at a value of 1. Note that even though the PI 2 was set during the attempt at path sensitization, it was not identified as one of the nodes involved in the failure because the test generator was unable to resolve the other requirements even with the requirement on PI 2 removed from the list. It might be possible to trace through the circuitry surrounding the nodes involved in a failure and determine why the attempt to generate a test was unsuccessful.

The final switch that can be set on the `analyze_faults` command is the `-fault_simulation` flag. In the case of path delay, all that this does is to add a line to the printout for a successful test indicating the type of test that the fault simulator has identified. Examples of this would be

```
Fault simulation status : DR
```

Indicating a robust test, or

```
Fault simulation status : DS
```

Indicating a non-robust test.

Examples

```
TEST> analyze_faults -class no
Fault analysis summary: #analyzed=106, #unexplained=95.
11 faults are connected to CLKPO.
3 faults are connected to CLOCK.
11 faults are connected to TS_ENABLE.
8 faults are connected from CLOCK.
```

```
TEST> analyze_faults -class an
Fault analysis summary: #analyzed=2375, #unexplained=17.
1602 faults are untestable due to constrain values.
674 faults are untestable due to constrain value blockage.
194 faults are connected to CLKPO.
106 faults are connected to CLOCK.
10 faults are connected to SET.
55 faults are connected to RESET.
189 faults are connected to TS_ENABLE.
116 faults are connected from CLOCK.
25 faults are connected from CAPTURE_CHANGE.
```

```
TEST> analyze_faults mux2/MUX2_UDP_1/A -stuck 1
-----
----
Fault analysis performed for mux2/MUX2_UDP_1/A stuck at 1 (input 0 of AND
```

```

gate 101).
Current fault classification = AN (atpg_untestable-not_det).
-----
---
Connection data: to=TLA
Fault site control to 0 was successful (data placed in parallel pattern
0).
Observe_pt=any test generation was unsuccessful due to atpg_untestable.
Observe_pt=101(AND) test generation was successful (data placed in
parallel pattern 1).
Observe_pt=102(OR) test generation was unsuccessful due to atpg_
untestable.
Warning: 1 patterns rejected due to 4 bus contentions (ID=57, pat1=0).
mux2 (34) is potential source of blockage for observe gate 102.
mux2 (94) is potential source of blockage for observe gate 102.

TEST> analyze_faults out1/EN -stuck 0
-----
---
Fault analysis performed for out1/EN stuck at 0 (input 0 of BUF gate 46).
Current fault classification = AP (atpg_untestable-pos_det).
-----
----
Connection data: to=REGPO,TLA,TS_ENABLE
Fault site control to 1 was successful (data placed in parallel pattern
0).
Observe_pt=any test generation was unsuccessful due to atpg_untestable.
Observe_pt=73(TSD) test generation was unsuccessful due to atpg_
untestable.
Warning: 1 patterns rejected due to 4 bus contentions (ID=57, pat1=0).

TEST-T> analyze_faults -class AN -source_points 5 0
Fault analysis summary: #analyzed=802, #unexplained=0.
420 faults are untestable due to constrain values.
382 faults are untestable due to constrain value blockage.
22 faults are connected to DSLAVE.
2 faults are connected to TIEX.
62 faults are connected to TLA.
28 faults are connected to CLOCK.
6 faults are connected to RESET.
34 faults are connected to NONSCAN_LOAD.
6 faults are connected from CLOCK.
22 faults are connected from TLA.
126 faults are connected from TIEX.

Blockage Points - Total 145:
-----

Blockage Value Source Points - Total 3:
-----
i_test_se 37.16%

```



```
Constraint Value Source Points - Total 27:
-----
test_model 17.08%
i_test_se 5.74%
```

See Also[report_faults](#)[set_atpg](#)[Fault Analysis](#)[Understanding the analyze_faults Report](#)

analyze_feedback_path

Overview

Use this command for determining whether a selected feedback network has a sensitizable feedback path.

Syntax

```
analyze_feedback_path
  loop_id
  [-verbose]
```

Arguments

loop_id

Specifies the [loop ID](#) number of the feedback path network to analyze. The loop ID is assigned during model building and can be displayed with the `report_feedback_paths` command.

-verbose

Displays verbose information.

Allowed Command Modes

Test

Description

Use this command for determining whether a selected feedback network has a sensitizable [feedback path](#). If a pattern can be created to satisfy a sensitization condition, you can display the simulation values for that pattern by setting pin data reporting to pattern 0.

Note: The feedback path analysis uses test generation, whose parameters are controlled using the `set_atpg` command. X1 analysis is affected by the `set_atpg -abort_limit` switch, since it uses ATPG to figure out if the feedback path is sensitizable. The default setting is 10 aborts before the analysis can complete.

Examples

```
TEST> report_feedback_path -all
id#  #gates  #sources  sensitization_status
---  -
0    2      1      pass
1    10     1      pass
2    10     1      pass
3    10     1      pass
4    10     1      pass
5    10     1      pass
6    5      1      pass
7    10     1      pass
8    2      1      fail
9  583     17     abort
```

```
TEST> analyze_feedback_path 6
Feedback path 6 passed sensitization check.
```

```
TEST> analyze_feedback_path 9
Feedback path 9 (408339,408768) aborted sensitization check.

TEST> report_feedback_path 8 -verbose
id# #gates #sources sensitization_status
---
 8   2     1         fail
    BUS core/u1/map/fifo/tsd (7019), cell=_BUS
    BUF core/gronk/tgh_2 (7020), cell=BKEEP
```

See Also

[report_feedback_paths](#)
[set_atpg](#)
[Loop ID](#)

analyze_nonscan_loading

Overview

This command analyzes the loading of nonscan state elements by simulating combinational random patterns and then reporting the number of nonscan cells that are loadable.

Syntax

```
analyze_nonscan_loading
  [-atleast {<percentage <0 | 1 | B>>}]
  [-atmost {<percentage <0 | 1 | B>>}]
  [-max number]
  [-patterns number]
```

Arguments

-atleast {<percentage <0 | 1 | B>>}

This option reports all nonscan cells that were loaded to either 0 or 1, or both 0 and 1 (by specifying the **B** argument) at least *percentage* times. The results are sorted by decreasing probability.

-atmost {<percentage <0 | 1 | B>>}

This option reports all nonscan cells that were loaded to either 0 or 1, or both 0 and 1 (by specifying the **B** argument) at most *percentage* times. The results are sorted by increasing probability.

-max *number*

This option limits the reporting to a specified *number* of cells.

-patterns *number*

Specifies the *number* of patterns to simulate. By default, 1024 random patterns are simulated.

Description

This command analyzes the loading of nonscan state elements by simulating combinational random patterns and then reporting the number of nonscan cells that are loadable.

By default, 1024 patterns are simulated and all nonscan cells are reported in a histogram of the percentage of patterns in which a non-X value was loaded.

The value of [set_simulation](#) -shift_cycles is taken into account during this analysis. To find the number of nonscan cells that are controllable by ATPG, specify 1. To find the number of nonscan cells that is simulated as loadable, use the number that is set during test generation.

Note that no instances are reported by default. To report instances, use either the **-atleast** or **-atmost** switches, while taking into account the minimum or maximum percentage of the time that it was loaded with the specified value (with **B** meaning either 0 or 1). To report the most possible instances, specify “-atleast {1 B}”. For example:

```
TEST-T> analyze_nonscan_loading -patterns 100 -atleast {10 0}
Begin nonscan loading analysis: #patterns=100, #nonscan_cells=7
Shift_pattern simulation setup completed: #shift_cycles=1, #shift_
```

```

patterns=3
Shift simulation completed: #shifts=1, #nonscancells_loaded=160
Shift simulation completed: #shifts=1, #nonscancells_loaded=160
Shift simulation completed: #shifts=1, #nonscancells_loaded=160
Shift simulation completed: #shifts=1, #nonscancells_loaded=20
Nonscan_loading analysis summary: #patterns=100, #nonscan_cells=7, CPU_
time=0.00 sec
-----
Range nonscan cell load count
-----
0          1
1-9        1
10-19      0
20-29      0
30-39      0
40-49      0
50-59      0
60-69      0
70-79      0
80-89      0
90-99      1
100        4
-----
-----
Nonscan_loading report: threshold=10, type=atleast, value=0, max=0
-----
-----
gate_id type count instance name
-----
91 DFF 93 n0and
95 DFF 48 l2andf
94 DFF 47 n0xor
93 DFF 30 n0aoi

```

In the previous example, only four instances are reported even though five instances were loaded with non-X values in more than 10% of the simulations. This indicates that the instances not listed failed to meet the `-atleast` condition that a 0 should be loaded in 10% of the simulations.

Note that when using the `analyze_nonscan_loading` command, you must set [set_simulation](#) **-shift_cycles** to a value greater than 0.

For complete information on using this command and related commands, see the topic "[Using Loadable Nonscan Cells in TestMAX ATPG](#)."

analyze_simulation_data

Overview

You use this command to compare the results from any two of three possible simulation sources, or to display the results from a single simulation source.

Syntax

```
analyze_simulation_data
  <file_name | -fast_sequential pattern | -full_sequential pat#1 pat#2>
  [-bidi_events]
  [-diag]
  [-max_fails_reported d]
  [-max_values_reported d]
  [-sensitive | -insensitive]
  [-top top_instance_name]
```

Arguments

file_name

Specifies the filename of the VCD input file that contains simulation to be displayed by itself or to be compared to a second form of simulation, either the `-fast_sequential` or `-full_sequential` data sources. The first 32 characters of this VCD simulation data is displayed without interpretation either on the schematic or in the output from the `report_primitive` command when `set_pindata` is set to `debug_sim_data`. A dash "-" is displayed for circuit locations for which there is no corresponding VCD data present in this file.

-fast_sequential pattern

Selects as one of the simulation sources the currently selected internal or external pattern buffer using the Fast-Sequential simulation algorithm. A single pattern must be specified. If the pattern is a Basic-Scan pattern the pattern is simulated with the Fast-Sequential simulator and the graphical display will show three character values corresponding to a preclock, active-clock, and postclock events. A minus character "-" is used as a delimiter between the values from different capture cycles and a pound sign '#' indicates a scan load.

-full_sequential pat#1 pat#2

Selects as one of the simulation sources the currently selected internal or external pattern buffer using the Full-Sequential simulation algorithm. A starting and stopping pattern range must be specified. In collecting the simulated values all patterns from the beginning of the pattern buffer to the selected ending pattern are simulated, but only the start to stop range is saved for comparison and display. When displaying the simulated pattern values, a minus character "-" is used as a delimiter between values from different patterns.

-bidi_events

When a VCD data source is specified, the default conversion of [simulation events](#) to internal events advances time only if primary inputs (PIs) change. The selection of `-bidi_events` causes a time advance and a new event to be stored if bidirectional ports change

(PIOs). By default, event changes that only involve bidirectional ports are merged until an event involving any PI occurs.

-diag

When two sources for simulation data compare are Fast-Sequential and Full-Sequential patterns *and* the same pattern is selected, then this option can also be specified to perform an automated diagnosis to search out the simulation differences. It will first compare the results of the simulation and report any POs or scan cells with pattern differences (an X for the Fast-Sequential simulation will not be considered a failure). If there is a failure, the first failure is automatically selected and displayed graphically. The failing node gate and the gates connected to it which have a simulation mismatch is displayed with the debug_sim_ data annotated on the pins. The cone of logic displayed is limited to a single input with a difference and is bounded by state element gates.

-max_fails_reported d

This specifies the maximum number of reported failures when the -diag option is selected. The default is 10.

-max_values_reported d

This specifies the maximum number of simulation values reported for each simulation source. The default is 32.

-sensitive | -insensitive

Specifies whether the VCD file is read in with or without case sensitivity for instance, net, and pin names present. The default is -sensitive.

-top top_instance_name

Specifies the [instance name](#) in the VCD data file corresponding to the current flattened design module in use by TestMAX ATPG. If a VCD source is selected without specifying how to recognize the instance name of the top module, then the first nested instance name is used (for instance, if \$scope TESTBENCH has nested \$scope DUT, the "DUT" is treated as the instance name corresponding to the currently selected top module in TestMAX ATPG memory).

Allowed Command Modes

Test

Description

You use this command to compare the results from any two of three possible simulation sources, or to display the results from a single simulation source. The three possible sources of simulation data are:

- A VCD (Value Change Dump) simulation file.
- Internal patterns using the Fast-Sequential simulator.
- Internal patterns using the Full-Sequential simulator.

The internal patterns refer to the existing selected pattern buffer, which can be either the internal pattern buffer (populated as by the run_atpg command), or the external pattern buffer (populated by the set_patterns external command).

After comparing the two simulation sources, you can view the results of this comparison graphically by selecting the `debug_sim_data` option of the `set_pindata` command and selecting the gates of interest for view in the graphical display.

An automated difference identification algorithm is possible using the `-diag` option if the two sources are the Fast-Sequential and Full-Sequential simulators and the pattern to compare is the same. This algorithm automatically identifies gates in the design that have simulation differences, and selects and displays the first such gate identified along with its connecting logic to state elements.

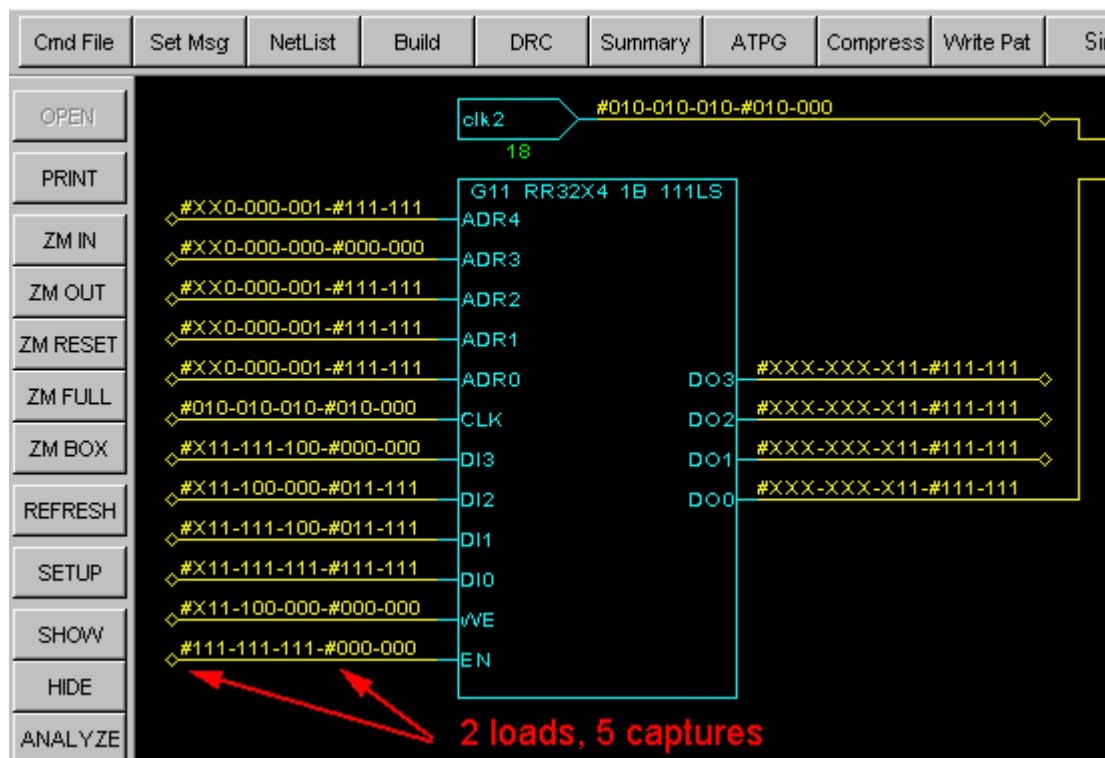
The data in the VCD file is presented without interpretation for a side-by-side comparison. Only the first 32 characters are shown. For best results, you should carefully collect the VCD that best matches the pattern of interest. This is best done by:

1. Using the parallel form of a Verilog testbench.
2. Collecting VCD data only from "capture_XXX" procedures.
3. Avoiding collection during scan chain load/unloads.
4. Avoiding collection during test_setup.

Not all VCD data will have a corresponding net or node location in the TestMAX ATPG-flattened design. TestMAX ATPG ignores VCD information for which no internal circuit location can be found. For internal circuit points that exist but are not present in the VCD data stream, a dash "-" is displayed.

Examples

```
TEST> analyze_simulation data -fast_sequential 10
TEST> set_pindata debug_sim_data
TEST> # show instances of interest in schematic
```



See Also

[run_simulation](#)

[set_patterns](#)

analyze_test_points

Overview

This command analyzes a design and prints a file containing a list of optimal observe and control test points.

Syntax

```
analyze_test_points
[-target <testability | pattern_reduction | fault_class>]
[-test_points_file file_name]
[-max_control_points number]
[-max_observe_points number]
[-num_observe_points_per_cell number]
[-class {list_of_fault_classes}]
[-exclude {list_of_instance_names}]
[-replace]
```

Arguments

-target <testability | pattern_reduction | fault_class>

Specifies the type of analysis to perform when determining the test points. You can use the following arguments:

- **testability**
 - Targets improved test coverage with control and observe points
 - Uses random patterns for iterative static analysis
 - Does not require prior ATPG
- **pattern_reduction**
 - Targets reduced pattern size with observe points
 - Uses [SCOAP](#) (Sandia Controllability and Observability Analysis Program) numbers for static analysis
 - Does not require prior ATPG
- **fault_class**
 - Targets improved test coverage with observe points for fault classes
 - Uses fault cone topology for dynamic analysis
 - Requires initial ATPG for analysis of fault cones

-test_points_file *file_name*

Specifies the name of the output file containing the list of test points.

-max_control_points *number*

Specifies the maximum number of control points selected in the design. By default, TestMAX ATPG selects a maximum of 1000 control points if this option is not specified. This option is supported only when the `-target testability` option is specified.

-max_observe_points *number*

Specifies the maximum number of observe points selected in the design. By default, TestMAX ATPG selects a maximum of 1000 observe points if this option is not specified.

-num_observe_points_per_cell *number*

Specifies the maximum number of observe points connected through XOR (observation) trees to one observe scan flip-flop or primary output. The default is eight observe points per observe flip-flop or primary output.

-class {*list_of_fault_classes*}

Specifies the class of faults targeted for analysis. This option is supported only when the `-target fault_class` option is specified.

You can specify this option multiple times to analyze multiple fault classes. For example, you can generate the test patterns, then execute the `analyze_test_points` command using with the `-class NO` and `-class AN` options. If you don't specify the `-class` option, all undetected faults are considered for test-point insertion.

In Tcl mode, you must use a list to specify a set of fault classes. For example:

```
-class {NO AN}
```

OR

```
-class [list NO AN]
```

The following list shows the fault classes and sub-classes. Note that fault sub-classes are highlighted in bold, and are specified in conjunction with the `-class` option. You cannot specify a major class with the `-class` option.

DT - Detected

DS - Detected by simulation

DI - Detected by implication

PT - Possibly detected

AP - ATPG untestable, possibly detected

NP - Not analyzed, possibly detected

UD - Undetectable

UU - Undetectable, unused

UO - Undetectable, unobservable

UT - Undetectable, tied

UB - Undetectable, blocked

UR - Undetectable, redundant

AU - ATPG untestable

AN - ATPG untestable, not-detected

ND - Not detected

NC - Not controlled

NO - Not observed

-exclude {*list_of_instance_names*}

Prevents TestMAX ATPG from creating test points for the specified instance or instances. This option is supported only when the `-target fault_class` option is specified. You can specify multiple `-exclude` options for multiple instances.

In Tcl mode, you must use a list to specify multiple instances. For example:

```
-exclude {instA instB}
```

OR

```
-exclude [list instA instB]
```

-replace

Replaces any existing output file previously specified by the `-test_points_file` option.

Allowed Command Modes

Test

Description

This command analyzes a design and prints a file containing a list of optimal observe and control test points. When you use the generated test points file with the [run_atpg](#) `-observe_file` command, TestMAX ATPG estimates the test coverage improvement.

For more information on using the `analyze_test_points` command, see [Improving Test Coverage With Test Points](#).

Examples

The following example uses a set of random patterns for iterative static analysis, and creates a file containing test points called `tp_file_a`:

```
analyze_test_points -target testability -test_points_file tp_file_a
```

A previous ATPG run is required only when you use the `-target fault_class` option with the `analyze_test_points` command.

analyze_violation

Overview

This command analyzes the cause of certain rule violations.

Syntax

```
analyze_violation  
  violation_id
```

Arguments

violation_id

Indicates the rule violation ID (such as "Z1-4") to analyze.

Allowed Command Modes

Test

Description

This command analyzes the cause of certain rule violations. The gates associated with the violation are displayed in the schematic viewer, and are annotated with data that was used to perform the rule checking. Most violations are associated with rules that are checked during the DRC process, which is performed by using the `run_drc` command.

Examples

```
TEST> analyze_violation z4-3  
Warning: Bus contention on my_asic/BIBUS/L34 (375) occurred at time 0 of  
test_setup procedure. (Z4-3)  
The pin data is now set to "test_setup".
```

See Also

[report_violations](#)
[run_drc](#)

analyze_wires

Overview

Use this command to analyze potential contention problems associated with WIRE primitives.

Syntax

```
analyze_wires  
  <id | -all> [-update]
```

Arguments

id

Specifies the [primitive ID](#) of the WIRE primitive on which analysis is to be performed. An error is reported if the ID specified is not a WIRE primitive.

-all

Selects all WIRE primitives for analysis. Summary information is displayed upon completion of the analysis.

-update

Selects updating of the wire contention status based on the results of the analysis. The default is not to update this status.

Allowed Command Modes

Test

Description

Use this command to analyze potential contention problems associated with WIRE primitives. If a pattern can be created to satisfy a contention condition, you can display the simulation values for that pattern by setting pindata reporting to pattern 0.

Note: The wire analysis is affected by settings of the `set_atpg` and `set_contention` commands.

Examples

```
TEST> analyze_wires -all  
Wire Contention results: #pass=32, #fail=0, #abort=0, CPU time=3.21  
  
TEST> analyze_wires 206 -update  
Wire 206 passed contention check.
```

See Also

[report_wires](#)
[set_atpg](#)
[set_contention](#)
[set_pindata](#)
[Primitive ID](#)

License Commands

This section describes the following commands:

- [get_licenses](#)
- [remove_licenses](#)
- [report_licenses](#)

get_licenses

Overview

Use this command to manually check out selected licenses.

Syntax

```
get_licenses {license}  
  [-quantity  
  [-wait time]
```

Arguments

**test-analysis | test-compression-atpg | test-faultsim | test-atpg-max |
test-diagnosis | test-fault-max | test-power**

Specifies the license that is to be checked out. You can request multiple licenses in one command.

-quantity

Checks out multiple licenses for a specified license name. This option is supported with the following licenses: test-compression-atpg, test-faultsim, test-atpg-max, test-diagnosis, test-fault-max, and test-physical. You can use the `-quantity` option of the `report_licenses` command to report how many copies of each license are currently checked out.

-wait time

This option attempts to get a license multiple times until *time* seconds have passed. The *time* argument is an integer in seconds. It does not have a default. If *time* < 60, ATPG will retry every second. If *time* > 60, ATPG retries every 60 seconds until the time left is less than 60 seconds. It then retries every second before it times out.

Description

Use this command to manually check out selected licenses. This is not normally required because licenses are automatically checked out as needed.

For more information, see "[Licensing](#)."

Example

In the following example, a standard ATPG license is checked out.

```
BUILD> get_licenses test-faultsim  
BUILD> report_licenses  
test-fault-max  
test-faultsim
```

In the next example, four licenses are checked out for the test-atpg-max license:

```
BUILD> get_licenses test-atpg-max -quantity 4  
BUILD> report_licenses -quantity  
Test-Compile  
Test-ATPG-Max (4)
```


See Also

[remove_licenses](#)

[report_licenses](#)

remove_licenses

Overview

Use this command to manually release and check in selected licenses.

Syntax

```
remove_licenses {license}
```

Arguments

test-analysis | test-compression-atpg | test-faultsim | test-atpg-max | test-diagnosis | test-fault-max | test-power

Specifies the license or licenses to release and to check in to the license server. Any number of licenses can be released using a single command.

Allowed Command Modes

All

Description

Use this command to manually release and check in selected licenses.

For more information, see "[Licensing](#)."

Examples

In this example, a standard ATPG license is checked in.

```
TEST> report_licenses
test-analysis
test-faultsim
TEST> remove_licenses test-faultsim
TEST> report_licenses
test-analysis
```

See Also

[get_licenses](#)

[report_licenses](#)

[set_faults](#)

report_licenses

Overview

Use this command to report all licenses currently checked out by TestMAX ATPG.

Syntax

```
report_licenses  
    [-quantity]
```

Arguments

-quantity

Reports how many copies of each license are currently checked out.

Allowed Command Modes

All

Description

Use this command to report all licenses currently checked out by TestMAX ATPG.

Examples

```
TEST-T> report_licenses  
Test-Analysis  
Test-Fault-Max  
  
TEST-T> report_licenses -quantity  
Test-Compile  
Test-ATPG-MAX (4)
```

See Also

[get_licenses](#)

[remove_licenses](#)

[Licensing](#)

Read Commands

This section describes the following commands:

- [read_cell_model](#)
- [read_drc](#)
- [read_faults](#)
- [read_image](#)
- [read_layout](#)
- [read_memory_file](#)
- [read_netlist](#)
- [read_nofaults](#)
- [read_sdc](#)
- [read_timing](#)

read_cell_model

Overview

This command loads cell test models (CTMs) for use in cell-aware diagnostics.

Syntax

```
read_cell_model  
  path/filename
```

Arguments

path

Specifies the path and names of the CTMs you want to load. Wildcards (*) are accepted with a file name extension to specify all files in a directory with a particular suffix.

Description

This command loads cell test models (CTMs) used for cell-aware diagnostics.

Examples

The following example loads all CTMs with a .ctm extension in the ctm_dir directory:

```
read_cell_model ctm_dir/*.ctm
```

See Also

[Understanding Cell Test Models](#)
[Cell-Aware Diagnosis](#)

read_drc

Overview

This command reads the specified STIL procedure file, but will not further process it through DRC.

Syntax

```
read_drc  
    filename  
    [-reorder_xtol_signals]
```

Arguments

filename

Specifies the STL procedure file to be read.

-reorder_xtol_signals

Reverses the order of signal usage for unload mode signals when writing X-tolerant mode data. You should use this option if the [run_drc](#) command uses a file written from the `read_drc` command, and generates R rule violations during DRC. Note that STIL procedure files generated from DFT Compiler might require this option if the `read_drc` command is specified before these files are used by the [run_drc](#) command.

Description

This command reads the specified STIL procedure file, but will not further process it through DRC. It must be specified immediately before the `update_wft`, [update_scale](#), and `update_clocks` commands. Any changes made with the update commands will only be saved, and usable, if the `write_drc_file` command is used immediately after these commands, and then the `run_drc` command is used with the newly generated file.

If you want to [modify the timing data in an existing STIL procedure file](#), a specific sequence of commands is required.

Note that the `read_drc` command uses limited design data to validate the contents of the STIL procedure file, and does not use the [run_drc](#) command for complete DRC checking. As a result, the contents of the STIL procedure file are not checked during this process. This means you should validate the contents of an STIL procedure file using the [run_drc](#) command before specifying the `read_drc` command.

In some cases, the STIL procedure file generated after specifying the [write_drc_file](#) command causes R rule violations or [M464](#) messages for compressed designs. But the original STIL procedure file used for the `read_drc` command does not generate R rule violations when used directly with the [run_drc](#) command. In this case, you can repeat the flow using the `-reorder_xtol_signals` option of the `read_drc` command to remove the R rule issues in the STL procedure file.

read_faults

Overview

This command provides a way for faults defined in a file to be added to or deleted from the current fault list.

Syntax

```
read_faults
  file_name [-retain_code | -force_retain_code | -maintain_detection] [-add
  | -delete]
```

Arguments

file_name

Specifies the file containing a fault list to read.

-retain_code

Causes the [fault codes](#) in the input file to be processed and potentially retained. Without this option, the fault location information is used, but the fault code is ignored.

For any NEW fault locations provided in the input file, if the fault code is DS, DI, UU, UT, UB, UR, AN, then the fault is added with the same fault code. For all other fault codes, an analysis is done to determine if the fault location should be classified as one of {UU, UT, UB, DI, AN}. If the fault location is found to be one of these fault classes, the new fault is added and the fault code is changed to the appropriate fault class.

For any fault locations provided in the input file that are already in the internal fault list, the fault code from the input file replaces the fault code in the internal fault list.

Note: If you wish to restore a previous fault list exactly use the `-force_retain_code` option.

-force_retain_code

Causes the [fault codes](#) in the input file to be read and retained as found in the input file.

-maintain_detection

While reading an external fault list, this option allows faults marked detected (with a DT, DS or DI fault class) to be added to the current TestMAX ATPG internal fault list without losing detection status that exists in the current TestMAX ATPG session. Faults added from this file will take on the status from the file, otherwise.

-add | -delete

Indicates whether to add or delete the faults in the file from the current fault list. The default is to add faults.

Allowed Command Modes

Test

Description

This command provides a way for faults defined in a file to be added to or deleted from the current fault list.

You can use this command to import fault lists from other tools. Bridging fault lists based on net names cannot be read by using the `read_faults` command; use the `add_faults` command to do this.

Examples

```
TEST> remove_faults -all
Warning: Internal pattern set is now deleted. (M133)
TEST> read_faults faults.all
88408 faults were read in and 88408 new faults were added to fault list.
TEST> report_faults -summary
Uncollapsed Fault Summary Report
-----
fault class          code #faults
-----
Detected             DT   11816
Possibly detected    PT     0
Undetectable         UD    840
ATPG untestable      AU   3312
Not detected         ND  72440
-----
total faults 88408
test coverage 13.49%
-----
```

```
TEST> remove_faults -all
TEST> read_faults faults.all -retain
88408 faults were read in and 88408 new faults were added to fault list.
TEST> report_faults -summary
Uncollapsed Fault Summary Report
-----
fault class          code #faults
-----
Detected             DT   83321
Possibly detected    PT    324
Undetectable         UD   1071
ATPG untestable      AU   3453
Not detected         ND    239
-----
total faults 88408
test coverage 95.59%
-----
```

```
TEST> remove_faults -all
TEST> add_faults -all
TEST> report_faults -summary
Uncollapsed Fault Summary Report
-----
fault class          code #faults
-----
Detected             DT   11816
Possibly detected    PT     0
Undetectable         UD    840
```



```

ATPG untestable      AU      3312
Not detected         ND      72440
-----
total faults 88408
test coverage 13.49%
-----
TEST> read_faults faults.drop -delete
  324 faults were read in and 324 faults were removed from fault list.
TEST> report_faults -summary
      Uncollapsed Fault Summary Report
-----
fault class          code #faults
-----
Detected             DT    11816
Possibly detected    PT         0
Undetectable         UD     840
ATPG untestable      AU     3312
Not detected         ND    72440
-----
total faults 88084
test coverage 13.54%
-----

```

See Also[Fault Interface File Format](#)[add_faults](#)[add_nofaults](#)[report_faults](#)[remove_faults](#)

read_image

Overview

Use this command to read and restore the TestMAX ATPG database created by the `write_image` command.

Syntax

```
read_image
  file_name
  [-password string]
```

Arguments

file_name

Specifies the pathname to the file containing the database.

-password string

Enables the reading of a secure image written by a `write_image -password` command. You must provide the password used during image creation; otherwise, an error results. You can use double quotation marks (" ") to enclose your password.

After a successful read, TestMAX ATPG operates in a secure state and allows only the specified allowed commands. The list of allowed commands is displayed after the image is read. Any commands not on the list are not executed and a warning message displays. Commands that write an image file are prohibited. Distributed ATPG cannot be run in a secure state.

Schematic viewing is permitted if the `-schematic_view` option of the `write_image` command is used when the secure image is created. The default is to read unprotected image files.

Allowed Command Modes

Build

Description

Use this command to read and restore the TestMAX ATPG database created by the `write_image` command. The `read_image` command deletes all existing design images or patterns. TestMAX ATPG transitions to the TEST mode if you read the image saved after running DRC. TestMAX ATPG will transition to the DRC mode if the image was saved after Build and before DRC.

If the `-physical` option of the [write_image](#) command is specified, TestMAX ATPG prints a message that identifies the number of fanout nets that are stored in the image. In the following example, a total of 916 nets are stored; this includes 491 single-fanout nets, 141 two-fanout nets, and 284 greater-than-two-fanout nets:

```
End reading subnets: 916 (491/141/284) subnets were read from the image.
```

Examples

```
BUILD> read_image i044_image.dat
Warning: All netlist and library module data are now deleted. (M41)
End read image: #bytes=1830406, CPU time=5.00 sec.
```

```
TEST> read_image image_enc.gz -password secret
Warning: All netlist and library module data are now deleted. (M41)
End read image: #bytes=11511, CPU time=0.02 sec.
Secure image read with schematic viewing enabled and allowing the \
following commands:
exit
help
run_diagnosis
set_patterns
run_simulation
```

See Also[Binary Image Files](#)[Creating and Reading Image Files](#)[Creating a Non-Secure Image File](#)[Creating a Secure Image File](#)[write_image](#)

read_layout

Overview

This command reads net topology and net pair data used for diagnostics.

Syntax

```
read_layout
  [-add]
  [-max_mismatch_report d]
  [-output file_name]
  [ bridging_pair_file -all_pairs | -likely_pairs ]
  [ subnet_file <-subnets> [-fault_list] ]
```

Arguments

-add

Adds the subnet or bridging net pairs information contained in the specified subnet file or bridging pair file to the existing data. If this option is not specified, all existing information is deleted and replaced with the data contained in the subnet file or bridging pair file. This option enables TestMAX ATPG to read multiple subnet or bridging net pairs files without removing the existing data.

-max_mismatch_report d

Controls the number of warning and error messages reported when reading layout information containing subnet definitions or likely net pairs.

By default, after a subnet file is read in, a summary message displays the total number of subnet definitions that have not been read, if any. To print more debug messages, you should use this option with $d > 0$. When bridging net pairs are read in and incorrect data is present in the input files, a maximum ten mismatches are printed, by default.

-output file_name

Writes out the subnet topology file for debugging purposes if the `read_layout` command errors out. If this option is not specified, the temporary subnet topology file is written out to the following location: `/tmp/___subnets.<top_module>.<process_id>`.

-subnets

Specifies the type of the subnet file that is read. This option is required when the specified subnet file is a net structure ASCII or binary file or subnet fault definition file.

When subnet file is a net structure file, at the completion of the read, the following status line is printed:

```
Read_subnets completed: #nets(total/added/overwritten)=3299/222/12
```

This message indicates the total number of net structures in memory, the number of net structures that have been added to the existing structure already in memory, and, finally, the number of net structures that were overwritten.

-fault_list

This option indicates that the specified subnet file to be read, *subnet_file*, is defining the fault to be ranked. At the end of the fault list reading, the following line is printed:

```
Read_subnet_faults completed: #faults(added/ignored)=N/K
```

Note the following:

The "added" reference corresponds to integer N. It indicates the number of subnet faults added for the ranking run.

The "ignored" reference corresponds to integer K. It indicates the number of subnet faults ignored for the ranking run. For example, say the net structure file is read; and, for a particular net, the number of subnet defined is 10, but only 3 subnet faults are added. In this case, the number of ignored subnet faults indicated is 7.

subnet_file

Specifies the name of either the net structure ASCII or binary file or subnet fault definition file. For details on how to produce the binary subnet file, see the description of the -file option of the [report_layout](#) command.

bridging_pair_file

This file contains specifications of the bridging net pairs in ASCII format. This file can be an unmodified Star-RCXT coupling capacitance report, or it can contain one node pair per line. The file format is described in detail in the "Bridging Fault ATPG" section of the *TestMAX ATPG User Guide* and the "[Node File Format for Bridging Faults](#)" topic in *TestMAX ATPG Online Help*.

The pairs that are read in from the file are used during diagnosis and can be reported using the [report_layout](#) command. If the net pairs are read in and an image is written, then the data is stored by default. See the [write_image](#) command for information on how not to store the net pair data.

A message is printed after the net pairs are read in. For example:

```
read_net_pairs completed: #net_pairs(total/added/incorrect)=997/997/0
```

This message indicates the total of net pairs that are successfully read (997, in the example). It also indicates the number of net pairs that were added since the last read (997), and the number of incorrect net pairs that were printed (0).

If an incorrect net pair is found, a warning message that points to the faulty line is printed. For example:

```
Warning: Line 1003, cannot find net driver for "u_top_2/n12103".
```

By default, a maximum of 10 warnings are printed. To change the default, use the `-max_mismatch_report d` option. When reading in a bridging net pairs file, other options that are dedicated for use with Milkyway libraries (that is, `-version`, `-cell_name`, and so forth.) cannot be used.

-all_pairs | -likely_pairs

These options define the type of bridging net pairs that are read in. The `-all_pairs` option indicates that only those bridging pairs listed in the bridging pair file are to be considered, and no other possible bridges are to be analyzed. The `-likely_pairs` option indicates that the bridging file contains the most likely bridging pairs, but allows other possible bridges to be analyzed. In this case, all possible bridges are considered for a fast analysis, but only the likely pairs are considered for the detailed analysis.

When adding new bridging pairs using the `-add` option, the same switch (`-likely_pairs` or `-all_pairs`) must be specified as when the existing set of bridging pairs was read. Otherwise, an [M636](#) error message is printed. Either the `-all_pairs` or `-likely_pairs` option must be specified when the *bridging_pair_file* specification is used. There is no default.

Allowed Command Modes

DRC and TEST

Description

This command reads net topology and net pair data used for diagnostics. The net topology information is stored in an ASCII file. For more information, see "[Subnets Format For Diagnosis](#)."

Examples

The following example shows the TestMAX ATPG output after reading a subnet file:

```
TEST-T> read_layout sbnt.file -subnet
      Read_subnets completed: #nets(total/added/overwritten)=280/280/0, CPU_
time=0.01
```

See Also

[report_layout](#)

[run_diagnosis](#)

[Subnets Formats for Diagnosis](#)

read_memory_files

Overview

Use this command to load a RAM or ROM instance with a memory image.

Syntax

```
read_memory_file
  <id | instance_name> file_name
  [-binary | -hex] [-range first_address last_address]
```

Arguments

id | instance_name

Specifies the RAM or ROM memory gate for which the memory image file is to be read. It is specified by either its [primitive ID](#) or its [instance pathname](#).

file_name

Indicates the pathname to a file containing the memory contents that are to be read. The memory contents must be in Verilog format.

-binary | -hex

Indicates the format of the memory image file. The default is `-hex`.

-range first_address last_address

Indicates the range for the memory words to be loaded. Specify the first and last address in decimal. The default is to load the entire memory.

Allowed Command Modes

DRC

Description

Use this command to load a RAM or ROM instance with a memory image. It can be used to alter the contents or to change the default contents that might have been specified in the netlist model for the RAM or ROM.

CAUTION: If the module defining the RAM or ROM specifies a memory image file, this file is initially loaded as specified by the Verilog specification. This convention pays attention to the order of the array index definition of the memory register. If the memory register is defined as "mymem [0:15]", the first data entry in the file is loaded into address 0. If the memory register is defined as "mymem[15:0]", the first data entry is loaded into address 15. By default, the ATPG tool always loads the first entry in the file into address 0, the second into address 1, and so on, proceeding upward through addresses.

Examples

```
DRC> read_memory i007/u1/mem/rom1/rom_core i007.d3 -hex
DRC> report_memory -all
type      ID      instance_path      memory_file
-----
ROM       5425    i007/u1/mem/rom1/rom_core    i007.d3
```

See Also

[report_memory](#)

[Example Memory Init File](#)

read_netlist

Overview

Use this command to read netlists.

Syntax

```
read_netlist
  file_name
  [-format <edif | verilog | vhdl>]
  [-library]
  [-define variable_name]
  [-master_modules]
  [-sensitive | -insensitive]
  [-delete]
  [-noabort]
  [-verbose]
```

Arguments

file_name

Indicates the path name to a file containing the netlist to read. The standard wildcard usage for filenames is supported, which results in reading all netlists that match the selected *file_name*.

-format <edif | verilog | vhdl>

Specifies the format of the netlist to read. If you don't specify this, the type of netlist is automatically determined. When reading EDIF-based designs, it is strongly suggested that you read in the design first and the libraries last. When mixing VHDL designs with Verilog libraries, you should read the libraries first and the design modules second.

-sensitive | -insensitive

Specifies whether the netlist is read in with or without case sensitivity. The default depends on the netlist format: case-sensitive for Verilog netlists and case-insensitive for EDIF and VHDL netlists.

-library

Specifies that all modules defined within the file name specified should be treated as if they were library modules. This is equivalent to surrounding each Verilog module with ``celldefine` and ``endcelldefine` directives. Use of this option results in a module definition that has fault sites only on the pin boundaries. Use of this option is generally not needed and can negatively affect the ability to create STIL or WGL patterns (see DRC rule [V15](#)). Use only if necessary.

-master_modules

Specifies that all modules defined by the file being read should be marked as Master Modules. Normally, when reading more than one module with the same name, the last module encountered determines the final module definition (or first module encountered if set -redefined_module first has been used). When a module is read with the `-master_`

`modules` option, it is marked as a Master Module and is not replaced if other modules with the same name are encountered. However, it can be replaced if a module with the same name is read and the `-master_modules` option is again used. This mechanism can be useful for reading specific modules that are intended as module replacements, independent of the order in which standard modules are read.

-define *variable_name*

Defines any variable, function, or expression while reading a netlist. This option is equivalent to the 'define Verilog statement. You can enter multiple names as a Tcl list, as shown in the following example:

```
read_netlist define.v -define {test1 test2}
```

-delete

Specifies that all previously read netlists currently in memory should be deleted. If a file name is also specified, then memory is cleared before reading that file.

-noabort

Specifies that reading multiple netlists will not terminate if a netlist receives an error. This allows reading all netlists with wildcard (*) filenames. TestMAX ATPG ignores this option and issues an M3 warning message if you use this option to read a single netlist.

-verbose

Specifies that individual netlist information is displayed when wildcard filenames are used. The default is to only give a single summary of all the netlists that were read.

Allowed Command Modes

Build

Description

Use this command to read netlists. TestMAX ATPG can automatically identify and accept any combination of netlists in Verilog, EDIF, and VHDL formats; either ASCII or compressed (GZIP or binary). By using filename wildcards, a single command can be used to read in multiple netlists.

You can use the `set_netlist` command to specify whether to replace duplicate modules with the last one read. This allows rereading a file after editing one module or reading multiple files where there might be some duplication of module definitions.

For more information on using the `read_netlist` command, see "[Reading a Netlist.](#)"

Examples

```
BUILD> read_netlist brand_x.lib
Begin reading netlist ( brand_x.lib )...
Warning: Rule N2 (unsupported entry) failed 472 times.
End parsing Verilog file brand_x.lib with 0 errors;
End reading netlist: #modules=517, top=ADD4, #lines=78240, CPU_time=3.22
sec
BUILD> report_modules -summary
Modules: #UNKNOWN_FORMAT=19 #STRUCT_VERILOG=474 (#unsupported_beh=18) #BEH_
VERILOG=24 (#unsupported_beh=24)

BUILD> read_netlist /net/vendor_a/tech/*.v -delete -noabort
Warning: All netlist and library module data are now deleted. (M502)
```

```
Begin reading netlists ( /net/vendor_a/tech/*.v )...
Warning: Rule N2 (unsupported entry) failed 4 times.
Warning: Rule N12 (invalid UDP entry) failed 8 times.
Warning: Rule N15 (incomplete UDP) failed 364 times.
Warning: Rule N16 (overspecified UDP) failed 17 times.
Warning: Rule N20 (underspecified UDP) failed 22 times.
Warning: Rule N21 (unsupported UDP entry) failed 9 times.
Warning: Rule N23 (inconsistent UDP entry) failed 9 times.
End reading netlists: #files=3584, #modules=2383, top=DPLL1C,
#lines=442821, CPU_time=72.11 sec
```

See Also[set_netlist](#)[set_rules](#)[write_netlist](#)

read_nofaults

Overview

Use this command to add the nofault attribute for faults defined in an external file.

Syntax

```
read_nofaults  
  file_name
```

Arguments

file_name

Specifies the pathname to the file containing the list of faults whose nofault attribute is to be set on.

Allowed Command Modes

DRC Test

Description

Use this command to add the nofault attribute for faults defined in an external file. The input file is in standard fault list format. The fault locations equivalent to the nofaults specified by the file can also be nofaulted using the `add_equivalent_nofaults` command.

Examples

```
DRC> read_nofaults nofaults.ub  
4001 faults were added to nofault list.
```

See Also

[Fault Interface File Format](#)
[add_equivalent_nofaults](#)
[add_nofaults](#)
[report_nofaults](#)

read_sdc

Overview

This command reads in setup timing exceptions directly from an SDC (Synopsys Design Constraints) file.

Syntax

```
read_sdc file_name
  [-echo]
  [-syntax_only]
  [-version sdc_version]
```

Argument

file_name

Specifies the name of the SDC file to be read.

-echo

Echoes all commands.

-syntax_only

Causes the `read_sdc` command to perform only syntax/semantic checks.

-version sdc_version

Specifies the SDC version — default is the latest; values: 1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, latest.

Description

This command reads in setup timing exceptions directly from an SDC (Synopsys Design Constraints) file. You must be in DRC mode to use this command (after you successfully run the `run_build` command, but before running `run_drc`).

Note that module pins are accepted as -through points for false and multicycle paths, but flattening during the build process will remove all references to them so they cannot be found. A workaround is to use `set_build -portfault_box` command for the module(s) whose pins are used as through points.

The `read_sdc` command is valid only Tcl mode. For more information on specifying timing exceptions, see the "Specifying Setup Timing Exceptions From an SDC File" section in the *TestMAX ATPG User Guide*.

For more information on the SDC file, see [How TestMAX ATPG Interprets SDC File Commands](#).

See Also

[set_sdc](#)

[report_sdc](#)

read_timing

Overview

This command causes TestMAX ATPG to read a file of data that is interpreted as the minimum slacks (timing margins) for each faultable pin in the design.

Syntax

```
read_timing
    file_name
    [-delete]
    [-verbose]
```

Arguments

file_name

Specifies the input file containing the pin-slack data that is read.

-delete

Causes any slack data currently in the data structure to be deleted before reading in the slack data from the specified file and storing it in the data structure. Without this option, the slack data in the file is added to the slack data already stored in the data structure.

Note that any existing data is overwritten if the input file contains slack data for the same pin.

-verbose

The option enables additional messages on missing slack data during when running the read timing command. By default, only the first 50 missing slack messages are printed.

This option enables the printing of all missing slack messages.

Description

This command causes TestMAX ATPG to read a file of data that is interpreted as the minimum slacks (timing margins) for each faultable pin in the design. This minimum slack information can be printed from PrimeTime using the PrimeTime command `report_global_slack`.

The format for the input file is a single line for each faultable pin. The line contains three fields: the pinpathname for the pin, the minimum slack for a rising transition through that pin, and the minimum slack for a falling transition through the pin.

See Also

```
report_faults
set_pindata
set_delay
```

Remove Commands

This section describes the following commands:

- [remove_atpg_constraints](#)
- [remove_net_connections](#)
- [remove_atpg_primitives](#)
- [remove_nofaults](#)
- [remove_capture_masks](#)
- [remove_physical](#)
- [remove_cell_constraints](#)
- [remove_pi_constraints](#)
- [remove_clocks](#)
- [remove_pi_equivalences](#)
- [remove_compressors](#)
- [remove_po_masks](#)
- [remove_delay_paths](#)
- [remove_scan_chains](#)
- [remove_display_gates](#)
- [remove_scan_enables](#)
- [remove_display_proc](#)
- [remove_sdc](#)
- [remove_distributed_processors](#)
- [remove_slow_bidis](#)
- [remove_faults](#)
- [remove_slow_cells](#)
- [remove_lbist_seeds](#)
- [remove_waveform_signals](#)
- [remove_licenses](#)

remove_atpg_constraints

Overview

Use this command to remove ATPG constraints previously added with the `add_atpg_constraints` command.

Syntax

```
remove_atpg_constraints <atpg_constraint_name | -all>
[-no_detection_credit]
```

Arguments

atpg_constraint_name

Specifies the name of the ATPG constraint to remove. This is the symbolic name specified in a prior `add_atpg_constraints` command.

-all

Specifies that all ATPG constraints are removed.

-no_detection_credit

Removes all current no detection credit cells specified by the `-no_detection_credit` option of the [add_atpg_constraints](#) command. This option changes the fault status of the current AU faults to NC and can be used in the TEST and DRC modes.

For a complete description of using this option, see [Defining, Reporting, and Removing No Detection Credit Cells](#).

Allowed Command Modes

DRC, TEST

Description

Use this command to remove ATPG constraints previously added with the `add_atpg_constraints` command.

Examples

```
DRC> report_atpg constraints -all
name                val DRC site
-----
my_constraint1       1   no   my_atpg_primitive1 (2217)
my_constraint2       0   no   BLASTER/alu/cpu/arith/U182 (1911)
my_constraint3       1   no   BLASTER/core/arc/pie/U44 (2683)
DRC> remove_atpg_constraints -all
  3 constraints have been removed from the ATPG constraints list.
DRC> report_atpg_constraints -all
Warning: Requested report contained no entries. (M13)
```


See Also

[report_atpg_constraints](#)
[add_atpg_constraints](#)

remove_atpg_primitives

Overview

Use this command to delete ATPG primitives previously added with the `add_atpg_primitives` command..

Syntax

```
remove_atpg_primitives
  <atpg_primitive_name | atpg_primitive_id | -all>
```

Arguments

atpg_primitive_name

Specifies the name of the [ATPG primitive](#) to remove.

atpg_primitive_id

Specifies the [ATPG primitive ID](#) number of the ATPG primitive to remove.

-all

Specifies that all ATPG primitives is removed.

Allowed Command Modes

DRC Test

Description

Use this command to delete ATPG primitives previously added with the `add_atpg_primitives` command. An ATPG primitive cannot be removed if it has a current ATPG constraint on it.

Examples

```
DRC> report_atpg_primitives -all -noverbose
name          id#      type  inputs
-----
MY_ATPG_GATE_1 101071  AND   47566 250

DRC> remove_atpg_primitives -all
1 primitives have been removed from the ATPG primitive list.
DRC> report_atpg_primitives
ATPG primitive summary: none
```

See Also

[report_atpg_primitives](#)
[report_atpg_constraints](#)
[add_atpg_primitives](#)
[atpg_primitive_id](#)
[atpg_primitive_name](#)

remove_capture_masks

Overview

Use this command to delete previously defined nonscan cell capture masks.

Syntax

```
remove_capture_masks
  <instance_path | gate_id | -all>...
```

Arguments

instance_path | gate_id

Specifies the [instance path name](#) or the [gate ID](#) of the nonscan sequential cell for which the capture mask should be removed. This gate should have been previously masked with the `add_capture_masks` command or an error is reported.

-all

Specifies the removal of all defined capture masks.

Allowed Command Modes

DRC

Description

Use this command to delete previously defined nonscan cell capture masks. If you add masks using the `-scan` option of the `add_capture_masks` command, the only way to remove these masks before the `run_drc` command is to use the `-all` option of the `remove_capture_masks` command.

Examples

```
DRC> report_capture_masks
gate_id type instance_name
-----
19252 DFF MAIN/U87/DATA_OUT
19286 DFF MAIN/U111/SHADOW
19287 DFF MAIN/U112/SHADOW
19288 DFF MAIN/U113/SHADOW

DRC> remove_capture_masks MAIN/U112/SHADOW
1 capture masks were removed from capture mask list.

DRC> report_capture_masks
gate_id type instance_name
-----
19252 DFF MAIN/U87/DATA_OUT
19286 DFF MAIN/U111/SHADOW
19288 DFF MAIN/U113/SHADOW
```

See Also

[add_capture_masks](#)

[report_capture_masks](#)

remove_cell_constraints

Overview

Use this command to delete cell constraints that were added with the `add_cell_constraints` command.

Syntax

```
remove_cell_constraints
  <<chain_name <-position cell_position | instance_name | -all>>
```

Arguments

chain_name -position cell_position

Specifies the chain name and position in the scan chain of a scan cell whose cell constraint is to be removed. The position number 0 indicates scan cell closest to chain output.

instance_name

Specifies the [instance name](#) of a scan cell whose cell constraint is to be removed.

-all

Specifies that all cell constraints are to be removed.

Allowed Command Modes

DRC

Description

Use this command to delete cell constraints that were added with the `add_cell_constraints` command.

Examples

```
DRC> report_cell_constraints
type chain   pos.   site name
-----
  1   ...     ...   MAIN/CPU/TP/FI/OFIFO/[3]
OX   ...     ...   MAIN/PER/PRT_1/PORTIN_1/[3]
XX  c34      6     ...
DRC> remove_cell_constraints -all
DRC> report_cell_constraints
Warning: Requested report contained no entries. (M13)
```

See Also

[report_cell_constraints](#)

[add_cell_constraints](#)

[Instance Name](#)

[Pin Pathname](#)

remove_clocks

Overview

Use this command to delete clocks previously added with the `add_clocks` command.

Syntax

```
remove_clocks
  <{port_name} | -all>
```

Arguments

port_name...

Specifies the [port name](#) to be removed from the defined clock list.

-all

Specifies that all defined [clocks](#) are to be removed.

Allowed Command Modes

DRC

Description

Use this command to delete clocks previously added with the `add_clocks` command or implicitly defined in the DRC procedure file from a previous `run_drc` command. Note that clocks is defined again if they exist in the DRC procedure file and another `run_drc` command is issued. To completely remove clocks defined in the DRC procedure file, you must edit the file.

Examples

```
DRC> report_clocks
clock_name      off usage
-----
/INC 0
/CLK 0
/RSTB 1
DRC> remove_clocks {CLK RSTB}
DRC> report_clocks
clock_name      off usage
-----
/INC 0
```

```
DRC> report_clocks
clock_name      off usage
-----
/CLK            0    master shift
/NRESET         1    PO master set reset
DRC> remove_clocks CLK
DRC> report_clocks
clock_name      off usage
-----
```

```
/NRESET 1 PO master set reset  
DRC> remove_clocks -all  
DRC> report_clocks  
Warning: Requested report contained no entries. (M13)
```

See Also[report_clocks](#)[add_clocks](#)

remove_compressors

Overview

This command removes compressors from the defined compressors list.

Syntax

```
remove_compressors  
  <compressor_name | -load | -unload | -all>
```

Arguments

compressor_name | **-load** | **-unload** | **-all**

You can specify to remove a particular compressor name from the defined compressors list, or use one of the following: **-load** removes all load compressors; **-unload** removes all unload compressors; **-all** removes all compressors.

Description

This command removes compressors from the defined compressors list. Note that this command is valid in DRC mode only.

See Also

[report_compressors](#)

remove_delay_paths

Overview

This command removes delay path definitions from TestMAX ATPG memory, thus excluding them from consideration from being used for fault seeding and ATPG.

Syntax

```
remove_delay_paths  
  <-all | path_name | -min_cycle d | -min_slack d  
  | -max_cycle d | -max_slack d>
```

Arguments

path_name

Remove a specific path from the [delay_path](#) set currently maintained by TestMAX ATPG.

-max_cycle d

Removes paths with cycle time more than d.

-max_slack d

Removes paths with slack more than d.

-min_cycle d

Removes paths with cycle time less than d.

-min_slack d

Removes paths with slack less than d.

-all

Removes all paths from memory.

Allowed Command Modes

Test

Description

This command removes delay path definitions from TestMAX ATPG memory, thus excluding them from consideration from being used for fault seeding and ATPG.

Examples

```
TEST> remove_delay_paths -min_slack 0.3  
58 delay paths were removed due to failure to satisfy minimum slack.
```

See Also

[add_delay_paths](#)

[report_delay_paths](#)

remove_display_gates

Overview

Use this command to selectively remove gates displayed in the GSV.

Syntax

```
remove_display_gates  
  <pin_pathname | gate_id | -all>
```

Arguments

pin_pathname | *gate_id* | -all

Removes gates associated with the [pin_pathname](#), the [gate ID](#), or all gates from the GSV.

Allowed Command Modes

DRC, Test

Description

Use this command to selectively remove gates displayed in the GSV.

Examples

```
TEST> remove_display_gates mux3/MUX2_UDP_1/Q
```

See Also

add_display_gates
report_display_gates

remove_distributed_processors

Overview

Use this command to removes machines from the list of distributed processors.

Syntax

```
remove_distributed_processors  
  {name_list} [-all | -grd | -lsf]
```

Arguments

name_list

Specifies the slave machine names being removed from the distributed processor list. These names must match the names specified in an `add_distributed_processors` command.

-all

Specifies to remove all slave machines from the distributed processor list.

-lsf

Specifies to remove all slave processors in the load sharing facility (LSF) queue from the distributed processor list.

-grd

Specifies to remove all slave processors in the load sharing facility (LSF) queue from the distributed processor list.

Allowed Command Modes

Test

Description

Use this command to removes machines from the list of distributed processors.

Examples

```
TEST> remove_distributed_processors {himalaya himalaya himalaya}
```

See Also

[add_distributed_processors](#)
[report_distributed_processors](#)
[set_distributed](#)

Table Column Outside Table:

remove_faults

Overview

This command removes faults from the current fault list.

Syntax

```
remove_faults
  [<pin_pathname | instance_name | -module module_name
  | <[-bridge_location { bridge_location1 bridge_location2}]
  | -class fault_class | -all | -clocks | -scan_enable | -retain_sample d |
  -stuck <0 | 1 | 01> | -slow <r | f | rf>]
  [-launch launch_clock]
  [-capture capture_clock]
  [-combinational_feedback]
  [-exclusive]
  [-shared]
  [-shared_launch]
  [-shared_capture]
  [-inter_clock_domain]
  [-intra_clock_domain]
  [-aggressor_node <first | second | both>]
  [-bridge <0|1|01>]
  [-dynamic_bridge <r|f|rf>]
  [-dominant_node <first | second | both>]
  [-non_strength_sensitive]
```

Arguments

Note: When the bridging fault model is selected, TestMAX ATPG supports only the following options of the `remove_faults` command:

```
<[bridge_location_location { bridge_location1 bridge_location2}] | -class
<fault_class> | -all | -retain_sample d> [-bridge <0|1|01>] [-aggressor_node
<first | second | both>] [-non_strength_sensitive]
```

pin_pathname

Specifies removal of all faults in the current fault list associated with the specified [pin pathname](#).

instance_name

Specifies removal of all faults in the current fault list that are associated with the specified [instance name](#).

-module name

Remove faults from all instances of the module, where 'name' could be a fixed string or a [limited regular expression](#).

-bridge_location { bridge_location1 bridge_location2 }

Specifies the nodes of the bridging or dynamic bridging fault being removed. Both nodes must be specified, and if not specified, then either the `-all` or `-retain_sample` options

must be used. Either node can be selected to be an aggressor node, victim node, both using the `-aggressor` option for the bridging fault model and the `-dominant` option for the dynamic bridging fault model.

`-class fault_class`

Specifies the fault class that TestMAX ATPG should delete. TestMAX ATPG supports multiple uses of the `-class` option.

`-combinational_feedback`

Removes the bridging faults if the aggressor and victim form a combinational feedback loop.

`-all`

Specifies that all faults are to be removed. If there are any patterns in the internal pattern set, they are deleted if the fault list is removed.

`-clocks`

Removes clock faults from the fault list.

`-scan_enable`

Removes scan enable faults from the current fault list.

`-retain_sample d`

Specifies that a portion of all faults should be removed and that the indicated percentage of faults should be retained. This can be used to create a sampled subset of the original fault list. The sample selection is performed by giving each fault the selected probability of being retained. The final result cannot be an exact percentage of the original fault list due to random effects.

`-stuck <0 | 1 | 01>`

Removes faults with the specified stuck value. The default is stuck 01.

`-slow <r|f|rf>`

Removes faults with the specified slow value.

`-launch launch_clock`

Specifies the launch clock of the faults to be removed from the fault list. You can use this switch independently, or in conjunction with the `-capture` switch. The `-launch` option removes all faults that are launched exclusively by the specified clock. You can specify the PI, NO_CLOCK, or MULTIPLE keywords (in all upper or all lowercase) with this option to remove faults that are launched by the PI/PIO, no clock domains, or multiple clock domains, respectively. For a complete description of how TestMAX ATPG distinguishes faults captured by a clock and launched by a clock, see [Using Clock Domain-Based Faults](#).

`-capture capture_clock`

Specifies the capture clock of the faults to be removed from the fault list. You can use this switch independently, or in conjunction with the `-launch` switch.

The `-capture` switch removes all faults to the fault list that are captured exclusively by the specified clock. This switch also accepts the keywords PO, NO_CLOCK, or MULTIPLE, in either all upper or all lowercase, to remove faults that are observed by PO/PIO, no clock domains, or multiple clock domains, respectively. For a complete description of how

TestMAX ATPG distinguishes faults captured by a clock and launched by a clock, see [Using Clock Domain-Based Faults](#).

-exclusive

Specifies that only the faults that are launched and captured exclusively (using a single launch and a single capture) are to be removed from the fault list. Faults exclusively launched by PI or observed by PO are also removed. For further details, see [Using Clock Domain-Based Faults](#).

-shared

Specifies that only the faults that are launched or captured by multiple clocks are removed from the fault list. This excludes all PI and PO faults. For further details, see [Using Clock Domain-Based Faults](#).

-shared_launch

Specifies that faults launched by the specified clock and other clocks are removed from the fault list. An error is reported if you specify this switch without also using the `-launch` option. This excludes all PI and PO faults. For further details, see [Using Clock Domain-Based Faults](#).

-shared_capture

Specifies that faults captured by the specified clock and other clocks are removed from the faults list. An error is reported if you use this switch without also using the `-capture` option. For further details, see [Using Clock Domain-Based Faults](#).

-inter_clock_domain

Removes only exclusive faults from the fault list that are launched and captured by different clock domains. For further details, see [Using Clock Domain-Based Faults](#).

-intra_clock_domain

Removes only exclusive faults from the fault list that are launched and captured by the same clock domains. For further details, see [Using Clock Domain-Based Faults](#).

-aggressor_node <first | second | both>

Specifies which node in a node pair is used as the aggressor node of a bridging fault: `first`, `second`, or `both`. The default is `both`. To see how combinations of the `-bridge` and `-aggressor_node` option settings interact, see the `-aggressor` option of the `add_faults` command.

Using the default settings, there are four faults removed for each node pair defined if the faults were included in the list. A message is given at the end of the command indicating how many faults were removed from the list.

-bridge <0|1|01>

Specifies the type of bridging fault being removed. The type can be either:

- 0 - ba0 (bridging fault at 0)
- 1 - ba1 (bridging fault at 1)
- 01 - both (bridging fault at 0 and bridging fault at 1)

The default is 01 (both).

-dynamic_bridge <r|f|rf>

Specifies the type of dynamic bridging fault being reported. The type can be any of the following:

- `r` - bsr (bridge slow to rise)
 - `f` - bsf (bridge slow to fall)
 - `rf` -both (bridge slow to rise and bridge slow to fall)
- The default is `rf` (both).

-dominant_node <first | second | both>

Specifies which node in a node pair is used as the dominant node of a dynamic bridging fault: `first`, `second`, or `both`. The default is `both`. Using `first` or `second` will limit the set of faults reported and will only report the faults with the dominant in the position specified. To see how combinations of the `-dynamic_bridge` and `-dominant_node` option settings interact, see the `-dominant_node` option of the [add_faults](#) command. Using default settings, four faults are removed for each node pair defined if the faults were included in the list.

-non_strength_sensitive

Removes faults where neither node is driven by a gate with a dominant value.

Allowed Command Modes

Test

Description

This command removes faults from the current fault list. If there are any patterns in the current internal pattern set, you cannot remove faults from the fault list unless you select the `-all` option. If you use the `-all` option, internal patterns are also removed.

Note the following:

- You can also remove faults defined in an external file from a fault list using the `-delete` option of the `read_faults` command.
- All of the `remove_faults` options, other than `-launch` and `-capture`, are exclusive and cannot be issued at the same time, or with any of the old options. If you issue these options together, a UI error message will appear

Examples

```
TEST> add_faults /reg4
TEST> report_faults -all
sa0 DI reg4/CK
sa1 DI reg4/CK
sa0 DI reg4/Q
sa1 DI reg4/Q
sa0 DI reg4/lat1/Q
sa1 DI reg4/lat1/D
sa0 DI reg4/lat1/D
sa1 DI reg4/lat1/CK
sa0 DI reg4/lat1/CK
sa1 UT reg4/lat1/RB
sa0 DI reg4/lat1/RB
```

```

sa1 UT reg4/lat1/SB
sa0 DI reg4/lat1/SB
sa1 DI reg4/lat1/Q
sa0 DI reg4/r/SB
sa1 DI reg4/r/CK
sa0 DI reg4/r/CK
TEST> remove_faults reg4/lat1
TEST> report_faults -all
sa0 DI reg4/CK
sa1 DI reg4/CK
sa0 DI reg4/Q
sa1 DI reg4/Q
sa0 DI reg4/r/SB
sa1 DI reg4/r/CK
sa0 DI reg4/r/CK
TEST> report_faults -summary
      Uncollapsed Fault Summary Report
-----
fault class          code #faults
-----
Detected             DT    11816
Possibly detected    PT         0
Undetectable         UD     840
ATPG untestable      AU     3312
Not detected         ND    72116
-----
total faults 88084
test coverage 13.54%
-----
TEST> remove_faults -all
Warning: Internal pattern set is now removed. (M133)
TEST> report_faults -summary
      Uncollapsed Fault Summary Report
-----
fault class          code #faults
-----
Detected             DT         0
Possibly detected    PT         0
Undetectable         UD         0
ATPG untestable      AU         0
Not detected         ND         0
-----
total faults 0
test coverage 0.00%
-----

TEST> report_faults -all
sa0 NC reg0/r/RB
sa0 NC out2/EN
sa1 NC out2/EN
sa0 NC sdo_en/A

```



```
sa1 NC sdo_en/A
sa0 NC adder/A0
sa1 NC adder/A0
sa0 NC adder/A1
sa1 NC adder/A1
sa0 NC adder/CI
sa0 NC adder/S0
sa1 NC adder/S0
sa0 NC adder/CO
sa1 NC adder/CO
TEST> remove_faults sdo_en/A -stuck 1
TEST> remove_faults out2/EN -stuck 0
TEST> remove_faults /adder/CO
TEST> remove_faults -all
sa0 NC reg0/r/RB
sa1 NC out2/EN
sa0 NC sdo_en/A
sa0 NC adder/A0
sa1 NC adder/A0
sa0 NC adder/A1
sa1 NC adder/A1
sa0 NC adder/CI
sa0 NC adder/S0
sa1 NC adder/S0
```

See Also

[report_faults](#)

[add_faults](#)

[read_faults](#)

[Instance Name](#)

[Pin Pathname](#)

remove_lbist_seeds

Overview

Use this command to remove any seed values previously specified using the [add_lbist_seeds](#) command.

Syntax

```
remove_lbist_seeds
```

Arguments

None.

Allowed Command Modes

Test

Description

Use this command to remove any seed values previously specified using the [add_lbist_seeds](#) command.

Syntax

```
remove_lbist_seeds
```

Arguments

None.

Allowed Command Modes

Test

Description

Use this command to remove any seed values previously specified using the [add_lbist_seeds](#) command. If you run ATPG after specifying this command, TestMAX ATPG reverts to using the default pseudo-random seed value generation process. Alternatively, you can use the [add_lbist_seeds](#) command to start a new set of seed definitions. For more information on using TestMAX ATPG with the DFTMAX LogicBIST tool, see the *DFTMAX LogicBIST User Guide*.

Example

```
TEST-T> remove_lbist_seeds
```

See Also

[add_lbist_seeds](#)
[report_lbist_seeds](#)

remove_licenses

Overview

Use this command to manually release and check in selected licenses.

Syntax

```
remove_licenses {license}
```

Arguments

test-analysis | test-compression-atpg | test-faultsim | test-atpg-max | test-diagnosis | test-fault-max | test-power

Specifies the license or licenses to release and to check in to the license server. Any number of licenses can be released using a single command.

Allowed Command Modes

All

Description

Use this command to manually release and check in selected licenses.

For more information, see "[Licensing](#)."

Examples

In this example, a standard ATPG license is checked in.

```
TEST> report_licenses
test-analysis
test-faultsim
TEST> remove_licenses test-faultsim
TEST> report_licenses
test-analysis
```

See Also

[get_licenses](#)

[report_licenses](#)

[set_faults](#)

remove_net_connectionss

Overview

Use this command to remove user-defined net connections that were added with a prior `add_net_connections` command..

Syntax

```
remove_net_connections  
  <{net_names} | -all >
```

Arguments

net_names

Indicates the [name of the net](#) from the defined net connections which is to be removed.

`-all`

Specifies that all previously defined net connections are to be removed.

Allowed Command Modes

Build

Description

Use this command to remove user-defined net connections that were added with a prior `add_net_connections` command.

Examples

```
BUILD> remove_net_connections level2/float1
```

```
BUILD> report_net_connections  
Connection PI, net core/dac/fifo/reg0/RB, disconnect  
Connection PO, net core/dac/fifo/U943/Z  
BUILD> remove_net_connections -all
```

See Also

[report_net_connections](#)
[add_net_connections](#)

remove_nofaults

Overview

Use this command to remove nofault attributes for faults.

Syntax

```
remove_nofaults
  <-all | instance_name | pin_pathname | -instance_name instance_name | -
  module {module_name [optional_pin_names]}>
  [-stuck <0|1|01>]
  [-slow <r|f|rf>]
```

Arguments

-all

Specifies that all nofaults are to be removed.

instance_name

Removes all nofault attributes for faults associated with the selected [instance name](#) from the nofault list. The *instance_name* argument must be a fixed string.

pin_pathname

Removes all nofault attributes for faults associated with the selected [pin pathname](#).

-instance instance_name

Removes the nofault attribute for all faults associated with the *instance_name* argument. The *instance_name* can be a fixed string or a [limited regular expression](#).

-module {module_name [optional_pin_names]}

Removes the nofault attribute on one or more pins of every instantiation of a given module. The *module_name* argument can be a fixed string or a [limited regular expression](#). If an optional pin name or names is specified, then only the faults associated with that module and pin are used. If no pins are provided, then all pins of the module are used.

-slow <r|f|rf>

Removes faults with the specified slow value.

-stuck <0|1|01>

Selects either stuck 0, stuck 1, or both. The default is both stuck 0 and stuck 1.

Allowed Command Modes

DRC Test

Description

Use this command to remove nofault attributes for faults. The nofault attributes may have been defined in the netlist, added with the `add_nofaults` command, or added with the `add_equivalent_nofaults` commands. Nofault attributes cannot be removed from the nofault list if there are any faults in the current fault list.

Examples

```
TEST> report_nofaults -all
sa1 ** mux0/SL
sa0 ** buf0/Y
sa1 ** mux1/SL
sa0 ** buf1/Y
sa1 ** mux2/SL
sa0 ** buf2/Y
sa1 ** mux3/SL
sa0 ** buf3/Y
TEST> remove_nofaults mux2/SL
  1 faults were removed from nofault list.
TEST> remove_nofaults /mux0
  2 faults were removed from nofault list.
TEST> remove_nofaults /buf2/Y -stuck 0
  1 faults were removed from nofault list.
```

```
TEST> remove_nofaults -instance uPOWERA/add_CTS_CNIVX8L_G6B1I1
  4 faults were removed from nofault list.
```

```
TEST> remove_nofaults -instance uPOWERA/uRAM40K/M322RAM48K_8K6_NOBACKA0_
b/U.*
  1342 faults were removed from nofault list in wildcard instance
uPOWERA/uRAM40K/M322RAM48K_8K6_NOBACKA0_b/U.*.
```

See Also

[report_nofaults](#)
[read_nofaults](#)
[add_nofaults](#)
[add_equivalent_nofaults](#)
[Instance Name](#)
[Pin Pathname](#)

remove_physical

Overview

This command deletes specific data from the physical model created by the [run_diagnosis](#) command.

Syntax

```
remove_physical  
  [-all]  
  [-pairs]  
  [-pins]  
  [-subnets]  
  [-tolerance]
```

Arguments

-all

Removes all extracted data from the physical database.

-pairs

Removes all net pairs extracted during diagnosis or all net pairs loaded by the [read_layout](#) command.

-pins

Removes all pin and instance coordinates extracted during diagnosis, including pins extracted by the [extract_pins](#) command.

-subnets

Removes subnet topologies extracted during diagnosis by the [extract_nets](#) command or subnet data loaded by the [read_layout](#) command.

-tolerance

Resets all tolerances specified by the `-tolerance` option of the [set_physical](#) command. Note that the `-all` option does not reset tolerances.

Description

This command deletes specific data from the physical model created by the [run_diagnosis](#) command. If no options are specified, only data reported by the [report_physical](#) command is deleted.

Example

```
TEST-T> run_diagnosis test.fail
Diagnosis summary for failure file test.fail
#failing_pat=2, #failures=2, #defects=1, #faults=1, CPU_time=0.00
Simulated : #failing_pat=2, #passing_pat=4 #failures=2
-----
Fault candidates for defect 1: stuck fault model, #faults=1, #failures=2
-----
match=100.00%, #explained patterns: <failing=2, passing=4>
sa1 DS i1/out (NOT)
sa0 -- i1/in (NOT)
sa1 -- f3/d (SDFE)
-----
TEST-T> remove_physical -tolerance
TEST-T> set_physical -tolerance [list METAL 50 METAL2 100 METAL3 200 \
METAL4 300 METAL5 400]
```

See Also

[run_diagnosis](#)

[set_physical](#)

remove_pi_constraints

Overview

Use this command to delete PI constraints previously added with the `add_pi_constraints` command.

Syntax

```
remove_pi_constraints  
<{port_names} | -all>
```

Arguments

port_names

Specifies the [port name](#) of the PI (primary input) constraint to be removed.

`-all`

Specifies that all PI constraints should be removed.

Allowed Command Modes

DRC

Description

Use this command to delete PI constraints previously added with the `add_pi_constraints` command or implicitly defined in the test protocol file.

Examples

```
DRC> report_pi_constraints  
port_name          constrain_value  
-----  
/test_mode         1  
/pll_test_mode     1  
DRC> remove_pi_constraints -all  
DRC> report_pi_constraints  
Warning: Requested report contained no entries. (M13)
```

See Also

[report_pi_constraints](#)

[add_pi_constraints](#)

[Pin Name](#)

remove_pi_equivalences

Overview

Use this command to delete PI equivalence relationships previously added with the `add_pi_equivalences` command.

Syntax

```
remove_pi_equivalences  
  <{port_names} | -All>
```

Arguments

port_names

Specifies the name of the PI (primary input) or bidirectional port whose PI equivalence relationships are to be removed.

`-all`

Specifies that all PI equivalence relationships are to be removed.

Allowed Command Modes

DRC

Description

Use this command to delete PI equivalence relationships previously added with the `add_pi_equivalences` command.

Examples

```
DRC> report_pi_equivalences  
port_name          equivalent pins  
-----  
/NL0               ~/NSYNC ~/NL1  
DRC> remove_pi_equivalences -all  
DRC> report_pi_equivalences  
Warning: Requested report contained no entries. (M13)
```

See Also

[report_pi_equivalences](#)

[add_pi_equivalences](#)

[Pin Name](#)

remove_po_masks

Overview

Use this command to delete PO masks previously added with the `add_po_masks` command.

Syntax

```
remove_po_masks  
  <port_name | -All>
```

Arguments

port_name

Specifies the name of a [port](#) whose PO (primary output) mask is to be removed.

`-all`

Specifies that all PO masks are to be removed.

Allowed Command Modes

DRC

Description

Use this command to delete PO masks previously added with the `add_po_masks` command.

Examples

```
DRC> report_po_masks  
output port_name  
-----  
/LINK_X2  
DRC> remove_po_masks -all  
DRC> report_po_masks  
Warning: Requested report contained no entries. (M13)
```

See Also

[report_po_masks](#)

remove_scan_chains

Overview

Use this command to delete scan chains defined with the `add_scan_chains` command.

Syntax

```
remove_scan_chains
  <chain_name | -all>
```

Arguments

chain_name | -all

Specify the scan chain to be removed by the label previously used to define the scan chain when a prior add scan chains command was issued. Use of the `-all` option will remove all scan chain definitions from previous add scan chains commands.

Allowed Command Modes

DRC

Description

Use this command to delete scan chains defined with the `add_scan_chains` command.

This command does not affect any scan chains defined by the test protocol file referenced by the `run_drc` command. Removal of scan chains defined in a test protocol file requires editing of that file.

Examples

```
DRC> add_scan_chain chain_1 si[1] so[1]
DRC> add_scan_chain chain_2 si[2] so[2]
DRC> add_scan_chain C3 si[3] so[3]
DRC> report_scan_chains
chain    group    length input_pin    output_pin
-----
chain_1  sg0          0      si[1]        so[1]
chain_2  sg0          0      si[2]        so[2]
C3       sg0          0      si[3]        so[3]
DRC> remove_scan_chain C3
DRC> report_scan_chains
chain    group    length input_pin    output_pin
-----
chain_1  sg0          0      si[1]        so[1]
chain_2  sg0          0      si[2]        so[2]
```

See Also

[add_scan_chains](#)
[report_scan_chains](#)
[run_drc](#)

remove_scan_enables

Overview

Use this command to delete scan enables previously defined with the `add_scan_enables` command.

Syntax

```
remove_scan_enables
  <pin_name | -all>
```

Arguments

pin_name | -all

Specify the scan enable initialization to be removed by the top-level design port name previously used to define the scan enable when a prior `add_scan_enables` command was issued. Use of the `-all` option will remove all scan enable definitions from previous `add_scan_enables` commands.

Allowed Command Modes

DRC

Description

Use this command to delete scan enables previously defined with the `add_scan_enables` command.

This command does not affect any scan enable pin initialization defined in any test protocol file referenced by the `run_drc` command. Removal of pin initialization in the load_unload procedure defined in a test protocol file requires editing of that file.

Examples

```
DRC> add_scan_enables 1 SCAN_EN
DRC> add_scan_enables Z bidi[0] bidi[1] bidi[2]
DRC> report_scan_enables
value  port
-----
1      SCAN_EN (6)
Z      bidi[0] (3210)
Z      bidi[1] (3211)
Z      bidi[2] (3212)
DRC> remove_scan_enables bidi[1]
DRC> report_scan_enables
value  port
-----
1      SCAN_EN (6)
Z      bidi[0] (3210)
Z      bidi[2] (3212)
```

See Also

[add_scan_enables](#)
[report_scan_enables](#)
[run_drc](#)

remove_sdc

Overview

This command, which can only be run in Tcl mode, removes all SDC (Synopsys Design Constraints) information (SDC clocks and exceptions) so that a different set of SDC data can be read in.

Syntax

```
remove_sdc -all
```

Arguments

-all

Removes all SDC information.

Allowed Command Modes

DRC

Description

This command, which can only be run in Tcl mode, removes all SDC (Synopsys Design Constraints) information (SDC clocks and exceptions) so that a different set of SDC data can be read in. It is used only when multiple DRC and ATPG runs are combined into a single TestMAX ATPG session. The `remove_sdc` command does not change the settings made by previous [set_sdc](#) commands.

See Also

[read_sdc](#)

[set_sdc](#)

remove_slow_bidis

Overview

This command is used to remove BUS primitives that were modified with the `add_slow_bidis` command..

Syntax

```
remove_slow_bidis  
  <port_name | -all>
```

Arguments

port_name

Specifies the [port](#) whose modified bidirectional outputs are being removed.

-all

Indicates that all modified bidirectional outputs should be removed.

Allowed Command Modes

DRC

TEST

Description

This command is used to remove BUS primitives that were modified with the `add_slow_bidis` command.

See Also

[add_slow_bidis](#)

[report_slow_bidis](#)

remove_slow_cells

Overview

This command is used to remove the modification behavior of scan DFF or DLAT cells added with the `add_slow_cells` command.

Syntax

```
remove_slow_cells  
  <instance_path | gate_id | -all>
```

Arguments

instance_path | gate_id | -all

Specifies the [instance path name](#), the [gate ID](#), or that all scan DFF or DLAT cells whose simulation behavior was modified should be removed.

Allowed Command Modes

DRC

Description

This command is used to remove the modification behavior of scan DFF or DLAT cells added with the `add_slow_cells` command.

See Also

[add_slow_cells](#)
[report_slow_cells](#)

remove_waveform_signals

Overview

Removes gates associated with the pin pathname, gate ID, net names, or all gates from the GSV.

Syntax

```
remove_waveform_signals  
  <pin_pathname | gate_id | net_names| -all>
```

Arguments

pin_pathname | gate_id | net_names | -all

Specifies the [pin pathname](#) or the [gate ID](#) of the element whose simulation behavior is being modified. If you specify the `-all` option, TestMAX ATPG removes all gates from the GSV.

Allowed Command Modes

DRC and Test

Description

Removes gates associated with the pin pathname, gate ID, net names, or all gates from the GSV.

See Also

[add_waveform_signals](#)

Report Commands

This section describes the following commands:

- [report_atpg_constraints](#)
- [report_lbist_seeds](#)
- [report_scan_ability](#)
- [report_atpg_primitives](#)
- [report_licenses](#)
- [report_scan_cells](#)
- [report_buses](#)
- [report_lockup_latches](#)
- [report_scan_chains](#)
- [report_capture_masks](#)
- [report_memory](#)
- [report_scan_enables](#)
- [report_cell_constraints](#)
- [report_modules](#)
- [report_scan_path](#)
- [report_clocks](#)
- [report_net_connections](#)
- [report_sdc](#)
- [report_commands](#)
- [report_nets](#)
- [report_serializers](#)
- [report_compressors](#)
- [report_nofaults](#)
- [report_settings](#)
- [report_delay_paths](#)
- [report_nonscan_cells](#)
- [report_nonscan_cells](#)
- [report_display_gates](#)
- [report_patterns](#)
- [report_slow_cells](#)
- [report_distributed_processors](#)
- [report_physical](#)
- [report_summaries](#)
- [report_faults](#)
- [report_pi_constraints](#)
- [report_timing](#)
- [report_fanin](#)
- [report_pi_equivalences](#)

- [report_toggle_weights](#)
- [report_fanout](#)
- [report_po_masks](#)
- [report_version](#)
- [report_feedback_paths](#)
- [report_power](#)
- [report_violations](#)
- [report_instances](#)
- [report_primitives](#)
- [report_wires](#)
- [report_layout](#)
- [report_rules](#)

report_atpg_constraints

Overview

Use this command to report data from the current ATPG constraints list.

Syntax

```
report_atpg_constraints
  [-summary | -all]
  [-max d]
  [-no_detection_credit [-max number]]
```

Arguments

-summary

Selects a summary report of all ATPG constraints. This is the default report type.

-all

Specifies that all ATPG constraints should be reported.

-max d

Specifies the maximum number of ATPG constraints to be reported. The default is no limit.

-no_detection_credit [-max number]

Reports information on the current no detection credit cells specified by the `-no_detection_credit` option of the [add_atpg_constraints](#) command. The `-max` option limits the number of cells reported.

The information reported for each no detection credit cell includes the following:

- Gate ID of the associated DLAT/DFF
- Instance name of the associated DLAT/DFF
- Chain name and position in the chain if the DLAT/DFF is a scan cell

For a complete description of using this option, see [Defining, Reporting, and Removing No Detection Credit Cells](#).

Allowed Command Modes

DRC, TEST

Description

Use this command to report data from the current ATPG constraints list.

Examples

```
DRC> report_atpg_constraints
ATPG constraint summary: #constraints=3, #DRC_constraints=0
```

```
DRC> report_atpg_constraints -all
name                               val DRC site
```

```
-----
my_constraint1      1  no  my_atpg_primitive1 (2217)
my_constraint2      0  no  MAIN/CPU/ARIT/U842 (911)
my_constraint3      1  yes bear/gol/pie/U44 (1683)

DRC> report_atpg constraints -all -max 2
name                val DRC site
-----
my_constraint1      1  no  my_atpg_primitive1 (2217)
my_constraint2      0  no  MAIN/CPU/ARIT/U842 (911)
```

See Also

- [add_atpg_constraints](#)
- [remove_atpg_constraints](#)
- [ATPG Constraints Report](#)
- Table Column Outside Table:

report_atpg_primitives

Overview

Use this command to report data from the current ATPG primitives list.

Syntax

```
report_atpg_primitives
  <atpg_primitive_name | -summary | -all>
  [-max d] [-verbose]
```

Arguments

atpg_primitive_name

Specifies the [name of the ATPG primitive](#) to report.

-summary

Specifies that a summary of all ATPG primitives is reported. This is the default if you don't select -All or an ATPG primitive name.

-all

Specifies that all ATPG primitives should be reported.

-max d

Selects the maximum number of ATPG primitives to be reported. The default is no limit.

-verbose

The verbose switch causes a detailed report, which includes the name of the ATPG primitive, its [primitive ID](#), and the source of its inputs.

Allowed Command Modes

DRC, Test

Description

Use this command to report data from the current ATPG primitives list.

Examples

```
TEST> report_atpg_primitives -all
name          id      type  inputs
-----
my_atpg_prim  20217    EQUIV 861 990
```

```
TEST> report_atpg_primitives -all -verbose
my_atpg_prim (20217) EQUIV (_EQUIV)
  --- I () 861-airball/core/CPU/TP/reg/U14/N01
  --- I () 990-airball/core/CPU/TP/sub/U3/N02
my_atpg_prim 0 ()
```

See Also[add_atpg_primitives](#)[remove_atpg_primitives](#)[ATPG Primitives Report](#)

Table Column Outside Table:

report_buses

Overview

Use this command to report data associated with BUS primitives.

Syntax

```
report_buses
  < gate_id | -behavior <buf | inv | and | or | xor | mux | tie0 | tie1 |
  tiez> | -contention <fail | pass | abort | bidi> | -zstate <fail | pass |
  abort | bidi> | -bidis | -clock | -keepers | -pull | -weak | -summary | -
  all >
  [-names]
  [-max d]
  [-verbose]
```

Arguments

gate_id

Specifies a specific BUS primitive for reporting by its [primitive ID](#). An error occurs if the ID is not a BUS primitive.

-behavior <buf | inv | and | or | xor | mux | tie0 | tie1 | tiez>

Specifies that BUS primitives that match a particular behavior category are to be reported.

-bidis

Specifies that bus data is displayed for BUS primitives that connect to bidirectional ports.

-clock

Specifies that bus data is displayed for BUS primitives that have a combinational path leading from a [clock input](#).

-contention <fail | pass | abort | bidi>

Specifies that BUS primitives of the selected contention category is reported.

-keepers

Specifies that bus data is displayed for BUS primitives that connect to [bus keepers](#).

-pull

Specifies that bus data is displayed for BUS primitives that have both strong and weak drivers connecting to BUS inputs.

-weak

Specifies that bus data is displayed for BUS primitives that have only weak drivers connecting to BUS inputs.

-zstate <fail | pass | abort | bidi>

Specifies that bus data is displayed for BUS primitives of the selected Z-state category.

-summary

Specifies that a summary of all BUS primitives is given.

-all

Specifies that bus data is displayed for all BUS primitives.

-max d

Indicates the maximum number of buses reported. The default is to have no limit.

-names

Prints the names of all buses; one bus is reported per line. You can redirect this output to a file if the design contains a large number of buses. You should use the `-all` option with this option. You can use the `-max` option to limit the number of reported buses. A bus defined in a `celldefined` module may have a partial or incomplete name due to the cell constructs.

-verbose

Indicates that verbose data is displayed for each BUS primitive. The default is not to display verbose data.

Allowed Command Modes

DRC, Test

Description

Use this command to report data associated with BUS primitives. For additional background on BUS classification see: Understanding Contention Analysis

Examples

```
report_buses -summary
  Bus summary: #bus_gates=577, #bidi=128, #weak=0, #pull=0, #keepers=0
  Contention status: #pass=257, #bidi=31, #fail=289, #abort=0, #not_
analyzed=0
  Z-state status : #pass=160, #bidi=128, #fail=286, #abort=3, #not_
analyzed=0
  Learned behavior : none
```

```
report_buses -all -noverbose -max 8
      contention    zstate    #drivers
gate_id status/capture status  strong/weak behavior_data
-----
12441  pass  stable  fail      2 0
12443  pass  stable  fail      2 0
12445  pass  stable  fail      2 0
12447  pass  stable  fail      2 0
12449  pass  stable  fail      2 0
12451  pass  stable  fail      2 0
12453  pass  stable  fail      2 0
12455  pass  stable  fail      2 0
```

```
report_buses -all -verbose -max 8
      contention    zstate    #drivers
gate_id status/capture status  strong/weak behavior_data
-----
12441  pass  stable  fail      2 0
```

```

inputs: 12408 (BUFZ) 12440 (BUFZ)
12443 pass stable fail 2 0
inputs: 12409 (BUFZ) 12442 (BUFZ)
12445 pass stable fail 2 0
inputs: 12410 (BUFZ) 12444 (BUFZ)
12447 pass stable fail 2 0
inputs: 12411 (BUFZ) 12446 (BUFZ)
12449 pass stable fail 2 0
inputs: 12412 (BUFZ) 12448 (BUFZ)
12451 pass stable fail 2 0
inputs: 12413 (BUFZ) 12450 (BUFZ)
12453 pass stable fail 2 0
inputs: 12414 (BUFZ) 12452 (BUFZ)
12455 pass stable fail 2 0
inputs: 12415 (BUFZ) 12454 (BUFZ)

```

```

report_buses -bidis -max 4
      contention  zstate  #drivers
gate_id status/capture status strong/weak behavior_data
-----
12667  bidi stable    bidi    2 0
37039  fail unstable  bidi    3 0
37040  fail unstable  bidi    3 0
37041  fail unstable  bidi    3 0

```

```

report_buses -contention fail -max 8
      contention zstate #drivers
      id status/capture status strong/weak behavior_data
-----
26469  fail unstable  fail    4 0
26473  fail unstable  fail    4 0
26477  fail unstable  fail    4 0
26481  fail unstable  fail    4 0
26484  fail unstable  fail    4 0
26488  fail unstable  fail    4 0
26491  fail unstable  fail    4 0
26495  fail unstable  fail    4 0

```

```

report_buses -contention pass -max 4
      contention zstate #drivers
      id status/capture status strong/weak behavior_data
-----
12441  pass stable    fail    2 0
12443  pass stable    fail    2 0
12445  pass stable    fail    2 0
12447  pass stable    fail    2 0

```

```

report_buses -contention bidi -max 8
      contention zstate #drivers
      id status/capture status strong/weak behavior_data

```

```

-----
12667 bidi stable   bidi    2 0
54286 bidi unstable bidi    3 0
54290 bidi unstable bidi    3 0
54304 bidi unstable bidi    3 0
54306 bidi unstable bidi    3 0
54317 bidi unstable bidi    3 0
54876 bidi unstable bidi    3 0
54866 bidi unstable bidi    3 0

```

```

report_buses -zstate fail -max 4
             contention zstate #drivers
             id status/capture status strong/weak behavior_data
-----

```

```

12441 pass stable   fail    2 0
12443 pass stable   fail    2 0
12445 pass stable   fail    2 0
12447 pass stable   fail    2 0

```

```

report_buses -zstate pass -max 6
             contention zstate #drivers
             id status/capture status strong/weak behavior_data
-----

```

```

29488 fail unstable pass    13 0
29494 fail unstable pass    13 0
29584 fail unstable pass    13 0
29595 fail unstable pass    13 0
29613 fail unstable pass    13 0
29624 fail unstable pass    13 0

```

```

report_buses -zstate abort -max 32
             contention zstate #drivers
             id status/capture status strong/weak behavior_data
-----

```

```

41395 pass stable   abort    22 0
41412 pass stable   abort    22 0
41442 pass stable   abort    22 0

```

```

report_buses -zstate bidi -max 4
             contention zstate #drivers
             id status/capture status strong/weak behavior_data
-----

```

```

12667 bidi stable   bidi    2 0
37039 fail unstable bidi    3 0
37040 fail unstable bidi    3 0
37041 fail unstable bidi    3 0

```

See Also

- [Buses Report](#)
- [Bus Gate](#)
- [Bus Keeper](#)
- [Contention Analysis](#)
- `analyze_buses`
- Table Column Outside Table:

report_capture_masks

Overview

Use this command to report any defined capture masks which can exist on nonscan sequential cells.

Syntax

```
report_capture_masks
```

Arguments

None

Allowed Command Modes

DRC, Test

Description

Use this command to report any defined capture masks which can exist on nonscan sequential cells.

Examples

```
DRC> report_capture_masks
gate_id type instance_name
-----
19252 DFF MAIN/U87/DATA_OUT
10286 DFF MAIN/U111/REG_1
 9287 DFF MAIN/U112/BARSQ
11288 DFF MAIN/U113/KJX1
```

See Also

[add_capture_masks](#)

[remove_capture_masks](#)

report_cell_constraintss

Overview

Use this command to report cell constraints added with the `add_cell_constraints` command.

Syntax

`report_cell_constraints`

Arguments

None

Allowed Command Modes

DRC, Test

Description

Use this command to report cell constraints added with the `add_cell_constraints` command. Any cell constraint that was not defined by name is reported as `"..."` under the site name.

Examples

```
TEST-T> report_cell_constraints
type chain    pos.  site name
-----
  1   c10      6     MAIN/cat/TP/FI/OFIFO/\regst[3]
OX   c34     17     MAIN/dog/PRT/PORTIN_1/\reg3[3]
XX   c34      6     ...
```

Note in the following example that the reported constraints are raw specifications and that no checking is performed during DRC that the specifications can actually be applied:

```
TEST-T> report_cell_constraints
# # type chain position site name
# # ----
# # OX 1 10-22
# # X 1 0-13
```

See Also

- [add_cell_constraints](#)
- [remove_cell_constraints](#)
- [Cell Constraints Report](#)
- Table Column Outside Table:

report_clocks

Overview

This command reports all pins defined to be clocks, either by the `add_clocks` command.

Syntax

```
report_clocks
  [-capture_matrix]
  [-constraints [-procedure name | -all>]]
  [-command_report | -verbose | -matrix | -intclocks | -pllclocks | -
  dominant] [-by_id]
  [-gating]
```

Arguments

-capture_matrix

Reports clock pulse overlapping information used for synchronizing multi frequency internal clocks. For more information, see "[Specifying Synchronized Multi Frequency Internal Clocks](#)" in the *TestMAX ATPG User Guide*.

-constraints [-procedure name | -all]

Reports internal clocking procedures as they are used by ATPG. The `-procedure name` option provides information only for a specific internal clocking procedure. The `-all` option provides information for all clocking procedures. If neither `-procedure` nor `-all` are specified, a summary of the usable clocking procedures is reported. For more information, see "[Using Internal Clocking Procedures](#)."

-command_report

Causes the clocks to be reported in the form of the `add_clocks` command that would be necessary to define the clocks in a [command file](#). This option only supports clocks defined at top of the design (external clocks). Statements to generate internal clocks are not created at this time.

-matrix

Displays the dynamic clock pair group ability in table form where each row indicates the potential grouping relationship of a candidate clock with each of the other candidate clocks. This option will report information for both external and internal clocks. Note the following:

- This option can only be used in Test mode.
- TestMAX ATPG can identify additional disturbed clocks at the beginning of the [run_atpg](#) command process. This means a clock matrix reported before the `run_atpg` command process can be more restrictive than a matrix used during ATPG. To determine if the clock matrix was actually used by TestMAX ATPG, you should use the `-matrix` option after the first invocation of TestMAX ATPG.

-verbose

Causes additional columns to be reported which display the number of scan and nonscan elements affected by each clock. The different counts are further broken down into the

number of sets, resets, and clocking connections involved. A determination of scan vs. nonscan can only be made after DRC has been performed and TestMAX ATPG is in TEST mode. **Note:** Connection counts to RAM/ROM devices are not reported. This option will report information for both external and internal clocks.

-intclocks

Reports all internal clocks and all information associated with each internal clock.

-pllclocks

Reports all PLL clocks and all information associated with each PLL clock.

-dominant

Reports the single external clock that contains the most connections in the design. Reports only one clock; if multiple clocks have the same number of connections only one is reported. Connection counts to RAM/ROM devices are not counted for this identification. Also note that internal clocks are not considered in this count, even if they have higher connection counts.

-by_id

This option applies only to the `-verbose` format. This option will suppress printing the clock name in the report, and prints the gate id instead. For reports where clock names are long, this option may be used to keep report columns organized. You can cross-reference the gate id to the name with the `report_primitives` command.

-gating

Used with the Power Aware ATPG feature, this switch (when also specified with the `-verbose` switch) reports whether a design contains clock gating cells within the context of ATPG. Note that `-gating` may only be used after your design has successfully passed the [run_drc](#) process. Keep in mind that this analysis is performed using the information contained within the STL procedure file. If you have a constraint in which the clock-gating cells are always transparent, this analysis will not show those (CG) cells, and they are not usable within the context of Power Aware ATPG. For details, see "[Power Aware ATPG](#)" in the *TestMAX ATPG User Guide*.

Allowed Command Modes

DRC, Test

Description

This command reports all pins defined to be clocks, either by the `add_clocks` command or implicitly defined in the test protocol file. These clock pins include active-high and active-low clocks, asynchronous sets, asynchronous resets, and write control lines.

After entering TEST command mode, the learned usage of each clock is also reported.

If you specify this command with no arguments, only the external clocks are identified. Internal clocks are listed with the `-verbose` or `-intclocks` options, PLL clock definitions are listed with the `-pllclocks` option, and internal clocking procedures are listed with the `-constraints` option. The `-matrix` option will also list interactions of all clocks (both internal and external).

Examples

```
report_clocks
clock_name    off  usage
```

```

-----
CLK      0      master shift
RSTB     1      master reset

```

```

report_clocks -command_report
add_clocks 0 CLK
add_clocks 1 RSTB

```

```

report_clocks -verbose
scan connections      nonscan connections
clock_name    off    clock set reset      clock set reset
-----
clk_80        0      1182  0    0          1    0    0
clk_40  0      7657  0    0      21    0    0
clk_20        0      156  0    0          2    0    0
clk_10        0      486  0    0          2    0    0
CH_3          0      1211  0    0  0    0    0
CH_2          0      217  0    0          1    0    0
CH_1          0    372  0    0  0    0    0
L1_clk  0        22  0    0          0    0
L2_clk        0        22  0    0          0    0  0
RESETB       1        0 266 5888          0    2    0

```

```

report_clocks -matrix
id# clock_name      type      0    1    2    3
---
0 Ram_Clk           CW      --- BPA --- B--
1 BPCICLK           C        BPA --- --A ---
2 Pixel_Clk         C        --- B-- --- ---
3 nReset  SR        --A --- --- ---

```

```

report_clocks -intclocks -verbose
# int_clock_instance_name gate_id off source sync period LE  TE  lat cycle
conditions
# -----
# pll_controller_M1/U2      6698  0    138      1  1  0    10  5    0
13337=1 (0,4)
#
13336=1 (0,5)
...
#
13324=1 (0,17)
# pll_controller_M2/U2  7347  0    139      1      1  0    10  5  0    13362=1
(0,4)
#
13361=1 (0,5)
...
#
13324=1 (0,17)
# pll_controller_M3/U2      8567  0    187      1      2  5    25  5    0
13387=1 (0,4)
#
...

```

```
# 13
13324=1 (0,17)
TEST> report_clocks -constraints -all
-----
Clock Constraints constraints1:
Maximum sequential depth: 2
Defined Clocking Procedures: 3
Usable Clocking Procedures: 3
PLL clocks off Procedure: ClockOff

U0to1:
  CLKIR=10010
  dutm/ctrl1/U17/Z=P0
  dutm/ctrl2/U19/Z=0P
-----
U1to0:
  CLKIR=01010
  dutm/ctrl1/U17/Z=0P
  dutm/ctrl2/U19/Z=P0
-----
ClockOff:
  CLKIR=00000
  dutm/ctrl1/U17/Z=00
  dutm/ctrl2/U19/Z=00
-----
```

See Also[add_clocks](#)[remove_clocks](#)[Understanding the report_clocks Output](#)

report_compressors

Overview

Generates a report containing data for the specified compressors.

Syntax

```
report_compressors
  <compressor_name | -load | -unload | -pipeline | -shift_power_controller |
  -all>
  [-verbose]
```

Arguments

compressor_name | **-load** | **-unload** | **-pipeline** | **-shift_power_controller** | **-all**

You can specify a report related to a particular compressor name (*compressor_name*), or use one of the following options: **-load** reports all load compressors; **-unload** reports all unload compressors; **-pipeline** displays information about the load and unload compressor pipelines (if any) for each pipeline input, in separate blocks (see the examples below); **-shift_power_controller** reports chain and connection information for shift power groups (SPC); **-all** reports all compressors.

-verbose

Reports the data for the specified compressors in verbose mode.

Description

This command generates a report containing data for the specified compressors. Note that this command is valid only in TEST mode.

Examples

The following example shows the information reported as a result of specifying the **-pipeline** option:

```
report compressor -pipeline
Compressor pipeline report: #input stages=1, #output_stages=1
-----
mode_port output_gate invert type clock pipeline_cells
-----
SI_5          9544      1      hold ----- 9544 (1)
```

In the previous example, information is reported for each mode port, as follows:

Port input name: SI_5

Output gate ID: 9544

Combined inversion status of all pipeline stages: 1

(In this example 1 stage is reported; 1 = inversion, 0= no inversion)

Load pipeline cell type: hold

(there are three possible cell types -- hold, shift, both. For a complete explanation of these cell types, see the [R27](#) and [R18](#) rules).

Clock gate ID: In this case, there is no clock gate required to hold, so the clockfield is empty.

Pipeline cells gate ID: 9544(1)

(Inversion is indicated in the parentheses.)

The following set of tables show pipeline information for the load compressor inputs, the unload mode ports (one table for each load mode mode_id), the unload enable port, and the unload outputs. Note that the capture behavior of output pipeline stages (that is, "hold", "shift" or "both") is not relevant -- thus it is not checked and is not displayed.

```
-----
load_compressor_input output_gate invert type clock pipeline_cells
-----
SI_0                  9534          1  hold ----- 9534(1)
-----

pipeline unload mode port data for mode_id 0 of unload compressor des_unit_
U_compressor
unload_mode_port output_gate invert type clock pipeline_cells
-----
SI_0                  9534          1  hold ----- 9534(1)
-----

pipeline unload mode port data for mode_id 1 of unload compressor des_unit_
U_compressor
unload_mode_port output_gate invert type clock pipeline_cells
-----
SI_0                  9534          1  hold ----- 9534(1)
-----

unload_enable_port output_gate invert type clock pipeline_cells
-----
SI_7                  9548          1  hold ----- 9548(0)
-----

unload_compressor_output input_gate invert pipeline_cells
-----
S0_0                  9466          1  hold ----- 9535(0)
-----
```

In the next example, all pipeline flip-flops that are in the same pipeline stage are reported within parenthesis separated by commas:

```
report_compressor -pipeline
Compressor pipeline report: #input stages=2, #output_stages=0
-----
mode_port      output_gate invert type  clock pipeline_cells
-----
test_si3 10544 0 both  0 (9628,9857,10086,10315,10544) (0) 9623(0) (Hold
```

```
condition:18=0)
  test_si4 10545 0 both 0 (9629,9858,10087,10316,10545)(0) 9624(0) (Hold
condition:18=0)
```

In the next example, the master and slave flip-flops are reported within brackets; the master flip-flops are appended with an "m" and the slave flip-flops are appended with an "s":

```
report_compressor -pipeline
Compressor pipeline report: #input_stages=2, #output_stages=0
-----
mode_port      output_gate invert type  clock pipeline_cells
-----
test_si3 3117 0 both 0 [3112m 3117s](0) (Hold condition:18=0)
test_si4 3116 0 both 0 [3111m 3116s](0) (Hold condition:18=0))
```

1. "0" in the invert path means the overall pipeline path has an even number of inversions (where "0" counts as even)
2. The [] are used to pair master-slave combinations. Many pipelines "fork" on the slave element and there can be multiple slaves associated with a single master. It really means that these multiple elements are identified as one pipeline stage.
3. "m" means master
4. "s" means slave

The next example shows the output when using the `-shift_power_controller` and `-verbose` options:

```
TEST-T> report_compressors -shift_power_controller -verbose
-----
Total number of Shift Power Controller Chains = 1
Shift Power Controller Chain = OCC
Total Number of Shift Power Control Registers = 30
Total Number of bits in chain OCC = 46
Shift Power Controller Register cell      gate# #chains chains in group
-----
tenx_U_DFT_spwr_cntrl_chain_0/serial_reg_0_ 16 74103 4 2 3 4 5
tenx_U_DFT_spwr_cntrl_chain_0/serial_reg_1_ 17 74102 4 6 8 9 7
tenx_U_DFT_spwr_cntrl_chain_0/serial_reg_2_ 18 74101 4 12 11 13 10
.....
```

The first four lines of the previous example show the summary information for the shift power groups. This information is always printed when you specify the `-shift_power_controller` option. These lines show the shift power controller chain information, including whether the chain contains other registers used for non-SPC purposes.

The remaining lines of the previous example are printed only when the `-verbose` option is specified. There are normally more lines than displayed in the example (the complete report for this example contains 30 lines). These lines display the cell, gate number, and number of chains associated with each shift power controller register. The "chains in group" field shows the chain names for all compressed chains controlled by a specific SPC chain register.

The following example demonstrates output of the `report_compressors -all -verbose` command. This report provides a summary of the decompressor and compressor connections from the internal chains to the scanin or scanout port for each mode.

```
report_compressors -all -verbose
-----
name type #inputs #outputs #modes          mode controls
-----
DFT__des_unit_U_decompressor_ScanCompression_mode  load 6  96 3 Mode 0:
test_si8=0 test_si7=0
                                         Mode 1: test_si8=1 test_
si7=0
                                         Mode 2: test_si8=0 test_
si7=1
-----
external port connection          input output external chain connection
-----
test_si1 0 0 1
test_si2 1 1 2
test_si3 2 2 3
test_si4 3 3 4
test_si5 4 4 5
test_si6 5 5 6
      --- 6 7
      --- 7 8
      ... ..
-----
output_id inv ports connected to output for mode 0
-----
0          yes test_si1
1          yes test_si2
2          yes test_si3
3          yes test_si4
4          yes test_si5
5          yes test_si6
.          .. ..
-----
output_id inv ports connected to output for mode 1
-----
0          yes test_si1
1          yes test_si2
2          yes test_si3
3          yes test_si4
4          yes test_si5
5          yes test_si6
.          .. ..
```

See Also

[remove_compressors](#)
[Understanding Compressor Connections in the Output from report_scan_chains](#)

Table Column Outside Table:

report_delay_paths

Overview

This command generates individual paths for viewing with the GSV and exports a delay path list.

Syntax

```
report_delay_paths
  <-all | path_name>
  [-verbose]
  [-display]
  [-pindata]
```

Arguments

-all | path_name

Reports all valid [delay paths](#) in TestMAX ATPG delay path definition format. If you provide a *path_name*, TestMAX ATPG reports only the single path associated with the *path_name*.

-display

Displays the specified *path_name* in the GSV.

-verbose

Reports detailed data about each path including: fault status, transition direction, number of [launch clocks](#), launch clock, launch clock node (ID), number of capture clocks, capture clock, capture clock node (ID), and node IDs and gate types for nodes defined along the path.

-pindata

Uses the pindata setting to delay data, and allows pindata to be annotated to the path view in the GSV, or reports pindata in a primitive report if requested.

Allowed Command Modes

Test

Description

This command generates individual paths for viewing with the GSV and exports a delay path list.

Examples

```
TEST> report_delay_paths path1 -v

$path {
  $name path1 ;
  # #launch_clocks=1, launch_clock=CK (1), launch_node=DFF (22)
  # #capture_clocks=1, capture_clock=CK (1), capture_node=DFF (21)
  # slow-to-rise fault class = DR
  # slow-to-fall fault class = DR
  $transition {
    reg_fd1/Q = ; # 22 (DFF)
```

```
        reg_fd2/D1 = ; # 18 (MUX)
    }
}
```

See Also

`add_delay_paths`

report_display_gates

Overview

Use this command to generate information about the gates currently being displayed in the GSV.

Syntax

```
report_display_gates [-command]
```

Arguments

-command

This option prefixes each reported gate with the [add_display_gates](#) command, which can be used in a command file to repopulate the current GSV display (see the Examples section below).

Allowed Command Modes

DRC, Test

Description

Use this command to generate information about the gates currently being displayed in the GSV. The report contains the pin_pathname and gate type, and the type and number of ports on each gate.

Examples

```
DRC> report_display_gates
U05 (46): type=OR #inputs=2 #outputs=1 #bidis=0
U75 (47): type=AND #inputs=2 #outputs=1 #bidis=0
muxme (48): type=MUX #inputs=3 #outputs=1 #bidis=0
flop_it (77): type=MUX #inputs=3 #outputs=1 #bidis=0
SCANOUT (126): type=PO #inputs=1 #outputs=1 #bidis=0
```

```
TEST-T> report_display_gates -command
add_display_gate 6574599 ; # u_padring/pad_inst_gpio_1 (BUS)
add_display_gate 6574958 ; # u_padring/pad_inst_gpio_1 (AND)
add_display_gate 6574987 ; # u_jtag/U110 (AND)
```

See Also

[add_display_gates](#)

[remove_display_gates](#)

report_distributed_processors

Overview

Use this command to get information on the distributed processors performing a distributed fault simulation task.

Syntax

```
report_distributed_processors
```

Arguments

None

Allowed Command Modes

Test

Description

Use this command to get information on the distributed processors performing a distributed fault simulation task.

Examples

```
TEST> report_distributed_processors
MACHINE himalaya [ARCH: sparc-64]
MACHINE himalaya [ARCH: sparc-64]
MACHINE himalaya [ARCH: sparc-64]
MACHINE himalaya [ARCH: sparc-64]
```

See Also

[add_distributed_processors](#)
[remove_distributed_processors](#)
[set_distributed](#)

report_fanin

Overview

This command reports fanin paths to a specific pin.

Syntax

```
report_fanin -from pin_name -to pin_name
  [-hide_sequential]
  [-shift] [-test_setup] [-pi_constraints]
```

Arguments

-from *pin_name*

Specifies the hierarchical pin name in the top-level netlist to be used for mapping/tracing. Wildcard pin names are supported, although they do not apply to instance names.

-to *pin_name*

Specifies the pin name to be used as the end point or sink for mapping/tracing. By default, the `report_fanin` command prints all the PIs, PIOs and flip-flops that are driving this pin. You can specify wildcard characters for specifying pin names, but not instance names. The `report_fanin` command will report, one-by-one, all the sources for all the matching endpoint pin names.

-hide_sequential

Hides printing flip-flops or latch endpoints, and outputs only the PIs and POs.

-shift

Outputs endpoints after the shift constraints are applied.

-test_setup

Outputs endpoints after applying the test_setup sequence.

-pi_constraints

Outputs endpoints after applying the pin constraints.

Description

This command reports fanin paths to a specific pin. If there are multiple sources, it will report all the names. This command is valid only in Tcl mode.

Examples

Default report:

```
report_fanin -to top/Udftmax_core1/DFTMAX821/ZN

Possible sources for pin top/Udftmax_core1/DFTMAX821/ZN .
Udftmax_core1/d1_reg_3_ (3826) DFF
dftmax_test_mode (28) PI
  test_si12 (19) PI
  test_si14 (21) PI
  test_si11 (18) PI
```

```
test_si12 (19) PI
test_si15 (22) PI
```

Hides the FF Endpoints:

```
report_fanin -to top/Udftmax_core1/DFTMAX821/ZN -hide

Possible sources for pin top/Udftmax_core1/DFTMAX821/ZN .
dftmax_test_mode (28) PI
test_si12 (19) PI
test_si14 (21) PI
test_si11 (18) PI
test_si12 (19) PI
test_si15 (22) PI
```

See Also

[report_fanout](#)

report_fanout

Overview

This command reports fanout paths from a specific pin.

Syntax

```
report_fanout -from pin_name -to pin_name
  [-hide_sequential]
  [-shift]
  [-test_setup]
  [-pi_constraints]
```

Arguments

-from *pin_name*

Specifies the hierarchical pin name in the top-level netlist to be used as the starting point for reporting fanout paths. Wildcard pin names are supported, although they do not apply to instance names.

-to *pin_name*

Specifies the pin name to be used as the end point or sink for mapping/tracing. You can specify wildcard characters for specifying pin names, but not instance names.

-hide_sequential

Hides printing flip-flops or latch endpoints, and outputs only the PIs and POs.

-shift

Outputs endpoints after the shift constraints are applied.

-test_setup

Outputs endpoints after applying the test_setup sequence.

-pi_constraints

Outputs endpoints after applying the pin constraints.

Description

This command reports fanout paths from a specific pin. Note that this command is valid only in Tcl mode.

See Also

[report_fanin](#)

report_faults

Overview

Use this command to display fault data.

Syntax

```
report_faults
  [-aggressor_node <first | second | both>]
  [instance_name | pin_pathname
  [bridge_location1 [bridge_location2]]
  [-bridge <0|1|01>]
  [-cell_aware none | only]
  [-class fault_class]
  [-bridge_feedback]
  [-bridge_strong]
  [-collapsed | -uncollapsed]
  [-dynamic_bridge <r|f|rf>]
  [-dominant_node <first | second | both>]
  [-external {file_names}]
  [-level d m]
  [-max d]
  [-max_tmgn float]
  [-min_tmgn float]
  [-max_delta float]
  [-ndetects <histogram | faults n>]
  [-pattern_id d]
  [-per_clock_domain]
  [-slack <tgn [integer|float] | tdet [integer|float] | delta
  [integer|float] | sdql | effectiveness>]
  [-stuck <0|1|01> | -slow <r|f|rf>]]
  [-unsuccessful | -summary | -profile | -all ]
  [-verbose]
```

Arguments

Note: The following arguments are not valid when the [set_faults](#) -model path_delay command is specified:

```
[-collapsed | -uncollapsed] [-level d <m>] [-unsuccessful | -profile]
```

-aggressor_node <first | second | both>

Specifies which node in a node pair is used as the aggressor node of a bridging fault: *first*, *second*, or *both*. The default is *both*. Using *first* or *second* limits the set of faults reported and only reports the faults with the aggressor in the position specified. To see how combinations of **-bridge** and **-aggressor_node** option settings interact, see the description of the **-aggressor** option of the **add_faults** command.

Using default settings, four faults are reported for each node pair defined if the faults were included in the list.

***bridge_location1* [*bridge_location2*]**

Specifies the nodes of the bridging or dynamic bridging fault being reported. Either node can be selected to be an aggressor node, victim node, or both using the `-aggressor` option for the bridging fault model and the `-dominant` option for the dynamic bridging fault model. If `bridge_location2` is not specified, then all bridges with `bridge_location1` (both aggressor and victim) are reported.

-bridge <0|1|01>

Specifies the type of bridging fault being added. The type can be either:

- 0 - ba0 (bridging fault at 0)
- 1 - ba1 (bridging fault at 1)
- 01 - both (bridging fault at 0 and bridging fault at 1)

The default is 01 (both).

-bridge_feedback

Reports all bridges where the aggressor is in the combinational fanout cone of the victim.

-bridge_strong

Reports only those faults that are strength-sensitive. These are faults where either the victim or aggressor node is driven by a gate with a dominant value (AND, OR, NAND, or NOR).

-cell_aware none | only

The `none` parameter excludes cell-aware faults from the fault report generated by the `report_faults` command. The `only` parameter includes only cell-aware faults in the fault report. See the Examples section below for more information.

-class *fault_class* | -unsuccessful | -summary | -profile | -all

`-class` - Specifies a specific [fault class](#) to be reported. Multiple uses of `-class` are supported and can also be combined with an `instance_name` or `pin_pathname`.

`-unsuccessful` - Reports all faults for which test generation created a test pattern but for which fault simulation failed to identify the faults as detected.

`-summary` - Selects the fault summary report.

`-profile` - Reports the number of faults detected per pattern in the active pattern set. This option is useful only when the currently selected pattern buffer is external, and those patterns are functional patterns (no scan loads present). It generates a report showing the pattern number and the number of new faults detected, as well as the total cumulative detected faults. The `set_faults -report` option determines whether the fault reporting is in terms of collapsed or uncollapsed faults. Patterns that detect no faults are not printed in the report.

`-all` - Reports all faults in the current fault list.

-collapsed | -uncollapsed

Selects whether the fault report is for collapsed or uncollapsed faults. The default is the current setting as specified with the `set_faults` command.

-dynamic_bridge <r|f|rf>

Specifies the type of dynamic bridging fault being reported. The type can be any of the following:

r - bsr (bridge slow to rise)
f - bsf (bridge slow to fall)
rf -both (bridge slow to rise and bridge slow to fall)
 The default is *rf* (both).

-dominant_node <*first* | *second* | *both*>

Specifies which node in a node pair is used as the dominant node of a dynamic bridging fault: *first*, *second*, or *both*. The default is *both*. Using *first* or *second* will limit the set of reported faults and will only report the faults with the dominant in the specified position. To see how combinations of the `-dynamic_bridge` and `-dominant_node` option settings interact, see the `-dominant_node` option of the [add_faults](#) command. Using default settings, four faults are reported for each node pair defined, if the faults were included in the list.

-external {*file_names*}

Specifies a file name, or a list of file names, containing a list of faults to be reported. External file fault site names are compared with faults already read into memory. This comparison includes the current ATPG run with its associated fault list. If the name of a fault does not match a name in a file, it is added to the virtual total. If a core-level fault list is used, none of the names will match the top-level fault site names because hierarchy levels are added to these fault names. To perform a proper comparison, you must manually add the top-level instance hierarchy to the core-level fault list names in the fault list. If the blocks are instantiated multiple times, you must create and specify multiple external fault list files using the Tcl list syntax.

***instance_name* | *pin_pathname* -stuck** <0|1|01> | **-slow** <*r*|*f*|*rf*>]]

instance_name - Reports all faults associated with the [selected instance](#) name.

pin_pathname - Reports faults associated with the selected [pin pathname](#).

-stuck - Reports only the stuck-at-1 or stuck-at-0 faults, or both stuck-at faults associated with the specified *pin_pathname*. The default is 01 (both).

-slow - Reports slow-to-rise, slow-to-fall, or both transition delay faults for the specified *pin_pathname*.

-level *d m*

Generates a fault report for specified hierarchical levels. The *d* argument specifies the hierarchical depth of the report and the *m* specifies a minimum number of faults required to display a given depth. For example, `-levels 6 32` reports faults from the top level down for a total of six levels, but shows only those hierarchical blocks that had 32 faults or more.

-max *d*

Specifies the maximum number of faults to report. The default is to report all selected faults.

-max_tmgn *float*

This option, which is valid only for small delay defect testing, reports only those faults with a tmgn (timing margin) less than or equal to the *float*. **Note:** This option must be used with an option that specifies the faults to be printed, such as `-all`, `-class fault_class`, or an instance name.

-min_tmgn *float*

This option, which is valid only for small delay defect testing, reports only those faults with a tmgn (timing margin) greater than or equal to the *float*. **Note:** This option must be used with an option that specifies the faults to be printed, such as `-all`, `-class fault_class`, or an instance name.

-max_delta *float*

This option, which is valid only for small delay defect testing, reports only those faults with a delta less than or equal to the *float*. **Note:** This option must be used with an option that specifies the faults to be printed, such as `-all`, `-class fault_class`, or an instance name.

-ndetects <histogram | faults *n*>

The `histogram` keyword displays a tabular histogram showing the number of faults that were detected 1 time, 2 times, up to the *n*-detect value specified in the previous ATPG or fault simulation run.

The `faults` keyword displays the faults that were detected *n* number of times following an ATPG or fault simulation run.

This option is not supported for full-sequential ATPG, fault simulation, or distributed ATPG.

-pattern_id *d*

Reports faults detected by the selected pattern, but no patterns preceding it. To see results from this argument, you need to issue a `run_fault_sim -detected_pattern_storage` command. If you want to use the `-detected_pattern_storage` argument on an internal pattern set, then you can write out the pattern, read it back in, and fault simulate it. For example:

```
...run_atpg -auto
...write_patterns pat.bin -format bin -rep
...reset_state
...set_patterns -external pat.bin
...run_fault_sim -detected_pattern_storage
...report_faults -pattern_id 2
```

-per_clock_domain

Reports all specified faults with extra information for their launch and capture clocks. All clocks are reported, even those classified as "shared" or "multiple" clocks. The format is as follows:

```
<type> <code> <fault_name> (Launch clocks: <clock1> <clock2> ...)
(Capture clocks: <clock1> <clock2> ...)
```

For example:

```
str NC u_ext1/y_reg_3/Q ( Launch clocks: clkext1 ) ( Capture clocks:
TOTO/U8 clkext3)
```

For further details, see [Using Clock Domain-Based Faults](#).

-slack <tmgn [integer | float] | tdet [integer | float] | delta [integer | float] | effectiveness | coverage>

This set of arguments produces various special reports related to slack-based transition fault testing:

The `tmgn` (timing margin) argument produces a histogram-based report of faults based on the minimum-slack data that was read into TestMAX ATPG via the `read_timing` command

and stored in the data structure. If you want to use the `set_delay -max_tmgn` command to select a particular number or percentage of faults with the smallest slacks to target in small delay defect test generation, this report will allow the determination of a reasonable cutoff point for the `-max_tmgn` parameter. The optional parameter can be either an integer or a floating point number. When the number is an integer, it determines the number of categories (or buckets) used in the report. When the number is a floating point (identified by the "." in the number), it determines the slack interval between two consecutive histogram buckets. The default of this parameter is an integer of 10.

The `tdet` (timing of detection) argument produces a histogram-based report of faults based on the slack data for the actual detection paths (detection slacks) in tests generated to detect the faults. This report can be compared with the report produced by the `tmgn` argument to see how close the generated tests came to the longest (minimum-slack) paths. The optional parameter can be either an integer or a floating point number. When the number is an integer, it determines the number of categories (or buckets) used in the report. When the number is a floating point (identified by the "." in the number), it determines the slack interval between two consecutive histogram buckets. The default of this parameter is an integer of 10.

The `delta` argument reports a histogram of faults based on the difference between detection slacks and minimum slacks. This distribution allows a direct view of how close the generated tests came to the longest path. A delta of 0 indicates that the fault was tested along the longest path. The optional parameter can be either an integer or a floating point number. When the number is an integer, it determines the number of categories (or buckets) used in the report. When the number is a floating point (identified by the "." in the number), it determines the slack interval between two consecutive histogram buckets. The default of this parameter is an integer of 10. An example report produced by the `delta` argument is as follows:

```
report_faults -slack delta 0.8
-----
Histogram of Uncollapsed Fault Distribution
      delta
  values      #fault
-----
  0.00 - 0.80: 11794
  0.80 - 1.60: 385
  1.60 - 2.40: 361
  2.40 - 3.20: 273
  3.20 - 4.00: 213
  4.00 - 4.80: 223
  4.80 - 5.60: 390
  5.60 - 6.40: 271
  6.40 - 7.20: 384
      > 7.20: 0
      Infinite: 94
      Undefined: 15692
```

The `effectiveness` argument produces a report of a single delay effectiveness number that describes how close the test set came to achieving tests along the longest (minimum slack) paths for each fault. The number is normalized on a scale from 0 to 100, to resemble familiar fault coverage or test coverage metrics. Unlike the SDQL number, this metric

requires no fabrication process data. A delay effectiveness of 100 indicates that every targeted fault was detected on the longest path. The closer the number is to 100, the more effective the test set has been toward achieving that goal. Note that this option only reports the effectiveness for detected faults of interest.

The `coverage` argument produces a report that measures how well the pattern set detects small delay defects on all detectable faults.

In addition to producing these special reports, the `-slack` option slightly alters the normal fault reporting for transition fault tests. The normal list of faults includes the slack number for the fault at the end of each line in the report. Also, if the fault has been detected or potentially detected, the report includes the delta number for the detection.

For more information on reporting small delay defects, see "[Slack-Based Transition Fault Testing](#)" in the *TestMAX ATPG User Guide*.

-verbose

Displays additional information with each fault. In addition to the standard fault data (stuck-at value, fault class, and pin path name), the library model name and internal fault ID information are also displayed. When used in conjunction with the `-level` option, displays additional columns of fault categories by hierarchy. For a more detailed explanation of the fault hierarchy report see the [Understanding the Faults Report](#) topic. The default is not to display this extra information.

Allowed Command Modes

Test

Description

Use this command to display fault data.

Note: The fault list reported can be a collapsed or uncollapsed list of faults. You can select the type of list by specifying it explicitly in the `report_faults` command or by using the `set_faults` command. The uncollapsed list of faults includes the primary faults as well as faults equivalent to them. Equivalent faults are indicated by a fault class of "--" and immediately follow the fault they are equivalent to.

Examples

```
TEST-T> report_faults -summary
Uncollapsed Fault Summary Report
-----
fault class                code  #faults
-----
Detected                   DT    79912
Possibly detected          PT     325
Undetectable               UD     876
ATPG untestable            AU    3378
Not detected               ND    3917
-----
total faults                88408
test coverage               91.48%
-----
```

```
TEST-T> report_faults -class UR
```

```
sa0 UR mux0/MUX2_UDP_1/A
```

```
sa1 UR mux0/A
```

```
TEST-T> report_faults /reg3 -class AN -class ut
```

```
sa1 UT reg3/lat1/SB
```

```
sa0 AN reg3/lat1/RB
```

```
sa1 UT reg3/r/SB
```

```
TEST-T> report_faults -all -max 10
```

```
sa0 NP INC
```

```
sa0 DI CLK
```

```
sa1 DI CLK
```

```
sa1 DI RSTB
```

```
sa0 DS RSTB
```

```
sa0 DI reg2/MX1/SDI
```

```
sa1 DS mux2/X
```

```
sa0 DS reg2/MX1/D
```

```
sa0 AN mux2/MUX2_UDP_1/A
```

```
sa1 AN mux2/A
```

```
TEST> report_faults -level 4 64
```

```
#faults testcov instance name (type)
```

```
-----
```

88148	91.48%	my_asic	(top_module)
86538	91.65%	my_asic/MX2	(MAIN)
258	2.33%	my_asic/MX2/ntree	(NandTree)
43666	95.38%	my_asic/MX2/CPU	(CPU)
25808	94.92%	my_asic/MX2/CPU/ARIT	(ALU)
17842	96.05%	my_asic/MX2/CPU/TP	(TOP)
38796	94.90%	my_asic/MX2/TGH	(TGH)
9644	94.80%	my_asic/MX2/TGH/ADCTRL	(AGC)
730	95.12%	my_asic/MX2/TGH/IOP	(POY)
11208	96.47%	my_asic/MX2/TGH/PIE	(ABC)
11602	96.38%	my_asic/MX2/TGH/PRT	(WDRE)
5556	88.70%	my_asic/MX2/TGH/PWM	(DRM)
82	0.00%	my_asic/MX2/JTAG_IR	(JTAG_IR3)
254	0.00%	my_asic/MX2/JTAG_TAP_CONTROLLER	(JTAG_TAP)

```
TEST-T> report_faults -level 3 256 -verbose
```

```
#faults #detect #posdet #undet #redund testcov instance name (type)
```

```
-----
```

88148	79679	321	7272	876	91.48%	my_asic	(top_module)
86538	78447	123	7092	876	91.65%	my_asic/MX2	(MAIN)
258	6	0	252	0	2.33%	my_asic/MX2/ntree	(NandTree)
43666	41310	0	2000	356	95.38%	my_asic/MX2/CPU	(CPU)
38796	36333	6	1949	508	94.90%	my_asic/MX2/TGH	(TGH)

```
TEST-T> set_faults -report collapsed
```

```
TEST-T> report_faults -profile
```

```
pattern ID# #faults detected total detected
```

```
-----
```

4	4	4
20	5	9

21	1	10	
22	5		15
27	1		16
28	2		18
31	10		28
32	7		35
33	9	44	
34	7		51
35	1		52

TEST-T> **remove_faults -all**

3027282 faults were removed from the fault list, CPU_time=0.21 sec.

TEST-T> **report_faults -summary**

Uncollapsed Stuck Fault Summary Report

fault class code #faults

Detected DT 0

Possibly detected PT 0

Undetectable UD 0

ATPG untestable AU 0

Not detected ND 0

total faults 0

test coverage 0.00%

TEST-T> **add_faults -all**

1031846 faults were added to fault list, CPU_time=0.94 sec.

TEST-T> **report_faults -summary**

Uncollapsed Stuck Fault Summary Report

fault class code #faults

Detected DT 172984

detected_by_implication DI (172984)

Possibly detected PT 0

Undetectable UD 23110

undetectable-unused UU (46)

undetectable-tied UT (1043)

undetectable-blocked UB (3088)

undetectable-redundant UR (18933)

ATPG untestable AU 194079

atpg_untestable-not_detected AN (194079)

Not detected ND 641673

not-controlled NC (641673)

total faults 1031846

test coverage 17.15%

TEST-T> **add_faults -cell_aware -all**

Warning: Cell aware faults not added in 577 cells due to lost pin in netlist optimization.

1995436 faults were added to fault list, CPU_time=80.73 sec.

TEST-T> **report_faults -summ -cell_aware only**

Uncollapsed Stuck Fault Summary Report

fault class code #faults

Detected DT 30645
detected_by_implication DI (30645)
Possibly detected PT 0
Undetectable UD 9217
undetectable-tied UT (834)
undetectable-blocked UB (2238)
undetectable-redundant UR (6145)
ATPG untestable AU 293570
atpg_untestable-not_detected AN (293570)
Not detected ND 1662004
not-controlled NC (1662004)

total faults 1995436
test coverage 1.54%

TEST-T> **report_faults -summ -cell_aware none**

Uncollapsed Stuck Fault Summary Report

fault class code #faults

Detected DT 172984
detected_by_implication DI (172984)
Possibly detected PT 0
Undetectable UD 23110
undetectable-unused UU (46)
undetectable-tied UT (1043)
undetectable-blocked UB (3088)
undetectable-redundant UR (18933)
ATPG untestable AU 194079
atpg_untestable-not_detected AN (194079)
Not detected ND 641673
not-controlled NC (641673)

total faults 1031846
test coverage 17.15%

TEST-T> **report_faults -summary**

Uncollapsed Stuck Fault Summary Report

fault class code #faults

Detected DT 203629
detected_by_ DI (203629)


```
Possibly detected PT 0
Undetectable UD 32327
undetectable-unused UU (46)
undetectable-tied UT (1877)
undetectable-blocked UB (5326)
undetectable-redundant UR (25078)
ATPG untestable AU 487649
atpg_untestable-not_detected AN (487649)
Not detected ND 2303677
not-controlled NC (2303677)
```

```
-----
total faults 3027282
test coverage 6.80%
-----
```

See Also

[add_faults](#)

[analyze_faults](#)

[remove_faults](#)

[read_faults](#)

[set_faults](#)

[write_faults](#)

[Fault Classes](#)

[Understanding the Faults Report](#)

Table Column Outside Table:

report_feedback_paths

Overview

Use this command to get a report on the feedback paths in the design.

Syntax

```
report_feedback_paths
  <path_id | -summary | -all | -gate <id
    | pin_pathname | instance_name> >
  [-verbose]
```

Arguments

path_id

Specifies the feedback network to report by its specific [loop ID number](#).

-summary

Selects display of a summary of feedback paths showing the number of feedback paths which exists.

-all

Selects all feedback paths for reporting. This displays the loop ID, number of contributing gates, number of sources, and sensitization status.

-gate <id | pin_pathname | instance_name>

Displays the feedback path that contains the specified primitive. You can identify the gate by its [primitive ID](#) number, by a [pin pathname](#) of a pin connected to the gate, or by its [instance name](#). An error is reported if the selected gate is not in a feedback path.

-verbose

Reports verbose data for the selected feedback paths. In addition to the general feedback path information, the verbose data identifies all instances in the feedback path network. The source gates appear at the top of the list, and the remaining gates are rank ordered and should appear in a physical order. The default is not to display this additional information.

Allowed Command Modes

DRC, Test

Description

Use this command to get a report on the feedback paths in the design.

Examples

```
TEST> report_feedback_paths -all
id# #gates #sources sensitization_status
---
0      2      1      pass
1     10      1      pass
```

```
2      5      1      pass
3      8      1      pass

TEST> report_feedback_paths 2 -verbose
id# #gates #sources sensitization_status
-----
2      5      1      pass
    BUF amd2910/register/U70 (731), cell=CMOA02
    INV amd2910/register/sub_23/U11 (232), cell=CMIN20
    NAND amd2910/register/U86 (273), cell=CMND30
    BUF amd2910/register/U70 (274), cell=CMOA02
    NAND amd2910/register/U70/M1 (735), cell=OAI211
```

See Also

[analyze_feedback_path](#)
[Feedback Paths Report](#)

report_instances

Overview

This command reports data associated with a specified instance.

Syntax

```
report_instances instance_name
  [-module name]
  [-netnames]
  [-garble | -ungarble]
```

Arguments

instance_name

Specifies the name of the instance to report. It might be the hierarchical name of an instance or the name of an instance inside a module selected with the `-module` option.

`-module name`

Specifies that the instance to be reported resides inside the specified module.

`-netnames`

Causes net names to be displayed in the report. By default, net names are not displayed.

`-garble | -ungarble`

The `-garble` option displays the garbled instance name to which the given original instance name is mapped in a garbled image created by the `-garble` option of the `write_image` command. This can be done only with an ungarbled design database.

The `-ungarble` option displays the original instance name of the given garbled instance name present in a garbled image created by the `-garble` option of the `write_image` command. This option can be used to translate a garbled name to an original name, but only with an ungarbled design database.

Allowed Command Modes

All

Description

This command reports data associated with a specified instance.

Examples

```
DRC> report_instances U6 -module L572
U6 _MUX
  "UNDEF" I ( H09 )
  "UNDEF" I ( H07 )
  "UNDEF" I ( H08 )
  "UNDEF" O U1/
```

```
DRC> report_instances U6 -module L572 -netnames
U6 _MUX
```

```
"UNDEF" I ( H09 )  
"UNDEF" I ( H07 )  
"UNDEF" I ( H08 )  
"UNDEF" O ( n1 ) U1/
```

See Also[report_primitives](#)[report_nets](#)[report_modules](#)

report_layout

Overview

This command creates a report of the net structure for all nets loaded in TestMAX ATPG.

Syntax

```
report_layout
  identifier
  net_driver
  output_file
  [-bridging_pairs ]
  [-compress gzip | off]
  [-file]
  [-format binary | ascii]
  [-max_reported number]
  [-replace]
  [-subnets]
  [-summary]
```

Arguments

identifier

Specifies one of the following: the instance name from layout, the net name from layout, or the cell type from either the TestMAX ATPG design or the layout. This string must be accompanied by an option that indicates the identifier type.

net_driver

Specifies the name of a net driver for which a report of its structure is printed (the hierarchical pinpath name to a valid pin). This specification must be accompanied by the `-subnets` option.

output_file

Specifies the name of the binary and compressed file containing the net structure information.

-bridging_pairs

Reports all stored bridging pairs. For all net pairs, the report includes the name of the first node and its gate ID. The report also includes a list of all second nodes associated with the first node and its gate ID. An example of the report is provided below:

```
Net pairs of i_addr[1] (9):
de_d/U168/O (3599)
de_d/U172/O1 (3719)
```

This report indicates that two pairs with the same first node are successfully read in. The pairs are:

```
i_addr[1] de_d/U168/O
i_addr[1] de_d/U172/O1
```

The initial bridging pairs includes either a net name or a pinpath name. TestMAX ATPG automatically converts net names into pinpath names, so only the pinpaths are available.

-compress gzip | off

Compresses the subnet data in gzip format.

-file

Writes the subnet data to a file for future reuse.

-format binary | ascii

Specifies the format of the output file containing subnet data. The default is to use binary format.

-max_reported number

Reports a maximum of the specified *number* of nets.

-replace

Overwrites an existing subnet binary file.

-subnets

Reports the net structure of the specified *net_driver* parameter. It is mandatory when the *net_driver* parameter is specified.

-summary

When this is the only option specified, the *report_layout* command reports some characteristics of either the subnet or bridging net pairs loaded. For example:

```
Subnets : #nets=35192, #subnets=1500342, max_subnets=25, memory=1MB
Subnet_distribution: <10 (80.00%) <20 (90.00%) <30 (100.00%)
Receiver_per_net: <10 (90.00%) <20 (90.00%) <30 (100.00%)
```

The first line reports the total number of nets (*#nets=*) which has been loaded. It also reports the total number of subnet defined (*#subnets=*) and the number of subnets defined for the net with maximum subnets (*max_subnets=*). Finally, the memory allocated to the subnet definition is reported (*memory=*).

The line beginning with "Subnet_distribution:" indicates the percentage of nets having less than N subnets defined, where N=10, 20, and so forth. For example: <10 (82.26%) means that 82.26% of the nets have less than 10 subnets defined.

The line beginning with "Receiver_per_net:" indicates the percentage of nets that have less than N receivers, where N=10, 20, and so forth. For example: <10 (79.43%) means that 79.34% of the nets have less than 10 receivers.

If net pairs are loaded, their characteristics are also printed. For instance:

```
Net_pairs : #nets=3650, #pairs=26559, max_pairs=4, memory=1MB
Pairs_distribution: <10 (80.00%) <20 (90.00%) <30 (100.00%)
```

The first line reports the total number of net (*#nets=*) and the total number of pairs (*#pairs=*). A net could be involved in several net pairs. The field "max_pairs=" indicates the maximum number of net constituted of the same net. The memory allocated to the net pairs definition is also reported (*memory=*).

The line beginning with "Pairs_distribution:" indicates the percentage of nets that have less than N pairs defined, where N=10, 20, and so forth. For example: <10 (82.26%) means that 82.26% of the nets have less than 10 pairs defined.

If `-summary` is used with `-subnets`, this option compacts the information for a single net on one line. For instance:

```
report_layout -summary -subnets -max 1
net=mic0/ir0/U6/Z #fanouts=12/12 #subnets=10
```

This line contains the net driver (`net=`), the number of fanout branches for the net driver (`#fanouts=`) and, finally, the number of subnet defined (`#subnets=`).

Description

This command creates a report of the net structure for all nets loaded in TestMAX ATPG. You can use it only if you previously specified the [read_layout subnet_file -subnets](#) command.

Examples

The following example shows how to report the net structure of a particular net driver:

```
TEST-T> report_layout mic0/ir0/U6/Z -subnets
.net
mic0/ir0/U6/Z
mic0/ir0/U9/D, inv_flag=0
mic0/ir0/U10/D, inv_flag=0
mic0/ir0/U11/D, inv_flag=0
mic0/ir0/U12/D, inv_flag=0
.subnets
3 4
2 3
1 4 3
```

Note that the `inv_flag` information indicates if BUF or INV gates are present between the net driver and the fanout branch. If the number of the INV gate is odd, then the flag is equal to 1.

The following command is an example of how to report the net information for any net driver:

```
TEST-T> report_layout -subnets -max_reported 2 -summary
net=mic0/ir0/U6/Z #fanouts=12/12 #subnets=10
net=mic0/ir0/U7/Z #fanouts=14/14 #subnets=10
```

In the previous example, the first number of `#fanouts` indicates the number of direct fanouts and the second number indicates the indirect fanouts. A direct fanout occurs when there is no gate in the path between the net driver and the fanout.

The following example reports subnet data to a gzip-formatted compressed file:

```
TEST-T> report_layout -subnets -file subnet_filename.gz -compress gzip -
format ascii -replace
```

See Also

[read layout](#)
[run diagnosis](#)
[Subnets Format for Diagnosis](#)

report_lbist_seeds

Overview

Use this command to report all current seed definitions and identify seeds that were used during LogicBIST ATPG.

Syntax

```
report_lbist_seeds
```

Arguments

None

Allowed Command Modes

Test

Description

Use this command to report all current seed definitions and identify seeds that were used during LogicBIST ATPG. Seed definitions are specified using the [add_lbist_seeds](#) command. For more information on using TestMAX ATPG with the DFTMAX LogicBIST tool, see the *DFTMAX LogicBIST User Guide*.

Examples

```
TEST-T> report_lbist_seeds
Used Seed Value (binary_normal)
-----
Used  101101010100101010101010101111011
Used  110111101010101010101001010101101
      101101010100101010101010101111011
      101101010100101010101010101111011
      11110100101011101010100100000111
      101010101010101010101010101010101
      1110101011110010101111011111111
      1110101011110010101111011111001
```

See Also

[add_lbist_seeds](#)

[remove_lbist_seeds](#)

report_licenses

Overview

Use this command to report all licenses currently checked out by TestMAX ATPG.

Syntax

```
report_licenses  
    [-quantity]
```

Arguments

-quantity

Reports how many copies of each license are currently checked out.

Allowed Command Modes

All

Description

Use this command to report all licenses currently checked out by TestMAX ATPG.

Examples

```
TEST-T> report_licenses  
Test-Analysis  
Test-Fault-Max  
  
TEST-T> report_licenses -quantity  
Test-Compile  
Test-ATPG-MAX (4)
```

See Also

[get_licenses](#)

[remove_licenses](#)

[Licensing](#)

report_lockup_latches

Overview

The `report_lockup_latches` command performs a topological analysis of scan paths to find missing, extra, or incorrectly clocked lockup latches.

Syntax

```
report_lockup_latches
  [-balanced_gates_file file_name]
  [-buffered_clocking | -external_clocking | -gated_clocking]
  [-intclocks_unbalanced | -nointclocks_unbalanced]
  [-latch_based_balanced]
  [-pipeline]
```

Arguments

-balanced_gates_file *file_name*

Specifies a file containing a list of skew-balanced clock gating sources. This option enables you to avoid debugging irrelevant [M831](#), [M832](#), [M833](#), and [M835](#) messages.

The syntax for the file containing skew-balanced clock gating sources is as follows:

```
balanced_clock_group name pin_pathname1 [pin_pathname2 ...]
```

This file identifies groups of skew-balanced clock gating sources. Each group is specified on separate lines that must begin with the `balanced_clock_group` keyword. The name of the group is specified by the `name` parameter. At least one output pin of a gate must be specified (*pin_pathname1*).

The following example shows the extra information provided by this file in an M832 message:

```
An extra lockup latch is added between scan cells \counter_reg[45](172)
clocked by clk(17) (through U105/Z(23)) LE(45ns) and \counter_reg[46]
(173) clocked by clk(17) (through U106/Z(24)) TE(55ns) (M832)
```

The through point in parentheses varies depending on the options used.

Note the following:

- When this option is used with the `-gated_clocking` option, the syntax is as shown. An ungated clock should be designated (directly) to show that there is no gated through point. Only the last gate in the clock path is reported.
- If the `-latch_based_balanced` option is specified, latch-based clock gates are ignored but other types of clock gates are reported.
- If the `-buffered_clocking` option is specified, the last buffer in the clock path is reported.
- By default, or with the `-external_clocking` option, the parentheses are omitted for external clocks or when the `-nointclocks_unbalanced` option is specified.
- By default, or with the `-intclocks_unbalanced` option, the internal clock source is reported.

-buffered_clocking

Causes each clock tree driven by a separate buffer or inverter to be treated as a separate clock source. This switch corresponds to the TestMAX DFT command `set_scan_configuration -internal_clocks single`. You should not specify `report_lockup_latches -buffered_clocking` after `set_build -portfault_box` because the build option creates extra virtual buffers which will result in unexpected reporting.

-external_clocking

Causes each clock tree driven by a separate external clock to be treated as a separate clock source. This option is the default and corresponds to the TestMAX DFT default setting.

-gated_clocking

Causes each clock tree driven by a separate multi input gate to be treated as a separate clock source. This switch corresponds to the TestMAX DFT command `set_scan_configuration -internal_clocks multi`.

-intclocks_unbalanced | -nointclocks_unbalanced

When the `-external_clocking` option of the `report_lockup_latches` command is specified or used based on its default setting, the `-intclocks_unbalanced` option assumes that each On-Chip Clocking (OCC) clock tree is unbalanced to any other OCC clock tree. This includes any situation when two trees share an external shift clock which requires a lockup latch. This option is the default and corresponds to the TestMAX DFT behavior when it inserts OCC controllers. The `-nointclocks_unbalanced` option interprets the external clocking specification strictly and does not make an exception for OCC clocks.

-latch_based_balanced

When the `-gated_clocking` option is specified, this switch will assume that all clock sub-trees that are separated only by clock gating inserted by Power Compiler are balanced so that they do not need lockup latches between them. These balanced sub-trees are detected by topological analysis, assuming that one of the Power Compiler latch-based styles is used. All other types of multiple-input gates are still treated as separate clock sources. This switch has no effect unless `-gated_clocking` is also specified.

-pipeline

Enables checking for the following areas:

- Within compression pipeline registers
- Between compression pipeline registers and scan chains

By default, these areas are ignored by the `report_lockup_latches` command. Note that the `report_lockup_latches` command cannot check the serializer register.

Command Mode

TEST

Description

The `report_lockup_latches` command performs a topological analysis of scan paths to find missing,

extra, or incorrectly clocked lockup latches. This analysis assumes that the multiplexed flip-flop scan style is used. Other scan styles are not supported by this command.

A missing lockup latch is reported if a lockup latch is missing between two adjacent scan cells belonging to two different clock domains that are both clocked on the leading (LE) or trailing edge (TE).

An extra lockup latch is reported if any of the following conditions apply:

- More than one lockup latch is identified in series.
- A lockup latch is identified between two adjacent scan cells that are driven by different clock edges. Namely, LE to TE and TE to LE.
- A lockup latch is identified between two adjacent scan cells driven by the same clock domain.

An incorrectly clocked lockup latch is reported if any of the following conditions apply:

- A lockup latch between two different scan cells belonging to two different clock domains belongs to a clock domain that is different than either the driving scan cell, or the observing scan cell.
- A lockup latch between two different scan cells belonging to two different clock domains belongs to a clock domain that is the same as the driving scan cell.
- A lockup latch between two different scan cells belonging to two different clock domains belongs to a clock domain that is the same as the observing scan cell, and whose clock is inverted with respect to the observing scan cell.

Examples

```
TEST-T> report_lockup_latches
No lockup latch violations were detected.
```

```
TEST-T> report_lockup_latches -gated_clocking
A lockup latch is missing between scan cells U1/D1_reg(101325) and
U1/D2_reg(101230) (M831)
An extra lockup latch is added between scan cells U2/N1_reg(887) and
U2/N2_reg(908) (M832)
Incorrectly clocked lockup latch detected between scan cells U1/D3_reg
(106829) and U1/D4_reg(101087) (M833)
3 lockup latch violations were detected.
```

Limitations

The following limitations apply to the `report_lockup_latches` command:

- Clock timing is not considered. All clock leading edges are treated as simultaneous, and all clock trailing edges are treated as simultaneous. Clock polarity is considered.
- Clock timing is not considered. All clock leading edges are treated as simultaneous, and all clock trailing edges are treated as simultaneous. Clock polarity is considered.
- Clock timing is not considered. All clock leading edges are treated as simultaneous, and all clock trailing

- edges are treated as simultaneous. Clock polarity is considered.
- This command does not check for incorrectly clocked terminal lockup latches.

Table Column Outside Table:

report_memory

Overview

Use this command to report memory gate data for selected memory gates.

Syntax

```
report_memory  
  <id | instance_name | -all | -summary>  
  [-max d] [-contents <address | all>] [-verbose]
```

Arguments

id | instance_name

Specifies a specific memory primitive for which information is to be displayed. A memory gate might be identified by its [primitive ID](#) or its [instance name](#).

-all

Causes the report to display information on all MEMORY primitives, including all RAMs and ROMs in the design.

-summary

Causes a summary report to be displayed. When this option is chosen, all MEMORY primitives are included in the report.

-max d

Specifies the maximum number of memory gates allowed to be reported. By default, there is no limit.

-contents <address | all>

Specifies that the contents of the selected memories is reported. The contents might be selected to be for a single address or for all addresses. By default, the contents displayed are the initial data values of the RAMs or ROMs. You can use the `-store_memory_contents` option of the `set_simulation` command along with the `run_simulation -sequential` command to determine RAM contents for a particular pattern.

-verbose

Displays a verbose memory gate report.

Allowed Command Modes

DRC, Test

Description

Use this command to report memory gate data for selected memory gates. The `report_primitives` command can also be used to show data for memories.

Examples

```
TEST> report_memory -summary  
RAM summary: #RAMS=4, #clock_unstable=4, #load_unstable=0, #read_only=0.  
ROM summary: #ROMS=1, #hot_read=0, #readoff_X=0, #readoff_0=0, #readoff_
```

1=0.

```
TEST> report_memory -all
type ID instance_path memory_file
-----
RAM 29366 momma/SPI_TRANSMIT/RAM_32X32/u1 none
RAM 29367 momma/lamb/RAM_44X8/u1 none
RAM 29368 momma/unscram/RAM_128X8/u1 none
RAM 29369 momma/spir_in/BISTRAM_16X23/u1 none
ROM 5425 intpe/u1/mem/rom1/rom_core i007.d1
```

```
TEST> report_memory -all -verbose
#ports width address stable off
type ID wrt/rd addr/data min/max clk/load val instance_type
-----
RAM 146 1 1 10 8 0 1023 yes yes X ram1024x8
RAM 149 1 1 9 8 0 499 yes yes X ram512x8
RAM 150 1 1 6 8 0 63 yes yes X ram64x8
ROM 147 0 1 4 16 0 15 yes yes X rom16x16
ROM 148 0 1 5 16 2 31 yes yes X rom32x16
```

```
TEST> report_memory /e037/intpe/u1/mem/rom1/rom_core
type ID instance_path memory_file
-----
ROM 5425 intpe/u1/mem/rom1/rom_core i007.d1
```

```
TEST> report_memory intpe/u1/mem/rom1/rom_core -contents all
type ID instance_path memory_file
-----
ROM 5425 intpe/u1/mem/rom1/rom_core i007.d1
0 : 0000000000000001
1 : 0000000000000010
2 : 0000000000000100
3 : 00000000000001000
4 : 00000000000010000
5 : 00000000000100000
6 : 00000000001000000
7 : 00000000010000000
8 : 00000000100000000
9 : 00000001000000000
a : 00000100000000000
b : 00001000000000000
c : 00010000000000000
d : 00100000000000000
e : 01000000000000000
f : 10000000000000000
10 : 10100000000000001
11 : 11000100000000010
12 : 11100100000000101
13 : 01110000000001011
14 : 00111000000010110
15 : 00011100000101100
16 : 00101110001011000
17 : 0000011110110000
```



```

18   : 0010001111100000
19   : 0000011111000000
1a   : 0010010111100000
1b   : 0000101101110000
1c   : 0011011000111100
1d   : 0010110000011100
1e   : 0111110000001110
1f   : 101100000000111

TEST> set_simulation -store_memory_contents 40
TEST> run_simulation -last 40 -sequential
Begin sequential simulation of 73 external patterns.
Ram contents for all RAM MEMORY gates stored for pattern=40.
Simulation completed: #patterns=41/144, #fail_pats=0(0), #failing_ meas=0
(0)
TEST> report_memory /u5 -contents all
type gate_ID instance_path      memory_file
-----
RAM    150      u5              none
0      : 11010000
1      : 00101111
2      : 11001100
3      : XXXXXXXX
4      : XXXXXXXX
5      : 00000000
6      : XXXXXXXX
7      : 10101000
8      : XXXXXXXX
9      : XXXXXXXX
a      : XXXXXXXX
b      : XXXXXXXX
c      : XXXXXXXX
d      : 00100111
e      : XXXXXXXX
f      : XXXXXXXX

```

See Also

[run_simulation](#)

[set_simulation](#)

[Understanding the Report Memory output](#)

[Memory Gate](#)

[Primitive ID](#)

report_modules

Overview

Use this command to report data associated with a selected netlist module.

Syntax

```
report_modules  
[ name | -unreferenced | -undefined | -unspecified | -udp  
-behavioral | -black_box | -errors | -summary | -all]  
[-garble | -ungarble]  
[-verbose]
```

Arguments

name

Specifies the name of the module to report.

-unreferenced

Reports all modules that are not referenced by another module. The last unreferenced module is the default top module choice for the `run_build_model` command.

-undefined

Reports all modules that are referenced but not defined.

-unspecified

Reports all modules that were defined with sequential syntax, usually by UDP table, but the actual description resulted in a combinational model as output instead of a sequential model.

-udp

Reports all modules identified as Verilog UDP definitions.

-behavioral

Reports all modules considered to be behavioral.

-black_box

Reports all black box modules (modules that have port definitions but no contents).

-errors

Reports all modules that have at least one violation of a rule with a severity of "error."

-summary

Generates a summary report on all modules.

-all

Generates a report on all known modules.

-garble | -ungarble

The `-garble` option displays the garbled module name to which that the given original module name is mapped in a garbled image created by the `-garble` option of the `write_image` command. This can be done only with an ungarbled design database.

-verbose

Generates verbose information for each selected module, including a list of input pins, output pins, and all instances within the module. For each instance, the report includes the [instance name](#), module referenced, and pin to pin connectivity. The default is nonverbose, which displays a single line per module showing the [module name](#), number of pins, number of internal ATPG modeling gates used to model the module, and number of times the module is referenced.

Allowed Command Modes

All

Description

Use this command to report data associated with a selected netlist module.

Examples

```
DRC> report_modules -summary
Modules: #STRUCT_VERILOG=423 #BEH_VERILOG=3 #COMB_UDP=4(#xdetector=4)
#SEQ_UDP=8
```

```
BUILD> report_modules -undefined
pins
module name      tot( i/ o/ io)  inst refs(def'd)  used
-----
iopad3            0( 0/ 0/ 0)    0   36  (N)         0
bsd6rclr          0( 0/ 0/ 0)    0   32  (N)         0
iobuf2            0( 0/ 0/ 0)    0   13  (N)         0
cr23              0( 0/ 0/ 0)    0    4  (N)         0
schmoo_flop       0( 0/ 0/ 0)    0    1  (N)         0
dualport256x3     0( 0/ 0/ 0)    0    1  (N)         0
ramdac            0( 0/ 0/ 0)    0    3  (N)         0
dll               0( 0/ 0/ 0)    0    1  (N)         0
-----
---
```

```
DRC> report_modules -behavioral
pins
module name      tot( i/ o/ io)  inst refs(def'd)  used
-----
J14D             19( 15/ 4/ 0)  46    0  (Y)         0
J14F             21( 17/ 4/ 0)  51    0  (Y)         0
J14H             23( 19/ 4/ 0)  56    0  (Y)         0
-----
---
```

```
DRC> report_modules -all
pins
```

```

module name      tot( i/ o/ io)  inst refs(def'd) used
-----
i22              12( 6/ 2/ 4)    19  0  (Y)    1
MUX2H            4( 3/ 1/ 0)     1  4  (Y)    4
DFFRLP           7( 5/ 2/ 0)     5  4  (Y)    4
DFFP             4( 2/ 2/ 0)     4  1  (Y)    1
INC4             11( 5/ 6/ 0)    10  1  (Y)    1
OR2H             3( 2/ 1/ 0)     1  1  (Y)    1
BON4T            4( 2/ 1/ 1)     3  4  (Y)    4
BICN             2( 1/ 1/ 0)     1  4  (Y)    4
SCANINP_UDP_1    4( 3/ 1/ 0)     4  1  (Y)    4
P_LAT_RS         5( 4/ 1/ 0)     1  2  (Y)    5
N_LATCH          6( 5/ 1/ 0)     1  2  (Y)    5
MUX2_UDP_1       4( 3/ 1/ 0)     4  1  (Y)    4

```

```
DRC> report_modules L572 -verbose
```

```

pins
module name      tot( i/ o/ io)  inst refs(def'd) used
-----
L572             14( 10/ 4/ 0)    9  0  (Y)    0
  Inputs : H01 ( ) H02 ( ) H03 ( ) H04 ( ) H05 ( ) H06 ( ) H07 ( )
          : H08 ( ) H09 ( ) H10 ( )
  Outputs: N01 ( ) N02 ( ) N03 ( ) N04 ( )
  U1      : _NAND conn=( I:n1 I:n2 O:N04 )
  U2      : _NAND conn=( I:n3 I:n2 O:N03 )
  U3      : _NAND conn=( I:n4 I:n2 O:N02 )
  U4      : _NAND conn=( I:n5 I:n2 O:N01 )
  U5      : _INV  conn=( I:H10 O:n2 )
  U6      : _MUX  conn=( I:H09 I:H07 I:H08 O:n1 )
  U7      : _MUX  conn=( I:H09 I:H05 I:H06 O:n3 )
  U8      : _MUX  conn=( I:H09 I:H03 I:H04 O:n4 )
  U9      : _MUX  conn=( I:H09 I:H01 I:H02 O:n5 )

```

```
DRC> report_modules -unreferenced
```

```

pins
module name      tot( i/ o/ io)  inst refs(def'd) used
-----
F422NFP          5( 4/ 1/ 0)     4  0  (Y)    0
F605NP           5( 3/ 2/ 0)     7  0  (Y)    0
F605NQ           4( 3/ 1/ 0)     6  0  (Y)    0
L572             14( 10/ 4/ 0)    9  0  (Y)    0
F602NQP          4( 3/ 1/ 0)     7  0  (Y)    0

```

```
DRC> report_modules WITHRAM -verbose
```

```
pins
```

```

module name      tot( i/ o/ io)  inst refs(def'd) used
-----
WITHRAM          65( 41/ 24/ 0)   1   0   (Y)      0
  Inputs         : set ( ) reset ( ) w1 ( ) a1[3] ( ) a1[2] ( ) a1[1] ( )
                  : a1[0] ( ) d1[7] ( ) d1[6] ( ) d1[5] ( ) d1[4] ( ) d1[3] ( )
                  : d1[2] ( ) d1[1] ( ) d1[0] ( ) w2 ( ) c2 ( ) a2[3] ( )
                  : a2[2] ( ) a2[1] ( ) a2[0] ( ) d2[7] ( ) d2[6] ( ) d2[5] ( )
                  : d2[4] ( ) d2[3] ( ) d2[2] ( ) d2[1] ( ) d2[0] ( ) r3 ( )
                  : a3[3] ( ) a3[2] ( ) a3[1] ( ) a3[0] ( ) r4 ( ) a4[3] ( )
                  : a4[2] ( ) a4[1] ( ) a4[0] ( ) r5 ( ) c5 ( )
  Outputs        : d3[7] ( ) d3[6] ( ) d3[5] ( ) d3[4] ( ) d3[3] ( ) d3[2] ( )
                  : d3[1] ( ) d3[0] ( ) d4[7] ( ) d4[6] ( ) d4[5] ( ) d4[4] ( )
                  : d4[3] ( ) d4[2] ( ) d4[1] ( ) d4[0] ( ) d5[7] ( ) d5[6] ( )
                  : d5[5] ( ) d5[4] ( ) d5[3] ( ) d5[2] ( ) d5[1] ( ) d5[0] ( )
  mymem          : _MEM (S R wr=2 rd=3) addrlines=4 datalines=8
initfile=d:file.gz address[0:15] read_off=LO write_trigger=E
  conn=( R.I:reset S.I:set wr.I:w1 a.I:a1[3] a.I:a1[2] a.I:a1
[1]
      a.I:a1[0] d.I:d1[7] d.I:d1[6] d.I:d1[5] d.I:d1[4] d.I:d1[3]

      d.I:d1[2] d.I:d1[1] d.I:d1[0] wr.I:!w2 C.I:!c2 a.I:a2[3]
      a.I:a2[2] a.I:a2[1] a.I:a2[0] d.I:d2[7] d.I:d2[6] d.I:d2[5]

      d.I:d2[4] d.I:d2[3] d.I:d2[2] d.I:d2[1] d.I:d2[0] rd.L.I:r3
      a.I:a3[3] a.I:a3[2] a.I:a3[1] a.I:a3[0] d.O:d3[7] d.O:d3[6]

      d.O:d3[5] d.O:d3[4] d.O:d3[3] d.O:d3[2] d.O:d3[1] d.O:d3[0]

      rd.hL.I:!r4 a.I:a4[3] a.I:a4[2] a.I:a4[1] a.I:a4[0] d.O:d4
[7]
      d.O:d4[6] d.O:d4[5] d.O:d4[4] d.O:d4[3] d.O:d4[2] d.O:d4[1]

      d.O:d4[0] rd.hE.I:!r5 C.I:c5 a.I:a4[3] a.I:a4[2] a.I:a4[1]
      a.I:a4[0] d.O:d5[7] d.O:d5[6] d.O:d5[5] d.O:d5[4] d.O:d5[3]

      d.O:d5[2] d.O:d5[1] d.O:d5[0] )
-----
---
```

See Also[report_primitives](#)[report_instances](#)[report_nets](#)

report_net_connections

Overview

Use this command to report all net connections added with the `add_net_connections` command.

Syntax

```
report_net_connections
```

Arguments

None

Allowed Command Modes

All

Description

Use this command to report all net connections added with the `add_net_connections` command.

Examples

```
BUILD> add_net_connections PI gwx12z/amd/dp/uto_reg_0/D -disconnect
BUILD> add_net_connections PO gwx12z/amd/dp/U943/Z
BUILD> report_net_connections
  Connection PI, net gwx12z/amd/dp/uto_reg_0/D, disconnect
  Connection PO, net gwx12z/amd/dp/U943/Z
```

See Also

[report_primitives](#)

[report_instances](#)

[report_modules](#)

[report_nets](#)

report_nets

Overview

This command reports the net name and pin pathnames connected to the net or subnet.

Syntax

```
report_nets
  net_name
  [-candidates <all | candidate_id>]
  [-garble | -ungarble]
  [-subnet subnet_id]
```

Arguments

net_name

Specifies the net to report. This can be a pin pathname on the net, the net name, or the primitive ID driving the net.

-candidates <all | candidate_id>

Reports detailed net connectivity information for all candidates or just a specific candidate, as part of class-based diagnostics reporting. This information can also be reported using the `-verbose` option of the [set_diagnosis](#) command. Note that this option is not supported for the default fault-based diagnostics reporting. For more information on creating a class-based diagnostics report, see the "[Class-Based Diagnosis Reporting](#)" section in the *TestMAX ATPG User Guide*.

-garble | -ungarble

The `-garble` option displays the garbled net name to which that the given original net name is mapped in a garbled image created by the `-garble` option of the `write_image` command. This can be done only with an ungarbled design database.

-subnet subnet_id

Reports only the receiver pin pathnames that are connected to the physical subnet ID of the specified net, and the driver pin pathname of the net. To use the candidate subnets IDs with this option, you must enable the physical subnet IDs by specifying the [set_diagnosis](#) `-show physical_subnet_id` command prior to the [run_diagnosis](#) command.

Allowed Command Modes

DRC

TEST

Description

This command reports the net name and pin pathnames connected to the specified net, subnet, or fault candidates.

Examples

```
TEST-T> report_nets I_RISC_CORE/I_ALU/U14/ZN -subnet 20
Net connections:
-----
I_RISC_CORE/I_ALU/n16 (218)
O I_RISC_CORE/I_ALU/U14/ZN
:
I I_RISC_CORE/I_ALU/Lachd_Result_reg_11__U3/S
I I_RISC_CORE/I_ALU/Lachd_Result_reg_12__U3/S
I I_RISC_CORE/I_ALU/Lachd_Result_reg_13__U3/S
:
-----
```

See Also

[report_primitives](#)

[report_physical](#)

report_nofaults

Overview

Use this command to report faults that have the nofault attribute.

Syntax

```
report_nofaults  
  <instance_name | pin_pathname [-stuck<0|1|01>]  
  | -summary | -all> [-max d]
```

Arguments

instance_name

Reports all faults with the nofault attribute that are associated with the specified [instance name](#).

pin_pathname [-stuck<0|1|01>]

Reports all faults with the nofault attribute that are associated with the specified [pin pathname](#). You can use the `-stuck` option to report only the stuck-at-1 or stuck-at-0 fault associated with the specified pin. The default is to report both.

-summary

Generates a summary report of nofault counts. This option can also be used in conjunction with an instance pathname to report the number of nofaults associated with that instance.

-all

Reports of all nofault locations.

-max d

Specifies the maximum number of faults with the nofault attribute that are allowed to be reported. By default, there is no limit.

Allowed Command Modes

DRC, Test

Description

Use this command to report faults that have the nofault attribute. Nofault attributes can exist because they were defined in the netlist, added with the `add_nofaults` command, or added with the `add_equivalent_nofaults` commands.

Examples

```
TEST> add_nofaults -module MUX2H -stuck 1 SL  
  4 faults were added to nofault list.  
TEST> report_nofaults -all  
sal ** mux0/SL  
sal ** mux1/SL  
sal ** mux2/SL  
sal ** mux3/SL
```

```
TEST> report_nofaults mux2
sal ** mux2/SL
```

See Also[add_equivalent_nofaults](#)[add_nofaults](#)[remove_nofaults](#)[read_nofaults](#)[Nofaults Report](#)

report_pi_constraints

Overview

This command reports all primary input and bidirectional ports that have been constrained either explicitly with the `add_pi_constraints` command or implicitly by the test protocol file.

Syntax

```
report_pi_constraints  
  [-command_report]
```

Arguments

`-command_report`

Causes the PI constraints to be reported in the form of `add_pi_constraints` commands that would be necessary to define the PI constraints.

Allowed Command Modes

DRC Test

Description

This command reports all primary input and bidirectional ports that have been constrained either explicitly with the `add_pi_constraints` command or implicitly by the test protocol file.

Examples

```
DRC> report_pi_constraints  
port_name          constrain_value  
-----  
/TEST_MODE         1
```

See Also

[PI Constraints Report](#)
[add_pi_constraints](#)
[remove_pi_constraints](#)

report_nonscan_cells

Overview

Use this command to report behavioral data on nonscan flip-flops and latches.

Syntax

```
report_nonscan_cells
  <-summary | -all | c0 | c1 | cu | l0 | l1
  | tla | le | te | ls | ram_out | unstable_set_resets | load | nonx_load>
  [-max d] [-tlas [no_clock | hot_clock | x_clock]]
  [-unique] [-verbose]
```

Arguments

-summary

Generates a summary report on all nonscan cells.

-all

Reports all nonscan cells.

```
c0 | c1 | cu | l0 | l1 | tla | le | te | ls | ram_out | unstable_set_resets
| load | nonx_load
```

Reports nonscan cells with specific behavior characteristics. The choices are as follows:

<u>Nonscan Cell Type</u>	<u>Description</u>
c0	Constant 0
c1	Constant 1
cu	Clock unstable
l0	Always loads 0
l1	Always loads 1
tla	Transparent latches
le	Leading edge clock behavior
te	Trailing edge clock behavior
ls	Level-sensitive behavior
ram_out	Macro Out bit of a memory cell
unstable_set_resets	Unstable set/resets
load	Loadable nonscan cells
nonx_load	Loadable nonscan cells which always have a non-X value (that is, 0 or 1) during the random pattern-based analysis performed when the set_drc -load_nonscan_cells command is specified.

-max d

Limits the number of nonscan cells that can be reported. By default, there is no limit.

-tlas [no_clock | hot_clock | x_clock]

Reports transparent latches (TLAs) of the specified type.

-unique

Prints the same report as the `-all` option, but filters out the reporting of latches that are also listed in the scan cell report. This option can be used together with the `-max` and `-verbose` options, but is incompatible with any other `report_nonscan_cells` command options.

-verbose

Displays verbose data. For a summary report, this includes information about load disturbances and TLA behavior. For nonsummary reports, this includes the primitive report for all nonscan cells.

Allowed Command Modes

Test

Description

Use this command to report behavioral data on nonscan flip-flops and latches.

Examples

```
TEST> report_nonscan_cells -summary
Nonscan cell summary: #DFF=416 #DLAT=227 tla_usage_type=no_clock_tla
Nonscan behavior: #CU=64 #TLA=131 #LE=407 #TE=9 #LS=32
Load disturbs : #CU=64 #TLA=131 #LE=407 #TE=9 #LS=32
TLA behavior: #no_clock=131, #hot_clock=0, #X_clock=0
```

```
TEST> report_nonscan_cells cu -max 10
type      behavior_data id      instance_name (type)
-----
CU        load_unstable 9286   core/fifo/reg_42 (LD1)
CU        load_unstable 9287   core/fifo/reg_61 (LD1)
CU        load_unstable 9288   core/fifo/reg_52 (LD1)
CU        load_unstable 9289   core/fifo/reg_6 (LD1)
CU        load_unstable 9291   core/fifo/reg_46 (LD1)
CU        load_unstable 9296   core/fifo/reg_2 (LD1)
CU        load_unstable 9297   core/fifo/reg_56 (LD1)
CU        load_unstable 9298   core/fifo/reg_54 (LD1)
CU        load_unstable 9302   core/fifo/reg_0 (LD1)
CU        load_unstable 9305   core/fifo/reg_9 (LD1)
```

```
TEST> report_nonscan_cells tla -max 5
type      behavior_data id      instance_name (type)
-----
TLA       no_clock_tla 89351   core/ge/rd_reg_46 (LD1)
TLA       no_clock_tla 89352   core/ge/rd_reg_2 (LD1)
TLA       no_clock_tla 89353   core/ge/rd_reg_42 (LD1)
TLA       no_clock_tla 89354   core/ge/rd_reg_61 (LD1)
TLA       no_clock_tla 89355   core/ge/rd_reg_52 (LD1)
```

```
TEST> report_nonscan_cells le -max 5
type      behavior_data id      instance_name (type)
-----
LE        load_unstable 90102   core/amd/lutindd_reg_13 (FD1Q)
LE        load_unstable 90103   core/amd/lutindd_reg_20 (FD1Q)
LE        load_unstable 90104   core/amd/cursout_reg_23 (FD1QP)
LE        load_unstable 90105   core/amd/canddd_reg (FD1Q)
LE        load_unstable 90106   core/amd/cursout_reg_10 (FD1Q)
```

```
TEST> report_nonscan_cells te -max 4
type      behavior_data id      instance_name (type)
-----
TE        load_unstable 91115   core/pll/clk_mux_reg (FD1)
TE        load_unstable 95916   non/gv_clk_en_reg (FD2_SYNC)
TE        load_unstable 95919   non/clk_mux/timer_reg4 (FD2_SYNC)
TE        load_unstable 95920   non/clk_mux/timer_reg2 (FD2_SYNC)
```

```
TEST> report_nonscan_cells ls -max 4
type      behavior_data id      instance_name (type)
-----
LS        load_unstable 89327   amd/dp/uto_reg9 (LD1P)
LS        load_unstable 89328   amd/dp/uto_reg21 (LD1)
LS        load_unstable 89329   amd/dp/uto_reg12 (LD1)
LS        load_unstable 89330   amd/dp/uto_reg0 (LD1)
```

See Also

[report_scan_cells](#)

[report_primitives](#)

[report_instances](#)

[Nonscan Cells Definitions](#)

[Understanding the Nonscan Cells Report](#)

[Using Loadable Nonscan Cells in TestMAX ATPG](#)

report_patterns

Overview

Use this command to report pattern data from the internal or external pattern buffer.

Syntax

```
report_patterns
  [-chain name]
  [-chain_mapping]
  [-clocking]
  [-clk_summary]
  [-cycles <pat_num | {pat1_num pat_num2} | -all>]
  [-internal | -external]
  [-path_delay]
  [-slack]
  [{start_pattern [stop_pattern]} | -all]
  [-summary [{first_pattern last_pattern}]]
  [-types]
```

Arguments

-chain name

Limits the displayed pattern load/unload data to the specified scan chain only.

-chain_mapping

Reports the internal scan chain names observed on each scan out port for any patterns using the 1:1 mode or N:1 X-tolerant unload modes. If different unload modes are used within a pattern, the range of shift cycles is reported in parenthesis after the pattern number for each unload mode used. Any pattern or range of shift cycles in full observability mode (all chains observed) are denoted with a * for the chain name. Any padding patterns are denoted with a - for the chain name because no chains are observed in a padding pattern. This option is supported for both DFTMAX patterns and DFTMAX Ultra patterns.

-clocking

Reports only the internal clocks used for each pattern. This report is meaningful only when On-Chip Clocking (OCC) is used. Clocks controlled from the primary inputs are found in the `report_patterns -type` results. The only other way to get a report of all internal and external clocks is the default `report_patterns` listing, but it can be extremely verbose. For more information on On-Chip Clocking, see [On-Chip Clocking Support](#) in the *TestMAX ATPG User Guide*.

-clk_summary

Used as part of the internal clocking procedures feature, this option provides a summary of the number of clocking procedures of each type that was used in a particular pattern set. For more information, see "[Using Internal Clocking Procedures](#)."

-cycles <pat_num | {pat_num1 pat_num2} | -all>

Reports the cycle number at the start of each pattern within the specified range, and the cycle number at the start of the shift cycle for each pattern. The cycle count is the total number of executed vectors; loops and shift operations are expanded or flattened into the vectors generated by those operations. The cycle count starts at 1 (the first vector is counted as cycle 1). Data is reported only for external pattern files in the STIL or WGL format; internal patterns and binary patterns do not report cycle data. You must specify a pattern range using a number (*pat_num*), a pair of numbers (*{pat_num1 pat_num2}*), or the **-all** argument with the **-cycles** option. When you specify this option, the output data for the **report_patterns** command contains only pattern and cycle counts. This command does not apply to DFTMAX Ultra patterns.

The following example shows the typical output produced by the **-cycles -all** option:

```
report_patterns -cycles -all
Pattern Cycle Shift-start cycle
-----
0 3 4
1 106 107
2      209 210
...
```

The first field is the pattern number, the second field is the cycle number associated with the vector that uses the referenced pattern, and the third field is the cycle number of the shift cycle starting point within the pattern.

Note that when you specify the **-all** argument, the last pattern (which is usually an unload pattern for the previous pattern) and the total number of cycles are also reported, as shown in the following example:

```
...
65(unload) 6801 6802
Last cycle is 6902
```

-internal | -external

Indicates whether the pattern to report is in the internal or external pattern set. The default is the internal pattern set.

-path_delay

Valid when the fault model has been set to **path_delay**. Specifies that the report include information about path delay faults detected by each pattern.

start_pattern

Specifies the starting pattern to be reported. The pattern number is a zero-based number and is selected from either the internal or external pattern set.

stop_pattern

Specifies the ending pattern to be reported. This value is optional. If it is not specified, only a single pattern is reported.

-all

Specifies that all patterns are to be reported.

-summary [{first_pattern last_pattern}]

Selects a summary report for the internal pattern set. If a current external pattern set exists, its summary is also printed. You can specify a pattern range, if needed.

-slack

Valid when the fault model has been set to `path_delay`. Specifies that the report include information about the range of delay path slack values for path delay faults detected by each pattern. Note that if any path delay faults are missing a `$slack` specification, their slack value is assumed to be 0.

-types

Specifies that a listing of pattern number vs. pattern type is reported for the selected patterns.

Allowed Command Modes

Test

Description

Use this command to report pattern data from the internal or external pattern buffer.

Examples

```
TEST> report_patterns -summary
      Pattern Summary Report
```

```
-----
#internal patterns          64
  #basic_scan patterns     36
  #fast_sequential patterns 28
-----
```

```
TEST> report_patterns {3 4}
Pattern 3 (basic_scan)
Time 0: load c1 = 0111
Time 1: force_all_pis = 0001001011 001101111
Time 2: measure_all_pos = 001111
Time 3: unload c1 = 0111
Pattern 4 (basic_scan)
Time 0: load c1 = 0101
Time 1: force_all_pis = 0000111011 110010111
Time 2: measure_all_pos = 000111
Time 3: unload c1 = 0101
```

```
TEST> report_patterns 6
Pattern 6 (fast_sequential)
Time 0: load c1 = 0111
Time 1: force_all_pis = 0000000000 00000ZZZZ
Time 2: pulse clocks ck2 (1)
Time 3: force_all_pis = 0000100100 00000ZZZZ
Time 4: measure_all_pos = 00ZZZZ
Time 5: pulse clocks ck1 (0)
Time 6: unload c1 = 0000
```

```
report_patterns -type -all
```

Patn num	Pattern type	Cycle count	Load count	Observe proc	Clocks used
-----	-----	-----	-----	-----	-----
0	basic_scan	1/0	1	-	-
1	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
2	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
3	basic_scan	1/1	1	-	sdr_clk,prst_n
4	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
5	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
6	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
7	basic_scan	1/1	1	-	sdr_clk,prst_n
8	basic_scan	1/1	1	-	sys_clk,pclk
9	basic_scan	1/1	1	-	prst_n
10	basic_scan	1/1	1	-	prst_n
11	basic_scan	1/1	1	-	prst_n
12	basic_scan	1/1	1	-	prst_n
13	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
14	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
15	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
16	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk
17	basic_scan	1/1	1	-	sdr_clk,sys_clk,pclk

In the previous example, the cycle count column lists the number of tester cycles (first number) and the number of cycles in which there is a clock pulse (second number).

```
TEST> report_patterns -path_delay
```

```
Pattern Fault Fault Fault Launch Capture
```

num	name	type	class	clock	clock
---	-----	-----	-----	-----	-----
0	TxC1k_1	stf	DR	TxC1k	TxC1k
0	TxC1k_9	stf	DS	TxC1k	TxC1k
0	TxC1k_22	str	DS	TxC1k	TxC1k
1	TxC1k_4	str	DR	TxC1k	TxC1k
1	TxC1k_17	stf	DS	TxC1k	TxC1k
:					
:					

```
TEST> report_patterns -slack
```

Pattern num	Min slack	Max slack
-----	-----	-----
0	0.51	0.56
1	-1.20	0.00
2	0.19	2.08
3	1.27	1.27
4	0.00	0.00
:		
:		

```
TEST> report patterns -chain_mapping
pattern port      chain
```

```

0 test_so2 -----
0 test_so3 -----
0 test_so4 -----
1 test_so2 1
1 test_so3 12
1 test_so4 14

```

```

TEST> report patterns {26 29} -clocking
Clocking only:
Pattern 26 (fast_sequential-parallel_clocking)
Pattern 27 (fast_sequential-parallel_clocking)
Pattern 28 (fast_sequential-parallel_clocking)
# PLL internal clock pulse: capture_cycle=0, node=TOTO0/U2 (654) # PLL
internal clock pulse: capture_cycle=1, node=TOTO0/U2 (654) Pattern 29 (fast_
sequential-parallel_clocking)
# PLL internal clock pulse: capture_cycle=0, node=TOTO3/U2 (793) # PLL
internal clock pulse: capture_cycle=1, node=TOTO3/U2 (793)

```

```

TEST> report_patterns -all -clk_summary

```

```

      Pattern Clocking Constraints Summary Report
-----
#Used Clocking Procedures
      #U0to1 6
      #U1to0 5
-----

```

See Also

[write_patterns](#)

[set_patterns](#)

[Patterns Report](#)

report_physical

Overview

This command prints a report of physical locations based on the specified options.

Syntax

```
report_physical
  [-bridge {net1 net2}]
  [-candidates <all | candidate_id>]
  [-instance instance_name]
  [-net net_name]
  [-pin pin_pathname]
  [-subnet {net_name subnet_id}]
  [-technology]
```

Arguments

-bridge {net1 net2}

Reports the bridgeable areas between the two specified nets. Bridgeable areas are defined by the default layer tolerance or by the `-tolerance` option of the [set_physical](#) command.

-candidates <all | candidate_id>

Reports detailed information on physical locations used for class-based diagnostics reporting. This information can also be reported using the [set_diagnosis](#) `-verbose` command. Note that this option is not supported for fault-based diagnostics reporting. For more information on creating a class-based diagnostics report, see the "[Class-Based Diagnosis Reporting](#)" section in the *TestMAX ATPG User Guide*.

-instance instance_name

Reports the cell bounding box for the specified instance and the geometries for all pins connected to the instance.

-net net_name

Reports the geometries for the specified net and for all pins connected to the net.

-pin pin_pathname

Reports the geometries for the specified pin, the bounding box of the connected cell, and either the geometries of the connected subnet if the fanout is greater than 1 or the connected net if there is only one fanout.

-subnet {net_name subnet_id}

Reports only the net geometries for the specified subnet of the specified net. To use the candidate subnets IDs with this option, you must enable the physical subnet IDs by specifying the [set_diagnosis](#) `-show physical_subnet_id` command prior to the [run_diagnosis](#) command.

-technology

Reports technology library data, such as defined metal layers.

Allowed Command Modes

DRC

TEST

Description

This command prints a report of physical locations based on the specified options. It reports only the physical details previously extracted by the `run_diagnosis`, `extract_nets`, or `extract_pins` commands. Otherwise, an [M967](#) error message is issued. For more information about this command, see ["Using Physical Data for Diagnosis."](#)

Examples

```
TEST-T> report_physical -subnet {I_RISC_CORE/I_ALU/U14/ZN 20}
```

```
Net: I_RISC_CORE/I_ALU/n16 Net ID: 218
```

```
Subnet: 20
```

```

METAL2 (796240 626030) (796440 626350)
METAL2 (797880 622400) (798080 625880)
METAL2 (797880 625620) (798080 625940)
METAL3 (796195 626090) (796485 626290)
METAL3 (796240 625680) (796440 626290)
METAL3 (796240 625680) (798080 625880)
METAL3 (797835 625680) (798125 625880)
VIA23 (796340 626190)
VIA23 (797980 625780)
```

```
TEST-T> report_physical -technology
```

```
-----
Reporting technology library data ...
-----
```

```
Layer: NWEELL
```

```
...
```

```
Layer: METAL
```

```
Layer: VIA
```

```
Layer: METAL2
```

```
Layer: VIA2
```

```
Layer: METAL3
```

```
Layer: VIA3
```

```
Layer: METAL4
```

```
Layer: VIA4
```

```
Layer: METAL5
```

```
Layer: VIA5
```

```
Layer: METAL6
-----
```

```
TEST-T> report_physical -candidates all
```

```
Candidate 1:
```

```
Physical details:
```

```
-----
```

```
~ METAL (872160 719050) (872485 719210) LR
```

```
~ METAL (872160 719210) (872485 719255) LR
```

```
~ METAL (872160 719210) (872485 719255) CC
~ METAL (872160 719255) (872485 719415) LR
~ METAL (872280 719120) (872485 719255) LR
~ METAL (872280 719255) (872485 719415) LR
~ METAL (872280 719255) (872485 719485) CC
~ METAL (872280 719415) (872485 719485) CC
~ METAL (872295 717725) (872485 717865) CC
~ METAL (872295 717725) (872485 718090) CC
~ METAL2 (872290 719160) (872500 719360) LR
~ METAL2 (872290 719160) (872500 719770) LR
-----
```

See Also[set_physical_db](#)[extract_nets](#)[extract_pins](#)[report_nets](#)[run_diagnosis](#)

report_pi_equivalences

Overview

This command reports all primary input and bidirectional ports that have been constrained to be equivalent with the `add_pi_equivalences` command.

Syntax

```
report_pi_equivalences  
  [-command_report]
```

Arguments

`-command_report`

Causes the PI equivalences to be reported in the form of `add_pi_equivalences` commands that would be necessary to define the PI equivalences.

Allowed Command Modes

DRC, Test

Description

This command reports all primary input and bidirectional ports that have been constrained to be equivalent with the `add_pi_equivalences` command.

Examples

```
DRC> add_pi_equivalences NL0 NL1 NL2  
DRC> add_pi_equivalences ENA_P -inv ENA_N  
DRC> report_pi_equivalences  
port_name          equivalent pins  
-----  
NL0                 NL1 NL2  
ENA_P               ~ENA_N
```

See Also

[PI Equivalences Report](#)
[add_pi_equivalences](#)
[remove_pi_equivalences](#)

report_pin_data

Overview

Use this command to report the current pin data for a selected pin pathname.

Syntax

```
report_pin_data  
  pin_pathname
```

Arguments

pin_pathname

Specifies the pin of an instance.

Allowed Command Modes

DRC, Test

Description

Use this command to report the current pin data for a selected pin pathname. A single line is reported that contains all the characters of the pindata for the selected pin pathname.

Examples

Given the following instance:

```
sdr_clk_iopad  
  input: sdr_clk  
  output: net_sdram_clk
```

With Cell type:

```
pc3d01  
input port PAD  
output port CIN
```

```
TEST> report_pin_data sdr_clk_iopad/CIN  
010
```

```
TEST> report_pin_data sdr_clk_iopad/PAD  
010
```

report_pin_name

Overview

Use this command to report the pin pathnames of all nodes in the traceback cone of a selected input of a gate at the boundary of the library cell.

Syntax

```
report_pin_name  
  gate_id input_pin_id
```

Arguments

gate_id

Specifies the gate at a library cell boundary where traceback begins.

input_pin_id

Specifies the input pin of *gate_id*.

Allowed Command Modes

DRC

Description

Use this command to report the pin pathnames of all nodes in the traceback cone of a selected input of a gate at the boundary of the library cell. A line is reported for each node in the traced cone and contains its full pathname and an indicator of the connection type: noninverted, inverted, or complex.

report_po_masks

Overview

This command reports all primary outputs that have been constrained to be masked with the `add_po_masks` command.

Syntax

```
report_po_masks
```

Arguments

None

Allowed Command Modes

DRC, Test

Description

This command reports all primary outputs that have been constrained to be masked with the `add_po_masks` command.

Examples

```
DRC> report_po_masks
Warning: Requested report contained no entries. (M13)
DRC> add_po_masks {D0 D1 D2 D3}
DRC> report_po_masks
output port_name
-----
/D0
/D1
/D2
/D3
```

See Also

[PO Masks Report](#)

[remove_po_masks](#)

Table Column Outside Table:

report_primitives

Overview

Use this command to report data on ATPG primitives in the model created by the `run build_model` command.

Syntax

```
report_primitives
  < id | instance_name | net_name | pin_pathname
    | -ports | -pis | -pos | -pios | -type type
    | -summary | -all > [-max d]
```

Arguments

id | instance_name | net_name | pin_pathname

Selects display of pin data for the specified object. The type of pin data is specified by a prior `set_pindata` command. An object is identified by its [primitive ID](#) number, its [instance name](#), a [net name](#) connected to the object, or a [pin pathname](#) of a pin of the instance. You can use a wildcard character with an instance name. However, when TestMAX ATPG reads an image file, the object cannot be identified by its instance name.

-all

Reports all primitives. The pin data, as specified by a prior [set_pindata](#) command, is displayed for each primitive.

-ports

Reports all external (top-level) ports; equivalent to `-pis -pos -pios`.

-pis

Reports all input ports (Primary Inputs).

-pos

Reports all output ports (Primary Outputs).

-pios

Reports all bidirectional ports (Primary I/O ports).

-type type

Reports all primitives of a specific classification type. The pin data, as specified by a prior [set_pindata](#) command, is displayed for each primitive.

Priority indicator "p" on an input shows that the input (which must be the set or reset of a DLAT or DFF) has priority over the other set or reset of the same primitive. Priority indicator "P" on an input shows that the input (which must be the set, reset, or clock input of a DLAT or DFF) has priority over other clocks non-set/reset) of the same primitive.

These indicators allow correlating the attributes of gate pins in the flattened model with the attributes of module ports in the netlist data (see the `report_modules` command).

The recognized types include simulation primitives and ATPG functions as well as learned behavior analysis:

Simulation and ATPG Function Types:

ADRBUS, AND, BUF, BUFZ, BUS, BUSK, DATABUS, DFF, DLAT, EQUIV, INV, MEMORY, MOUT, MUX, NAND, NOR, OR, PI, PIO, PO, RPORT, SEL01, SEL1, SW, TIE0, TIE1, TIEX, TIEZ, TSD, WBUF, WIRE, XNOR, XOR

Learned Behavior Types:

blocked, common_input, common_tied_input, conblocked, constrained, equivalences, implications, invert_inputs, learn_buf, learn_inv, learn_tied_gate, tied, weak. For more information, see "[ATPG-Specific Learning Processes](#)."

-summary

Displays a summary of primitives used in the ATPG simulation model. This summary includes the total number of primitives as well as a count of each primitive type used in the ATPG model.

-max d

Limits the number of primitives reported to specified maximum. This option does not work with the `-summary` or `-all` options. To report a specific number out of all primitives, use the `report_primitives * -max d` command.

Allowed Command Modes

DRC, Test

Description

Use this command to report data on ATPG primitives in the model created by the `run build_model` command.

Examples

```
TEST> report primitives -summary
      Primitive Summary Report
```

```
-----
#primitives          20201
#primary_inputs      49
#primary_outputs     41
#primary_bidis       40
#DFFs                1713
  #nonscan           201
  #scan              1512
#BUSs                40
#BUFs               1573
#INVs               1963
#ANDs               2384
#NANDs              5381
#ORs                3298
#NORs               972
#XORs               208
#XNORs              141
#TIE0s              55
#TIE1s               3
#MUXs               2259
#TSDs               81
```

```
-----
# by instance pathname
DRC> report_primitives reg0
reg0 (48) INV (DFFRLP)
  ---      I      127-reg0/r/Q
  QB       O      50-reg1/MX1/SDI
```

```
# by pin pathname
DRC> report_primitives /reg1/MX1/SDI
reg1/MX1 (50) AND (SCANINP_UDP_1)
  SDI      I 48-reg0/QB
  ---      I 40-
  ---      O 85-
```

```
# by primitive ID number
DRC> report_primitives 128
reg1/lat1 (128) DLAT (P_LAT_RS)
  !SB      I (TIE_1)
  RB       I 26-
  CK       I 16-
  D        I 85-reg1/MX1/Q
  Q        O 129-reg1/r/D
```

```
# by type of modeling primitive
DRC> report_primitives -type XOR
-----
List of XOR gates
-----
adder (53) XOR (INC4)
  ---      I 51-
  ---      I 47-
  S0       O 56-mux0/B
adder (71) XOR (INC4)
  ---      I 69-
  ---      I 54-
  S1       O 74-mux1/B
adder (91) XOR (INC4)
  ---      I 89-
  ---      I 72-
  S2       O 94-mux2/B
adder (110) XOR (INC4)
  ---      I 107-
  ---      I 92-
  S3       O 113-mux3/B
Total number of reported XOR gates = 4
```

```
DRC> report_primitives -type dlat
-----
List of dlat gates
```

```

-----
(6) DLAT (_DLAT)
    --- pP I 4-SET
    !--- P I 5-RESET
    --- I 0-CLK1
    --- I 2-DATA1
    --- I 1-CLK2
    --- I 3-DATA2
    --- O 7-LOUT
Total number of reported dlat gates = 1

```

```

TEST> report_primitives -pis
gate_id type  port_name (behavior_data)
-----
    0  PI      CLK clock(off=0,master,shift)
    1  PI      RSTB clock(off=1,master,reset)
    2  PI      SDI2 scanin(c1)
    3  PI      SDI1 scanin(c2)
    4  PI      INC
    5  PI      SCAN

```

```

TEST> report_primitives -pos
gate_id type  port_name (behavior_data)
-----
    62  PO      SDO2 scanout(c1)
    63  PO      COUT

```

```

TEST> report_primitives -pios
gate_id type  port_name (behavior_data)
-----
    6  PIO      D0
    7  PIO      D1 scanout(c2)
    8  PIO      D2
    9  PIO      D3

```

```

TEST> report_primitives -ports
gate_id type  port_name (behavior_data)
-----
    0  PI      CLK clock(off=0,master,shift)
    1  PI      RSTB clock(off=1,master,reset)
    2  PI      SDI2 scanin(c1)
    3  PI      SDI1 scanin(c2)
    4  PI      INC
    5  PI      SCAN
    6  PIO      D0
    7  PIO      D1 scanout(c2)
    8  PIO      D2
    9  PIO      D3
    62  PO      SDO2 scanout(c1)

```

```

63    PO        COUT

# for a RAM or ROM
TEST> report_primitives u4
u4 (149)      MEMORY (ram512x8)
---          I (TIE_0)
---          I (TIE_0)
wclk         I 15-
---          I (TIE_1)
---          I 102-
---          I 120-
---          O 103-

Memory data: #data_lines=8, #address_lines=9/0, range=0-499,
file=ram512x8.dat
Read port usage: #read_ports=1, #cam_ports=0, read_off=X
Write port usage: #write_ports=1, edge_trigger=yes, type=stable_low
Stability results: clock_off=yes, load=yes, read_only_ability=no
Conflict behavior: write_write=XBIT, read_write=READ_NEW, read_read=READ_
NORMAL

```

See Also

- [Gate Type](#)
 - [Primitive IDs](#)
 - [Primitives Report](#)
 - [set_pindata](#)
 - [set_primitive_report](#)
 - [Understanding Pin Data](#)
 - [ATPG-Specific Learning Processes](#)
- Table Column Outside Table:

report_power

Overview

Use this command to report various types of power data.

Syntax

```
report_power
  [-average]
  [-capture]
  [-csv filename]
  [-first_pattern d]
  [-last_pattern d]
  [-max d]
  [-module name]
  [-peak]
  [-percentage]
  [-per_clock_domain]
  [-per_cycle]
  [-per_pattern]
  [-shift]
```

Arguments

-average

Includes average power data in the power report. The default is to include both average and peak power. This option only works with the `-per_pattern` option. If this option is issued without the `-per_pattern` option, then the `-per_pattern` option is automatically selected.

-capture

Uses capture cycles as a basis for power reporting. By default, shift, capture, and load cycles are reported.

-csv *filename*

Directs the output of the `report_power` command to a comma-separated file (csv). This option does not work with the `-percentage` option.

-first_pattern *d*

Specifies the first pattern to run the analysis on. This is applied to `-per_cycle` and `-per_pattern` options, and well as the case where neither is specified.

-last_pattern *d*

Specifies the last pattern to run the analysis on. This setting is applied to the `-per_cycle` and `-per_pattern` options, or when neither option is specified.

-max *d*

Includes a sorted list of shift, capture and load power switching for the maximum *d* patterns in the report generated from the `-per_pattern` option. If the `-per_pattern` option is not selected, the `-max` option automatically selects the `-per_pattern` option.

-module *name*

Specifies the module name used as a basis for calculating the power. Only the scan cells within the specified module is considered during power calculations.

-peak

Includes peak power in the power report. The default is to include both average and peak power. This option only works with the `-per_pattern` option. If this switch is issued without `-per_pattern`, then `-per_pattern` is automatically selected.

-percentage

Reports, as a percentage, all scannable flip-flops that change relative to all scannable flip-flops contained within the design and recognized as a result of the [run_drc](#) command.

-per_clock_domain

Creates individual capture power reports for each clock. By default, the `report_power` command creates a consolidated report for all clock domains.

-per_cycle

Reports switching information on a cycle-by-cycle basis. These cycles are arranged based on the actual sequence that would occur while applying the actual ATPG vector. For a basic scan vector, the power switching results for a number of shift cycles are equal to the length of the longest scan chain, followed by a single capture cycle, and this is repeated for the number of ATPG patterns in the pattern set.

For a fast-sequential pattern with one load and multiple capture cycles, the switching activity of several capture cycles are reported per pattern. As previously mentioned, a pattern set that includes both basic scan and fast-sequential patterns results in a report in which some patterns include one capture per pattern in their `-per_cycle` report, while some patterns include more than one. From the `-per_cycle` report, you cannot identify which patterns are basic scan patterns and which patterns are fast-sequential patterns, except that a pattern with more than one capture cycle can never be a basic scan pattern.

The `-per_cycle` switch is not supported with the multicore ATPG feature.

-per_pattern

Reports the average and peak switching results for shift, load and capture on a pattern by pattern basis. Peak shift results are reported for each pattern by comparing the instantaneous switching for different cycles and different load's of the same pattern to determine the load and shift cycle during which the maximum power switching occurs. The average shift results are derived by calculating the average switching for all cycles and all loads.

Peak capture results are calculated by comparing the capture power for different capture cycles of the same pattern to determine the cycle at which the peak power switching occurs. Average capture results are calculated by taking the average capture power of different capture cycles for the same pattern. Since basic scan patterns include only one capture cycle, average and peak power is always the same.

Peak load results are calculated by comparing the total power for different load cycles of the same pattern to determine the load cycle at which the peak power switching occurs. Average load results are calculated by taking the average load power of different load

cycles for the same pattern. Since basic scan pattern include only one load or unload cycle, average and peak power is always the same.

-shift

Uses shift cycles as a basis for power reporting. By default, shift, capture, and load cycles are reported.

Description

Use this command to report various types of power data.

The `-per_cycle`, `-first`, `-last`, and `-module` options of this command have no effect if the `-calculate_power` option of the [set_atpg](#) command is used. If you want to generate switching information on a cycle-by-cycle basis, use the following commands:

```
set_atpg -nocalculate_power
report_power -per_pattern -shift -capture -per_cycle
```

For related details, see "[Power Aware ATPG](#)" in the *TestMAX ATPG User Guide*.

Examples

```
TEST> report_power -per_pattern -percentage
-----
                Power Analysis Summary
-----
Number of Scan Cell           750534
Number of Patterns           0-2680
Cycle Per Load               3412
Average Shift Changes:       2400.38 3.20%
Average Capture Changes:     9058.04 12.07%
Maximum Shift Cell Changes:  37510 49.97% (pattern: 0 cycle: 3411)
Maximum Capture Cell Changes: 30742 40.96% (pattern: 1)
```

See Also

[Patterns Report](#)
[write_patterns](#)
[set_patterns](#)

report_rules

Overview

Use this command to report rule data.

Syntax

```
report_rules
  [rule_id | rule_type | -all]
  [-fail]
```

Arguments

rule_id

Reports all rules of the selected rule ID. You can use `-fails` to display only failing rules for this rule ID.

rule_type

Reports all rules of the selected rule type. You can use `-fails` to display only failing rules for this rule type.

`-all`

Reports all rules. This is the default choice. You can use `-fails` to display only the failing rules.

`-fail`

Reports only the rules that have violations.

Allowed Command Modes

All

Description

Use this command to report rule data.

Examples

```
TEST> report_rules -fail
rule severity #fails description
-----
B6    warning    2      undriven module inout pin
B7    warning   178      undriven module output pin
B10   warning    32      unconnected module internal net
B13   warning     2      undriven instance input pin
S23   warning    64      unobservable potential TLA
S29   warning     1      invalid dependent slave operation
C3    warning    32      no latch transparency when clocks off
C6    warning     1      TE port captured data affected by new capture
Z1    warning   289      bus contention ability check
Z2    warning   289      Z-state ability check
Z4    warning   360      bus contention in test procedure
```

```
TEST> report_rules C17
rule severity #fails description
-----
C17  warning  3      clock connected to PO
```

```
rule fails #fails description
-----
error invalid argument "TO" (M1)
```

```
TEST> report_rules T
rule severity #fails description
-----
T1   error    0      maximum number of periods exceeded
T2   error    0      minimum time for period not satisfied
T3   error    0      maximum time for period exceeded
T4   error    0      period timing fails resolution check
T5   error    1      minimum time for shift period not satisfied
T6   error    0      maximum number of pins exceeded
T7   error    0      maximum number of waveform shapes exceeded
T8   error    0      maximum number of time sets exceeded
T9   error    0      unsupported strobe type
```

See Also

[Rules Report](#)

[set_rules](#)

[report_violations](#)

report_scan_ability

Overview

Use this command to report nonscan cells that have been selected to behave as scan cells using the `set_scan_ability` command.

Syntax

```
report_scan_ability
  [-max d]
```

Arguments

-max d

Specifies the maximum number of virtual scan cells allowed to be reported. By default, there is no limit.

Allowed Command Modes

DRC, Test

Description

Use this command to report nonscan cells that have been selected to behave as scan cells using the `set_scan_ability` command. You can use this report to predict the effects of nonscan cells being converted to scan cells.

Examples

```
DRC> report_scan_ability
gate_id type instance_name
-----
There were 0 DLATs/DFFs with scan ability set to ON.
DRC> set_scan_ability on core/host/d_l2odd_reg
DRC> set_scan_ability on core/host/mrd_st_reg_1
DRC> report_scan_ability
gate_id type instance_name
-----
97299    DFF host/d_l2odd_reg
97302    DFF host/mrd_st_reg_1
```

See Also

[Scan Ability Report](#)
[set_scan_ability](#)
[report_nonscan_cells](#)
[report_scan_cells](#)

Table Column Outside Table:

report_scan_cells

Overview

This command reports scan cell information for selected scan cells.

Syntax

```
report_scan_cells
< { chain_names [position] } | -shadows | -unstable_set_resets | -all >
[-clocks]
[-ignore_constraints]
[-master_only]
[-max d]
[-physical]
[-pins]
[-reverse_order]
[-shift_clocks]
[-verbose]
```

Arguments

chain_names [position]

Reports the scan cells for the specified scan chain(s). If you do not specify the position within the scan chain, all scan cells of the scan chain are reported. If you specify the position, only the scan cell in that position of the scan chain is reported. Scan cell positions are numbered sequentially, with 0 identifying the scan cell closest to the scan output pin. Note that if you specify the scan cell position, you must enclose both the chain name and the scan cell position within curly braces, as shown in the following example:

```
TEST-T> report_scan_cells { chain0 0 }
```

-shadows

Reports only the shadow gates of scan cells.

-unstable_set_resets

Reports scan cells that have [unstable set/reset](#) controls.

-all

Reports all scan cells.

-clocks

Reports the capture clocks associated with each scan cell.

-ignore_constraints

Modifies the `-verbose` report format so that all clock ports that fan into the scan cell are listed regardless of constraints. This command is valid only if the `-verbose` option is also specified.

-master_only

The report normally includes the master cell as well as any associated slave, dependent slave, shadow, or observable shadow cell types. The `-master_only` option causes only the master cells to be reported.

-max d

Specifies the maximum number of scan cells to report. By default, there is no limit.

-physical

Includes physical locations for the reported scan cells. This option requires that physical locations were previously extracted using the [extract_pins](#) command.

-pins

Specifies that the input and output pin information of each scan cell are also to be reported, including the pin pathname and the inversion information relative to the scan chain input and output.

-reverse_order

By default, the report starts with the scan cell closest to the scan output port (cell 0). The `-reverse_order` option causes the report to start with the scan cell closest to the scan input port.

-shift_clocks

Reports the shift clock associated with each scan cell. Constraints are taken in account when tracing back the clock ports of each scan cell. Note that this option is incompatible with the `-verbose` or `-ignore_constraints` options.

-verbose

Modifies the report format to include the following additional information:

1. Leading edge ([LE](#)) vs. trailing edge ([TE](#)) clocking.
2. The capture clock polarity of the cell relative to the top-level clock port, where "+" indicates a 0-to-1 transition (at the top-level port) causes capture, "-" indicates a 1-to-0 transition causes capture, and "?" indicates a scan [TLA](#) cell for which polarity is not relevant.
3. A list of capture clocks connected to the scan cell state element. Constraints are taken into account when tracing back the clock ports of each scan cell.

Note that the `-verbose` option is incompatible with the `-shift_clocks` option.

Allowed Command Modes

TEST

Description

This command reports scan cell information for selected scan cells. Only sequential devices that are associated with scan cells are reported. Other sequential devices found on the scan data path that do not have a value derived from a scan cell (that is, flush latches, certain lockup latches, certain remodeled devices) will not appear in this report. For a list of all devices in the physical scan path use the `report_scan_paths` command.

The standard report lists the name of the scan chain, the position within the scan chain of the cell, the type of behavior identified for the cell (master, slave, shadow, and so on), the inversion information between the scan chain input and the cell and from the cell to the scan chain output (I = inversion, N = no inversion), the [primitive](#)

[ID](#) of the cell, and the [instance name](#) of the cell. For more information, see [Understanding the Scan Cells Report](#).

The bit position is counted relative to the scan output port for the scan chain. It represents the number of shifts required for the scan cell to be visible at the scan chain output. The scan cell closest to the scan output is bit position 0, the next one is 1, and so on.

Examples

```
TEST-T> report_scan_cells c1
chain  cell type      inv gate# instance_name (type)
-----
c1      0      MASTER IN  147      reg4/r (N_LATCH)
c1      1      MASTER IN  145      reg3/r (N_LATCH)
DSLAVE  IN  146      reg4/lat1 (P_LATCH)
c1      2      MASTER NI 143      reg2/r (N_LATCH)
SCANTLA IN 144      reg3/lat1 (P_LATCH)
```

```
TEST-T> report_scan_cells -all -reverse -verbose
chain  cell type      edge inv gate# clocks instance_name (type)
-----
-
chain1  6      MASTER  TE   NN  23      - clk1 flop1 (FD1S)
chain1  5      MASTER  TE   NN  24      - clk2 flop2 (FD1S)
chain1  4      MASTER  LE  NN  25      + clk1 flop3 (FD1S)
chain1  3      MASTER  LE  NN  26      + clk1 flop4 (FD1S)
chain1  2      MASTER  LE  NN  27      + clk1 flop5 (FD1S)
chain1  1      MASTER  LE  NN  28      + clk1 flop6 (FD1S)
DSLAVE  LS   NN   22      - clk1 lock2 (LD2)
chain1  0      MASTER  LE  NN  29      + clk2 flop7 (FD1S)
```

```
TEST-T> report_scan_cells c1 -pins
chain  cell type      inv gate# instance_name (type)
-----
c1      0      MASTER IN  147      reg4/r (N_LATCH)
          input I  147      reg4/r/D (N_LATCH)
output N 147      reg4/r/Q (N_LATCH)
c1      1      MASTER IN  145      reg3/r (N_LATCH)
DSLAVE  IN  146      reg4/lat1 (P_LATCH)
input I  145      reg3/r/D (N_LATCH)
output N 146      reg4/lat1/Q (P_LATCH)
c1      2      MASTER NI 143      reg2/r (N_LATCH)
SCANTLA IN 144      reg3/lat1 (P_LATCH)
input N  143      reg2/r/D (N_LATCH)
          output N 144      reg3/lat1/Q (P_LATCH)
```

```
TEST-T> report_scan_cells {c1 1} -pins
chain  cell type      inv gate# instance_name (type)
-----
c1      1      MASTER IN  145      reg3/r (N_LATCH)
DSLAVE  IN  146      reg4/lat1 (P_LATCH)
input I  145      reg3/r/D (N_LATCH)
```



```
output  N    146    reg4/lat1/Q (P_LATCH)
```

```
TEST-T> report_scan_cells -all -master_only
chain   cell type      inv gate# instance_name (type)
-----
chain_1 0    MASTER  IN   147    reg4/r (SDFF)
chain_1 1    MASTER  IN   145    reg3/r (SDFF)
chain_1 2    MASTER  NI   143    reg2/r (SDFF)
c2       0    MASTER  IN   141    reg1/r (SDFF)
c2       1    MASTER  NI   139    reg0/r (SDFF)
```

```
TEST> report_scan_cells -all -reverse_order -master_only
chain   cell type      inv gate# instance_name (type)
-----
chain_1 2    MASTER  NI   143    reg2/r (SDFF)
chain_1 1    MASTER  IN   145    reg3/r (SDFF)
chain_1 0    MASTER  IN   147    reg4/r (SDFF)
c2       1    MASTER  NI   139    reg0/r (SDFF)
c2       0    MASTER  IN   141    reg1/r (SDFF)
```

```
chain   cell type      inv gate# instance_name (type)
-----
c1       0    MASTER  IN   147    reg4/r (N_LATCH)
chain_1 1    SHADOW  NN    61    reg3_shadow (dff)
c2       0    SHADOW  NN    59    reg1_shadow (dff)
```

See Also

[Understanding the Scan Cells Report](#)

[extract_pins](#)

[report_nonscan_cells](#)

[report_scan_chains](#)

[Scan Cell Types](#)

report_scan_chains

Overview

Use this command to report scan chain information.

Syntax

```
report_scan_chains
  [-command_report]
  [-shift_clocks]
  [-verbose]
  [-xchains]
```

Arguments

-command_report

Causes the scan chains to be reported in the form of `add_scan_chains` commands that would be necessary to define these same scan chains.

-shift_clocks

Reports the shift clocks for each scan chain while taking constraints into account.

-verbose

Selects reporting additional information about each scan chain such as all the clocks connected to state elements in the scan chain (these clocks can function as shift, capture, set, reset), the number of leading-edge ([LE](#)), trailing-edge ([TE](#)), or level-sensitive ([LS](#)) devices. Also reported is the number of dependent slaves, shadows, and nonscan transparent latches found in the scan chain, if any.

-xchains

Reports only the scan chains designated as X-chains.

Allowed Command Modes

DRC, Test

Description

Use this command to report scan chain information. The standard report for a chain includes its name, length, input port, and output port. The verbose report includes associated clocks and a breakdown of leading-edge (LE), trailing-edge (TE), and level-sensitive (LS) master cells.

Examples

```
TEST> report_scan_chains
chain    group length input_port output_port
-----
c1       sg0    230    a_scanin1 a_scanout1
c2       sg0    225    a_scanin2 a_scanout2
c3       sg0    230    a_scanin3 a_scanout3
c4       sg0    224    a_scanin4 a_scanout4
```

```
c5      sg0      175      a_scanin5 a_scanout5
```

```
TEST> report_scan_chains -shift clocks
chain1 master clocks: gck1(0) gck2(1)
chain1 slave clocks:
```

```
TEST> report_scan_chains -verbose
chain  group #LE/#TE/#LS input_port output_port clocks
-----
c1      sg0    230/0/1      a_scanin1  a_scanout1  clk1 rst clk2 #dslaves=2
c2      sg0    200/25/0     a_scanin2  a_scanout2  clk2 #shadows=14
c3      sg0    115/115/0    a_scanin3  a_scanout3  phi_1 phi_2
c4      sg0    224/0/0     a_scanin4  a_scanout4  clk1 rst #scantlas=3
c5      sg0     0/175/0     a_scanin5  a_scanout5  nclk
```

See Also

[Understanding the Scan Chains Report](#)

[Understanding Compressor Connections in the Output from report_scan_chains](#)

[report_scan_cells](#)

[report_scan_path](#)

[Scan Cell Types](#)

report_scan_enables

Overview

Use this command to report scan enable attributes previously defined with the `add_scan_enables` command.

Syntax

```
report_scan_enables
```

Arguments

None

Allowed Command Modes

DRC, Test

Description

Use this command to report scan enable attributes previously defined with the `add_scan_enables` command.

Examples

```
TEST> report_scan_enables
value port
-----
1      SCAN_EN (6)
Z      bidi[0] (3210)
Z      bidi[1] (3211)
Z      bidi[2] (3212)
```

See Also

- [add_scan_chains](#)
- [report_scan_chains](#)
- [report_scan_cells](#)
- [report_scan_path](#)

Table Column Outside Table:

report_scan_path

Overview

This command reports the gates in a segment of a scan chain path.

Syntax

```
report_scan_path  
  chain_name <sco | cell_position> <sci | cell_position>  
  [-physical]  
  [-reverse]  
  [-verbose]
```

Arguments

chain_name

Specifies the name of the scan chain selected for reporting the scan path segment.

sco | cell_position

Specifies the end of the scan path segment. If you select *sco*, the trace begins at the scan chain output port. If you select a *cell_position*, the trace is bounded by the master of that scan cell. When there are multiple scan chains that share the same output port, specify the position as 0 rather than *sco* for best results.

sci | cell_position

Specifies the beginning of the scan path segment. If you select *sci*, the trace continues to the scan chain input port. If you select a *cell_position*, the trace is bounded by the master of that scan cell. If you specify both ends of the segment to be reported, the second *cell_position* number must be greater than the first.

-physical

Includes physical locations for the reported gates. This option requires that physical locations for cells on the scan paths were previously extracted using the [extract_pins](#) command.

-reverse

Reports the gates in reverse order.

-verbose

Provides a one line report of each gate in the selected scan chain path.

Allowed Command Modes

DRC, TEST

Description

This command reports the gates in a segment of a scan chain path.

Note the following usage for specifying multiple arguments in Tcl mode:

```
TEST-T> report_scan_path c0 {SC0 SCI}
```

Examples

```

TEST-T> report_scan_path c1 sco sci
Scan path for chain=c1: begin_position=SCO, end_position=SCI
SDO2 (148) PO (_PO)
    --- I 137-reg4/Q
    SDO2 O
PO usage: scanout(c1)
reg4 (137) BUF (DFFP)
    --- I 147-reg4/r/Q
    Q O 148-SDO2
reg4/r (147) DLAT (N_LATCH)
    !SB I (TIE_1)
    !RB I (TIE_1)
    CK I 14-
    D I 146-reg4/lat1/Q
    Q O 137-
scan_behavior: MASTER(LS/-) chain=c1 cell_id=0 invert_data=IN obs=noproc
reg4/lat1 (146) DLAT (P_LATCH)
    !SB I (TIE_1)
    !RB I (TIE_1)
    CK I 19-
    D I 116-reg3/Q
    Q O 147-reg4/r/D
scan_behavior: DSLAVE(LS/-) chain=c1 cell_id=1 invert_data=IN
reg3 (116) BUF (DFFRLP)
    --- I 145-reg3/r/Q
    Q O 146-reg4/lat1/D
        118-out3/DO ...
reg3/r (145) DLAT (N_LATCH)
    !SB I (/TIE_1)
    RB I 31-
    CK I 13-
    D I 144-reg3/lat1/Q
    Q O 116-
scan_behavior: MASTER(LS/-) chain=c1 cell_id=1 invert_data=IN obs=noproc
reg3/lat1 (144) DLAT (P_LATCH)
    !SB I (TIE_1)
    RB I 30-
    CK I 18-
    D I 136-reg3/MX1/Q
    Q O 145-reg3/r/D
scan_behavior: SCANTLA(UP/D) chain=c1 cell_id=2 invert_data=IN
reg3/MX1 (136) OR (SCANINP_UDP_1)
    --- I 96-
    --- I 135-
    Q O 144-reg3/lat1/D
reg3/MX1 (96) AND (SCANINP_UDP_1)
    SDI I 94-reg2/QB
    --- I 46-

```

```

    --- O 136-
reg2 (94) INV (DFFRLP)
    --- I 143-reg2/r/Q
    QB O 96-reg3/MX1/SDI
reg2/r (143) DLAT (N_LATCH)
    !SB I (TIE_1)
    RB I 29-
    CK I 12-
    D I 142-reg2/lat1/Q
    Q O 95-
        94-
scan_behavior: MASTER(LS/-) chain=c1 cell_id=2 invert_data=NI obs=noproc
reg2/lat1 (142) DLAT (P_LATCH)
    !SB I (/TIE_1)
    RB I 28-
    CK I 17-
    D I 115-reg2/MX1/Q
    Q O 143-reg2/r/D
non_scan_behavior: TLA(hot_clock_tla) port_data=0-0C-1C single_port_id=2
reg2/MX1 (115) OR (SCANINP_UDP_1)
    --- I 49-
    --- I 114-
    Q O 142-reg2/lat1/D
reg2/MX1 (49) AND (SCANINP_UDP_1)
    SDI I 2-SDI2
    --- I 45-
    --- O 115-
SDI2 (2) PI (_PI)
    SDI2 O 49-reg2/MX1/SDI
PI usage: scanin(c1)

```

```
TEST-T> report_scan_path 1 {2 3} -physical
```

```
Scan path for chain=1: begin_position=2, end_position=3
```

```
gate_id gate_type cell_id cell_type inv instance_name (type) location
```

```
-----
```

```

1601 DFF 2 MASTER  IN I_RISC_CORE/I_STACK_TOP/I_STACK_FSM/TOS_int_reg_0_
(sdcrb1) L=878135 R=891255 B=716595 T=720285 RT=180 M=false
1499 MUX ---      --- -- I_RISC_CORE/I_STACK_TOP/I_STACK_FSM/TOS_int_reg_0_
(sdcrb1) L=878135 R=891255 B=716595 T=720285 RT=180 M=false
1596 DFF 3 MASTER  IN I_RISC_CORE/I_STACK_TOP/I_STACK_FSM/STACK_FULL_reg
(sdcrq1) L=858045 R=870755 B=676005 T=679695 RT=0 M=true

```

See Also

[Understanding the report_scan_path Output](#)

[report_scan_cells](#)

[report_scan_chains](#)

[extract_pins](#)

report_sdc

Overview

This command, which is valid only in Tcl mode, reports timing exceptions read in from a SDC (Synopsys Design Constraints) file.

Syntax

```
report_sdc
  [-clocks]
  [-to_cells]
  [-groups]
  [-case_analysis]
  [-false_paths]
  [-multicycle_paths]
  [-case_paths]
  [-disable_paths]
  [-all_paths]
```

Arguments

-clocks

Reports SDC clocks specified by the `create_clock` and `create_generated_clock` SDC commands. This does not include clocks specified only by regular TestMAX ATPG commands.

-to_cells

Reports SDC clocks and the registers driven by these clocks. This report is useful for debugging clock-to-clock exceptions, but can be very long.

-groups

Reports exceptions specified by the `set_clock_groups` SDC command.

-case_analysis

Reports constant signals specified by the `set_case_analysis` SDC command.

-false_paths

Reports exceptions specified by the `set_false_path` SDC command.

-multicycle_paths

Reports exceptions specified by the `set_multicycle_path` SDC command.

-case_paths

Reports exceptions specified by the `set_case_analysis` SDC command (with `set_sdc -case_paths`).

-disable_paths

Reports exceptions specified by the `set_disable_timing` SDC command.

-all_paths

Reports all exceptions.

Description

This command, which is valid only in Tcl mode, reports timing exceptions read in from a SDC (Synopsys Design Constraints) file. It is used only in TEST mode, and must be used in conjunction with the `read_sdc` command, which reads in an SDC file.

When exception paths are specified for specific pins of sequential cell instances, they can be modified to fit into the TestMAX ATPG database. In this case, they are reported as they exist in the TestMAX ATPG database — not as they were specified in the SDC file. The most common change is that `-to` exceptions for input pins are converted to `-through` exceptions for the same input pins. Exceptions specified as `-through` output pins can be converted to `-from` exceptions.

For more information on specifying timing exceptions, see the "[Specifying Setup Timing Exceptions From an SDC File](#)" section in the *TestMAX ATPG User Guide*. For more information on the SDC file and the valid commands accepted by TestMAX ATPG, see [How TestMAX ATPG Interprets SDC File Commands](#).

See Also

[How TestMAX ATPG Processes Setup and Hold Violations](#)

`read_sdc`

`set_sdc`

report_serializers

Overview

This command generates a report containing data for the specified serializers.

Syntax

```
report_serializers
  <<serializer_name> | -load | -unload | -all> -chain_segments
  [-verbose]
```

Arguments

<serializer_name>

The name of the serializer for which you want to generate a report.

-load

Reports all load serializers.

-unload

Reports all unload serializers.

-all

Reports all serializers.

-chain_segments

Reports the scanin port and length for each load serializer segment and the scanout port and length for each unload serializer segment.

-verbose

Reports the data for the specified serializers in verbose mode.

Allowed Command Modes

TEST

Description

This command generates a report containing data for the specified serializers.

Examples

```
TEST-T> report_serializers -load
-----
name                                     type length
-----
U0/U_deserializer_my_serial            load 8
-----
-----
Scanin Index Parallel Outputs
invert
-----
```

```

-----
SI_0 0 U0/U_decompressor_my_serial/\serial_update_reg[0] no
SI_0 1 U0/U_decompressor_my_serial/\serial_update_reg[1] no
SI_0 2 U0/U_decompressor_my_serial/\serial_update_reg[2] no
SI_0 3 U0/U_decompressor_my_serial/\serial_update_reg[3] no
SI_0 4 U0/U_decompressor_my_serial/\serial_update_reg[4] no
SI_0 5 U0/U_decompressor_my_serial/\serial_update_reg[5]
no
SI_0 6 U0/U_decompressor_my_serial/\serial_update_reg[6] no
SI_0 7 U0/U_decompressor_my_serial/\serial_update_reg[7] no

```

```
TEST-T> report_serializers -unload
```

```

-----
name                                     type length
-----
U0/U_deserializer_my_serial             load 8
-----

```

```

Scanin Index Parallel Outputs
invert
-----

```

```

-----
SO_0 0 U0/U_compressor_my_serial/U8 no
SO_0 1 U0/U_compressor_my_serial/U102 no
SO_0 2 U0/U_compressor_my_serial/U197 no
SO_0 3 U0/U_compressor_my_serial/U301 no
SO_0 4 U0/U_compressor_my_serial/U413 no
SO_0 5 U0/U_compressor_my_serial/U414 no
SO_0 6 U0/U_compressor_my_serial/U415 no

```

```
TEST-T> report_serializers -chain_segments
```

```

-----
serializer input port length
-----
SI_0                                     8
-----
serializer output port                length
-----
SO_0
SO_0 7 U0/U_compressor_my_serial/U417

```

report_settings

Overview

This command reports the current settings defined by the set commands, such as the [set_contention](#), [set_atpg](#), and [set_build](#) commands. For a complete list of the set commands in TestMAX ATPG, see [Set Commands](#).

Note: The bidi_map options of the wgl category are reported only when the mapping causes a change from its default setting. For example, the [set_wgl](#) [list z0 z0] command specifies that there is no BIDI mapping. In this case, the output of the `report_settings wgl` command does not report the BIDI mapping.

Examples

```
BUILD> report_settings contention
contention =          multiple_on=yes, atpg=yes, bidi=yes, bus=yes,
                      dff_dlat=no, float=no, preclock=yes,
                      post_capture_clock_check=yes, ram=no, retain_bidi_
direction=no,
                      severity=warning, verbose=no, wire=no,
                      capture_constraint_check=no;

BUILD> report_settings atpg
atpg =                allow_clockon_measures=no, abort_limit=10, analyze_
untestable_faults=no, capture_cycles=0,
                      checkpoint=none, save_patterns=none,
                      coverage=100.00, decision=norandom, di_analysis=yes,
                      full_seq_abort_limit=10, full_seq_atpg=no, full_seq_
time=(10.0,0.0),
                      full_seq_merge=off, merge=off 5000, basic_min_
detects_per_pattern=(0,40),
                      fast_min_detects_per_pattern=(0,40), full_min_
detects_per_pattern=(0,40),
                      lete_fastseq=no, min_ateclock_cycles=none,
                      new_capture=no, patterns=0, prevention=norandom,
reset_bidis=yes,
                      store=yes, resim_atpg_patterns=off,
                      summary=yes, time=(0.0,0.0), verbose=no,
                      post_capture_contention_prevention=no,
                      fill=random, adjacent_fill_range=0,
                      optimize_bridge_strengths=no, min_patterns_
threshold=0,
                      power_budget=0,
                      domain={},
                      partition={},
                      power_effort=low,
                      seq_comp_optimizations=yes, misr_measure=once_per_
pattern, seq_comp_chain_test=no,
                      seq_comp_shift_power=100, xtol_enable_equivalence=no,
                      shift_power_budget=none, shift_power_effort=none,
                      chain_test=0011, xtol_chain_diagnosis=OFF, single_
load_per_pattern=no,
```

```

        quiet_chain_test=no, load_mode=0, load_value=0,
        fast_path_delay=yes,
        timing_exceptions_au_analysis=no, calculate_power=no,
parallel_strobe_data_file=none,
        replace_parallel_strobe_data_file=0, num_processes=0;

BUILD> report_settings
    atpg =                allow_clockon_measures=no, abort_limit=10, analyze_
untestable_faults=no, capture_cycles=0,
                        checkpoint=none, save_patterns=none,
                        coverage=100.00, decision=norandom, di_analysis=yes,
                        full_seq_abort_limit=10, full_seq_atpg=no, full_seq_
time=(10.0,0.0),
                        full_seq_merge=off, merge=off 5000, basic_min_
detects_per_pattern=(0,40),
                        fast_min_detects_per_pattern=(0,40), full_min_
detects_per_pattern=(0,40),
                        lete_fastseq=no, min_ateclock_cycles=none,
                        new_capture=no, patterns=0, prevention=norandom,
reset_bidis=yes,
                        store=yes, resim_atpg_patterns=off,
                        summary=yes, time=(0.0,0.0), verbose=no,
                        post_capture_contention_prevention=no,
                        fill=random, adjacent_fill_range=0,
                        optimize_bridge_strengths=no, min_patterns_
threshold=0,
                        power_budget=0,
                        domain={},
                        partition={},
                        power_effort=low,
                        seq_comp_optimizations=yes, misr_measure=once_per_
pattern, seq_comp_chain_test=no,
                        seq_comp_shift_power=100, xtol_enable_equivalence=no,
                        shift_power_budget=none, shift_power_effort=none,
                        chain_test=0011, xtol_chain_diagnosis=OFF, single_
load_per_pattern=no,
                        quiet_chain_test=no, load_mode=0, load_value=0,
                        fast_path_delay=yes,
                        timing_exceptions_au_analysis=no, calculate_power=no,
parallel_strobe_data_file=none,
                        replace_parallel_strobe_data_file=0, num_processes=0;

    bist =                dbist=yes, debug=off, hex=yes, hex_diag_data=no,
                        max_intervals=1000, max_pattern_cells=250,
                        max_seed_patterns=16, max_total_cells=250, min_lfsr_
length=20,
                        multi_seeds_per_interval=no, num_dbist_patterns_per_
interval=32,
                        num_patterns_per_interval=256, randomize_pis=no,
chain_test=no,
                        sim_misr=yes, verbose=no, dump=none, max_routing=4

```

```

pi_assumed_scan=no, po_assumed_scan=no, assume_bist_
setup=no,
force_lfsr_load=no, force_xdbist_scalable_
selector=no,
use_cell_constraints=no, use_constant_value_
cells=yes,
optimize_fault_sim=yes, trailing_edge_checking=yes;

    build =
boundary=lowest,
undriven_bidi=PIO,
directions=no,

merge: bus_keepers=yes
cascaded_gates_with_pin_loss=no
equivalent_dlat_dff=on
feedback_paths=yes
flipflop_from_dlat=on
mux_from_gates=pin-preserve
tied_gates_with_pin_loss=no
global_tie_propagate=yes
wire_to_buffer=yes
xor_from_gates=pin-preserve
bus_handling=none
instance_modify=none
black_box=none design_box=none portfault_box=none
empty_box=none tie0_box=none tiel_box=none

nobox=none;

    buses = external_z=Z, fault_contention=TIEX,
contention_status=noignore all, zstate_
status=noignore all;

    colors =
user_commands=0,0,0 file_commands=0,128,0
error=0,0,220 warning=220,0,220

commands = abort=yes, history=yes;

contention = multiple_on=yes, atpg=yes, bidi=yes, bus=yes,
dff_dlat=no, float=no, preclock=yes,
post_capture_clock_check=yes, ram=no, retain_bidi_
direction=no,

severity=warning, verbose=no, wire=no,
capture_constraint_check=no;

    delay = diagnostic_propagation=no, launch_cycle=clock_launch, mask_
nontarget_paths=no,
pi_changes=yes, po_measures=yes, relative_edge=no,
robust_fill=yes,
simulate_hazards=yes, allow_reconverging_paths=no,
common_launch_capture_clock=no, allow_multiple_
common_clocks=yes,

```

```

slow_equivalence=no, two_clock_transition_
optimization=yes, extra_force=yes
data=none, disturb_clock_grouping=yes, transition_
clocking_checks=no,
slackdata_for_atpg=no, slackdata_for_faultsim=no
infinite_slack_checks=no, negative_slack_checks=no
max_delta_per_fault=0, max_tmgn=100%,
sdql_exponential_function=yes,
allow_functional_sensitization=no, allow_nonrobust_
tests=yes, multicycle_length=0;

diagnosis = time_limit=0, sample=(100000:96), check_expected_
data=yes,
verbose=no, incomplete_failures=no, layout_data=yes,
show_version=off, show_cycles=off,
internal_cell_type=no, max_defective_chains=3, min_
report_score=1
report_net_data=no, max_report_failures=10, cycle_
offset=0,
failure_memory_limit=0, composite=no, auto=yes,
dftmax_chain_format=no, per_pin_limit=no,
num_processes=0, post_procedure=none, process_
fail=stop
pattern=all on,
mapping_report=no, subnet_type=yes, delay_type=no;
use_phds=all

drc = test_proc_file=none,
allow_unstable_set_resets=no, bidi_control_pin=no,
clock=dynamic,
controller_clock=no, disturb_clock_grouping=yes,
disturb_limits=1000 20,
initialize_dff_dlat=X, multi_captures_per_load=yes,
oscillation=500,
remove_false_clocks=no, chain_trace=none, extract_
cascaded_clock_gating=no,
shadows=on, limit_shadow=none,
skew=1, TLAs=yes, trace=off,
store_setup=no, store_stability_patterns=no, store_
unload_mode_data=no,
store_setup_cycles=no,
unstable_lsrams=no, compressor_debug_data=no,
dslave_remodel=yes, reclassify_invalid_dslaves=yes,
constraints=yes, z_check_with_all_constraints=no,
use_cell_constraints=yes, clockdrc_use_cell_
constraints=no,
clock_constraints=none, internal_clock_
timing=default,
seq_comp_accumulator_mode=no,
seq_comp_extend_test_setup=no,
set_unload_mode_ports_to_x=yes,
seq_comp_jtag_lbist_mode=off,
multiple_pipes_per_port=no,

```

```

dftmax_shift_cycles=0,
load_nonscan_cells=no,
store_full_cycles=NO_PROC,
analyze_drc_violation=off
blockage_aware_clock_grouping=no,
regular_scan_shift_cycles=0,
seq_comp_power_mode=no -all,
dynamic_clock_equivalencing=off, scan_shift_
clocks=none,
fast_multifrequency_capture=spf, multiframe_paths=no,
observe_procedure=any, clock_gating_init_cycles_
integer=0,
pll_launch_on_shift=no, num_pll_cycles=0,
check_multiple_shift_clocks=no,
max_pll_simulation_passes=100, pll_simulate_test_
setup=no,
pipeline=no, pipeline_in_compressor=yes,
report_x_sink=no, xchain_threshold_probability=0,
freerunning_expansion=yes,
allow_clocks_as_scan=no;
check_user_serializer_bits=yes;
lockup_after_compressor=no;
independent_sync_clocks=no;
distributed = shell_timeout=30, slave_setup_
timeout=100, print_stats_timeout=30,
script=none, work_dir=none,
verbose=-noverbose,
shell=rsh;

faults = atpg_effectiveness=no, au_credit=0, ax_credit=0, equiv_code=(-
-),
fault_coverage=no, model=stuck, report=uncollapsed,
bridge_inputs=no, persistent_fault_models=no,
multi_fanout_di_faults=yes, summary=noverbose;

iddq = atpg=no, float=yes, strong=yes, toggle=no, weak=yes, write=yes,
bridge_equivalence=yes,
exclude_ports=all,
interval_size=1;

learning = atpg_equivalence=on(sim_passes=32, test_passes=5000),
common_input=yes, equivalent_latches=yes,
implication=medium,
module_learning=yes, disable_time_limits=no,
max_feedback_sources=100, verbose=no;

messages = display=yes, leading_comment=no, level=standard, transcript_
comments=yes,
logfile=off

netlist = celldefine=yes, check_only_used_udps=yes, dominance_
detection=on, conservative_

```



```

                                mux=combinational_udp,enable_portfaults=yes,
escape=cond,
                                max_errors=10, v2k=no, work_dir=".", pin_assign=no,
redefined_module=last, scalar_net=no,
                                sequential_modeling=no, suppress_faults=yes,
xmodeling=yes;

    patterns = source=internal,
                                split_patterns=none,
                                histogram_summary=no, load_summary=no, verilog_last_
scan=yes,
                                netlist_independent=no, measure_forced_bidis=no,
verbose=no;

    physical_db = hostname=none, port_number=none,
                                device=none, version=none,
                                lef_directory=none,
                                def_directory=none,
                                database=none,
                                top_def_file=none,
                                technology_lef_file=none;

    pindata =                    none;
                                -shift_character=S, -constrain_character=X;
                                primitive_report= interval=0, max_fanout=2, time=all,
verbose=no;
                                random_patterns = clock=none, length=1024, observe_
type=master;

    sdc = verbose=no, setup=yes, hold=yes, mark_gui_gates=no, show_all_
matches=no
                                case_paths=no, false_paths=yes,
                                multicycle_paths=yes, disable_paths=yes,
                                delete_common_clock_exceptions=no,
                                instance=none,
                                environment=none;

    simulation = basic_scan=yes, bidi_fill=off, data=(0:-1), measure=pat,
                                analyze_x_sources=no, analyze_x_effects=no,
                                xclock_gives_xout=no, xfill_out_of_range_write=no,
                                pipeline_cells=yes, timing_exceptions_for_stuck_
at=no,
                                oscillation=(10,2), store_memory_contents=no,
                                shift_cycles=0,
                                progress_message=0,
                                words_per_pass=32, verbose=no,
                                dominance_use=off, two_clock_resim=no, num_
processes=0;

    wgl = chain_list=shift, forces_during_load=previous, group_bidis=no,
initial_zx=as_is,
                                inversion_reference=master, last_scan=yes, macro_
usage=no, pad=no,

```

```
scan_map=dash, scan_data_format=pre_measured,  
bidi_map=(Z0,-0) (Z1,-1) (0X,0-) (1X,1-) (XX,X-) (ZX,-X) (-  
X,--) (ZZ,-Z) (Z-,--)  
workspace_sizes = atpg_gates=1024, connectors=20000,  
decisions=10000,  
DRC_buffer_size=1000, line=50000, string=2048,  
command_line=50000, command_words=10000;  
TEST-T> report_settings match_names  
match_names = name_manipulation_flag=ON.  
Prefix 'risc1' to be substituted by 'risc2'.
```

See Also

[Settings Report](#)

report_slow_bidis

Overview

This command is used to report all BUS primitives modified with the `add_slow_bidis` command.

Syntax

```
report_slow_bidis
```

Arguments

None

Allowed Command Modes

DRC
TEST

Description

This command is used to report all BUS primitives modified with the `add_slow_bidis` command.

See Also

[add_slow_bidis](#)
[remove_slow_bidis](#)

report_slow_cells

Overview

This command is used to report scan DFF or DLAT cells modified with the `add_slow_cells` command.

Syntax

```
report_slow_cells
```

Arguments

None

Allowed Command Modes

DRC

Description

This command is used to report scan DFF or DLAT cells modified with the `add_slow_cells` command.

See Also

[add_slow_cells](#)

[remove_slow_cells](#)

Table Column Outside Table:

report_summaries

Overview

Use this command to report any combination of the primitive, fault, pattern, library cell, memory, optimization, sequential depth, or CPU usage summaries.

Syntax

```
report_summaries
[primitives] [faults] [patterns]
[library_cells] [memory_usage] [optimizations]
[sequential_depths] [cpu_usage] [-launch launch_clock] [-capture capture_clock]
[-exclusive] [-shared] [-inter_clock_domain] [-intra_clock_domain]
[-per_clock_domain]
```

Arguments

primitives

Reports the ATPG primitive summary.

faults

Reports the fault summary. The display of collapsed versus uncollapsed fault categories or normal versus verbose fault class detail is controlled by use of the `set_faults` command.

patterns

Reports the pattern count.

library_cells

Reports the library cells. A report includes the name of the library cell and the number which have been instantiated in the flattened design. **Note:** This count can differ slightly from an ASIC vendor's report of the same data due to circuit optimization done for ATPG purposes which can result in the elimination of certain internal gates.

memory_usage

Reports total RAM usage by the process and well as the RAM used by the data space patterns + design of the process. **Note:** This report is available only on UNIX platforms.

optimizations

Reports the gate optimization and removal performed during the run build process.

sequential_depths

Reports the maximum Fast-Sequential depth for controlling, observing, and detection, and a representative [gate ID](#) for each.

cpu_usage

Reports the cumulative CPU usage since starting the tool for selected operations such as reading netlists, performing model builds, generating ATPG patterns, compressing patterns, and so on.

-launch *launch_clock*

When specified with the `faults` option, the `-launch` option specifies the launch clock of the faults to be reported on. You can use this switch independently, or in conjunction with the `-capture` switch. This switch also accepts the keywords `PI`, `NO_CLOCK`, and `MULTIPLE` in either all upper or all lowercase, to report on faults that are launched by PI/PIO, no clock domains, or multiple clock domains, respectively. For a complete description of how TestMAX ATPG distinguishes faults captured by a clock and launched by a clock, see "[Using Clock Domain-Based Faults](#)."

`-capture capture_clock`

When specified with the `faults` option, the `-capture` option specifies the capture clock of the faults to be reported on. This switch can be used independently or in conjunction with the `-launch` switch. This switch also accepts the keywords `PO`, `NO_CLOCK`, and `MULTIPLE` in either all upper or all lowercase, to report on faults that are observed by PO/PIO, no clock domains, or multiple clock domains, respectively. For a complete description of how TestMAX ATPG distinguishes faults captured by a clock and launched by a clock, see "[Using Clock Domain-Based Faults](#)."

`-exclusive`

This option is used for reporting clock domain-based faults. When specified with the `faults` option, the `-exclusive` option excludes the multiple launch and capture section from the report. For further details, please see Reporting Clock Domain-Based Faults.

`-shared`

When specified with the `faults` option, the `-shared` option reports only the section relating to multiple launch and capture clocks. For further details, please see Reporting Clock Domain-Based Faults.

`-inter_clock_domain`

When specified with the `faults` option, the `-inter_clock_domain` option reports on only the exclusive faults that are launched and captured by different clock domains. For further details, please see Reporting Clock Domain-Based Faults.

`-intra_clock_domain`

When specified with the `faults` option, the `-intra_clock_domain` option reports on only the exclusive faults that are launched and captured by the same clock domains. For further details, please see Reporting Clock Domain-Based Faults.

`-per_clock_domain`

This option reports all specified faults with extra information for their launch and capture clocks. All clocks are reported, even those classified as "shared" or "multiple." For further details, see "[Using Clock Domain-Based Faults](#)."

Allowed Command Modes

DRC, Test

Description

Use this command to report any combination of the primitive, fault, pattern, library cell, memory, optimization, sequential depth, or CPU usage summaries. With no arguments, the default is to report the current fault summary and the current pattern summary.

Examples

```
TEST> set_faults -fault_coverage
TEST> report_summaries faults
      Uncollapsed Fault Summary Report
```

```
-----
fault class          code #faults
-----
Detected              DT   144501
Possibly detected     PT    4224
Undetectable          UD    1634
ATPG untestable       AU    9335
Not detected          ND    6811
-----
```

```
total faults 166505
test coverage 91.53%
fault coverage 86.78%
-----
```

```
TEST> report_summaries patterns
      Pattern Summary Report
```

```
-----
#internal patterns          974
  #basic_scan patterns      950
  #fast_sequential patterns  24
-----
```

```
TEST> report sum memory
Memory usage summary: total=102.60MB
```

```
TEST> set_faults -summary verbose
TEST> report_summaries
      Collapsed Fault Summary Report
```

```
-----
fault class          code #faults
-----
Detected              DT   28689
  detected_by_simulation DS   (1822)
  detected_by_implication DI  (26867)
Possibly detected     PT    617
  not_analyzed-pos_detected NP  (617)
Undetectable          UD    976
  undetectable-unused   UU     (8)
  undetectable-tied     UT   (505)
  undetectable-blocked  UB   (463)
ATPG untestable       AU    6795
  atpg_untestable-not_detected AN  (6795)
Not detected          ND  102341
  not-controlled        NC   (6171)
  not-observed          NO  (96170)
-----
```

```
-----
total faults 139418
test coverage 20.95%
-----
```

Pattern Summary Report

```
-----
#internal patterns      25
  #basic_scan patterns  25
-----
```

```
TEST> report_summaries primitives
      Gate Summary Report
```

```
-----
#primitives      101071
#primary_inputs   251
#primary_outputs  148
#primary_bidis    128
#DLATs           227
  #TLAs          195
  #nonscan        32
#DFFs            11369
  #nonscan        416
  #scan           10953
#BUSs            577
  #contention_fails 289
#BUFs            4108
#INVs            6425
#ANDs            9740
#NANDs           21086
#ORs             5689
#NORs            2981
#XORs            2207
#XNORs           944
#TIE0s           11357
#MUXs            20017
#WIRES           23
#BUFZs           138
#TSDs            3656
-----
```

```
TEST> report_summaries library
      Library Cells Report
```

```
-----
Cell name used
-----
```

```
an2p1      2589
an2p2       111
an8p1        58
aoi211p1    376
fdrtpl     4416
na4p1         8
```



```

no2p1      4593
no7p1       4
no8p1      13
oai31p1     82
or2p8      16
or7p2       1
or8p1      53
sfdrstp1   134
-----

```

```

TEST> report_summaries optimizations
      Optimizations Report

```

```

-----
optimization #occurrences #primitives #pins #modules
type          eliminated  lost   optimized
-----
unused gates 105129        105129    759      74
tied gates   0             24       0       0
buffers      220542        220542     0     560
inverters     64131        64131     0     209
cascaded gates 808         808       0       6
SWs as BUFs   24           0       0       1
DLATs as BUFs 0           0       0       0
MUXs          22061        109195    45      33
XORs          0           0       0       0
equiv. DLAT/ DFF 43         0       0      10
DLATs as DFFs 0           0       0       0
DFFs as DLATs 0           0       0       0
BUS keepers   0           0       0       0
feedback paths 0           0       0       0
-----
total         412738        499829    804     590
-----

```

```

TEST> report_summaries sequential_depths
type    depth gate_id
-----

```

```

Control    7    91788
Observe    5    1496
Detect    10    1496

```

```

TEST> report_summaries cpu_usage
      CPU Usage Summary Report

```

```

-----
Read netlist CPU time      1.26
Model build CPU time       4.55
DRC CPU time               0.81
Atpg CPU time              106.28
Pattern compression CPU time 0.00
Fault Sim CPU time         76.07
Good Sim CPU time          1.13

```

```

Diagnosis CPU time          0.00
Analyses CPU time          0.00
Total session CPU time      192.23
-----

```

```

BUILD> report_summaries faults -launch A
# Uncollapsed Stuck Fault Summary Report
# From: Clock Domain A
# To: Clock Domain A
# -----
# fault class          code #faults
# -----
# Detected             DT    603
# Possibly detected    PT     32
# Undetectable         UD     10
# ATPG untestable      AU      2
# Not detected         ND      3
# -----
# total faults 650
# test coverage 96.72%
# -----
(Report continues for each clock domain.)

```

```

BUILD> report_summaries f -capture B
# -----
# From: Clock Domain A
# To: Clock Domain B
# -----
# -----
# fault class          code #faults
# -----
# Detected             DT     8
# Possibly detected    PT     2
# Undetectable         UD     1
# ATPG untestable      AU     0
# Not detected         ND     1
# -----
# total faults 12
# test coverage 81.92%
# -----
(Report continues for each clock domain.)

```

```

BUILD> report_summaries f -per_clock_domain
# Uncollapsed Stuck Fault Summary Report
# From: Clock Domain A
# To: Clock Domain A
# -----
# fault class          code #faults
# -----
# Detected             DT    603
# Possibly detected    PT     32
# Undetectable         UD     10
# ATPG untestable      AU      2

```

```
# Not detected      ND      3
# -----
# total faults 650
# test coverage 96.72%
# -----
(Report continues for each clock domain.)
```

See Also

[Coverage Calculations](#)

[Fault Classes](#)

[report_faults](#)

[report_primitives](#)

[report_patterns](#)

[set_faults](#)

[Understanding Flattening Optimization](#)

[Understanding Summaries Report](#)

report_timing

Overview

This command generates a report of the pin slack data used in small delay defect testing in TestMAX ATPG.

Syntax

```
report_timing
  < instance_name | primitive_id | -all >
  [-max number]
```

Arguments

instance_name

Specifies an instance name used for reporting pin slack data. You can use a wildcard character with the instance name (see [Examples](#)).

primitive_id

Specifies the [primitive ID](#) used for reporting pin slack data.

-all

Reports pin slacks for all gates.

-max number

Specifies the maximum number of gates reported when using a wildcard character with an instance name. To report a specific number out of all instances, use the `report_timing * -max number` command.

Description

This command generates a report of the pin slack data used in small delay defect testing in TestMAX ATPG. Note that slack data is not reported for pins without names.

For complete information on using pin slack data, see [Small Delay Defect Testing](#).

In the following set of examples, the first field in the generated reports is the pin pathname of the node on which the slack is applied. The second field is the rising transition slack. The third field is the falling transition slack. Also note that `inf` is printed to indicate the infinite slack.

Examples

```
TEST-T> report_timing SO1
Pin name Rise Slack Fall Slack
-----
SO1      1.21      1.33

TEST-T> report_timing SO*
Pin name Rise Slack Fall Slack
-----
SO0      0.81      0.90
SO1      1.21      1.33
```

SO2	1.14	1.09
SO3	1.27	1.37
SO4	0.92	1.06
SO5	1.54	1.62

TEST-T> report_timing SO* -max 4

Pin name	Rise	Slack	Fall	Slack
-----	-----	-----	-----	-----
SO0	0.81	0.90		
SO1	1.21	1.33		
SO2	1.14	1.09		
SO3	1.27	1.37		

See Also
[read_timing](#)

report_toggle_weights

Overview

This command prints a report of all non-default toggle weights applied using the `set_toggle_weights` command.

Syntax

```
report_toggle_weights
```

Arguments

This command has no arguments.

Description

This command prints a report of all non-default toggle weights applied using the `set_toggle_weights.htm` command.

Examples

```
DRC-T> set_toggle_weights path/to/my/FF -weight 5 -shift -capture
DRC-T> report_toggle_weights
Non-default Toggle Weights:
Shift Weights:
a_reg_2_: 5
Capture Weights:
a_reg_2_: 5
```

See Also

[set_toggle_weights](#)

report_version

Overview

This command reports the current version of TestMAX ATPG or the virtual address space in which TestMAX ATPG is executing.

Syntax

```
report_version  
  [-full | -short | -address]  
  [-banner]  
  [-verbose]
```

Arguments

-full

Generates the full version report. This consists of the short version string, the date, and the time of day stamp of the currently running version.

-short

Generates the short version report. This consists of a letter code and the date string of the form YYMMDD.

-address

Reports whether the executable is using 32-bit or 64-bit virtual address space. A 32-bit executable can use at most 2.1 to 3.8 GB of virtual memory, depending on the platform. So, your design plus libraries plus patterns must all fit in this range of memory. A 64-bit executable does not have this limit and is typically limited by the size of physical RAM plus swap space of the workstation on which it is running.

-banner

Generates the product name and the full version report. This consists of the product name (TestMAX ATPG), short version string, the date, and the time of day stamp of the currently running version.

-verbose

Generates the product name and the full version report (same as the `-banner` option), followed by the date version for subsystem tasks.

Allowed Command Modes

All

Description

This command reports the current version of TestMAX ATPG or the virtual address space in which TestMAX ATPG is executing. With no arguments, the main version string is displayed.

Examples

```
BUILD-T> report_version
```

```
H-2013.03-SP5-CS1-tcl

BUILD-T> report_version -full
H-2013.03-SP5-CS1-i131004_180753-tcl

BUILD-T> report_version -short
i131004-tcl

BUILD-T> report_version -address
32bit executable

BUILD-T> report_version -banner
TestMAX ATPG (TM) H-2013.03-SP5-CS1-i131004_180753 -tcl

BUILD-T> report_version -release
Standard release code

BUILD-T> report_version -verbose
TestMAX ATPG (TM) H-2013.03-SP5-CS1-i131004_180753
util020306 edif102400 udp030603 otdd111798 veri091708 net031604 flat121611
stil032009 syn121400 vhd1120203 wgl1100308 api101507 vcde_102602
tpapi040700 lbist111207-tcl
```

See Also

[Version Report](#)

report_violations

Overview

Use this command to report rule violation data.

Syntax

```
report_violations
  <violation_id | rule_id | rule_type | -all>
  [-instance <name | id>] [-max d]
```

Arguments

violation_id

Reports only the rule violation with the selected violation ID.

rule_id

Reports all rule violations of the selected rule ID.

rule_type

Reports all rule violations of the selected rule type.

-all

Reports all rule violations.

-instance <name | id>

This option reports violations on a given instance. The instance can be specified with a name or a TestMAX ATPG primitive id. The name can also contain a wildcard character "*". This option can be combined with other options to filter out certain type of violations. For example, `report_violations S1 -instance scan_cell*` reports all S1 violations on instances with names that start with "scan_cell".

-max d

Specifies the maximum number of violations allowed to be reported. By default, there is no limit.

Allowed Command Modes

All

Description

Use this command to report rule violation data.

Examples

```
TEST> report_rules -fail
rule severity #fails description
----
C17  warning  3      clock connected to PO
C19  warning  1      clock connected to non-contention-free BUS
V7   warning  1      unsupported construct
Z4   warning  3      bus contention in test procedure
```

```
Z9    warning 2      bidi bus driver enable affected by scan cell
TEST> report_violations -all
Warning: Clock prun is connected to primary output D[15]. (C17-1)
Warning: Clock prun is connected to primary output D[14]. (C17-2)
Warning: Clock prun is connected to primary output D[13]. (C17-3)
Warning: Clock prun is connected to at least one non-contention-free BUS
(430). (C19-1)
Warning: Line 99 (job.spf), unsupported construct (Procedure "test_setup"
should be defined and is used as a Macro). (V7-1)
Warning: Bus contention on my_asic/Lic03 (373) occurred at time 0 of test_
setup procedure. (Z4-1)
Warning: Bus contention on my_asic/Lic03 (373) occurred at time 10 of test_
setup procedure. (Z4-2)
Warning: Bus contention on my_asic/Lic03 (373) occurred at time 20 of test_
setup procedure. (Z4-3)
Warning: Enable of bidi bus driver my_asic/Logic30 (73) connected to scan
cell gate (191). (Z9-1)
Warning: Enable of bidi bus driver my_asic/Logic32 (374) connected to scan
cell gate (199). (Z9-2)

TEST> report_violations c17
Warning: Clock prun is connected to primary output D[15]. (C17-1)
Warning: Clock prun is connected to primary output D[14]. (C17-2)
Warning: Clock prun is connected to primary output D[13]. (C17-3)

TEST> report_violations z4-2
Warning: Bus contention on my_asic/Lic03 (373) occurred at time 10 of test_
setup procedure. (Z4-2)
```

See Also

[Violations Report](#)
[analyze_violation](#)
[report_rules](#)
[set_rules](#)

report_wires

Overview

Use this command to report data associated with wire gates.

Syntax

```
report_wires
[id | -summary | -all ]
[-contention <fail | pass | abort>] [-max d]
[-verbose]
```

Arguments

id

Displays wire data for the selected [primitive ID](#). An error is reported if the object requested is not a wire gate.

-summary

Gives a summary of all wire gates.

-all

Displays wire data for all wire gates.

-contention <fails | pass | abort>

Reports all wire gates of the selected contention category.

-max d

Specifies the maximum number of wire gates allowed to be reported. By default, there is no limit.

-verbose

Displays verbose data for each reported wire gate.

Allowed Command Modes

DRC, Test

Description

Use this command to report data associated with wire gates.

Examples

```
TEST> report_wires -contention pass
      contention      zstate #drivers
gate_id status/capture status strong/weak behavior_data
-----
    79  pass ...          ...      4 0    clock
   206  pass ...          ...      5 0
```

```
TEST> report_wires -all
      contention      zstate #drivers
```

```
gate_id status/capture status strong/weak behavior_data
-----
    79  pass ...          ...          4 0    clock
    206  pass ...          ...          5 0

TEST> report_wires -all -verbose
      contention      zstate #drivers
gate_id status/capture status strong/weak behavior_data
-----
    79  pass ...          ...          4 0    clock
      inputs: 75 (BUF) 76 (BUF) 77 (BUF) 78 (BUF)
    206  pass ...          ...          5 0
      inputs: 203 (BUF) 204 (BUF) 205 (BUF) 201 (BUF) 202 (BUF)

TEST> report_wires -all -max 1
      contention      zstate #drivers
gate_id status/capture status strong/weak behavior_data
-----
    79  pass ...          ...          4 0    clock
```

See Also

- [Wires Report](#)
- [Wire Gate](#)
- [report_buses](#)
- [Primitive ID](#)

Run Commands

This section describes the following commands:

- [run atpg](#)
- [run build model](#)
- [run diagnosis](#)
- [run drc](#)
- [run fault sim](#)
- [run justification](#)
- [run observe analysis](#)
- [run simulation](#)
- [run testpoint analysis](#)

run_atpg

Overview

Use this command to generate ATPG patterns for current set of faults using the pattern source set by the `set_patterns` command.

Syntax

```
run_atpg
  [-auto_compression]
  [basic_scan_only | fast_sequential_only | full_sequential_only]
  [-distributed]
  [-jtag_lbist { seed_values pattern_count capture_cycles }]
  [-ndetects n]
  [-nodisturb_clock_grouping]
  [-observe_file file_name]
  [-only_chain_diagnosis]
  [-only_chain_test]
  [-optimize_patterns]
  [-random]
  [-rerun_for_timing_exceptions]
  [-resolve_differences]
```

Arguments

-auto_compression

Selects an algorithm that automatically produces compact patterns and high test coverage using minimal CPU time. This option also enables global testability analysis, including analysis for two-clock transition ATPG (specified by the `set_delay -two_clock_transition_optimization` command).

See "[Using Automatic Mode to Generate Optimized Patterns](#)" for more information. You should always use this option for both multicore ATPG and distributed ATPG, and all types of fault models and all design sizes.

Note the following:

- When using DFTMAX Ultra with the `-auto_compression` option, the log file includes two sets of pattern summary reports. You should refer only to the second set of summary reports.
- This option is ignored by the TestMAX ATPG multithreading engine. The only effect of this option is to add faults if the fault list is empty. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

basic_scan_only

Performs only Basic-Scan pattern generation. Basic-Scan must be specified to take advantage of the default two-clock transition ATPG when running distributed ATPG for system clock transition. This option is silently ignored by TestMAX multithreading. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

What does it want to say

fast_sequential_only

Performs only Fast-Sequential pattern generation. A `-capture_cycles > 1` should have been previously selected with the `set_atpg` command. If this is not the case, then a

setting of 4 is applied by default and this setting persists for subsequent use of the `run_atpg` command.

full_sequential_only

Causes only Full-Sequential pattern generation to be performed.

The Full-Sequential algorithm should have been previously enabled with the `-full_seq_atpg` option of the `set_atpg` command. If this is not the case, then this option is enabled and persists for subsequent use of the `run_atpg` command. This option switches to non-threaded processing. For more information, see [Multithreading Limitations](#).

-distributed

Runs ATPG in the distributed processing mode. Running distributed ATPG can greatly reduce your turnaround time for pattern generation. Use the `add_distributed_processors` command to identify the distributed processors. Note that this option cannot be used in conjunction with the `-optimize_patterns` option. This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-jtag_lbst { seed_values pattern_count capture_cycles }

This option computes the signature values for LogicBIST self-test. The values specified by this option are: number of seed values (always 1), pattern count, and number of capture cycles (usually 1). The maximum pattern count value is $(2^{\text{pattern_count_width}})-2$, which allows for an additional load-only pattern at the beginning of self-test. For more information, see "Using LogicBIST Self-Test" in the *DFTMAX Design-For-Test User Guide*.

-ndetects number (n)

Detects a fault up to and including n times. The default is one fault detection during ATPG. This option is not supported for full-sequential ATPG, full-sequential fault simulation, or distributed ATPG. It allows fault detections to be active until the fault has been detected n times. A fault detected less than d times is placed in the not analyzed, possibly detected (NP) fault class. When a fault is detected n times, it is then placed in the Detected by Simulation (DS) fault class, and removed from the active fault list.

Note the following:

- Fault reports are enhanced to display the number of times each fault was detected.
- You should specify a `set_atpg -decision random` command before running ATPG for best results. This will improve chances of detecting the fault in different ways, but does not guarantee it.
- TestMAX ATPG does not guarantee that each fault is detected in different ways.

-nodisturb_clock_grouping

Runs ATPG without disturb clock grouping. When running compression using the default X-tolerant compressor, you should rerun the ATPG a second time using this option to minimize the number of Xs captured in flip-flops.

-observe_file file_name

Used with the `-auto_compression` option, the `-observe_file` option reads the test point list (which can contain both observe and control points) written by the [analyze_test_points](#), [run_observe_analysis](#), or [run_testpoint_analysis](#) commands. This option shows you the potential benefit of executing ATPG after inserting these test points by proceeding as if the test points specified in this file were actually present in the design. The resulting test coverage and pattern count are representative of what to expect if the

design were changed. The pattern set is automatically deleted at the end of the ATPG session because the test point logic is yet to be added to the design to achieve such results.

-only_chain_diagnosis

Specifies ATPG to generate only X-tolerant chain test and capture patterns. See the description of the `-xtol_chain_diagnosis` option of the [set_atpg](#) command for more information on these patterns. When ATPG finishes pattern generation, the [M234](#) warning message is printed.

-only_chain_test

Specifies ATPG to generate chain test patterns only. When ATPG finishes the pattern generation, the [M234](#) warning message is printed.

For compression mode, the necessary padding patterns are also generated. The generated scan chain patterns include regular scan chain test patterns, augmented chain test patterns (for designs with X-tolerance) or quiet chain test patterns.

Note that if multicore ATPG and the `-only_chain_test` option are both specified, multicore ATPG runs one slave and stops when the chain test patterns are created.

-optimize_patterns

Selects an algorithm designed to produce a very compact pattern set with high test coverage. The trade-off is that this optimization increases the run time by dynamically adjusting the internal settings, such as abort limits, minimum detects, and merge limits during ATPG. This option enables you to use a single command instead of iterating multiple `run_atpg` commands and manually adjusting the various parameters.

The `-optimize_patterns` option will work with the `-chain_test`, `-coverage`, and `-patterns` options of the [set_atpg](#) command. This option also works with all power aware options of the [set_atpg](#) command. However, the power aware options might impact the effectiveness of the pattern optimization process.

Note the following:

- This option is not supported for distributed ATPG.
- This option generates two-clock ATPG patterns as basic scan patterns. But they are stored, read, and simulated as fast-sequential patterns. As a result, a fault simulation that uses two-clock ATPG patterns usually takes longer than the original ATPG run.
- If you are using a DFTMAX Ultra design, the `-optimize_patterns` option works, but for better quality of results, you should use the `-auto` option instead.
- See "[Optimizing Patterns During the run_atpg Process](#)" for more information on using this option.
- TestMAX ATPG ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-random

Specifies that random patterns with all possible clocking variations are to be used as the initial part of the Basic-Scan ATPG process before switching to deterministic patterns. You can only use this option when the pattern source is set to internal. The default is not to use random patterns. This option switches to non-threaded processing. For more information, see [Multithreading Limitations](#)

-rerun_for_timing_exceptions

This option causes the `run_atpg` command to run twice when timing exceptions are used. The first ATPG run is done normally and is not affected by this switch. The second ATPG run is made if there are single -through exceptions, or if `-case_paths` is enabled and the `set_`

`case_analysis` pin or port lists do not match ATPG or PI constraints. The second pass run adds ATPG constraints to prevent switch on these nodes. Patterns generated in the second pass are added to the total from the first pass. This option can increase the test coverage, but the runtime can be very long.

TestMAX ATPG ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-resolve_differences

This option is used only after performing per-cycle pattern masking of external patterns. It performs the fault simulation of masked external patterns and updates the patterns. The fault simulation of masked patterns could also be performed by using `run_fault_sim` (the recommended flow). The difference between both methods is that the `run_atpg -resolve_difference` command removes patterns that do not contribute to fault coverage after fault simulating them.

This option can be used if a large number of measures are masked using `set_patterns -external_patterns_file -resolve mask_file`. The drawback is that new failures could appear because of the pattern suppression. This is why it is recommended that you specify `run_simulation` after `run_atpg -resolve`. If new failures occur, you can mask them by executing another time `set_patterns -resolve_differences` command.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

Allowed Command Modes

Test

Description

You use this command to generate ATPG patterns for current set of faults using the pattern source set by the `set_patterns` command. This command requires Test-ATPG-Max and Test-FaultSim licenses.

Select the parameters for ATPG using the `set_atpg` command. Upon completion of the run, TestMAX ATPG automatically performs a `report_summaries` command. After an ATPG process, you can analyze faults that have detection problems with the `analyze_faults` command.

Examples

```
TEST-T> set_atpg -patterns 300 -merge low
TEST-T> run_atpg
ATPG performed for stuck fault model using internal pattern source.
-----
#patterns #faults          #ATPG faults test      process
stored    detect/active red/au/abort coverage CPU time
-----
Begin deterministic ATPG: #uncollapsed_faults=72025, abort_limit=10...
32         52907 19118          0/0/1  74.17%      17.11
64          6750 12366          1/0/1  81.90%      25.85
96          3005  9361          1/0/1  85.34%      33.38
128         1733  7628          1/0/1  87.32%      40.57
160         1360  6268          1/0/3  88.88%      47.22
192         1139  5129          1/0/4  90.18%      52.88
224         1016  4113          1/0/7  91.35%      58.21
256          718  3395          1/0/9  92.17%      63.27
```

```

288          662  2733          1/0/11 92.93%          67.56
Warning: ATPG terminated due to meeting pattern count limit. (M234)
300          192  2541          1/0/14 93.15%          69.13

```

Uncollapsed Stuck Fault Summary Report

```

-----
fault class          code #faults
-----
Detected             DT   81330
Possibly detected    PT     6
Undetectable         UD   1092
ATPG untestable      AU   3445
Not detected         ND   2535
-----

```

```

total faults 88408
test coverage 93.15%
-----

```

Pattern Summary Report

```

-----
#internal patterns          300
  #basic_scan patterns      300
-----

```

TEST-T> **run_atpg -random**

ATPG performed for stuck fault model using internal pattern source.

```

-----
#patterns    #faults    #ATPG faults test    process
stored      detect/active red/au/abort coverage CPU time
-----

```

Begin random patterns: capture_clock = CLK, observe_point = master...

```

25          31125 40898          49.30%  0.89
35           1983 38915          51.57%  1.53
53           3042 35873          55.04%  2.16
67           3579 32294          59.13%  2.77
81           2249 30045          61.70%  3.33
96            960 29085          62.79%  3.89
111          2529 26556          65.69%  4.40
122           285 26271          66.02%  4.87

```

Begin random patterns: capture_clock = NRES, observe_point = master...

```

131          877 25394 67.03%   5.33
132           3 25391          67.03%   5.78

```

Begin random patterns: capture_clock = none, observe_point = master...

```

139          47 25344          67.06%   6.24

```

Begin deterministic ATPG: #uncollapsed_faults=25344, abort_limit=10...

```

169          12907 12437          0/0/1    81.81%  16.00
201           3927  8510          0/0/1    86.31%  22.98
233           1956  6554          0/0/2    88.55%  29.27
265           1237  5317          0/0/4    89.97%  34.83
297           1087  4230          0/0/5    91.21%  39.68

```

Warning: ATPG terminated due to meeting pattern count limit. (M234)

```

300           81  4149          0/0/7    91.31%  40.16

```

Uncollapsed Stuck Fault Summary Report

```

-----
fault class      code  #faults
-----
Detected         DT    79722
Possibly detected PT     5
Undetectable     UD   1092
ATPG untestable  AU   3445
Not detected     ND   4144
-----

```

```

total faults      88408
test coverage     91.31%
-----

```

Pattern Summary Report

```

-----
#internal patterns      300
#basic_scan patterns    300
-----

```

```

TEST-T> set_atpg -capture_cycles 4 -abort_limit 50 -merge high
TEST-T> run_atpg

```

```

*****
* NOTICE: The following DRC violations were previously *
* encountered. The presence of these violations is an *
* indicator that it is possible that the ATPG patterns *
* created during this process can fail in simulation. *
* *
* Rules: N20 *
*****

```

```

ATPG performed for stuck fault model using internal pattern source.
-----

```

```

#patterns #faults      #ATPG faults test      process
stored    detect/active red/au/abort coverage CPU time
-----

```

```

Begin deterministic ATPG: #uncollapsed_faults=214, abort_limit = 50...

```

```

9          47 167          0/0/0      30.22%    0.01

```

```

Begin fast-seq ATPG: #uncollapsed_faults=167, abort_limit=10, depth=4... 10

```

```

56 111 0/0/0 51.12% 0.02

```

```

11          39 72          0/0/0      65.67%    0.02

```

```

12          16 56 0/0/0      71.64%    0.02

```

```

13           1 55 0/0/0      72.01%    0.02

```

```

14           3 52          0/0/0      73.13%    0.02

```

```

15           7 45          0/0/0      75.75%    0.02

```

```

16           6 39          0/0/0      77.99%    0.02

```

```

17           4 35          0/0/0      79.48%    0.02

```

```

18          18 17          0/0/0      86.19%    0.02

```

```

19  1   16          0/0/0      86.57%    0.03

```

```

20           3 13          0/0/0      87.69%    0.03

```

```

21  4    9          0/0/0      89.18%    0.04

```

```

22           3 6          0/0/0      90.30%    0.04

```

23	1	5	0/0/0	90.67%	0.04
24	1	4	0/0/0	91.04%	0.04
25	2	2	0/0/0	91.79%	0.04
26	1	1	0/0/0	92.16%	0.04
27	1	0	0/0/0	92.54%	0.04

Uncollapsed Fault Summary Report

fault class	code	#faults
Detected	DT	248
Possibly detected	PT	0
Undetectable	UD	2
ATPG untestable	AU	20
Not detected	ND	0

total faults 270
test coverage 92.54%

Pattern Summary Report

#internal patterns	27
#basic_scan patterns	9
#fast_sequential patterns	18

TEST-T> **run_atpg -auto_compression -observe_file Design_Observe_points.tpl**

Fanout-free region identification completed: #regions=32661, CPU time=0.12 sec.

228 observe points were successfully read in.

ATPG performed for stuck fault model using internal pattern source.

#patterns simulated	#patterns eff/total	#faults detect/active	#ATPG faults red/au/abort	test coverage	process CPU time
Begin deterministic ATPG: #uncollapsed_faults=424527, abort_limit=1					
32	32 0	281819 142708	0/0/0	71.61%	1.40
64	32 0	32843 109864	1/0/0	77.78%	2.10
96	32 0	16862 93002	1/0/0	80.94%	2.73
128	32 0	14016 78980	3/1/0	83.58%	3.32
160	32 0	10204 68764	7/2/0	85.49%	3.89

ATPG performed for stuck fault model using internal pattern source.
Fast-sequential simulation is used to verify Basic-Scan patterns.

Uncollapsed Stuck Fault Summary Report

fault class	code	#faults
-------------	------	---------

```

Detected                DT    521464
detected_by_simulation  DS    (421796)
detected_by_implication DI    (99668)
Possibly detected       PT          3
atpg_untestable-pos_detected AP    (3)
Undetectable            UD    2858

undetectable-tied       UT    (121)
undetectable-blocked    UB    (454)
undetectable-redundant  UR    (2283)
ATPG untestable         AU    8510
atpg_untestable-not_detected AN    (8510)
Not detected            ND    439
not-controlled          NC    (73)
not-observed            NO    (366)
-----
total faults            533274
test coverage           98.31%
-----
Pattern Summary Report
-----
#internal patterns      311

#basic_scan patterns    311
-----
CPU Usage Summary Report
-----
Testable analysis CPU time    70.90
Test generation CPU time      702.04
Pattern compression CPU time   28.72
Total CPU time                 802.20
-----
Warning: Internal pattern set is now deleted. (M133)

```

See Also

[Automatic Mode of Run ATPG](#)
[Aborted Faults](#)
[add_faults](#)
[analyze_faults](#)
[report_faults](#)
[report_summaries](#)
[set_atpg](#)
[set_patterns](#)
[set_random_patterns](#)
[Test Coverage](#)
[Understanding the Run ATPG output](#)

run_build_model

Overview

This command builds the in-memory simulation model from the design modules that have been read in.

Syntax

```
run_build_model  
  [top_module]  
  [-weakgates]  
  [-remove_pio_pull]
```

Arguments

top_module

Specifies the name of the module from which to create the simulation model. The default is to use the last unreferenced module encountered while reading the netlist. If there is more than one unreferenced module, the one defined last is used and a warning message is issued indicating the choice.

-weakgates

Specifies that when a non-three-state driver occurs on the same net as one or more three-state drivers, the non-three-state driver is considered to have weak drive strength. The default is not to assume a weak drive under these conditions and to issue a B18 violation.

-remove_pio_pull

Removes pull-type connections from PIO gates. A pull-type connection is a weak input to a BUS gate driven by TIE1/TIE0 or by a TSD (tri-state driver) or a SW (switch) gate whose data input is driven by TIE1/TIE0. Only pull-type connections of BUS gates connected to a PIO gate are removed.

Allowed Command Modes

Build DRC

Description

This command builds the in-memory simulation model from the design modules that have been read in. You can use the `set_build` command to control the build process. After the build process is completed, TestMAX ATPG performs a learning process, which you can control with the `set_learning` command. If a simulation model already exists, it is automatically deleted before the new model is built.

Examples

```
BUILD> run_build_model asic_top  
-----  
Begin build model process on topcut = asic_top ...  
-----  
Warning: Rule B6 (undriven module inout pin) failed 2 times.  
Warning: Rule B7 (undriven module output pin) failed 178 times.  
Warning: Rule B8 (unconnected module input pin) failed 923 times.
```

```
Warning: Rule B13 (undriven instance pin) failed 2 times.  
End build model: #primitives=101004, CPU_time=8.12 sec, Memory=34702381  
-----  
Begin learning analyses...  
End learning analyses, total learning CPU time=12.3  
-----
```

See Also[set_learning](#)[set_build](#)[Understanding the run_build_model output](#)

run_diagnosis

Overview

You use this command to perform a diagnosis of a failing device using an input file that identifies its failing tester measures.

Syntax

```
run_diagnosis
  file_name
  [-assume_chain_defect < <stuck|slow|fast|all> chain_name >]
  [-chain_failure]
  [-display]
  [-file {file_list}]
  [-find_offset {begin end}]
  [-noreport_defects]
  [-only_report_failures]
  [-rank_faults]
  [-streaming_report_chains_only filename]
  [-truncate pattern#]
  [-verbose]
  [-write_fault_list out_file [-replace] [-compress <bin | gzip>]]
```

Arguments

file_name

Specifies the name of the failure data file. When parallel diagnostics is enabled by the `set_diagnosis -number_processes` command, you can specify a list of datalogs, as shown in the following example:

```
run_diagnosis [list {datalogs/ff_[1-9].log} \
  {datalogs/ff_[1-9][0-9].log} \
  {datalogs/ff_100.log}]
```

Note that wildcards are also accepted when specifying datalogs, as shown in the following example:

```
run_diagnosis datalogs/ff_*.log
```

For more information on specifying a list of datalogs, see "[Parallel Diagnosis](#)."

-assume_chain_defect < <stuck|slow|fast|all> chain_name >

This option and set of arguments enforce the execution of scan chain diagnosis for a particular defect type and scan chain. There are two mandatory arguments. The first, `<stuck|slow|fast|all >`, is the defect type. The second, `chain_name`, is the chain on which the faults should be ranked. Only one defect type and chain name can be specified at a time.

At the completion of the command, the tool reports the top candidates for each fault type. If the "stuck" type is specified, then all stuck-at types (stuck-at-0, stuck-at-1) are ranked. If the "slow" or "fast" types are specified, then all slow or fast types (slow-to-rise/fall/rise-fall and fast-to-rise/fall/rise-fall) are ranked. If "all" is specified, then all defect types for the specified scan chain are ranked.

The `-assume_chain_defect` option overwrites the `-auto` option. It cannot be used on the same command line with the `-chain_failure` or `-rank_faults` options.

If there is unexplained failure for all simulated candidates, then following message is printed:

```
Warning: Insufficient data to locate <fault_type> fault in chain
chain_name
```

-chain_failure

If your patterns show failures during the chain test, then you might have a device failure that prevents the proper loading and unloading of the scan chains. Use the `-chain_failure` option locate the scan chain and bit position at which the flaw most likely occurs. This option adjusts the diagnostic algorithm to troubleshoot scan chain shifting instead of assuming the data load and unload work fine. For best results, failure data from ten or more patterns is required.

The diagnostic algorithm identifies the location of stuck-at, slow-to-rise, slow-to-fall, fast-to-rise or fast-to-fall. The latter two types cover hold time problems affecting the scan chain shift operation. If pattern 0 is not the chain test, a simple chain diagnosis algorithm with lower accuracy is used.

The output displays a set of possible defect locations (chain, cell position, and instance name), along with a score, which indicates the confidence of each location. This score is a percentage measuring how much the failures seen on the tester matches a simulated chain defect at that location. It also shows the predicted type of defect. For example:

```
fail.log scan chain diagnosis results: #failing_patterns=79
-----
defect type=fast-to-rise
match=100% chain=c0 position=178 master=CORE/c_rg0 (46)
match=100% chain=c0 position=179 master=CORE/c_rg2 (57)
match= 98% chain=c0 position=180 master=CORE/c_rg6 (54)
```

-display

Displays the diagnosis results schematically, unless you use the `-chain_failure` option. It does not support graphic display of failures in the chain test patterns. For patterns other than chain test patterns, it displays each potential failure site along with the fault codes to indicate the fault positions associated with the failure. The fault code is always "DS/--" or "--/DS", indicating a stuck-at-0 or stuck-at-1 detection location, respectively.

Note: The primary output display mechanism affects the fault codes displayed on PO primitives. As a result, the faults codes displayed near PO primitives might be incorrect. To confirm the values shown in the schematic, refer to the values in the transcript.

[-file {file_list}]

When using the split pattern feature in Tcl mode, this option indicates the list of the second and following failure log files. The specification must be a valid Tcl list. For example:

```
TEST-T> run_diagnosis fail1.log -file {fail2.log fail3.log ...} [options]
```

The `run_diagnosis` report includes a set of lines that display each pattern file characteristic, as in the `set_pattern ext <file> -split` report. If a failure log file contains failures, another line displays the cycle offset and truncation options values. The line is highlighted in bold in the example shown below:

```
TEST-T> run_diagnosis split_fail/fail_stil_0.txt \
```

```

-file {split_fail/fail_stil_1.txt \
split_fail/fail_stil_2.txt \
split_fail/fail_stil_3.txt}
; Pattern_file=prog.stil_0.gz, basic_scan=[0(3),4(66)], fast_sequential=
[-1,-1]
Cycle_offset=0, truncate=0/0, incomplete_failures=0, failure_memory_
limit=0
  Pattern_file=prog.stil_1.gz, basic_scan=[5(82),9(146)], fast_
sequential=[-1,-1]
  Pattern_file=prog.stil_2.gz, basic_scan=[10(162),14(226)], fast_
sequential=[-1,-1]
Cycle_offset=159, truncate=3/13, incomplete_failures=0, failure_memory_
limit=0
Warning: 0 failure(s) were read, 6 failure(s) were ignored. (M267)
  Pattern_file=prog.stil_3.gz, basic_scan=[15(242),16(258)], fast_
sequential=[-1,-1]
Cycle_offset=239, truncate=0/0, incomplete_failures=0, failure_memory_
limit=0

```

To avoid generating a long report, only the first failure log file containing valid failures is reported — even if the diagnosis is performed using several failure log files. You can retrieve all the file names using the "name" attribute from the collection returned by [get_diag_files](#) -all Tcl API command. If you use this command in conjunction with the previous example, the diagnosis report will indicate the following statement:

Diagnosis summary for failure file split_fail/fail_stil_0.txt

Also note the following:

Cycle_offset=NBR means that the first cycle number of the patterns contained in the split pattern read is cycle number NBR.

truncate=A/B means that the .truncate value in the failure log file is A and it corresponds to pattern number B.

incomplete_failures is equal to 0 if .incomplete_failures is not set and 1 if it is set.

failure_memory_limit=NBR is an echo of value of .failure_memory_limit directive.

See [Split Pattern Diagnosis](#) for more information on the split pattern flow.

See [Using Split Datalogs to Perform Parallel Diagnosis for Split Patterns](#) for more information on the split pattern flow for parallel diagnostics.

-find_offset {begin end}

This option suppresses normal diagnostics reporting and instead finds and prints a specific cycle offset in the search range between the *begin* and *end* parameters. When this offset value is added to the cycle-based failures in the failure data file (specified by *file_name* parameter of the `run_diagnosis` command), it allows the expected data for all failures to match the expected responses in the external pattern set. The failure data file must contain expected data for each failure, and there must be at least 10 failures to use this option. If the patterns run on the tester have not been modified, the cycle offset is usually 0 (the default) or 1 (if the tester counts cycles starting at 0.) However, if additional cycles have been added before or while running on the tester, you will need to specify the correct cycle offset (usually a negative value) so diagnosis can map cycle numbers in the failure data file to the cycle numbers of the external pattern set used for diagnosis.

The cycle offset reported with this option should be specified using the `set_diagnosis - cycle_offset d` option prior to all subsequent diagnosis runs using the same external pattern set.

-noreport_defects

This option suppresses printing the full diagnostics report for flows that always use the [report_defects](#) command for class-based diagnostics.

-only_report_failures

This option suppresses diagnostics reporting and prints only the failing measures. Do not use this option for class-based diagnostics (triggered by the [set_diagnosis - organization class](#) command). Instead, use the `set_diagnosis -verbose` command. In the case of normal scan patterns, the failing measures report looks like the following:

```
Report all failures: #failures=2
-----
-----
pattern  chain/pin_name  position  gate_id
-----
3         4              2         654
4         4              1         655
```

The descriptions of the fields are as follows:

`pattern` is the pattern number.

`chain/pin_name` indicates the chain name where the failure occurs when the failure could be mapped back. This column indicates the pin name in case of failure in capture cycle or if the failure could not be mapped back to a scan cell.

`position` is the scan cell position where the failure occurs. If the failure was in capture cycle, it is -1.

`gate_id` is the gate id of the failing scan cell or a scanout.

For Adaptive Scan, an additional column called "`type`" indicates if the failure has been successful (*unique*) or not (*unmapped*) at mapping back to a single scan cell.

A third mapping type, called *potential*, can be displayed. This type is displayed if more than one scan cell explains a particular set of external failures for a particular shift position. In summary, one failure maps to different chains. As an example, say there is one failure for pattern 7, and the diagnosis cannot determine the exact source of the failure. Then the diagnosis indicates that the failure observed on pattern 7 is coming potentially from chains 6, 81 or 90. The following report is printed:

```
-----
pattern chain/pin_name position  gate_id type
-----
7 6 131 636837 potential
7   81 131 659098 potential
7 90 131 665290 potential
```

The diagnosis engine then assumes one of these flip-flops is failing and tries to find the best candidate.

Another example is as follows:

```

-----
pattern chain/pin_name position gate_id type
-----
7      48              1      62063    unique
26     out1            -1      62084    unique
33     o_tdata_dd[5]  12      62076    unmapped

```

The mapping report printed by this option and the report printed by [set_diagnosis -mapping_report](#) are the only complete reports. The first mapping report printed at the beginning of the diagnosis report looks like the following example and is not intended to be completed. See "Understanding run_diagnosis Output" for more details.

```

-----
pattern chain pos# output pin_names
-----
9 1 4 test_so1 test_so2 test_so3
14 1 3 test_so1 test_so2 test_so3

```

-rank_faults

Ranks a set of faults based on how well they match the defective device response on the tester. The supported fault models are stuck-at, transition, bridging, and IDDQ.

TestMAX ATPG simulates the collapsed faults (indicated by the DS code in the diagnosis report) in the active fault list (added by the `add_faults` or `read_faults` command) and using the Fast-Sequential simulator, scores the faults, and displays the match score. As the fault list is typically small, this run should be relatively quick.

TestMAX ATPG simulates only a subset of the passing patterns as with normal diagnosis, but you can control it with the `-sample` option of the `set_diagnosis` command.

Note: TestMAX ATPG ignores most of the other diagnosis option when you use the `-rank_faults` option because you are not running diagnosis. The following is an example flow:

```

run_drc
set_patterns -external pattern.bin.gz

# write out fault candidates into file callouts.txt
run_diagnosis fail.txt -write callouts.txt -replace

# read the faults back in
read_faults callout.txt

#run fault simulation to rank the fault
run_diagnosis failures.txt -rank_faults

```

TestMAX ATPG uses the following formula for ranking the faults:

$$\text{match \%} = \frac{\text{TFSF}}{\text{TFSF} + \text{TFSP} + \text{TPSF}} \times 100$$

Where (use the `-verbose` option to display these parameters):

TFSP—tester fail, simulator fail	Number of measures that failed on the tester and the fault candidate was detected in the simulator at the same location and pattern.
TFSP—tester fail, simulator pass	Number of measures where the tester failed, but the fault candidate was not detected in the simulator at the same location and pattern.
TPSF—tester pass, simulator fail	Number of measures where the fault candidate was detected, but the tester measure passed at the same location and pattern.

`-streaming_report_chains_only filename`

Reports chain test failures used for performing chain diagnostics using DFTMAX Ultra compression. For example:

```
run_diagnosis failure_log_file1.log -streaming_report_chains_only chain_fail_report.txt
```

For more information on using this option, see [Chain Diagnosis for Multiple Chain Defects](#).

`-truncate pattern#`

Specifies the last pattern applied in obtaining the failures. Use this option when the patterns applied on the tester were not the complete set. This tells TestMAX ATPG to stop its analysis at a similar point for its copy of these patterns. The tester failure file should contain no patterns greater than the pattern number indicated as the truncation point, or else an error is reported. The default is the last pattern available in the pattern set. This option should not be used with the `-incomplete_failures` option of `set_diagnosis`. This option specifies that the tester failure memory limitations might have caused the failure log to have only partial failure data for the last failing pattern reported in the log.

Therefore, the last failing pattern and all subsequent patterns are ignored during diagnosis.

The behavior of this option is unchanged in the case of the split pattern feature. This option still specifies a single pattern range to ignore at the end of the pattern set. The specified pattern number can be in any of the pattern files.

`-verbose`

Causes the diagnostics report to include additional detailed information such as observable points and an explained patterns list. For more information on this detailed information, see the "Verbose Format" section of the "[Understanding the Diagnosis Report](#)" topic. Do not use this option for class-based diagnostics (triggered by the `set_diagnosis -organization class` command). Instead, use the `set_diagnosis -verbose` command.

Allowed Command Modes

Test

Description

You use this command to perform a diagnosis of a failing device using an input file that identifies its failing tester measures. Note that you use the `set_diagnosis` command to make specifications relevant to this command.

The diagnosis assumes that the patterns were originally generated by TestMAX ATPG; these patterns must be available during the diagnosis. You must specify the pattern set as external using the `set_patterns` command. For a selected fault, use the `run_simulation` command to create a representative failure file for diagnosis.

TestMAX ATPG does not consider fault sites that are no-faulted as potential defect sites. If you have examined a site and not found any physical defect, use this command to identify additional sites that might explain the tester failures. Use the `add_nofaults` command before executing the `run_diagnosis` command. Use the `remove_nofaults` command consider fault sites as potential defect sites.

Note: The `run_diagnosis` command does not support diagnosis on pattern sets containing full-sequential patterns.

Examples

```
TEST-T> run_diagnosis /project/mars/lander/chipA_failure.dat -display
Diagnosis summary for failure file /project/mars/lander/chipA_failure.dat
#failing_pat=2, #failures=2, #defects=1, #faults=2, CPU_time=0.05
Simulated : #failing_pat=2, #passing_pat=35, #failures=2
-----
---
Fault candidates for defect 1: stuck fault model, #faults=2, #failing_
pat=2, #passing_pat=35, #failures=2
-----
---
match=100.00%, #explained patterns: <failing=2, passing=35>
sa1 DS de_encrypt/C264/U36/O (L434ND)
sa0 -- de_encrypt/C264/U36/I1 (L434ND)
sa0 -- de_encrypt/C264/U36/I2 (L434ND)
sa0 -- de_encrypt/C264/U28/O (L434ND)
sa1 -- de_encrypt/C264/U26/I2 (L434ND)
-----
---
match=50.00%, #explained patterns: <failing=1, passing=35>
sa1 DS de_encrypt/C264/U28/I1 (L434ND)
```

Example failure data file used with the `-check_expected_dataoption`:

```
5 vga_out0 1 # nonscan output, expecting a 1
8 scanout5 34 0 # scan output, expecting a 0
```

The following is a sample output of the `run_diagnosis` command using the `cycle_offset` <integer> and the verbose with the cycle-based failure log file. Comments indicate the failure cycle used to generate the pattern-based failure. It also shows whether the cycle was a capture or a shift cycle.

```
4 po0 # Cycle conversion from cycle 34; fail in capture
4 so 2 # Cycle conversion from cycle 38; fail in shift
```

The following is an example flow:

```
run_drc ...
set_patterns -external pat.stil
```

```
set_diagnosis -cycle_offset -1
run_diagnosis fail.log
```

Fault Ranking Example

The following is an example output of fault ranking.

```
#failing_pat=2, #failures=4, #faults=2, CPU_time=0.00
Simulated : #failing_pat=2, #passing_pat=12 #failures=4
-----
match=25.00%, #explained patterns: <failing=1, passing=2>
sa1 DS i1/out
```

In this example:

The first part displays:

- The number of failing patterns on the tester
- The number of failures on the tester
- Number of faults simulated and ranked
- CPU time used for simulation

The second part displays the data related to the fault simulation such as:

- Number of failing patterns simulated
- Number of passing patterns simulated
- Number of failing measures across all patterns
- The match score
- Number of failing and passing patterns that the fault explains
- The fault itself.

To display the verbose output, use the `-verbose` option.

```
#failing_patterns=3, #failures=4, #faults=5, CPU=-0.00
#simulated patterns: failing=3, #passing=2 #failures=4
-----
match=25.00%, (TFSF=1/TFSP=3/TPSF=0), #perfect/partial match: <failing=1/1,
passing=2>
sa1 NO i1/out
sa0 -- i1/in
-----
```

This shows additional information such as:

- TFSF, TFSP, TPSF values
- Number of explained failing patterns with an exact or perfect match; shown before the `/`.
- Number of explained failing patterns with a partial match; shown after the `/`.
- Number of patterns that passed on the tester
- Number of patterns that did not detect the fault in simulation.

See Also

[add_faults](#)
[Failure Data Files](#)
[read_faults](#)
[read_layout](#)
[set_delay](#)

[set_diagnosis](#)

[Understanding the Diagnosis Report](#)

[write_image](#)

run_drc

Overview

This command performs design rule checking (DRC).

Syntax

```
run_drc  
  [file_name]  
  [-append]  
  [-dft]  
  [-patternexec exec_name]  
  [-prescan]  
  [-test | -delete]
```

Arguments

file_name

Specifies the name of the STIL procedures file that defines the scan chains and test procedures. If not specified, the file name specified in the most recent `set_drc` command is used. Any file used becomes the default for future `run_drc` commands. The syntax of the STIL procedures file is a limited subset of the IEEE STIL language.

-append

Applies the DRC operation based on information specified from command line settings (the same as if the `run_drc` command was performed without a specified test protocol file). A test protocol filename must be specified with this option, however only the MacroDefs and Procedures sections of this file are included in the applied data.

The test protocol file provides the capability to define the `test_setup` and/or the `load_unload` procedure in a flow that would otherwise use command line statements to establish the design information. As a result, you can override the defaults established for these functions when command line operations are used exclusively. The test protocol file should only contain either one or both of the MacroDefs and Procedures blocks; any other data in this file is ignored. These blocks should contain the following statements at the beginning of the line (no other names should appear on these blocks): "`Procedures {`" and "`MacroDefs {`".

If the `MacroDefs` block is specified, it must include the definition of the `test_setup` procedure. If the `Procedures` block is specified, it must include the definition of the `load_unload` procedure, and may also define definitions of capture procedures.

-dft

This option, which is applicable to DRC mode only, causes TestMAX ATPG to check post-scan DRC rules (includes all DRC checks except for D rules). TestMAX ATPG returns to DRC mode after checking, regardless of whether the rule check is successful.

-patternexec *exec_name*

Selects between different PatternExec blocks in the STIL procedure file. DRC can only run on one of the PatternExec blocks, so if the STIL procedure file has more than one, this

switch must be used. Failure to use this switch might cause DRC to run on only the last PatternExec block in the file.

-prescan

Causes TestMAX ATPG to check only pre-scan DRC rules (D rules). TestMAX ATPG returns to the DRC mode after the checking is complete. The prescan context assumes certain netlist environments and requires certain options to be set to for the checking to accurately take place. For instance, if this analysis is performed on a netlist before scan insertion, the command [set_scan_ability](#) on `-all` is required. Other netlist contexts might require additional switches.

-test | -delete

Indicates the type of design rule checking to be performed. If you select `-test` (the default), design rule checking is performed for TEST command mode. Upon successful completion, the command mode is set to TEST. The `-delete` option deletes all DRC data, and must be run in TEST mode. After the data is deleted, TestMAX ATPG returns to DRC command mode.

Allowed Command Modes

DRC

TEST

Description

This command performs design rule checking (DRC). You must perform DRC before entering the TEST command mode, which performs test generation and fault simulation. This command requires a Test-Analysis license. However, the `-prescan` and `-dft` options do not need a Test-Analysis license.

During DRC, TestMAX ATPG performs a variety of checks to ensure that the scan structure is correct and to determine how to use the scan structure for test generation and fault simulation. These checks include ensuring that the scan chains operate properly, identifying scan cells, identifying nonscan cell behavior, and ensuring that clocks obey the required rules.

You can define clocks, clock timing, tester cycle periods, output measure values, scan chains, and scan shift initialization both at the command line and by the appropriate sections of the DRC file. To avoid potential conflict, the following rules apply:

- When a DRC file is not specified, all items defined by commands are used as defined.
- When a DRC file is specified, any scan chains defined by command lines are deleted from the scan chain list, before processing the DRC file.
- When a DRC file is specified, any pin initialization defined by an `add_scan_enable` command is ignored.
- When a DRC file is specified and that file contains a `Timing` block, then clocks defined in that block override any timing defined by an [add_clocks](#) command.

Note the following:

- The `test` command can also be used to perform the DRC process.
- The DRC process uses parameters that are affected by both the `set_drc` and `set_atpg` commands. In particular, the `-abort_limit` option of the `set_atpg` command affects the effort of some DRC checks.
- A specific sequence of commands is required to [modify the timing data in an existing STIL procedure file](#).

Examples

```

DRC> run_drc slo_gin.spf
-----
Begin scan design rule checking...
-----
Begin reading test protocol file slo_gin.spf...
End parsing STIL file slo_gin.spf with 0 errors.
Test protocol file reading completed, CPU time=0.08 sec.
-----
Begin Bus/Wire contention ability checking...
Bus summary: #bus_gates=40, #bidi=40, #weak=0, #pull=0, #keepers=0
    Contention status: #pass=0, #bidi=40, #fail=0, #abort=0, #not_analyzed=0

    Z-state status : #pass=0, #bidi=40, #fail=0, #abort=0, #not_analyzed=0
Bus/Wire contention ability checking completed, CPU time=0.04 sec.
-----
Begin simulating test protocol procedures...
Nonscan cell constant value results: #constant0 = 4, #constant1 = 7
Nonscan cell load value results : #load0 = 4, #load1 = 7
Warning: Rule Z4 (bus contention in test procedure) was violated 12 times.
Test protocol simulation completed, CPU time=0.15 sec.
-----
Begin scan chain operation checking...
Chain c1 successfully traced with 31 scan_cells.
Chain c2 successfully traced with 31 scan_cells.
Chain c3 successfully traced with 31 scan_cells.
Chain c4 successfully traced with 31 scan_cells.
Chain c5 successfully traced with 31 scan_cells.
    : : : : : : :
Chain c44 successfully traced with 30 scan_cells.
Chain c45 successfully traced with 30 scan_cells.
Chain c46 successfully traced with 30 scan_cells.
Scan chain operation checking completed, CPU time=0.47 sec.
-----
Begin clock rules checking...
Warning: Rule C17 (clock connected to PO) was violated 16 times.
Warning: Rule C19 (clock connected to non-contention-free BUS) was violated
1 times.
Clock rules checking completed, CPU time=0.15 sec.
-----
Begin nonscan rules checking...
Nonscan cell summary: #DFF=201 #DLAT=0 tla_usage_type=none
Nonscan behavior: #C0=4 #C1=7 #LE=11 #TE=179
Nonscan rules checking completed, CPU time=0.04 sec.
-----
Begin DRC dependent learning...
DRC dependent learning completed, CPU time=1.01 sec.
-----
Begin contention prevention rules checking...
26 scan cells are connected to bidirectional BUS gates.
Warning: Rule Z9 (bidi bus driver enable affected by scan cell) was

```

```
violated 24 times.  
Contention prevention checking completed, CPU time=0.03 sec.  
-----  
DRC Summary Report  
-----  
Warning: Rule C17 (clock connected to PO) was violated 16 times.  
Warning: Rule C19 (clock connected to non-contention-free BUS) was violated  
1 times.  
Warning: Rule Z4 (bus contention in test procedure) was violated 12 times.  
Warning: Rule Z9 (bidi bus driver enable affected by scan cell) was  
violated 24 times.  
There were 54 violations that occurred during DRC process.  
Design rule checking was successful, total CPU time=2.27 sec.  
-----
```

See Also

[add_clocks](#)
[add_scan_chains](#)
[add_scan_enables](#)
[analyze_violation](#)
[report_violations](#)
[set_atpg](#)
[set_drc](#)
[set_rules](#)
[test](#)
[Understanding the run_drc Output](#)

run_fault_sim

Overview

Performs fault simulation using the current fault list and the current selection of the pattern source.

Syntax

```
run_fault_sim
[-checkpoint fault_filename]
[-detected_pattern_storage] [-distributed]
[-first_pattern d] [-last_pattern d]
[-ndetects n] [-nodrop_faults] [-sequential] [-store]
[-strong_bridge]
```

Arguments

-checkpoint *fault_filename*

Specifies that a fault list file should be saved to the named file after every pass of fault simulation. This option is supported only if the `-sequential` option is also used. The file named is overwritten if it already exists. The default is not to perform the checkpointing process.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-detected_pattern_storage

Stores the pattern ID for the first detection of a fault when operating on an external pattern set. Adds an additional field to the fault report indicating the pattern ID of the first detection of detected faults.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-distributed

Causes TestMAX ATPG to look for the list of distributed processors to start a distributed fault simulation environment. The machine issuing this command becomes the "master" machine. The master machine drives the distributed process, but does not participate in the fault simulation task. The fault simulation task is deferred to distributed processors only. This means that if you want to use two machines for fault simulation, you need to use a master plus two distributed processor machines.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-first_pattern *d*

Specifies an individual pattern or a range of patterns to be fault-graded during simulation.

-last_pattern *d*

Specifies a subset of the current pattern set on which to perform fault simulation. When this option is used, fault simulation is performed from the first pattern, pattern 0, through

the last pattern specified by integer *d*. The default is to perform fault simulation on all patterns.

-nodrop_faults

Allows and independent estimation of the test coverage for each pattern. After fault simulation, you can check both the number of all detected and first detected faults for each simulated pattern with the `report_faults -profile` command.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-ndetects *number* (*n*)

Detects a fault up to and including *n* times. The default is one fault detection during ATPG. This option is not supported for full-Sequential ATPG, full-sequential fault simulation, or distributed ATPG.

The `-ndetects` option keeps fault detections active until the fault has been detected *n* times. A fault detected less than *n* times is placed in the not analyzed, possibly detected (NP) fault class. When a fault is detected *n* times, it is placed in the Detected by Simulation (DS) fault class, and removed from the active fault list.

Fault reports are enhanced to display the number of times each fault was detected.

-sequential

Specifies that fault simulation will use a sequential simulation algorithm. This is required if the design has sequential elements and the functional patterns expect data effects to persist from one pattern to another. It is optional if the patterns are ATPG patterns consisting of Basic-Scan or Fast-Sequential patterns, or if the functional patterns do not expect data to persist from one pattern to another.

Without the `-sequential` option, a combinational (scan-based) simulation is performed, which assumes that data does not persist beyond the current pattern. This type of simulation is faster and can identify redundant faults, which can increase test coverage. This default mode is appropriate if the patterns were originally generated by TestMAX ATPG using Basic-Scan or Fast-Sequential ATPG.

With the `-sequential` option, a sequential simulation is performed, which retains the design state from pattern to pattern. This type of simulation is slower and cannot identify redundant faults. Sequential simulation is generally required to fault-simulate functional patterns imported with the `set patterns external` command. Use of the `-sequential` option is not recommended if the patterns can be simulated combinational. A test for whether patterns can be simulated combinational is to use the `run simulation` command without its `-sequential` option, and seeing whether the good machine simulation runs successfully without any differences between simulated and expected values.

When the `-verbose` option of the `set_simulation` command is in effect, the use of the `-sequential` option causes the `run_fault_sim` command to display the good machine vs. faulty machine response for each pattern as the fault simulation progresses.

You might want to review the `-sequential_modeling` option of the `set_netlist` command. It can affect the derived ATPG model. Sometimes a more precise (less pessimistic) model is needed for fault simulation than when doing ATPG. This `set_netlist` option can produce a more precise model during library reading.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-store

Copies functional patterns and the measure conditions that detect faults into the internal pattern buffer during fault simulation. This option is ignored if the `-sequential` option is also used or if the pattern source is internal. For scan-based patterns, any patterns that fail to detect a fault are not copied to the internal pattern buffer. For designs with compression, the pattern sequence is maintained. However, any measure conditions that do not detect a fault are set to an X value to indicate that there is no useful detection for the specified fault list.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-strong_bridge

Marks only those faults meeting the criteria for fully-optimized bridging fault detection as detected. This includes those faults where the aggressor has the strongest drive strength and the victim has the weakest drive strength.

Allowed Command Modes

Test

Description

Performs fault simulation using the current fault list and the current selection of the pattern source. This command requires a Test-FaultSim license.

You can select the type of fault simulation to be scan-based or sequential. You can display the results of the fault simulation using the `faults` option of the `report_summaries` command.

Examples

```
TEST-T> run_fault_sim -sequential
-----
Begin sequential fault simulation of 4540 faults on 36 external patterns.
-----
#faults      pass #faults      cum. #faults      test      process
simulated    detect/total  detect/active  coverage  CPU time
-----
1675         550 1675         550 3990         13.57%    05.45
3326         669 1651         1219 3321         29.36%    09.12
4540         390 1214         1609 2931         40.22%    12.22
Fault simulation completed: #faults_simulated=4540, test_coverage=40.22%,
CPU time=12.22
```

See Also

[set_patterns](#)

[set_simulation](#)

[run_simulation](#)

[Understanding the run_fault_sim Output](#)

run_justification

Overview

You can use this command to create a pattern that satisfies user-specified conditions placed on any number of internal (or external) circuit nodes.

Syntax

```
run_justification
  [-set {<id | pin_pathname> <0|1|z|r|f>}... ]
  [-full_sequential] [-noconstraints] [-noprevention]
  [-previous] [-verbose] <[-store] | [-replace_pattern d]>
```

Arguments

-set {<id | pin_pathname> <0|1|z|r|f>}...

Indicates the circuit location and the desired set value to justify. The circuit location is identified by a [primitive ID](#) number or [pin_pathname](#). 0, 1 or Z set a static value at the specified location. R indicates a rising transition and F indicates a falling transition. The `-set` option can specify multiple set points. You can repeat the circuit location and set value as needed.

The `r` and `f` arguments create last-shift launch transitions, but do not apply any capture clocks. Basic scan and full-sequential patterns are supported. It is not possible to launch transitions from the last cells of the scan chains.

-full_sequential

Specifies that the full-sequential ATPG algorithm is to be used during justification.

-noconstraints

Causes all constraints to be ignored during test generation. The default is to satisfy all constraints.

-noprevention

Causes contention prevention to be ignored during test generation. The default is to enforce conditions that avoid contention.

-previous

Causes all set points from the preceding `run_justification` command to also be satisfied for the current justification.

-replace_pattern d

The argument `d` to the `-replace_pattern` option is a decimal pattern number of an existing pattern. The care bits for the justification pattern replaces values in the pattern specified by the argument `d`. This is done to the internal or external pattern buffer, depending on the current setting (report settings patterns). The expected value for the pattern is not updated. Use the `run_simulation -update` command to get correct expected values.

-store

Causes storage of the pattern that satisfies the set points into the internal pattern set. The default is to not store the pattern. When the pattern is stored, it is appended to the end of any existing patterns in the internal pattern set. To avoid confusion, it is suggested that you delete any existing internal patterns using the `set_patterns -delete` command before performing the justification.

-verbose

Displays the pattern that satisfies the set points. The default is to not display the pattern.

Allowed Command Modes

Test

Description

You can use this command to create a pattern that satisfies user-specified conditions placed on any number of internal (or external) circuit nodes. If this is successful, the resulting pattern can optionally be placed into the internal pattern buffer and then written out to a file. If the justification is not successful, a reason is reported.

Note the following:

The justification process does not attempt to create patterns which honor PI equivalence relations unless they are explicitly listed as `-set` arguments. So it is possible to generate a pattern which satisfies the user-defined list of set points, and then have that pattern discarded when it does not satisfy the constraints required by the equivalence relationship. When this occurs, the [M179](#) message will indicate the pattern is rejected. If you add at least one of the pins from the PI equivalence relationship to the `-set` list, you can avoid pattern rejection due to this issue.

The justification process uses a set of test generation algorithms controlled by the `set_atpg` command.

Examples

```
# id #899 output set to 1 and pin pathname set to zero
TEST> run_justification -set {899 1 top/adder/carry 0 }
Unsuccessful justification: test status was redundant.
```

```
# id #1000 output set to 0
TEST> run_justification -set {1000 0} -store
Successful justification: pattern values available in pattern 0.
Warning: 1 patterns rejected due to 127 bus contentions (ID=37039, pat1=0).
(M181)
Justification pattern is stored in internal pattern set.
```

```
# cumulative run justification
TEST> run_justification -set {1999 0} -previous
Successful justification: pattern values available in pattern 0.
Warning: 1 patterns rejected due to 127 bus contentions (ID=37039, pat1=0).
(M181)
```

See Also

[set_atpg](#)

[set_patterns](#)

[set_pindata](#)

[Understanding the run_justification Output](#)

run_observe_analysis

Overview

You use this command to perform an observability analysis of the design based on controllability and observability SCOAP numbers.

Syntax

```
run_observe_analysis
  [-max_observe_points d]
  [-num_observe_points_per_cell d]
  <-observe_file file_name>
  [-type_observe <pos | scancells>]
  [-replace]
```

Arguments

-max_observe_points d

Specifies the maximum number of observe points to be selected in the design. By default, TestMAX ATPG will do an analysis to select an optimum number of observe points required for the design.

-num_observe_points_per_cell d

Specifies the maximum number of observe points that should be connected through XOR (observation) trees to one observe scan flip flop or primary output. The default is 8 observe points per observe flip flop or primary output. The selection of observe flip flop versus primary output is done with the `-type` option.

-observe_file file_name>

Specifies the name of the output file that will contain the list of selected observe points. By default, 8 locations are listed per line and rank ordered from the most beneficial to the least beneficial.

-type_observe <pos | scancells>

Specifies the type of observation technique to be used for the selected observe point. You can select between a primary output and a observe scan cell to make the selected location observable. The default is a scan cell.

-replace

If the file specified by the `-observe_file` option exists, then this option is needed to overwrite that file, otherwise, an error condition is reported.

Allowed Command Modes

Test

Description

You use this command to perform an observability analysis of the design based on controllability and observability SCOAP numbers. This command requires a Test-Analysis license.

The objective is to minimize the number of ATPG patterns required to achieve a given test coverage by making some selected internal nodes observable. It ranks the top "N" best design locations that would provide a benefit to ATPG pattern generation if those sites were better observable by being connected to a primary output or additional (that is, new) scan cells. The locations identified are reported to the transcript in rank order as the analysis progresses. These locations are recorded in the observe file in a form intended to be useful during scan insertion should you wish to modify your design to have these additional observation points.

When calculating the best observe points, fault sites that are detected by implication are excluded from consideration. In addition, no location connected to a clock source is considered.

When the observe file is saved, the potential benefit to the ATPG process can be observed by specifying this observe list with the `-observe_file` option during the use of the `run_atpg` command. The ATPG process will then proceed as if the observe points specified in this file were actually present in the design. The resulting test coverage and pattern count are representative of what to expect if the design were changed.

Examples

```
TEST> run_observe_analysis -max 500
-----
                        List of added observe points
-----
cnt obs #faults node
-----
  1 145 484   /ARIT/mult/U860/N01 (7041)
  2 115 178   /ARIT/mult/U756/N01 (7528)
  3 39  486   /ARIT/mult/U798/N01 (6361)
  4 21  850   /ARIT/mult/U220/N01 (6445)
  5 74  194   /ARIT/mult/U850/N01 (7051)
  6 87  148   /ARIT/mult/S2_11_7/N01 (6822)
  7 66  162   /ARIT/mult/U859/N01 (6918)
  8 38  352   /ARIT/mult/U227/N01 (6785)
  9 96   98   /ARIT/mult/lt/mu/S2_12_12/N02 (5745)
 10 123  74   /ARIT/mult/m32/L14_11/N02 (5991)
   :   :   :
498 42   14   /ARIT/mult/U801/N01 (6596)
499 98    6   /ARIT/mult/U527/N01 (7344)
500 28   21   /PIE_1/PEASCH/U103/N01 (11958)
-----
Total number of added observe points = 500, CPU tiime=25.87
```

```
#
# --- example OBSERVE FILE format
#
set_test_point { /ARIT/mult/U860/N01 } -type observe.
set_test_point { /ARIT/mult/U860/N01 } -type observe
set_test_point { /ARIT/mult/U860/N01 } -type observe
set_test_point { /ARIT/mult/U860/N01 } -type observe
set_test_point { /ARIT/mult/U860/N01 } -type observe
```

See Also

[run_atpg](#)
[run_testpoint_analysis](#)

run_simulation

Overview

Use this command to perform simulation of the current pattern source determined by the `set_patterns` command and report any differences between simulated and expected values..

Syntax

```
run_simulation
  [-bridge <wand | wor | adom | bdom <nodea> <nodeb>>]
  [-cell_aware_fault "path/instance_name defect_id"]
  [-compare_x | -nocompare_x]
  [-disable_masking]
  [-failure_file file_name]
  [-fast <<<r | f | rf> <chain_name position>+>]
  [-last_pattern d]
  [-max_fails d]
  [-nocompare]
  [-nox_difference]
  [-override_differences]
  [-parallel_strobe_data_file filename]
  [-replace]
  [-resolve_differences]
  [-sequential]
  [-sequential_update]
  [-slow <<<r | f | rf> <pathname | chain_name position>+>]
  [-stuck <<<0 | 1 | 01 | x> <pathname | chain_name position>+>]
  [-store]
  [-subnet subnet_id]
  [-update]
```

Arguments

-bridge <wand | wor | adom | bdom nodeanodeb>

This option injects a bridging fault in the functional logic; only one injection is possible. Note that injection of bridging defect in a scan chain is not possible: only one injection is possible. More specifically, if the internal pattern buffer is empty, this will simulate the patterns in the external buffer as if a bridging fault exists between the two specified pinpaths, referred to as *nodea* and *nodeb*. The pinpaths must be output pins of nonscan cells. The supported bridge types are wired-AND (WAND), wired-OR (WOR), pinpathA dominating pinpathB (aDom) or pinpathB dominating pinpathA (bDom). For dominant faults, the victim node takes on the value of the dominating (aggressor) node. The fault is assumed to not affect the load_unload operation. The output will display the mismatches caused by the presence of the fault.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-cell_aware_fault path/instance_name defect_id

Injects an internal cell defect based on the specified path, instance name and associated defect ID, as shown in the following example:

```
run_simulation -failure_file [list outfile1 outfile2] -cell_aware_fault
instance_name1 defect_ID1 -replace
```

For more information, see "[Cell-Aware Diagnosis](#)."

-compare_x | -nocompare_x

Checks all X values in the compared pattern data against the X values in the simulated results and reports any differences. The default is to not compare X values (the -nocompare_x option). By default, X values are treated as no-measure conditions and any differences in X states are ignored. The -compare_x option is incompatible with the set_po_masks -all command because this option identifies mismatches with all known simulated results due to the use of an X value as a mask or no-measure operation. This option is valid only for library validation flows.

-disable_masking

Enables you to automatically disable or ignore all user-defined masks during the good machine simulation. All capture and observable masks, along with disturbed clock grouping and contention checks, are disabled during this operation (this does not include PO measures). The assumption is that the patterns will pass on the tester. In this case, only X measures from the external patterns have to be taken into account during the simulation.

The -disable_masking option will typically remove a large number of simulation mismatches. However in some cases, a few mismatches may still exist. In this situation, you will need to take into account the remaining X measures and update the patterns accordingly; this is the purpose of the -resolve_differences option.

-failure_file file_name

Writes all failure data to the specified file. This option forces a fast-sequential mode simulation of the basic scan patterns, which affects the behavior of the faults. The behavior difference reflects the differences between the fast-sequential and basic scan simulators.

-fast <<<r | f | rf> < chain_nameposition>+>

If the internal pattern buffer is empty, this option will simulate the patterns in the external buffer as if transitions arrive too early at the specified scan cell during the load_unload procedure. The fast fault types are similar to hold time violations. They can be fast-to-rise (r), fast-to-fall (f) or fast-to-rise-fall (rf). The scan cell is identified by the chain name and the position in the scan chain. Position 0 is the one closest to the scan-out port. The output will display the mismatches caused by the presence of the fault. Note that this option can be used to inject chain defects only. It is not possible to inject the same fault type in functional logic.

The -fast option supports multiple arguments of the same type. Up to two chain faults can be specified in different scan chains.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-last_pattern d

Specifies a subset of the current pattern set on which to perform simulation. When this option is used, simulation is performed from the first pattern, pattern 0, through the last pattern specified by the d integer. The default is to perform simulation on all patterns.

-max_fails *d*

Specifies a limit on the number of simulated data vs. expected data failures to allow before stopping the simulation. The default is no limit. However, TestMAX ATPG displays all failures for the last failing pattern even if it exceeds this limit.

-nocompare

Disables the comparison of simulated data to expected data. The default is to perform comparisons.

-nox_difference

Specifies that a simulated value of X on an output or bidirectional port, when the expected data is not X, should not be considered a failure. This option applies only to the simulation of basic scan or sequential scan patterns and is ignored when sequential simulation is specified using the `-sequential` option. The default is to treat X differences as failures.

-override_differences

This option updates an existing pattern set but only replaces certain measure values. It does not replace X values. Instead, it replaces 0/1 and 1/0 mismatches by the simulated boolean values. This option is useful for ECO changes.

-parallel_strobe_data_file *file_name*

Generates an enhanced parallel strobe data (PSD) file used for debugging parallel simulation failures using the CPV flow. The PSD file supports the backward compatibility of an existing STIL file. For details, see "[Using the run_simulation Command to Create a PSD File](#)."

-replace

Allows an existing file to be overwritten. The default is not to overwrite an existing file.

-resolve_differences

This option updates the external pattern sets without re-running ATPG. It performs a simulation of a pattern set in the external pattern buffer. Any differences between the expected values listed in the pattern set and the obtained simulation values will cause the mismatched value in the pattern to be replaced by an X. The modified pattern set will remain in the external pattern buffer.

All cases are supported, including 0 -> 1, 0 -> X, 1 -> 0, 1->X.

In addition, you can use the `-disable_masking` option to reduce the number of X (unknown) simulation values. This approach might reduce the number of added X measures in the external pattern buffer with the assumption that the initial pattern set will pass on the tester.

For related information, see "[Updating Pattern Sets Without Re-Running ATPG](#)."

-sequential

Invokes a standalone simulator meant to be used only for functional or partial scan patterns. This simulator is event-driven like VCS and designed for very fast simulation of nonscan-based patterns.

Note: For TestMAX ATPG-generated patterns, it is recommended that you use the `run_simulation` command without any options since TestMAX ATPG will automatically use the

appropriate simulation algorithm based on the type of pattern input -- whether it is basic-scan or fast-sequential.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-sequential_update

Equivalent to specifying the `-sequential` and `-update` options simultaneously.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-slow <<<r | f | rf> <pathname | chain_nameposition>>>

If you use the arguments `-slow <r | f | rf> pathname` and the internal pattern buffer is empty, this will simulate the patterns in the external buffer as if the specified pin has a rising (r), falling (f) or rising-falling (rf) system clock transition fault. A rising-falling transition fault will delay both rising and falling transitions. The fault is assumed to not affect the load_unload operation. The output will display the mismatches caused by the presence of the fault. The transition fault types are rising (r), falling (f) or rising-falling (rf). A rising-falling transition fault will delay both rising and falling transitions. Note that the arguments `-slow <r | f | rf> pathname` are not supported in Full-Sequential mode.

The arguments `-slow <r | f | rf> chain_namepos_number` will simulate the patterns in the external buffer as if a slow-to-rise, fall or rise-fall fault is affecting the specified scan cell during the load_unload procedure. The scan cell is identified by the chain name and the position in the scan chain. Position 0 is the one closest to the scan-out port.

The `-slow` option does not support multiple arguments of the same type for logic defects. For chain defects, up to two chain faults can be specified in different scan chains.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-store

Specifies that all patterns are to be stored in the internal pattern set as simulation progresses. This option is ignored if the pattern source is set to internal or if the `-sequential` option is used. This option is not supported for Full-Sequential patterns. The default is not to store patterns.

-stuck <<<0 | 1 | 01 | x> pathname | chain_nameposition>>>

This option injects a stuck-at fault in functional logic or a chain. To inject a logic defect, you should specify the path name. To inject a chain defect, you should specify the chain name and position.

For a chain defect, if the internal pattern buffer is empty, this option will simulate the patterns in the external buffer as if a stuck-at fault is affecting the specified scan cell during the load_unload procedure. The scan cell is identified by the chain name and the position in the scan chain. Position 0 is the one closest to the scan-out port. The output will display the mismatches caused by the presence of the fault.

For a logic defect, there are four types of fault behavior that you can select: stuck-at-0 (O), stuck-at-1 (1), stuck-at-01 (01), and stuck-at-X (X). For the stuck-at-01 fault type, the value is fixed either to 0 or 1 for each pattern for all affected fanout branches. For the stuck-at-X fault type, the value is fixed either to 0 or 1 for each time frame for all affected fanout branches. All affected fanout branches have the same value.

For a chain defect, you can select any fault type except for stuck-at-01.

For a logic defect, If the internal pattern buffer is empty, this option will simulate the patterns in the external buffer as if the specified pin has a stuck-at 0 or 1 or 01 fault. A stuck-at 01 fault is activated with either a 0 or 1 good circuit value. The fault(s) are assumed to not affect the `load_unload` operation. The output will display the mismatches caused by the presence of the fault(s).

The `-stuck` option supports multiple arguments of the same type. You can specify a maximum of ten logic faults for different pins, however only one injection is supported for a stuck-at-01 logic defect. Only one fault per chain can be injected at a time. Simulating two or more faults on the same chain will cause TestMAX ATPG to exit with an error.

To inject two separate defects include both injections in the same `-stuck` option bracket, as shown in the following example:

```
run_simulation -stuck {0 100 42 0 423 20}
```

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-subnet *subnet_id*

Specifies the subnet ID (*subnet_id*) to indicate where to inject the subnet fault. The subnet ID should be in the range of the defined ID.

The following example shows how to inject a fault on a subnet:

```
run_simulation -stuck {01/X net_driver -subnet n}
```

Note that the `-subnet` option must be specified within the node definition and between brackets.

-update

Updates the responses from test in memory. This option can be used only in conjunction with the `-sequential` option. You can write out the new responses from simulation with the `write_patterns` command.

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

Allowed Command Modes

Test

Description

Use this command to perform simulation of the current pattern source determined by the `set_patterns` command and report any differences between simulated and expected values. This simulation can be performed in the presence of a selected fault and will report all failures that result from the fault. If the failures are stored in a file, that file can be used as input for the `run_diagnosis` command.

If the patterns are read in the external pattern buffer using the `-split_patterns` option, you should be aware of the following: When a fault is injected, if the output of the `run_simulation` command is stored in a file using `-failure_file` option, the failure file contains all the necessary directives to run the split diagnosis.

For related information, please see ["Updating Pattern Sets Without Re-Running ATPG."](#)

The output from the `run_simulation` command cannot be redirected using the ">" operator in native mode. This operator is valid in Tcl mode, however.

The following options are not supported for multicore simulation: , -sequential, -sequential_update, -update, -override_differences, and -store.

Examples

The following example runs a sequential simulation:

```
TEST-T> run_simulation -sequential
Begin sequential simulation of 36 external patterns.
 10 o_y2 (exp=1, got=0)
 20 o_y10 (exp=1, got=0)
Simulation completed: #patterns=36/102, #fail_pats=2(0), #failing_meas=2
(0), CPU time=0.05
```

The following example simulates a single sa01 fault:

```
run_simulation -stuck { 01 ucore/freg/u540 }
```

The following example simulates a stuck-at 0 and a stuck-at 1 fault:

```
run_simulation -stuck { 0 ucore/freg/u540 1 ucore/alu/u27 }
```

The following example simulates a rising transition delay fault in functional logic:

```
run_simulation -slow { r ucore/freg/u540 }
```

The following example simulates a transition delay problem in the scan chain:

```
run_simulation -slow { rf chn9 428 }
```

See Also

[set_simulation](#)

[set_patterns](#)

[run_fault_sim](#)

[Understanding the run_simulation Output](#)

[Specifying Lists in Tcl Mode](#)

run_testpoint_analysis

Overview

This command controls test-point insertion based on undetected fault topology.

Syntax

```
run_testpoint_analysis
  -test_point_file file_name
  [-num_observe_points_per_cell number]
  [-class fault_class]
  [-replace]
  [-dont_touch module_name]
  [-max_test_points number]
```

Arguments

-test_point_file *file_name*

Specifies the name of the output file that will contain the list of test points. This file uses the same file format as the `run_observe_analysis` command to remain compatible with TestMAX DFT.

-num_observe_points_per_cell *number*

Specifies the maximum number of observe points that should be connected through XOR (observation) to one observe scan flip flop or primary output. The default is 8 observe points per observe flip flop or primary output.

-class *fault_class*

Specifies the class of faults to be targeted. This switch can be specified multiple times if you need to specify multiple fault classes. For example, you can first generate test patterns, then execute the `run_testpoint_analysis` command using with the `-class NO` and `-class AN` options. If the `-class` switch is not specified, then all undetected faults is considered for test-point insertion.

In Tcl mode, lists are used to specify additional classes. For example:

```
-class {NO AN}
```

OR

```
-class [list NO AN]
```

You can specify the following fault classes using the `-class` option:

DS — Detected by simulation

DI — Detected by implication

AP — ATPG untestable possibly detected

NP — Not analyzed, possibly detected

UU — Undetectable unused

UO — Undetectable unobservable

UT — Undetectable tied

UB — Undetectable blocked

UR — Undetectable redundant

AN — ATPG untestable not-detected

NC — Not controlled

NO — Not observed

-replace

Causes TestMAX ATPG to replace any existing output file (specified by the `-test_point_file` option).

-dont_touch module_name

This option prevents the creation of test points in the specified instance. Multiple `-dont_touch` options can be used for additional instances. In Tcl mode, lists are used to specify additional instances. For example:

```
-dont_touch {instA instB}
```

OR

```
-dont_touch [list instA instB]
```

-max_test_points number

Specifies the largest number of test points to be inserted. TestMAX ATPG will stop writing out test-points before reaching the maximum number specified with the `-max_test_points` option if no improvement is seen during analysis. If this option is not specified, the default maximum is 1000 test-points.

Description

This command controls test-point insertion based on undetected fault topology. Note that this command can only be used in the TEST command mode.

For complete details on how to run this command, see "[Improving Test Coverage With Test Points](#)" of the *TestMAX ATPG User Guide*.

See Also

[run_observe_analysis](#)

Set Commands

This section describes the following commands:

- [set_atpg](#)
- [set_learning](#)
- [set_build](#)
- [set_match_names](#)
- [set_buses](#)
- [set_messages](#)
- [set_colors](#)
- [set_netlist](#)
- [set_commands](#)
- [set_patterns](#)
- [set_contention](#)
- [set_physical](#)
- [set_delay](#)
- [set_physical_db](#)
- [set_diagnosis](#)
- [set_pindata](#)
- [set_distributed](#)
- [set_primitive_report](#)
- [set_drc](#)
- [set_random_patterns](#)
- [set_environment_gui](#)
- [set_rules](#)
- [set_environment_info](#)
- [set_scan_ability](#)
- [set_environment_reports](#)
- [set_sdc](#)
- [set_environment_transcript](#)
- [set_simulation](#)
- [set_environment_viewer](#)
- [set_toggle_weights](#)
- [set_faults](#)
- [set_wgl](#)
- [set_iddq](#)
- [set_workspace_sizes](#)
- [set_ydf](#)

set_atpg

Overview

Use this command to set the parameters that control the ATPG process.

Syntax

```
set_atpg
[-abort_limit comb_max_remade_decs]
[-add_setreset_test | -noadd_setreset_test]
[-adjacent_fill_range d]
[-allow_clockon_measures | -noallow_clockon_measures]
[-analyze_untestable_faults | -noanalyze_untestable_faults]
[-basic_min_detects_per_pattern d [d]]
[-calculate_power | nocalculate_power]
[-capture_cycles d]
[-chain_test <off | 0011 | 0101 | 1000 | 0111 | string<r|c>>]
[-checkpoint {time fault_file pattern_file} | -nocheckpoint]
[-coverage max_percent]
[-decision <random | norandom>]
[-di_analysis | -nodi_analysis]
[-domain {clock_names}]
[-extra_cycle | -noextra_cycle]
[-fast_min_detects_per_pattern { d [d] }]
[-fast_path_delay | -nofast_path_delay]
[-fill <random | 0|1|X| adjacent>]
[-group_clk_constraints { first_pass middle_pass final_pass }]
[-lete_fastseq | -nolete_fastseq]
[-load_mode <n | all>]
[-load_value [0 | 1]]
[-merge <off | low | medium | high | d [low | medium | high | d]>]
[-min_ateclock_cycles d]
[-new_capture | -nonew_capture]
[-num_processes number | max>]
[-num_threads <ipsd_file] | -noparallel_strobe_data_file]
[-patterns max_patterns]
[-post_capture_contention_prevention | -nopost_capture_contention_prevention]
[-power_aware_asyncs | -nopower_aware_asyncs]
[-power_budget n] [-prevention <random | norandom>]
[-power_effort <high | low>]
[-quiet_chain_test | -noquiet_chain_test]
[-quiet_padding_patterns | -noquiet_padding_patterns]
[-replace_parallel_strobe_data_file | -noreplace_parallel_strobe_data_file]
[-resim_atpg_pattern <off | fault_sim | nofault_sim | test_gen>]
[-save_patterns {file_name_prefix n}]
[-seq_contention_prevention | -noseq_contention_prevention]
```

```

[-shared_io_analysis | -noshared_io_analysis]
[-shift_controller_peak percentage]
[-shift_power_budget number]
[-shift_power_effort <off | low | medium | high | d >]
[-single_load_per_pattern | -nosingle_load_per_pattern]
[-store | -nostore]
[-streaming_xtol_chain_test on | off]
[-streaming_xtol_chain_diagnosis on | off | number_of_chain_patterns]
[-summary | -nosummary]
[-timing_exceptions_au_analysis | -notiming_exceptions_au_analysis]
[-target_clock_faults | -notarget_clock_faults]
[-time_max_secs_per_fault [max_secs_per_run]]
[-verbose | -noverbose]
[-xtol_chain_diagnosis <off | low | high [d]>]

```

Arguments

-abort_limit comb_max_remade_decs

Specifies the maximum number of remade decisions before terminating a test generation effort during the Basic-Scan and Fast-Sequential ATPG algorithms. The default is 10. X1 analysis is affected by the `-abort` switch, since it uses ATPG to figure out if the feedback path is sensitizable.

TestMAX ATPG ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-add_setreset_test | -noadd_setreset_test

The `-add_setreset_test` option improves the resolution for chain diagnosis. This option causes the set test pattern and reset test pattern to load values, 0 and 1, in scan chains, and then apply the set and reset clock pulse to all the applicable scan cells. Both set and reset test patterns use full observability mode. If all the scan cells have both set and reset signals, then each stuck-at chain defect is distinguished based on these two patterns. These patterns are not chain test patterns, but they can be annotated as chain tests if all the clocks are off (inactive). This case might occur when either a 0 or a 1 value is loaded into all scan chains, but the clock pulse is not applied.

When the `-add_setreset_test` option is set, the following message is printed during the `run_atpg` process:

```

Adding reset test pattern using 1 clocks.
Adding set test pattern using 1 clocks.

```

The `-add_setreset_test` option is compatible with the `-xtol_chain_diagnosis` option or the `-quiet_chain_test` option. The order in which you specify these options is important: the `-xtol_chain_diagnosis` option or the `-quiet_chain_test` option must be specified before the `-add_setreset_test` option.

The `-chain_test` option does not generate set and reset patterns.

-adjacent_fill_range d

This option only applies if the `-fill adjacent` option is used. It offers a compromise between power usage and pattern count by limiting the number of adjacent bits having the same value. The value `d` includes the care bit and typically has a value of 2 or greater. The

default adjacent fill range is 0. Higher values result in less compact, lower switching patterns, while lower values result in more compact, and higher switching patterns. The range is reset at each care bit, or when the previous range expires. When a range expires, a random value is selected for the next fill range.

-allow_clockon_measures | -noallow_clockon_measures

Specifies whether to allow patterns that perform output measures while clocks are on. By default, measures are not performed with any clocks on. Certain types of fault locations cannot be fully tested under this restriction. For additional information, see ["Supporting Clock ON Patterns in STIL."](#)

This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-analyze_untestable_faults | -noanalyze_untestable_faults

Use this option to enable global analysis to identify untestable faults when `run_atpg` (without `-auto_compress`) is used, and also enables global testability analysis for 2-clock transition ATPG. This analysis is useful for reducing pattern count when many targeted faults are untestable. This option is always enabled when `run_atpg -auto_compress` is used. The default is `-noanalyze_untestable_faults`.

-basic_min_detects_per_pattern *d* [*d*]

When the `-merge` option is enabled, this option avoids merging patterns that fail to detect a minimum number of collapsed faults during basic scan ATPG and two-clock (transition) ATPG. Note that the `-merge` option is on by default in TestMAX ATPG II, so this option is always applied.

The `-basic_min_detects_per_pattern` option can be set at any time before running ATPG; the default is 0 (not used). The second *d* parameter specifies to terminate basic scan ATPG when the number of consecutive rejected patterns due to failure to achieve minimum detections reaches the value specified. The default for this parameter is 40. Changing this value to 0 means "no limit" on the number of consecutive rejected patterns. A non-zero value here will reduce ATPG run-time, with a potential corresponding reduction in coverage at the point of rejection if the remaining faults require some effort to detect. This option provides a trade-off between run-time and early detection of faults when pattern generation starts to drop in effectiveness.

When the second parameter is non-zero, TestMAX ATPG might report termination of pattern generation due to a specified number of consecutive rejected patterns. The number of rejected patterns specified in this message might exceed the setting of this second parameter. This is because TestMAX ATPG operates on pattern sets for basic-scan and two-clock environments in groups of 32, in parallel, and the results at the end of this operation is reported across this entire group.

An exception to the usual effect of `-fast_min_detects_per_pattern` occurs when the two-clock option is used for transition fault ATPG. In this case, the patterns are simulated in groups of 32, rather than one at a time, so there is no measure of the number of detects for each pattern at the time each pattern is generated. In this case, the `-basic_min_detects_per_pattern` option is applied at the time the patterns are fault simulated rather than as each pattern is being generated. In TestMAX ATPG, the size of the pattern group varies, however patterns are still simulated in groups.

Note: When running TestMAX ATPG, the `-basic_min_detects_per_pattern` option is applied separately to basic scan (or two-clock) patterns and to fast-sequential patterns. This means that when fast-sequential ATPG is enabled, the rejection limit is applied twice. First, basic scan and two-clock patterns are generated. If these patterns hit the rejection limit, that part of ATPG is halted. But when fast-sequential pattern generation begins, both parameters of the `-basic_min_detects_per_pattern` option are applied. This is in contrast to non-threaded TestMAX ATPG, which uses the separate parameters of the `-fast_min_detects_per_pattern` option for fast-sequential ATPG.

-calculate_power | nocalculate_power

Improves the runtime performance of the [report_power](#) command by enabling the ATPG process to internally calculate the switching activity of the scan flip-flops. Note the following limitations of the `-calculate_power` option:

- It cannot be used with the `-domain` option of the `set_atpg` command.
- It is not supported for Full-Sequential ATPG.
- It does not compute power numbers for padding patterns. Power numbers for padding patterns are displayed in the output report with the "?" symbol.
- The `-per_cycle`, `-first`, `-last`, and `-module` options of the [report_power](#) command have no effect if the `-calculate_power` option is used. If you want to generate switching information on a cycle-by-cycle basis, use the following commands:

```
set_atpg -nocalculate_power (This is the default. It must be set before the run\_atpg command.)
```

```
report_power -per_pattern -shift -capture -per_cycle
```

Note: This option is ignored by TestMAX ATPG. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-capture_cycles d

Sets the Fast-Sequential ATPG algorithm effort level. Acceptable values are integers between 2 and 10, or 0. A value of 0 disables Fast-Sequential ATPG pattern effort and results in Basic-Scan (combinational) ATPG patterns, the default. A value of 2 or higher enables Fast-Sequential ATPG pattern effort. A recommended starting value is 4. Be careful not to set this control too high, as it can cause long runtimes. Instead, generate patterns, increase the value, generate more patterns, increase the value again, and repeat until you achieve the desired test coverage.

Note: When running TestMAX ATPG, two-cycle patterns can be generated when the setting is 0.

-chain_test <off | 0011 | 0101 | 1000 | 0111 | string>

Generates a chain test as pattern 0 during ATPG. The choices are:

`off` - Do not generate pattern.

`0011` - (Default) Generate a repeating pattern of 0011...

`0101` - Generate a repeating pattern of 01...

`1000` - Generate a pattern of 0's lead by a 1 (that is, 10000...000)

`0111` - Generate a pattern of 1's lead by a 0 (that is, 01111...111)

`string` - Generate a user-defined sequence of 1's and 0's having a final character of "C" to repeat the last digit as a constant or "R" to repeat sequence For example:

```
011001R --> 011001 011001 011001
```

```
011011C --> 011011 111111 111111
```


-checkpoint {time fault_file pattern_file} | -nocheckpoint

Periodically saves patterns and fault lists to files at a specified checkpoint interval during ATPG pattern generation. The checkpoint interval time is specified in CPU seconds, with a minimum value of 60 seconds. The fault and pattern files are saved in compressed format and overwrite any files of the same name. The default is the `-nocheckpoint` option.

Example:

```
set_atpg -checkpoint { 3600.00 chkp_fault chkp_patt }
```

For more information on checkpoint files, see [Using ATPG Checkpoint Files](#).

-coverage max_percent

Specifies a test coverage limit for terminating the ATPG effort. The value can be any decimal number between 0 and 100. Basic-scan ATPG is performed in groups of 32 patterns and this can cause the final test coverage to be slightly higher than the level specified by the `-coverage` option. If you combine the `-coverage` option with the `-num_processes` option, you might see coverage overruns. Normally, when a coverage limit is met, a slave process finishes its current interval and stops. However, because slave processes work concurrently in ATPG, these finishing intervals might add additional fault coverage beyond the set limit. The more slave processes that are used, the more likely overruns will occur.

-decision <random | norandom>

Specifies how the initial decision choice is to be made when making a decision during ATPG. The default is `norandom`, which makes the initial choice based on controllability criteria. When performing ATPG pattern compression, the random decision method can sometimes result in lower pattern counts.

Note: TestMAX ATPG ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-di_analysis | -nodi_analysis

The `-di_analysis` option (the default) specifies DRC to perform DI (Detected by Implication) fault analysis. This analysis marks fault sites that must correctly operate to enable successful scan shift operation. These faults are credited as DI faults in the fault list.

If you do not want to perform DI analysis, you must set the `-nodi_analysis` option before running DRC. This prevents TestMAX ATPG from crediting any detected faults on the scan path or enable port to the flip-flop, and so on. In this case, ATPG targets these faults and attempts to detect them using regular patterns. If you set `-nodi_analysis` after running DRC and before running ATPG, faults on the scan path are still credited as DI faults.

-domain {clock_names}

Sets a capture budget for individual clock domains. This option must be used with the `-power_budget` option, and must be specified in the order shown in the following example:

```
set_atpg -power_budget 48 -domain clock1
```

You can specify a capture budget for multiple individual clock domains, as shown in the following example:

```
set_atpg -power_budget min -domain {clock1 clock2 clock3}
```

For more information, see ["Setting a Capture Budget for Individual Clocks"](#) in the *TestMAX ATPG and TestMAX Diagnosis User Guide*.

-extra_cycle | -noextra_cycle

The `-extra_cycle` option (the default) allows ATPG to generate 2-cycle patterns for stuck-at faults even when fast sequential ATPG is not enabled. In most cases, only a small number of 2-cycle patterns are generated near the end of the ATPG session. To prevent the 2-cycle patterns from being generated (which is always the case with single-process and multicore ATPG), set the `-noextra_cycle` option. If you are using TestMAX ATPG, you can control the clocking of the extra cycle for stuck-at faults using the [set_delay -common_launch_capture_clock](#) command.

-fast_min_detects_per_pattern {d [d] }

When the `-merge` option is enabled, this option avoids creating patterns that fail to detect a minimum number of collapsed faults during fast-sequential ATPG compression. This option can be set any time before running ATPG; the default is 0 (not used).

The second `d` parameter specifies to terminate Fast-Sequential ATPG when the number of consecutive rejected patterns due to failure to achieve minimum detections reaches the value specified. The default for this parameter is 40. Changing this value to 0 means there is no limit on the number of consecutive rejected patterns. A non-zero value will reduce ATPG run-time, with a potential corresponding reduction in coverage at the point of rejection if the remaining faults require some effort to detect. This option provides a trade-off between run-time and detection of faults when pattern generation starts to drop in effectiveness.

If you use the `-fast_min_detects_per_pattern` option to specify two parameters, you must use Tcl list syntax (for example, enclose the values in curly braces):

```
set_atpg -fast_min_detect {100 10}
```

Note the following:

The `-fast_min_detects_per_pattern` option does not apply to the two-clock option with transition fault ATPG; the two-clock option is controlled by the `-basic_min_detects` option. In this case, the patterns are simulated in groups of 32 rather than one at a time, so there is no measure of the number of detects for each pattern at the time each pattern is generated. In this case, the `-basic_min_detects_per_pattern` option is applied at the time the patterns are fault-simulated rather than as each pattern is being generated.

The `-fast_min_detects_per_pattern` option is ignored in TestMAX ATPG. However, you should use the `-basic_min_detects_per_pattern` option for equivalent functionality on basic scan patterns.

-fast_path_delay | -nofast_path_delay

Targets a list of path delay faults used by fast-sequential ATPG for generating path delay patterns. The default is `-fast_path_delay`.

Note: This option switches to non-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-fill <random |0|1|X| adjacent>

Specifies fill options for running internal scan and compressed scan patterns. During ATPG, the values at some primary inputs and scan cells do not have affect the targeted faults. These don't care bits can be filled based on user specification. Except as noted below, PIs are filled with random values.

The `random` parameter (the default) causes the don't care scan cells to be filled with random values. This option produces the most compact pattern set, because non-targeted faults are more likely to be detected with every don't care bit taking on a random value.

The `0` parameter causes don't care scan cells to be filled with 0s.

The `1` parameter causes don't care scan cells to be filled with 1s.

The `x` parameter causes don't care scan cells and PIs to be filled with Xs. This option produces the least compact pattern set.

The `adjacent` parameter causes don't care scan cells to be filled with the value of the first downstream (closer to scan output port) scan cell with a defined/care value. Fill values are generated based on values at the scan input port (and not the values in the scan cells). This option can be useful to minimize power usage during scan shifting by reducing signal switching at the expense of higher pattern count.

Note: The `adjacent` parameter is supported in the presence of compression (i.e. DFTMAX compression). However, all other fill values (`0`, `1`, `X`, and `random`) are treated as `random`.

-group_clk_constraints { *first_pass middle_pass final_pass* }

This option groups patterns into sets, each of which uses only one of the defined `ClockingProcedure` blocks in the STL procedure file. These numbers are specified just once, but they are applied for each individual clocking procedure. ATPG categorizes each fault by the clocking procedures that can test it. Only the appropriate subset is considered by ATPG as it generates tests for each clocking procedure. The grouping does not apply to fast sequential patterns. It applies to two-clock-optimized transition delay patterns and basic scan patterns for stuck-at.

The following arguments are specified in terms of percentages of the fault list:

- The *first_pass* argument specifies the target percentage of the fault list in the first pass through each clocking procedure. The first pass results in long blocks of patterns with just one clocking procedure.
- The *middle_pass* argument specifies the target percentage of the fault list in the subsequent passes through each clocking procedure. These passes are repeated until the *final_pass* number is reached. The middle passes result in shorter blocks of patterns with just one clocking procedure.
- The *final_pass* argument specifies the target percentage of the fault list in the final pass in which any clocking procedure might be used. In this pass, there is no guarantee that any two consecutive patterns will share the same clocking procedure.

For more information, see "[Using Internal Clocking Procedures](#)."

-lete_fastseq | -nolete_fastseq

When the `-lete_fastseq` option is set, Fast-Sequential ATPG is target faults based on Leading Edge (LE) Trailing Edge (TE) relationships between sequential elements. With `-nolete_fastseq` (the default), Fast-Sequential ATPG will target all the faults as if there is no such LE/TE clocking relationship, which is the typical mode of Fast-Sequential ATPG. The `-lete_fastseq` option is intended to get extra fault coverage when there are many faults whose detection depends on LETS effects.

Note: TestMAX ATPG ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-load_mode <n | all>

Generates quiet chain test patterns for a specific and existing load mode or all load modes. When you use compression mode to generate a quiet chain test you cannot independently

load one internal scan chain. This is because, for a fixed compressor load mode, the same internal scan chains are always fed by one load compressor scan input. Thus, it is not possible to decouple two chains which are always tied together in same load mode.

To overcome this situation, use the `-load_mode` option to enforce the application of the quiet chain test for a particular load mode or all load modes. You should specify the `-load_mode` option after setting the `-quiet_chain_test` option.

`-load_value [0 | 1]`

Configures the constant value loaded into chain test patterns, and is particularly useful for quiet chain test patterns. The constant value can be 0 or 1. The default is 0.

`-merge <off | low | medium | high | d [low | medium | high | d]>`

Specifies whether to perform pattern merging during Basic-Scan and Fast-Sequential ATPG pattern generation. The argument indicates how much effort to spend doing merging. The default is off (0), which means no merging is performed. Unless a value for this option is specified or the `run_atpg -auto_compression` command is being used, no merging will occur and there is no verbose output to report.

The first value sets normal ATPG secondary test generation merge effort. The second value sets the merge effort for multiple fault sensitization, which occurs when `-auto_compression` is used. The labels low, medium, and high are accepted for both, as well as an integer value. The first group of labels have corresponding numeric values of low=20, medium=100, and high=500. The second group of labels have corresponding numeric values of low=200, medium=1000, and high=5000. To help determine whether a chosen effort level is appropriate, you can use the `-verbose` option.

Enables 2-clock optimization, `set_delay -two_clock_transition_optimization`, when `-merge` is set to a value other than 0.

Note: If you are using TestMAX ATPG, this option is ignored. For more information on TestMAX ATPG, see "Running TestMAX ATPG."

Does tool really do this, which tool doesn't, is it wrt to -auto

`-min_ateclock_cycles d`

Specifies the minimum number of system cycles each pattern must have. Each system cycle pulses all defined reference clocks (if any). By default, the number of ATE Clock cycles is the same as the value for `set_drc -num_pll_cycles` (or the PLLCycles value in the STIL procedure file). This might have to be increased to account for OCC Controller latency, or if one of the PLLs has a low frequency.

```
set_atpg -min_ateclock_cycles 3 # example
```

A minimum number of ATE clock cycles can be defined to allow sufficient time between the end of a `load_unload` and the start of the next `load_unload` for the clock generator and the PLL controller to switch from shift to system mode, pulse the designated internal clocks, and switch back to shift mode.

`-new_capture | -nonew_capture`

The simulation behavior of trailing-edge (TE) devices is to always capture the state after the leading-edge (LE) and level-sensitive (LS) devices have captured. This pattern always simulates correctly, but it might not detect the target fault. ATPG recognizes all forms of "capture new" behavior for DLATs and DFFs, and it creates patterns that are much more likely to detect such target faults, including:

- Leading edge to TE state elements
- LS to TE state elements
- LS to LS state elements

The default is the `-nonew_capture` option. For DFTMAX Ultra, the default is the `-new_capture` option.

Note: TestMAX ATPG ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

`-num_processes number | max>`

Specifies the number of slave processes to be used for multicore ATPG. The *number* specification refers to the number of slave processes that is used in ATPG. If `max` is specified, then TestMAX ATPG computes the maximum number of processes available in the host based on number of CPUs. If TestMAX ATPG detects that the host has only one CPU, then single-process ATPG is performed instead of multicore ATPG with only one slave. Note that you should not specify more processes than the number of CPUs available on the host. You should also consider whether there is other CPU-intensive processes running simultaneously on the host when specifying the number of processes. If too many processes are specified, performance will degrade and might be worse than single-process ATPG. On some platforms, TestMAX ATPG cannot compute the number of CPUs available and will issue an error if `max` is specified.

Note the following:

- When the `-num_processes` option is selected, the parameters for the following options need to be set according to number of slave processes used: `-basic_min_detects_per_pattern d [d]`, `-fast_min_detects_per_pattern d [d]`, and `-full_min_detects_per_pattern d [d]`. This is because these options are applied independently to each slave process. The number of consecutive rejected patterns due to failure to achieve minimum detections is lower in case of a slave process when compared to single-process ATPG. Hence, to get the benefit of reduction in ATPG run-time when pattern generation starts to drop in effectiveness, the second *d* parameter has to be set to a lower value than specified in a single-process ATPG. If a high value is specified for this parameter, performance will degrade and might be worse than single-process ATPG.
- If you combine the `-num_processes` option with ATPG limit settings such as `-pattern` or `-coverage`, you might see pattern or coverage overruns. Normally, when a pattern or coverage limit is met, a slave process finishes its current interval and stops. However, because slave processes work concurrently in ATPG, these finishing intervals might add additional patterns and fault coverage beyond the set limit. The more slave processes that are used, the more likely overruns will occur.
- To turn off multicore ATPG, specify `set_atpg -num_processes 0`.
- This option has no effect on TestMAX. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).
- For more information on using the `-num_processes` option, see "[Running Multicore APTG](#)."

`-num_threads number`

Specifies the number of threads to use for multithreading. This option only works if you have installed TestMAX. By default, the number of threads is set to 8. This option is used with the `-num_threads` option of the `set_simulation` command. Both options must be set to the same value, as shown in the following example:

```
set_atpg -num_threads 12
set_simulation -num_threads 12
```

If you set this option to 0, multithreading is disabled and non-threaded processing is used.

Note: You cannot read image files into TestMAX ATPG after changing the value of the `-num_threads` option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

`-optimize_bridge_strengths | -nooptimize_bridge_strengths`

Selects strength-sensitive bridging fault ATPG and fault simulation. TestMAX ATPG attempts to generate patterns with fully optimized detection for all faults. If it fails to obtain a fully optimized detection, it falls back to minimal or partially optimized bridging fault detection. During ATPG, faults in any of these three bridging fault detection categories is marked as detected. For additional information about detection categories, see "Categories of Bridging Faults" in the TestMAX ATPG User Guide. The default is `-nooptimize_bridge_strengths`.

`-parallel_strobe_data_file [psd_file] | -noperallel_strobe_data_file`

Generates a parallel strobe data (PSD) file used for debugging parallel simulation failures using the CPV flow. The data in the PSD file corresponds to the expected strobe (unload scan chain) data. For details, see "[Using the run_atpg Command to Create a PSD File](#)."

Note: TestMAX ATPG ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

`-patterns max_patterns`

Specifies the termination of the ATPG effort when the pattern count equals or exceeds the specified value. A value of 0 disables this limit. The default is no pattern limit. If you combine this option with the `-num_processes` option, you might see pattern overruns. Normally, when a pattern limit is met, a slave process finishes its current interval and stops. However, because slave processes work concurrently in ATPG, these finishing intervals might add additional patterns beyond the set limit. The more slave processes that are used, the more likely overruns will occur.

`-post_capture_contention_prevention | -nopost_capture_contention_prevention`

Activates an algorithm that avoids postclock contention during pattern generation. The older (and default) method is to check for postclock contention after a pattern is generated, and to discard it if contention is found. Enabling this option will consume more CPU time during pattern generation but might result in finding and retaining more patterns than the default algorithm.

`-power_aware_asyncs | -nopower_aware_asyncs`

Activates testing of asynchronous sets and resets from primary inputs for legacy scan designs. Note that this feature is not implemented for DFTMAX designs. Also note that the design should be capable of propagating asynchronous signals so there is sufficient time for them to fit within the given ATE vector. The default is `-nopower_aware_asyncs`.

`-power_budget number | min`

Sets a power (switching) budget for the Power Aware ATPG feature. The *number* specification is an integer that specifies the maximum number (as a percentage) of scannable flip-flops budgeted to change during capture. The *min* specification is the minimum recommended setting based upon the clock-gating analysis. The clock-gating analysis is dependent upon the user-provided STL procedure file. For details, see "[Power Aware ATPG](#)" in the *TestMAX ATPG and TestMAX Diagnosis User Guide*.

-power_effort <high | low>

Generates patterns that do not exceed the power budget specified for capture. The default is `low`. If you set this option to `high`, ATPG is forced to generate patterns that do not exceed the budget specified by the `set_atpg -power_budget` command. Note that over constraining the power budget might cause longer run times and generate fewer patterns when the `-power_effort` option is set to `high`. Because of this, it is strongly recommended that you do not set power budgets below that recommended by the `report_clocks -gating` command.

-prevention <random | norandom>

Specifies the algorithm for prevention of Z-state contention. A setting of `random` selects a random BUS primitive input to control to a non-Z state. A setting of `-norandom` (the default) selects the first BUS input.

-quiet_chain_test | -noquiet_chain_test

Enables the generation of quiet chain test patterns. When this option is specified, TestMAX ATPG generates chain test patterns with lower switching activity than regular chain test patterns. This is useful when dealing with power issues or for diagnostics purposes. The 0011 sequence associated with regular scan chain patterns is applied to all scan inputs, which could lead to power issues. However, a quiet chain test pattern loads a single chain with specified pattern data and loads all other chains with a constant value. You can use the `set_atpg -load_value` command to select the constant value.

In legacy scan mode, the 0011 sequence, or any other specified pattern data, is independently applied to each scan chain, while all other scan chains are set to 0 by default. This means that one pattern loads the 0011 sequence in one single scan chain at a time. To load N scan chains, where N is the total number of scan chain, the tool generates N quiet chain test patterns.

In scan compression mode, the 0011 sequence, or any other specified pattern data, is independently applied to each scan channel, while all other scan channels are set to 0 by default. The compressor load mode is maintained to a constant value, which is 0 by default. One scan channel fanouts multiple chains due to the input load compressor. Thus, to load the P scan channels, where P is the total number of load compressor scan inputs, the tool generates P quiet chain test patterns.

If legacy scan mode is used, then N quiet chain test patterns are generated, where N is the total number of scan chains. The following message is printed during the ATPG run:

```
<N> quiet chain patterns were generated.
```

If scan compression mode is used, then the load mode 0 is used by default and P quiet chain test patterns are generated, where P is the total number of input compressor inputs. The following message is printed during the ATPG run:

```
<P> quiet chain patterns were generated using load mode 0.
```

To generate quiet chain test patterns for a specific and existing load mode or all load modes, use the following command:

```
set_atpg -load_mode <<n> | all>
```

Note: This command has no significance for DFTMAX Ultra. All quiet chain test patterns are generated with load mode 0.

If the following sequence of commands is issued, no chain test patterns, including quiet chain test patterns, are generated:

```
set_atpg -chain_test off
set_atpg -quiet_chain_test
```

When quiet chain test patterns are generated, all other chain test patterns are not generated, including regular chain test pattern and augmented chain test patterns.

The first generated pattern is a preliminary pattern which initializes the chains with a constant value. This pattern is necessary to contribute to the low switching activity of the first quiet chain test. After this pattern is generated, the quiet chain test patterns are generated.

Note: The preliminary pattern is not generated for DFTMAX Ultra.

The default, `-noquiet_chain_test`, disables the generation of quiet chain test patterns.

-quiet_padding_patterns | -noquiet_padding_patterns

Forces the suppression of clocks when generating padding patterns during ATPG. By default (the `-noquiet_padding_patterns` parameter), the clocking of the pattern preceding the pattern is used, except if a power budget is set in which case the clocks are suppressed. This option affects only TestMAX ATPG.

-replace_parallel_strobe_data_file | -noreplace_parallel_strobe_data_file

Replaces an existing PSD (parallel strobe data) file used for debugging parallel simulation failures using the CPV flow. The data in the PSD file corresponds to the expected strobe (unload scan chain) data. For details, see "[Using the run_atpg Command to Create a PSD File](#)."

-resim_atpg_patterns <off | nofault_sim | fault_sim | test_gen>

This option (the `off` argument is the default) enables the resimulation of ATPG-generated patterns to increase the robustness of patterns by masking all differences between the simulation performed during the ATPG process and the resimulation. As a result, all [run_simulation](#) failures are eliminated. This functionality is supported for all fault models using basic scan and fast-sequential patterns for all scan and DFTMAX flows. It is not supported for full-sequential ATPG. **Note:** This option requires significant CPU resources, even if no mismatches are identified.

When the `-resim_atpg_patterns` option is turned on (using the `nofault_sim`, `fault_sim`, or `test_gen` arguments), the following actions are performed at the end of the ATPG process:

- Good machine resimulation is performed for all basic scan and fast-sequential patterns.
- Measures which have a difference are replaced with Xs.
- The lines which indicate failures are suppressed, but all other lines from the `run_simulation` command are retained.
- If any measures are replaced with Xs and the `fault_sim` argument is selected, the following events will occur:
 - All Detected by Simulation (DS) faults are reset to Not Controlled (NC).
 - All active faults are simulated over all patterns using the adjusted measures.

- The output during fault simulation is the same as the output produced by the [run_fault_sim](#) command.
- The `test_gen` argument behaves the same as the `fault_sim` argument, except that an additional [run_atpg](#) pass is automatically performed, if needed, to recover coverage lost during the masking process.
- An expert-level message is issued at the end of the process indicating the number of measures replaced with Xs and the total CPU time.

Note: ATPG uses multithreading, but resimulation uses non-threading processing.

-save_patterns {file_name_prefix n}

This option enables ATPG to save GZIP-compressed parallel pattern sets that can be simulated in STIL format for every *n* patterns during the ATPG process. The names of the pattern files use the naming convention *file_name_prefix_0.stil*, *file_name_prefix_1.stil*, and so forth. The last set of patterns uses the naming convention *file_name_prefix_end.stil*. If a file with the same name already exists, it will not be overwritten. Also, since Basic scan ATPG creates 32 patterns at a time, specifying a value less than 32 will automatically default to 32 patterns per STIL file.

Note that in Tcl mode, the file name prefix and the number of patterns must be specified in a Tcl list format, as shown in the following example:

```
set_atpg -save_patterns {file_prefix 3}
```

The differences between this option and the `-checkpoint` option are: 1) The `-checkpoint` option saves patterns at constant time intervals, while `-save_patterns` saves patterns on pattern intervals. 2) The `-checkpoint` option saves all patterns in one file, while `-save_patterns` saved patterns in multiple files so that they can be simulated simultaneously. To disable this option, use the following command: `set_atpg -save_patterns file_name_prefix 0`.

Note: TestMAX ATPG ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-seq_contention_prevention | -noseq_contention_prevention

When post-clock bus contention checking is enabled with the [set_contention](#) -capture command, you can use the `-seq_contention_prevention` option to enable ATPG support to actively prevent post-clock bus contention during pattern generation. Designs with many [Z1](#) violations and [M181](#) messages might benefit with better test coverage and/or lower pattern count at the expense of increased run-times. The default is `-noseq_contention_prevention`, which disables ATPG support for post-clock bus contention.

Note: This option has no effect if there are no failing buses capable of post-clock contention or if post-clock bus contention checking is disabled with the non-default setting of [set_contention](#) -nocapture.

-shared_io_analysis | -noshared_io_analysis

Performs a quick analysis to identify all identical circuit networks in designs that use shared codec I/O implementation for identical cores. This analysis, which is performed before ATPG, leads to improved pattern generation. The default is `-noshared_io_analysis`. If you specify the `-shared_io_analysis` option, the following statement reports if the current design context can benefit from this feature:

```
Shared IO instance analysis completed. Priority faults identified: n
```

If the number of reported priority faults is zero, then the `-shared_io_analysis` option has no effect and should not be used. A non-zero number indicates that the `-shared_io_analysis` option can improve ATPG efficiency.

Note: TestMAX silently ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

`-shift_controller_peak percentage`

Specifies the maximum percentage of scan cells that can switch in a shift cycle when controlled by the shift power groups hardware that can be added to DFTMAX compression. Shift power groups reduce power consumption during scan shift. The `-shift_controller_peak` option cannot be used if shift power groups were not specified during DFT insertion using the `-shift_power_groups true` option of the `set_scan_compression_configuration` DFT Compiler command. If shift power groups were specified, the `-shift_controller_peak` option is required unless the shift power disable signal is constrained to disable this feature.

Patterns that exceed the percentage specified by the `-shift_controller_peak` option are rejected during ATPG, but the minimum percentage that can be used for successful pattern generation depends on the shift power chain length or ratio specified for DFT insertion. The default peak percentage is 100.

Additional information on configuring shift power groups is provided in "Reducing Scan Shift Power Using Shift Power Groups" in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

`-shift_power_budget number`

Sets a budget for TestMAX ATPG to reject patterns that exceed the specified maximum percentage of scan cells switching in a shift cycle. This option must be used with the `-shift_power_effort` option, and works only with basic scan, fast-sequential, and two-clock transition ATPG. Set the `-shift_power_budget` option to 0 to turn it off, as shown in the following example:

```
set_atpg shift_power_budget 0
```

The default budget is 100.

`-shift_power_effort <off | low | medium | high | d >`

Specifies a relative level of effort for reducing switching activity. You can use the `d` argument to specify a number between 1 and 100. The lower the specified number, the more effort is applied to reducing switching activity. The `low` argument corresponds to a setting of 50, the `medium` argument corresponds to a setting of 30, and the `high` argument corresponds to a setting of 10. The default is the `off` argument, which disables the `-shift_power_effort` option.

The `-shift_power_effort` settings do not result in an exact reduction of switching activity. In some cases, there might not be a discernible difference in switching activity reduction, even though different levels are specified.

`-single_load_per_pattern | -nosingle_load_per_pattern`

The `-single_load_per_pattern` option limits the number of scan loads in any given pattern to 1. The default is `-nosingle_load_per_pattern`.

`-store | -nostore`

Specifies whether ATPG patterns created is stored. Selecting `-nostore` performs ATPG to determine a test coverage without saving the patterns. The default is `-store`.

-summary | -nosummary

Specifies whether a completion summary is issued after the ATPG pattern effort is completed. By default, a summary of pattern count, test coverage, and fault report is displayed after each `run_atpg` command.

-streaming_xtol_chain_diagnosis on | off | *number_of_chain_patterns*

Turns on and off chain test patterns for DFTMAX Ultra 1:1 mode patterns. When this option is turned on, one chain test pattern is used for a chain. By default, one chain test and four default patterns are used to verify the integrity of the DFTMAX Ultra architecture; this includes the first three patterns and last pattern of a pattern set. The *number_of_chain_patterns* parameter generates high resolution patterns for scan position localization. Ten capture patterns are generated by default.

-streaming_xtol_chain_test on | off

Controls the generation of test patterns for DFTMAX Ultra chain diagnostics. To turn off the generation of patterns for DFTMAX Ultra chain diagnostics, specify the `off` argument.

-target_clock_faults | -notarget_clock_faults

Enables Fast-Sequential ATPG to sensitize faults through the clock input of a latch, if necessary. This option is typically used if sensitization through a clock pin was previously blocked in Fast-Sequential ATPG, and some coverage might have been lost because of this restriction. The `-notarget_clock_faults` option resets Fast-Sequential ATPG back to the default settings.

-time *max_secs_per_fault* [*max_secs_per_run*]

Specifies the maximum CPU time, in seconds, allowed per fault during the Basic-Scan or Fast-Sequential ATPG before a fault detection effort is terminated. The optional second argument indicates the maximum CPU time allowed for the entire Basic-Scan and Fast-Sequential portions of the `run_atpg` command. The default is no time limits. After it is turned on, the time limit can be turned off again by specifying a 0 for the time values.

Note for Tcl mode: Multiple values specified by the `-time` option must appear as a list and be enclosed by curly braces, for example:

```
TEST-T> set_atpg -time {5 15}
```

-timing_exceptions_au_analysis | -notiming_exceptions_au_analysis

This setting enables a special type of AU analysis to be performed at the start of the [run_atpg](#) process. For each fault affected by SDC (Synopsys Design Constraints) timing exceptions, if all the gates in both the backward and forward logic cones are part of the same timing exception simulation path, then the fault is marked AU. If any gates on either cone are not part a timing exception simulation path, or are only on the paths for other exceptions, the fault remains ND and is targeted by ATPG. This analysis completes before any patterns are generated, and no additional faults are marked as AU because of timing exceptions during the rest of the ATPG process. The default is `-notiming_exceptions_au_analysis`.

The AU faults identified as a result of using the `-timing_exceptions_au_analysis` switch are only reported separately when the `set_faults -summary verbose` command has been set. They will appear in the fault summary report under the "AX" subclass of faults.

You can report the faults assigned to the AX subclass by specifying the command `report_faults -class AX`. The command `update_faults -reset_au` resets the AU faults for timing exceptions to ND.

This analysis can require significant CPU time and memory. This usually occurs when there are a combined large number of exceptions and gates affected by those exceptions. This analysis only takes place before the first pattern is generated, so if the `run_atpg` command appears to hang before generating the first pattern, you should consider stopping it and trying it again without specifying the `set_atpg -timing_exceptions_au_analysis` switch.

-verbose | -noverbose

With `-verbose` enabled and, extra messages are displayed during the pattern merge operation. The default is `-noverbose`. Additional discussion and examples of the affects of the `-verbose` option can be found under the "Verbose Format With Merge" topic of [Understanding the run_atpg Report](#).

-xtol_chain_diagnosis <off | low | high [d]>

This option improves the diagnosis of failing scan chains by generating additional patterns that use the available X-tolerant modes for direct chain observation. These patterns are used for testing scan chains and capturing logic values, and improve the identification of failing scan chains when there are multiple chain defects that might not be diagnosed using standard ATPG patterns. You should use the `-xtol_chain_diagnosis` option for designs with DFTMAX compression with high X-tolerant mode. For more information, see "[Creating Test Patterns for Diagnosing Scan Chain Failures](#)" in the *TestMAX ATPG and TestMAX Diagnosis User Guide*.

When the `-xtol_chain_diagnosis` option is specified, two sets of patterns are generated:

- The first pattern set consists of augmented chain test patterns using 1:1 X-tolerant modes to uniquely observe each individual chain. The `off` parameter (the default) generates augmented chain test patterns only for compressors with [R10](#) violations.
- The second pattern set consists of X-tolerant capture patterns using the available X-tolerant modes. These patterns target primary and secondary faults similar to standard ATPG, and all detected faults are classified accordingly and removed from the list of faults for ATPG to target.

The `low` parameter generates 64 additional patterns (32 capture patterns plus 32 padding patterns) that use only N:1 X-tolerant modes. The `high` parameter generates additional patterns that use all available 1:1 X-tolerant modes. The default is to generate 10 capture patterns for each available 1:1 X-tolerant mode. The optional `d` parameter can specify the number of capture patterns generated for each 1:1 mode.

The `-xtol_chain_diagnosis` option is not supported for distributed ATPG. You can use the `-only_chain_diagnosis` option of the `run_atpg` command to terminate ATPG after generating the X-tolerant chain test and capture patterns. You can then enable distributed ATPG to complete pattern generation in a second pass (for example, using the `run_atpg -auto` command).

Allowed Command Modes

All (DRC mode only for `-nomulti_clocks_per_load` option)

Description

Use this command to set the parameters that control the ATPG process. It also affects all analyses that use test generation (including DRC and justification). You can display the current settings with the `report_settings` command.

Examples

```
TEST-T> set_atpg -patterns 400 -abort_limit 5
```

```
DRC-T> set_atpg -coverage 80
```

```
TEST-T> set_atpg -abort_limit 10 -patterns 100 -time {5 15} -merge off -
noverbose
```

```
BUILD-T> set_atpg -abort 5 -patterns 1500
```

```
BUILD-T> report_settings
```

```
atpg = limits(abort=5 time=0.00/0.00 patterns=1500 coverage=100.00)
        clock=any, decision=norandom, learn_depth=0, merge=off
        prevention=norandom, store=on, verbose=off
        capture_cycles=0, multi_clocks_per_load=on
        :
        :
workspace sizes = string=2048, line=50000, decisions=5000
```

See Also

[analyze_buses](#)

[analyze_faults](#)

[analyze_feedback_path](#)

[analyze_wires](#)

[report_settings](#)

[run_atpg](#)

[run_drc](#)

[run_justification](#)

[Pin Name](#)

[Specifying Lists in Tcl Mode](#)

set_build

Overview

This command sets the parameters that control how TestMAX ATPG builds a simulation model when you specify the `run_build_model` command.

Syntax

```
set_build
[-black_box module_name | -design_box module_name | -empty_box module_name
| -nobox module_name | -portfault_box module_name | -reset_boxes]...
[-bus_handling <none | pulldown | pullup | keeper>]
[-coerce_port_directions | -nocoerce_port_directions]
[-delete_unused_gates | -nodelete_unused_gates]
[-fault_boundary <lowest | hierarchical>]
[-hierarchical_delimiter c]
[-inout_force {input_port_names}]
[-inout_reset]
[-instance_modify {instance_name gate_type_name}| -noinstance_modify
{instance_name} | -all]
[-limit_fanout d]
[-merge {<bus_keepers | nobus_keepers> | <cascaded_gates_with_pin_loss |
nocascaded_gates_with_pin_loss> | <dlat_from_flipflop | nodlat_from_
flipflop> | <equivalent_dlat_dff | equivalent_initialized_dlat_dff |
noequivalent_dlat_dff> | <feedback_paths | nofeedback_paths> | <flipflop_
from_dlat | flipflop_cell_from_dlat | noflipflop_from_dlat> | <global_tie_
propagate | noglobal_tie_propagate> | <mux_from_gates | muxpins_from_gates
| muxx_from_gates | nomux_from_gates> | <tied_gates_with_pin_loss |
notied_gates_with_pin_loss> | <wire_to_buffer | nowire_to_buffer> | <xor_
from_gates | xorpins_from_gates | noxor_from_gates> }}]...
[-net_connections_change_netlist | -nonet_connections_change_netlist]
[-undriven_bidi <pio | pi |po>]
```

Arguments

-black_box module_name | -empty_box module_name | -nobox module_name | -design_box module_name | -portfault_box module_name | -reset_boxes

The `-black_box` option specifies a module to be treated as a [black box](#). This option is mutually exclusive with the `-portfault_box` option.

The `-empty_box` option specifies a module to be treated as an [empty box](#).

The `-design_box` option specifies the module is to be considered a design library cell. In this case, faults are placed only at the pin boundaries of the module.

The `-portfault_box` option forces the build process to preserve pin names of modules that would normally be flattened in the model. This option can be used as a workaround for the `read_sdc` command when module pins are accepted as -through points for false and multicycle paths, but flattening during the build process will remove all reference to them so they cannot be found. In this case, the `-portfault_box` option used for the module(s)

whose pins are used as through points. Note that the `-portfault_box` option is mutually exclusive with the `-blackbox` option.

The `-nobox` option is used to remove the black box, empty box, portfault box, or design box setting from a module previously set to one of these types.

The `-reset_boxes` option provides a convenient method for doing a `-nobox` on all of the modules at once. You can see which modules have been assigned the black box or empty box attribute using the report modules command. A (BB) or (EB) indicator appears in the rightmost column to indicate this attribute.

You can use a Tcl list to specify more than one module using the `-black_box`, `-empty_box`, or `-nobox` option. For example:

```
DRC-T> set_build -empty_box {MY_MODULE PLL_CORE}
```

For best results, you should read all netlists before declaring any options related to boxes.

-bus_handling <none | pulldown | pullup | keeper>

When `pulldown` is selected, a weak pulldown driver is added to all BUS gates that don't already have a TIE connection of any strength. When `pullup` is selected, a weak pullup driver is added to all BUS gates that don't already have a TIE connection of any strength. When `keeper` is selected, a bus keeper is added to all BUS gates without bus keepers. In this case, a half-keeper (that is, a bus keeper that holds only 0 or 1) is upgraded to a full-keeper (one that holds both 0 and 1). The default is `none` and no additional actions are performed.

Note: Using a non-default setting of the `-bus_handling` option might result in the generation of patterns by the ATPG engine that can fail VCS simulation or fail on silicon, if the actual design does not have the same BUS behavior that ATPG considers.

-coerce_port_directions | -nocoerce_port_directions

Selecting `-coerce_port_directions` transforms PO gates that connect to nets with one or more other sinks into PI gates and no B7 is issued. This transformation is done hierarchical, bottom-up. The model built reflects the new port direction. The netlist data is also changed: a subsequent `write_netlist` command will show the module having a PI instead of the PO; also, a subsequent `run_build` command will not show the transformation again because the internal netlist data has already been changed. At the end of build, message M435 is issued if one or more POs have been transformed into PIs.

The default setting is `-nocoerce_port_directions`, which is the old behavior (that is, if a PO gate is connected to a net with no sources, then a TIEZ drives the PO and a B7 violation is issued).

-delete_unused_gates | -nodelete_unused_gates

Selects whether unused gates are deleted from the simulation model. An unused gate has no outputs connected to other gate inputs or bidirectional pins. With unused gates deleted, the corresponding fault sites are also deleted. These faults are normally the UD = Undetectable [class of faults](#). Since the UD faults are included in the [fault coverage](#) calculation, you should be aware of how unused gates can affect this reported value. The default is to delete all unused gates.

By default, all unused gates are deleted during model optimization. An unused gate is defined as a gate that has no fanout to another gate (not even a "black box" or TIEX gate), except possibly to other unused gates. Removal of unused gates during hierarchical, bottom-up model build can significantly improve performance and reduce memory usage.

However, all fault sites on pins of unused gates are also removed (these faults could not be detected anyway, because they do not connect to an output).

Conversely, `-nodelete_unused_gates` preserves the undetectable faults, but can significantly degrade performance and increase memory usage.

Often, `-nodelete_unused_gates` is used to facilitate comparison between TestMAX ATPG fault population and the fault population of other ATPG tools that use less optimized models. This has the unfortunate side-effect of degraded performance, yet doesn't provide a fair comparison. Here is why:

All faults on unused gates are in the UD category and do not affect test coverage.

However, only a fraction of the unused gates are due to unconnected logic in the design; the rest are the result of previous optimizations (for example, replacing equivalent DLAT/DFFs). The latter gates, if not removed, would contribute to both UD and detectable (DT) fault categories, thereby increasing the total fault count, although the pattern set generated would not differ.

Therefore, a better method to compare TestMAX ATPG to other tools is to keep the default `-delete_unused_gates` and compare the numbers of detected faults.

`-fault_boundary <lowest | hierarchical>`

Specifies where faults is placed in the absence of any ``celldefine`, ``suppress_faults`, or ``enable_portfaults` directives within a Verilog netlist. A selection of `lowest` causes faults to be placed on the ports of the lowest-level modules (modules not containing instances of other modules). A selection of `hierarchical` causes faults to be placed on all modules in the hierarchy that contain at least one instance of a primitive. The default is `lowest`.

`-hierarchical_delimiter c`

Specifies the character to use as the hierarchical delimiter for pin, net, and instance path names. Do not change this character after performing a `run_build_model` command, or else unpredictable fault lists will result. The default is a forward slash `"/"`.

`-instance_modify {instance_name gate_type_name} | -noinstance_modify {instance_names} | -all`

Defines a list of instance modifiers that specify how selected instances are defined in the flattened model. Note that in Tcl mode you must use curly braces to specify a list.

The `-instance_modify {instance_name gate_type_name}` option specifies that *instance_name* should be replaced with a gate of type *gate_type_name*. The supported gate types and their respective restrictions are as follows:

BUF, INV: the instance must have a single input and no BIDI.

AND, NAND, OR, NOR, XOR, XNOR: the instance must have no BIDI.

TIEX, TIE0, TIE1, TIEZ: no restrictions.

DLATDFF2BUF: all single-port DLAT and DFF primitives (with tied-off set or reset) within the specified instance, if any, are replaced with buffers from the data input to the output; the set, reset and clock inputs are disconnected.

Example:

```
set_build -instance_modify {mybuffer TIEX}
```

Note that the command syntax and the instance type are checked immediately when the command is entered; any error will cause the command to be rejected. The restrictions for

each instance type are checked at the beginning of build. Error conditions are reported in M496 messages and cause the build process to terminate immediately.

Message Examples:

```
Instance_modify instance_name gate_type_name was not found and removed
(M496)
```

Explanation: The specified instance was not found. Check the path name entered.

```
Instance_modify instance_name gate_type_name cannot be applied to module
module_name with number bidis and was removed (M496)
```

Explanation: The specified instance has BIDI pins and cannot be remodeled as the Boolean gate chosen. Change the instance or model it as a TIE gate.

```
Instance_modify instance_name gate_type_name cannot be applied to module
module_name with number inputs and was removed (M496)
```

Explanation: The specified instance has more than one input and cannot be remodeled as the Boolean gate chosen. Change the instance or model it as a multi input gate.

Note that all invalid instance modifiers are removed from the list of instance modifiers. You can then add additional instance modifiers and restart the build process.

The `-noinstance_modify {instance_names} | -all` option deletes the selected or all instance modifiers, thus restoring the netlist definition of an instance or all instances.

These options can be repeated in the command line to add or remove additional instance modifiers.

Boxed modules (see `[-black_box module_name ...]`) are considered before instance modifiers, so no instances can be found inside a black/empty-boxed module.

The list of all current instance modifiers is displayed as part of the `report_settings build` command output.

The `report_instances` command output and the block information window in the GSV identify if a particular instance has been modified.

`-inout_force {input_port_names}`

Defines a set of input signals in the top-level module that are redefined as bidirectional signals (inout signals). The `input_port_names` argument can be a single name or a list of port names. Only the input ports in the input netlist can be modified. This modification does not create a full bidirectional signal in the design. Instead, it changes the test program to reference the signal as an inout signal.

The `-inout_force` option only provides a mechanism to uniformly ignore the defined signals as input signals in the test program. Changed signals must be constrained to a single value and you should use the `add_po_masks` command for these signals.

When the `-inout_force` option is specified, the top module specified for the [run_build_model](#) command will change the port names to `original_module_inout_force`, and the [run_build_model](#) command will use this modified interface. Full port names are required; wildcard expansion of these references is not supported.

You can replace full vectored ports (buses) by referencing the signal name without the bracket notation. These signal references cannot be validated until the top-level module is identified using the [run_build_model](#) command; errors with incorrect names are not reported until after the [run_build_model](#) command is executed. The [read_netlist -delete](#) command will remove these definitions, so the names must be specified again after specifying this command.

-inout_reset

Removes any signal names defined using the `-inout_force` option from subsequent processing when executing the [run_build_model](#) command.

-limit_fanout d

Limits the number of fanouts a gate is allowed to have. You can set this limit using the `-limit_fanout d` option. The default is 256. It can set to any value between 16 and 65536. A higher limit can slightly reduce the number of buffers and inverters in the final model, but can also significantly increase memory and CPU time. This fanout limit applies only to netlist nets and does not apply to internal connections between RPORT, ADRBUS, DATABUS, and MEMORY primitives.

Limiting the fanout (for designs that have a large number of gates with very large fanout) results in a small increase in the number of buffers and inverters in the final model, but the peak memory usage is decreased significantly. Reducing the fanout also increases the ATPG performance.

```
-merge {<bus_keepers | nobus_keepers> |
<cascaded_gates_with_pin_loss | nocascaded_gates_with_pin_loss> |
<dlat_from_flipflop | nodlat_from_flipflop> |
<equivalent_dlat_dff | equivalent_initialized_dlat_dff
    | nonequivalent_dlat_dff> |
<feedback_paths | nofeedback_paths> |
<flipflop_from_dlat | flipflop_cell_from_dlat | noflipflop_from_dlat> |
<global_tie_propagate | noglobal_tie_propagate>
<mux_from_gates | muxpins_from_gates | muxx_from_gates
    | nomux_from_gates> |
<tied_gates_with_pin_loss | notied_gates_with_pin_loss> |
<wire_to_buffer | nowire_to_buffer>
<xor_from_gates | xorpins_from_gates | noxor_from_gates>}
```

TestMAX ATPG has 16 model flattening optimization algorithms, of which 11 are under user control. You control them by using the `-merge` option before you use the `run_build_model` command. These algorithms are all designed to reduce the number of gates in the final image, allowing larger designs in the same memory and less CPU effort than the original design. These controls are independent of each other. To adjust more than one control, repeat the `-merge` option as needed. All optimization algorithms are enabled by default except for `tied_gates` and `cascaded_gates`. For additional information, see [Understanding Flattening Optimization](#).

-net_connections_change_netlist | -nonet_connections_change_netlist

If `-nonet_connections_change_netlist` is specified, then net connections only affect the flatten model. The default is `-net_connections_change_netlist`.

All net connections of type `-module` or `-all` are resolved during flattening at the netlist level, thus modifying the netlist.

Note that, in concordance with existing restrictions in the `add_net_connections` command, net connections of type `-module` or `-all` must be TIEs (no PI, PO, PIO) and moreover, they cannot have a port name or use the `-remove` argument (`-disconnect` is supported).

Because netlist data can now be changed by net connections of type `-module` or `-all` (not just the flatten model), there are several implications:

Modules changed by net connections are marked by "(nc)" at the end of the line in the report modules output. Such modules cannot also have the "(BB)" or "(EB)" marker (black box and design box).

If a `write_netlist` command is issued and the modules written include one or more modules modified by net connections, a new warning message is output:

```
"There were number modules changed by net connections (M248) "
```

Net connections that were used to modify one or more modules cannot be removed by a `remove net connections` command unless the netlist data is first deleted with a `read_netlist -delete` command. If an attempt is made to remove such net connections, a new warning message is output:

```
"Cannot remove net connection (<connection_name>) because it changed netlist data. (M249) "
```

or, for multiple net connections:

```
"There were number net connection not removed because they changed netlist data. (M249) "
```

-undriven_bidi <pio | pi | po>

Selects how to model undriven bidirectional nets. By default, a BIDI pin that is not driven internally remains a bidi pin, even though it is behaving as an input. This can be changed to an input or an output pin through this switch.

Allowed Command Modes

Build

Description

This command sets the parameters that control how TestMAX ATPG builds a simulation model when you specify the `run_build_model` command. Any changes you make using the `set_build` command only affect the future use of the `run_build_model` command. They do not affect an existing flattened image created from the `run_build_model` command.

Examples

```
BUILD-T> set_build -hierarchy . -nodelete_unused_gates
```

```
BUILD-T> set_build -delete_unused_gates
```

```
BUILD-T> set_build -reset_boxes
```

```
BUILD-T> set_build -black_box [ list MY_MODULE PLL_CORE ]
```

See Also

[report_settings](#)
[run_build_model](#)
[S18](#)

set_buses

Overview

Use this command to set the parameters that control the behavior of bus gates.

Syntax

```
set_buses
  [-contention_status <<ignore | noignore> <gate_id | pin_pathname | all>>...]

  [-external_z <z | 0 | 1 | x>]
  [-fault_contention <and | or | x>]
  [-zstate_status <<ignore | noignore> <gate_id | pin_pathname | all>>...]
```

Arguments

-contention_status <<ignore | noignore> <gate_id | pin_pathname | all>>....

Ignores selected buses for contention.

-external_z <z | 0 | 1 | x>

Specifies the behavior of the Z state for external pins and determines the usability of the Z state for fault detection. The default is *z*, which enables detection of Z states on outputs and bidirectional ports. If your target tester does not support this behavior, you need to disable it by selecting *x*.

-fault_contention <and | or | x>

This option affects only the fault simulator, and has no effect on the test generator itself. Adjusts the fault machine simulated value for the bussed net when buses are in contention (multiple drivers on). The default is *X*. When a value of *and* is selected, the simulated value in the faulty machine when contention occurs is the AND of the values, which is often 0. Likewise, a selection of *or* produces a fault machine-simulated value, which is the OR of the data values, most often 1. Using a simulated contention value, which is 0 or 1 instead of *X*, can make it easier to detect faults on the tristate enable control lines. This is not possible if the faulty machine simulated value is *X*. In essence, when a value other than *X* is selected, the fault simulator "pretends" it can still be able to detect faults on tristate enable lines, even though contention is occurring. This setting does not affect the simulated value of the GOOD machine.

This option is not yet implemented for the Full-Sequential pattern generator.

-zstate_status <<ignore | noignore> gate_id | pin_pathname | all>>...

Ignores selected buses for z-state checking.

Allowed Command Modes

All

Description

Use this command to set the parameters that control the behavior of bus gates. You can display the current settings with the `report_settings bus` command.

Examples

```
DRC> set_buses -external_z z
```

See Also

[report_settings](#)

[report_buses](#)

set_colors

Overview

This command sets the colors for selected transcript messages.

Syntax

```
set_colors  
  <error | warning | user_commands | file_commands>  
  <red_number> <green_number> <blue_number>
```

Arguments

error

Changes the color of error messages in the transcript. The default color is red.

warning

Specifies changing the color of warning messages in the transcript. The default color is magenta.

user_commands

Changes the color of user command messages in the transcript. The default color is blue.

file_commands

Changes the color of file command messages in the transcript. The default color is green.

red_number

The intensity of the red component, an integer between 0 and 255 inclusive. The higher the number, the greater the amount of red.

green_number

The intensity of the green component, an integer between 0 and 255 inclusive. The higher the number, the greater the amount of green.

blue_number

The intensity of the blue component, an integer between 0 and 255 inclusive. The higher the number, the greater the amount of blue.

Allowed Command Modes

All

Description

This command sets the colors for selected transcript messages. All message text of the selected type is changed in color, including existing text in the transcript.

Examples

```
# the color of error transcript messages is set to blue  
DRC> set_colors error 0 0 255
```

```
# the color of user command transcript messages is set to red  
DRC> set_colors user_commands 255 0 0
```

See Also[report_settings](#)

set_commands

Overview

Use this command to control the command history or to specify the handling of errors during command file execution.

Syntax

```
set_commands
[history | nohistory] [abort | noabort | exit]
[-secure <command | all> | -nosecure <command | -all>]
```

Arguments

history | nohistory

Specifies whether a command history is stored. The default is `history`.

abort | noabort | exit

Specifies what happens when an error occurs during execution of a [command file](#). `Abort` causes execution of the command file to stop at the error. `Noabort` allows execution to continue after the error. `Exit` causes termination of the TestMAX ATPG session at the error. The default is `abort`. Note the following:

- You cannot use the Control-c keyboard combination to replicate the `abort` option during the execution of the set.
- The `-noabort` option will override Tcl script variables "set sh_script_stop_severity E" and "set sh_continue_on_error False." You should use either the `set_commands` options or the Tcl variables in a script flow. Using both may result in unpredictable results.

-secure command | -all> | -nosecure <command | -all>

Specifies which commands are included in a password-protected secure image file created when you next issue a `write_image` command using the `-password` option. You must be careful when choosing these commands as all options of a command are permitted. Any action that requires writing an image is not allowed in the secure state.

`-secure command`: Specifies that the given command or list of commands is permitted in a subsequently secure written image file. Command abbreviations can be used.

`-secure -all`: Allows all commands in a subsequently secure written image file. This is used to generate a password-protect image file, but without limiting a legitimate receiver.

`-nosecure command`: Disallows a previously permitted command in a subsequently secure written image file. Command abbreviations can be used.

`-nosecure -all`: Disallows all commands in a subsequently secure written image file except for the following commands which are always allowed: `exit`, `alias`, `unalias`, `help`, `source`, `cd`, `pwd`, `usage`. This is the default setting when a secure image file is created without using the `-secure` option.

Allowed Command Modes

All

Description

Use this command to control the command history or to specify the handling of errors during command file execution. You can display the current settings with the `report_settings` command.

This command can also be used to specify the command or commands allowed after reading in a password-protect image file. For more information on using this command in conjunction with creating an image file, see ["Creating a Secure Image File."](#)

Note: Commands executed from a command file are *NOT* recorded into the command history.

Examples

```
DRC> set_commands noabort # don't quit command files on error
```

```
DRC> set_commands history # turn on command history
```

```
TEST> set_commands -secure run_diagnosis # allow run diagnosis command in  
secure image file
```

See Also

[Binary Image Files](#)

[report_settings](#)

[report_commands](#)

Table Column Outside Table:

set_contention

Overview

Use this command to specify the types of contention checking performed and the reporting of these conditions.

Syntax

```
set_contention
[abus | noabus] [bidi | nobidi] [bus | nobus] [dff_dlat | nodff_dlat]
[float | nofloat] [ram | noram] [wire | nowire]
[-atpg | -noatpg] [-capture | -nocapture]
[-capture_constraint_check | -nocapture_constraint_check]
[-preclock | -nopreclock]
[-multiple_on | -nomultiple_on]
[-retain_bidi | -noretain_bidi]
[-severity <error | warning | ignore>]
[-verbose | -noverbose]
```

Arguments

abus | noabus

Specifies whether contention checking is performed for address ports on RAMs. ABUS contention occurs when an illegal address appears at an address ports of a RAM during write operation for this address port. When you select the `abus` option, any patterns generated that have ABUS contention are identified and discarded. The default is `noabus`.

bidi | nobidi

Specifies whether contention checking is performed for bidirectional ports. Bidirectional port contention occurs when both the device under test and the testbench drive values onto the same pin, whether or not they are driving the same value. Patterns that cause contention are discarded in favor of other patterns. The default is `bidi`, which causes this type of contention checking to be performed.

bus | nobus

Specifies whether contention checking is performed for buses. Bus contention occurs when there are opposing strong values driven on a BUS primitive. When you select the `bus` option, any patterns generated that have bus contention are identified and discarded. The point in the cycle at which the bus contention check is performed (preclock, clock-on, postclock) is affected by the `-capture` and `-preclock` settings. The default is `bus`.

dff_dlat | nodff_dlat

Specifies whether contention checking is performed for clock/set/reset pins on latches and flip-flops. Contention occurs when more that one clock/set/reset pin is active at the same time on a DLAT or DFF primitive. When you select the `dff_dlat` option, any patterns generated that have port contention are identified and discarded. The default is `nodff_dlat`, which means that this type of contention checking is not performed.

float | nofloat

Specifies whether a floating bus condition is considered a form of contention that is to be avoided. Contention occurs when there are no drivers for an internal BUS gate. With a selection of `float`, patterns generated with a floating bus condition are discarded and additional pattern generation is attempted. The point in the cycle at which the floating check is made (preclock, clock-on, postclock) is affected by the `-capture` and `-preclock` settings. The default is `nofloat`.

ram | noram

Specifies whether contention checking is performed for write ports on RAMs. RAM contention occurs when more than one write port of a RAM is active at the same time on a RAM primitive. When you select the `ram` option, any patterns generated that have RAM contention are identified and discarded. The default is `noram`.

wire | nowire

Specifies whether contention checking is performed for wire gates. Wire contention occurs when there are opposing values driven on a wire primitive. When you select the `wire` option, patterns that have wire contention are identified and discarded. The default is `nowire`.

-atpg | -noatpg

Specifies whether ATPG methods are used to avoid preclock and clock-on contention. When `-noatpg` is selected, patterns are generated and then checked for contention, and if contention is found, the pattern is discarded. With `atpg` selected, patterns are generated while considering contention so that there is no need to discard patterns. Use of ATPG methods to avoid contention while patterns are being generated is more CPU intensive. The ATPG avoidance algorithm is only available for the preclock and clock-on contention checks. postclock contention is not supported by the ATPG contention avoidance algorithm. The default is `-atpg`.

-capture | -nocapture

Specifies whether contention checking is performed after a capture clock is applied (postclock). The default is to check for contention due to the application of a capture clock. If any pattern has this type of contention, it is discarded and other patterns is attempted. (See the following figure.)

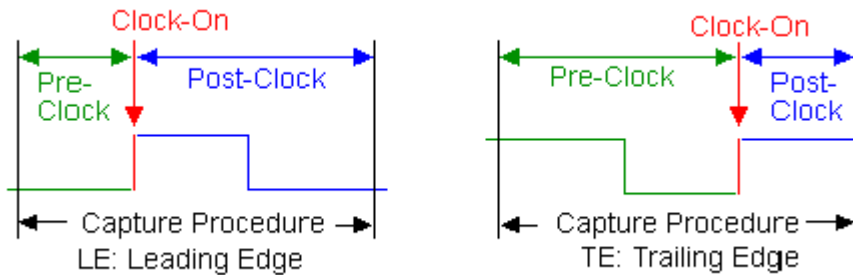
Note: This option is not yet implemented for Full-Sequential ATPG, which defaults to `-capture` behavior.

-capture_constraint_check | -nocapture_constraint_check

Specifies whether to check for ATPG constraints after a capture clock is applied. The default (`-nocapture_constraint_check`) does not check for ATPG constraints after applying a capture clock.

-preclock | -nopreclock

Specifies whether contention checking is performed from the point at which inputs are changed (before any optional capture clocks). The default is to check for contention from the start of a cycle. If any pattern has this type of contention, it is discarded and other patterns are attempted.



This figure shows the different portions of a typical capture clock cycle and where contention checking occurs. The preclock contention checking is the portion of the cycle from the force PI events until the clock-on event. The clock-on contention checking occurs at the instant the clock is asserted, but before the changes which would occur from propagating sequential gate inputs to their outputs. The postclock contention checking is done after the clock event and after the sequential gate outputs are updated due to the clock. preclock and postclock contention checking have individual controls. Clock-on contention checking is generally controlled with the `bus/nobus` option.

Note: This option is not yet implemented for Full-Sequential ATPG, which always uses `-preclock` behavior.

-multiple_on | -nomultiple_on

Specifies whether bus contention is considered to occur when more than one strong driver is active, even if they drive the same value. Selecting `-multiple_on` means that multiple drivers on with the same value are allowed and not considered contention. Selecting `-nomultiple_on` means that having multiple drivers on is considered contention, regardless of the driven value. The default is `-multiple_on`.

-retain_bidi | -noretain_bidi

Specifies whether the bidirectional ports must retain their original direction after a clock is applied. The default is not to require the signal direction to remain the same.

-severity <error | warning | ignore>

Indicates the handling of a contention condition. If you select `error` and a contention condition occurs, the simulation is terminated and an error message reports the contention site and the number of rejected patterns. You can view the contention pattern by using the `error_data` option of the `set_pindata` command. If you select `warning` and a contention condition occurs, a warning message reports the contention site and the number of rejected patterns, and the simulation continues. If you select `ignore` and a contention condition occurs, no message is given and the simulation continues. The default is `warning`.

-verbose | -noverbose

Specifies whether all contention conditions are reported. The default is `-noverbose`, where only a single contention message per simulation generation pass is reported.

Allowed Command Modes

All

Description

Use this command to specify the types of contention checking performed and the reporting of these conditions. Patterns that violate contention conditions are discarded.

Examples

```
DRC> set_contention bus -atpg -capture -multi -severity warn -verbose
```

See Also

[report_settings](#)

[Contention Analysis](#)

set_delay

Overview

Use this command to specify options for transition fault and path delay fault test generation and fault simulation.

Syntax

```
set_delay
[-allow_nonrobust_tests | -noallow_nonrobust_tests]
[-allow_reconverging_paths | -noallow_reconverging_paths]
[-allow_multiple_common_clocks | -noallow_multiple_common_clocks]
[-data path_name]
[-common_launch_capture_clock | -nocommon_launch_capture_clock]
[-diagnostic_propagation | -nodiagnostic_propagation]
[-disturb | -nodisturb]
[-extra_force | -noextra_force]
[-infinite_slack_checks | -noinfinite_slack_checks]
[-launch_cycle <last_shift | system_clock | extra_shift | any>]
[-mask_nontarget_paths | -nomask_nontarget_paths]
[-max_delta_per_fault float | inf | tmgn%]
[-max_tmgn float | defect%]
[-multicycle_length N]
[-negative_slack_checks | -nonegative_slack_checks]
[-pi_changes | -nopi_changes]
[-po_measures | -nopo_measures]
[-relative_edge | -norelative_edge]
[-robust_fill | -norobust_fill]
[-simulate_hazards | -nosimulate_hazards]
[-slackdata_for_atpg | -noslackdata_for_atpg]
[-slackdata_for_faultsim | -noslackdata_for_faultsim]
[-slow_equivalence | -noslow_equivalence]
[-transition_clocking_checks | -notransition_clocking_checks]
[-two_clock_transition_optimization | -notwo_clock_transition_
optimization]
```

Arguments

-allow_nonrobust_tests | -noallow_nonrobust_tests

By default, TestMAX ATPG accepts all robust and nonrobust tests for path delay faults, and retains all patterns that provide this type of detection for one or more faults. If you specify the `-noallow_nonrobust_tests` option, TestMAX ATPG does not recognize faults detected from nonrobust tests, and does not retain the patterns containing only nonrobust detections. If a pattern provides both robust and nonrobust detections of path delay faults, it is retained in the pattern set. However, TestMAX ATPG recognizes only the faults detected from robust tests. You can reset the default setting using the `-allow_nonrobust_tests` option (the default).

This option applies only to the fast-sequential path delay test generator. Nonrobust tests are accepted in full-sequential mode.

-allow_reconverging_paths | -noallow_reconverging_paths

The `-allow_reconverging_paths` option causes TestMAX ATPG to reconverge off-path segments when redundant logic prevents a specified path segment from being individually tested. The standard definition of a [path delay fault](#) causes TestMAX ATPG to classify these paths as untestable. However, it might be possible for TestMAX ATPG to test the on-path and off-path segments together using this relaxed criteria. A [reconverging path](#) might cause a path to be immediately marked as untestable, but this option would allow test generation to occur if allowed. The default is not to allow reconverging paths (the `-noallow_reconverging_paths` option).

NOTE: The `-allow_reconverging_paths` option might cause confusing output from the `analyze_faults` command. It might be best to use the default, `-noallow_reconverging_paths`, when doing fault analysis.

-allow_multiple_common_clocks | -noallow_multiple_common_clocks

Uses multiple clocks for common launch/capture clocking for clock-launch transition fault patterns. Enabled by default if the `-common_launch_capture_clock` option is issued before DRC.

The `-allow_multiple_common_clocks` option identifies clocking pairs that can be grouped with infinite skew. These combinations are then allowed when ATPG is run with both the `-allow_multiple_common_clocks` and `-common_launch_capture_clock` options specified.

-common_launch_capture_clock | -nocommon_launch_capture_clock

This option disables dynamic clocking and uses only a single clock for launch and capture. That is, it avoids generating patterns with one clock being used for launching the transition and another clock being used for capturing the transition. You cannot specify `-common_launch_capture_clock` in the TEST command mode. The default is `-nocommon_launch_capture_clock`.

TestMAX ATPG performs conservative grouping (which assumes infinite tester skew) during clock grouping analysis when both `-allow_multiple_common_clocks` and `-common_launch_capture_clock` are specified.

The `-common_launch_capture_clock` option is supported by the `-launch_cycle system_clock` or `-launch_cycle extra_shift` options of the `set_delay` command. When TestMAX ATPG is generating last shift-launch patterns (using either the `set_delay -launch_cycle last_shift` command or the `set_delay -launch_cycle any` command), there is no simple way to prevent transitions from being launched in one clock domain and captured in another.

This option has the same effect in both TestMAX ATPG and TestMAX ATPG. But in TestMAX ATPG it is used during stuck-at ATPG to constrain two-cycle patterns.

-data path_name

Sets the GSV and primitive report data to load the path set by this option. The `set_pindata delay_data` command allows viewing this data in the GSV.

-diagnostic_propagation | -nodiagnostic_propagation

Enables ATPG to be performed with a more stringent set of rules for [off-path inputs](#). For a diagnostic propagation path delay fault test, all off-path inputs must have a constant propagating value, which means that only the target path is tested at its endpoint. This

type of test might be useful for characterizing the timing of a particular path, or for better diagnostic resolution of a failing path delay test. Note that requiring such path delay fault tests will result in lower overall coverage. The default is to not generate diagnostic propagation tests: `-nodiagnostic_propagation`.

-disturb | -nodisturb

Enables disturbed clock grouping for two-clock optimized transition fault ATPG. The default is `-disturb`.

TestMAX ATPG silently ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-extra_force | -noextra_force

The default, the `-extra_force` option, provides additional delay for force PI behavior. Some ATPG engines place an additional force of all PI values at the start of the capture operation before the launch sequence starts for that particular transition capture sequence. This behavior enables the assertion of PI states in the additional vector or capture procedure before the clock events of the launch operation and provides extra settling time for the PI states before the launch. Depending on the capture procedures defined in the STL procedure file, this additional force PI might be an inline vector, a call to an unclocked capture procedure, or a call to the multiclock_capture procedure, before the vector initiates the launch operation.

If you do not want the additional test cycles and the additional PI settling time, suppress the force PI event using the `-noextra_force` option.

TestMAX silently ignores this option. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

-infinite_slack_checks | -noinfinite_slack_checks

Enables or disables infinite slack check during read timing file, slack-based transition fault test generation, or fault simulation. During read timing file, warning message [M751](#) is printed if this check is enabled. During slack-based transition fault test generation or fault simulation, warning message [M753](#) is printed if this check is enabled. The default is `-noinfinite`.

-launch_cycle <last_shift | system_clock | extra_shift | any>

This option specifies how the transition is launched based on the transition fault model. The `last_shift` parameter launches the transition on the last shift clock of the scan chain load. A capture clock procedure is then used to observe the transition. The `system_clock` parameter (the default) launches a transition launch procedure that captures data from its functional data input, and a second capture clock procedure is used to observe it. If you use the `system_clock` parameter, fast-sequential patterns or two-clock optimized patterns are generated, and the `-capture_cycle` option of the `set_atpg` command is automatically set to 2 if it is not already set to 2 or higher. The `extra_shift` parameter launches a logic value based on one more shift than the `last_shift` parameter. This ensures that all clock domains receive their last scan shift before the internally controlled capture clock pulse. Unlike the `last_shift` parameter, the `extra_shift` parameter does not place additional timing requirements on an OCC controller. The `any` parameter allows the use of any of the `-launch_cycle` option parameters, including the `last_shift`, `system_clock`, and `extra_shift` parameters.

Note the following:

- The `-launch_cycle` option affects ATPG, fault simulation, and diagnostics.
- This option has no affect when the `Path_delay` fault model is selected.
- The `last_shift` and `any` parameters work do not respect the `-common_launch_capture_clock` option of the `set_delay` command. Note that the `any` parameter allows the generation of last-shift patterns.
- This option switches to non-threaded processing. The only exception is when `system_clock` or `extra_shift` is specified as a value, which is supported by multithreading.

`-mask_nontarget_paths | -nomask_nontarget_paths`

With this option, only the observation point for a detected, targeted path is measured, and all other scan cell and primary output values are masked out. This might be useful when there are many false paths or multicycle timing paths in the circuit that could be unintentionally exercised by path delay test patterns. Note that the `-mask_nontarget_paths` option must be used in high X-tolerant mode only. The default, `-nomask_nontarget_paths`, is to capture and measure all valid simulation output values.

`-max_delta_per_fault float | inf | tmgn%`

Sets the maximum delta value (detection slack - minimum slack) considered acceptable for slack-based transition fault testing. This value can be specified as a floating number (*float*), by *inf* value, or as a percentage (*tmgn%*) of *tmgn* (timing margin). The *inf* value specifies an infinite delta, so that all faults detected after being targeted by the slack-based transition fault testing algorithm are categorized as DS (detected by simulation).

Faults detected with a delta less than or equal to the `-max_delta_per_fault` setting are categorized as DS faults and are not simulated again. Faults detected with a delta larger than this value are categorized as TP (transition partially detected) faults, but are simulated again and targeted for DS detection later in the test set. The default setting is 0, which categorizes faults as DS faults only if they are detected on their longest (minimum slack) path. If you specify a percentage, the valid percentage range is from 0 to 100.

Note the following:

If you specify the maximum delta value using the *tmgn%* specification, this percentage value is used until the first invocation of the [run_atpg](#) command. At this point, TestMAX ATPG computes the actual maximum delta value for each fault based on the specified percentage of the fault's *tmgn* number, and stores these values in the fault records.

A low setting specified by the `-max_delta_per_fault` option produces higher quality test sets, but also increases CPU time and produces higher pattern counts.

Detected faults with slacks greater than the `-max_tmgn` setting, including faults with infinite slack, are still given a TP fault status when "inf" is specified."

`-max_tmgn float | defect%`

Controls the test generation target faults for slack-based transition fault test generation. You can specify this setting with either a floating number (*float*) or as the percentage of faults as the slack-based testing target (*defect%*). Faults with a minimum slack less than or equal to the setting specified by the `-max_tmgn` option is targeted by the slack-based transition fault testing algorithm during test generation. Other faults are targeted by the normal transition fault test generation algorithm.

The default behavior is for all faults to be used as target faults for slack-based transition fault test generation. Note that setting the `-max_tmgn` parameter to a large number, or

leaving it at its default, might produce a slightly higher quality test set, but will do so at the cost of increased CPU time and higher pattern counts. The `set_delay -max_tmgn 3.0` command targets all faults with tmgn less than 3.0 with the slack-based test generation algorithm. For more information, see ["Slack-Based Transition Fault Testing."](#)

If you specify `set_delay -max_tmgn 50%`, you are setting the max_tmgn to include 50% of all the faults with the smallest tmgn faults as the slack-based transition fault test target. A message is printed at the start of the test generation that informs you of the numerical value of the max_tmgn calculated from the percentage. Note that the percentage remains as a percentage until the first [run_atpg](#) invocation; at which point it is frozen into the delay number appropriate at that time.

This implementation is based on following assumptions:

1. All faults are considered, not just the active faults. This is to maintain the same fault base in case of multiple ATPG runs. Otherwise the set of slack-based transition target faults is a moving target.
2. The max_tmgn is adjusted according to the percentage you specify. As a result, it might not be possible to have the exact percentage of faults sampled. For example, if 1/2 of the faults have tmgn=1, and 1/2 of faults have tmgn=infinite, an 80% setting is essentially the same as 100% since it will internally set the max_tmgn to infinite.
3. The number of faults can be counted either as collapsed or uncollapsed, which is controlled by the command `report_faults -collapsed | uncollapsed`.

Example:

```
TEST> set_delay -max_tmgn 80%
TEST> run_atpg -auto
The max tmgn for slack-based transition faultshas been set to
13.23
...
```

-multicycle_length N

Used in conjunction with specifications in an SDC file, this option treats multicycle paths below a certain number as if they are single-cycle paths. Based on this option, all exceptions specified by the `set_multicycle_path` SDC command with numbers of N or less are ignored. The default is to treat all multicycle paths of length 2 or greater as exceptions. Note that this option is valid only in Tcl mode.

For more information on the `set_multicycle_path` SDC command, see [How TestMAX ATPG Interprets SDC File Commands](#).

-negative_slack_checks | -nonegative_slack_checks

Enables or disables negative slack check during read timing file, slack-based transition fault test generation, or fault simulation. During slack-based transition fault test generation or fault simulation, warning message [M752](#) is printed if this check is enabled. The default is `-nonegative_slack_checks`.

-pi_changes | -nopi_changes

Specifies whether patterns are allowed to change Primary Inputs (PIs) between the [launch clock](#) and the capture clock for clock launch transition fault patterns or for any [path delay fault](#) patterns. This will have no affect on last_shift_launch transition fault patterns.

Note that the `-nopi_changes` option might in some cases produce results that are inferior to simply constraining all PIs to a known value or even to X. The reason for this somewhat paradoxical result is that this option essentially imposes a constraint on the PIs, but it is such a complex constraint that it might increase the backtracks that lead to aborted faults. The `-nopi_changes` option causes an extra unlocked tester cycle to be added to each generated transition fault or path delay pattern. The use of a `set_drc -clock -one_hot` command might interfere with the addition of this unlocked cycle and is not recommended for use when the `-nopi_changes` option is in effect. The unlocked cycle is not added when internal clocking is used, nor is it added by the Full-Sequential test generator.

The default allows the inputs to change between the launch and capture events: `-pi_changes`.

`-po_measures | -nopo_measures`

Specifies whether primary outputs can be measured between the [launch clock](#) and the capture clock. Use this option when the ATE does not have sufficient timing accuracy to strobe outputs before the capture event. This option has no effect when the transition fault model has been selected. For transition fault ATPG, the `add_po_masks -all` command can be used to prevent primary output measurements. The default is to allow primary output measurements: `-po_measures`.

`-relative_edge | -norelative_edge`

Specifies whether both the slow to rise and slow to fall faults is added to the fault list even though only one fault was defined in the path definition file read into TestMAX ATPG. This option should be set before paths are added. The default is to honor the absolute fault definitions: `-norelative_edge`.

`-robust_fill | -norobust_fill`

ATPG will normally fill all unused scan cells and primary inputs with random values. For path delay tests, ATPG will first fill unused scan cells when the data being clocked into the cell is known so that the output remains stable. This can increase the likelihood of a [robust test](#). The default is to include this additional filling algorithm: `-robust_fill`.

`-simulate_hazards | -nosimulate_hazards`

Allows the fault simulator to ignore potential glitches on [off-path inputs](#) that might mask [path delay fault](#) propagation. The default, `-simulate_hazards`, is to use an extended simulation algorithm that determines when such hazards are potentially present on off-path inputs. This results in a slightly looser interpretation of [robust detection](#).

`-slackdata_for_atpg | -noslackdata_for_atpg`

Turns on and off the slack-based transition fault testing during ATPG. If slack data exists, the default is `-slackdata_for_atpg`.

`-slackdata_for_faultsim | -noslackdata_for_faultsim`

Turns on and off slack-based transition fault testing during fault simulation. If slack data exists, the default is `-slackdata_for_faultsim`.

`-slow_equivalence | -noslow_equivalence`

Considers that both slow-to-rise and slow-to-fall transition faults at a fault site are detected when either is detected. This ability is supported for all Basic Scan and Fast-

Sequential fault simulation and test generation flows. The additionally detected faults are placed in the DI fault category. The default is `-noslow_equivilance`.

`-transition_clocking_checks` | `-notransition_clocking_checks`

Identifies the conditions to launch (considering hold behavior), capture a fault, and immediately assign the values during two-clock transition fault test generation. The default is `-notransition_clocking_checks`.

`-two_clock_transition_optimization` | `-notwo_clock_transition_optimization`

Enables an optimized algorithm for generating system-clock launch transition patterns, if the `-launch_cycle` option does not use the `last_shift` argument. If the number of capture cycles is set to greater than 2 using the `set_atpg -capture` command, then another pass with Fast-Sequential ATPG is performed. The default is the `-two_clock_transition_optimization` option.

To run two-clock optimized ATPG, the following conditions must be satisfied:

The transition fault model and the clock launch transition type must be specified

Merging must be enabled using the (`run_atpg -auto_compression` command or the `set_atpg -merge` command

Basic Scan ATPG must be enabled

Note that the following changes occur when using two-clock optimized ATPG:

The `capture_clock` value is set to 2, and remains at that value for the complete session unless it is redefined.

The `set_drc -multi_captures_per_load` command is enabled for the complete session, unless it is redefined.

Fast-sequential ATPG is disabled (Note that two-clock optimized ATPG is considered a Basic Scan process).

The `set_atpg -analyze_untestable_faults` command is enabled.

All ATPG progress messages are written in a verbose form

Allowed Command Modes

All, except the `-common_launch_capture_clock` option cannot be specified in the TEST command mode.

Description

Use this command to specify options for transition fault and path delay fault test generation and fault simulation.

Examples

```
DRC> set_faults -model transition
DRC> set_delay -launch_cycle system_clock -nopi_changes
```

See Also

[set_faults](#)

set_diagnosis

Overview

This command specifies the parameters associated with performing a diagnosis of a failing device.

Syntax

```
set_diagnosis
  [-add_default_offset {value}]
  [-auto | -noauto]
  [-check_expected_data | -nocheck_expected_data]
  [-composite | -nocomposite]
  [-cycle_offset d]
  [-default_offset {pat_namevalue}]
  [-dftmax_chain_format | -nodftmax_chain_format]
  [-failing_compressor { number | patterns | failures }]
  [-failure_memory_limit d | -nofailure_memory_limit]
  [-fault_type [ all | shared | transition | stuck ]
  [-filter_bridges { [ambiguous <on | off>] [same_cell < none | feedback |
all >]] ]
  [-filter_candidates {score_distance <distance | off> | [min_score
<distance | off> ] max_number <number | off>}} ]
  [-hide value]
  [-incomplete_failures | -notincomplete_failures]
  [-intermediate_chain_faults]
  [-mapping_report | -nomapping_report]
  [-max_defective_chains n]
  [-max_fail_gates n]
  [-max_report_failures d]
  [-num_processes number]
  [-organization class | fault]
  [-original_pattern_id]
  [-pattern { p1 p2 on|off }]
  [-per_pin_limit | -no_per_pin_limit]
  [-post_procedure procedure_name | none]
  [-process_fail stop | continue | rerun]
  [-reset_filters]
  [-sample d1 d2]
  [-show value]
  [-time_limit d | -notime_limit]
  [-use_default_offset | -nouse_default_offset]
  [-use_dictionary | -nouse_dictionary]
  [-use_phds {values}]
  [-verbose | -noverbose]
```

Arguments

-add_default_offset {value}

This option, which is used for split patterns diagnostics, adds the specified *value* to all default offsets. A default offset is the cycle offset between each split patterns file. Default offsets are computed automatically when the split patterns are read for diagnosis; one default offset is defined for each split pattern file.

You can use this option if the offset value is constant between all split patterns files. Any manual initialization changes directly affect the value of the default offsets. The failures log file must use the cycle-based format for this option to work.

The `set_diagnosis` command prints the updated default offsets, as shown in the following example:

```
Pattern_file=pat.ser_0.stil, basic_scan=[0(3),9(137)], fast_sequential=[-1,-1], default_offset=0
Pattern_file=pat.ser_1.stil, basic_scan=[10(152),19(287)], fast_sequential=[-1,-1], default_offset=14
Pattern_file=pat.ser_2.stil, basic_scan=[20(302),29(437)], fast_sequential=[-1,-1], default_offset=28
Pattern_file=pat.ser_3.stil, basic_scan=[30(452),32(482)], fast_sequential=[-1,-1], default_offset=42
```

The `set_diagnosis -add_default_offset {10}` command prints the following updated default offsets:

```
Pattern_file=pat.ser_0.stil, basic_scan=[0(3),9(137)], fast_sequential=[-1,-1], default_offset=0
Pattern_file=pat.ser_1.stil, basic_scan=[10(152),19(287)], fast_sequential=[-1,-1], default_offset=24
Pattern_file=pat.ser_2.stil, basic_scan=[20(302),29(437)], fast_sequential=[-1,-1], default_offset=48
Pattern_file=pat.ser_3.stil, basic_scan=[30(452),32(482)], fast_sequential=[-1,-1], default_offset=72
```

The effect of the `-add_default_offset` option is cumulative. For example, if the command is used twice, the specified values from each command are combined:

```
TEST-T> set_diagnosis -add_default_offset {1}
Pattern_file=pat.ser.stil_0.gz, basic_scan=[0(87),399(3103505)], fast_sequential=[-1,-1], default_offset=0
Pattern_file=pat.ser.stil_1.gz, basic_scan=[400(3111283),799(6214703)], fast_sequential=[-1,-1], default_offset=863
Pattern_file=pat.ser.stil_2.gz, basic_scan=[800(6222481),1199(9325903)], fast_sequential=[-1,-1], default_offset=5726

TEST-T> set_diagnosis -add_default_offset {1}
Pattern_file=pat.ser.stil_0.gz, basic_scan=[0(87),399(3103505)], fast_sequential=[-1,-1], default_offset=0
Pattern_file=pat.ser.stil_1.gz, basic_scan=[400(3111283),799(6214703)], fast_sequential=[-1,-1], default_offset=864
Pattern_file=pat.ser.stil_2.gz, basic_scan=[800(6222481),1199(9325903)], fast_sequential=[-1,-1], default_offset=5728
```

For more information, see "[Split Patterns Diagnosis for Cycle-Based Failures.](#)"

-auto | -noauto

When the `-auto` option is set and the chain test pattern fails, the `run_diagnosis` command performs chain diagnostics (the same behavior as if you specify the `-chain_failure` option). If you specify the `-noauto` argument, the `run_diagnosis` command

performs functional logic diagnosis (the same behavior as if you do not specify the `-chain_failure` option).

-check_expected_data | -nocheck_expected_data

This option recognizes a field in the tester failure data file as the expected data for that measure. It performs a comparison between this data and the expected data from the pattern file. If the compared values do not match, this option displays the [M530](#) warning and stops the diagnosis. Otherwise, diagnosis proceeds normally. It does not compare masked outputs. It supports scan and adaptive scan designs.

The expected value field is the last one on the line (fourth field from the left for scan shifting failures, and third field from the left for capture cycle failures). The output format from the `run_simulation` command is also supported. The H/L format is not accepted. It is treated as a comment and simply ignored. The result is that the expected data will not be checked.

Note: To indicate the expected data during a capture of a scan chain output and use `-check_expected_data` option, you must use the `exp=` syntax. For example:

```
103 scan_out3 1 #invalid
103 scan_out3 (exp=1, got=0) #valid
103 scan_out3 exp=1 #valid
```

Use this option as a sanity check of the process that translates tester failures into the TestMAX ATPG failure data file. Mismatches indicate that the tester cycles are not properly mapped to the pattern numbers. Diagnosis will report the number of compared values. The default is `-check_expected_data`. If no expected data is present in the failures log file, no message is printed.

-composite | -nocomposite

The `-composite` option is used for the diagnosis ranking flow (`run_diagnosis <file> -rank_faults`). It enables diagnosis to perform a more precise fault ranking using complex fault models, such as bridging or transition. The default is `-nocomposite`. As a result, during ranking, only simple stuck-at models are used.

If both `sa0` and `sa1` are defined for the same instance, the usage of the `-composite` option will rank the `sa01` fault. If you want to rank the `sa0` and the `sa1` faults independently, use `-nocomposite`.

If both `str` and `stf` are defined for the same instance, the usage of the `-composite` option will rank the `strf` fault. If you want to rank the `str` and the `stf` faults independently, use `-nocomposite`.

If default setting is used (that is, `-nocomposite`), and the bridging faults are defined (even if the faults defined are equivalent to a bridging composite fault), then the component faults (`ba0` and `ba1`) are ranked.

Here is an example:

```
#Fault defined in fault.txt
node1 node2 ba0
node2 node1 ba0

#TestMAX ATPG Script
set_faults -model bridging
add_faults -node fault.txt
```

```
set_diagnosis -nocomposite
run_diagnosis fail.txt -rank_faults
```

In the previous example, the definition of the bridging faults is equivalent to defining a WAND bridging fault (node1 node2 wand). However, because the `-nocomposite` option of the `set_diagnosis` command is used, only the component faults is ranked.

If the `-composite` option of `set_diagnosis` command is used, and the faults defined are equivalent to a bridging composite fault, then the composite faults are ranked.

-cycle_offset d

Enables you to adjust the cycle count when the cycle numbering for cycle-based failure log files does not correspond to the cycle numbering for the pattern. TestMAX ATPG adds the specified positive or negative integer offset to the cycle counts of the failure log file. The cycle counts derived by counting the vector or V statements in the STIL or WGL pattern set always starts at 1. If the ATE counts cycle number starting with cycle 0 as the first cycle, use an argument of 1 to adjust this. If the tester failure log includes additional initialization cycles that are not accounted in the patterns, specify a negative integer as the argument. The default is 0.

An example usage of this option is as follows:

If you use test patterns generated by TestMAX ATPG directly in the ATE, the test_setup cycles on the ATE are, by default, the same as the test_setup cycles in the pattern file read for diagnosis. In this case, you shouldn't set a `cycle_offset` value. However, it's possible that the ATE program could add more cycles to accommodate the patterns to the ATE. This change could also be made by the engineers who adapt the patterns to the ATE. For example, an engineer could add more wait cycles to reset the chip. In this case, you should use a negative integer for the `cycle_offset` value which corresponds to the number of added wait cycles.

-default_offset {pat_name value}

This option adjusts some or all of the default offsets used for split patterns diagnosis if the failures log file format is cycle-based. A default offset is the cycle offset between each split patterns file. Default offsets are computed automatically when the split patterns are read in for diagnosis; one default offset is defined for each split pattern file. Any manual initialization changes directly affect the value of the default offsets.

When *value* is the only argument specified with this option, all default offsets use the specified value. The default offsets are echoed to the log. For example:

```
set_diagnosis -default_offset {14}
```

In this case, the `set_diagnosis` command prints following report:

```
Pattern_file=pat.ser_0.stil, basic_scan=[0(3),9(137)], fast_sequential=[-1,-1], default_offset=0
Pattern_file=pat.ser_1.stil, basic_scan=[10(152),19(287)], fast_sequential=[-1,-1], default_offset=14
Pattern_file=pat.ser_2.stil, basic_scan=[20(302),29(437)], fast_sequential=[-1,-1], default_offset=28
Pattern_file=pat.ser_3.stil, basic_scan=[30(452),32(482)], fast_sequential=[-1,-1], default_offset=42
```

You can also define the default offset for each split pattern. In this case, both the *pattern_name* and *value* arguments must be specified, as shown in the following example:


```

set_diagnosis -default_offset {pat.ser_0.stil 0}
set_diagnosis -default_offset {pat.ser_1.stil 14}
set_diagnosis -default_offset {pat.ser_2.stil 28}
set_diagnosis -default_offset {pat.ser_3.stil 42}

```

For more information, see "[Split Patterns Diagnosis for Cycle-Based Failures.](#)"

-dftmax_chain_format | -nodftmax_chain_format

This option enables the usage of the pattern-based failure format, which uses the failing scan chain name instead of the failing output pin name. When this option is enabled, the diagnosis does not check the expected data even if it is present in the failures log file.

The pattern-based failure format with scan chain name does not require a specific switch for legacy scan. This switch is only needed for DFTMAX failures that use this format.

After the `set_diagnosis -dftmax_chain_format` command is set, the failures log file reader expects this format. Thus, it is not possible to use another format since DFTMAX failures use the output pin. To unset this option, use `set_diagnosis -nodftmax_chain_format` command.

The pattern-based failure format is obtained from a DFTMAX patterns simulation in parallel mode. When this mode is enabled, the following `set_diagnosis` options are not supported: `-mapping_report` and `-max_report_failures d`. Chain diagnosis is also not supported for this mode. The default is `-nodftmax_chain_format`.

-failing_compressor { number | patterns | failures }

Targets a failing compressor during diagnostics based on the specified order associated with the number of failing patterns or failures. The **patterns** parameter specifies that failing compressors are ordered based on the number of failing patterns. The **failures** parameter specifies that failing compressors are ordered based on the number of failures.

For example, the following command specifies diagnostics to be run on the second compressor associated with the second number of failing patterns:

```
set_diagnosis -failing_compressor { 2 patterns }
```

In the example, compressors are ranked based on the number of failing patterns. Diagnosis is run on the second compressor because it is associated with the second highest number of failing patterns.

To target either the first or third compressor, specify 1 or 3 with the `-failing_compressor` option.

By default, failing compressors are ordered based on the compressor ID.

-failure_memory_limit d | -nofailure_memory_limit

This option specifies the maximum number of failures that can be captured by the tester. It ensures accurate diagnosis results when the tester truncates failure data before reaching the end of the pattern set. The *d* argument is the maximum number of failures that can be captured by the tester. The value 0 disables this feature (this is the default, which is the same as the `-nofailure_memory_limit` option).

If the number of failures in the failure log file is less than *d*, all failing and passing patterns are considered for diagnosis. If the number of failures in the failure log file is greater than or equal to *d*, the last failing pattern and all subsequent patterns are ignored during diagnosis (the same as specifying the `-incomplete_failures` option). If there are more

failures than the value specified by *d*, only the failures in the last failing pattern are ignored.

If this option is used in the context of the split pattern feature, it ignores patterns in the range beginning with the last failing pattern recorded in each failure log file to the last pattern in the corresponding pattern file – if the number of failures in that file is at least *d*.

-fault_type [all | shared | transition | stuck]

This option enables diagnosis to simulate both stuck fault sites and transition fault sites in a single run.

When the *all* argument is specified, all fault sites are simulated for both transition and stuck types.

The *shared* argument is the default and enables simulation of both stuck and transition types, but only at stuck fault sites. This reduces diagnosis runtime when transition defects are less likely.

The *transition* argument simulates only transition types and the *stuck* argument simulates only stuck types.

-filter_bridges { [ambiguous <on | off>] [same_cell <none | feedback | all>] }

Removes all specified bridging candidates when class-based diagnostics reporting is enabled (using the *-organization class* setting of the *set_diagnosis* command). The candidates must be exercised by either of the following excitation conditions: an ambiguous bridge or any bridge candidates between two nets connected to the same cell. An ambiguous bridge is any bridge candidate in which every explained pattern has the same value on the aggressor node. These bridge candidates are not distinguishable from a stuck candidate on the victim node. A same-cell bridge is any bridge candidate between two nodes that are connected to the same library cell. Such bridge candidates are not distinguishable from bridges inside the cell. The *feedback* parameter filters only output/input bridging candidates; the *all* parameter filters both output/input and input/input bridging candidates. These filters must be applied before specifying the *run_diagnosis* command, otherwise they will not affect the candidate list reported by the *report_defects* or *write_ydf* commands. For more information on creating a class-based diagnosis report, see the "[Class-Based Diagnosis Reporting](#)" section in the *TestMAX ATPG User Guide*.

-filter_candidates {score_distance <distance | off> | [min_score <score | off>] max_number <number | off>}] }

Specifies various filters to remove candidates when class-based diagnostics reporting is enabled:

- The *score_distance* parameter removes all candidates with a match score that is further than the specified distance from the maximum defect score reported (*distance*). The default score distance is 20 percent.
- The *min_score* parameter removes all candidates with a match score that is less than the specified score (*score*). At least one candidate with the maximum defect score is reported, even if it is below the specified minimum score. The default minimum score is 50 percent.
- The *max_number* parameter removes all but the top-scoring specified number of

candidates for each defect (*number*). All candidates with the same score are reported, even if the number of candidates exceeds the specified value. The default is 10.

These filters must be specified before the `run_diagnosis` command, otherwise they will not affect the candidate list reported by the `report_defects` or `write_ydf` commands. For more information on creating a class-based diagnosis report, see the "[Class-Based Diagnosis Reporting](#)" section in the *TestMAX ATPG User Guide*.

-hide value

This option suppresses the information from the diagnostics report that was added by the `set_diagnosis -show` command. The *value* argument has four possibilities: `version`, `cycles`, `key_value_pairs`, and `physical_subnet_id`.

To suppress the version name from the report, use the `-hide version` option.

To suppress the cycle conversion use `-hide cycles` option.

To suppress reporting the header values contained in the header of the failures log file, use the `-hide key_value_pairs` option. For more information on using this option, see "[Adding Header Information to a Failure Data File](#)."

To suppress reporting the physical subnet ID in the diagnostics report, use the `-hide physical_subnet_id` option. For more information on using this option, see "Reporting Physical Subnet ID Data."

Only one value can be specified at a time. By default, the version, cycles, header values, and physical subnet ID information are hidden.

-incomplete_failures | -noincomplete_failures

This option specifies that the tester failure memory limitations might have caused the failure log to have only partial failure data for the last failing pattern contained there. Therefore, the last failing pattern and all subsequent patterns are ignored during diagnosis. Do not use `-incomplete_failures` together with the `-truncate` option of `run_diagnosis`. The default is `-noincomplete_failures`.

In the context of the split pattern feature, this option ignores patterns in the range beginning with the last failing pattern recorded in each failure log file to the last pattern in the corresponding pattern file.

-intermediate_chain_faults

Enables the scoring and identification of candidates on pins along the scan chain when diagnosing chain failures. This includes the scan in and scan out pins of each scan cell and the pins of any combinational cells along the scan chain. Without this option, only the scan cell state elements are scored and identified. The default is not to enable scoring of intermediate scan chain pins. When this option is enabled, each intermediate candidate is reported with the associated chain name and position indices of the subsequent and previous scan cells.

-mapping_report | -nomapping_report

This option enables the detailed report of failure mapping results for Adaptive Scan technology.

If `-nomapping_report` (default) is used, only a one line summary of the failure mapping result is printed:

```
Failure mapping completed: #failing_pats=815, #skipped_pats=0,
```

```
#masked_cycles=35, CPU_time=15.90
```

Note:

#failing_pats indicates the number of failing patterns for which a potential failure mapping is found.

#skipped_pats indicates the number of failing patterns for which a potential failure mapping has not been found and thus is skipped.

#masked_cycles indicates the number of cycles for which any failure mapping has not been found and thus is ignored.

If the `-mapping_report` option is used, additional lines are printed at the head of the one-line summary. For example, successful mapping is reported per compressor as follows:

```
-----
Failure mapping report:
-----

Successful mapping for des_chip2_U_compressor:
Pattern 1, position 54, chains: 13
Pattern 1, position 55, chains: 13
Pattern 1, position 57, chains: 13
```

If some failures are not successfully mapped, potential observe point and their score are added to the report. For example:

```
Failed mapping for des_chip2_U_compressor:
Pattern 36, position 9, potential chains: 31
Pattern 105, position 9, potential chains: 31
-----
Compressor des_chip2_U_compressor, shift_position=3
score=100.00%, max_score=100.00%, chain=31, cell=u_des_hard_macro_
1/u_des_hard_macro_core_u_03/kdin_reg_91_
-----
Compressor des_chip2_U_compressor, shift_position=9
score=75.00%, max_score=100.00%, chain=31, cell=u_des_hard_macro_
1/u_des_hard_macro_core_u_03/kdin_reg_85_
score=25.00%, max_score=100.00%, chain=32, cell=u_des_hard_macro_
1/u_des_hard_macro_core_u_03/kdin_reg_124_
```

Refer to the topic "[Understanding the Failure Mapping Report](#)" for a complete description of this report.

Note: The mapping report printed by this option and the report printed by the [run diagnosis](#) `-only_report_failures` command are the only complete reports. The first mapping report printed at the beginning of the diagnosis report is not intended to be complete. See "[Understanding the Diagnosis Report](#)" for more details.

```
-----
-----
pattern chain pos# output pin_names
-----
-----
9 1 4 test_so1 test_so2 test_so3
```

```
14 1 3    test_so1 test_so2 test_so3
```

-max_defective_chains *n*

Aborts running the chain diagnosis engine if the number of defective chains is larger than the specified number *n*. By default, *n* is 3. This default is a trade-off between simulation time and quality of the results. The value 0 is used to disable this feature.

If the scan chain engine is halted because of meeting the maximum defective scan chain specification, then the following message is printed:

```
Scan chain diagnosis aborted due to exceeding the maximum number of
defective chains.
```

-max_report_failures *d*

For adaptive scan designs, the output includes the failing measures. The argument for this option specifies the number of failures to be displayed. The default is 10.

-num_processes *number*

Specifies the number of cores to use during a parallel diagnostics run. Using this option conserves memory usage when processing a large number of failure files. For more information on using this option, see "[Parallel Diagnosis](#)" in the *TestMAX ATPG User Guide*.

-organization class | fault

Enables the class-based or fault-based diagnostics report. The default is the fault-based report. The class-based report identifies each physically distinguishable location as a separate candidate, and each candidate may contain different fault types. In the fault-based report, a fault and all of its equivalent faults are considered a single candidate, and each fault type is considered a separate defect. For a complete description of both reports, see the "[Diagnosing Manufacturing Test Failures](#)" chapter in the *TestMAX ATPG User Guide*.

-original_pattern_id

When split patterns are used, additional patterns may need to be added to each split such that there are more total patterns than in the original pattern set. In this case, patterns after the first split will have different pattern numbers than in the original pattern set. This option specifies that diagnosis should use the original (pre-split) pattern IDs in the split pattern files as the pattern numbers instead of counting patterns in the external pattern set. This option should be used if failures from the tester are pattern-based, and if the tester uses the pattern IDs in the pattern files rather than counting the number of applied patterns.

-pattern {*p1 p2 on|off*}

This option specifies a range of patterns to include or ignore during a diagnosis run.

The range *p1 p2* can be specified as a single pattern. To include a pattern, use the parameter *on*. To exclude a pattern, use the parameter *off*. If the neither *on* nor *off* is specified, the default is *off*.

The parameters of this option should be specified as a Tcl list when TestMAX ATPG is run in Tcl mode. The following example shows how to specify this option as a list in Tcl mode:

```
set_diagnosis -pattern {1 5 off}
```

The native mode equivalent command is as follows:

```
set diagnosis -pattern 1 5 off
```

The `-pattern` option can be used to optimize the match score if a specific pattern from the external pattern buffer has not been run on the tester but produces a TPSF. For example:

```
# read all patterns
set_patterns -external pat.bin
# exclude pattern 3
set_diagnosis -pattern {3 off}
run_diagnosis fail.log
```

The `-pattern` option can also be used when the TPSF needs to be computed for only passing patterns. In this case, the flow is to read all patterns, exclude the failing patterns (but include the passing patterns), and then run the diagnosis with `-rank` option:

```
# read all patterns [0 to 9]
set_patterns -external pat.bin
# exclude all patterns
set_diagnosis -pattern {0 9 off}
# include only a single passing pattern
set_diagnosis -pattern {3 on}
# read fault list
read_fault fault.txt
run_diagnosis fail_2.log -rank
```

`-per_pin_limit` | `-no_per_pin_limit`

The `-per_pin_limit` option causes TestMAX ATPG to set a per-pin limit truncation mechanism commonly used by testers. Because of tester memory limits, the total amount of failures per pin for the device under test is usually limited. If TestMAX ATPG does not recognize this situation when it occurs, the failures that are not captured are considered as passing by the diagnostics engine. This can degrade the match score of the fault candidates.

When the `-per_pin_limit` option is set, TestMAX ATPG recognizes a pin as truncated if it has at least one failure.

TestMAX ATPG will also recognize only certain pins as truncated if you set the `-per_pin_limit` option, and use the following directive to define the truncated pins in the failures log file:

```
.pin_truncate pin_name
```

If this directive is specified, only the identified pins are considered truncated.

The default is `-no_per_pin_limit`.

This option is not supported for DFTMAX Ultra.

`-post_procedure procedure_name` | `none`

Specifies the name of a post processing procedure to run during parallel diagnostics. You can specify diagnostics commands, such as `write_ydf`, or Tcl API commands, such as `get_diag_files`, within a procedure. To disable a post processing procedure, specify the `none` argument (`none` is the default).

The following example shows how to define a post processing procedure and specify it using the `-post_procedure` option:

```
proc pp {} {
  write_ydf -append my_ydf.ydf
```

```

    set datalog [get_attribute [index_collection [get_diag_files -all] 0]
name]
    foreach_in_collection cand [get_candidates -all] {
        echo [get_attribute $cand pinpath] $datalog >> candidates.list
    }
}
set_diagnosis -post_procedure pp

```

For more information on using this option, see "[Parallel Diagnosis](#)" in the *TestMAX ATPG User Guide*.

-process_fail stop | continue | rerun

Sets the diagnosis behavior when a slave process crashes during the parallel diagnosis flow. The `stop` argument is the default and causes all processes to stop. The `continue` argument allows the remaining processes to continue without interruption. The `rerun` argument schedules the crashed job to rerun at the end of the diagnosis process.

-reset_filters

Resets all user-specified candidate filters back to their defaults for class-based diagnostics reporting (see the descriptions of the `-filter_candidates` and `-filter_bridges` options). For more information creating a class-based diagnosis report, see the "[Class-Based Diagnosis Reporting](#)" section in the *TestMAX ATPG User Guide*.

-sample d1 d2

Controls the number of patterns simulated during logic diagnosis. Simulates a loose upper limit of `d1` failing patterns and `d2` passing patterns. The actual number of patterns simulated might be higher due to the word-parallel simulation algorithm. This option can be used to get results faster, especially in the case of large designs, at the expense of lower precision. Minimum value for `d1` is 1 and for `d2` is 0. A large value can be specified for either argument to simulate all patterns. The default is to simulate all failing patterns and 96 passing patterns.

The usage of this option has a direct impact on the match score because the match score computation is not weighted with the number of patterns simulated. Note that using too many passing patterns could decrease your ranking score. It is recommended that you leave the default as it is.

-show parameter

This option adds information to the diagnostics report related to the following parameters: `version`, `cycles`, and `key_value_pairs`.

```

run_diagnosis ./fail1.log
Version: K-2015.06-SP4-CS1-i131004_180753-tcl

```

The `cycles` parameter prints data from the conversion of cycle-based failures into pattern-based failures by TestMAX ATPG diagnostics as shown in the following example:

```
16 test_so5 11 # Cycle conversion from cycle 270; fail in shift
```

The example shows that a failure at tester, cycle 270, corresponds to a failure measured on the test_so5 pin at pattern 16. The scan cell which captures the failure is at position 11.

The `key_value_pairs` parameter prints the values from the header of the failure log file. For more information on using this option, see "[Adding Header Information to a Failure Log File](#)."

Note the following:

- You can specify only one `-show` parameter at a time. To display the version name, cycles, and header, specify three separate `set_diagnosis -show` commands.
- By default, the version, cycles, and header values are hidden (see the `set_diagnosis -hide value` command).
- When both the `-show` and `-hide` options are specified, only the last option in the command line is recognized.

`-time_limit d | -notime_limit`

Causes diagnosis to abort if the specified integer number of seconds have passed since diagnosis started running. The time limit is a loose upper limit as the time limit is only checked after each set of 32 patterns have been simulated. Default is `-notime_limit`.

`-use_default_offset | -nouse_default_offset`

This option enables or disables the default offset definitions for split patterns diagnosis if the failures log file format is cycle-based. A default offset is the cycle offset between each split patterns file. When split patterns are read in for diagnosis, these offset values are computed but are not used by default. The default is the `-nouse_default_offset` option. You should use the `-use_default_offset` option if there is a continuous cycle count between each split pattern. For more information, see "[Split Patterns Diagnosis for Cycle-Based Failures](#)."

`-use_dictionary | -nouse_dictionary`

Enables or disables the use of a dictionary during diagnostics (the default is the `-use_dictionary` option).

`-use_phds {values}`

Specifies the type of data to query from the PHDS database. You can specify one or more values as a Tcl list. The possible values are: `all` (the default), `candidates`, `subnets`, `bridges` and `none`.

The `all` keyword extracts all data types.

The `candidates` keyword extracts the physical information related to a diagnostics candidate (for example, the cell bounding box).

The `subnets` keyword extracts the subnets topology for the diagnostics candidates.

The `bridge` keyword extracts the bridging pairs for the diagnostics candidates.

The `none` keyword disables the use of the PHDS database during diagnosis.

For more information on using this option, see "[Static Subnet Extraction Using a PHDS Database](#)."

`-verbose | -noverbose`

Reports detailed information such as net connectivity and physical locations for class-based reporting, and data on observable points, the explained pattern list, and match score components. The default is `-noverbose`. For more information, see the "Verbose Format" section of the "[Understanding the Diagnosis Report](#)" topic.

Description

This command specifies the parameters associated with performing a diagnosis of a failing device (specified by the `run_diagnosis` command).

See Also

[run_diagnosis](#)

[set_physical_db](#)

[Understanding the Diagnosis Report](#)

[Subnets Format for Diagnosis](#)

set_distributed

Overview

Use this to control the amount of time the system is given to perform various distributed processor-related communications.

Syntax

```
set_distributed
[-print_stats_timeout d]
[-script file_name]
[-shell remsh | rsh | ssh]
[-shell_timeout d]
[-slave_setup_timeout d]
[-work_dir dir_name]
[-verbose | -noverbose]
```

Arguments

-print_stats_timeout d

Controls the amount of time allowed for displaying distributed processor statistics.

-script file_name

Specifies the pathname of a user-defined script to be invoked when launching distributed processors. This file should be executable and must contain all settings required for launching TestMAX ATPG on the slave machines. Here's an example script:

```
#!/usr/bin/csh -f
setenv SYNOPSISYS /tools/synopsys/W-2004.12
set path = ($SYNOPSISYS/bin $path)
tmax $*
```

When authoring user-defined scripts, you should:

- Make sure your script does not include echo or exit statements

- Verify the syntax and semantics before running your script.

-shell remsh | rsh | ssh

Controls the shell used for launching slaves on remote hosts specified by the add distributed processors command. Default is `rsh`.

-shell_timeout d

Controls the amount of time in seconds allowed to start the UNIX shell specified by the `-shell` option.

-slave_setup_timeout d

Controls the amount of time in seconds allocated for the distributed processors to be up and running. The argument *d* is given in seconds. This value must be inclusive and between 30 and 4,000,000. The default is 30 seconds. You should be cautious when you specify a

high timeout value, because it might appear that the tool is hung while it's waiting until the timeout limit that you specified for slaves to be ready expires. TestMAX ATPG prints out the wait messaging, indicating how many slaves are up and running and how many other slaves are still not ready; for example:

```
Master: Spawning the slaves ...
        Waiting for 3 (of 3) slaves to respond ...
        Waiting for 3 (of 3) slaves to respond ...
        Waiting for 2 (of 3) slaves to respond ...
Slaves: About to get licenses ...
```

-work_dir *dir_name*

Specifies the pathname of the shared working directory. Every machine performing the distributed fault simulation tasks must have read/write/execute permissions in this directory. This must be an absolute path starting from the system root; relative paths are not supported. Default is the current working directory.

-verbose | -noverbose

When `-verbose` is specified, additional messages from the shell command prints to the log file. This information is useful for debug purposes. Default is `-noverbose`.

Allowed Command Modes

Test

Description

Use this to control the amount of time the system is given to perform various distributed processor-related communications. This includes checking out the status of the distributed processors (using an UNIX `rsh` command), printing statistics regarding the current jobs, and so on. This command also identifies the location of the shared working directory that the machines performing a distributed fault simulation task use to exchange information.

See Also

[Running Distributed ATPG](#)
[add_distributed_processors](#)
[remove_distributed_processors](#)
[report_distributed_processors](#)

set_drc

Use this command to set the parameters that control the DRC process.

Syntax

```
set_drc
[-allow_unstable_set_resets | -noallow_unstable_set_resets]
[-bidi_control_pin <0|1> name | none]
[-blockage_aware_clock_grouping | -noblockage_aware_clock_grouping]
[-check_multiple_shift_clocks | -nocheck_multiple_shift_clocks]
[-check_user_serializer_bits | -nocheck_user_serializer_bits]
[-clock <pin_name | -any | -chain_capture | -dynamic | -one_hot | -seq_
capture>]
[-clockdrc_use_cell_constraints | -noclockdrc_use_cell_constraints]
[-clock_constraints block_name]
[-compressor_debug_data | -nocompressor_debug_data]
[-constraints | -noconstraints]
[-controller_clock pin_name | -nocontroller_clock]
[-disturb_clock_grouping | -nodisturb_clock_grouping]
[-deterministic_zlcheck | -nodeterministic_zlcheck]
[-dftmax_shift_cycles value]
[-dlat_checking_type <stable | hot_clock | transparent >]
[-dslave_remodel | -nodslave_remodel]
[-dynamic_clock_equivalencing <off | disturb | nodisturb>]
[-extract_cascaded_clock_gating | -noextract_cascaded_clock_gating]
[-fast_multifrequency_capture < on | off | spf >]
[ file_name | -nofile]
[-forked_pipes | -noforked_pipes]
[-freerunning_expansion | -nofreerunning_expansion]
[-independent_sync_clocks | -noindependent_sync_clocks]
[-initialize_dff_dlat <0|1|x|random> instance_name]
[-internal_clock_timing name | -nointernal_clock_timing]
[-limit_shadow d]
[-load_nonscan_cells | -noload_nonscan_cells]
[-lockup_after_compressor | -nolockup_after_compressor]
[-master_slave_pipes]
[-max_pll_simulation_passes d]
[-max_unidentifiable_scan_chains number]
[-multi_captures_per_load | -nomulti_captures_per_load]
[-multiframe_paths | -nomultiframe_paths]
[-multiple_pipes_per_port | -nomultiple_pipes_per_port]
[-num_pll_cycles d ]
[-observe_procedure <master | any>]
[-oscillation d]
[-pipeline | -nopipeline]
[-pipeline_in_compressor | -nopipeline_in_compressor]
[-pll_launch_on_shift | -nopll_launch_on_shift]
[-pll_simulate_test_setup | -nopll_simulate_test_setup]
[-random_zlcheck | -norandom_zlcheck]
[-reclassify_invalid_dslaves | -noreclassify_invalid_dslaves]
```

```

[-remove_false_clocks | -noremove_false_clocks]
[-regular_scan_shift_cycles value]
[-set_unload_mode_ports_to_x | -noset_unload_mode_ports_to_x]
[-shadows | -noshadows | -serial_shadows_only]
[-skew d] [-store_initial_shifts | -nostore_initial_shifts]
[-spc_chain spc_chain_name]
[-store_setup | -nostore_setup]
[-store_stability_patterns | -nostore_stability_patterns]
[-store_unload_mode_data]
[-tlas | -notlas]
[-trace | -chain_trace chain_name | -notrace]
[-use_cell_constraints | -nouse_cell_constraints]
[-unstable_lsrams | -nounstable_lsrams]
[-xchain_threshold_probability n]
[-z_check_with_all_constraints | -noz_check_with_all_constraints]

```

Arguments

-allow_unstable_set_resets | -noallow_unstable_set_resets

Specifies whether DFF/DLAT devices with unstable sets or resets are considered DRC violations. An [unstable set/reset](#) condition exists when the control net is internally generated. The default is to consider an unstable set/reset a DRC violation. Using the `-allow_unstable_set_resets` option allows the presence of internally generated set/resets, but might result in decreased test coverage and increased pattern count, especially when using scan compression. However, the `-allow_unstable_set_resets` option might be required in some situations, such as when a primary input is used as both a synchronous and asynchronous set or reset.

When `-allow_unstable_set_resets` is specified, the clock grouping analysis is extended to include flip-flops with set or reset lines controlled by other flip-flops clocked from different clocks. This will increase the disturb flop list and possibly change the clock matrix (specified by [report_clocks](#) `-matrix`) and clock grouping definitions. This behavior defines more appropriate clock relationships in the clock matrix for these conditions and prevents generating patterns that might violate these additional dependencies. The use of `-allow_unstable_set_resets` can result in different pattern generation and coverage opportunities.

-bidi_control_pin <0|1> *name* | none

Specifies the top-level port that operates as a global enable/disable for bidirectional pins when used in conjunction with a ReflectIO capture procedure; or it can be used before any [run_drc](#) command has been performed to cause the `write_drc_file` command to create a template DRC file containing an example ReflectIO procedure. The value 0 or 1 specifies the state to apply to this port during the ReflectIO procedure so that bidirectional ports are inputs. Specifying `none` turns off this control.

-blockage_aware_clock_grouping | -noblockage_aware_clock_grouping

The `-blockage_aware_clock_grouping` option performs analysis on all previously identified disturb clock groups if any of the clocks in the group connects to a candidate "control cell," that is, a scan cell that holds value during capture. During this analysis, the effect of setting the control cells first to 0 and then to 1 is simulated, and the scan cells that are undisturbed in both simulations are removed from the disturb list. The `-blockage_aware`

`aware_clock_grouping` option is intended to support DFT techniques in which the path between clock domains, or one of the clocks, is blocked. This extra analysis increases DRC run time. If DFT logic is not available to block the disturbed paths in both states of the control cells, then it is unlikely that the number of disturbed cells is reduced. The default is `-noblockage_aware_clock_grouping`.

`-check_multiple_shift_clocks | -nocheck_multiple_shift_clocks`

Enables checking during the scan shift operation. TestMAX ATPG reports a [C4](#) violation for each scan cell with a shift clock that is derived from multiple clocks that are active during shift, or from a single clock which fans out through multiple paths that reconverge. The default is `-nocheck_multiple_shift_clocks`.

`-check_user_serializer_bits | -nocheck_user_serializer_bits`

The `-check_user_serializer_bits` is the default. It checks the validity of the user-specified load and unload serializer registers in the STL procedure file when the serializers are used during DRC. This check determines if the specified registers have the expected simulation values as the serializer registers. If an invalid bit is specified in the STL procedure file, an [M873](#) message is issued and the specification for that bit is ignored. If a valid bit is specified with incorrect inversion, a [M874](#) message is issued and the user specification is accepted after the inversion is corrected.

Validity checking is useful when incorrect registers are unintentionally specified. The `-nocheck_user_serializer_bits` option overrides the validity check of the load and unload serializer registers. This is useful for debugging [R33](#) errors, which are generated when TestMAX ATPG is unable to find a register that behaves as an unload serializer register. This causes the validity check to fail. If you specify unload serializer registers in the STL procedure file, and specify the `-nocheck_user_serializer_bits` option, TestMAX ATPG recognizes the specified unload serializer registers without performing the identification process. This enables you to continue the DRC process, which might show additional S or R violations (other than [R33](#) to [R38](#) errors).

For more information on specifying serializer registers in the STL procedure file, refer to the "Providing Guidance for R34 and R36 DRC Errors" section in the *Adaptive Scan With Serializer User Guide*.

`-clock <pin_name | -any | -chain_capture | -dynamic | -one_hot | -seq_capture>`

The `-clock` option specifies restrictions on clock usage for pattern generation. Multiple usage on the same line is allowed, but only the last one encountered will remain in effect. The default is `-dynamic`, which activates the multiple clocking feature. All PI equivalences are removed (except differential). Dynamic clock grouping will not take effect if the `-nomulti_captures_per_load` option is used with this option or if clocking constraints are enabled.

The `-any` argument allows pattern generation using any single clock, including patterns where no clock is used.

The `pin_name` argument restricts patterns to using only the specified clock, but this restriction applies only to Basic Scan test generation. However, if the fault model is IDDQ (established when a [set_faults](#) `-model iddq` command has been issued), patterns are generated with the IDDQ capture procedure having `pin_name` clock pulse. By default, there is no clock pulse in an IDDQ capture procedure.

The `-chain_capture` argument specifies the use of the special retention cell chain test, rather than the usual chain test. This test is applied by the full-sequential test generator. For more information, see [Retention Cell Testing](#).

The `-one_hot` argument restricts patterns to always use one of the defined clocks (while avoiding the non-clocking pattern). Chain test patterns are not generated. Threaded ATPG may generate non-clocking patterns, but this can be prevented by setting `add_po_masks -all`.

The `-seq_capture` argument selects the special Full-Sequential capture procedure defined in the DRC file for Full-Sequential ATPG. Otherwise, by default, the Full-Sequential algorithm uses the same capture procedures as the Basic-Scan and Fast-Sequential ATPG algorithms.

The clock restrictions specified by the `-clock` option are considered during DRC and can affect the recognition of constant-value nonscan cells. For a clock selection of `port_name` or `-one_hot` it will not be possible to create chain test patterns.

TestMAX ATPG silently ignores this option, except when the `-dynamic` or `-one_hot` options are specified. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

`-clock_constraints block_name`

This option, which is used in the internal clocking procedures feature, selects the `ClockConstraints` block in the STL procedure file that is used for DRC. Note that only one `ClockConstraints` block can be used during a single DRC run. The default is to not use any of the `ClockConstraints` blocks in the STL procedure file. For more information, see "[Using Internal Clocking Procedures](#)."

This option switches to single-threaded processing. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

`-clockdrc_use_cell_constraints | -noclockdrc_use_cell_constraints`

Enhances constant value cell identification so the DRC process considers cell constraints affected by the [clock \(C\) rules](#). Because this analysis is performed before scan cell identification, cell constraints are considered only if they are defined by their instance name and not by a position in a scan chain. The default (the `-noclockdrc_use_cell_constraints` option) is for the DRC process to use constant value cell identification to consider cell constraints without applying the [clock \(C\) rules](#). For related information, see the description of the `-use_cell_constraints` option.

`-controller_clock pin_name | -nocontroller_clock`

Specify `-controller_clock` to generate patterns with capture cycles that always have clock pulses from a controller clock regardless of the existence of any other clock pulses in that particular capture cycle. This option is useful for JTAG designs that guarantee the TCK is pulsing in every capture cycle. The default is `-nocontroller_clock`, which does not guarantee that every capture cycle has at least one clock pulse.

`-constraints | -noconstraints`

DRC simulates the `load_unload` functionality as part of the scan-shifting validation process, and will apply all constrained signals to their constrained states at the start of that validation. These values are set in WGL output patterns, but the presence of constrained state data in STIL output is not guaranteed unless these signals are explicitly specified in the `load_unload` procedure. The `-noconstraints` option identifies if these constrained

signals (not otherwise applied in the load_unload procedure) affect the behavior of the shift operation. This will identify missing signals in the definition of the load_unload procedure that should be present to define a self-contained operation. See also the [DRC rule V33](#). The default is `-constraints`.

`-compressor_debug_data | -nocompressor_debug_data`

Provides debug information when DRC fails to find the connection between core scan chains and the unload compressor. This option has no effect when DRC passes.

`-deterministic_zlcheck | -nodeterministic_zlcheck`

Applies a deterministic approach to find buses that are skipped during the bus contention process. These buses are unlikely to produce bus contention under certain conditions. However, they are checked for contention like all other failing buses. Using the deterministic approach, contention analysis is rerun for [Z1](#) violations using all constraints and other information before analysis is performed for [Z8](#) violations (contention prevention). The default is `-nodeterministic_zlcheck`. The buses that were skipped can be reported using the [report_buses](#) `-contention fail` command, as shown in the following example:

```
report_buses -contention fail
  contention zstate #drivers
gate_id status/capture status strong/weak behavior_data
-----
811 fail stable pass 2 0
3166 fail stable pass 4 1 keep01 clock skip
3171 fail stable pass 4 1 keep01 clock skip
```

`-dftmax_shift_cycles value`

Specifies an overshift scan operation value for DFTMAX compression designs. Overshift provides the ability to write out patterns at a user-specified length greater than the longest chains in the design.

`-disturb_clock_grouping | -nodisturb_clock_grouping`

The default, `-disturb_clock_grouping`, adds parallel clock pairs with a limited number of cells having sequential behavior to a clock group. Together with the `-clock -dynamic` option, this option controls whether or not the analysis should be performed for these clock pairs during DRC by default.

`-dlat_checking_type <stable | hot_clock | transparent >`

By default, a [D4](#) violation check is performed on all clock ports in their off state. When the `-dlat_checking_type` option is set with the `stable` argument, a D4 violation is reported when a clock pin of a DLAT is not at the off state. When the `hot_clock` argument is used, a D4 violation is reported if the DLAT clock pin value is X. The default is the `transparent` argument.

`-dslave_remodel | -nodslave_remodel`

Specifies whether to remodel a dependent slave (dslave) in the presence of [S29](#) messages. The default is to remodel the dslave (the `-dslave_remodel` option). The `-nodslave_remodel` option prevents the switching of devices in a master-slave combination, and forces TestMAX ATPG to use the conventional scan chain ordering that puts a master device before the slave device instead of basing the ordering on their

simulated behavior which may lower the test coverage but prevents parallel simulation mismatches. Alternatively, because remodeled/reclassified latches might fail MAX TestBench parallel simulation, you can use the [write_patterns](#) -cellnames type command to eliminate parallel simulation errors.

-dynamic_clock_equivalencing <off | disturb | nodisturb>

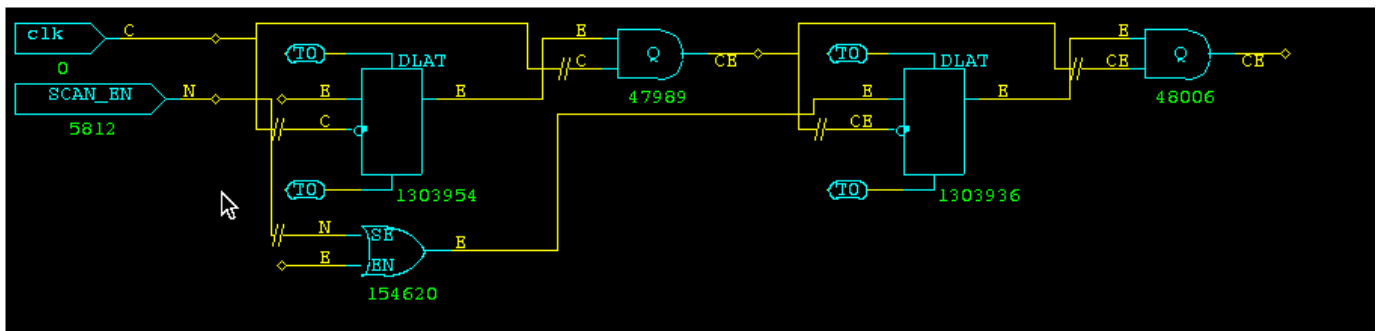
Enables the use of pin equivalences for dynamic clock grouping. The `off` option (the default) automatically deletes pin equivalences for clocks during DRC when dynamic clock grouping is enabled. The `disturb` option allows the use of pin equivalences for clocks when dynamic clock grouping is enabled using normally calculated disturb cells. In this case, all dynamic clock pairings for equivalent clocks are used with no limits on disturb cells. The `nodisturb` option allows the use of pin equivalences for clocks when dynamic clock grouping is enabled with no consideration for disturb cells. In this case, all dynamic clock pairings for equivalent clocks are used with no limits on disturb cells.

-extract_cascaded_clock_gating | **-noextract_cascaded_clock_gating**

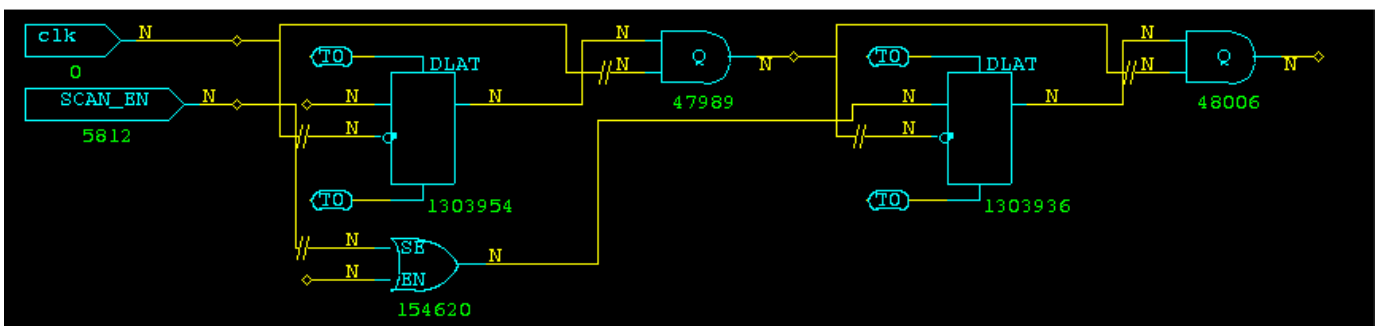
Extracts the entire clock-gating tree structure during DRC and excludes violations on cascaded clock-gating cells. By default, only leaf-level nodes are extracted.

Stuck-inactive faults of the scan enable pins of clock-gating cells receive detected by implication (DI) credit. Use the `-extract_cascaded_clock_gating` option to ensure that DI credit is applied to all cells in the cascade.

In the following figure, only the leaf-level nodes are extracted, which results in a series of [C8](#) violations for cascaded clock-gating cells:



In the next figure, the `-extract_cascaded_clock_gating` option is set and the entire clock-gating tree structure is extracted. As a result, no C8 violations are issued for cascaded clock-gating cells.



This option removes all cascaded clock-gating cells from the list of DRC violations. All other violations, if present, are still included in the DRC summary report.

-fast_multifrequency_capture < on | off | spf >

When a `ClockTiming` block exists in the STL procedure file, this option specifies whether to overlap the clock pulses. The default is to overlap clocks if the `ClockTiming` block contains one or more `MultiCyclePath` statements, but not to overlap the clocks otherwise. When `off` is selected, the clocks are pulsed synchronously, but only at the frequency of the slowest synchronized clock. When `on` is selected, the clocks are overlapped to achieve single-cycle timing even if there is no `MultiCyclePath` statement in the STL procedure file. The default, `spf`, restores the default behavior.

`file_name | -nofile`

Specifies the name of the DRC file (also known as the test protocol file or STIL procedure file) used for design rule checking. This setting remains in effect until you use the `set_drc` command again or you specify a DRC file name in the `run_drc` command. To cancel a previous setting and use no DRC file, use the `-nofile` option.

`-forked_pipes | -noforked_pipes`

The `-forked_pipes` option (the default) causes TestMAX ATPG to recognize the use of pipeline registers before and after a fork, and those used for common scan data bits of a distributed processor. You can view an example of the output produced when this option is set in the description of the [report_compressors](#) command.

`-freerunning_expansion | -nofreerunning_expansion`

Specifies whether or not the clock events defined by a `FreeRunning` block should be included as part of the pattern data. TestMAX ATPG will assume these clocks always pulse as part of the ATPG assumptions for pattern generation. The default, `-freerunning_expansion`, will also fold these clocks into the pattern data. (Some types of patterns do not support some types of `FreeRunning` timing; see the description of the [M669](#) message.) If these clocks are specified from external sources and should not be part of the pattern data, then you can specify `-nofreerunning_expansion`. An [M669](#) message is generated for all `write_patterns` operations for all `FreeRunning` clocks when `-nofreerunning_expansion` is specified. If the design simulation flow requires the clocks to be present and you have specified the `-nofreerunning_expansion` option, then you must manually add the clocks to the simulation. If these clocks are not pulsed, the simulation will fail.

`-independent_sync_clocks | -noindependent_sync_clocks`

Accounts for independent synchronized clocks during DRC when a `ClockTiming` block exists in the STL procedure file. This option modifies the capture matrix for the independent clocks, which allows ATPG to use these clocks to reduce the pattern count. For more information on the capture matrix, see the description of the `-capture_matrix` option of the [report_clocks](#) command. The `-independent_sync_clocks` option requires the `set_drc -fast_multifrequency_capture on` command. The `-noindependent_sync_clocks` option (the default), does not perform this analysis.

`-initialize_dff_dlat <0|1|x|random> instance_name`

Specifies the initial simulation value for all sequential devices or for the named sequential device. The default is `x`. A choice of `random` results in random initialization to either 1 or 0. Note: In order for a non-X default to be kept for a particular sequential device, that device must have clocks/sets/reset, which are off at T=0. Any initialization effects due to the `test_setup` procedure at T=0 will override the initial setting defined by this option.

`-internal_clock_timing name | -nointernal_clock_timing`

This setting is used when multiple `ClockTiming` blocks are defined in an STL procedure file when specifying synchronized multi frequency internal clocks in an OCC Controller. By default, the last `ClockTiming` block defined in an STL procedure file is used. However, you can use the `-internal_clock_timing` switch to specify the use of a specific block. You can specify the `-nointernal_clock_timing` switch to ignore all `ClockTiming` blocks and return to legacy non-synchronized internal clocks behavior.

For more information, see "[Specifying Synchronized Multi Frequency Internal Clocks for an OCC Controller](#)" in the *TestMAX ATPG and TestMAX Diagnosis User Guide*.

This option switches to non-threaded process. For more information, see [TestMAX ATPG Multithreading Command Option Support](#).

`-limit_shadow d`

Causes TestMAX ATPG to not report any shadows in a design that are deeper than the specified `d` value. For example, if your design has a multibit cell length of 500, you can specify DRC to limit the shadow depth to 100.

`-load_nonscan_cells | -noload_nonscan_cells`

Enables test generation support and performs constrain value calculation for loadable nonscan state elements. Note that the command [set_simulation](#) `-shift_cycles` must be set to a value greater than 0. For complete information on using this option and related commands, see the topic "[Using Loadable Nonscan Cells in TestMAX ATPG](#)."

`-lockup_after_compressor | -nolockup_after_compressor`

The `-lockup_after_compressor` option causes the DRC process to report all lock-up latches located between compressors and unload pipelines, if unload pipelines are detected. The default, specified by the `-nolockup_after_compressor` option, is to ignore this check.

`-master_slave_pipes`

This option causes the tool to support pipeline stages with master/slave flip-flops. The tool detects a LE (leading edge) flip-flop followed by a TE (trailing edge) flip-flop and considers this as a single stage of pipeline. You can view an example of the output produced when this option is set in the description of the [report_compressors](#) command.

`-max_pll_simulation_passes d`

Extends simulation of the load procedure to apply a selected number of PLL clock pulses at tester period and procedure boundaries. Simulation continues until all state elements reach a stable value, or until the maximum number of passes specified by `d` is reached. A warning message is given if the maximum number of passes is reached without stabilization. The default maximum number of passes is set to 100.

`-max_unidentifiable_scan_chains number`

Sets a user-specified threshold for unidentifiable scan chain messages ([M464](#)). When the limit is reached, DRC stops without further processing. The default limit is 100 scan chains. If you set a value of 0 for the limit, an M464 message is printed for every unidentifiable scan chain. The scan chains are identified in the same order as they are defined in the ScanStructures block of the STIL protocol file (SPF). This means when the limit is reached, all chains without an error that are defined in the ScanStructures block before the last M464 message are successfully identified. But all chains after the last M464 message are unchecked.

-multi_captures_per_load | -nomulti_captures_per_load

Specifies whether to allow the creation of patterns that have more than one capture clock procedure between scan load and scan unload. The default is to allow this. Use of `-nomulti_captures_per_load` causes there to be no more than one capture clock procedure between each scan load/unload. Disabling multiple capture clocks can increase total tester cycles and reduce test coverage, so you should do it only if required by your ASIC vendor.

-multiframe_paths | -nomultiframe_paths

This switch affects clock timing when synchronized multi frequency clocks is enabled. By default, a `MultiCyclePath` definition inside a `SynchronizedClocks` group in the STIL procedure file is used only to move clock edges closer together in time. When `-multiframe_paths` is specified, a `MultiCyclePaths` definition of greater than 1 can cause clock edges to be moved farther apart. This switch has no effect on the behavior of timing exceptions defined by [read_sdc](#) command.

-multiple_pipes_per_port | -nomultiple_pipes_per_port

When `-multiple_pipes_per_port` is specified, DRC considers multiple DFFs for each bit of each input and output pipeline stage. This is needed to support the following architecture types: scanin → fanout → input pipeline stages → load decompressor → scan chains → unload compressor → output pipeline stages → MUX → scanout. The default is `-nomultiple_pipes_per_port`.

-num_pll_cycles d

Defines the number of PLL clock cycles supported per load. You must define this before defining internal clocks.

-observe_procedure <master | any>

Restricts pattern usage to always apply a master observe procedure.

-oscillation d

Specifies the number of simulation passes allowed during DRC simulation before oscillation is declared. A simulation pass is the effort required to stabilize the new state element values from the previous pass, where the activity is bounded by state elements. The default is 500.

-pipeline | -nopipeline

Performs a special pipeline analysis during DRC to identify states of the pipeline during shifting.

-pipeline_in_compressor | -nopipeline_in_compressor

When enabled, DRC unload pipeline identification allows multi input gates between stages. This is useful if timing optimizations have pushed the unload pipeline DFFs into the XOR network of the unload compressor. Every path must still have the same number of pipeline DFFs.

-pll_launch_on_shift | -nopll_launch_on_shift

The `-pll_launch_on_shift` option must be used for transition-fault testing while `launch_on_shift` uses PLL-controlled internal clocks (the default is `-nopll_launch_on_shift`). The design netlist must include pipelined scan enable, and the STL procedure file must

include a single shift with scan enable changed to functional mode, following the general shift. DRC performs a special simulation of the load_unload procedure but the correctness of the clock controller must be verified in a timing-sensitive simulator, outside TestMAX ATPG DRC.

After `run_drc` is completed, all POs are masked, all nonclock PIs are constrained, and the transition type is set to last-shift-launch. After running DRC in this mode, `run_atpg` will issue an error if ATPG is attempted using any fault model other than Transition.

`-pll_simulate_test_setup | -nopll_simulate_test_setup`

Extends simulation of the `test_setup` procedures with the ability to apply a selected number of PLL clock pulses at tester period boundaries. Simulation of `test_setup` always performs PLL clocking at the beginning and end of the procedure.

`-random_zlcheck | -norandom_zlcheck`

Applies a random approach to find buses that are skipped during the bus contention process. These buses are unlikely to produce bus contention under certain conditions. However, they are checked for contention like all other failing buses. Using the random approach, contention-free buses are determined based on random pattern generation and the calculation of bus contention for all buses in the failing list. The default is `-norandom_zlcheck`. The buses that were skipped can be reported using the [report_buses](#) - `contention fail` command, as shown in the following example:

```
report_buses -contention fail
      contention zstate #drivers
gate_id status/capture status strong/weak behavior_data
-----
811 fail stable pass 2 0
3166 fail stable pass 4 1 keep01 clock skip
3171 fail stable pass 4 1 keep01 clock skip
```

`-reclassify_invalid_dslaves | -noreclassify_invalid_dslaves`

This option controls the reclassification of a dependent slave (dslave) device that violates the [S29](#) rule because it holds a different state than its master device. The default, `-reclassify_invalid_dslaves`, reclassifies the appropriate slave devices as master devices if there is a difference in the device states. This reclassification improves the generation of test patterns that validate the behavior of these devices.

If you set the `-nodslave_remodel` option, the slave reclassification is disabled (the `-noreclassify_invalid_dslaves` option is disabled). Try to avoid simultaneously specifying both the `-reclassify_invalid_dslaves` option and the `-nodslave_remodel` option. This combination of options removes the observe X normally placed on master-slave devices, which is intended to prevent bad pattern generation.

For parallel STIL pattern generation, you should consider using the `-noreclassify_invalid_dslaves` option (for details, see the description of the [M404](#) message). The `-noreclassify_invalid_dslaves` option applies a mask to an invalid master-slave device, which lowers the test coverage but prevents parallel simulation miscompares. This is important since not reclassifying the master-slave operation might violate the sequence of events assumed in basic-scan ATPG. Alternatively, because remodeled/reclassified latches might fail MAX TestBench parallel simulation, you can use the [write_patterns](#) - `cellnames` type command to eliminate parallel simulation errors.

-regular_scan_shift_cycles value

Specifies an overshift scan operation value for regular scan designs. Overshift provides the ability to write out patterns at a user-specified length greater than the longest chains in the design.

-remove_false_clocks | -noremove_false_clocks

When a port has been defined as a clock using an [add_clocks](#) command, and the circuit analysis indicates that the port cannot capture data into a sequential device, then a C4 violation is issued.

Specifying `-remove_false_clocks` causes these false clocks to be removed from the clock list without generating a [C4 violation](#). In addition, the use of the `-remove_false_clocks` option enables the `-allow_unstable_set_resets` option and causes an additional search for any defined clock that goes only to set/reset inputs and at least one DFF/DLAT data input but no DFF/DLAT clock inputs. If this condition exists, then these clock definitions are also removed.

This control only affects clocks defined by command and not those defined by the test protocol file. The default is `-noremove_false_clocks`.

Reference clocks, by definition, are false clocks; they do not connect to any real primitives in the design.

-set_unload_mode_ports_to_x | -noset_unload_mode_ports_to_x

When you specify the `-noset_unload_mode_ports_to_x` option, the unload-mode ports are placed to binary values when checking the [R8](#), [R23](#), and [R31](#) compressor rules.

You should use the default setting (`-set_unload_mode_ports_to_x`) first because it performs a more thorough checking. This option is also faster because it runs fewer simulations by leaving inputs unchanged that are unaffected at X.

If spurious R8, R23, or R31 violations are reported, use the `-noset_unload_mode_ports_to_x` option. This option simulates 1024 random patterns, which is enough to find problems in most cases, but also takes longer to complete.

-shadows | -noshadows | -serial_shadows_only

Specifies the type of shadow identification performed. When `-shadows` is selected (the default), the DRC process attempts to identify shadow registers in both serial and parallel configurations to their master cell. A parallel shadow obtains its value in parallel with the master scan cells during the Shift procedure. The `-serial_shadows_only` option selects the identification of only serially configured shadow registers. A serial shadow register obtains its value during the post-amble sequence after the Shift procedure. Using `-noshadows` disables the identification of shadow cells. For DFMAX Ultra, only the `-shadows` option is supported.

-skew d

When the design has two state elements, A and B, with the output of A connected to the input of B, and each element is clocked by a different clock, and those clocks have nearly identical timing, then the two clocks have a parallel grouping relationship only if the capture edge of clock B occurs at or before the capture edge of clock A minus the skew value. Otherwise, the clocks are ungrouped, or have a disturbed grouping. Note that the specified *d* value uses the same time units used in the protocol file.

-spc_chain spc_chain_name

Specifies the name of the shift power control (SPC) chain used to control the shift power groups hardware added to DFTMAX compression during DFT insertion. Shift power groups reduce power consumption during scan shift. The `-spc_chain` option cannot be used if shift power groups are not specified during DFT insertion using the `-shift_power_groups true` option of the `set_scan_compression_configuration` DFT Compiler command. If shift power groups are specified, the `-spc_chain` option is required unless the shift power disable signal is constrained to disable this feature.

The `spc_chain_name` argument is the chain name specified by the `-class spc` option or the `-class occ` option of the `set_scan_path` TestMAX DFT command during DFT insertion. The chain name is written to the SPF file, but it is not identified as a particular entity in that file.

When the `-spc_chain` option is specified, the scan enable signal must be constrained to its inactive state during capture. The following example adds the `test_se` constraint:

```
add_pi_constraints 0 test_se
```

For more information on configuring shift power groups, see "Reducing Scan Shift Power Using Shift Power Groups" in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

`-store_initial_shifts | -nostore_initial_shifts`

Specifies whether to store the first simulated vectors of the shift procedure until the first internal shift is performed for serializer. When `-store_initial_shifts` is selected, the stored values can be displayed with the `set_pindata -shift` command, followed by `run_drc`. With `-store_initial_shifts` selected, TestMAX ATPG DRC is guaranteed to fail. After debugging and fixing serializer DRC violations, you will need to specify `set_drc -nostore_initial_shifts` before invoking `run_drc`, to pass TestMAX ATPG DRC.

`-store_setup | -nostore_setup`

Specifies whether to store all simulated time periods of the `test_setup` procedure for potential display in TestMAX ATPG. The default is to store only the final value of the `test_setup` procedure. The `-store_setup` option can cause long runtimes.

`-store_stability_patterns | -nostore_stability_patterns`

Specifies whether to store the patterns used to determine the values placed on nonscan state elements at the beginning of a pattern for potential display. The default is not to store these patterns.

`-store_unload_mode_data`

A schematic is annotated with `unload_mode` data, that is, shift data plus unload mode pins for the unload mode given as an argument in the `set_pindata -drcdata unload_mode <mode_name>` command. This data is stored automatically for an R23 violation, or explicit storage can be requested by the `set_drc -store_unload_mode_data` option. For non-shared X-tolerant compressors, this option stores unload mode data for all unload modes plus R23-like data for `unload_enable`. For shared X-tolerant compressors, this option only stores R23-like data for `unload_enable`.

`-tlas | -notlas`

Specifies whether transparent latch identification is performed. The default is to identify all potential transparent latches.

`-trace | -chain_trace <chain_name> | -notrace>`

Specifies whether to display primitives in scan chains that were sensitized during scan chain tracing. The default is `-notrace`. Using `-trace` causes all primitives in all scan chains to be traced. Using `-chain_trace` causes only the specified chain to be traced.

`-use_cell_constraints | -nouse_cell_constraints`

Enhances constant value cell identification so the DRC process considers cell constraints (the default). Because this analysis is performed before scancell identification, cell constraints are considered only if they are defined by their instance name and not by a position in a scan chain. This option does not affect [clock \(C\) rules](#). See the description of the `-clockdrc_use_cell_constraints` option to control DRC for cells constraints for [clock \(C\) rules](#).

`-unstable_lsrams | -nounstable_lsrams`

Lets level-sensitives RAMS, which are not stable when clocks are off, to be used during Fast-Sequential ATPG. This argument must be active before DRC to have an effect. The default is `-nounstable_lsrams`.

`-xchain_threshold_probability n`

Defines the minimum probability of X-capture of a scancell that causes rule [D39](#) to fail. The default is 0, which indicates that checking will not be performed. The value of *n* must be between 0 and 100, inclusive.

If this analysis is to be performed on a netlist before scan insertion, the following commands are also required: [set_scan_ability](#) on `-all` and [run_drc](#) `-prescan file_name`.

`-z_check_with_all_constraints | -noz_check_with_all_constraints`

Specifies whether constrained pins and constant value gates are considered while performing bus checks (specifically Z-rule checking). The default is `-noz_check_with_all_constraints`.

Allowed Command Modes

BUILD, DRC

Description

Use this command to set the parameters that control the DRC process. You can display the current settings with the `report_settings` command.

Examples

```
DRC> set_drc -oscillation 20 -store_setup
```

```
DRC> set_drc -pipeline -clock 12clk
```

```
Begin simulating test protocol procedures...
Begin simulate_pipeline_shifting: group=sg0, pass=1, n=95
End simulate_pipeline_shifting: group=sg0, pass=1, n=0
Begin simulate_pipeline_shifting: group=sg0, pass=1, n=95
End simulate_pipeline_shifting: group=sg0, pass=1, n=0
Nonscan cell constant value results: #constant0 = 29, #constant1 = 45
Nonscan cell load value results : #load0 = 29, #load1 = 65
```



```
Test protocol simulation completed, CPU time=1.00 sec.
```

See Also[add_clocks](#)[Clock Grouping](#)[run_drc](#)[Sequential Capture Procedure](#)[set_rules](#)[Understanding Clock Grouping Analysis](#)[write_drc](#)

set_environment

Overview

This command provides a command line equivalent for many of the changes which can be interactively adjusted in the Environment dialog box.

Syntax

```
set_environment  
[gui | transcript | viewer | reports | info] options...
```

Arguments

gui options

This option selects the GUI subset of the set environment controls. For details, see:
`set_environment_gui`

transcript options

This option selects the Transcript subset of the set environment controls. For details, see:
`set_environment_transcript`

viewer options

This option selects the Viewer subset of the set environment controls. For details, see:
`set_environment_viewer`

Reports options

This option selects the Reports subset of the set environment controls. For details, see:
`set_environment_reports`

info options

This option selects the Info subset of the set environment controls. These are detailed on the page:
`set_environment_info`

Allowed Command Modes

All

Description

This command provides a command line equivalent for many of the changes which can be interactively adjusted in the Environment dialog box. This dialog box is accessed from the Edit->Environment menu.

See Also

[set_environment_gui](#)
[set_environment_info](#)
[set_environment_reports](#)
[set_environment_transcript](#)
[set_environment_viewer](#)

set_environment_gui

Overview

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the Environment dialog box.

Syntax

```
set_environment_gui
  [-cmdbar | -nocmdbar]
  [-error_messages <transcript | message_box>]
  [-history_order <newest | oldest>]
  [-max_history_length d]
  [-title_prefix string]
  [-toolbar <text | bitmap>]
```

Arguments

-cmdbar | -nocmdbar

A selection of `-cmdbar` displays the command bar and while you resize the main window, hides or discloses buttons. Click the ">>" symbol on the far right to view and select a hidden button. A selection of `-nocmdbar` hides the command bar. The default is `-cmdbar`.

-history_order <newest | oldest>

This option specifies the order in which the command history appears in the popup window when the down arrow at the right of the command line input buffer is selected and the command history window opens up. A selection of `newest` causes the most recent command to appear first (at the top) of the window. A selection of `oldest` causes the oldest command to appear first. The default is `Oldest`.

-max_history_length d

This option controls the maximum number of commands stored in the command history buffer. The default is 100 lines.

-title_prefix *string*

This option displays a text string in the GUI title. To make the hostname visible in the GUI title, use the hostname as the *string*.

-toolbar <text | bitmap>

This option determines whether the GUI buttons are displayed with text labels or as graphical icons. The default is `Text`.

Allowed Command Modes

All

Description

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the Environment dialog box. This dialog box is accessed from the Edit->Environment menu.

Examples

```
BUILD> set_environment_gui -toolbar bitmap
```

See Also

[set_environment_info](#)

[set_environment_reports](#)

[set_environment_transcript](#)

[set_environment_viewer](#)

set_environment_info

Overview

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the Environment dialog box.

Syntax

```
set_environment_info  
  [-always_on_top | -noalways_on_top]  
  [-save_dimensions | -nosave_dimensions]
```

Arguments

-always_on_top | -noalways_on_top

Selects whether the gate information window stays on top or not. The default is not to stay on top.

-save_dimensions | -nosave_dimensions

Selects whether last size and position of information window are saved or not. The default is to save and reuse this size and dimensions last used.

Allowed Command Modes

All

Description

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the Environment dialog box. This dialog box is accessed from the Edit->Environment menu.

Examples

```
BUILD> set_environment_info -always_on_top
```

See Also

[set_environment_gui](#)

[set_environment_reports](#)

[set_environment_transcript](#)

[set_environment_viewer](#)

set_environment_reports

Overview

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the Environment dialog box.

Syntax

```
set_environment_reports  
  [-color <text | background> <red> <green> <blue>]  
  [-font_size d]  
  [-height d]  
  [-line_length d]  
  [-max_lines d]  
  [-margin d]  
  [-width d]  
  [-xpos d]  
  [-ypos d]
```

Arguments

-color <text | background> <red> <green> <blue>

Specifies the color of the text or background used in report windows opened by various "report" commands. The color is specified as a triplet of three numbers in the range of 0 to 255, with 0 0 0 for black or 255 255 255 for white. The default for the background is white and for the text is black.

-font_size d

Selects the size of the font in points used in the popup report windows. The default is 16 points.

-height d

Selects the initial height of the window in pixels used by the report window. The default is 400 pixels.

-width d

Selects the initial width of the window in pixels used by the report window. The default is 600 pixels.

-line_length d

Selects the maximum number of characters per line to store in the report window. Lines longer than this are truncated. The default is 200 characters.

-max_lines d

Selects the maximum number of lines held in the report window. When more lines are written to the report window than this limit, the oldest lines are removed. The default is 5000 lines.

-margin d

Selects the size of the left margin of the report window in pixels. The default is 5 pixels.

-xpos *d*

Selects the initial X position of the upper left corner of the report window relative to the upper left corner of the screen. The default is 20 pixels offset to the right.

-ypos *d*

Selects the initial Y position of the upper left corner of the report window relative to the upper left corner of the screen. The default is 20 pixels offset down.

Allowed Command Modes

All

Description

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the Environment dialog box. This dialog box is accessed from the Edit->Environment menu.

Examples

```
BUILD> set_environment_reports -line_length 300
```

See Also

[set_environment_gui](#)

[set_environment_info](#)

[set_environment_transcript](#)

[set_environment_viewer](#)

set_environment_transcript

Overview

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the **Environment** dialog box.

Syntax

```
set_environment_transcript  
  [-font_size d]  
  [-max_lines d]
```

Arguments

-font_size d

Selects the font size in points of the text displayed in the transcript. The default font size is 16 points.

-max_lines d

Selects the maximum number of lines to store in the transcript. When this limit is reached, the oldest lines are removed as new lines are added. The default is 15000 lines.

Allowed Command Modes

All

Description

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the **Environment** dialog box. This dialog box is accessed from the Edit->Environment menu.

Example

```
BUILD> set_environment_transcript -max_lines 10000
```

See Also

[set_environment_gui](#)
[set_environment_info](#)
[set_environment_reports](#)
[set_environment_viewer](#)

set_environment_viewer

Overview

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the Environment dialog box.

Syntax

```
set_environment_viewer
[-buf_inv | -nobuf_inv]
[-color {<background | bidi | block | bufinv_removed
| edge_connector | highlight | id | instance_name | net
| pindata_text | pinname_text | rubberband | tie
| title_background | title_text | type_name_text>
<red_value> <green_value> <blue_value> }]}...
[-design_symbols <block | primitives>]
[-dff_dlat_symbol <model | mode2 | mode3>]
[-font <system | hershey>]
[-instance_names | -noinstance_names]
[-left_justify_pis | -noleft_justify_pis]
[-max_gates d]
[-max_pindata_length d]
[-primitive | -noprimitive]
[-redraw <always | if_necessary>]
[-scheme <default | black_on_white | synopsys>]
[-time_out d]
[-truncate | -notruncate]
```

Arguments

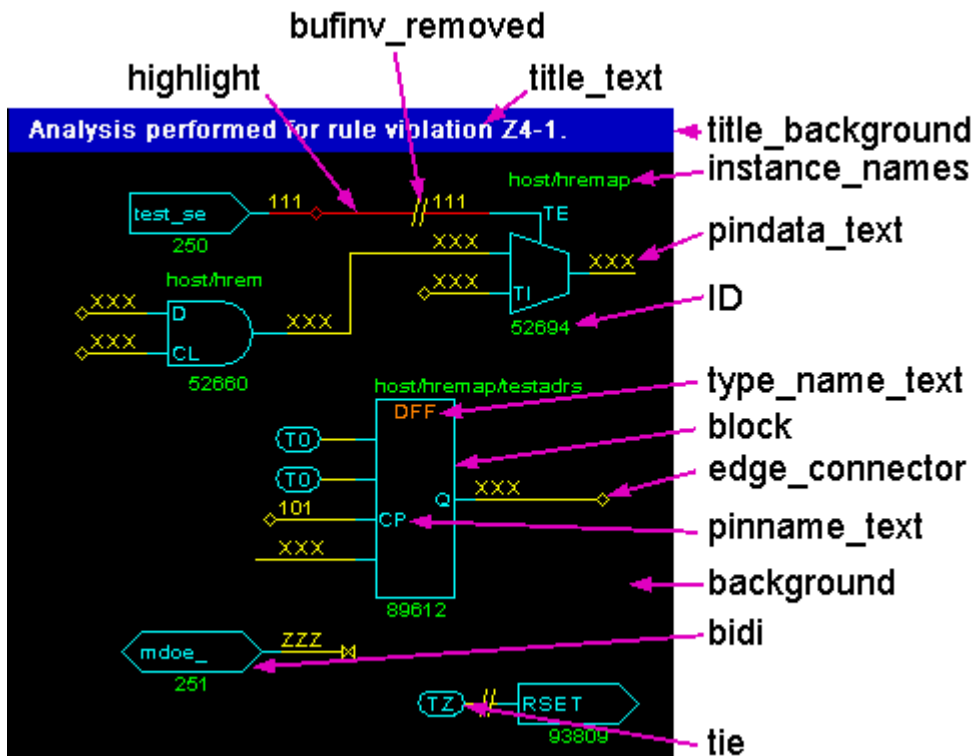
-buf_inv | -nobuf_inv

Selects whether buffers and inverters are removed from the schematic and replaced with collapsed buf/inv symbols. This applies only if the view is "primitive". The default is to show all buffers and inverters.

-color {<object> <red_value> <green_value> <blue_value>}

This option sets the display color for a type of object or text label. The *object* is one of the field names as illustrated in the following diagram. The color values are specified as a color triplet of red, green, and blue values, with a range of 0 to 255 for each value. For example, 0 0 0 = black, 255 255 255 = white, and so forth. Make sure you specify the object and color values enclosed within curly braces, for example:

```
set_environment_viewer -color { block 0 255 0 }
```



-design_symbols <block | primitives>

Specifies whether design view symbols are drawn as rectangles, or as more familiar graphics. With a selection of `primitives`, if the design view has a simple underlying ATPG primitive, then the graphic for the ATPG primitive is drawn instead of a rectangle. The default is `block`.

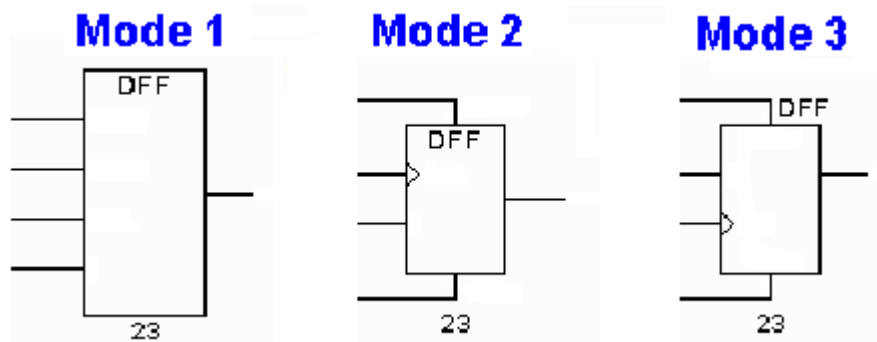
-dff_dlat_symbol <model1 | model2 | model3>

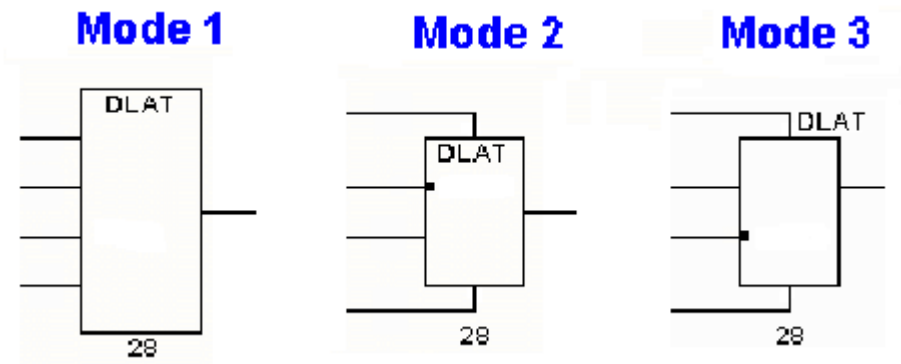
Changes the symbol displayed for DFF and DLAT graphics when the display view of "primitives" has been selected.

Mode 1: Input order from top to bottom is set, reset, clock, data.

Mode 2: Input order from top to bottom is clock, data with set on top and reset on bottom.

Mode 3: Input order from top to bottom is data, clock, with set on top and reset on bottom.





-font <system | hershey>

Selects the type of font used in the dialog boxes and menus. The default is `System`.

-instance_names | -noinstance_names

Specifies whether to display [instance names](#). The default is to not display instance names.

-left_justify_pis | -noleft_justify_pis

Selects whether when primary input symbols (PIs) are placed on the left side of the schematic or are allowed to exist anywhere. The default is to left justify.

-max_gates d

Specifies the maximum number of gates to try to draw before giving up. The default is 200.

-max_pindata_length d

Specifies the length of the simulation string centered around the current wave cursor reference point. The default is 10, and there is no upper limit. When the value is set to 0, all available simulation pindata values are displayed. Note that the pindata length count includes all "information" characters displayed between the waveform characters. An ellipsis (...) is added to the annotation to signify that only a partial value is displayed.

-primitive | -noprimitive

Selects whether the Schematic Viewer displays the design view or the primitive view. The design view shows vendor library cell level. The primitive view shows ATPG primitive level.

-redraw <always | if_necessary>

Specifies whether adding new objects to an existing view causes the view to be redrawn every time or only when necessary (when the new object will not fit unless other objects are moved around). The default is to redraw only as necessary.

-scheme <default | black_on_white | synopsys>

Selects the color scheme. The default setting is color. Using `black_on_white` is useful for screen snapshots used in black-and-white printed documentation.

-time_out d

Specifies the maximum time to work on the drawing before giving up. Generally the placement of instances and the routing of nets can become difficult for a large draw list. This feature ensures that the algorithm does not get stuck trying to solve a drawing problem for which the answer would take too long. The default is 20 seconds.

-truncate | -nottruncate

When the `-instance_names` argument has been selected, specifies to shorten the display of instance names to a fixed width. This might be useful to avoid clutter and one long string overlaying another. Truncation occurs on the leading characters, leaving the rightmost characters visible. To get a complete instance name, you can click a gate in the GSV and the full instance name displays in status window.

Allowed Command Modes

All

Description

This command provides a command line equivalent for many of the changes that can be interactively adjusted in the Environment dialog box. This dialog box is accessed from the Edit->Environment menu.

Examples

```
BUILD> set_environment_viewer -max 50 -time_out 25  
BUILD> set_environment_viewer -dff_dlat_symbol mode3
```

See Also

[set_environment_gui](#)

[set_environment_info](#)

[set_environment_reports](#)

[set_environment_transcript](#)

set_faults

Overview

Use this command to set the parameters that control the fault manager.

Syntax

```
set_faults
[-atpg_effectiveness | -noatpg_effectiveness]
[-au_credit d]
[-ax_credit d]
[-bridge_input | -nobridge_input]
[-delete_dictionary]
[-equiv_code code_name | -noequiv_code]
[-interval_fault_coverage | -nointerval_fault_coverage]
[-fault_coverage | -nofault_coverage]
[-model <stuck | iddq | iddq_bridging | transition | path_delay | hold_
time | bridging | dynamic_bridging>]
[-multi_fanout_di_faults | -nomulti_fanout_di_faults]
[-persistent_fault_models | -nopersistent_fault_models]
[-pt_credit d]
[-report <collapsed | uncollapsed>]
[-store_dictionary max_num_fail_pat]
[-summary <verbose | noverbose>]
[-test_coverage_exclude {ud_class_fault_list}]
[-test_coverage_include {ud_class_fault_list}]
[-use_dictionary | -nouse_dictionary]
```

Arguments

-atpg_effectiveness | -noatpg_effectiveness

Indicates whether the fault summary reports the [ATPG effectiveness](#) coverage. The default is not to report this information.

-au_credit d

Defines the percentage of credit given to AN (ATPG Untestable Not-Detected) faults during the calculation of [test coverage](#). The default is 0, which gives 0% credit. This option accepts integer values from 0 to 100, which are interpreted as a percentage.

-ax_credit d

Defines the percentage of credit given to AX (ATPG Untestable Timing Exceptions) faults. The default is 0. This option accepts integer values from 0 to 100, which are interpreted as a percentage.

-bridge_input | -nobridge_input

Supported for distributed ATPG; not supported for Full-Sequential ATPG and Full-Sequential fault simulation.

Selecting `-bridge_input` accepts input pins for both the victim and the aggressor nodes. For the victim node, the specified pin is used to propagate the fault effect. For the

aggressor node, an attempt is made to map the input pin to the driving output pin. If this mapping fails, a warning is issued. The default is `-nobridge_input`.

-delete_dictionary <internal | external | both>

Deletes all internal or external diagnostics dictionaries or both. For more information, see [Using a Dictionary for Diagnosis](#).

-equiv_code code_name | -noequiv_code

Selects the equivalence code to use when reporting uncollapsed faults. The default is to use a double dash "--". Use of `-noequiv_code` suppresses the use of this character string and uses instead the [fault class](#) of the fault to which this fault is equivalent. When specifying an alternative equivalence code, use a 2-character string. Also note that the current setting of the equivalence code can affect fault lists read using the `read_faults` command.

-fault_coverage | -nofault_coverage

Indicates whether the fault summary reports the [fault coverage](#). The default is not to report this information. The calculation of the fault coverage is affected by the inclusion or exclusion of faults for unused gates. By default, unused gates and their associated faults are removed from the design during flattening to extend gate capacity. If you want an accurate fault coverage report, you can have TestMAX ATPG retain the unused gates during the circuit flattening process. This is controlled with the `-nodelete_unused_gates` option of the `set_build` command.

-interval_fault_coverage | -nointerval_fault_coverage

Prints the fault coverage to standard output and log file for each pattern when the `-interval_fault_coverage` option is set during the processing of the [run_atpg](#) command. In each interval summary line, the test coverage is always printed first for each interval, followed by the fault coverage. The default is the `-nointerval_fault_coverage` option.

-model <stuck | iddq | iddq_bridging | transition | path_delay | hold_time | bridging | dynamic_bridging>

Indicates the type of fault model to be used. The choices are as follows:

`stuck` — Generates standard ATPG patterns (the default).

`iddq` — Uses the IDDQ model to generate patterns to measure IDDQ.

`iddq_bridging` — Uses the IDDQ bridging model to generate patterns to increase IDDQ coverage.

`transition` — Applies the transition fault model for using last shift or system clock to launch a transition.

`path_delay` — Applies the [Path delay fault](#) model, which enables pattern generation that targets a list of path delay faults.

`hold_time` — Applies the hold time fault model, which targets hold time faults during pattern generation.

`bridging` — Applies the bridging fault model for detecting bridge defects. In this case, the faults cannot be collapsed, so the collapsed and uncollapsed fault counts will always be the same. This is supported for distributed ATPG; but not supported for full-sequential ATPG and fault simulation.

`dynamic_bridging` — Applies the dynamic bridging fault model. In this case, there is no fault collapsing, and full-sequential ATPG and fault simulation are not supported.

Note that the IDDQ measurement criteria is adjusted by specifying a separate `set_iddq` command. The criteria selections are valid for both IDDQ and IDDQ_bridging fault models. If you use this option, make sure you either turn off the chain test with the `set_atpg -chain_test` command, or add 1 to your maximum pattern limit using the `set_atpg -patterns` command.

Also note that the full-sequential pattern generation algorithm does not support the IDDQ fault modes and IDDQ bridging fault models. These models are supported by the basic scan and fast-sequential ATPG algorithms.

The `transition`, `path_delay`, `hold_time`, `bridging`, `iddq_bridging`, and `dynamic_bridging` options all require a Test-Fault-Max license.

`-multi_fanout_di_faults | -nomulti_fanout_di_faults`

In default mode (`-multi_fanout_di_faults`), faults in the scan chain path which have multiple fanouts is assigned as detected by implication (DI) faults. Note that this action is valid only if the `set_faults` command is run before the `add_faults` command. If `-nomulti_fanout_di_faults` is specified, then these same faults will not be assigned as DI faults.

`-persistent_fault_models | -nopersistent_fault_models`

Turns on (`-persistent_fault_models`) or off (`-nopersistent_faults_models`) the persistent fault model flow, which automatically maintains fault lists for all supported fault models in both DRC and TEST modes. The default is `-nopersistent_faults_models`. For more information, see "[Persistent Fault Model Support](#)" in the *TestMAX ATPG User Guide*.

`-pt_credit d`

Defines the percentage of credit given to possible detected faults (PT) during calculation of [test coverage](#), fault coverage, or ATPG effectiveness. The default is 50, which results in 50% credit. If this credit is set to zero, no patterns are saved during pattern generation. When this credit is nonzero, one pattern is saved during pattern generation which shows the faulty machine simulates an X where the good machine simulates a 1 or 0. If you set this credit to zero, you should not change this setting after patterns have been created.

`-report <collapsed | uncollapsed>`

Indicates whether fault counts, test coverages, and reports are given in terms of collapsed or uncollapsed faults. The default is uncollapsed.

`-store_dictionary max_num_fail_pat`

Enables the creation of a diagnostics dictionary during fault simulation, The `max_numb_fail_pat` parameter specifies the maximum number of failing patterns per pin (the default is 0).

`-summary <verbose | noverbose>`

Indicates the type of fault summary report to display. The default is `noverbose`.

`-test_coverage_exclude {ud_class_fault_list}`

This option is used to identify and exclude a class or subclass of undetectable faults that you do not want to subtract from the total number of faults. By default, the test coverage calculation subtracts all undetectable faults from the total number of faults, since these

faults cannot be detected and bias this calculation. This will reduce the calculated test coverage based on the number of undetectable faults that remain in the total fault count. The classes you can specify are UU, UO, UR, UB, and UT. For complete descriptions of these classes, see Fault Classes.

-test_coverage_include {ud_class_fault_list}

This option is used to add classes back into the test coverage calculation after the `-test_coverage_exclude` option is used to remove a class or classes. By default, the calculation subtracts all undetectable faults from the total number of faults, since these faults cannot be detected and bias the calculation. The classes you can specify are UU, UO, UR, UB, and UT. For a complete description of these classes, see Fault Classes.

Allowed Command Modes

All

Description

Use this command to set the parameters that control the fault manager. You can display the current settings with the `report_settings` command.

Examples

```
TEST> set_faults -report collapsed
TEST> report_faults -summary
Uncollapsed Fault Summary Report
-----
fault class                code #faults
-----
Detected                   DT    121025
Possibly detected          PT     4003
Undetectable               UD     1443
ATPG untestable            AU     7193
Not detected               ND     5754
-----
total faults                139418
test coverage               89.17%
-----
```

See Also

[Fault Classes](#)
[read_faults](#)
[report_faults](#)
[report_settings](#)
[set_atpg](#)
[set_build](#)
[set_delay](#)
[set_iddq](#)
[update_faults](#)
[write_faults](#)

set_iddq

Overview

Use this command to specify the criteria considered in determining whether a pattern is used for an IDDQ measure.

Syntax

```
set_iddq
  [-atpg | -noatpg]
  [-bridge_equivalence | -nobridge_equivalence]
  [-exclude_ports <none | bidis | all>]
  [-interval_size 1|32]
  [-toggle | -notoggle]
  [float | nofloat] [strong | nostrong]
  [weak | noweak] [write | nowrite]
```

Arguments

-atpg | -noatpg

Specifying **-atpg** causes the test generator to attempt to satisfy all of the IDDQ constraints while generating patterns. Specifying **-noatpg** results in patterns checked for IDDQ constraints after pattern generation; any failing pattern is discarded. The default is **-noatpg**.

-bridge_equivalence | -nobridge_equivalence

This option detects one pair of bridges to imply detection of the other pair. For example, say that "ba0 net1 net2" and "ba1 net2 net1" are the same collapsed fault. When this option is specified, there is an implied detection of "ba1 net1 net2" and "ba0 net2 net1". Note that this option has no effect when the test generator generates a test. If one fault is detected, the fault simulator marks the equivalent faults as detected. The default is **-bridge_equivalence**.

-exclude_ports <none | bidis | all>

Controls contention checking. Specifying **all** ignores IDDQ contention checking on buses connected to the top level ports. Specifying **bidis** ignores IDDQ checking on bidir ports. Specifying **none** checks IDDQ contention on all buses and bidir ports. The default is **all**.

-interval_size 1 | 32

Sets the interval size to either 1 (the default) or 32. Using an interval length of 1 will give better QOR at the expense of execution time. Using an Interval length of 32 gives a better runtime, and is useful for generating a large number of IDDQ patterns.

-toggle | -notoggle

Specifying **-toggle** causes a relaxation of the IDDQ criteria and an input pin fault site does not require sensitization and propagation to an output pin of the same gate. With the **-toggle** option, the input pin fault site needs only to toggle states to be given a IDDQ fault detection credit. Setting the **-toggle** option has no effect if the IDDQ bridging fault model is set (**set_faults -model iddq_bridging**). The default is **-notoggle**.

For input faults (using the IDDQ fault model) on sequential primitives, TestMAX ATPG does not propagate the fault effect to the output of the sequential primitives.

float | nofloat

Selecting float specifies that BUS gates cannot be at a Z state during an IDDQ measure. Any pattern failing this criteria will not be considered for an IDDQ pattern. The default is float.

strong | nostrong

Selecting strong specifies that the IDDQ measure is required to be contention free for strong drivers of BUS gates. This functionality is not yet implemented.

weak | noweak

Selecting weak specifies that BUS gates with weak inputs cannot be at a competing state to other strong or weak BUS inputs during an IDDQ measure. The default is weak.

write | nowrite

Selecting write specifies that RAMs cannot have an active write port during an IDDQ measure. The default is write.

Allowed Command Modes

All

Description

Use this command to specify the criteria considered in determining whether a pattern is used for an IDDQ measure. Enabling the creation of IDDQ patterns also requires adjusting the fault model using the `set_faults` command.

The current settings for IDDQ can be reviewed with the `report_settingsiddq` command.

Additional IDDQ constraints can be constructed using the `-iddq` option of the [add_atpg_constraints](#) command.

Examples

```
BUILD> set_faults -model iddq
BUILD> set_iddq float strong write -atpg
```

See Also

[add_atpg_constraints](#)

[report_settings](#)

[set_faults](#)

Table Column Outside Table:

set_learning

Overview

Use this command to set the parameters that control the learning process.

Syntax

```
set_learning
[-atpg_equivalence | -noatpg_equivalence]
[-common_input | -nocommon_input]
[-disable_time_limit | -nodisable_time_limit]
[-equivalent_latches | -noequivalent_latches]
[-implication [low | medium | high] | -noimplication]
[-max_feedback_sources d]
[-sim_passes d]
[-test_passes d]
[-verbose | -noverbose]
```

Arguments

-atpg_equivalence | -noatpg_equivalence

Specifies whether the ATPG equivalence analysis is to be performed. The default is to perform this learning analysis.

-common_input | -nocommon_input

Specifies whether the common input analysis is to be performed. The default is to perform this learning analysis.

-disable_time_limit | -nodisable_time_limit

Enables these two time limiting learning options (removes the time limits and finishes the learning to completion): `set_learning -max_implication 0` and `set_learning -max_atpg_learning 0`.

The default option is `-nodisable_time_limit`.

-equivalent_latches | -noequivalent_latches

Specifies whether the equivalent latch analysis is to be performed. When this analysis is enabled, TestMAX ATPG performs a search for latches with common clock and data relationships (with potential inversions). The default is to perform this learning analysis.

-implication [low | medium | high] | -noimplication

Specifies whether the implication learning analysis is to be performed. The default is to perform this learning analysis with an effort of medium.

-max_feedback_sources d

Specifies the limit on the number of sources in a combinational feedback path. The default is 100.

-sim_passes d

Specifies the number of simulation passes to be performed during the ATPG equivalence analysis. The default is to perform 32 passes.

-test_passes *d*

Specifies the number of test generation passes to be performed during the ATPG equivalence analysis. The default is to perform 5000 passes.

-verbose | -noverbose

Specifies the amount of detail in the ATPG equivalence analysis report. The default is -noverbose.

Allowed Command Modes

All

Description

Use this command to set the parameters that control the learning process. You can display the current settings with the [report_settings](#) command. To view the learned data for a given gate, set the verbose option for the [set_primitive_report](#) command and run the [report_primitive](#) command for the selected gate.

Note: The learning process occurs immediately after the build process, so any setting changes for learning must occur before that point.

Examples

```

BUILD> set_messages -level expert
BUILD> set_learning -atpg_equivalence
BUILD> run_build_model flight_sim
-----
Begin build model for topcut = flight_sim ...
-----
Primitive elimination results: #tied=239, #unused=6079, #BUF=18456 (3
added), #inv=5202
Number common input primitives=11533
Ranking completed: #ranked_gates=77591, #feedback_paths=0, CPU time=0.34
End build model: #primitives=101071, CPU_time=3.00 sec, Memory=34529279
-----
Begin learning analyses...
Fanout-free region identification completed: #regions=37758, CPU time=2.44
Begin ATPG equivalence analysis: #sim_passes=16, #max_test_passes=1000 ...
Random simulation results: #pos_ties=496/1882, #pos_equiv=41051
(#groups=15601), CPU time=4.33
ATPG learned equiv relationship: 11802 (MUX) == 11804 (WIRE).
ATPG learned equiv relationship: 11816 (WIRE) == 11817 (WIRE).
ATPG learned equiv relationship: 11819 (WIRE) == 11820 (WIRE).
ATPG learned equiv relationship: 40187 (TSD) == 40402 (TSD).
ATPG learned equiv relationship: 12975 (NAND) == 12977 (NAND).
ATPG learned equiv relationship: 12978 (NAND) != 12979 (AND).
ATPG learned equiv relationship: 13048 (OR) == 13051 (OR).
ATPG learned equiv relationship: 13957 (OR) == 13963 (NAND).
ATPG learned equiv relationship: 13916 (XOR) != 13929 (XOR).
ATPG learned equiv relationship: 13955 (NAND) == 13958 (NAND).
ATPG learned equiv relationship: 69502 (MUX) == 69505 (AND).

```

```
ATPG learned equiv relationship: 14293 (NOR) != 14300 (OR).
ATPG learned tied relationship: 14347 (OR) = 1.
ATPG learned tied relationship: 15774 (AND) = 0.
ATPG learn analysis aborted after 1000 test generation analyses.
ATPG equiv learning completed: #ties=5, #equiv=90, #abort=3/2, CPU
time=10.21
Tied analysis completed: #tied_gates=11716, #blocked_gates=71/0, #blocked_
inputs=431
End learning analyses, total learning CPU time=10.25
-----
```

See Also[report_settings](#)[run_build_model](#)[set_messages](#)[ATPG-Specific Learning Processes](#)

set_match_names

Overview

This command specifies a set of name manipulation parameters that enable the `match_names` command to correctly match the referenced logical names with the physical names in the physical database.

Syntax

```
set_match_names
  [-auto]
  [-exclude_modules {verilog_module_names}]
  [-instance name]
  [-reset]
  [-sub_prefix {prefix1 prefix2}]
  [-sub_str {string1 string2}]
```

Arguments

-auto

Enables the [match_names](#) command to automatically define name matching rules to resolve hierarchy conflicts related to top-level wrappers.

-exclude_modules {verilog_module_names}

Sets a flag on the specified logical module or modules which do not have counterpart physical modules. As a result of this setting, the diagnosis operation does not report physical data in the YDF tables for the specified modules. You should specify only Verilog module names; the module names provided by the [report_physical](#) command should not be used.

-instance name

Applies the name matching operation to the specified instance and all instances under its hierarchy. Note that this option is intended only for debugging mismatches. It does not accept a Tcl list of instances or regular expressions.

-reset

Resets all name-manipulation specifications originally set by the `set_match_names` command.

-sub_prefix {prefix1 prefix2}

Replaces the instance prefix *prefix1* with *prefix2* from the flattened logical instance names.

-sub_str {string1 string2}

Replaces all occurrences of *string1* in flattened logical instance names with *string2*. Note that in the reverse name matching logic used in bridge extraction, the whole pin (for example, `'/D'` and if the `'/D'` is the end of the string) or hierarchy level (the name between `'/'` to be matched) is required. For details on reverse name matching logic used in bridge extraction, see the Description section below.

Allowed Command Modes

All

Description

The `set_match_names` command specifies a set of name manipulation parameters that enable the `match_names` command to correctly match the referenced logical names with the physical names in the physical database.

You should specify both the `set_match_names` and [match_names](#) command to ensure that the physical pin names are properly associated with the logical pin names and the subnet information is annotated to the logical view.

If you specify the `set_match_names -sub_prefix {A B}` command, all occurrences of A are deleted and replaced with B. The `match_names` command is then able to match the logical name with the new prefix in the physical database.

The following example replaces the logical prefix "Uxxx" with the physical prefix "Top_inst/Uxxx":

```
set_match_names -sub_prefix {logical_top_level_design_name/Uxx physical_top_level_design_name/Top_inst/Uxxx}
```

Assuming the logical and physical names are matched, the `write_ydf` command automatically converts the logical candidate names to the physical names. It then obtains the relevant physical information from the composite image and writes this data to a table.

To list the current name matching rules, specify the [report_settings](#) `match_names` command.

If you use the `-sub_str` option, the reverse name matching logic used in bridge extraction has some specific requirements. If the forward name matching rule is `/FF0/D -> /D0`, the reverse name matching works as follows:

- `A/B/D0/Z->-> A/B/FF0/D/Z`

This specification is valid because the forward slash (/) is the hierarchy delimiter and the "/D0/" string is identified.

- `A/B/D0_C/Z -> A/B/D0_C/Z`

This specification is valid because the reverse name matching rule is not applied in this case.

Note the following constraints:

- Reverse name matching is not supported in the following case:

```
set_match_name -sub_str {"/sub1/" "_a1/"}
```

- Empty strings are not supported

Forward: `top/block1/wrapper_a1/block/inst/pin => top/block1/a1/block/inst/pin`

Reverse: `top/block1/a1/block/inst/pin => top/block1/a1/block/inst/pin`

In this case, the following commands will work:

```
set_match_name -sub_str {"block1/wrapper_a1" "block1/a1"}
```

```
set_match_name -sub_str {"block1/wrapper_ a2" "block1/a2"}
...
set_match_name -sub_str {"block1/wrapper_ ax" " block1/ax"}
```

Note that the following use cases work:

- **Case 1**

```
set_match_name -sub_str {"/wrapper/block/" "/blockA/"}
```

Forward: top/block1/wrapper/block/inst/pin => top/block1/blockA/inst/pin

Reverse: top/block1/blockA/inst/pin => top/block1/wrapper/block/inst/pin

- **Case 2**

```
set_match_name -sub_str {"/instA/sub1/pinA" "/instA/pinX"}
```

Forward: top/block1/instA/sub1/pinA => top/block1/instA/pinX

Reverse: top/block1/instA/pinX => top/block1/instA/sub1/pinA

- **Case 3**

```
set_match_name -sub_str {"/FF0/D" "/D0"}
```

Forward: top/block1/instA/FF0/D => top/block1/instA/D0

Reverse: top/block1/instA/D0 => top/block1/instA/FF0/D

Reverse: top/block1/D0/inst/pin=> top/block1/FF0/D/inst/pin

Case 4

In the following case, the forward name matching works, but the reverse name matching does not work:

```
set_match_name -sub_str {"/sub1/" "_a1/"}
```

Forward: top/block1/block1/sub1/block/inst/pin => top/block1/block1_a1/block/inst/pin

Reverse: top/block1/block1_a1/block/inst/pin => top/block1/block1/sub1/block/inst/pin

Case 5

This case works:

```
set_match_name -sub_str {"_a1/" "/sub1/"}
```

Forward: top/block1/block1_a1/block/inst/pin => top/block1/block1/sub1/block/inst/pin

Reverse: top/block1/block1/sub1/block/inst/pin => top/block1/block1_a1/block/inst/pin

Note the following:

- There are some name matching rules that cannot be reversed (with empty strings, constraint Case 2).
- If you avoid using rules with empty strings, and use additional rules with non-empty strings, the reverse operations will work. This improves bridge diagnostics results.

Examples

```
// command to change the hierarchy prefix from top_i to i_core
set_match_names -sub_prefix {top_i i_core}

-----

Name Match Options Summary
-----

Name Manipulation Flag = ON
Prefix to be deleted: top_i
Prefix to be added: i_core
-----

// command to replace top_i/i_core with top_i1/i_core1
set_match_names -sub_str {top_i/i_core top_i1/i_core1}

// command to reset the match settings
set_match_names -reset
```

See Also

[Name Matching Using a PHDS Database](#)

[match_names](#)

[report_physical](#)

[report_layout](#)

[write_ydf](#)

set_messages

Overview

Use this command to set the parameters that control message handling.

Syntax

```
set_messages
[-display | -nodisplay] [-leading_comment | -noleading_comment] [-nolog |
-log file_name [-replace | -append]]
[-level <expert | standard>]
[-transcript_comments | -notranscript_comments]
```

Arguments

-display | -nodisplay

Controls the display of text to the command transcript without affecting what is recorded in a logfile. With a setting of `-nodisplay`, the output of commands are suppressed except those commands entered interactively. The default is `-display`.

-leading_comment | -noleading_comment

Specifies whether a leading comment (the pound sign (#) in Tcl mode or a pair of forward slashes "/" in native mode) precedes message text in the command transcript. (This setting has no effect on the output from "report" commands.) The default is `-noleading_comment`.

-nolog | -log *file_name* [-replace | -append]

The `-log` option records commands and transcript output to a file. If the specified file already exists, an error occurs, unless you use the `-replace` or `-append` option. Using `-nolog` turns off message logging. The default is `-nolog`. Note that in non-Tcl mode, a dash prefix is not used for the `nolog` and `log` options.

-level <expert | standard>

Selects the general level of message reporting. The default is standard level. With the level set to expert, more information is displayed for many commands.

-transcript_comments | -notranscript_comments

Selects whether comment lines and blank lines encountered in [command files](#) appear in the transcript window as the command file is executed. When you specify `-notranscript_comments`, only commands are shown in the transcript. Note that `-notranscript_comments` only works in native mode; it does not work in Tcl mode. When you specify `-transcript_comments`, both commands and comments from a command file are shown in the transcript; this is the default.

Allowed Command Modes

All

Description

Use this command to set the parameters that control message handling. You can display the current settings

with the `report_settings` command. The primary use is to establish a log file and to turn on and off the text displayed in the command transcript window.

Examples

```
TEST> report_clocks
clock_name      off usage
-----
/CLK            0  master shift
/RSTB           1  master reset
TEST> set_messages -nodisplay
TEST> report_clocks
TEST> report_settings
TEST>
```

```
TEST> report_version
2009.06
TEST>
```

See Also

[report_settings](#)

Table Column Outside Table:

set_netlist

Overview

Use this command to set the parameters that control the reading of netlists.

Syntax

```
set_netlist
  [-celldefine | -nocelldefine]
  [-check_only_used_udps | -nocheck_only_used_udps]
  [-conservative_mux < none | combinational_udp | all > ]
  [-dominance_detection <on | off | boolean>]
  [-enable_portfaults | -noenable_portfaults]
  [-escape <both | all | none>]
  [-max_errors d]
  [-pin_assign | -nopin_assign]
  [-redefined_module <first | last>]
  [-scalar_net | -noscalar_net]
  [-sequential_modeling | -nosequential_modeling]
  [-suppress_faults | -nosuppress_faults]
  [-xmodeling | -noxmodeling]
```

Arguments

-celldefine | -nocelldefine

Selects whether to ignore the Verilog compiler directive `celldefine in netlists. The default is -celldefine.

-check_only_used_udps | -nocheck_only_used_udps

Specifies check and warning behavior for reading design netlists and libraries. With the default, -check_only_used_udps, the checks are performed during the build phase and only the modules and UDPs actually used by the design are checked. Using -nocheck_only_used_udps causes all library and netlist files to be checked as they are read.

-conservative_mux < none | combinational_udp | all >

Specifies if [conservative MUX gates](#) are extracted from conservative UDP models of a MUX. In a conservative MUX, if the select input is X, then the output is X, even if the data inputs have the same Boolean value. In a [regular "optimistic" MUX](#), if the select input is X and the data inputs have the same Boolean value, then the output has the value of the data inputs.

If the UDP model specifies optimistic MUX behavior, then this is the behavior that will take place in TestMAX ATPG. If the UDP model specifies conservative MUX behavior however, this option will control whether the resulting TestMAX ATPG MUXes will have optimistic or conservative behavior.

If none is selected, then all MUXes extracted are regular (and appropriate violations are issued if necessary).

If combinational_udp is selected (the default), then conservative MUXes can be extracted from combinational UDPs, but not from sequential UDPs. Previously, UDPs describing a conservative MUX were extracted into a regular MUX and rule violations N20

(underspecified UDP) and N12 (inconsistent usage of x) were issued. When a conservative MUX is extracted, these N20s and N12s are no longer issued. To support this, the checking of N12 was moved from before to after the extraction of the UDP's function into gates. This has the side effect that the remaining N12 violations of combinational UDPs are, in some cases, fewer but using more wildcards.

If `all` is selected, then conservative MUXes can be extracted from any UDP, combinational or sequential. When this setting is selected, extraction of sequential UDPs is less tolerant to incomplete and inconsistent UDPs, which might be extracted into TIEX, even though a gate-level model could be extracted if `-conservative_mux` is set to `none` or `-combinational_udp`. Use the `all` setting only if the sequential UDPs you parse are properly defined.

-dominance_detection <on | off | boolean>

Specifies whether the derived ATPG model recognizes dominance behavior between the set/reset/clock pins. A selection of `off` disables this checking, whereas `on` enables it. A selection of `Boolean` enables the checking but checks only the state table entries for which clock/set/reset are Boolean values, and ignores the entries where they are Xs. The default is `on`.

-enable_portfaults | -noenable_portfaults

Specifies whether to ignore the Verilog compiler directive ``enable_portfaults` in netlists. The default is `-enable_portfaults`.

-escape <both | all | none>

Specifies whether the escape character is preserved as part of the identifier's name. The choices are: to store the instance names both with and without escape characters (`both`) (the default); to always preserve escape characters (`all`); to never preserve escape characters (`none`). You should use the `none` argument with caution. For example, if you specify the `none` argument, `"\net[4]"` changes to `"net[4]."` As a result, this name becomes indistinguishable from bit 4 of the "net" vector net.

-max_errors d

Specifies the maximum number of parsing errors allowed before terminating netlist parsing of the current file. The default setting is 10.

-pin_assign | -nopin_assign

Specifies whether a netlist assign statement results in artificial pin names (and faults) being assigned to all inputs and to the output of the assignment. The default is `-nopin_assign`. An assign statement can only appear in Verilog and VHDL netlists.

-redefined_module <first | last>

Specifies how to treat the redefinition of a module. In case of multiple definitions, one definition is kept and the others are ignored. You can specify the first or last encountered netlist definition as the one to keep. The default is `last`.

-scalar_net | -noscalar_net

Specifies whether Verilog nets such as `"wire [N:N] netname"` are considered 1-bit vector nets or scalar nets. The default is a 1-bit vector net. If you select `-scalar_net`, such nets instead are considered scalar nets (as if the definition were in the form of `"wire netname"`).

-sequential_modeling | -nosequential_modeling

Specifies whether the derived ATPG model is generated to precisely model set/rst/clock simultaneous effects during sequential simulation. This is usually not needed for ATPG pattern generation but might be needed for fault grading of functional patterns when X-miscompares need to be avoided. The default is `-nosequential_modeling`, and this produces a memory-efficient derived ATPG model that is accurate for Basic-Scan and Fast-Sequential ATPG, but is slightly more pessimistic (output=X when simultaneous set/rst/clk active) than the more precise model.

`-suppress_faults | -nosuppress_faults`

Specifies whether to ignore the Verilog compiler directive ``suppress_faults` in netlists. Using `-nosuppress_faults` causes this directive to be ignored. The default is `-suppress_faults`.

`-xmodeling | -noxmodeling`

When a UDP table is missing a line, it implies the output should be X for the omitted input condition. However, for translation into Boolean gates, the expected output is sometimes non-X to map to a gate-level equivalent. When `-xmodeling` is selected and this situation occurs, then additional gates are added to the derived ATPG model in an attempt to exactly match the defined UDP table. If `-noxmodeling` is selected, then a simpler, more optimal gate-level model is used. The default is `-xmodeling`.

Allowed Command Modes

All

Description

Use this command to set the parameters that control the reading of netlists.

Any changes made with this command only affect future `read_netlist` commands. It does not affect netlists that have already been read by TestMAX ATPG.

For more information on using the `set_netlist` command and the Set Netlist dialog box (its equivalent in the TestMAX ATPG GUI), see "[Preparing a Netlist](#)."

Examples

```
DRC> set_netlist -escape none -scalar_net -redefined_module last
```

```
DRC> set_netlist -xmodeling
```

See Also

[Design Netlists and Model Libraries](#)
[report_settings](#)
[read_netlist](#)
[DRC Rule N15](#)

set_patterns

Overview

Use this command to select the source of patterns to use for simulation, fault simulation, or test generation.

Syntax

```
set_patterns
  [-adjacent | -discontiguous]
  [-append]
  [-delete]
  [-internal | -random | -external filename]
  [-cycle_count | -nocycle_count]
  [-keep_dictionary]
  [-histogram_summary | -nohistogram_summary]
  [-load_summary | -noload_summary]
  [-netlist_independent | -nonetlist_independent]
  [-novcd_clock | vcd_clock <auto | <0|1> port>]
  [-resolve_differences failures_log file]
  [-sensitive | -insensitive]
  [-split_patterns]
  [-strobe_comments comments]
  [-strobe_period < d <fs|ps|ns|us|ms|s>>]
  [-strobe_rising port_name]
  [-strobe_falling port_name]
  [-strobe_event port_name]
  [-strobe_offset < d <fs|ps|ns|us|ms|s>>]
  [-verbose]
```

Arguments

-adjacent | -discontiguous

These options are used only when the `-external` and `-append` options of the `set_patterns` command are specified. The default, the `-append` option, assumes that the current appended pattern block is generated in sequential order from the previously appended pattern set. The `-discontiguous` option is used when reading patterns that add initial patterns that are not generated in sequential order or are appended in a different order than the previously appended pattern set.

-append

This option must be used with the `-external` option (`set_patterns -external filename -append`). It reads the external pattern file and appends the patterns to any existing patterns in the internal pattern buffer. TestMAX ATPG rennumbers the patterns based on their order of occurrence in the new internal pattern set.

-delete

Deletes all current internal and external patterns before reading external patterns or switching the pattern source.

-internal | -random | -external *filename*

These options indicate the pattern source:

-internal specifies that the pattern source is internally generated.

-random specifies that the pattern source is random (defined by the parameters set with the `set_random_patterns` command). TestMAX ATPG does not support good machine simulation of a random pattern source if you select the `-nobasic_scan` option of the `set_simulation` command.

-external specifies that the pattern source is an external file. The file formats accepted are binary, extended VCD, and limited-syntax forms of STIL, Verilog, VHDL, and WGL. TestMAX ATPG reads only the syntax and format similar to what it writes.

Use binary formats whenever possible to read patterns into TestMAX ATPG. Other pattern formats such as WGL and STIL have limited capabilities to store all data about the patterns. For example, when you read a STIL or WGL pattern file back, TestMAX ATPG might interpret a fast-sequential pattern as a full-sequential pattern.

-cycle_count | -nocycle_count

By default, cycle counting will always occur on execution of the `set_patterns -external` command with a cycle-formatted file. Should this option cause problems, it can be disabled with the command `-nocycle_count` option.

The cycle count must be generated when reading external patterns. This is the only mechanism to identify cycles relative to the actual formatted (cyclized) data used at test. This allows for test formatting changes to be properly interpreted and used as the basis for identifying failing patterns and offsets. For example, if the final test program adds a cycle into the load_unload procedure at test, then reading this modified test data back into the `set_patterns -external` command creates a properly referenced baseline for interpretation of the cycle offset data in the failure file.

-histogram_summary | -nohistogram_summary

Selects additional reporting for the `report_summaries` command that displays a histogram of the number of N-cycle patterns as well as the number of N-load patterns.

-keep_dictionary

Retains the current dictionary when reading in an external pattern set during diagnostics. For more information, see [Using a Dictionary for Diagnostics](#).

-load_summary | -noload_summary

Selects additional reporting for the `report_summaries` command that displays the total number of scan loads used by the basic-scan and fast-sequential patterns.

-netlist_independent | -nonetlist_independent

If you specify the `-netlist_independent` option when the DRC file has the design in the adaptive scan mode, any subsequent "write patterns" in binary format create a special pattern file that can be used to translate adaptive scan patterns so they can be used in scan mode for improved diagnostic resolution (no compression). This special pattern set must be read back in after a scan mode DRC file is read in for the user-transparent translation to occur. After the patterns are read back, a simulation is done internally to compute the expected values to complete the translation process. The translated scan mode patterns can then be written out. The default is `-nonetlist_independent`.

Note that the netlist-independent binary patterns do not store response data. This is necessary if they are to be truly netlist-independent. When TestMAX ATPG reads the

netlist-independent patterns back in, it simulates them to determine the proper response data for the current netlist. Note also that this means that the timing exceptions need to be in place at the time the netlist-independent patterns are read into TestMAX ATPG, as the effect of the exceptions will not otherwise show up in the pattern response data.

-novcd_clock | vcd_clock <auto | <0|1> port>

When the pattern source is extended VCD, it is possible to recognize clock events in the pattern stream and convert them into a single pattern event that pulses a clock rather than retain them as multiple events corresponding to clock off, then clock on, then clock off. This reduces the size of the patterns after they are read into TestMAX ATPG, and might result in faster simulation, fault grading, and reduced memory needs. By default, no clocks are defined when reading VCDE patterns.

`-vcd_clock auto` passes the known clocks are passed to the pattern reader for event collapsing and clock recovery. Use the `-vcd_clock auto` option if clocks have already been defined for the current design.

`-vcd_clock <0|1> port` defines clocks (if you have not already defined them using the `add_clocks` command) or creates a subset list of clocks for clock recovery efforts.

`-novcd_clock` disables automatically passing the known list of clocks to the pattern reader.

Note: If you define clocks, TestMAX ATPG checks the patterns for unsupported events such as PO measures while clocks are active, PI changes while clocks are active, multiple clocks on simultaneously that do not have the exact same start time and duration. If any of these events occur, TestMAX ATPG displays a warning message and abandons the event collapsing effort for the pattern.

-resolve_differences failures_log_file

This option is used to update the external pattern sets without re-running ATPG. It resolves tester failures or simulation failures — that is, it performs per-cycle pattern masking.

This option does not cause a simulation of the patterns. It simply reads a failure file showing mismatches and places an X in the pattern set located in the external pattern buffer (`-external file_name`) in the position of each of the mismatches. The presumption is that the pattern set has had either Verilog simulation mismatches or failures on the ATE, and the failure file contains a record of these mismatches/failures.

All formats used in diagnosis are supported. Support includes both scan and adaptive scan patterns. After masking, the updated patterns could be fault simulated using `run_fault_sim` and written out to be run on the tester.

For related information, please see [Updating Pattern Sets Without Re-Running ATPG](#).

-sensitive | -insensitive

Specifies case sensitivity for the pattern file. By default, TestMAX ATPG determines the case-sensitivity based on the language: case-insensitive for EDIF and VHDL, or case-sensitive for STIL, Verilog, and Extended VCD.

-split_patterns

When this option is specified, the patterns from the specified file are appended to the patterns already in the external pattern buffer if the previous `set_patterns -external` command also used the `-split_patterns` option. If there are patterns in the external buffer that were not placed there by using the `-split_patterns` option, the existing

patterns in the external buffer are first deleted. TestMAX ATPG will store some data regarding the pattern offset and pattern file names for use during diagnosis. The pattern file names must be unique when the hierarchical directory name is removed.

Multiple pattern files can be read into the external pattern buffer for diagnosis by issuing the following command sequence:

```
set_patterns -external pat1.bin -split_patterns
set_patterns -external pat2.bin -split_patterns
```

During the execution of the `set_patterns -split_patterns` command, a report of each pattern file read in is displayed. It includes the pattern file name and the pattern range considered during diagnosis. The cycle range of the patterns considered for diagnosis is displayed, if available. The line is highlighted in bold in the examples shown below:

```
TEST> set_patterns -external split_pattern/prog.stil_0.gz -split
End parsing STIL file split_pattern/prog.stil_0.gz with 0 errors.
End reading 5 patterns, CPU_time = 0.01 sec, Memory = 0MB
Pattern_file=prog.stil_0.gz, basic_scan=[0(3),4(66)], fast_sequential=[-
1,-1]
```

```
TEST> set_patterns -external split_pattern/prog.fast.stil_0.gz -split
End parsing STIL file split_pattern/prog.fast.stil_0.gz with 0 errors.
End reading 5 patterns, CPU_time = 0.01 sec, Memory = 0MB
Pattern_file=prog.stil_0.gz, basic_scan=[0(3),4(66)], fast_sequential=[-
1,-1]
Pattern_file=prog.fast.stil_0.gz, basic_scan=[-1,-1], fast_sequential=[5
(82),9(150)]
```

In the previous example, the pattern ranges noted `[-1, -1]` means that no pattern of this type has been read in. The text `[0(3),4(66)]` indicates the pattern and cycle range read for the first pattern file. The first pattern is number 0 and it starts at cycle 3. The last pattern is number 4 and it starts at cycle 66. the text `[5(82),9(150)]` indicates the pattern and cycle range read for the second pattern file. The first pattern is number 5 and it starts at cycle 82. The last pattern is number 9 and it starts at cycle 150.

```
-strobe_comments comments
-strobe_period < d <fs|ps|ns|us|ms|s>>
-strobe_rising port_name
-strobe_falling port_name
-strobe_event port_name
-strobe_offset < d <fs|ps|ns|us|ms|s>>
```

Use these options when reading an external pattern source, which is Extended VCD (VCDE). These options define a trigger condition for measuring expected values based upon events at a primary input, output, or bidirectional port. The `-strobe_offset` option acts as a qualifier on the trigger choice.

The *port_name* is any input, output, or bidirectional port in the current top module.

The `-strobe_rising`, `-strobe_falling`, `-strobe_event`, `-strobe_period`, and `-strobe_comment` options are mutually exclusive; you can specify only one of these options. You cannot combine the `-strobe_offset` option with the `-strobe_comments` option.

The `-strobe_rising` option triggers a measure whenever an event, for which the specified port changes to a logic one, occurs. This includes the transitions 0->1, 1->1, X->1, Z->1. Without an offset, TestMAX ATPG inserts the measure before processing the pattern event that caused the trigger. With a non-zero offset, TestMAX ATPG inserts the measure at the specified time offset after the time of the trigger.

The `-strobe_falling` and `-strobe_event` options are similar to the rising option. The `-strobe_falling` option causes a trigger for any event, which changes the named port to a logic 0. The `-strobe_event` option causes a trigger for any event on the named port, including transition with no level change such as 1->1 or 0->0. You can combine both the falling and event option with the strobe offset.

The `-strobe_period` option defines a repeating time interval at which TestMAX ATPG inserts a measure into the external pattern set. The VCDE data file specifies the expected values, but the strobe period specifies the time of measure. The *d* value represents a nonzero positive integer. FS | PS | NS | US | MS|S specify units of time: fs = femtoseconds, ps = picoseconds, ns = nanoseconds, us = microseconds, ms = milliseconds, and s = seconds.

You can combine `-strobe_offset` with `-strobe_rising`, `-strobe_falling`, `-strobe_event`, or `-strobe_period`. Use the offset to adjust the measure point in the external patterns relative to the trigger event. Like the `-strobe_period` option, the `-strobe_offset` option requires an integer and a unit of time. This integer can be zero.

If you do not define an offset, TestMAX ATPG considers it as an offset of zero. An offset of zero means that TestMAX ATPG inserts the measure before the trigger event and before processing the VCDE data changes that caused the trigger event.

When you use `-strobe_offset` in conjunction with `-strobe_period`, then the offset defines the offset from time=0 in the pattern stream at which TestMAX ATPG starts the first measure. For example, "`-strobe_period {200 ns} -strobe_offset {399 ns}`" causes the first measure to occur at absolute time of 399 ns and at 200 ns intervals thereafter, or at the intervals 399, 599, 799, 999 ...

If you specify a strobe period without any strobe offset, then TestMAX ATPG inserts the first measure at an offset just before the period. For example, `-strobe_period {100 ns}` produces a measure at times 100, 200, 300, 400...

If you use a strobe offset in conjunction with a rising, falling, or event trigger, then once a data event on the specified port causes the trigger, TestMAX ATPG ignores any additional events on that port until the offset time elapses.

If a strobe offset occurs simultaneously with events in the pattern data, then TestMAX ATPG inserts the measure point before processing the event data.

Use the `-strobe_comments` option to indicate the location of measure points in the VCDE file with the location of (`$comment ... $end` blocks as markers for measure points. For best results use the comment blocks at #NNNNN event times by themselves; do not combine with any other PI or expected PO changes. You cannot combine the comments option with any other `-strobe` option.

-verbose

This option lists all unused primary inputs and scan cells due to DRC differences between compression and regular scan mode when reading netlist-independent patterns.

Allowed Command Modes

Test

Description

Use this command to select the source of patterns to use for simulation, fault simulation, or test generation. You must select an external source to read in ATPG or functional patterns.

To display the current settings, use the `report_settings` command.

Examples

```
TEST> set_patterns -delete
Warning: Internal pattern set is now deleted. (M133)
```

```
TEST> set_patterns -random
```

```
TEST> set_patterns -external b010.vil
End parsing Verilog file b010.vil with 0 errors;
End reading 36 patterns, CPU_time = 5.38 sec, Memory = 28MB
```

```
TEST> set_patterns -external func_pat.vcde -strobe_offset {95 ns} -strobe_
period {100 ns }
End reading 83 patterns, CPU_time = 1.91 sec, Memory = 84MB
```

```
TEST> set_patterns -external data.vcde -strobe_rising CLK -strobe_offset {50
ns}
End reading 752 patterns, CPU_time = 9.41 sec, Memory = 244MB
```

```
TEST> set_patterns -external func_fifo2.vcde -strobe_period {100 ns} -
strobe_offset {40 ns }
-vcd_clock 0 clk1 -vcd 0 clk2 -vcd 0 clk3
End reading 169 patterns, CPU_time = 3.21 sec, Memory = 12MB
```

```
TEST> set_patterns -external chk2_fifo.vcde \
-strobe_period {100 ns} -strobe_offset {90 ns} -vcd_clock auto
Note: adding -vcd_clock 0 c1
Note: adding -vcd_clock 0 c2
Note: adding -vcd_clock 0 c3
End reading 52 patterns, CPU_time = 3.90 sec, Memory = 9MB
```

```
TEST> report_summaries patterns
Pattern Summary Report
-----
#internal patterns 66
#basic_scan patterns 19
#fast_sequential patterns 47
-----
```

```
TEST> set_patterns -histogram_summary
```

```
TEST> report_summaries patterns
      Pattern Summary Report
```

```
-----
#internal patterns 66
  #basic_scan patterns 19
  #fast_sequential patterns 47
    # 2-cycle patterns 7
    # 3-cycle patterns 24
    # 4-cycle patterns 16
    # 2-load patterns 14
    # 3-load patterns 33
-----
```

```
TEST> set_patterns -load_summary
TEST> report_summaries patterns
      Pattern Summary Report
```

```
-----
#internal patterns 66
  #basic_scan patterns 19
  #fast_sequential patterns 47
  Total #loads 146
-----
```

See Also

[report_patterns](#)

[report_settings](#)

[report_summaries](#)

[set_random_patterns](#)

[write_patterns](#)

Table Column Outside Table:

set_physical

Overview

This command sets options for the `report_physical` and [run_diagnosis](#) commands for physical diagnostics..

Syntax

```
set_physical  
  [-num_processes number]  
  [-tolerance {metal tolerance}]  
  [-via_bridges | -novia_bridges]
```

Arguments

-num_processes *number*

Enables multicore static extraction from a PHDS database. This option is supported by the following commands: [match_names](#), [extract_nets](#) -all, and [extract_pins](#) -scan_path.

-tolerance {*metal distance*}

Specifies a tolerance distance, in nanometers, for extracting neighbor nets for specific layers. The default tolerance per layer is equal to the pitch if this information can be found in the technology information. If not, the default is 1000nm. You can determine the appropriate tolerance by analyzing technology data, such as pitch distance. You must use a Tcl list to specify each layer and its tolerance, as shown in the following example:

```
set_physical -tolerance {METAL 50 METAL2 100 METAL3 200 }
```

For details on using this option, see "[Setting Up and Running Physical Diagnosis](#)."

-via_bridges | -novia_bridges

When the `-via_bridges` option is specified, metal layer shapes included in via definitions are considered when extracting net neighbors for bridge candidates. The runtime is affected when metal layer shapes are included. However, net neighbors are more thoroughly extracted and bridge defects between a net and a via or between two vias are more accurately diagnosed. This option requires that the PHDS database was created using TestMAX ATPG version J-2014.09-SP1 or later, or Yield Explorer version J-2014.03-SP3 or later. The default, the `-no_viabridges` option, does not consider metal layer shapes included in the via definitions.

Allowed Command Modes

All

Description

This command sets options for the `report_physical` and [run_diagnosis](#) commands for physical diagnostics. For more information on using the `set_physical` command, see "[Using Physical Data for Diagnosis](#)."

See Also

[set_diagnosis](#)

set_physical_db

Overview

This command sets the parameters for creating, validating, and opening the PHDS database, and connecting TestMAX ATPG diagnostics to the PHDS database.

Syntax

```
set_physical_db
  [-create_phds | -nocreate_phds]
  [-database dirpath]
  [-device {name version}]
  [-def_directory dirpath]
  [-hostname name]
  [-lef_directory dirpath]
  [-password name]
  [-phds_validation_file file_name]
  [-port_number number]
  [-retry_connection {max_attempts time_between_attempts}]
  [-technology_lef_file {tech_file_names} | -notechnology_lef_file]
  [-top_def_file filename]
  [-top_design top_design_name]
  [-user name]
  [-validation | -novalidation]
```

Arguments

-create_phds | -nocreate_phds

Enables or disables the creation of the PHDS database. The PHDS database is created by default. If you do not want to create the PHDS database, use the `-nocreate_phds` option. If you previously used the `set_physical_db -nocreate_db` command, and want to create the full PHDS database in a subsequent run, use the `-create_phds` option.

-database *dirpath*

Specifies the output directory for the PHDS database. This option enables TestMAX ATPG diagnostics to open a connection with the PHDS database.

-def_directory *dirpath*

Specifies the directory containing the DEF files used for building the PHDS database.

-device {*name version*}

Specifies the name and version of the design used for establishing a connection with the PHDS database. The parameters must be specified in Tcl list format. For example:

```
set_physical_db -device {DES 4}
```

-hostname *name*

Specifies the name of the machine on which to connect to the PHDS database. This machine must have a process called DAPListener, which is launched by the [open_physical_db](#) command.

-lef_directory *dirpath*

Specifies the directory containing the LEF files used for building the PHDS database.

-password *name*

Specifies the password for connecting to the PHDS database created by Yield Explorer.

-phds_validation_file *file_name*

Specifies the name of a file containing validation information (Y rule violations) from a previous session.

-retry_connection {*max_attempts time_between_attempts*}

Changes the number of maximum connection attempts and the time between attempts in seconds. The following example sets the maximum number to 100 and the time in between attempts at 60s.

```
BUILD-T> set_physical_db -retry_connection {100 60}
```

-port_number *number*

Establishes a connection to the PHDS database by specifying the port number of the host in which TestMAX ATPG is running. The default port number is 9998. This is an option specification that you make before opening the physical database (using the [open_physical_db](#) command). However, you must specify the port number to connect to an existing PHDS connection (using the `set_physical_db -hostname` command).

-technology_lef_file {*tech_file_names*} | -notechnology_lef_file

Specifies the name(s) of a LEF technology file(s) to use for creating the PHDS database. By default, TestMAX ATPG automatically uses a single LEF technology file if that file has the string "tech" embedded in its name. The `-notechnology_lef_file` option clears any previously specified list of LEF technology files.

-top_def_file *filename*

Specifies the name of the top-level DEF file to use when the LEF/DEF database contains multiple DEF files.

```
set_physical_db -top_def_file top_def_file
```

-top_design *top_design_name*

Specifies the name of the top design to use as a basis for creating or validating the PHDS database.

-user *name*

Specifies the user login name for connecting to the PHDS database created by Yield Explorer.

-validation | -novalidation

Enables or disables the PHDS database validation process. The validation is run by default. If you want to create the PHDS database without validating it, use the `-novalidation` option. If you previously used the `set_physical_db -novalidation` command, but want

to validate the PHDS database in a subsequent run, use the `-validation` option. For related information, see the "[Category Y - PHDS Rules](#)" topic.

Description

This command sets the parameters for creating, validating, and opening a PHDS (Synopsys Physical Design Store) database, and connecting TestMAX ATPG diagnostics to the PHDS database.

For details on using this command, see the "[Using TestMAX ATPG to Create and Validate the PHDS Database](#)" topic.

Examples

The following set of commands create the PHDS database in the phds directory:

```
set_physical_db -lef_directory ./lef -def_directory ./def
set_physical_db -top_def_file top_design.def
set_physical_db -database ./phds
set_physical_db -device [list DES 4]
write_physical_db -replace -verbose
```

The following set of commands open a connection to a PHDS database:

```
set_physical_db -database ./phds -port_number 9990
open_physical_db
```

The following command connects TestMAX ATPG diagnostics to the PHDS database:

```
set_physical_db -hostname localhost101 -port_number 9990 -user tester -
password safel234
Setting user name ('tester') for physical connection.
Setting password ('safel234') for physical connection.
Setting host name ('localhost101') for physical connection.
Setting port number ('9990') for physical connection.
Connecting to physical database.
Successfully connected to physical database.
Available Devices:
-----
DES 1
DES 2
DES 3
DES 4
TST 1
```

See Also

[write_physical_db](#)

Table Column Outside Table:

set_pindata

Overview

This command controls the type of information displayed by the `report_primitives` command or displayed within the graphical schematic viewer.

Syntax

```
set_pindata
  [-bidi_control_value | -clock_cone port_name | -clock_off | -clock_on port_name |
  -constrain_data | -debug_sim_data | -delay_data | -error_data | -
  fault_sim_results <pin_pathnamegatepin_number> <0|1> pattern | -fault_data
  | -full_seq_scoap_data | -full_seq_tg_data | -good_sim_results pattern |
  -load | -master_observe | -none | -pattern <d | all | fast_sequential> |
  scoap_data | -seq_sim_data | -shadow_observe | -shift | -slack | -
  stability_patterns | -test_setup | -tie_data ]
  [-constrain_character c]
  [-drcdata {unload_mode d}]
  [-gatedata]
  [-lfsr_shift | -lfsr_load]
  [-prpg_shadow_shift | -prpg_shadow_load>]
  [-refresh]
  [-scoap_data]
  [-sdc_case_analysis]
  [-seq_sim_data]
  [-setup_cycles {n1 n2}]
  [-shadow_observe]
  [-shift]
  [-shift_character c]
  [-slack]
  [-stability_patterns]
  [-test_setup]
  [-tie_data]
```

Arguments

-bidi_control_value

This option displays the simulated values that occur when the `bidi_control` pin is set to its [off state](#) (as defined by the `set_drc` command).

-clock_cone *port_name*

Displays clock cone information for the specified clock. This information includes:

C indicates the pin is in the [clock cone](#).

E indicates the pin is in the [effect cone](#).

CE indicates the pin is in both clock and effect cones.

N indicates the pin is not in either clock or effect cone.

-clock_off

Displays the simulated value that results when all [clocks](#) are off.

-clock_on

Displays the simulated value that results when all [clocks](#) are on.

-constrain_character

Specifies the constrain character to be used when displaying X values for constrain value data.

-constrain_data

Six fields are displayed that show the tied values and constrained values for both combinational and sequential ATPG, and the detection blockages that result from these constraints. The characters are displayed in the format "T/B1,C/B2,S/B3" where:

T = constant value due to tied logic

B = fault blockage due to tied value "T", if any

C = constrained value during combinational ATPG, if any

B = fault blockages due to C, if any

S = constrained value during sequential ATPG

B = fault blockages due to S, if any

An "X" indicate there is no constant or constrained value. A "-" indicates there is no blockage. A "~" in front, for example "~1Z", means constrained to "not 1 or Z", that is, values of 0 and X.

-debug_sim_data

Displays mapping error simulation values, either a single selected simulation source or a comparison of two simulation sources. When two sources are shown, the data is separated by a slash, as in "AAA / BBB ". For a Fast-Sequential simulation source, a scan load is shown by "#", and a dash "-" is used to separate different capture procedures. For Full-Sequential or VCD simulation sources, a single continuous stream of characters is shown. A VCD source may not contain data for all circuit points; the absence of data is shown with a single dash as in "-" or " AAA / - ".

-delay_data

The [delay path](#) (path_name) currently loaded by the `report_delay path path_name -p` or `set_delay -data path_name` is available for display in the GSV or reporting in a primitive report. See [Understanding Pin Data](#) for more details.

-drcdata {unload_mode d}

Annotates the schematic with unload mode data (stores both shift data and unload mode pins). The unload mode is specified by the *d* argument. This data is stored automatically for R21 and R23 violations. You can specify explicit storage using the `set_drc -store_unload_mode_data` command.

The `set_pindata -unload_mode` and `set_pindata -drcdata unload_mode all` commands have the same effect: data for all patterns is shown at the same time (32 values) for each gate. This is verbose and typically unnecessary, unless you are an expert user in unusual circumstances, such as comparing all unload modes at the same time.

-error_data

The pin data associated with the current error condition is displayed.

-fault_data

The current fault codes for the pin stuck 0 (first field) and stuck 1 (second field) are displayed. If there is no fault on a pin, two dashes "--" are displayed. Pins with a nofault attribute appear as "##".

-fault_sim_results <pin_pathname | gate pin_number> <0|1> pattern

This displays the good machine value and faulty machine value for a selected fault and a selected pattern. If they are different, the good machine value is given first followed by the faulty value separated by a slash "/". If there is no difference, only the good machine value is displayed. Selecting `fault_sim_results` is not supported if the `-nobasic_scan` option of the `set_simulation` command has been used.

-full_seq_scoap_data

If you set the pin data display mode to `-full_seq_scoap_data`, the pin data field shows the set of [SCOAP](#) controllability and observability numbers for the pin. For a three-state node, the data display format is:

```
c0.c1.cZ.obs
```

In this case, `c0` is the control-to-0 measure, `c1` is the control-to-1 measure, `cZ` is the control-to-Z measure, and `obs` is the observability measure. For an ordinary (not three-state) node, the data display format is the same, except that there is no control-to-Z field:

```
c0.c1.obs
```

-full_seq_tg_data

This displays the Full-Sequential test generator logic values for the pin. For example, the pin data for a nonscan DFF element could be displayed as "?0x0" before justification and "#010-010" after justification. The question mark (?) means the need for a scan load has not yet been determined, while a pound sign (#) indicates a scan load will occur at this position.

-gatedata

This option displays the X probability for reported and displayed gates associated with the `analyze_compressors -xchain_analysis` command.

-good_sim_results pattern

This option displays the good machine value for the selected Basic scan or Fast-Sequential ATPG pattern. The selected pattern is resimulated and the results are made available for display on the schematic or by use of the [report_primitives](#) command. For Full-Sequential or functional patterns that require sequential simulation, use the `seq_sim_data pindata` option. Also refer to "[Viewing Pattern Data in the Graphical Schematic Viewer](#)" for more details.

-lfsr_shift | -lfsr_load

The simulation values for the linear feedback shift register (LSFR) shift and load are displayed.

-load

The [simulation event values](#) for all time periods of the load_unload test procedure are displayed. The position of the shift period is denoted by "{}" characters.

-master_observe

The simulation event values for all time periods of the master_observe procedure are displayed.

-none

Specifying **-none** turns off the display of pin data in the `report_primitives` command as well as future updates of the graphical schematic viewer.

-pattern <d | all | fast_sequential>

There is a general-purpose pattern simulation buffer that holds 32 patterns. It is used for a number of purposes, including generation of ATPG patterns, holding patterns to illustrate DRC violations as a result of an `analyze_violation` command, holding the successful or unsuccessful results from a `run_justification` command, and other operations.

You can select and display a single pattern from this buffer by providing an integer number between 0 and 31, or all 32 patterns by specifying `all`. If there are any Fast-Sequential patterns in the buffer, the time events of the last Fast-Sequential pattern is selected by specifying `fast_sequential`.

For Basic-Scan patterns, the analysis time of the displayed data can be changed from preclock, to clock, to postclock, plus other combinations. This is controlled with the `-time` option of the `set_primitive_report` command.

Note: Due to the general reuse of this buffer by many commands, the data contents cannot always be what you expect. The best time to view the pin data is immediately after you use the command that saves data to the buffer.

-prpg_shadow_shift | -prpg_shadow_load

The simulation values for the pseudo random pattern generator (PRPG) shadow shift and load are displayed.

-refresh

This option causes the displayed values on any gates in the graphical schematic viewer to be updated with the newly selected pindata values. By default, no update will take place.

-scoap_data

This option displays the [SCOAP](#) controllability and observability numbers. Two sets of numbers are displayed. The first set is for combinational ATPG, shown in the form of "A-B-C". The fields indicate the minimum number of:

A = PIs and scan cells needed to place a 0 at the pin.

B = PIs and scan cells needed to place a 1 at the pin.

C = PIs and scan cells needed to observe the value on the pin.

The second set is for sequential ATPG, shown in the form of "A-B-C-D". These fields indicate the minimum scan depth needed to:

A = place a 0 at the pin.

B = place a 1 at the pin.

C = to propagate the value to an observe point.

D = to sensitize the gate to an observe point.

An asterisk "*" indicates the value exceeds the 254 limit the program can track.

-sdc_case_analysis

Controls the display of the SDC pindata type. The format of the data is: N/M, where N is the case analysis setting from the SDC, and M is the TestMAX ATPG constraint value.

Unconstrained values are printed as X. You can also specify the display of this pindata type directly from the GSV Setup menu. Note that this option is valid only in Tcl mode.

For more information, see "[Specifying Setup Timing Exceptions From an SDC File](#)" in the *TestMAX ATPG User Guide*.

-seq_sim_data

If you have previously performed a `run_simulation -sequential` command with data collection enabled by a `set_simulation -data` option, then the `-seq_sim_data` option presents that data for review in the schematic or by the report primitives command.

-setup_cycles {n1 n2}

Selects a range of cycles for display when `test_setup` procedure patterns are stored using the `-store_setup` option of the [set_drc](#) command. The `n1` argument is the first zero-based `test_setup` procedure cycle; this specification must be greater than or equal to 0. The `n2` argument is the last zero-based `test_setup` procedure cycle; this specification must be greater than or equal to the `n1` specification. The range of cycles available for display is also affected by the `-store_setup_cycles` option of the [set_drc](#) command.

-shadow_observe

Displays the simulation event values for all time periods of the `shadow_observe` procedure.

-shift

Displays the [simulation event values](#) for all time periods of the shift test procedure.

-shift_character c

Specifies the shift character to be used when displaying X values for shift data on the scan path.

-slack

Displays the slack data that is used in small delay defect testing. The first number displayed is the smallest slack for any rising transition through the indicated pin, and the second number is the smallest slack for any falling transition through the pin.

-stability_patterns

The simulated event values for all time periods of the `load_unload`, `shift`, and `capture` cycle are displayed. This display shows the data used to determine the stable state element values. The stability pattern is only stored if there is at least one DLAT/DFF non-X at the end of `test_setup`.

These values are normally not stored unless the `-store_stability` option of the `set_drc` command has been performed before the `run_drc` command.

-test_setup

The [simulation event values](#) for all time periods of the `test_setup` procedure are displayed. By default, the ATPG tool retains only the last simulated value of the `test_setup` procedure. To make full use of this displayed data, you need to use the `set_drc -store_setup` command and rerun DRC checks.

-tie_data

The simulated value that results from tied circuitry is displayed.

Allowed Command Modes

All

Description

This command controls the type of information displayed by the `report_primitives` command or displayed within the graphical schematic viewer.

For additional discussion and examples, see [Understanding Pin Data](#).

Examples

```
TEST> set_pindata -clock_cone CLK
TEST> report_primitives /e006/reg2/r
e006/reg2/r (104) DLAT (N_LATCH)
  !SB I () (TIE_1)
  !RB I (N) 18-
  CK I (C) 13-
  D I (E) 103-e006/reg2/lat1/Q
  Q O (E) 53-
scan_behavior: MASTER(LS/-) chain=c1 cell_id=2 invert_data=NN obs=noproc
```

```
TEST> set_pindata -clock_off
TEST> report_primitives /e006/reg2/r
e006/reg2/r (104) DLAT (N_LATCH)
  !SB I () (TIE_1)
  !RB I (1) 18-
  CK I (0) 13-
  D I (X) 103-e006/reg2/lat1/Q
  Q O (X) 53-
scan_behavior: MASTER(LS/-) chain=c1 cell_id=2 invert_data=NN obs=noproc
```

```
TEST> set_pindata -constrain_data
TEST> report_primitives e006/buried3
e006/buried3 (80) NAND (NAN2)
  B I (X/-,0/B) 79-e006/buried2/QB
  A I (X/-,1/-) 109-e006/buried1/Q
  X O (X/-,1/B) 111-e006/buried4/RB
TEST> report_primitives /e006/buried4
e006/buried4 (111) DFF (DFFRP)
  --- I () (TIE_0)
  !RB I (X/-,1/B) 80-e006/buried3/X
  CK I () (TIE_0)
  D I () (TIE_0)
  --- O (X/-,0/-) 81-
non_scan_behavior: C0/-
```

```
TEST> set_pindata -load
TEST> report_primitives D2
D2 (8) PIO (_PIO)
  --- B (ZZZ11Z) 60-
```

```

TEST> set_pindata -scoap_data
TEST> report_primitives /e006/reg2/r
e006/reg2/r (104) DLAT (N_LATCH)
  !SB I () (TIE_1)
  !RB I (1-1-0) 18-
  CK I (1-1-0) 13-
  D I (4-4-1) 103-e006/reg2/lat1/Q
  Q O (1-1-0) 53-
scan_behavior: MASTER(LS/-) chain=c1 cell_id=2 invert_data=NN obs=noproc

```

```

TEST> set_pindata -shift
TEST> report_primitives /e006/reg2/r
e006/reg2/r (104) DLAT (N_LATCH)
  !SB I () (/TIE_1)
  !RB I (111) 18-
  CK I (010) 13-
  D I (XXX) 103-e006/reg2/lat1/Q
  Q O (XXX) 53-
scan_behavior: MASTER(LS/-) chain=c1 cell_id=2 invert_data=NN obs=noproc

```

```

TEST> set_pindata -stability_patterns
TEST> report_primitives e006/reg0/r
e006/reg0/r (100) DLAT (N_LATCH)
  !SB I () (TIE_1)
  !RB I (111111/1XX) 16-
  CK I (000000/0XX) 11-
  D I (XXXXXX/XXX) 99-e006/reg0/lat1/Q
  Q O (XXXXXX/XXX) 29-
scan_behavior: MASTER(LS/-) chain=c1 cell_id=4 invert_data=NN obs=noproc

```

```

TEST> set_pindata -test_setup
TEST> report_primitives /e006/buried3
e006/buried3 (80) NAND (NAN2)
  B I (1111000) 79-e006/buried2/QB
  A I (0011111) 109-e006/buried1/Q
  X O (1100111) 111-e006/buried4/RB
TEST> report_primitives /e006/buried4
e006/buried4 (111) DFF (DFFRP)
  --- I () (TIE_0)
  !RB I (1100111) 80-/e006/buried3/X
  CK I () (TIE_0)
  D I () (TIE_0)
  --- O (XX00000) 81-
non_scan_behavior: C0/-

```

```

TEST> set_pindata -tie_data
TEST> report_primitives /e006/adder
e006/adder (32) XOR (INC4)
  --- I (X) 30-
  --- I (1) 28-

```



```
S0 O (X) 35-e006/mux0/B
```

See Also[Fault Classes](#)[report_primitives](#)[report_settings](#)[set_drc](#)[set_primitive_report](#)[set_simulation](#)[SCOAP](#)[Understanding Pin Data](#)[Understanding the report_primitives Output](#)

set_primitive_report

Overview

This command controls the type of information displayed by the `report_primitives` command.

Syntax

```
set_primitive_report
  [-interval d]
  [-max_fanout d]
  [-time <clock | preclock | postclock | lete | all>]
  [-verbose | -noverbose]
```

Arguments

-interval d

Specifies an interval at which to place a blank space for some selections of pin data (includes test procedure simulated data and pattern data). This is intended to improve the readability of the pin data. A selection of 0 indicates no interval insertion. The default is 0.

-max_fanout d

Specifies the maximum number of fanout connections that are displayed. The default is 2.

-verbose | -noverbose

Specifies whether to display all learned data for a gate. The default is -noverbose.

-time <clock | preclock | postclock | lete | all>

Specifies the time segment of the capture clock simulation to display when viewing Basic-Scan patterns. This setting has no effect when viewing Fast-Sequential or Full-Sequential patterns. You should use the `analyze_simulation_data` command to view Fast-Sequential or Full-Sequential data.

The `clock` argument displays the time segment with all clocks active and all state elements at their new captured state.

The `preclock` argument displays the time segment with all clocks off and all state elements at their previous state.

The `postclock` argument displays the time segment with all clocks off and the state elements at their new captured state.

The `lete` argument displays the time segment with the capture clock on and [LE/LS](#) state elements at their new value used by a trailing-edge state element.

The `all` argument (the default) causes display of clock, preclock, and postclock data as a 3-character field; any unavailable information is displayed as a question mark character "?". A question mark is used because the additional time periods are only simulated when required.

Allowed Command Modes

All

Description

This command controls the type of information displayed by the `report_primitives` command.

Examples

```
TEST> set_primitive_report -max_fanout 5 -verbose
```

See Also

[analyze_simulation_data](#)

[report_settings](#)

[Understanding Report Primitives](#)

Table Column Outside Table:

set_random_patterns

Overview

Use this command to set the parameters that control the application of random patterns.

Syntax

```
set_random_patterns
  [-clock <pinname | -clock_none>]
  [-length d]
  [-observe <master | slave | shadow>]
```

Arguments

-clock <pinname | -clock_none>

Specifies the name of the capture clock to be used for random patterns. The default is `-clock_none` (no capture clock).

-length d

Specifies the number of random patterns to apply. The default is 1024 patterns.

-observe <master | slave | shadow>

Specifies the type of scan cell observe points to use for random patterns. The default is `master`. In addition to this choice, every pattern is assumed to also measure primary outputs.

Allowed Command Modes

DRC, Test

Description

Use this command to set the parameters that control the application of random patterns. You can display the current settings with the `report_settings` command.

Examples

```
DRC> set_random_patterns -clock_none -length 3000 -observe master
```

See Also

[report_settings](#)
[set_patterns](#)

set_rules

Overview

Use this command to set the severity of a rule violation.

Syntax

```
set_rules rule_id
[-autofix | -noautofix]
[error | warning | ignore]
[-mask | -nomask]
[-max_fails number]
[-min_area value]
[-reset]
[-verbose | -noverbose]
```

Arguments

rule_id

Specifies the rule for which the severity is to be changed. Rules are identified by an alphabetic character followed by a number, such as [B1](#), [C6](#), [S22](#), and so on. [Categories of rules](#) are: B, C, L, N, P, S, T, V, X, and Z. Messages for information, warnings, or errors start with the letter M and are not rules, and are not affected by the `set_rules` command.

-autofix | -noautofix

Supported for the [V33 rule](#) only, this option automatically modifies output to correct the violating condition. When this option is set, rule violations are no longer be reported. You should review the output modifications when using this option.

error | warning | ignore

Specifies the new severity level of the rule. If the rule violation is a fatal error by default, you cannot set it to a lower severity level. If you do not specify a severity level, the existing setting is retained.

-mask | -nomask

For selected DRC rules, the `-mask` option enables masking of either the control or observe functionality of the affected sequential elements, as appropriate for the specific rule. This masking improves the ability to create ATPG patterns that pass in simulation. However, the tradeoff is usually lowers test coverage. Not all rules support this option. The current list of DRC rules that accept the `-mask` option are: [C1](#), [C5-C14](#), [C22](#), [C25](#), [S18](#), and [S29](#).

You can use the `-mask` option to compensate for DRC violations by setting the rule severity to warning. This causes the ATPG algorithm to mask off the observe value to ensure that any ATPG pattern generated will pass simulation. The tradeoff is a reduction in test coverage.

-max_fails number

Sets the maximum number of failures reported for all DRC rules. The default is 1000000 (one million) and can be adjusted individually for each rule by specifying the rule ID (*rule_*

id). If the maximum number of rule violations is reached, a message is printed to the log at the point where the rule is checked, as shown in the following example:

```
Rule R45 has reached its max_fails limit.
```

-min_area

Defines the minimum area of a cell that will trigger a Y20 violation (this option is only supported for the Y20 violation). You can find the area of the cell in the "Size" category defined in the cell's LEF file.

-reset

Restores all rule severities and mask or autofix settings back to the original documented defaults. This option must be specified as the only option to a `set_rules` command; it cannot be used with any options as it overrides any other rule settings with this command.

-verbose | -noverbose

The `-verbose` option prints an expanded list of [S14](#), [S22](#), or [C39](#) violations. In the case of [S14](#), the `-noverbose` option (the default) prints only the first [S14](#) violation that is encountered. If the `-verbose` option is specified with [S14](#), then all [S14](#) violations are printed. In the case of [S22](#), the `-verbose` option prints all instances and clocks associated with the violating transition. If checking is still not sufficiently improved, use the [report_lockup_latches](#) command instead. For [C39](#), by default, only one violation is printed per clock. If the `-verbose` option is specified with [C39](#), then violations are reported by scan cell, which usually results in a significant increase in the number of reported violations. Note that the `-verbose` option applies only to [S14](#), [S22](#), and [C39](#) violations.

Allowed Command Modes

All

Description

Use this command to set the severity of a rule violation. You can use the `report_rules` command to report the rule settings.

Examples

```
DRC> report_rules n2
rule severity #fails description
-----
N2    warning  0      unsupported entry
DRC> set_rules N2 ignore
# observe changes for N2
DRC> report_rules n2
rule severity #fails description
-----
N2    ignore   0      unsupported entry
```

```
DRC> set_rules c39 warning
DRC> ...
DRC> ... report_violations c39
```

```
Warning: Nonlogical clock dft_clock connects to 4 scancells (u1/ci_reg_3_
(827)). (C39-1)
Warning: Nonlogical clock clkref connects to 19 scancells (u8/LOCKUP
(812)). (C39-2)
```

```
DRC> set_rules c39 warning -verbose
DRC> ...
DRC> run_drc
DRC> ...
DRC> report_violations C39

Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_3_
(827). (C39-1)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_2_
(828). (C39-2)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_0_
(829). (C39-3)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_1_
(836). (C39-4)
Warning: Nonlogical clock clkref connects to scancell u8/LOCKUP (812).
(C39-5)
Warning: Nonlogical clock clkref connects to scancell u8/ai_reg_3_ (873).
(C39-6)
.
.
.
```

See Also

[report_rules](#)
[report_violations](#)
[Rule List](#)

set_scan_ability

Overview

Use this command to explore the test coverage and testability effects of converting selected nonscan elements into scan elements.

Syntax

```
set_scan_ability
  <on | off> <nonscan_cell_gate | -dff | -dlat | -all>
```

Arguments

<on | off>

Specifies whether to consider the named nonscan cells scan or nonscan. Use `on` to add nonscan cells to the list or `off` to remove them. The instances in the scan ability list are treated as if they belong to scan chains and are controlled and observed accordingly. Added scan cells are placed in a non-existing scan chain named `"_sc1"`.

nonscan_cell_gate

Identifies a nonscan cell gate to be added to or removed from the scan ability list. It can be defined by [primitive ID](#), [instance name](#), or [pin pathname](#).

-dff

Adds or removes all DFF type nonscan cells.

-dlat

Adds or removes all DLAT type nonscan cells.

-all

Adds or removes all nonscan cells (DFF and DLAT).

Allowed Command Modes

DRC

Description

Use this command to explore the test coverage and testability effects of converting selected nonscan elements into scan elements. You must set up the clocks before issuing this command.

Note the following:

You cannot write patterns to a file if any of the patterns were generated with instances on the scan ability list. Any such patterns would be invalid for the current design.

The `set_scan_ability` command is not compatible with any type of scan compression. DRC will fail if the STIL procedure file contains a CompressorStructures block.

Examples

```
DRC> report_scan_ability
gate_id type instance_name
-----
```


There were 0 DLATs/DFFs with scan ability set to ON.

```
DRC> set_scan_ability on core/host/d_l2odd_reg
DRC> set_scan_ability on core/host/status1
DRC> report_scan_ability
gate_id type instance_name
-----
97299 DFF core/host/d_l2odd_reg
97302 DFF core/host/status1
```

See Also

[report_scan_ability](#)

Table Column Outside Table:

set_sdc

Overview

This command controls how TestMAX ATPG interprets an SDC file used for specifying setup timing exceptions.

Syntax

```
set_sdc
  [-case_paths | -nocase_paths]
  [-delete_common_clock_exceptions |
  -nodelete_common_clock_exceptions]
  [-disable_paths | -nodisable_paths]
  [-environment < sdc_case_analysis | tmax_drc | none >]
  [-false_paths | -nofalse_paths]
  [-hold | -nohold]
  [-instance instance_path]
  [-mark_gui_gates | -nomark_gui_gates]
  [-multicycle_paths | -nomulticycle_paths]
  [-setup | -nosetup]
  [-show_all_matches | -noshow_all_matches]
  [-verbose | -noverbose]
```

Arguments

-case_paths | -nocase_paths

Creates -through exceptions for paths blocked by the `set_case_analysis` settings in the SDC file. The `set_case_analysis` is still used for clock tracing regardless of how this switch is set. The `-nocase_paths` option restores the default.

-delete_common_clock_exceptions | -nodelete_common_clock_exceptions

Removes the timing exceptions between the clocks that are defined with the same source objects. For limitations in interpreting these clocks, see [How TestMAX ATPG Interprets SDC File Commands](#). All `set_clock_group`, `set_false_path`, or `set_multicycle_path` SDC commands are removed, but only when the two points specified are clock objects. The default is `-nodelete_common_clock_exceptions`.

-disable_paths | -nodisable_paths

The default (`-disable_paths`) uses `set_disable_timing` commands in the SDC file for exceptions. You can set the `-nodisable_paths` option to make sure these commands are not used for exceptions.

-environment < sdc_case_analysis | tmax_drc | none >

Causes clocks to be traced without reference to logic values either defined in the SDC `set_case_analysis` commands or found by the TestMAX ATPG `run_drc` command. This causes the clock to propagate to the maximum possible number of registers.

The default is `none`. Use the `sdc_case_analysis` argument to trace clocks based on the SDC `set_case_analysis` commands. This is useful for SDC constraints created for the functional mode of a design in which the test mode clocks are different.

If you specify the `set_sdc -environment sdc_case_analysis` command, but the SDC file read by the [read_sdc](#) command does not contain any `set_case_analysis` commands, then the `-environment` switch is reset to `none` during the [run_drc](#) process. The only observable effect of this change occurs if you go back to DRC mode and read additional SDC files in the same TestMAX ATPG session.

You should use the `tmax_drc` argument to trace clocks based on the TestMAX ATPG `run_drc` results. This is useful for SDC created specifically for test mode, which will give the most accurate results.

-false_paths | -nofalse_paths

The default (`-false_paths`) creates timing exceptions for a false path. Use the `-no_false_paths` option to make sure timing exceptions are not created for a false path.

-hold | -nohold

The `-hold` option (the default) enables you to use the SDC file to specify hold timing exceptions. Transitions that violate a hold time requirement during a clock pulse are masked during good machine simulation. If you require different behavior for a particular SDC file, you can change this setting before specifying the [read_sdc](#) command again. If TestMAX ATPG reads the SDC file when the `-hold` option is enabled, the DRC process also uses this setting.

-instance *instance_path*

Prepends the specified argument (*instance_path*) to all instances in the SDC file. This option should be used in case the SDC file was written for a submodule of the top-level design. For example:

```
# First SDC is for instance "a" inside
top-level instance "top"
set_sdc -instance /top/a
read_sdc my.sdc

# Second SDC is for the top-level design
set_sdc -instance "/"
read_sdc second.sdc
```

-mark_gui_gates | -nomark_gui_gates

Causes cells and primitives that are affected by timing exceptions to be displayed in a distinctive color in graphical schematic viewer (GSV). Cells that begin or end a timing exception are displayed in green, and cells within a timing exception path are displayed in orange. This marking has no effect on ATPG or simulation, but might take substantial time during the [run_drc](#) process for multicycle paths. Patterns generated without the `-mark_gui_gates` option setting can be debugged by saving them and then reading them into a TestMAX ATPG session that uses this option. The GUI marking is not saved in image files. The default is `-nomark_gui_gates`.

-multicycle_paths | -nomulticycle_paths

The default (`-multicycle_paths`) uses `set_multicycle_path` commands in the SDC file for exceptions. The `-nomulticycle_paths` option prevents creating timing exceptions for multicycle paths.

-setup | -nosetup

The `-setup` option (the default) enables you to use the SDC file to specify setup exceptions. Transitions that violate a setup requirement during a clock pulse are masked during good machine simulation. If you require different behavior for a particular SDC file, you can change this setting before specifying the [read_sdc](#) command again. If TestMAX ATPG reads the SDC file when the `-setup` option is enabled, the DRC process also uses this setting.

`-show_all_matches | -noshow_all_matches`

Specifies that for each SDC command, all TestMAX ATPG database objects that match the specified points are reported. This results in an extremely verbose listing. The `-noshow_all_matches` option (the default) makes sure that these database objects are reported.

`-verbose | -noverbose`

Prints messages for SDC commands and options not supported by TestMAX ATPG. The `-noverbose` option (the default) restores the default messaging. The `-verbose` option does not enable printing of all messages in all situations. To print all messages, you need to redirect the output using the redirect command: `read_sdc >...`

Allowed Command Modes

DRC

Description

This command controls how TestMAX ATPG interprets an SDC file used for specifying setup timing exceptions. You must specify it before the `read_sdc` command. You must be in DRC mode to use this command (after you successfully run the `run_build` command, but before running `run_drc`).

All `set_sdc` command settings are cumulative; you can run this command multiple times to prepare for a `read_sdc` command. If multiple `read_sdc` commands are required, you can also specify a `set_sdc` command before each `read_sdc` command to specify its verbosity, instance, and so forth.

Note that the `set_sdc` command is valid only Tcl mode. For more information on specifying timing exceptions, see [Specifying Setup Timing Exceptions From an SDC File](#).

For more information on the SDC file, see [How TestMAX ATPG Interprets SDC File Commands](#).

See Also

[How TestMAX ATPG Processes Setup and Hold Violations](#)

`read_sdc`

`report_sdc`

set_simulation

Overview

Use this command to set the simulation-related control parameters that are in effect for the `run_simulation` and `run_fault_sim` commands.

Syntax

```
set_simulation
  [-analyze_x_effects | -noanalyze_x_effects]
  [-analyze_x_sources | -noanalyze_x_sources]
```

```

[-basic_scan | -nobasic_scan]
[-bidi_fill | -strong_bidi_fill | -weak_bidi_fill | -nobidi_fill]
[-check_padding_patterns | -nocheck_padding_patterns]
[-data {first_pattern last_pattern}]
[-dominance_use <off | on | sr_persist>]
[-interval_pattern pattern_name]
[-num_processes number | max]
[-num_threads number]
[-oscillation {num_passes num_additional_fault_passes}]
[-pipeline_cells | -nopipeline_cells]
[-progress_message number]
[-shift_cycles number]
[-store_memory_contents < pattern_num | last> |
-nostore_memory_contents]
[-timing_exceptions_for_stuck_at |
-notiming_exceptions_for_stuck_at]
[-truncate file_name | -notruncate]
[-two_clock_resim | -notwo_clock_resim]
[-use_timing_exceptions | -nouse_timing_exceptions]
[-verbose | -noverbose]
[-words_per_pass d]
[-xclock_gives_xout | -noxclock_gives_xout]
[-xfill_out_of_range_write | -noxfill_out_of_range_write]

```

Arguments

-analyze_x_effects | -noanalyze_x_effects]

This option turns on and off the analysis of X effects performed during a TestMAX ATPG simulation. It is supported only for the simulation of basic scan and fast-sequential patterns. This analysis includes a printed report, and is performed during both the [run simulation](#) process and the simulation that occurs as part of the test generation process when the [run_atpg](#) command is used.

When the `-analyze_x_effects` option is specified, each pattern is evaluated individually. TestMAX ATPG prints an analysis of the simulation of patterns that have at least one scan cell with a final simulated value of X, as shown in the following example:

```

X-effect analysis results: pattern=0, #scancells_at_X=115313 (42.27%)
  shift results for non-Xchains: 0X=1 6X=1 9X >9X=202, maxX=1244
  non-Xchain results: 1351=118 804=111 743=110 1128=110 1048=107
  1255=107 1257=107 1338=107 916=106 1051=106

```

This analysis includes the following information:

- A summary message that indicates the pattern number, number of scan cells at X for the pattern, and the percent of scan cells at X.
- Shift results for non-X-chains. This message provides an analysis of the distribution of simultaneous Xs shifted out of non-X-chains. The tool analyzes the captured values across the chains at each [shift position](#), counts the number of X values in each shift position row, then prints the distribution of those counts: zero Xs, one X, two Xs, three Xs, four Xs, five Xs, six Xs, seven Xs, eight Xs, nine Xs, and more than nine Xs. The maximum number of Xs for any shift is also provided.
- Non-X-chain results. This message includes an ordered list of the top 10 chains with X counts that have the most Xs for the pattern.

At the end of the simulation, a summary report is printed, as shown in the following example:

```
Final X-effect analysis results: #scancells_at_X=275038 (3.15%)
  shift results for non-Xchains: 0X=187 1X=162 2X=343 3X=407
  4X=470 5X=498 6X=702 7X=753 8X=730 9X=569 >9X=1739 maxX=2241
  non-Xchain results: 652=3133 1235=2209 1236=2128 1237=2127
  802=2005 1362=1664 803=1177 653=1145 638=1122 656=926
```

This summary report includes the following information:

A summary message that indicates the total number of scan cells at X and the percentage of scan cells at X.

- Shift results for non-X chains. This message includes a list of the number of [shift positions](#) for the entire set of patterns: zero Xs, one X, two Xs, three Xs, four Xs, five Xs, six Xs, seven Xs, eight Xs, nine Xs, and more than nine Xs. Note that this count set only considers non-X chains, and does not print counts less than one. The maximum number of Xs for any shift is also provided.
- Non-X chain results. This message includes an ordered list of the top 10 chains with X counts that have the most Xs for the entire set of patterns.

-analyze_x_sources | -noanalyze_x_sources

This option turns on and off the analysis of X sources performed during the [run simulation](#) command process. This is a non-default option, and is supported for simulation of basic scan and fast-sequential patterns. When the `-analyze_x_sources` option is specified, each pattern is individually evaluated and the following messages are printed if any scan cells have an X value:

- A summary message indicating the pattern number, number of scan cells at X, and percent of scan cells at X.
- For a given X source type that explains at least 1 scan cell, a message is given indicating number of scan cells at X due to this type, a description of the type, and the gate ID number of the first scan cell at X due to this type. If the scan cell captured an X through its data input, then the gate ID of source of captured X is given.

The following X source types are considered:

- User capture mask
- Cell constraint to X
- Capture masking due to any of the following rule failures: S29, S18, C1, C5, C6, C7, C8, C9, C10, C11, C12, C13, C14, C22, C25.
- Clock disturb
- Multiple clocks on
- Clock at X
- Excess simulation passes
- SDC timing exceptions
- Capture X from any of the following: nonscan cell, transparent latch, scan master, scan slave, scan dslave, scan shadow, RAM, ROM, TIE, global TIE, TIEZ, global TIEZ, floating BUS, contending BUS, from tri-state driver, WIRE, PI, or feedback path
- Unknown capture X source
- Unknown simulation X source

-basic_scan | -nobasic_scan

This option specifies how Basic-Scan patterns is simulated during the good machine simulation. The default is `-basic_scan`, which simulates Basic-Scan ATPG patterns using

the combinational simulation algorithm. Using `-nobasic_scan` forces simulation of all Basic-Scan patterns using the sequential scan algorithm. This might be useful if a `run_simulation` command reports simulation mismatches.

`-bidi_fill | -strong_bidi_fill | -weak_bidi_fill | -nobidi_fill`

These options are useful only when the pattern source is internal. All of the bidi-fill options cause the ATPG tool to create Basic-Scan, Fast-Sequential, and Full-Sequential capture patterns that drive each bidirectional port to a high or low state when there is no internal driver enabled. The ports that are candidates for bidi-fill are those that have no internal driver enabled and that also have not had an input value specified by the ATPG process for the current pattern. This fill process can assist in the avoidance of floating inputs (Z state) on the bidis, but it is not guaranteed to be successful. The occurrence of a floating bidirectional port produces internal Z states that lead to the propagation of Xs. The differences among the various bidi-fill options depend upon how and on which types of bidirectional ports they perform fill.

The `-bidi_fill` option performs random fill on all candidate bidirectional ports. In the case of bidis with a weak pullup or pulldown device, this random fill might cause weak contention with the pullup or pulldown.

The `-strong_bidi_fill` option is similar to the `-bidi_fill` option, however `-strong_bidi_fill` does not perform fill on any bidirectional ports that have a weak pullup or pulldown. In those cases, the weak driver is left to provide the input value for the bidirectional port.

The `-weak_bidi_fill` option is similar to the `-strong_bidi_fill` option, however `-weak_bidi_fill` also puts a fill value on candidate bidirectional ports with a weak pullup or pulldown. The value placed on these ports is the same as that provided by the weak driver; that is, a 1 in the case of a pullup, and a 0 in the case of a pulldown. It can be useful to have this fill value provided, even though it is the same as the value that the weak driver would have provided; this is because the RC time constant of the pullup or pulldown might be too great to drive values reliably at the desired speed of the ATPG patterns.

The `-nobidi_fill` option performs no fill on any bidirectional ports, which would leave all the candidate ports for bidi-fill floating (Z value) or controlled only by a weak pullup or pulldown.

Note: The use of any form of bidi fill does not affect the patterns defined by the `load_unload` procedure. You should manually initialize bidirectional ports to appropriate values in the `load_unload` procedure. Use the bidi fill options for test generation only, and do not change the options between test generation and simulation of the generated patterns

`-check_padding_patterns | -nocheck_padding_patterns`

When the `-check_padding_patterns` option is specified, checking for ATPG and cell constraints is performed during the `run_simulation` command and `run_atpg` command processes. During the `run_atpg` command process, all failing patterns are eliminated and the unload values of the previous pattern become X values. Fault simulation is then performed to give the correct coverage numbers.

This option is supported only in TestMAX ATPG. The default is the `-nocheck_padding_patterns` option.

If you specify the `-check_padding_patterns` option and are using expert-level messaging, the following messages appear (these are examples):

Fault simulation will be repeated for current patterns after 174 padding patterns were eliminated due to constraint violations.

Pattern checking completed: #patterns_eliminated=174, test coverage was changed from 76.70% to 76.30%

-data {*first_pattern last_pattern*}

For sequential simulation, use this option to specify that the simulation values are to be stored and the range of patterns to be stored. The stored values can be displayed later by using the `-seq_sim_data` option of `set_pindata` command in conjunction with the [report_primitives](#) command. The default is not to store any patterns. Pattern storage can be turned off by using `-data {0 -1}`.

-dominance_use <off | on | sr_persist>

Controls the application of dominance information during simulation. The default is off.

Dominance information is extracted from ATPG models for set/reset/clock pins during netlist interpretation (for more information, see the description of the `-dominance_detection` option of the [set_netlist](#) command). When [set_netlist](#) `-dominance_detection` is set to `on` or `boolean`, the dominance information can be applied during simulation by specifying `set_simulation -dominance_use` with `on` or `sr_persist`.

When `-dominance_use on` is applied, known states on the set or reset lines (which also depends on dominance between these signals) will override unknown clock states and result in known output values. This reduces pessimistic behavior in the presence of Xs on clock lines. It is critical that you accept the resulting known values. Setting these options without validating the resulting behavior might generate patterns that fail in external simulation or at the tester due to the reduced pessimism.

When `-dominance_use sr_persist` is specified, the previous behavior is applied with the following extension: If the set or reset line is active before a clock, and the clock goes active at the same time as the set or reset goes inactive, the set or reset condition will persist and override the clock event. This condition might be necessary if the set or reset states continue past the current clock event in the actual design or simulation context with timing present, and result in a device response that reflects the set or reset and not the clock operation. This option also requires that you specify the [set_drc](#) `-allow_unstable_set_resets` command to have an effect on the simulation.

-interval_pattern *pattern_name*

Specifies the name of the interval pattern.

-num_processes *number* | max

Specifies the number of slave processes to be used for multicore simulation. The *number* specification refers to the number of slave processes that is used for simulation. If `max` is specified, then TestMAX ATPG computes the maximum number of processes available in the host based on number of CPUs. If TestMAX ATPG detects that the host has only one CPU, then single-process simulation is performed instead of multicore simulation with only one slave.

Note the following:

- You should not specify more processes than the number of CPUs available on the host.
- You should consider whether there is other CPU-intensive processes running simultaneously on the host when specifying the number of processes. If too many processes are specified, performance will degrade and might be worse than single-process simulation. On some platforms, TestMAX ATPG cannot compute the number of CPUs available and will issue an error if `max` is specified.

- This option has no effect when TestMAX ATPG is enabled. For more information on TestMAX ATPG, see "Running TestMAX ATPG."

-num_threads *number*

Specifies the number of threads to use for multithreading when running TestMAX ATPG. This option cannot be used unless you have installed TestMAX ATPG. By default, the number of threads is set to 8. This option is used with the `-num_threads` option of the [set_atpg](#) command. Both options must be set to the same value, as shown in the following example:

```
set_simulation -num_threads 12
set_atpg -num_threads 12
```

If you set this option to 0, TestMAX ATPG is disabled and TestMAX ATPG is used.

You cannot read image files into TestMAX ATPG after changing the value of the `-num_threads` option. For more information on this option and TestMAX ATPG, see "Running TestMAX ATPG."

-oscillation {*num_passes num_additional_fault_passes*}

This specifies the number of simulation passes to allow while waiting for logic simulation values to stabilize before declaring an oscillation condition. This is meaningful only for sequential simulation. The first value is the limit used for the good machine simulation (default 10) and the second value is the additional passes allowed for the faulty machine to stabilize (default 2).

-pipeline_cells | -nopipeline_cells

When `-pipeline_cells` is specified (the default), input pipeline DFFs are simulated with being in shift mode or capable of holding state during capture. When `-nopipeline_cells` is specified, input pipeline DFFs are considered X for such patterns.

Note: Use of this switch in designs with input pipelines and scan enable not constrained to its system value can result in parallel testbench failing VCS because input pipe DFFs remain X in the parallel simulation. Parallel testbench with serial input_pipe_depth shifts should pass; or, use `set_simulation -nopipeline_cells`.

-progress_message *number*

This option prints a progress message every specified *number* of simulation passes. A simulation pass is 32 basic scan patterns, or from 1 to 10 (but usually 3) fast-sequential patterns. For more information, see ["Understanding the run_simulation Output."](#)

-shift_cycles *number*

This switch enables the simulation of loadable nonscan cells for the last specified *number* of shift cycles. The default is *number* = 0, which means no simulation of loading nonscan cells. You can set the *number* to specify the depth of the loading to be simulated. The maximum value of the specified number is 10, but also no more than the length of the longest scan chain (plus 1 for compression).

The `-shift_cycles` option cannot be set to any value other than 0 when you run the `analyze_compressors` command.

For complete information on using the `-shift_cycles` option and related commands, see the topic ["Using Loadable Nonscan Cells in TestMAX ATPG."](#)

-store_memory_contents < *pattern_num* | last> | -nostore_memory_contents

This option specifies that during sequential simulation, the data contents of all RAMs are to be stored for a specific pattern. The pattern can be selected by a specific pattern number or the keyword last, in which case the last pattern simulated determines the RAM contents. The data contents can then be reported with the `report_memory` command. The default is `-nostore_memory_contents`.

-timing_exceptions_for_stuck_at | -notiming_exceptions_for_stuck_at

Controls timing exceptions simulation for stuck-at faults. The default is `-notiming_exceptions_for_stuck_at`. For more information, see the "Specifying Setup Timing Exceptions From an SDC File" section in the *TestMAX ATPG User Guide*.

-truncate file_name | -notruncate

The `-truncate` option specifies the name of the file containing observe point definitions used for the good simulation of pin data. Each entry in the file contains one observe point that identifies a scan cell or primary output. A line that defines a scan cell must contain two entries: the first entry identifies the scan chain and the second entry identifies the position of the scan cell in the scan chain. The chain position is zero-based, which means a scan cell specified at the 0 position is the closest to the scanout pin. A line that defines a primary output must use only one entry to identify the port name.

The `-truncate` option is used only in TEST command mode. This option creates a list of the gates that must be simulated to ensure that an observe point has the correct value for single-cycle and two-cycle simulation.

A message displays the number of gates and sequential cells that are in the simulation list. The truncated simulation is used for both shift simulation and capture-cycle simulation when there are fewer than two cycles. The truncated simulation is activated only when the `-good_sim_results pattern_number` option of the [set_pindata](#) command is specified. The `-notruncate` option removes truncation for the good simulation of pin data.

-two_clock_resim | -notwo_clock_resim

The `-two_clock_resim` option causes the [run_simulation](#) and [run_fault_sim](#) commands to simulate certain single-load, two-cycle patterns exactly as they were simulated during the two-clock ATPG process using the [set_delay -two_clock_transition_optimization](#) command. A total of 32 patterns are used for each simulation pass. For more information on this type of simulation, see the description of the `-two_clock_transition_optimization` option in the [set_delay](#) command topic.

Note that only single-load, two-cycle patterns that occur immediately after the basic-scan patterns (if any) with a minimum of 10 patterns are considered for this type of simulation. You can expect a runtime performance improvement when a significant number of single-load, two-cycle patterns are simulated.

The default is the `-notwo_clock_resim` option, which uses the fast-sequential simulation mode for multi cycle patterns and all other patterns that do not meet the required criteria. By default, many two-clock patterns are defined as three-cycle operations due to the presence of an extra initial force-PI operation. This additional operation affects the performance improvement gained by using the `-two_clock_resim` option. You can specify the `-noextra_force` option of the [set_delay](#) command to create tests that are applicable in this context.

-use_timing_exceptions | -nouse_timing_exceptions

This switch globally enables or disables the use of timing exceptions during simulation. The default is the `-use_timing_exceptions` option. For example, if you specify the `set_simulation -timing_exceptions_for_stuck_at` command and the fault model is stuck, the `-nouse_timing_exceptions` option overrides the `-timing_exceptions_for_stuck_at` command.

ATPG Untestable (AU) analysis is disabled by default, but is honored when enabled by the `set_atpg -timing_exceptions_au_analysis` command.

-verbose | -noverbose

Specifies whether to display the good machine vs. faulty machine differences and including the gate id and pin number of faults detected in a pattern during fault simulation performed by the `run_fault_sim -sequential` command. For example:

```
17 test_so (exp=0, got=1) fault: 20 pin 2 stuck 1
```

Where:

17 is the pattern

test_so is the output measured

(exp=0, got=1) is the good machine vs faulty machine results

fault: 20 pin 2 stuck 1 is stuck at 1 fault on gate id 20 pin 2.

The default is `-noverbose`.

Note: This option is not yet implemented for the Full-Sequential pattern generator.

-words_per_pass d

Specifies the number of words per pass to be used during fault simulation. The maximum and default is 32 words. Each word supports tracking 32 faults for a default of $32 \times 32 = 1024$ faults processed per pass during fault simulation. In a design with 50,000 faults to grade, it will take $50,000 / 1,024$ passes of fault simulation to process all of the faults at the default settings.

-xclock_gives_xout | -noxclock_gives_xout

This option specifies the logic simulation behavior for all DFF and DLAT primitives when the clock has a value of X. Selection of `-xclock_gives_xout` produces a Q output of X whenever the clock is at X, regardless of the current or previous values on D and Q. A selection of `-noxclock_gives_xout`, which is the default, reviews the current and previous values on D and Q when the clock has a value of X. When current D equals previous Q, then the Q output remains known, otherwise it is set to X.

-xfill_out_of_range_write | -noxfill_out_of_range_write

These options apply to a write operation in an illegal memory address -- including active values for all control pins. By default, X will appear at the memory outputs.

When `-xfill_out_of_range_write` is specified, the memory content will also be set to X. This affects only simulation. If you specify `set_contention abus`, the previous condition is treated as contention during simulation and test pattern is rejected. This option requires ATPG support similar to this for bus contention. For example, during ATPG, the address at each memory bus needs to be set so that the illegal address cannot be generated when all "don't care" bits are randomly set to either 0 or 1.

Allowed Command Modes

All

Description

Use this command to set the simulation-related control parameters that are in effect for the `run_simulation` and `run_fault_sim` commands. You can display the current settings with the `report_settings` command.

Examples

```
DRC> set_simulation -measure pat -oscillation {20 2} -verbose
TEST> set_simulation -store_memory last
```

See Also

[report_memory](#)
[report_primitives](#)
[report_settings](#)
[run_fault_simulation](#)
[run_simulation](#)
[set_contention](#)

set_toggle_weights

Overview

This command is used in the power aware ATPG flow to specify separate toggle weights for both shift and capture cycles.

Syntax

```
set_toggle_weights  
  [{list_of_sequential_elements}]  
  [-capture]  
  [-delete]  
  [-shift]  
  [-weight number]
```

Arguments

list_of_sequential_elements

Specifies a list of one or more sequential elements for which the specified weight is to be applied. This list is specified as instance path names. An error is generated if the specified information does not correspond to a valid sequential element.

-capture

Specifies that the weight should apply to the capture cycle. Can be used in conjunction with the `-shift` option to specify the same weight for both shift and capture.

-delete

Deletes any previously specified weight before applying the weight specified in the command (if any) to any specified sequential elements.

-shift

Specifies that the weight should apply to the shift cycle. Can be used in conjunction to `-capture` to specify the same weight for both shift and capture.

-weight *number*

Specifies the toggle weight as an integer from 0 to 100. Numbers exceeding 100 are accepted but not used.

Description

This command is used in the power aware ATPG flow to specify separate toggle weights for both shift and capture cycles. By default, each toggle or transition at a flip-flop (FF) is scored as a 1. This scoring only considers the flip-flop and does not take into account the fan-out of the flip-flop. By using toggle weighting, you can place an integer number, 1 or higher, onto those flip-flops to represent larger fan-out nets and cones of logic.

Example

```
DRC-T> set_toggle_weights core1/ff1 -weight 5 -shift -capture
DRC-T> set_toggle_weights core1/ff2 -weight 2 -shift
DRC-T> set_toggle_weights core1/ff2 -weight 2 -shift
DRC-T> set_toggle_weights core1/ff2 -weight 7 -capture
DRC-T> report_toggle_weights
    Non-default Toggle Weights:
    Shift Weights:
    a_reg_2_: 5
    Capture Weights:
    a_reg_2_: 5
```

See Also

[report_toggle_weights](#)

[report_power](#)

[set_atpg](#)

[report_clocks](#)

set_wgl

Overview

You use this command to set the parameters for waveform generation language (WGL) pattern generation.

Syntax

```
set_wgl
[-bidi_map {<ab> <cd>}]...
[-chain_list <all | shift>]...
[-forces_during_load <allow_x | previous_values | no_x>]
[-group_bidis | -nogroup_bidis]
[-initial_zx < as_is | x_only | z_only >]
[-inversion_reference <cell | master | omit>]
[-last_scan | -nolast_scan]
[-macro_usage | -nomacro_usage]
[-pad | -nopad]
[-scan_map < dash | bidi | keep | none >]
[-tester_ready | -notester_ready | -pre_measured]
```

Arguments

Note: This command and its arguments do not apply to the WGL_flat format.

-bidi_map <ab> <cd>

Specifies an alternative mapping of force and measure characters in the WGL output. This applies only to bidirectional ports, not to input-only or output-only ports. There are 9 mappings that can be adjusted: 3 for when the bidirectional port is an input, 4 for when the bidirectional port is an output, and 2 for when the bidirectional port is a scan input or scan output.

The **ab** choice the input side of the translation. The first character is the force value and the second character is the measure value. A dash "-" as a force character is a placeholder, meaning a scan chain input, while a dash as a measure character is a placeholder meaning a scan chain output.

The **cd** choice the corresponding output side of the translation. These are the characters that appear in the WGL output file. The default mapping can viewed with the `report_settings` command.

Here is a mapping example:

```
ab : cd
== : ==
0x : 0- # force 0, no measure
1x : 1- # force 1, no measure
xx : x- # force unknown, no measure
z0 : -0 # force Z, measure 0
z1 : -1 # force Z, measure 1
zx : -x # force Z, no measure
zz : -z # force Z, measure Z
-x : -- # bidi is a scan input
z- : -- # bidi is a scan output
```

The `ab` combinations in the previous example represent all legal values that can be mapped. All other combinations for the `ab` characters are illegal. A mapping can be repeated. If you repeat a mapping, the last one specified is used.

For more information, see [Understanding WGL Options](#).

The `set_wgl` command that implements the table above is as follows:

```
BUILD> set_wgl -bidi_map { 0x 0- 1x 1-  xx x-  z0 -0 \
                        z1 -1  zx -x  zz -z  x  --  z-  -- }
```

Note in the previous example that you can specify the `-bidi_map` option only once, and the parameters must be in a list structure. Alternatively, you can repeat the entire command line for each entry, as shown in the following example:

```
set_wgl -bidi_map {0x 0-}
set_wgl -bidi_map {1x 1-}
set_wgl -bidi_map {xx x- }
set_wgl -bidi_map {z0 -0 }
set_wgl -bidi_map {z1 -1}
set_wgl -bidi_map {zx -x}
set_wgl -bidi_map {zz -z}
set_wgl -bidi_map {x  --}
set_wgl -bidi_map {z- --}
```

To use some BIDI mappings, you must disable the detection of Z states on device I/O pins.

This means the `set_buses -external_z x` command must be specified. Also, when writing WGL output, you cannot specify the `-measure_forced_bidis` option of the `write_patterns` command.

-chain_list <all | shift>

In some designs, typically containing JTAG, there are one or more shifts of the scan chain done outside of the "shift" procedure within the load_unload procedure. When this occurs, the length of the data vector provided for the WGL "scan" statement is reduced by the number of shifts done with "vector" statements. Some WGL translators require that the length of the scan chain defined in the WGL scanchain block match the number of bits in the `scan()` load. Other WGL translators require that the length of the defined scan chain match the physical scan chain at all times. The selection of `all` causes all instances in the scan chain to occur in the definition. The selection of `shift` causes only the instances loaded by the "scan()" statement to appear. The default is `shift`.

-forces_during_load <allow_x | previous_values | no_x>

Specifies how primary inputs not explicitly forced in the load_unload procedure should be forced. A selection of `previous_values`, which is the default, causes vector and scan statements associated with the load_unload procedure to keep the values from the prior capture procedures. A selection of `no_x` is identical to a selection of `previous_values`, but in addition, if the prior value for an input was an X, it is changed to the pad character in effect while writing patterns. A selection of `allow_x` disables this pattern persistence, so that all inputs not explicitly set in the load_unload procedure are forced to X. This option has no effect if the macro form of WGL is used and the behavior is that of `allow_x`.

-group_bidis | -nogroup_bidis

Selecting `-group_bidis` specifies that WGL patterns for designs with bidirectional ports should have the pattern data for bidirectional ports presented in a single column, 2-character grouped form. Selecting `-nogroup_bidis` produces data where the bidirectional

values occur in two separate single character columns, one for input and the other for output. The default is `-nogroup_bidis`.

`-initial_zx < as_is | x_only | z_only >`

This option, which affects the WGL timeplate description, enforces a specific state (either Z or X) only at time-zero (0ns) of any waveform that starts with a Z or X. The default, `as_is`, does not change the waveform description from the timing definition in the incoming STL procedure file or provided by default.

When `z_only` is specified, any waveform event at time-zero that specifies an X is changed to a Z (all other events remain the same, including any P values). When `x_only` is specified, any Z event at time-zero is changed to an X. It is recommended that if a waveform needs a specific value, this value should be specified with the incoming timing of the STL procedure file rather than using this option. However, for situations in which the WGL output must not have a Z state at time-zero, but it is required by the STIL operation, the `-initial_zx x_only` specification can be used to change only the WGL timing.

This option is applied to [write_patterns](#) `-format wgl` or `wgl_flat` output. It is strongly recommended that you use the `x_only` option when `wgl_flat` output is generated, since `z_only` generates drive operations that are likely not desired in the output WGL file.

`-inversion_reference <master | cell | omit>`

This option selects the reference scheme used to construct the inversion markers ("!" characters) written to the WGL pattern file header. The selection of `cell` identifies an inversion from each scan cell input to the next scan cell input. The use of `master` identifies an inversion from each master cell to the next. This is a match to TestCompiler and is the default. The selection of `omit` causes all inversion markers to be omitted. This selection also affects the data written when the `-notester_ready` option is selected (see below). See also: [Understanding WGL Options](#)

`-last_scan | -noLast_scan`

Use of the `-last_scan` option results in the last patterns in the WGL file being a "scan" vector. This is the default. Use of `-nolast_scan` causes the final patterns in the WGL file to be parallel "vector" patterns. This ending with "vector" statements ensures that there are no active clocks, there are no bidirectional port conflicts or internal bus contention problems, and the same number of cycles/vectors occur for all patterns. Use of `-nolast_scan` is required for compatibility with some ASIC vendor requirements for WGL, such as LSI Logic's LNI format.

`-macro_usage | -nomacro_usage`

Selecting `-nomacro_usage` specifies that WGL patterns should be simplified without the use of macros. Use of `-macro_usage` allows for the definition and use of macros, which can make the pattern file more complex but result in a more compact file. The default is `-nomacro_usage`.

`-pad | -nopad`

Selecting `-pad` specifies that scan chains shorter than the longest scan chain should be padded to match the length of the longest. Selecting `-nopad` specifies that no padding will occur. The default is not to pad. The choice of padding characters can be altered by using the `-pad` option of the `write_patterns` command.

`-scan_map <dash | bidi | keep | none>`

For WGL output, the mapping of characters for "vector()" constructs and "scan()" constructs can be handled differently from each other. Selecting `dash` causes scan chain inputs and outputs that are also bidirectional ports to be mapped to "--" for force and measure. Selecting `bidi` causes scan chain bidi ports to be mapped as determined by the `-bidi_map` settings in effect. Selecting `keep` causes the scan chain bidi ports to keep the same value as the prior vector() construct from the load_unload procedure. Selecting `none` turns off any mapping of scan chain bidi ports. The default is `dash`.

-tester_ready | -notester_ready | -pre_measured

There are three variations for representing the scan chain load and unload data in the WGL output file. The `-tester_ready` mode causes the data to be created in a form that is ready to shift into the device pins on the tester, and requires no additional processing to compensate for inversion information of the scan cells which is loaded.

The `-notester_ready` or parallel mode causes the data to be created in a form that is ready to directly load into the state element of the scan cell which TestMAX ATPG has classified as the "master" without additional processing for scan cell inversion data. This means that the scan load and unload data is modified from the data that would have been used for `tester_ready` by adjusting the data for the effect of inversions between scan input and the scan cell for scan loads, and between the scan cell and the scan output for scan unloads. The reference point for inversions is selected by the `-inversion_reference` option.

The `pre_measured` option is similar to the `-notester_ready` option in that it is a parallel form of data, but it produces a different output when an end-of-cycle protocol is used (measure comes after shift clock). The end-of-cycle protocol requires a scan output pre-measure vector for the first scan bit and the expected scan unload data must be shifted by one bit to compensate for this pre-measure. During this one-shift adjustment, the effect of inversions between cells is considered and the expected data is modified accordingly. If a preclock measure protocol is used, then the WGL produced is the same as if a selection of `-notester_ready` was made. However, if an end-of-cycle protocol is used, the WGL is different if the scan chains contain inversions. This parallel form is the more traditional form expected by many test translation utilities.

The default is `-pre_measured`.

Description

You use this command to set the parameters for waveform generation language (WGL) pattern generation.

Examples

```
TEST> set_wgl -nomacro_usage
```

```
TEST> set_wgl -nomacro -bidi zx -x -bidi -x -- -bidi z- --
```

```
BUILD> set_wgl -bidi_map { 0x 0- 1x 1- xx x- z0 -0 \
z1 -1 zx -x zz -z x -- z- -- }
```

```
BUILD> set_wgl -bidi_map -x -x
```

```
BUILD> set_wgl -bidi_map z- z-
```

```
BUILD> set_wgl -bidi_map 0x 0x
```

```
BUILD> set_wgl -bidi_map 1x 1x
```

```
BUILD> set_wgl -bidi_map xx xx
BUILD> set_wgl -bidi_map z0 z0
BUILD> set_wgl -bidi_map z1 z1
BUILD> set_wgl -bidi_map zx zx
BUILD> set_wgl -bidi_map zz zz
BUILD> set_wgl -chain_list shift
BUILD> set_wgl -group_bidis
BUILD> set_wgl -inversion_reference master
BUILD> set_wgl -last_scan
BUILD> set_wgl -nomacro
BUILD> set_wgl -nopad
BUILD> set_wgl -notester_ready
BUILD> set_wgl -scan_map dash
```

See Also

[Creating LSI Compatible WGL](#)

[Creating NEC Compatible WGL](#)

[report_settings](#)

[set_buses](#)

[Understanding WGL Options](#)

[write_patterns](#)

set_workspace_sizes

Overview

Use this command to select the size of user-adjustable workspaces.

Syntax

```
set_workspace_sizes
[-atpg_gates integer]
[-command_line max_length]
[-command_words max_strings]
[-connectors integer]
[-decisions integer]
[-drc_buffer_size integer]
[-line integer]
[-max_trace_gates]
[-string integer]
```

Arguments

-atpg_gates *integer*

Specifies the maximum number of entries in the storage table used to keep track of user-defined ATPG gates resulting from the `add_atpg_primitives` command. The default is 1024 entries.

-command_line *max_length*

Specifies the maximum length of any command line that can be entered. Note that the command line limit is still in effect even when a command line is split up using backslashes in an effort to avoid exceeding the maximum length. The default is 50000.

-command_words *max_strings*

Specifies the maximum number of strings that can be processed as part of a single command line. The default is 10,000.

-connectors *integer*

Specifies the maximum number of fanout connections supported when calculating the draw image for the graphical schematic viewer (GSV). The default is 20000.

-decisions *integer*

Specifies the maximum storage table size used to keep track of all of the active decisions. This table is used during the ATPG process. The default size is 10000 entries. Occasionally, a design can require a larger decision table.

-drc_buffer_size *integer*

Specifies the maximum number of state element candidates that can be analyzed to determine the source for shift data associated with a multiple sensitized path during a scan

chain trace. The default setting, 1000, can be increased to 10000 when an [M251](#) message is issued due to insufficient workspace size.

-line *integer*

Specifies the maximum length of any line that can be processed when reading files. The default is 50000 characters per line.

-max_trace_gates *integer*

Specifies the maximum depth of gate tracing during DRC. By default, DRC tracing events terminate when the size of a trace exceeds 500 gates. Use the `-max_trace_gates` option when a [M738](#) warning message is generated when tracing [R17 violations](#) during DRC. For example, try doubling the maximum trace value to determine if the change affects subsequent DFT results and eliminates the R17 violations. Note that specifying a large increase in gate tracing affects the overall DRC runtime.

-string *integer*

Specifies the maximum length of a single string that can be processed. The default is 2048 characters. Consider increasing this value. For example, when the number of inputs and bidirectionals defined in the design or when the number of outputs and bidirectionals in the design exceeds the default of this parameter. An [M441](#) message indicates the increase to the default during processing.

Allowed Command Modes

All

Description

Use this command to select the size of user-adjustable workspaces. You can display the current settings with the `report_settings` command.

Examples

```
DRC> set_workspace_sizes -string 5000 -line 10000
```

See Also

[report_settings](#)

set_ydf

Overview

This command sets parameters for the [write_ydf](#) command, which creates the Yield Explorer Data Format (YDF) file that stores the physical information for a design loaded into Yield Explorer.

Syntax

```
set_ydf
  file_name
  [-apply_match_names | -noapply_match_names]
  [-schema]
  [-version version]
```

Arguments

file_name

Specifies the file name of the header schema file, which inserts a header section into a failure log file.

-apply_match_names | -noapply_match_names

Applies match name rules to instance and net names and prints the corresponding values in Yield Explorer Data Format (YDF). This option is applicable only if you also specify the [set_match_names](#) command and TestMAX ATPG is unable to match the physical instance or net names using the rules specified by this command. When the `-apply_match_names` option is enabled, TestMAX ATPG forces the [set_match_names](#) rules to apply to physical names even if no matches are found. The `-noapply_match_names` parameter disables this option.

-schema

Specifies the use of a header schema file, which contains a list of custom columns to add to the DFTCandidates table.

-version version

Sets the YDF version. The default is 1.2. If you are using class-based diagnosis reporting, the version must be set to 1.2. For more information creating a class-based diagnosis report, see the "[Class-Based Diagnosis Reporting](#)" section in the *TestMAX ATPG User Guide*.

Allowed Command Modes

All

Description

This command sets parameters for the [write_ydf](#) command, which creates the Yield Explorer Data Format (YDF) file that stores the physical information for a design loaded into Yield Explorer. The parameters set by the `set_ydf` command include specifying a header schema file, which is a configuration file that controls the format of the data contained in the header section of the failure log file. For more information about creating and using the header schema file and its format, see "[Adding Header Information to a Failure Data File](#)."

See Also

[run_diagnosis](#)

Shell Commands

This section describes the following commands:

- [cat](#)
- [clear](#)
- [cd](#)
- [cp](#)
- [ls](#)
- [ltran_shell](#)
- [mkdir](#)
- [mv](#)
- [pwd](#)
- [rm](#)
- [system](#)

cat

Overview

This command displays the contents of a selected file.

Syntax

```
cat  
  filename [-max <n>] [-line_numbers]
```

Arguments

filename

Specifies the filename whose content is to be displayed.

-max <n>

Selects the maximum number of lines to display.

-line_numbers

Specifies that line numbers should be displayed for each line.

Allowed Command Modes

All

Description

This command displays the contents of a selected file.

Examples

```
DRC> cat hello_file  
Hello!!!
```

See Also

[Other Shell Commands](#)

cd

Overview

Use this command to change the current working directory.

Syntax

```
cd  
  <directory_path>
```

Arguments

directory_path

Specifies the pathname of a directory that is set to be current working directory.

Allowed Command Modes

All

Description

Use this command to change the current working directory.

Examples

```
DRC> pwd  
  E:\BIN  
DRC> cd e:  
DRC> pwd  
  E:\BIN  
DRC> cd ..  
DRC> pwd  
  E:\  
DRC> cd regress  
DRC> pwd  
  E:\regress
```

See Also

[Other Shell Commands](#)

clear

Overview

Use this command to clear the text in the transcript window.

Syntax

```
clear
```

Arguments

None

Allowed Command Modes

All

Description

Use this command to clear the text in the transcript window.

- bullet
- more text
-

Examples

```
DRC> clear
```

See Also

[Other Shell Commands](#)

cp

Overview

Use this command to copy a file to a selected destination.

Syntax

```
cp
  source_filename destination_filename
```

Arguments

source_filename

Specifies the filename that is copied.

destination_filename

Specifies the filename where the source file is copied.

Allowed Command Modes

All

Description

Use this command to copy a file to a selected destination. Wild cards (*) are not supported.

Examples

```
DRC> cp flt_file1 flt_file2
```

See Also

[Other Shell Commands](#)

ls

Overview

Use this command to display file information for a selected file or directory.

Syntax

```
ls  
  [-l] [pathname]
```

Arguments

- l**
Selects the long format output.
- pathname***
Specifies the pathname of the directory or file that is listed. The current working directory is listed by default.

Allowed Command Modes

All

Description

Use this command to display file information for a selected file or directory. Standard wildcard usage is supported.

Examples

```
DRC> ls  
[.]  
[..]  
Makefile  
[BIN]  
[log_archive]  
[oacs_data]  
[DESIGNS]  
[WORK]  
[DATA]  
[DOCS]  
[tmp]  
r-all_i970527.txt  
cumulative_log  
I000.rpt  
I001.rpt  
r-i_i970527.txt
```

```
DRC> ls -l  
05/23/1997 12:49:38 <DIR> .  
05/23/1997 12:49:38 <DIR> ..  
05/28/1997 23:09:30 8146 Makefile
```

```
05/23/1997 12:49:38 <DIR> BIN
05/23/1997 12:49:38 <DIR> log_archive
05/23/1997 12:49:42 <DIR> oacs_data
05/23/1997 12:51:14 <DIR> DESIGNS
05/23/1997 12:52:58 <DIR> WORK
05/23/1997 12:53:04 <DIR> DATA
05/23/1997 12:53:10 <DIR> DOCS
05/23/1997 21:20:42 <DIR> tmp
05/28/1997 23:06:24 97293 r-all_i970527.txt
05/28/1997 23:16:30 32952 cumulative_log
05/28/1997 23:16:24 616 I000.rpt
05/28/1997 23:16:44 1166 I001.rpt
05/28/1997 23:16:44 1834 r-i_i970527.txt
                16 File(s) 142007 bytes
```

See Also

[cat](#)
[cd](#)
[cp](#)
[mkdir](#)
[mv](#)
[pwd](#)
[rm](#)

mkdir

Overview

Use this command to create a new directory.

Syntax

```
mkdir  
  <pathname>
```

Arguments

pathname

Specifies the name of the directory to create.

Allowed Command Modes

All

Description

Use this command to create a new directory.

Examples

```
DRC> mkdir flt_files
```

See Also

[cat](#)
[cd](#)
[cp](#)
[ls](#)
[mv](#)
[pwd](#)
[rm](#)

mv

Overview

Use this command to move a file to a selected destination.

Syntax

```
mv  
  <source_filename> <destination_filename>
```

Arguments

source_filename

Specifies the name of the file to move.

destination_filename

Specifies the name of the file to which the source file is moved.

Allowed Command Modes

All

Description

Use this command to move a file to a selected destination. Wildcards (*) are not supported.

Examples

```
DRC> mv flt_file1 flt_file2
```

See Also

[cat](#)
[cd](#)
[cp](#)
[ls](#)
[mkdir](#)
[pwd](#)
[rm](#)

pwd

Overview

Use this command to display the current working directory.

Syntax

pwd

Arguments

None

Allowed Command Modes

All

Description

Use this command to display the current working directory.

Examples

```
DRC> pwd
E:\BIN
DRC> cd e:
DRC> pwd
E:\BIN
DRC> cd ..
DRC> pwd
E:\
DRC> cd regress
DRC> pwd
E:\regress
```

See Also

[cat](#)
[cd](#)
[cp](#)
[ls](#)
[mkdir](#)
[mv](#)
[rm](#)

rm

Overview

Use this command to delete an existing file.

Syntax

```
rm filename
```

Arguments

filename

Specifies the name of the file to delete.

Allowed Command Modes

All

Description

Use this command to delete an existing file. Wildcards (*) are not supported.

Examples

```
DRC> rm flt_file1
```

See Also

[cat](#)
[cd](#)
[cp](#)
[ls](#)
[mkdir](#)
[mv](#)
[pwd](#)

system

Overview

Use this command for executing UNIX shell commands.

Syntax

```
system
  <command_string>
```

Arguments

command_string

Specifies the command to be passed to a separate shell interpreter.

Allowed Command Modes

All

Description

Use this command for executing UNIX shell commands. The command string is passed to the user's default shell and executed. The command string can be enclosed in single or double quotation marks. Text surrounded by single quotation marks is passed to the system unexpanded. Text that is not quoted or surrounded by double quotation marks has TestMAX ATPG and environment variables expanded and replaced before calling the system.

Note: The standard input and standard output of the command is not available within TestMAX ATPG. The command is passed blind to your default shell and the shell command is executed. You will not see any progress or status from this command reflected back in the TestMAX ATPG transcript.

Examples

```
DRC> system ls\line
DRC> system "ps -elf > ps_snapshot.txt"
ATPG> system "cd $MYLIB/verilog/models ; ls"
ATPG> system A_$TEST_ENV1.C
ATPG> system cp $VAR1/xxx $VAR2/yyy
BUILD> system "ls -al $MYVAR > junk"
BUILD> system "this ; and ; that"

ATPG> system "touch ${VAR1}_${VAR2}/done" # expanded before passing
ATPG> system 'touch ${VAR1}_${VAR2}/done' # not expanded before passing

# Bad form, the file 'snapshot' is empty as the "> snapshot" is not
# passed to the system but interpreted by TestMAX ATPG command parser.
DRC> system ps -fu $USER > snapshot
# proper forms, the '> snapshot' is passed as part of the command
DRC> "system ps -fu $USER > snapshot"
DRC> 'system ps -fu $USER > snapshot'

BUILD> system "date > tempfile" # save output in a file
BUILD> cat tempfile # record that file into TestMAX ATPG log
```

See Also

[Other Shell Commands](#)

Tcl API Commands

This section describes the following commands:

- [add_display_proc](#)
- [get_fanout](#)
- [all_clocks](#)
- [get_faults](#)
- [all_inputs](#)
- [get_instances](#)
- [all_outputs](#)
- [get_modules](#)
- [all_registers](#)
- [get_patterns](#)
- [filter_collection](#)
- [get_pins](#)
- [foreach_in_collection](#)
- [get_ports](#)
- [get_attribute](#)
- [get_primitive](#)
- [get_candidates](#)
- [get_scan_chains](#)
- [get_cells](#)
- [get_settings](#)
- [get_clocks](#)
- [get_timing](#)
- [get_diag_files](#)
- [list_attributes](#)
- [get_fanin](#)
- [remove_display_proc](#)

add_display_proc

Overview

This command registers a Tcl procedure name that is called when you select a context-sensitive menu item in the GSV.

Syntax

```
add_display_proc proc_name
```

Arguments

proc_name

The name of the procedure to be added.

Description

This command registers a Tcl procedure name that is called when you select a context-sensitive menu item in the GSV. After this command is specified, it determines if the procedure has been defined. If the procedure being added is not defined, the following error message is displayed:

```
Error: No command or proc named "%s" is found.
```

When you select a menu item, the corresponding Tcl procedure is called with an argument. If the GSV is at the design level (library cells shown), the instance name of the selected cell is sent as the argument to the proc call. If the GSV is at a primitive level, the gate id is used. You can use a Tcl form such as “The arg value is \$arg” to distinguish between the gate id or the instance name.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

[remove_display_proc](#)

all_clocks

Overview

The `all_clocks` command creates a collection of all clocks in the current design.

Syntax

```
collection all_clocks
```

Arguments

None.

Description

The `all_clocks` command creates a collection of all clocks in the current design. You can assign these clocks to a variable or pass them into another command. If you do not define any clocks, the empty collection (empty string) is returned.

By default, a maximum of 10 objects are displayed. You cannot change this maximum.

If you want only certain clocks, use the `get_clocks` command to create a collection of clocks matching a specific pattern.

If this command is part of SDC file, it returns all SDC clocks. If this command is executed outside SDC file, it returns all clocks defined for TestMAX ATPG.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

Examples

```
TEST-T> all_clocks
```

See Also

`get_clocks`

all_inputs

Overview

The `all_inputs` command creates a collection of all input or inout ports in the current design.

Syntax

```
collection all_inputs
```

Arguments

None

Description

The `all_inputs` command creates a collection of all input or inout ports in the current design.

If you specify `all_inputs` at the command prompt, it displays the names of inputs.

By default, a maximum of 10 objects are displayed. You cannot change this maximum.

If you want only certain ports, use the `get_ports` command to create a collection of ports matching a specific pattern.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

Examples

```
TEST-T> all_inputs
```

See Also

`get_ports`

all_outputs

Overview

The `all_outputs` command creates a collection of all output or inout ports in the current design.

Syntax

```
collection all_outputs
```

Arguments

None

Description

The `all_outputs` command creates a collection of all output or inout ports in the current design.

When issued from the command prompt, `all_outputs` behaves as though `query_objects` had been called to display the objects in the collection.

By default, a maximum of 10 objects are displayed. You cannot change this maximum.

If you want only certain ports, use the `get_ports` command to create a collection of ports matching a specific pattern.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

Examples

```
TEST-T> all_outputs
```

See Also

`get_ports`

all_registers

Overview

The `all_registers` command creates a collection of cells related to registers.

Syntax

```
collection all_registers
```

Arguments

None

Description

The `all_registers` command creates a collection of cells related to registers. You can assign the resulting collection to a variable or pass it into another command.

When issued from the command prompt, the `all_registers` command behaves as though `query_objects` had been called to display the objects in the collection.

By default, a maximum of 10 objects are displayed. You cannot change this maximum.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

Examples

```
TEST-T> all_registers
```

See Also

`get_cells`

filter_collection

Overview

This command filters a collection after the collection is created.

Syntax

```
collection filter_collection
  [-regexp]
  [-nocase]
  collection1
  expression
```

Arguments

-regexp

Indicates that TestMAX ATPG regular expressions are used by the following filter operators: `=~` and `!~`. By default, these filter operators use simple wildcard pattern matching with the `'*'` and `'?'` wildcards.

-nocase

Makes the pattern match case-insensitive.

Attributes

collection1

Specifies the base collection to be filtered. This collection is copied to the result collection. Objects are removed from the result collection if they are evaluated as false by the conditional expression.

expression

Specifies the expression used to filter the collection.

Description

This command filters a collection after the collection is created. In many cases, the commands that create the collections support a `-filter` option, which performs the filtering as part of the collection process. Using the `-filter` option in those commands is usually more efficient than using the `filter_collection` command to filter the collection after it has been created. The `filter_collection` command is most useful when you want to filter the same large collection many times using different criteria.

The `filter_collection` command results in either a new collection containing the subset of the objects in the input specified by using the base collection, or it results in the empty string (an empty collection) indicating that the expression filtered out all elements of the input base collection.

The relational operators are as follows:

`==` Equal

`!=` Not equal

- > Greater than
- < Less than
- >= Greater than or equal to
- <= Less than or equal to
- =~ Matches pattern
- !~ Does not match pattern

The basic relational rules are as follows:

String attributes can be compared with any operator.

Numeric attributes cannot be compared with pattern-match operators.

Boolean attributes can be compared only with == and !=. The value can be only true or false.

Examples

The following example echoes all `pin_pathnames` containing the string “slow_clk”:

```
TEST-T> foreach_in_collection pin [filter_collection [get_pins] pin_
pathname=~*slow_clk* ] {
? echo [get_attribute $pin pin_pathname]
? }
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/CP
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/Q
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/QN
TOTO/slow_clk_clkgt/U2/Z
TOTO/slow_clk_clkgt/U2/A
TOTO/slow_clk_clkgt/U2/B
TOTO/slow_clk_clkgt/cg_latch_reg/Q
TOTO/slow_clk_clkgt/cg_latch_reg/GN
TOTO/slow_clk_clkgt/cg_latch_reg/D
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/CD
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/D
```

The following example echoes all `pin_pathnames` containing the string “slow_clk”, but using TestMAX ATPG regular expressions:

```
TEST-T> foreach_in_collection pin [filter_collection -regexp [get_pins] \
? pin_pathname=~.*slow_clk.* ] {
? echo [get_attribute $pin pin_pathname]
? }
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/CP
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/Q
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/QN
TOTO/slow_clk_clkgt/U2/Z
TOTO/slow_clk_clkgt/U2/A
TOTO/slow_clk_clkgt/U2/B
TOTO/slow_clk_clkgt/cg_latch_reg/Q
TOTO/slow_clk_clkgt/cg_latch_reg/GN
```

```
TOTO/slow_clk_clkgt/cg_latch_reg/D
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/CD
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/D
```

The following example finds whether the `set_delay -nopi_changes` command has been set:

```
TEST-T> if {[get_attribute [filter_collection [get_settings delay] \
? command_type==pi_changes] value] == "false"} {
? echo "-nopi_changes is set"
? }
-nopi_changes is set
```

foreach_in_collection

Overview

This command is used to iterate each element in a collection.

Syntax

```
string foreach_in_collection
    itr_var
    collections
    body
```

Arguments

itr_var

Specifies the name of the iterator variable.

collections

Specifies a list of collections to iterate.

body

Specifies a script to execute per iteration.

Description

This command is used to iterate each element in a collection. You cannot use the Tcl-supplied `foreach` command to iterate collections because this command requires a list, and a collection is not a list. Using the `foreach` command on a collection causes the collection to be deleted.

The arguments for the `foreach_in_collection` command parallel those of the `foreach` command: an iterator variable, the collections to iterate, and the script to apply at each iteration. Note that the `foreach_in_collection` command does not allow a list of iterator variables.

During each iteration, the `itr_var` option is set to a collection of exactly one object. Any command that accepts collections as an argument accepts the `itr_var` option, because they are of the same data type (collection).

You can nest the `foreach_in_collection` command within other control structures, including the `foreach_in_collection` command.

Note that if the body of the iteration is modifying the design, it is possible that all or part of the collection involved in the iteration are deleted. The `foreach_in_collection` command is safe for such operations. If a command in the body causes a collection to be removed, at the next iteration, the iteration ends with a message indicating that the iteration ended prematurely.

Examples

The following example iterates a filtered collection and echoes the `pin_pathname` for each member of the collection:

```
TEST-T> foreach_in_collection pin [filter_collection [get_pins] pin_
pathname=~*slow_clk* ] {
? echo [get_attribute $pin pin_pathname]
```

```
? }
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/CP
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/Q
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/QN
TOTO/slow_clk_clkgt/U2/Z
TOTO/slow_clk_clkgt/U2/A
TOTO/slow_clk_clkgt/U2/B
TOTO/slow_clk_clkgt/cg_latch_reg/Q
TOTO/slow_clk_clkgt/cg_latch_reg/GN
TOTO/slow_clk_clkgt/cg_latch_reg/D
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/CD
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/D
```

get_attribute

Overview

This command searches the *object_spec* for the specified attribute and returns a list of attribute values.

Syntax

```
list get_attribute
  [-class class_name]
  [-quiet]
  [-value_list]
  object_spec
  attr_name
```

Arguments

-class *class_name*

If the *object_spec* attribute is a name, its class can be any of the following: Clock, ScanChains, Cell, Primitive, Pin, Port, Modules, Instances, Timings, Settings, Faults, Patterns, Candidates, DiagFile.

-quiet

Does not report messages.

-value_list

Forces the return of a single object as a list.

Attributes

object_spec

Object(s) used to get the attributes.

attr_name

Name of the attribute.

Description

This command searches the *object_spec* for the specified attribute and returns a list of attribute values. An empty list is returned if the attribute is not found on any of the specified objects.

Example

The following example iterates over a filtered collection and gets the *pin_pathname* attribute of each pin so that it can be echoed:

```
TEST-T> foreach_in_collection pin [filter_collection [get_pins] pin_
pathname=~*slow_clk* ] {
? echo [get_attribute $pin pin_pathname]
? }
TOTO/U_clk_control_i_0/slow_clk_enable_l_reg/CP
```



```
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/Q  
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/QN  
TOTO/slow_clk_clkgt/U2/Z  
TOTO/slow_clk_clkgt/U2/A  
TOTO/slow_clk_clkgt/U2/B  
TOTO/slow_clk_clkgt/cg_latch_reg/Q  
TOTO/slow_clk_clkgt/cg_latch_reg/GN  
TOTO/slow_clk_clkgt/cg_latch_reg/D  
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/CD  
TOTO/U_clk_control_i_0/slow_clk_enable_1_reg/D
```

get_candidates

Overview

This command returns a collection of defect candidate objects which contain the results of a previous `run_diagnosis` command.

Syntax

```
collection get_candidates  
  -filter expression | -pinpath pinpath | -defect defect_id | -all
```

Arguments

-filter *expression*

Filters the collection with the expression. For any candidates that match patterns, the expression is evaluated based on the candidate's attributes. If the expression evaluates to true, the candidate is included in the result.

-pinpath *pinpath*

Returns the fault object for the collapsed fault(s) associated with the specified hierarchical pinpath.

-defect *defect_id*

Returns the fault object for the collapsed fault(s) associated with the specified *defect_id*.

-all

Returns all defect candidate objects.

Attributes

pinpath (string)

Pinpath of the collapsed fault location if functional diagnosis was done.

pin (collection of pin objects)

Pin object of fault location, if functional diagnosis was done.

cell (collection of cell objects)

Cell associated with defect site.

chain_name (string)

Chain name of the defective chain (if chain diagnosis was done, empty otherwise).

defect_failing_patterns (integer)

Number of failing patterns for all candidates of a particular defect group. In the case of chain diagnosis, the attribute value is -1.

defect_passing_patterns (integer)

Number of passing patterns for all candidates of a particular defect group. In the case of chain diagnosis, the value is -1.

explained_passing_patterns (integer)

Number of passing patterns for a particular candidate. This information is equivalent to "passing=63" in the diagnosis report line "#perfect/partial match: <failing=18/18, passing=63>". To get the number of failing patterns for a particular candidate, use the attribute `explained_patterns` and determine the size of the collection. This will give the number of fully explained patterns for a given candidate.

physical_subnet_id

Integer that represents the physical subnet ID value.

subnet_driver

String that represents the physical subnet driver name.

scan_cell_position (integer)

Scan chain position of defective scan cell (if chain diagnosis was done, -1 otherwise).

scan_cell_name (string)

Hierarchical instance name of the defective scan cell (if chain diagnosis is done).

subnet_id (integer)

Indicates the subnet id if the diagnosis finds a subnet as a fault candidate. For each candidate, the attribute is available. If a stuck-open fault candidate that affects a particular subnet is not found, the attribute value is -1.

defect_id (integer)

From previous diagnosis run, this is the ID of the defect group to which this fault belongs (if functional diagnosis was done, -1 otherwise).

explained_patterns (Pattern object collection)

From previous diagnosis run, patterns (fully or partially) explained by this fault (if functional diagnosis was done).

is_partially_explaining (boolean)

From previous diagnosis run, this is true if this fault only partially explained a pattern (if functional diagnosis was done).

internal_cell_defect_flag (boolean)

True if the fault candidate is of type `internal_cell`.

type (string)

Indicates the type of the fault candidate. For example, `sa0`, `stf`.

class_behavior (string)

Contains the fault class. For example, `DS`, `--`, `**`.

TFSF (integer)

Number of failures where the corresponding measure location failed for both the tester and simulation of this fault.

TFSP (integer)

Number of failures where the corresponding measure location failed for the tester but passed for the simulation of this fault.

TPSF (integer)

Number of failures where the corresponding measure location passed for the tester but failed for the simulation of this fault.

match_score (float)

This is the match score in percentage from either functional or chain diagnosis, for example, 98 or 50.

causes_passing_pattern_fail (boolean)

True if this candidate caused a passing pattern to fail (if functional diagnosis was done).

module_name (string)

Name of the module of the candidate.

cone_id (int)

ID of the cone of the candidate.

bridging_module_name (string)

Name of the module of the second node reported for a bridging candidate.

bridging_pinpath (string)

Pinpath of the second node reported for a bridging candidate.

combined_pinpath (string)

Pinpath of the best candidate.

combined_score (float)

Score of best candidate.

equiv_faults (collection of fault objects)

- Includes fault objects that are equivalent to the fault candidate, and includes the fault candidate.
- Only the pinpath and type attributes of the fault object in this collection are valid since there are no active faults after diagnosis.

object_class (constant string: "defect")**Description**

The `get_candidates` command returns a collection of defect candidate objects which contain the results of a previous `run_diagnosis` command. A defect candidate object is available for a collapsed diagnosis fault candidate (functional diagnosis), or a scan cell (chain diagnosis). If the `-filter` option is used on a pinpath that is not the collapsed pinpath, no results are returned.

Note that the `get_candidates` command is supported only for fault-based diagnostics reporting, but not class-based reporting.

Use the attribute pins to retrieve the net name connected to a fault candidate's pin and the X/Y coordinates for the pin. See the attributes list for the `get_pins` command.

This command is disabled when reading a secure image file.

For more information, see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

[get_pins](#)

get_cells

Overview

This command returns a collection of cell objects.

Syntax

```
collection get_cells  
  -id d | -type type | -nonscan | -filter expression | -all | -selected | -  
  clock clock | [-regexp] [-nocase] [-hsc separator] [patterns]
```

Arguments

-id *d*

Returns a collection of cell objects corresponding to the specified gate ID.

-type *type*

Returns a collection of cell objects with primitives of the type specified in the `report primitives` command.

-nonscan

Returns a list of nonscan sequential elements.

-all

Returns all cell objects.

-selected

Returns a collection of cells that are selected in the GSV.

-clock *clock*

Returns cells with the specified clock.

-filter *expression*

Returns cells with the specified filter. The `-filter` option cannot be used when the `patterns` argument is specified.

-regexp

Recognizes patterns as full regular expressions. This option applies only to the cell name. Note that you cannot use the `-filter` and `-regexp` options at the same time.

-nocase

When combined with the `-regexp` option, the `-nocase` option causes case-insensitive matches. You can use the `-nocase` option only when you also use the `-regexp` option.

-hsc *separator*

Specifies a separator character. The default is "/". Valid values are /, @, ^, #, ., and |

patterns

Matches cell names against the specified patterns.

Attributes

cell_id (integer)

The ID of the cell.

cell_type (string)

Primitive type of this cell.

primitive_name (string)

Primitive path name of the output pin of a primitive. This attribute is valid only for primitives that drive an output pin of a library cell.

cell_name (string)

Instance name of the parent library cell of this primitive.

cell_module_name (string)

The module name of the cell.

is_load (boolean)

True if this is a loadable nonscan cell and if it appears in the output list produced by the [report_nonscan_cells](#) load command.

is_nonx_load (boolean)

True if this is a loadable nonscan cell that always loads a non-X value and if it appears in the output list produced by the [report_nonscan_cells](#) load command.

is_scan (boolean)

True if this cell is a scan cell.

is_masked (boolean)

True if this cell is masked by `add capture masks` or `set rules -mask`. Valid for DFF or DLAT primitives.

load_unload_constraint (string)

For example, "X". PI constraint from STL procedure file from the load_unload constraint.

capture_constraint (string)

For example, "OX". ATPG, PI, cell constraint or constraint in capture procedure in STL procedure file. Same format as `add cell_constraints`, `add atpg constraints`, or `add pi constraints`. Cell constraints are valid for DFF, DLAT primitives. PI constraints are valid for PI primitives.

nonscan_behavior (string)

For example, "C0", "TLA". Data as it appears on `report nonscan cells` output.

pins (pin object collection)

Collection of pin objects.

is_clock (boolean)

True if this is a PI primitive that is a clock.

module_weight (float)

Weight assigned to library cell.

fanin_gates (cell object collection)

Collection of cells that immediately fanin to this cell. 1:1 with input_pins.

fanout_gates (cell object collection)

Collection of cells that immediately fanout from this cell.

chain_name (string)

If scan cell, this is the chain name.

edge (string)

The trigger type of the clock port of the cell.

is_async (boolean)

True if the cell has unstable set/reset.

is_constrained (boolean)

True if the cell has load_unload or capture constraint.

is_disturbed (boolean)

True if the cell is load-disturbed. False when the cell is listed as load-stable in the [report_nonscan_cells](#) command.

is_placement_valid (boolean)

True if the cell has physical placement data available.

is_slow (boolean)

True if the cell is a slow cell or the BIDI is slow BIDI.

scan_position (integer)

If scan cell, this is the scan cell position. Position 0 is closest to scanout port; -1 if invalid.

scan_type (string)

Scan type of the cell.

placement_left, placement_right, placement_top, placement_bottom (integer)

Valid if physical data has been read in. X location of left or right edge of the cell or Y location of the top or bottom of the cell; -1 if invalid.

rotation (integer)

Valid if physical data has been read in. Counter-clockwise rotation value in degrees: 0, 90, 180, 270; -1 if invalid.

mirror_x (boolean)

Valid if physical data has been read in. True if the cell has been flipped along the X axis. Y axis flip can be transformed with mirror_x and rotation combinations.

is_displayed (boolean)

True if this gate is displayed in the GSV.

is_selected (boolean)

True if this gate is selected in the GSV (shown as red).

color (string)

Non-empty string if this gate is highlighted. If highlighted, this is the color name.

object_class (constant string: "cell")

A constant string that indicates the class type of the object returned by `get_cells`.

Description

This command returns a collection of cell objects. This command is disabled when reading a secure image file.

For more information, see ["An Introduction to the TestMAX ATPG Tcl API."](#)

Example

Use the following code to build a collection, called `cell_col`, of cell objects with names that contain the word "add"; and to display the `cell_name` and `cell_module_name` attribute:

```
TEST-T> foreach_in_collection cell_col [get_cell -filter "cell_name=~*add*"]
{
set name [ get_attribute $cell_col cell_name]
set mod [get_attribute $cell_col cell_module_name]
puts "$name $mod"
}
add_24/U31 IVI
add_24/U27 ND2I
add_24/U26 IVI
add_24/U2 EOI
add_24/U15 ENI
add_24/U28 AN2I
add_24/U25 ND2I
add_24/U14 ND2I
add_24/U13 ENI
add_24/U24 IVI
add_24/U12 ENI
add_24/U29 AN2I
add_24/U23 ND2I
add_24/U11 ND2I
add_24/U10 ENI
add_24/U22 IVI
add_24/U9 ENI
```

get_clocks

Overview

The `get_clocks` command creates a collection of clocks in the current design that match certain criteria.

Syntax

```
collection get_clocks
  [-all]
  [-regexp]
  [-nocase]
  [patterns]
list patterns
```

Arguments

-all

Returns all clocks.

-regexp

Views the *patterns* argument as real regular expressions rather than simple wildcard patterns. This option applies only to the clock name.

-nocase

When combined with `-regexp`, makes matches case-insensitive. You can use `-nocase` only when you also use `-regexp`.

patterns

Matches clock names against the specified patterns. Patterns can include the wildcard characters "*" and "?".

Attributes

clock_id (int)

The ID of the clock.

clock_name (string)

Name of the clock.

disturbed_clocks (collection)

A pulse on this clock affects (disturbs) states established on elements by other clocks.

is_differential (boolean)

This clock is differential; there are two related signals for this clock.

is_internal (boolean)

This attribute is true if the clock is an internal clock; otherwise it is false. An internal clock is defined by the command [add_clocks](#) `-intclock` or by the `Internal` keyword in the `ClockStructures` block of the STL procedure file.

is_pll (boolean)

This attribute is true if the clock is a PLL clock; otherwise it is false. A PLL clock is defined by the command [add_clocks](#) -pllclock or by the `PLL` keyword in the `ClockStructures` block of the STL procedure file.

nonscan_connections (int)

Indicates the number of nonscan cell ports connected to this clock. The ports taken into account are clock, set and reset ports. Cells with constraints, such as "constant values cells" might not be counted here.

object_class (string)

Constant string "clocks."

off_state (string)

Off state of the clock.

parallel_clocks (collection)

A collection of parallel clocks in the clock matrix.

pll_pin_name (string)

Identifies the pin name of the clock if the `is_pll` attribute is true.

is_reference (boolean)

This attribute is true if the clock is a reference clock; otherwise it is false. A reference clock is defined by the command [add_clocks](#) -refclock or by the `Reference` keyword in the `ClockStructures` block of the STL procedure file.

scan_connections (int)

Indicates the number of scan cell ports connected to this clock. The ports taken into account are clock, set and reset ports.

ungroupable_clocks (collection)

The set of clocks that cannot be pulsed together (in parallel) because they affect the state of elements in common (disturb).

usage (string)

Indicates the functional usage of the clock.

Description

The `get_clocks` command creates a collection of clocks in the current design that match certain criteria. The command returns a collection if any clocks match the patterns and pass the filter (if specified). If no objects match the criteria, the empty string is returned.

You can use the `get_clocks` command at the command prompt, or you can nest it as an argument to another command (for example, `query_objects`). In addition, you can assign the `get_clocks` command result to a variable.

When issued from the command prompt, the `get_clocks` command behaves as though the `query_objects` command had been called to display the objects in the collection.

By default, a maximum of 100 objects is displayed; you can change this maximum using the variable `collection_result_display_limit`.

The "implicit query" property of the `get_clocks` command provides a fast, simple way to display clocks in a collection. However, if you want the flexibility provided by the `query_objects` command options (for example, if you want to display the object class), use the `get_clocks` as an argument to the `query_objects` command.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

Example

```
TEST-T> get_clocks
```

See Also

`all_clocks`
`report_clock`

get_diag_files

Overview

This command returns a collection of objects for diagnosis files, which include the failure log files and the patterns file.

Syntax

```
collection get_diag_files  
    -name file_name | -all
```

Arguments

-name *file_name*

Returns the diagnosis file object pertaining to the specified file name.

-all

Returns all diagnosis file objects.

Attributes

attr_file_name (string)

Sets user-defined attributes for a particular failure log file. You can then access the specified string value using the Tcl API. For example, an attribute can describe the ATE clock frequency or the pattern type used for testing a chip. You can then retrieve the string using the `attr_file_name` attribute returned by the `get_diag_files` Tcl API command.

basic_first_pat_id (int)

ID of the first basic scan pattern.

basic_last_pat_id (int)

ID of the last basic scan pattern.

basic_total_offset (int)

Cycle offset for basic scan patterns.

chain_test_patterns (int)

Number of chain test patterns in the external pattern buffer.

checked_failures (int)

Number of failures that are checked for the expected value.

failing_patterns (int)

Number of failing patterns in the failure log file.

failure_memory_limit (int)

Value of the `.failure_memory_limit` directive.

fastseq_first_pat_id (int)

ID of the first fast-sequential pattern.

fastseq_last_pat_id (int)

ID of the last fast-sequential pattern.

fastseq_total_offset (int)

Cycle offset for fast-sequential patterns.

ignored_failures (int)

Number of failures that are ignored due to a mapping process failure or the presence of X measures.

incomplete_failures (int)

Value of the `.incomplete_failures` directive.

last_diag_proc (string)

Indicates the type of diagnosis performed in the last run. The string is "logic_diag" if functional logic diagnosis is performed. The string is "chain_diag" if scan chain diagnosis is performed.

name (string)

Name of the failure log file.

partially_mapped_patterns (int)

Number of patterns that are partially mapped.

passing_patterns (int)

Number of passing patterns.

pat_file (string)

Name of the pattern file corresponding to the current failure log file. The string is the value of `.pattern_file_name` directive.

skip_all_patterns (int)

Indicates the presence of skipped patterns during the current diagnosis run. If all the patterns are skipped, the integer value is 1. If some or none of the patterns are skipped, the integer value is 0.

skipped_failures (int)

Number of failures that are skipped during the failure mapping process.

skipped_patterns (int)

Number of patterns that are skipped during the failure mapping process because of too many unmapped shifts or some other reason. These patterns are ignored during diagnosis.

total_failures (int)

Number of failures in the failures log file.

total_patterns (int)

Number of patterns that are read in the external pattern buffer.

truncate (int)

Value of the `.truncate` directive.

unmapped_patterns (int)

Number of unmapped patterns due to at least one unmapped shift. These patterns can still be used during diagnosis.

used_failures (int)

Number of failures that are used for the diagnosis.

version (string)

The current version of the tool.

xmeasure_failures (int)

Number of failures that are ignored because they occur during a masked cycle (X measure) in the external pattern file. This number refers to the value of [M266](#) message ("*Number failure(s) were ignored due to X measures. (M266)*").

Description

This command returns a collection of objects for diagnosis files, which include the failure log files and the patterns file. The directives for each failure log file are also available.

This command is disabled when reading a secure image file.

For more information on the Tcl API, see "[An Introduction to the TestMAX ATPG Tcl API](#)."

get_fanin

Overview

This command returns a collection of fanin pins.

Syntax

```
collection get_fanin  
  -to pin_name [-shift] [-test_setup]
```

Arguments

-to *pin_name*

Specifies the pin name to be used as the end point or sink for mapping/tracing. By default, the `get_fanin` command prints all the PIs, PIOs and flip-flops that are driving this pin. You can specify wildcard characters for specifying pin names, but not instance names. The `get_fanin` command will report, one-by-one, all the sources for all the matching endpoint pin names.

-shift

Outputs endpoints after the shift constraints are applied.

-test_setup

Outputs endpoints after applying the `test_setup` sequence.

Description

This command returns a collection of fanin pins

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

[report_fanin](#)

[get_fanout](#)

get_fanouts

Overview

This command returns a collection of fanout pins

Syntax

```
collection get_fanout  
  -from pin_name [-shift] [-test_setup]
```

Arguments

-from *pin_name*

Specifies the hierarchical pin name in the top-level netlist to be used as the starting point for reporting fanout paths. Wildcard pin names are supported, although they do not apply to instance names.

-shift

Outputs endpoints after the shift constraints are applied.

-test_setup

Outputs endpoints after applying the test_setup sequence.

Description

This command returns a collection of fanout pins

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

[report_fanout](#)

[get_fanin](#)

get_faults

Overview

The `get_faults` command returns a collection of fault objects.

Syntax

```
collection get_faults  
  -filter <expression> | -pinpath <pinpath> | -all
```

Arguments

-filter <expression>

Filters the collection with the expression. For any fault objects that match patterns, the expression is evaluated based on the fault objects's attributes. If the expression evaluates to true, the fault object is included in the result.

-pinpath <pinpath>

Returns the fault object for the collapsed fault(s) associated with the specified hierarchical pinpath.

-all

Returns all fault objects.

Attributes

pinpath (string)

Pinpath of the collapsed fault location if functional diagnosis was done.

pin (pin object)

Pin object of fault location, if functional diagnosis was done.

type (string)

For example, "sa0", "stf".

equiv_faults (collection of fault objects)

Collection of un-collapsed fault object.

gate_id (int)

Gate ID of the cell where the fault is defined.

id (int)

ID of the fault.

module_name (string)

Module name of the cell where the fault is defined.

object_class (string) (constant string: "fault")

A constant string that indicates the class type of the object returned by `get_faults`.

class_behavior (string)

For example, "DS", "AU", "--", "***"

Description

The `get_faults` command returns a collection of fault objects. There is a fault object per collapsed fault. If `filter` is used on a pinpath that is not the collapsed pinpath, no results are returned.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

`report_faults`

get_instances

Overview

The `get_instances` command returns a collection of instances.

Syntax

```
collection get_instances -type <instance_type> | instance_name
```

Arguments

-type <instance_type>

Returns a collection of instance objects.

instance_name

Returns the instance object associated with the specified instance type.

Attributes

instance_name (string)

Name of the instance.

module_name (string)

Name of the module associated with the instance.

object_class (string)

Constant string "instances."

pins (collection)

Pins of this instance.

Description

The `get_instances` command returns a collection of instances.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

`report_instances`

get_modules

Overview

This command returns a collection of module objects.

Syntax

```
collection get_modules -type type | module_name
```

Arguments

-type *type*

Returns the module object associated with the specified type.

module_name

The name of the module to return.

Attributes

box_type (string)

Returns the type of module (blackbox, emptybox and so forth)

instances (collection)

Instances associated with this module.

is_black_box (boolean)

True if this module is a black box.

is_defined (Boolean)

True if the module is defined.

is_empty_box (boolean)

True if this module is an empty module.

is_nobox (boolean)

True if the module is declared as NOBOX.

module_name (string)

Name of the module.

object_class (string)

Description

This command returns a collection of module objects.

You can use `get_modules` command with an image file if the `-netlist_data` option of the [write_image](#) command is specified.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API.](#)"

See Also

report_modules

get_patterns

Overview

The `get_patterns` command returns a collection of pattern objects.

Syntax

```
collection get_patterns  
  -pattern d | -filter <expression> | -all
```

Arguments

-pattern *d*

Return a collection of pattern objects with the specified pattern number (internal or external buffer).

-filter <expression>

Filters the collection with the expression. For any pattern objects that match patterns, the expression is evaluated based on the patterns object's attributes. If the expression evaluates to true, the pattern object is included in the result.

-all

Returns all pattern objects.

Attributes

number (integer)

Pattern number, relative to the buffer.

is_internal (boolean)

True if in internal pattern buffer.

failing_gates (Cell object collection)

Collection of cells of either scan cells and/or POs where failure(s) occurred on the tester. This data comes directly from the failure log file.

is_explained (boolean)

True if previous diagnosis run explained this pattern.

is_partial_explained (boolean)

True if multiple faults were required to explain all the failures of this pattern, or if only part of the failures were explained by a fault.

is_tester_fail (boolean)

True if this pattern failed on the tester based on the failure log file.

explaining_candidates (fault candidate object collection)

After diagnosis, the list of fault candidates that explain this pattern.

```
object_class (constant string: "pattern")
```

Description

The `get_patterns` command returns a collection of pattern objects.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

`report_patterns`

get_pins

Overview

The `get_pins` command creates a collection of pins in the current design, relative to the current instance, that match certain criteria.

Syntax

```
collection get_pins [-filter expression] [-regexp] [-nocase]
                  [-hsc separator] [patterns]
                  string expression
                  list patterns
```

Arguments

-filter *expression*

Filters the collection with *expression*. For any pins that match patterns (or objects), the *expression* is evaluated based on the pin's attributes. If the *expression* evaluates to true, the pin is included in the result. Note that this option cannot be specified at the same time as the *patterns* argument.

-regexp

Views the *patterns* argument as real regular expressions rather than simple wildcard patterns. This option applies only to the pin name. You cannot use the `-filter` and `-regexp` options at the same time.

-nocase

When combined with `-regexp`, makes matches case-insensitive. You can use `-nocase` only when you also use `-regexp`.

-hsc *separator*

Separator character. The default is /. Valid values are : /, @, ^, #, ., |

patterns

Matches pin names against patterns. Patterns can include the wildcard characters "*" and "?" or regular expressions, based on the `-regexp` option.

Attributes

pin_pathname (string)

Hierarchical pinpath to this pin.

gate_id (integer)

Gate ID of the primitive to which this pin belongs.

gate (cell object collection)

Cell object of primitive to which this pin belongs.

pin_id (integer)

0 is the output.

direction (string)

IN or OUT.

net_name (string)

The net name connected to this pin.

net_weight (float)

Weight assigned to this pinpath; -1 if invalid.

pin_data (string)

Selected by [set_pindata](#). Data values shown in OLH "Understanding Pin Data".

constraint_data (string)

Constrain value of pin; 0, 1, X, Z.

location_x (integer)

Valid if physical data has been read in. X location of a vertex of the pin; -1 if invalid.

location_y (integer)

Valid if physical data has been read in; X location of a vertex of the pin; -1 if invalid.

layer (string)

Valid if physical data has been read in. Layer name where the pin is located physically.

is_clock (boolean)

True if this pin a clock pin.

is_inverted (boolean)

True if there is an inversion at the gate input (as represented by a '!' symbol in [report_primitive](#) command). It is false if there is not inversion at the input gate. The output pin of gate always returns "false".

is_scanin (boolean)

True if this pin is a scan in pin.

is_scanout (boolean)

True if this pin a scan out pin.

object_class (constant string)

Constant string "pins."

Description

The `get_pins` command creates a collection of pins in the current design, relative to the current instance, that match certain criteria. The command returns a collection if any pins match the patterns or objects and pass the filter (if specified). If no objects match the criteria, the empty string is returned.

Regular expression matching is the same as in the Tcl `regexp` command. When using the `-regexp` option, take care in the way you quote the patterns and filter expression; use rigid quoting with curly braces around

regular expressions. Regular expressions are always anchored; that is, the expression is assumed to begin matching at the beginning of an object name and end matching at the end of an object name. You can widen the search simply by adding "." to the beginning or end of the expressions as needed.

You can use the `get_pins` command at the command prompt, or you can nest it as an argument to another command (for example, `query_objects`). In addition, you can assign the `get_pins` command result to a variable.

When issued from the command prompt, the `get_pins` command behaves as though the `query_objects` command has been called to display the objects in the collection. By default, a maximum of 10 objects are displayed.

The "implicit query" property of the `get_pins` command provides a fast, simple way to display cells in a collection. However, if you want the flexibility provided by the `query_objects` command options (for example, if you want to display the object class), use the `get_pins` command as an argument to the `query_objects` command.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

Example

The following example queries the 'CP' pins of cells that begin with 'o'. Although the output looks like a list, it is just a display.

```
TEST-T> get_pins o*/CP
{"o_reg1/CP", "o_reg2/CP", "o_reg3/CP", "o_reg4/CP"}
```

The following example shows that given a collection of cells, you can query the pins connected to those cells.

```
TEST-T> set csel [get_cells o_reg1]
{"o_reg1"}
```

The following example shows how to create a clock using a collection of pins.

```
TEST-T> create_clock -period 8 -name CLK [get_pins o_reg*/CP]
1
```

See Also

`get_cells`

get_ports

Overview

The `get_ports` command creates a collection of ports in the current design that match certain criteria.

Syntax

```
collection get_ports [-regexp] [-nocase]  
    [patterns]
```

```
list patterns
```

Arguments

-regexp

Views the *patterns* argument as real regular expressions rather than simple wildcard patterns. This option applies only to the port name.

-nocase

When combined with the `-regexp` option, makes matches case-insensitive. You can use `-nocase` only when you also use `-regexp`.

patterns

Matches port names against patterns. Patterns can include the wildcard characters "*" and "?" or regular expressions, based on the `-regexp` option. Patterns can also include collections of type port.

Attributes

Port name

Object class

Transitive fanin

Sequential cells and inputs in fanin of the port.

Transitive fanout

Sequential cells and outputs in fanout of the port.

Description

The `get_ports` command creates a collection of ports in the current design that match certain criteria. The command returns a collection if any ports match the patterns and pass the filter (if specified). If no objects match the criteria, the empty string is returned.

Regular expression matching is the same as in the Tcl `regexp` command. When using `-regexp`, be careful in the way you quote the patterns and filter expression; use rigid quoting with curly braces around regular

expressions. Regular expressions are always anchored; that is, the expression is assumed to begin matching at the beginning of an object name and end matching at the end of an object name. You can widen the search simply by adding `".*"` to the beginning or end of the expressions as needed.

You can use the `get_ports` command at the command prompt, or you can nest it as an argument to another command (for example, `query_objects`). In addition, you can assign the `get_ports` command result to a variable.

When issued from the command prompt, the `get_ports` command behaves as though the `query_objects` command has been called to display the objects in the collection. By default, a maximum of 10 objects are displayed..

The "implicit query" property of the `get_ports` command provides a fast, simple way to display cells in a collection. However, if you want the flexibility provided by the `query_objects` command options (for example, if you want to display the object class), use the `get_ports` command as an argument to the `query_objects` command.

For information about collections and the querying of objects, see the collections man page. In addition, refer to the man pages for the `all_inputs` and `all_outputs` commands, which also create collections of ports.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

Example

The following example queries all input ports beginning with 'mode'. Although the output looks like a list, it is just a display.

```
TEST-T> get_ports "mode*"
{"mode[0]", "mode[1]", "mode[2]"}
```

The following example sets the driving cell for ports beginning with "in" to an FD2.

```
TEST-T> set_driving_cell -cell FD2 -library my_lib [get_ports in*]
```

See Also

`all_inputs`
`all_outputs`

get_primitive

Overview

The `get_primitive` command returns a collection of primitive objects.

Syntax

```
collection get_primitive  
    <primitive_id>
```

Arguments

<primitive_id>

Returns a collection of primitive objects corresponding to the specified primitive ID. Note that only a single ID can be specified.

Attributes

cell_id (integer)

The primitive cell's ID.

cell_type (string)

Primitive type of this cell.

primitive_name (string)

Primitive ID.

cell_name (string)

Instance name of the parent library cell of this primitive.

cell_module_name (string)

Cell's module name

is_scan (boolean)

True if this cell is a scan cell.

is_masked (boolean)

True if this cell is masked by `add capture masks` or `set rules -mask`. Valid for DFF or DLAT primitives.

load_unload_constraint (string)

For example, "X". PI constraint from STL procedure file from the `load_unload` constraint.

capture_constraint (string)

For example, "OX". ATPG, PI, cell constraint or constraint in capture procedure in STL procedure file. Same format as `add cell_constraints`, `add atpg constraints`, or `add pi constraints`. Cell constraints are valid for DFF, DLAT primitives. PI constraints are valid for PI primitives.

nonscan_behavior (string)

For example, "C0", "TLA". Data as it appears on `report nonscan cells` output.

pins (pin object collection)

A collection of pin objects. If the primitive belongs to a library cell that consists of several primitives -- for example, a MUX of a muxed scan flip-flop -- the pins collection represents the internal node of a library cell. This is different than the collection returned by the command `get_cells -id d` for the same gate ID. In the case of `get_cells`, the pin collection contains the external pins of the parent library cell.

is_clock (boolean)

True if this is a PI primitive that is a clock.

module_weight (float)

Weight assigned to library cell.

fanin_gates (cell object collection)

Collection of cells that immediately fanin to this cell. 1:1 with input_pins.

fanout_gates (cell object collection)

Collection of cells that immediately fanout from this cell.

chain_name (string)

If scan cell, this is the chain name.

edge (string)

The trigger type of the clock port of the cell.

is_async (boolean)

True if the cell has unstable set/reset.

is_constrained (boolean)

True if the cell has load_unload or capture constraint.

is_disturbed (boolean)

True if the cell has unstable set/reset.

is_placement_valid (boolean)

True if the cell has physical placement data available.

is_slow (boolean)

True if the cell is slow cell or bidi is slow bidi.

scan_position (integer)

If scan cell, this is the scan cell position. Position 0 is closest to scanout port; -1 if invalid.

scan_type (string)

Scan type of the cell.

placement_left, placement_right, placement_top, placement_bottom (integer)

Valid if physical data has been read in. X location of left or right edge of the cell or Y location of the top or bottom of the cell; -1 if invalid.

rotation (integer)

Valid if physical data has been read in. Counter-clockwise rotation value in degrees: 0, 90, 180, 270; -1 if invalid.

mirror_x (boolean)

Valid if physical data has been read in. True if the cell has been flipped along the X axis. Y axis flip can be transformed with mirror_x and rotation combinations.

is_displayed (boolean)

True if this gate is displayed in the GSV.

is_selected (boolean)

True if this gate is selected in the GSV (shown as red).

color (string)

Non-empty string if this gate is highlighted. If highlighted, this is the color name.

object_class (constant string: "cell")

A constant string that indicates the class type of the object returned by [get_cells](#).

Description

The `get_primitive` command returns a collection of primitive objects. The attributes of a primitive object are primarily the same as those for a cell object. The main difference is in the pin collection. If the primitive belongs to a library cell that consists of several primitives -- for example, a MUX of a muxed scan flip-flop -- the pin collection represents the internal node of a library cell. The collection returned by the [get_cells](#) command for the same gate ID contains the external pins of the parent library cell.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

Example

If the primitive is the MUX of a muxed scan flip-flop, then its output is connected to an internal node which is an input of a DFF primitive. An example of the `report_primitive` command for this ID is as follows:

```
TEST-T> report_primitive 547
U_TOP/PrimCntr/Incr/i_Reg0 (547) MUX (YDFFX1)
SE I 54-SE
D I 421- U_TOP/PrimCntr/Incr/U23/Y
SI I 76-SI
--- O 832-
```

The pin collection of this primitive is as follows:

```
foreach_in_collection prim [get_primitive 547] {
  foreach_in_collection pin [get_attribute $prim pins] {
    puts "pin: [get_attribute $pin pin_pathname]"
    puts "dir: [get_attribute $pin direction]"
  }
}
```



```
pin: U_TOP/PgCntr/Incr/i_Reg0/  
dir: OUT  
pin: U_TOP/PgCntr/Incr/i_Reg0/SE  
dir: IN  
pin: U_TOP/PgCntr/Incr/i_Reg0/D  
dir: IN  
pin: U_TOP/PgCntr/Incr/i_Reg0/SI  
dir: IN
```

get_scan_chains

Overview

The `get_scan_chains` command returns a collection of scan chain objects associated with the specified `chain_name`.

Syntax

```
collection get_scan_chains chain_name | -all
```

Arguments

chain_name

Specifies the name of the `chain_name` to return.

-all

Returns all scanchain objects.

Attributes

chain_name (string)

Name of scan chain.

clock_names (string)

List of all clocks associated with Master and other cells in the scan chain.

group_name (string)

Group name associated with the chain.

input_pin (string)

Name of input pin.

length (int)

Length of the scan chain.

object_class (string)

Constant string "ScanChain".

output_pin (string)

Name of output pin.

Description

The `get_scan_chains` command returns a collection of scan chain objects associated with the specified `chain_name`.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

`report_scan_chains`

get_settings

Overview

The `get_settings` command returns a collection of setting objects associated with either DRC or delay settings.

Syntax

```
collection get_settings drc | delay
```

Arguments

drc

Returns DRC settings.

delay

Returns delay settings.

Attributes

command_type (string)

Name of the command setting.

object_class (string)

Indicates the class type of the object: Setting.

setting_name (string)

Indicates the name of the setting name: drc or delay.

value (string)

Value of the command setting.

Description

The `get_settings` command returns a collection of setting objects associated with either DRC or delay settings.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

`report_settings`

get_timing

Overview

The `get_timing` command returns a collection of timing objects associated with the argument specifications.

Syntax

```
collection get_timing -proc <proc_name> | -wft <wft_name> | port_name
```

Arguments

-proc <proc_name>

Returns the timing on a specified procedure.

-wft <wft_name>

Returns the timing on a waveform table.

port_name

Returns the timing base on the specified port name.

Attributes

clkoff_time (int)

Clock off time for the clock.

clkon_time (int)

Clock on time for the clock.

force_time (int)

The earliest input (drive) event time across all PIs of the design for the primary capture cycle behavior.

is_clock (boolean)

True if the signal is clocked.

measure_time (int)

The latest output (measure) event time across all Pos of the design for the primary capture cycle behavior.

object_class (string)

Constant string "timing."

period (int)

Period of the clock.

waveform_table (string)

Waveform table for the clock.

Description

The `get_timing` command returns a collection of timing objects associated with the argument specifications.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, please see "[An Introduction to the TestMAX ATPG Tcl API](#)."

list_attributes

Overview

This command displays an alphabetically sorted list of currently defined attributes.

Syntax

```
list list_attributes
    [-application]
    [-class class_name]
    [-nosplit]
```

Arguments

-application

Lists the application attributes.

-class *class_name*

Lists only attributes of the *class_name*, which can be any of the following: Clock, ScanChains, Cell, Primitive, Pin, Port, Modules, Instances, Timings, Settings, Faults, Patterns, Candidates, DiagFile.

-nosplit

Prevents split lines when columns overflow.

Description

This command displays an alphabetically sorted list of currently defined attributes. The attributes are divided into two categories: application defined and user-defined. By default, the `list_attributes` command lists all user-defined attributes.

Using the `-application` option adds all the application attributes to the listing. Since there are many application attributes, you can limit the listing to a specific object class using the `-class` option.

Examples

The following example lists only those attributes of the class clock:

```
TEST-T> list_attributes -class clock
*****
```

Report : List of Attribute Definitions

Version: F-2011.09-SP4

Date : Mon Apr 16 18:01:03 2012

```
*****
```

Properties:

A - Application-defined

Attribute Name	Object	Type	Properties	Constraints

clock_id	Clock	int		U
clock_name	Clock	string	U	
disturbed_clocks	Clock	collection		U
is_differential	Clock	boolean		U

```
is_internal          Clock boolean U
is_pll               Clock boolean U
is_reference         Clock boolean U
nonscan_connections Clock int U
object_class         Clock string U
off_state            Clock      string U
parallel_clocks      Clock collection  U
pll_pin_name         Clock string U
scan_connections     Clock int U
ungroupable_clocks   Clock      collection  U
usage                Clock string U
```

remove_display_procs

Overview

This command removes all Tcl procedures or a specific Tcl procedure that was previously defined using the `add_display_proc` command.

Syntax

```
remove_display_proc -all | proc_name
```

Arguments

-all

Removes all previously registered procedures or commands.

proc_name

Specifies the name of the procedure or command to be removed.

Description

This command removes all Tcl procedures or a specific Tcl procedure that was previously defined using the `add_display_proc` command.

If the procedure was not previously registered, the following warning message is issued:

```
Error: No command or proc named " name" is found."
```

The GSV is then notified of the deletion of the menu item if it was previously registered.

This command is disabled when reading a secure image file. It is not yet possible for the secure image writer to enable this command as secure.

For more information, see "[An Introduction to the TestMAX ATPG Tcl API](#)."

See Also

`add_display_proc`

Update Commands

This section describes the following commands:

- [update_clock](#)
- [update_faults](#)
- [update_scale](#)
- [update_streaming_patterns](#)
- [update_wft](#)

update_clock

Overview

This command is used to change the pulse timing of a specific clock in a specific named WaveformTable.

Syntax

```
update_clock
  -wft {list_of_wft_names}
  -clock {list_of_clock_names}
  [-pulse {integer integer}]
  [-unit <ps|ns>]
```

Arguments

-wft {list_of_wft_names}

Specifies the required name or list of the specific WaveformTable(s) you want to modify. At least one **-wft** option must be provided. You may specify a list of names if you want to change a list of WaveformTables with a single command. For more information, see "[Specifying Lists in Tcl Mode](#)."

-clock {list_of_clock_names}

Specifies the required name of the clock signal or group of clock signals to be modified. At least one **-clock** specification must be provided. You may specify a list of clock names to change a set of clocks with a single command. The name specified by the **-clock** option must match the name used in the WaveformTable definition; aliases or groups containing a named clock will not be modified unless the group name as defined in the WaveformTable is used in the `update_clock` command.

-pulse {integer integer}

Specifies the timing for the leading edge and trailing edge of the specified clock pulse.

-unit <ps|ns>

Identifies the unit for specified all time values.

Description

This command is used to change the pulse timing of a specific clock in a specific named WaveformTable. It can be used only after the `read_drc` command. All optional parameters not otherwise specified in this command will remain at existing values. The time values specified by this command, scaled based the specified unit value, are modified to fit within the current scale of the WaveformTable being changed. If the units are not specified, then the values are treated relative to the current scale of the WaveformTable being changed.

Changes made by this command will only be saved, and usable, if the `write_drc` command is specified after this command and then the `run_drc` command is specified with the newly generated file. All WaveformTables across all present Timing blocks that also match the set of **-wft** names specified by this command are modified. There is no wildcard support for name matching.

When the `update_clock` command is finished, a statement identifying the number of WaveformTables referenced for modification is displayed. Not all the referenced WaveformTables can be modified if the WaveformTables do not contain constructs pertinent to the requested modification.

a specific sequence of commands is required to [modify the timing data in an existing STIL procedure file](#).

Example

```
update_clock -wft _multiclock_capture_WFT_ -clock CLK1 -pulse {90000  
110000} -unit ps
```

See Also

[update_wft](#)

[update_scale](#)

update_faults

Overview

This command updates fault parameters controlled by the fault manager.

Syntax

```
update_faults
  [-direct_credit]
  [-external file_name]
  [-reset_au]
```

Arguments

-direct_credit

Enables direct fault crediting for the following fault models:

Dynamic bridging (victim only) faults to transition delay faults

Dynamic bridging faults to static bridging faults

Dynamic bridging (victim only), static bridging (victim only), and transition delay faults to stuck-at faults

Depending on the current fault model, the appropriate source fault model is selected from the cache, if it exists. If not, you must also specify the `-external` option.

For more information, see "[Persistent Fault Model Support](#)" in the *TestMAX ATPG User Guide*.

-external file_name

This option, which is used with the `-direct_credit` option, specifies a transition, bridging, or dynamic bridging fault list (*file_name*). For more information, see "[Persistent Fault Model Support](#)" in the *TestMAX ATPG User Guide*.

-reset_au

Resets the fault classification of the faults in the AN (ATPG Untestable Not-Detected) and AX (ATPG Untestable Timing Exceptions) fault categories to ND (not detected). This allows further ATPG effort to be applied in an attempt to detect these faults.

Description

This command updates fault parameters controlled by the fault manager.

Examples

```
TEST-T> update_faults -direct_credit
1544361 stuck-at faults were changed to DS from the inactive transition
fault list.
0 stuck-at faults were changed to DI from the inactive transition fault
list.
387 stuck-at faults were changed to NP from the inactive transition fault
list.
13062 inactive transition faults were not found in the stuck-at fault
list.
```

```

TEST-T> report_summaries
      Uncollapsed Fault Summary Report
-----
fault class      code #faults
-----
Detected         DT   27161
Possibly detected PT   3515
Undetectable     UD   1634
ATPG untestable  AU   10044
Not detected     ND  124151
-----
total faults 166505
test coverage 17.26%
-----

TEST> update_faults -reset_au
TEST> report_summaries
      Uncollapsed Fault Summary Report
-----
fault class      code #faults
-----
Detected         DT   27161
Possibly detected PT   3515
Undetectable     UD   1634
ATPG untestable  AU     0
Not detected     ND  134195
-----
total faults 166505
test coverage 16.47%
-----

```

See Also

[add_faults](#)

[add_nofaults](#)

[read_faults](#)

[report_faults](#)

[remove_faults](#)

[set_faults](#)

Table Column Outside Table:

update_scale

Overview

This command is used only after specifying a `read_drc` command.

Syntax

```
update_scale
  -wft {list_of_wft_names}
  -unit <ps|ns>
```

Arguments

-wft {list_of_wft_names}

Specifies the required name or list of the specific WaveformTable(s) you want to modify. At least one `-wft` option must be provided. You may specify a list of names if you want to change a list of WaveformTables with a single command. For more information, see "[Specifying Lists in Tcl Mode](#)."

-unit <ps|ns>

Identifies the new scale value for all time values contained in the WaveformTable.

Description

This command is used only after specifying a `read_drc` command. All time values present in a WaveformTable that match a specified name are rescaled with this command and will represent the same time value but is relative to the new scale value. If the existing scale in a specified WaveformTable, and the `-unit` value on this command are the same (for example, `update_scale -unit ps`, and the WaveformTable is already in ps units), then the WaveformTable is not changed. If the WaveformTable scale is increased (for example, it was ps in the WaveformTable, and this command requests ns), then time values present in the Waveform Table can be truncated as necessary. An [M408](#) message is generated in this situation.

All time values in a modified WaveformTable will have the same units after specifying this command.

Changes made with the update commands will only be saved, and usable, if the `write_drc` command is specified after these commands, and then `run_drc` is specified with the newly generated file. All WaveformTables across all Timing blocks that might be present and that match the set of names specified by this command is modified. There is no wildcard support for name-matching.

When this command is finished processing, a statement identifying the number of WaveformTables referenced for modification is displayed. The WaveformTables might not have been modified; this information identifies how many WaveformTables were processed.

A specific sequence of commands is required to [modify the timing data in an existing STIL procedure file](#).

See Also

[update_clock](#)

[update_wft](#)

update_streaming_patterns

Overview

This command modifies, removes, reorders, or optimizes patterns for designs that use DFTMAX Ultra architectures.

Syntax

```
update_streaming_patterns
  [-load_scan_in {scan_pin_name1 value1 scan_pin_name2 value2}]
  [-insert {pattern_num insert_at_pattern_num}]
  [-max_shifts shift_length]
  [-remove_padding_patterns]
  [-remove {pattern_num pattern_num}]
  [-virtual_head_pipelines <head_value | tail_value>]
```

Arguments

-load_scan_in {scan_pin_name1 value1 scan_pin_name2 value2}

Causes the specified scan-in pins to maintain a constant value during the shift operations so you can control the peak and average power during shifting. For example, if you specify a value of 0 for the test1 scan-in pin, the pattern is modified so that all test1 pins maintain a constant 0 value during load shifting.

You can specify values for as many scan-in pins as required using Tcl list syntax. This option reduces overall power consumption during shifting, and it can be used for both internal and external patterns. However, this option also causes some coverage loss and simulation mismatches might occur if you specify scan-in pins connected to on-chip clocking (OCC) chains. For more information on using this option, see "[Controlling the Peak and Average Power During Shifting](#)."

-insert {pattern_num insert_at_pattern_num}

Use this option with the `-remove` option to remove patterns and reinsert them at a different location in the pattern set.

For each pattern number or block provided in the removal list, you must specify the pattern number where the removed patterns should be reinserted in the insertion list. You can specify blocks of patterns by providing the first and last pattern numbers of the block as a sublist inside the pattern removal list. For removed patterns that you not want reinserted, specify an insertion pattern value of X.

For example, the following command removes pattern 3 and reinserts patterns 4 through 6 at pattern 0:

```
update_streaming_patterns -remove {3 {4 6}} -insert {X 0}
```

For complete information on using the `-insert` option, see "[Removing and Reordering Patterns](#)."

-max_shifts shift_length

Specifies the maximum shift length, which enables you to increase the size of internal and external patterns from the optimal value set by TestMAX ATPG. You can use this option

before an ATPG run so the generated patterns use the specified shift length or you can apply it to external patterns. This option prevents overshifting and makes the pattern shift lengths equal across different blocks, which assures correct pattern porting.

You can apply this option in one session to increase the shift length of the patterns, and use these same patterns in another session by writing and reading them back in. For subsequent sessions, make sure you first set the pattern shift length to the same value set in previous session. Otherwise, you will see errors and simulation mismatches. For complete information on the flow for using this option, see "[Increasing the Maximum Shift Length of Patterns](#)."

-remove {*pattern_num pattern_num*}

Removes individual patterns or blocks of patterns. To remove one or more patterns, specify a list of pattern numbers. You can use this option with the `-insert` option to reorder patterns.

For complete information on using this option, see "[Removing and Reordering Patterns](#)."

-remove_padding_patterns

Performs padding pattern optimization by deleting all padding patterns, except for the first and last padding patterns. Use the following commands to apply padding pattern optimization to an internal pattern set generated by ATPG:

```
run_atpg ...
update_streaming_patterns -remove_paddding_patterns
write_patterns ...
```

Use the following commands to apply padding pattern optimization to an external pattern set:

```
set_patterns -external stil_file_name
update_streaming_patterns -remove_padding_patterns
write_patterns ...
```

For more information on using this option, see "[Optimizing Padding Patterns](#)."

-virtual_head_pipelines <*head_value* | *tail_value*>

DFTMAX Ultra can stretch the pattern file with pipeline bits to match the top-level architecture. This option translates the core-level patterns to match the actual top-level pattern pipeline structure and ATE log file. The *head_value* parameter or *tail_value* parameters align longer top-level patterns to the top-level ATE log when running diagnostics on a block-level database.

Allowed Command Modes

TEST

Description

This command modifies, removes, reorders, or optimizes patterns for designs that use DFTMAX Ultra architectures. It does not support codec configurations. For complete information on using this command, see "[Manipulating Patterns for DFTMAX Ultra](#)."

Examples

```
TEST-T> update_streaming_patterns -load_scan_in \
      {test_si1 0 test_si3 1}
```

```
TEST-T> update_streaming_patterns -max_shifts 300
```

```
TEST-T> update_streaming_patterns -remove_padding_patterns
```

```
TEST-T> update_streaming_patterns -remove {3 {4 6}} -insert {X 0}
```

Table Column Outside Table:

update_wft

Overview

This command is used to change any of the period, force-event timing, or measure-event timing parameters within a specific named WaveformTable.

Syntax

```
update_wft
  -wft {list_of_wft_names}
  [-period integer]
  [-force integer]
  [-strobe integer]
  [-unit <ps|ns>]
```

Arguments

-wft {list_of_wft_names}

Specifies the required name or list of the specific WaveformTable(s) you want to modify. At least one **-wft** option must be provided. You may specify a list of names if you want to change a list of WaveformTables with a single command. For more information, see "[Specifying Lists in Tcl Mode](#)."

-period integer

Specifies the value for the period of the specified WaveformTable.

-force integer

Specifies the timing offset value to drive all inputs and bidirectionals. The specified value is applied to all 0, 1, N, and Z waveforms to define the assert time of the drive edge for these waveforms across all signals that have these waveforms defined (in the affected WaveformTables).

-strobe integer

Specifies the measure time for all outputs and bidirectionals. The specified value is applied across all H, L, T, and X waveforms to define the measure-strobe time for these waveforms across all signals that have these waveforms defined (in the affected WaveformTables).

-unit <ps|ns>

Identifies the unit for all specified time values.

Description

This command is used to change any of the period, force-event timing, or measure-event timing parameters within a specific named WaveformTable. It can be used only after the `read_drc` command. All optional parameters not otherwise specified in this command will remain at existing values. The time values specified by this command, scaled based the specified unit value, is modified to fit within the current scale of the WaveformTable being changed. If the units are not specified, then the values are treated relative to the current scale of the WaveformTable being changed.

Changes made by this command will only be saved, and usable, if the `write_drc` command is specified after this command and then the `run_drc` command is specified with the newly generated file. All

WaveformTables across all present Timing blocks that also match the set of `-wft` names specified by this command are modified. There is no wildcard support for name matching.

When the `update_wft` command is finished, a statement identifying the number of WaveformTables referenced for modification is displayed. Not all the referenced WaveformTables can be modified if the WaveformTables do not contain constructs pertinent to the requested modification.

a specific sequence of commands is required to [modify the timing data in an existing STIL procedure file](#).

Example

```
update_wft { _launch_WFT_ _capture_WFT_ _launch_capture_WFT_ } -period 100
-unit ns
```

See Also

[update_clock](#)

[update_scale](#)

Write Commands

This section describes the following commands:

- [write_drc_file](#)
- [write_faults](#)
- [write_image](#)
- [write_netlist](#)
- [write_patterns](#)
- [write_physical_db](#)
- [write_simtrace](#)
- [write_testbench](#)
- [write_ydf](#)

Table Column Outside Table:

write_drc_file

Overview

Use this command to write a test protocol file in STIL format based on the currently defined clocks, PI constraints, PI equivalences, pin timing, and known STIL procedure files or protocol files.

Syntax

```
write_drc_file
  filename
  [-compress <gzip | binary>]
  [-generic_captures | -nogeneric_captures]
  [-master_observe_generation | -nomaster_observe_generation]
  [-replace]
  [-scancells]
  [-scanstructures <all | min>]
```

Arguments

filename

Specifies the path name of the file into which the test protocol data is written.

-compress <gzip | binary>

Specifies that the test protocol file is to be written in compressed format and specifies the type of compression: either GZIP or binary. The `binary` keyword creates a proprietary format that can be read only by TestMAX ATPG. The default is to write the file in ASCII STIL format with no compression.

-generic_captures | -nogeneric_captures

Writes single-cycle generic capture procedures (`multiclock_capture`, `allclock_capture`, `allclock_launch`, and `allclock_launch_capture`) into the output file. The default is `-generic_captures`; use the `-nogeneric_captures` option to get the previous behavior. When the `-generic_captures` option is used, the default-generated procedures are suppressed (`capture_<clockname>` will not be produced if they were not defined), but any explicitly defined clocked capture procedure from a prior `run_drc` command will still be written out. The unclocked capture procedure will not be written when this option is used. Also the default timing produced is written to be compatible with single-cycle capture procedures (a Z event is produced by default for the measure events H, L, T, and X, at time zero) when this option is used. If the default timing is defined, only one Waveformtable is generated from this option, and all procedures will reference that same timing. If you want multiple WaveformTables to be created ("`_launch_WFT_`", "`_capture_WFT_`", and "`_launch_capture_WFT_`"), then use the command `set_faults -model transition` or `set_faults -model path_delay` before the `write_drc_file` command to identify that the data should be generated to cover this mode of operation. Note that the WGL format does not support 3-cycle generic capture procedures, and third-party (Itran) formats also might have restrictions.

-master_observe_generation | -nomaster_observe_generation

Controls whether a default master_observe procedure is written into the output file. This procedure is generated only if slave clocks are detected and no master_observe procedure was defined. Note that the master_observe procedure is intended to be a template and will likely require manual edits to complete this definition. The default, `-master_observe_generation`, provides this template if the context indicates it might be useful. If you don't want additions to the set of STL procedure file procedures, you can select `-nomaster_observe_generation` to suppress the generation of this template.

-replace

Specifies that the test protocol file is allowed to replace an existing file. The default is to not allow a replacement. Attempting to overwrite an existing file without using `-replace` results in an error.

-scancells

Causes the test protocol file to include information describing each scan chain and the instances of each scan chain. The default is not to include this information. Note: The `-scancells` option does not affect scan cells on internal chains.

-scanstructures <all | min>

Controls the generation of scan chain data in the `ScanStructures` block of a STIL file. The default setting, `all`, generates all supported fields for all applicable scan chain data. If you specify the `min` keyword, the scan chain data is truncated for compressed designs that reference internal scan input or scan output ports (internal scan chains). This protects the identification of internal design elements.

The following set of examples shows how the `-scanstructures` option affects the contents of a generated STIL file. The first example shows the scan chain data using the default setting (`-scanstructures all`):

```
ScanChain "1" {
  ScanLength 11;
  ScanIn "core_i/ctl/U199/ZN";
  ScanOut "core_i/ctl/ctl_de_reg_2_/Q";
  ScanInversion 1;
  ScanCells ! "des_unit.core_i.ctl.ctl_dd_reg_0_.SI" "des_unit.core_
i.ctl.ctl_dd_reg_1_.SI"
  "des_unit.core_i.ctl.ctl_dd_reg_2_.SI" "des_unit.core_i.ctl.ctl_dd_reg_
3_.SI"
  ...
  "des_unit.core_i.ctl.ctl_de_reg_0_.SI" "des_unit.core_i.ctl.ctl_de_reg_
1_.SI"
  "des_unit.core_i.ctl.ctl_de_reg_2_.SI" ;
  ScanMasterClock "clk_st" ;
}
```

```
ScanChain "2" {
  ScanLength 11;
  ScanIn "core_i/ctl/U198/ZN";
  ScanOut "core_i/ctl/key_dd_reg_5_/Q";
  ScanInversion 1;
  ScanCells ! "des_unit.core_i.ctl.ctl_de_reg_3_.SI" "des_unit.core_
i.ctl.ctl_de_reg_4_.SI"
  "des_unit.core_i.ctl.ctl_de_reg_5_.SI" "des_unit.core_i.ctl.ctl_de_reg_
```

```

6_.SI"
...
"des_unit.core_i.ct1.key_dd_reg_5_.SI" ;
ScanMasterClock "clk_st" ;
}

```

If the `-scanstructures min` option is specified, the generated scan chain data is truncated, as shown in the following example:

```

ScanChain "1" { }
ScanChain "2" { }

```

Note the following:

All `ScanIn` and `ScanOut` references in the `Signals` group that identify pseudo signals are also removed.

The `-scanstructures min` option conflicts with parallel simulation requirements, and is disabled if parallel simulation options are set. In this case, an [M479](#) message is reported.

If you are using a mixture of design styles, and want to remove internal names, you should specify both the `-scanstructures min` and `-nointernal_scancells` options.

Allowed Command Modes

DRC, Test

Description

Use this command to write a test protocol file in STIL format based on the currently defined clocks, PI constraints, PI equivalences, pin timing, and known STIL procedure files or protocol files.

A specific sequence of commands is required to [modify the timing data in an existing STIL procedure file](#).

Examples

```

DRC> read_netlist i22.v -delete
Begin reading netlist ( i22.v )...
End parsing Verilog file i22.v with 0 errors;
End reading netlist: #modules=8, top=i22, #lines=61
DRC> run_build_model
-----
Begin build model process on topcut = i22 ...
End build model process: #gates=150, CPU_time=0.01 sec
-----
Begin learning analyses...
End learning analyses, total learning CPU time=0.02
-----
DRC> add_clocks 0 CLK
DRC> add_clocks 1 RSTB
DRC> write_drc_file template.spf -replace
End writing STIL test procedure file, File_size = 2140
DRC>

```

And here is the resulting file "template.spf":

```

STIL 1.0 {
  Extension Design P2000.5;
}
Header {

```



```

Title " TestMAX ATPG (TM) 2000.05-i000622_173054 STIL output";
Date "Wed Dec 31 17:21:05 1999";
History { }
}
Signals {
  CLK In; RSTB In; SDI2 In; SDI1 In; INC In; SCAN In; HACKIN In; si4 In;
  six In; D0 InOut; D1 InOut; D2 InOut; D3 InOut; SDO2 Out; COUT Out;
  HACKOUT Out; so4 Out; sox Out;
}
SignalGroups {
  _pi = 'D0 + D1 + D2 + D3 + CLK + RSTB + SDI2 + SDI1 + INC +
  SCAN + HACKIN + si4 + six';
  _default_Clk1_Timing_ = 'RSTB';
  _io = 'D0 + D1 + D2 + D3' { WFCMap 0X->0; WFCMap 1X->1; WFCMap
  ZX->Z; WFCMap NX->N; }
  _po = 'SDO2 + COUT + D0 + D1 + D2 + D3 + HACKOUT + so4 + sox';
  _default_In_Timing_ = 'D0 + D1 + D2 + D3 + CLK + RSTB + SDI2 +
  SDI1 + INC + SCAN + HACKIN + si4 + six';
  _default_Out_Timing_ = 'SDO2 + COUT + D0 + D1 + D2 + D3 + HACKOUT
  + so4 + sox';
  _default_Clk0_Timing_ = 'CLK';
}
ScanStructures {
  # Uncomment and modify the following to suit your design
  # ScanChain chain_name { ScanIn chain_input_name; ScanOut chain_output_
name; }
}
Timing {
  WaveformTable _default_WFT_ {
    Period '100ns';
    Waveforms {
      _default_In_Timing_ { 0 { '0ns' D; } }
      _default_In_Timing_ { 1 { '0ns' U; } }
      _default_In_Timing_ { Z { '0ns' Z; } }
      _default_In_Timing_ { N { '0ns' N; } }
      _default_Clk0_Timing_ { P { '0ns' D; '50ns' U; '80ns' D; } }
      _default_Clk1_Timing_ { P { '0ns' U; '50ns' D; '80ns' U; } }
      _default_Out_Timing_ { X { '0ns' X; } }
      _default_Out_Timing_ { H { '0ns' X; '40ns' H; } }
      _default_Out_Timing_ { T { '0ns' X; '40ns' T; } }
      _default_Out_Timing_ { L { '0ns' X; '40ns' L; } }
    }
  }
}
PatternBurst _burst_ { PatList {
  _pattern_ {
  }
}}
PatternExec {
  PatternBurst _burst_;
}
Procedures {

```

```

capture_CLK {
    W _default_WFT_;
    forcePI: V { _pi=\r13 # ; _po=\j \r9 X ; }
    measurePO: V { _po=\r9 # ; }
    pulse: V { CLK=P; _po=\j \r9 X ; }
}
capture_RSTB {
    W _default_WFT_;
    forcePI: V { _pi=\r13 # ; _po=\j \r9 X ; }
    measurePO: V { _po=\r9 # ; }
    pulse: V { RSTB=P; _po=\j \r9 X ; }
}
capture {
    W _default_WFT_;
    forcePI: V { _pi=\r13 # ; _po=\j \r9 X ; }
    measurePO: V { _po=\r9 # ; }
}

# Uncomment and modify the following to suit your design
# PRE_CLOCK_MEASURE Procedures {
# load_unload {
#     W _default_WFT_;
#     C { test_so=X; test_si=0; test_si2=0; test_so2=X; clk=0; tclk=0;
reset=1; test_se=1; }
#     Shift { W _default_WFT_;
#     V { _si=#; _so=#; CLK = P; }
#     }
# }
# TMAX GENERATED POST_CLOCK_MEASURE (Closer to DFTCompiler Procedures {
# load_unload {
#     W _default_WFT_;
#     C { test_si=0; test_si2=0; clk=0; tclk=0; reset=1; test_se=1; }
#     V { _so=#; }
#     Shift { W _default_WFT_;
#     V { _si=#; _so=#; clk=P; }
#     }
# }
}
MacroDefs {
    test_setup {
        W _default_WFT_;
        V { CLK=0; RSTB=1; }
    }
}
}

```

See Also

[add_clocks](#)

[add_scan_chains](#)

[add_scan_enables](#)

[Controlling Bidirectional Pins in STIL](#)

[Defining Circuit Initialization in STIL](#)

[Declaring Clocks](#)

[Defining PI Constraints in STIL](#)

[Defining Scan Chain Load/Unload in STIL](#)

[Defining Scan Chain Shifting in STIL](#)

[Defining Scan Chains in STIL](#)

[run_drc](#)

[set_drc](#)

Table Column Outside Table:

write_faults

Overview

Use this command to write fault data to an external file.

Syntax

```
write_faults
  filename [ instance_name | pin_pathname]
  [-stuck <0|1|01> | -slow <r|f|rf>] |
  [-class fault_class... | -summary | -all >
  [-collapsed | -uncollapsed]
  [-compress <gzip | binary>]
  [-profile] [-max d] [-replace]
```

Arguments

filename

Specifies the pathname and file to which the fault data is written.

instance_name

Specifies that faults associated with the selected [instance name](#) are written.

pin_pathname [-stuck <0|1|01> | -slow <r|f|rf>]

Selects faults associated with the specified [pin pathname](#) for writing. You can use the `-stuck` option to write only the stuck-at-1 or stuck-at-0 faults associated with the specified pin pathname, or both; or use the `-slow` option to write the slow-to-rise, slow-to-fall, or both to the specified file.

-class fault_class

Specifies that only faults associated with the specified [fault class](#) should be written.

Multiple occurrences of `-class` can be used, and this option can also be combined with an *instance_name*.

Note for Tcl mode: See the following example.

-summary

Writes a fault summary (rather than a fault list) to the file.

-all

Specifies that all fault classes in the current fault are selected for writing.

-collapsed | -uncollapsed

Selects whether the fault list written will show collapsed or uncollapsed faults. The default is the current setting as specified with the `set_faults` command. When `-uncollapsed` is selected or is the current setting of set faults, the equivalent faults follow the primary fault and are indicated by a double dash "--". This behavior can be adjusted with the `set_faults` command.

-compress <gzip | binary>

Specifies that the output file is to be written in compressed format and specifies the type of compression: GZIP or binary. The `binary` keyword creates a proprietary format that can be read only by TestMAX ATPG. The default is to write the file with no compression.

-profile

Selects a report of the number of faults detected per pattern in the external pattern set. The pattern set must be external and a prior `run_fault_sim -sequential` command must have been run. This option is primarily useful for fault grading of functional patterns. It creates report showing the pattern number and the number of new faults detected as well as the total cumulative detected faults. The `set_faults -report` command determines whether fault reporting is in terms of collapsed or uncollapsed faults. Patterns that do not detect faults are not printed in the report.

-max d

Specifies the maximum number of faults that are written. The default is to write all selected faults.

-replace

Indicates that the fault file is allowed to replace an existing file. The default is to not allow a replacement.

Allowed Command Modes

Test

Description

Use this command to write fault data to an external file. These faults can then be read back in as the source of a future fault list.

The fault list written can be a collapsed or uncollapsed list of faults. You can select the type of fault list by using the `set_faults` command or by explicitly specifying the type in the `write_faults` command. The uncollapsed list of faults includes the primary faults as well as faults equivalent to them. Equivalent faults are indicated by a fault class of "--" and always immediately follow the fault that they are equivalent to.

Examples

Tcl mode:

```
TEST-T > write_faults ud.au_tcl-test.flt -class {UD AU ND} -replace
```

Native mode:

```
TEST> set fault -report uncollapsed
TEST> write fault fault.summary -summary
TEST> write fault faults.all -all -replace
TEST> write faults faults.AU -class au -replace
TEST> wr fa faults.detected -cl ds -cl dA
TEST> write faults faults.NP -class np -verbose
TEST> wri fault selected.faults reg3 -class AN -class ut
```

See Also

[set_faults](#)
[report_faults](#)
[read_faults](#)
[Fault Classes](#)

[Fault Summary](#)

write_image

Overview

This command writes the TestMAX ATPG database to a file after a successful build or a successful DRC run.

Syntax

```
write_image
  file_name
  [-compress <gzip | binary | off>]
  [-design_view]
  [-garble]
  [-netlist_data]
  [-password string]
  [-replace]
  [-schematic_view]
  [-violations]
  [-nosubnets]
  [-nonet_pairs]
```

Arguments

file_name

Specifies the pathname of the file into which the database is written.

-compress <gzip | binary | off>

Specifies whether to compress the database. The default is gzip. The binary keyword creates a proprietary format that can be read only by TestMAX ATPG. The *off* keyword creates a file without compression.

-design_view

Enables schematic viewing in Design View of the GSV for password-protected and unprotected images. Note that the image must always be written using the *-netlist_data* option. The netlist data is required to draw the schematic with design names and related information. The default is not to enable schematic viewing.

To write a password-unprotected image for the GSV in Design View, execute the following command:

```
write_image design.image -design_view -netlist_data
```

To write a password-protected image for the GSV in Design View, execute the following command:

```
write_image design.image.passwr -password topsecret -schematic_view -
design_view -netlist_data
```

-garble

Obfuscates names to a single hierarchy level name before the image is written. Instance names become of the format *u###*, net names become *n###* and module names become *m###*, where "###" is an integer number of any length. Some names are not garbled, such as primary IOs, to allow some commands to work with the original names. Command

scripts might have to be modified to use the garbled name. Garbled names can be translated to the original names by using the `report_nets` or `report_instances` commands if an ungarbled design database is read by TestMAX ATPG. This option can only be used in TEST mode. It is not supported for physical data (you cannot simultaneously specify both the `-garble` and `-physical` options of the `write_image` command).

-netlist_data

Adds net and instance names to the image files. By default, these names are not stored in the image file. Commands involving net and instance names, such as `report_nets`, `report_instances`, or `set_diagnosis -report_net_data`, are functional after TestMAX ATPG reads in an image with netlist data. Using this option will increase the image file size. If you want to protect IP information for certain design data, such as net names, do not use this option.

-password *string*

Creates a password-protected secure image, where *string* is the password to be used for encryption. Commands specified by previous `set_commands -secure` commands are designated as allowed in secure state. The encrypted image is compressed by default. You can use double quotation marks (" ") to enclose your password if you desire. This password must be used to read the secure image. The output of this command lists the commands that are allowed when that image is read. The default is to create unprotected image files. Settings from commands such as `set_atpg` might remain unchanged when a secure image is read by `read_image`.

-replace

Specifies that the image file is allowed to replace an existing file. The default is not to replace an existing file and generate an error message indicating the file already exists.

-schematic_view

Enables schematic viewing in primitive view only when used with the `-password` option. Note that primitive view minimizes the image file size and protects the IP by not displaying certain design data, such as net names and intermediate levels of hierarchy. To write a password-protected image for the GSV in Design View you must also include the `-design_view` and `-netlist_data` options. The default is not to enable schematic viewing.

-violations

Stores DRC violations for the C, D, L, S, X, and Z rules.

-nosubnets

This option avoids saving the subnet data to the image. If the subnet data has been read, and after that, an image was written, then the data is stored by default.

-nonet_pairs

This option prevents all bridging net pairs used for physical diagnostics (specified by the [read_layout -likely_pairs](#) command) from being saved and included in the output image. The net pairs data is read by default, and is saved and included in the output image. The saved net pairs are intended for physical diagnostics only.

Allowed Command Modes

DRC, Test

Description

This command writes the TestMAX ATPG database to a file after a successful build or a successful DRC run.

Note the following:

This command does not write patterns to the image file.

You must use either or both the `-netlist_data` and `-design_view` options of the `write_image` command to create image files that generate parallel patterns. Otherwise, the image file incurs a loss of design information that affects the patterns, and TestMAX ATPG issues an [M269](#) message.

Examples

```
TEST> write_image i044_image.dat -replace
End write image: #bytes=1830406, CPU time=3.00 sec.

TEST> write_image image_enc.gz -password secret
End write image: #bytes=11403, CPU time=0.03 sec.
Secure image created with schematic viewing disabled and \
  allowing the following commands:
help
run_diagnosis
set_patterns
run_simulation
```

See Also

[Binary Image Files](#)

[Creating and Reading Image Files](#)

[Creating a Non-Secure Image File](#)

[Creating a Secure Image File](#)

[read_image](#)

[set_diagnosis](#)

write_netlist

Overview

Use this command to write netlist data to an external netlist file in a specified format.

Syntax

```
write_netlist
  file_name [-replace] [-top top_name]
  [-format <edif [external] | verilog | verilog_tmax | verilog_vcs> ]
  [-stop [edif | verilog | vhdl | design_level]]
  [-compress <gzip | binary>]
```

Arguments

file_name

Indicates the pathname of the file in which the netlist data is written.

-replace

Indicates that the netlist file is allowed to replace an existing file. The default is to not allow a replacement. Attempting to write over an existing file without the -replace option results in an error.

-top top_name

Specifies the name of the top-level module whose netlist data is to be written. If a top module is specified, only that module and all subsequently referenced modules are included in the output file. Any other modules in memory but not referenced by the top module or its descendants are not included. If a top module is not specified, the default is to write all modules.

-format <edif [external] | verilog | verilog_tmax | verilog_vcs>

Selects the netlist output format. If you select `edif`, no external library is written, unless you specify the external option. The default output format is Verilog.

The `verilog` and `verilog_tmax` arguments have the same effect: the netlist is written in most compact form, using TMAX netlist primitives (such as `_AND`, `_DFF`, `_MEMORY`, and so forth.). The `verilog_vcs` argument writes a standard Verilog netlist, using only Verilog primitives, whenever possible. For example, latches and flops are written as UDPs. In some cases, the `verilog_vcs` option cannot write out an instance in standard Verilog, and will write as Verilog_Tmax instead. An [M467](#) message is issued each time this occurs. M467 messages can be as follows:

```
"Instance <inst_name> of module <mod_name> written in TMAX-style Verilog
(M467)"
```

```
"Memory instance <inst_name> of module <mod_name> written in TMAX-style
Verilog (M467)."
```

```
"_BUSK0 in module <mod_name> written in TMAX-style Verilog (M467)."
```

```
"_BUSK1 in module <mod_name> written in TMAX-style Verilog (M467)."
```

-stop [edif | verilog | vhdl | design_level]

This is an additional option for blocking certain segments of the design from appearing in the written file. For example, if the design is in EDIF and the libraries are in Verilog, then `-stop verilog` can be used to write the design but not the library modules. Use of `-stop design_level` blocks the writing of any low-level cell considered a library level or "design" cell. Without the use of any `-stop` option, no filtering of modules is performed.

`-compress <gzip | binary>`

This option specifies that the file is to be written in compressed format and specifies the type of compression, either GZIP or binary. The default is to write the file without compression.

Allowed Command Modes

All

Description

Use this command to write netlist data to an external netlist file in a specified format. You can use compression to minimize the size of the file.

Note the following:

Net connections added with the `add_net_connections` command are not reflected in the output netlist.

Do not use the `write_netlist` command as a tool to convert netlists between Verilog, EDIF, and VHDL formats, or to produce legal Verilog syntax for some other tool. The netlist written by this command is intended for ATPG tools only and does not contain many of the items which are in the original netlist or module definitions. For example, any behavioral module, with the exception of recognized RAM/ROM definitions, is converted to [black box](#) modules. All timing information is stripped away. Also, "specify" blocks are stripped away. Any undefined modules become black box definitions. Verilog UDPs are converted to gate-level functional netlists using ATPG primitives.

The `write_netlist` command outputs primitives only. If the design behavior depends on attributes, for example the `reset_dominance` attribute of the `_DFF` or `_DLAT` primitives, then the circuit behavior will change. In this case, use the [write_image](#) command instead.

Examples

```
TEST> write_netlist mux81.v -replace -top MUX81
      End writing Verilog netlist, CPU_time = 0.20 sec, File_size = 500
```

See Also

[read_netlist](#)
[set_netlist](#)

write_patterns

Overview

Use this command to write ATPG patterns to a file in one of the supported formats: FTDL, STIL, TDL91, TSTL2, WGL, WGL_FLAT, and the ATPG tool's proprietary binary.

Syntax

```
write_patterns filename
  [-cellnames <internal | verilog | parallel | type | module | garbled>]
  [-compress <gzip | binary>]
  [-compressor_based]
  [-cycle_count | -nocycle_count]
  [-exclude <setup | repeat_setup | patterns | chain_test | atpg_patterns |
all>]
  [-first d] [-last d]
  [-format <binary | ftdl | tdl91 | tstl2 | wgl | wgl_flat | stil | stil99>]

  [-config_file cfilename]
  [-internal | -external]
  [-internal_force_clocks | -nointernal_force_clocks]
  [-internal_scancells | -nointernal_scancells]
  [-longtokensplit | -nolongtokensplit]
  [-measure_forced_bidis]
  [-nocompaction]
  [-nooverlap_load]
  [-occ_load_split]
  [-order_pins]
  [-pad_character < 0 | 1 | x >]
  [-patinfo | -nopatinfo]
  [-patname pat_block_name]
  [-replace]
  [-scanstructures <-all | min>]
  [-serial | -parallel]
  [-sorted | -reorder file | -type <basic_scan
| clock_on_measure | fast_sequential | full_sequential>]
  [-split d]
  [-stil | -wgl]
  [-top_scanports | -notop_scanports]
  [-unified_stil_flow | -nounified_stil_flow]
  [-use_delay_capture_start number]
  [-use_delay_capture_end number]
```

Arguments

filename

Specifies the base path name of the file in which TestMAX ATPG writes the patterns. If the file already exists, TestMAX ATPG displays an error unless you use the `-replace` option.

Note that the *filename* must be the first argument specified on the `write_patterns` command line.

-cellnames <internal | verilog | parallel | type | module | garbled>

Controls the format of the scan cell information in the `ScanStructures` block of a STIL file.

When `-cellnames internal` is used, you can specify scan cell names that match the names reported by the `report_scan_cells` command.

When `-cellnames type` is used for STIL (but not STIL99) output, the `ScanStructures` block contains `ScanCellType` statements with compact references to the scan cell information.

When `-cellnames module` is used, the `ScanCellType` statement uses the cell module type in the design (with additional references to scan-in and scan-out connections) and each scan cell identifies the specific module.

When `-cellnames verilog` is used, the generated names are specified with Verilog syntax.

When `-cellnames parallel` is used, the names are the same as `-cellnames verilog`, with an additional reference to the scan-input port of these cells.

When `-garbled` is used, the internal scan cell names (instance-level names only) are generated in a garbled format. This enables TestMAX ATPG to read netlists containing garbled names without affecting subsequent pattern usage. Note that garbled names are not appropriate for parallel simulation and errors are generated if a pattern file is used in this context.

The naming constructs used for the `type` or `module` options require 1450.1 constructs and are supported for the STIL (not STIL99) format only. Checks are not made on the module names as appropriate for a STIL flow, and the `-cellnames module` option may generate STIL files that are not appropriate for other translators. For STIL99 files and STIL files written with `-parallel` or `-unified_stil_flow`, the default is `-cellnames parallel`. If parallel simulation failures are reported in certain situations (see the description of the [M404](#) message for more information), you can also try using `-cellnames type` for STIL (not STIL99) parallel files since this format supports the [set_drc](#) `-dslave_remodel` context for parallel simulation.

For more information, see "[Output Examples for write_patterns -cellnames](#)."

-compress <gzip | binary>

Specifies the compressed format (gzip or binary) of the output file that TestMAX ATPG writes. Binary compression is a proprietary format of TestMAX ATPG. The default is to write the file without compression. If you select the `gzip` format, add the `.gz` suffix to the filename. You cannot use this option with the `Ftdl`, `TDI91`, or `TSt12` option.

-compressor_based

This option, which is used in conjunction with the `-format binary` option, writes out a parallel mode pattern set which is read back into serial mode when running TestMAX ATPG on serializer designs. It is used only for translating parallel mode patterns to serial mode patterns in DFTMAX compression. For details on this flow, see "Translating Parallel Mode Patterns to Serial Mode Patterns" in the *DFTMAX Compression User Guide*.

-config_file cfilename

This option sets the user configuration file to control options in the Ftdl, TDI91, and TSt12 output formats. If you do not set this option, TestMAX ATPG uses a default configuration file from the \$SYNOPSYS/auxx/syn/ltran directory. To create a custom configuration file, copy the default file and modify the user-editable fields and controls. For details, see [Customizing Ltran Output](#).

-cycle_count | -nocycle_count

The cycle count is generated by default when the `write_patterns -format stil | stil99` command is specified. If this option causes problems, you can disable it using the `-nocycle_count` option.

The cycle count is only generated when writing STIL patterns. There are two values reported after the patterns are written. The first value identifies the number of V statements encountered in these patterns; this includes counts of the V statements contained in each executed procedure for each call to that procedure. The second value (number of cycles) identifies the number of flattened cycles produced by these operations, which includes the number of cycles in the expanded Loop and Shift statements.

-exclude <setup | repeat_setup | patterns | chain_test | atpg_patterns | all>

Selects from the following pattern types to exclude from the output:

`setup` — Patterns for the test_setup macro.

`repeat_setup` — Patterns for the test_setup macro when you use the `-split` option. Note that this option should not be used for WGL patterns.

`patterns` — Includes the test_setup vector; excludes ATPG patterns and chain tests. When chain tests are present, they start at ATPG pattern 0. These patterns are also controlled by options of the `set_atpg` command.

`chain_test` — Patterns associated with the chain test.

`atpg_patterns` — All patterns NOT associated with the chain test (converse of the `chain_test` option).

`all` — Excludes all pattern types. You can repeat this option to exclude multiple pattern types.

You can use a list with the `-exclude` option to exclude multiple pattern types. Excluding patterns with compressed designs might result in errors if there are dependencies between the patterns.

-first d

Specifies the first pattern number that TestMAX ATPG writes. The default is to begin with pattern 0. Note: For Full-Sequential patterns, this option might cause simulation mismatches.

-format <binary | ftdl | tdl91 | tst12 | wgl | wgl_flat | stil | stil99>

Specifies the output format for the patterns that TestMAX ATPG writes. The default is `binary`.

The `binary` option writes patterns in a TestMAX ATPG proprietary binary format. TestMAX ATPG can write and read this format. To read these pattern files, use the `set_patterns -external` command.

The `ftdl` option writes patterns in the Fujitsu TDL pattern format. You can specify an optional user configuration file using the `-config_file` option. Note: The FTDL pattern format is supported for write only; TestMAX ATPG cannot read this pattern format.

The `tdl91` option writes patterns in the Texas Instruments TDL91 pattern format. You can specify writing the patterns in a parallel format for parallel TDL simulation. You can specify an optional user configuration file using the `-config_file` option. The Texas Instruments TDL91 pattern format is supported for write only; TestMAX ATPG cannot read this pattern format.

The `tstl2` option writes patterns in the Toshiba TSTL2 pattern format. An optional user configuration file can be specified using the `-config_file` option. The Toshiba TSTL2 pattern format is supported for write only; this pattern format cannot be read back into TestMAX ATPG.

The `wgl` option writes patterns in TSSI's Waveform Generation Language (WGL) pattern format. Note: The `set_wgl` command controls many aspects of creating WGL pattern output. The subset of the WGL language written by TestMAX ATPG can also be read back in.

The `wgl_flat` option produces a WGL pattern file in which the scan loads have been serialized (flattened) into a series of "vector()" statements. The WGL file contains only vector() statements and does not contain any "scan()", scan cell, scan states, or scan chain information.

The `stil` option writes patterns in the official IEEE Std. 1450.1 Standard Test Interface Language (STIL) for Digital Test Vectors format. TestMAX ATPG can both write and read this format; however, it supports only a subset of the language it writes for reading back in.

The `stil99` option writes patterns in the official IEEE-1450.0 Standard Test Interface Language (STIL) for Digital Test Vectors format. This format can be both written and read, but only the subset of the language written by TestMAX ATPG is supported for reading back in.

Note: You must use a 1450.0-compliant DRC procedure as input to write output in the `stil99` format.

Selecting `-format stil` or `stil99` generates a STIL file which contains a name in the format `<pfile>.<ext>`, where you specified `write_patterns <pfile>.<ext>`. The default STIL testbench is serial.

`-internal` | `-external`

Specifies whether to write patterns from the internal or external pattern set. The initial default is to use the internal pattern set, but this can be modified by the [set_patterns](#) command.

`-internal_force_clocks` | `-nointernal_force_clocks`

Controls the generation of additional events in the pattern data. If these events are present for the design context, then they are required to properly drive certain internal clocks during VCS or external simulation. The default is `-nointernal_force_clocks`.

`-internal_scancells` | `-nointernal_scancells`

Controls the generation of data for the `ScanCell` and `ScanClock` sections of a STIL file for internal chains if this data is not already provided. For non-compressed designs, the default behavior is not to generate the scan cell and scan clock data (the `-nointernal_scancells` option). For compressed designs, the default is to generate the scan cell and

scan clock data (`-internal_scancells`). If the scan cell data is required (for example, if the `-parallel` option is specified), the internal scan cell data is written irrespective of this option setting and [M479](#) messages are generated.

`-last d`

Specifies the last pattern number for the patterns to be written. The default is to end with the last available pattern.

`-longtokensplit` | `-nolongtokensplit`

The default, the `-longtokensplit` option, avoids excessively long lines in the STIL file by splitting long scan cell names into several tokens using the "." (point) STIL statement. This default behavior generates lines containing a maximum of 1024 characters. The `-nolongtokensplit` option avoids splitting scan cell names and can generate lines longer than 1024 characters. This option is intended to support testers that cannot interpret the "." (point) STIL statement.

`-measure_forced_bidis`

By default, no patterns written will attempt to force and measure bidirectional ports in the same tester cycle. Selecting this option allows writing patterns which can attempt to both force and measure in the same tester cycle. **Note:** You should not use this option if you are writing WGL output or if your target tester will not support force/measures on the same pin in the same cycle.

`-nocompaction`

Applies to STIL format only. By default, compaction is allowed in STIL by replacing repeated character strings using the '\r NN' construct. Using this option disallows this type of compaction.

`-nooverlap_load`

Applies to STIL format only. When patterns are written, the scan chain loading for pattern N+1 is overlapped with the scan chain unloading of pattern N. By specifying `-nooverlap_load`, the patterns are written so that no overlapping occurs. Total number of tester cycles will essentially double when `-nooverlap_load` is used. The default is to overlap load and unload operations. Note that this option is not supported for Adaptive Scan Technology compression patterns.

`-occ_load_split`

This option, which is used in the internal clocking procedures feature, splits patterns into a separate file for each clocking procedure. Note that a new pattern file is saved each time a clocking procedure changes from one pattern to the next, which can result in a large number of pattern files. Because of this, you should use the `-occ_load_split` option only in combination with the [set_atpg](#) `-group_clk_constraints` command. For more information, see "[Using Internal Clocking Procedures](#)."

`-order_pins`

Selects a special ordering of port data for pattern outputs. The default order of pattern data follows the order in which the ports are defined in the design's top module. Specifying the `-order_pins` option causes the order of pattern data to be inputs first, then bidirectional ports, then outputs. This option is supported only for WGL patterns.

`-pad_character < 0 | 1 | x >`

Selects the padding character to be used on standard WGL patterns when shorter scan chains are padded up to the same size as the longest scan chain, or during the last chain unload operation when padding data is shifted in while checking the last pattern shifted out. The default padding is 0. There is a separate control for WGL patterns found under the `set_wgl` command for enabling/disabling padding when writing WGL patterns.

-patinfo | -nopatinfo

Applies to STIL format only. The default is `-nopatinfo`. If `-patinfo` is specified, a `<filename-ext>.patinfo` file is generated when STIL patterns are written.

-patname *pat_block_name*

Replaces the default STIL pattern block name `"_pattern_"` with the name specified to this option. Affects `-format stil` or `stil99` files only.

-replace

Specifies that the pattern file is allowed to replace an existing file. The default is not to replace an existing file and generate an error message indicating the file already exists.

-scanstructures <all | min>

Controls the generation of scan chain data in the `ScanStructures` block of a STIL file. The default setting, `all`, generates all supported fields for all applicable scan chain data. If you specify the `min` keyword, the scan chain data is truncated for compressed designs that reference internal scan input or scan output ports (internal scan chains). This protects the identification of internal design elements.

The following set of examples shows how the `-scanstructures` option affects the contents of a generated STIL file. The first example shows the scan chain data using the default setting (`-scanstructures all`):

```
ScanChain "1" {
  ScanLength 11;
  ScanIn "core_i/ctl/U199/ZN";
  ScanOut "core_i/ctl/ctl_de_reg_2_/Q";
  ScanInversion 1;
  ScanCells ! "des_unit.core_i.ctl.ctl_dd_reg_0_.SI" "des_unit.core_
i.ctl.ctl_dd_reg_1_.SI"
  "des_unit.core_i.ctl.ctl_dd_reg_2_.SI" "des_unit.core_i.ctl.ctl_dd_reg_
3_.SI"
  ...
  "des_unit.core_i.ctl.ctl_de_reg_0_.SI" "des_unit.core_i.ctl.ctl_de_reg_
1_.SI"
  "des_unit.core_i.ctl.ctl_de_reg_2_.SI" ;
  ScanMasterClock "clk_st" ;
}
```

```
ScanChain "2" {
  ScanLength 11;
  ScanIn "core_i/ctl/U198/ZN";
  ScanOut "core_i/ctl/key_dd_reg_5_/Q";
  ScanInversion 1;
  ScanCells ! "des_unit.core_i.ctl.ctl_de_reg_3_.SI" "des_unit.core_
i.ctl.ctl_de_reg_4_.SI"
  "des_unit.core_i.ctl.ctl_de_reg_5_.SI" "des_unit.core_i.ctl.ctl_de_reg_
5_.SI"
```

```

6_.SI"
...
"des_unit.core_i.ct1.key_dd_reg_5_.SI" ;
ScanMasterClock "clk_st" ;
}

```

If the `-scanstructures min` option is specified, the generated scan chain data is truncated, as shown in the following example:

```

ScanChain "1" { }
ScanChain "2" { }

```

Note the following:

All `ScanIn` and `ScanOut` references in the `Signals` group that identify pseudo signals are also removed.

The `-scanstructures min` option conflicts with parallel simulation requirements, and is disabled if parallel simulation options are set. In this case, an [M479](#) message is reported.

If you are using a mixture of design styles, and want to remove internal names, you should specify both the `-scanstructures min` and `-nointernal_scancells` options.

-serial | -parallel

You can use the `-serial` or `-parallel` options in combination with the `-format stil` or `-format stil99` options, but this does not affect the patterns that are written. The `-serial` option adds a flag to the pattern file to force serial Verilog simulation, but this can be done more easily by writing only a single parallel STIL pattern and then using `+define+tmx_serial` option at simulation time to do the serial simulation.

-sorted | -reorder file | -type <basic_scan | clock_on_measure | fast_sequential | full_sequential>

The `-sorted` option writes the patterns in a predefined order, from low complexity to high. The definition of complexity considers the number of scan loads and capture procedures involved. Fewer scan loads and fewer clocks are considered less complex. The `-sorted` command is not supported for adaptive scan technology compression options.

The `-reorder` option accepts a user-specified ASCII file that specifies the output order of patterns. Each line must start with an integer that specifies the pattern number to be written (remember that pattern numbers start with 0). These numbers form a column in the file. Additional columns in this file are ignored. The file can contain blank lines and comment lines. For example, line 1 containing a 10 and line 2 containing a 5 would produce the pattern order 10,5.

The `-reorder` option is not supported for adaptive scan technology compression options.

The `report_patterns -all -type` command generates a report that can be sorted or edited to create this reorder file. The following example was created by using the output of a `report_patterns -types` command to select a representative sample of patterns that exercise at least one of each clock in the design. This type of pattern block would be most valuable for performing serial simulation and checking for timing problems.

The `-types` option limits the patterns written to the type of pattern specified, either Basic-Scan, Fast-Sequential, Full-Sequential, or Clock_on_measure.

```

#
# --- example input file for -ORDER option produce by
# editing the output of 'report_patterns -all -types'
#
0 basic_scan 1 1 - -

```

```

9 basic_scan(COM) 1 1 - -
15 fast_sequential 1 1 - RST
30 fast_sequential 2 2 - clk1,clk2
16 fast_sequential 1 1 - clk2
7 basic_scan 1 1 master clk3
2 basic_scan 1 1 - clk5
34 fast_sequential 3 0 - clk6,clk6
41 fast_sequential 4 4 - WRT,READ,clk1
45 fast_sequential 4 3 - WRT,WRT,READ,clk1

```

-split d

Using this option causes the pattern output file to be split into multiple files of d patterns each, as long as d is smaller than the total number of patterns. The pattern filenames are created by inserting "_0", "_1", "_2", and so on before the last period "." found in the user-specified filename. By default, each pattern file will contain the patterns associated with the test_setup procedure, unless the `-exclude repeat_setup` or `-exclude setup` options are used. Only the first pattern file contains the chain test patterns.

For Full-Sequential patterns, the `-split` option is only useful for tester (ATE) memory requirements, and loading the files in sequential order is mandatory.

-stil | -wgl

This switch is typically used when specifying the TDL91, TSTL2, or FTDL pattern formats with the `-format` option. It creates either a STIL or WGL pattern format intermediate file that is passed to the external translator. A STIL intermediate file is created by default. In most cases, writing a STIL file is much faster than writing WGL file; this can save minutes of time for large pattern files. For more information on generating and using the intermediate file, see "[FTDL, TDL91, TSTL2 Configuration Files](#)."

-top_scanports | -notop_scanports

Controls the naming context of the internal scan chain ScanIn and ScanOut references in the ScanStructures block of the STIL pattern files for compressed designs. The `-notop_scanports` option (the default) uses the names reported from the [report_scan_chains](#) command and generates a STIL file accepted by the [set_patterns](#) `-external` command. In some library modeling contexts, the scan chain names directly reference Verilog primitives in the design. In this case, you can use the `-top_scanports` option to generate STIL pattern files that can be simulated with MAX Testbench. If the library used by TestMAX ATPG is sensitive to Verilog primitive references, [V12](#) messages are generated. For most designs, the `-top_scanports` option has no effect, and the [V12](#) messages are not reported.

If a [V12](#) message is reported, you can still use the `-top_scanports` option. Make sure this option doesn't conflict with a STIL file that is read back into TestMAX ATPG (for example, for a diagnostic operation) or used for parallel simulation in the pattern validation flow.

-unified_stil_flow | -nounified_stil_flow

The `-unified_stil_flow` option (the default) generates parallel STIL patterns with a precomputed `n_shifts` value for both regular scan and scan compression logic. This format is always tester-ready and is not affected by any other specified options. The `-nounified_`

`stil_flow` option is obsolete and should never be used because the resulting STIL patterns are not usable on the tester.

-use_delay_capture_start *number*

This option inserts calls to the `delay_capture_start` procedure at the start of each capture option in the patterns. The *number* specification (an integer value) is required and controls the number of calls made to this procedure. For most environments, the value 1 is appropriate. If you want multiple instances of this wait procedure, this value would be greater than 1.

The `delay_capture_start` procedure is created if it was not defined in the input STIL procedure file. Note that the `-use_delay_capture_start` option must be specified for each `write_patterns` command that requires the presence of this delay. This includes situations in which patterns that contain these procedures are read back. Because this procedure is not preserved in the pattern data, it must be present with any `write_patterns` command that requires the presence of these wait procedures. For additional information on the `delay_capture_start` procedure, including the default specification, see "[Using the delay_capture_start Procedure](#)."

-use_delay_capture_end *number*

This switch adds extra cycles after the capture procedure to allow capture clocks to propagate. The *number* specification (an integer) is the number of `delay_capture_end` procedures that are added. If this procedure is not defined in the STIL procedure file, it is created automatically with one cycle using the default waveform table.

When on-chip clocking is used, the `set_atpg -min_ateclock_cycles` command should be specified to provide extra delay for clock propagation. By extending the capture procedure, it allows better support for ATPG and simulation. The `-use_delay_capture_end` switch is needed only for ATPG runs in which on-chip clocking is absent or bypassed.

For additional information on the `delay_capture_end` procedure, including the default specification, see "[Using the delay_capture_end Procedure](#)" in the *TestMAX ATPG User Guide*.

Allowed Command Modes

Test

Description

Use this command to write ATPG patterns to a file in one of the supported formats: FTDL, STIL, TDL91, TSTL2, WGL, WGL_FLAT, and the ATPG tool's proprietary binary. Optionally, the file can be written using one of two forms of compression: GZIP or binary.

Using FTDL, TDL91, TSTL2, or WGL_FLAT format causes TestMAX ATPG to invoke a separate translation process, which opens a new window in which the translation process runs independently. The pattern formats FTDL, TDL91, and TSTL2 are *write only*; they cannot be read back in. This is important if you want to use the diagnostics capability of TestMAX ATPG to assist in diagnosing device failures at the tester. If supporting diagnostics is part of your flow, you should also save the same patterns in a format that can be read back in, such as STIL or binary.

When the `-format [-stil | stil99]` option is specified, the default (if the `-parallel [n_shifts]` or `-serial` options are not specified) is the unified STIL flow that supports parallel simulation with a precomputed `n_shifts` value. This format is always tester-ready when no options are specified. For

compression scan designs, if the `-parallel` option is specified, the dual STIL parallel format is used (that is, not unified STIL). This format is not tester-ready.

If the patterns are written in STIL format, an additional information line is printed, as shown in the following example:

```
Patterns written reference <n> V statements, generating <m> test cycles
```

The first part of this line, `Patterns written reference <n> V statements`, indicates the number of V statements contained in the patterns. The second part of the statement, `generating <m> test cycles`, means that the V statements are found in Loop and Shift constructs, and the test execution creates <m> number of executed (flattened) test cycles.

Note: Launch and capture events are pertinent only to transition and path delay fault environments. If the fault model is set to the default model (stuck), then the presence of launch and capture events are likely to be dropped. TestMAX ATPG will attempt to maintain this information when possible. But because of the variety of flows and the ability to process patterns generated for one fault model, under a different model (for instance, re-grading transition patterns under a stuck model), care must be exercised if this information needs to be maintained. Before the write patterns operation is executed in the file that reads back the binary patterns, add the [set_faults](#) `-model transition` command. Then the launch and capture events will remain across all outputs.

Examples

```
TEST> write_patterns pat.bin -format binary -last 49 -replace
End writing 50 patterns, CPU_time = 0.72 sec, File_size = 28692
```

```
TEST> write_patterns pat.stil -format stil -last 19 -compress gzip -rep
Patterns written reference 1050 V statements, generating 163221 test cycles
End writing file 'pat.stil' with 20 patterns, File_size = 81002, CPU_time =
0.34 sec.
```

```
TEST> write_patterns pat.wgl -format wgl -last 19 -comp gzip -replace
End writing 20 patterns, CPU_time = 0.43 sec, File_size = 221926
```

See Also

[Defining PI Equivalents in STIL](#)

[report_patterns](#)

[set_atpg](#)

[set_patterns](#)

[set_drc](#)

[set_wgl](#)

[Troubleshooting Simulation Failures](#)

write_physical_db

Overview

This command creates and validates the PHDS database. You can use options of the `set_physical_db` command to perform only a PHDS validation run or only a PHDS creation run.

Syntax

```
write_physical_db
  [-replace]
  [-verbose]
```

Arguments

-replace

Overwrites any previously loaded device with the same name and version.

-verbose

Prints an additional set of information messages for the LEF/DEF reader and Yield Explorer PHDS loader.

Description

This command starts a process that translates a LEF/DEF database to Yield Explorer YDF format, loads the resulting collection of files into a PHDS database, and validates the database based on a set of [Y rules](#). You can load multiple devices into the same PHDS database.

Before using the `write_physical_db` command, use the `set_physical_db` command to specify the location of the LEF/DEF files and the output directory for the PHDS database. You can also use the `-novalidation` option to create only the PHDS database without validating it. Alternatively, you can use the `-nocreate_phds` option to only validate the PHDS database.

For more information on using the `write_physical_db` command, see "[Creating and Validating the PHDS Database for Physical Diagnosis](#)" topic.

Example

```
write_physical_db -replace
Writing Physical Database...
LEF input directory : ./lef
DEF input directory : ./def
Top DEF file name : top_design.def
PHDS output directory: ./phds
Device name : DES
Device version : 4
Running PHDS validation...
PHDS validation completed successfully.
```

```
-----
Validation Summary Report
```


Warning: Rule Y18 (DEF Without Corresponding LEF) was violated 2 times.
There were 2 violations that occurred during Validation process.
Running PHDS creation...
PHDS creation completed successfully.
Total_time = 36.76

See Also[set_physical_db](#)[open_physical_db](#)[close_physical_db](#)

Table Column Outside Table:

write_simtrace

Overview

This command assists in debugging ATPG pattern mismatches found during Verilog simulation.

Syntax

```
write_simtrace
  filename
  [-gate netname_or_id]
  [-length int]
  [-replace]
  [-scan chainname:cellnumber]
```

Arguments

filename

Specifies the name of the output file that defines a standalone module containing Verilog debug statements.

-gate netname_or_id

References a single net, device port, or module name that is monitored during simulation. The reference can be a TestMAX ATPG name or gate id. If the reference is a single net, then the monitor will contain the name of the object (subject to a length restriction of 20 characters or as specified by the `-length` option), followed by a colon, and a single bit that is the state of that net at the specified time. This is terminated by a semicolon. An example would be: "ABC.DI:0;". If the reference is to a module, then the output of that module must be displayed first in the format above, followed by a comma instead of the semicolon, and all the named inputs of the module are listed after an "INs:" identifier. The inputs are identified by port-name only (not the full path name), and input ports that are constant values or have no name are not displayed. A semicolon follows the last input port of the module.

-length int

Specifies the length of the names as they appear in the output statements. By default, names are truncated to the last 20 characters; this option allows you to specify longer or shorter names. This option is position-dependent and can be used multiple times; you may place this option between sequences of `-scan` and `-gate` options and it will affect only the remaining options.

-replace

Indicates that the output file is allowed to replace an existing file. The default is to not allow a replacement.

-scan chainname:cellnumber

Identifies the scan chain and cell that is monitored during simulation. This option requires a chain name, followed by a colon, and the bit index in that chain. The output of the cell is indicated first, followed by ", INs:", followed by all named, non-constant input ports of that cell. In the output region, if there is a scan cell before this cell, then the state of that

previous cell will also be monitored, and it is placed in parenthesis before the output state of this cell. Likewise, if there is a scan cell after this cell, then the output of the next cell is also displayed, with parenthesis around that state value as well. For example, a scan cell number 1 of chain c1 can be displayed as: "c1:1 (0)0(1) INs: DI:0 SE: 0 D:1;".

Description

This command assists in debugging ATPG pattern mismatches found during Verilog simulation. It facilitates the creation of a Verilog module by adding simulation data to be used for comparison with TestMAX ATPG.

The output file created by the `write_simtrace` command contains Verilog syntax to define a standalone module that contains debug statements to invoke Verilog operations referencing nodes specified by the `-scan` and `-gate` options. Because this is a standalone module, it references these nets as instantiated in the simulation through the testbench module; there are dependencies on these references based on the naming convention of the top module in the testbench module. You may specify multiple occurrences of the `-scan` and `-gate` options. The order of these options is important, as the defined order is the order placed in the generated file.

After running this command, if all nodes specified were found and the file was written as expected, TestMAX ATPG will print the following:

```
End writing file 'filename' referencing integer nets, File_size = integer
```

This statement identifies how many nets were placed in the output file to be monitored. Note that the file name is not include the path information.

The `write_simtrace` command is intended to be used as part of an iterative debugging flow. "

write_testbench

Overview

This command eases the generation of an equivalent Verilog testbench by allowing you to run MAX Testbench without leaving the TestMAX ATPG environment.

Syntax

```
write_testbench
  -input [stil_filename | {-split_in \{list_of_stil_files_for_split_in\}}]
  -output testbench_name
  [-replace]
  [-config_file config_filename]
  [-parameters {list_of_parameters}]
```

Arguments

-input [*stil_filename* | {-split_in \{*list_of_stil_files_for_split_in*\}}]

The *stil_filename* argument specifies the path name of the previous TestMAX ATPG-generated STIL file requested by the equivalent Verilog testbench. You can use a previously generated STIL file as input. This file can originate from either the current session or from an older session using the `write_patterns` command.

The following syntax is used for specifying split STIL pattern files as input (note that backslashes are required to escape the extra set of curly braces):

```
{-split_in \{list_of_stil_files_for_split_in\}}
```

The following example shows how to specify a set of split STIL pattern files:

```
write_testbench -input {-split_in \{patterns_0.stil patterns_1.stil\}} -
output pat_mxtb
```

-output *testbench_name*

Specifies the names used for the generated Verilog testbench output files. Files are created using the naming convention `<testbench_name>.v` and `<testbench_name>.dat`.

-replace

Forces the new output files to replace any existing output files. The default is to not allow a replacement.

-config_file *config_filename*

Specifies the name of a configuration file that contains a list of customized options to the MAX Testbench command line. See "[Customized MAX Testbench Parameters Used in a Configuration File with the write_testbench Command](#)" for a complete list of options that can be used in the configuration file.

-parameters {*list_of_parameters*}

Enables you to specify additional options to the MAX Testbench command line. See "[MAX Testbench Command-Line Parameters Used with the write_testbench Command](#)" for a complete list of parameters you can use with the `-parameters` option.

If you use the **-parameters** option, make sure it is the last specified argument in the command line, otherwise you might encounter some Tcl UI conversion limitations.

A usage example for this option is as follows:

```
write_testbench -parameters { -v_file \"design_file_names\" -v_lib  
\"library_file_names\" -tb_module module_name -config_file config1}
```

Note the following:

All the parameters must be specified using the Tcl syntax required in the TMAX shell. For example: -

```
parameters {param1 param2 -param3 \"param4\"}
```

quotation marks must have a backslash, as required by Tcl syntax, to be interpreted correctly and passed directly to the MAX Testbench command line.

Parameters specified within a `-parameters {}` list are order-dependent. They are parsed in the order in which they are specified, and are transmitted directly to the MAX Testbench command line. These parameters must follow the order and syntax required for the MAX Testbench command line.

Allowed Command Modes

All

Description

This command eases the generation of an equivalent Verilog testbench by allowing you to run MAX Testbench without leaving the TestMAX ATPG environment.

When you use the `write_testbench` command, there are many pattern files to translate, or you might be using a batch-mode environment. In this case, use the [LTRAN_SHELL](#) command. This command uses a shell version of the `stil2verilog` script even if Xwindows is set up.

Example

```

TEST-T > ...
TEST-T > write_patterns pat.stil -formal stil -serial -replace
Patterns written reference 7158 V statements, generating 18624 test cycles
End writing file 'pat.stil' with 127 patterns, File_size = 207207, CPU_time
= 0.0 sec.
TEST-T> write_testbench -input pat.stil -output maxtb -replace
#####
# #
# STIL2VERILOG #
# #
# Copyright (c) 2007-2013 SYNOPSYS INC. ALL RIGHTS RESERVED      #
# #
#####
maxtb> Parsing command line...
maxtb> Checking for feature license...
maxtb> Parsing STIL file "pat.stil" ...
... STIL version 1.0 ( Design 2005) ...
... Building test model ...
... Signals ...
... SignalGroups ...
... Timing ...
... ScanStructures : "1" "2" "3" "4" ...
... PatternBurst "_burst_" ...
... PatternExec ...
... Procedures : "multiclock_capture" "allclock_capture" "allclock_launch"
"allclock_launch_capture" "load_unload" ...
... MacroDefs : "test_setup" ...
... Pattern block "_pattern_" ...
maxtb> STIL file successfully interpreted (PatternExec: "").
maxtb> Detected a Normal Scan mode.
maxtb> Test bench files "maxtb.v" and "maxtb.dat" generated successfully.
maxtb> Memory usage: 5.15 Mbytes. CPU usage: 0.710 seconds.
maxtb> End.
TEST-T> ...

```

See Also

[Using write_testbench to Customize MAX Testbench Output](#)

LTRAN_SHELL

write_ydf

Overview

This command writes a Yield Explorer Data Format (YDF) file.

Syntax

```
write_ydf
  file_name
  [-append | -replace]
  [-all | -candidates]
  [-apply_match_names]
  [-branch]
  [-bridge]
  [-cell]
  [-cell_pin]
  [-cell_instance_pin_net]
  [-chain_def]
  [-custom_field file_name]
  [-device name]
  [-diex number]
  [-diey number]
  [-fab fab_name]
  [-include_scan_path | -noinclude_scan_path]
  [-instance_cell]
  [-lot number]
  [-nets]
  [-net_contact_position]
  [-net_layer]
  [-net_path]
  [-replace]
  [-test_condition {condition_list}]
  [-version string]
  [-via]
  [-via_position]
  [-wafer number]
```

Arguments

file_name

Specifies the file name of the Yield Explorer Data Format (YDF) file.

-append | -replace

The **-append** option appends the YDF data to an existing YDF file. If a YDF file does not exist, it is created. The **-replace** option replaces an existing YDF file. There is no default; you must specify one of these two options if there is an existing YDF file.

-all | -candidates

The `-all` option writes physical information for all instances and nets. The `-candidates` option writes data only for diagnostics candidates. If you specify any table name, except for an EDCT table, all physical information is written for the specified tables by default.

The following example generates Via and Cell physical data tables for all elements:

```
TEST-T> write_ydf file.ydf -cell -via -replace
```

The previous example is equivalent to the following example:

```
TEST-T> write_ydf file.ydf -cell -via -replace -all
```

If you don't specify a table name, all tables are written with only the physical information for the diagnostics candidates. The following example generates Via, Cell, CellPin, InstanceCell, CellInstancePinNet, Net, NetPath, ViaPosition, NetLayer, EDCT, ChainDef, Branch, and Bridge physical data tables for diagnostic candidates:

```
TEST-T> write_ydf file.ydf -replace
```

In this case, the default is the `-candidates` option. The command in the previous example is equivalent to the following command:

```
TEST-T> write_ydf file.ydf -candidates -replace
```

-apply_match_names

Applies match name rules to instance and net names, and prints the corresponding values in YDF. This option is applicable only if you also specify the [set_match_names](#) command.

-branch

Writes a subnet information table. You must also specify the `-candidate` option when using this option.

-bridge

Writes a bridge information table for the physical diagnostics flow. You must also specify the `-candidate` option when using this option.

-cell

Writes a cell information table.

-cell_instance_pin_net

Writes an information table for cells, instances, pins, and nets.

-cell_pin

Specifies a Yield Explorer data segment that reports the pin geometries for individual MACRO cells defined in the LEF file.

-chain_def

Writes a table that describes the chains in a design.

-custom_field file_name

Specifies the name of a custom fields file that contains descriptions of custom fields to add to the DFTCandidate table. The format of this file is the same as a schema file (see the description of the `-schema` option of the `set_ydf` command).

For example, to add equipment information to the DFTCandidate table of the YDF file:

1. Edit the schema file with the text shown in the following example:

```
Equipment STRING
```

2. Add "Equipment" as a custom field in file, called "custfields.txt," for example:

```
.header
Equipment ATE2
.end_header
```

3. Specify the schema file, using the following command:

```
set_ydf -schema schema_file_name
```

4. To use the data from the custom fields file in the YDF file, specify the following command:

```
write_ydf tester.ydf -custom_field custfields.txt -replace
```

-device *name*

Specifies the device name in the DFTCandidate table.

-diex *number*

Specifies the die X value in the DFTCandidate table.

-diey *number*

Specifies the die Y value in the DFTCandidate table.

-fab *fab_name*

Identifies where a chip is fabricated in the header of the DFTCandidate table.

-include_scan_path | -noinclude_scan_path

The `include_scan_path` option (the default) enables TestMAX ATPG to write out the intermediate logic between scan cells in the ChainDefinition table. This option must be enabled with the `-chain_def` option.

-instance_cell

Writes an information table for instances and cells.

-lot *number*

Specifies the lot number in the DFTCandidate table.

-nets

Writes a net information table.

-net_contact_position

Reports all via, center, X and Y coordinates for a given net.

-net_layer

Writes a net layer information table.

-net_path

Writes a net path information table.

-replace

Replaces an existing Yield Explorer data format (YDF) file.

-test_condition {*condition_list*}

Adds a description of the specified TestCondition row to the header of the DFTCandidate table. In the following example, the TestConditions row with the value "VDD, FREQUENCY" is added to the YDF table header:

```
write_ydf file.ydf -test_condition {VDD FREQUENCY} -replace
```

-version *string*

Specifies the version number in the DFTCandidate table. You can specify any alphanumeric string as an argument.

-via

Writes a via information table.

-via_position

Writes a net contact information table.

-wafer *number*

Specifies the wafer number in the DFTCandidate table.

Allowed Command Modes

All

Description

This command writes a Yield Explorer Data Format (YDF) file, which contains diagnostics results in a DFTCandidate table. Starting with the E-2010.12 release, Yield Explorer reads the output from the `write_ydf` command.

Following each `run_diagnosis` command, you should specify the `write_ydf` command and include the name of the YDF file without any options. This command writes the diagnostics candidates table, and all other tables, with the physical data for the candidates. This data enables Yield Explorer to perform volume diagnostics analysis. The following example reports the physical data elements for the diagnostics candidates:

```
write_ydf top_chip.ydf -candidates -append
```

In some cases, you might want to write individual tables to a specific file. If you specify one or more physical data options, TestMAX ATPG reports only the physical data tables related to the specified options.

If you specify the following example, TestMAX ATPG reports only the physical data for the vias and the LEF macro cells in their respective data segments:

```
write_ydf top_chip.ydf -replace -via -cell
```

You need to write the chain definition table only once using the following command:

```
write_ydf top_chip.chain_def.ydf -chain_def
```

This command writes the chain definition data for the entire database.

The DFTCandidate table contains the following columns:

`FileName` — Failure log file name without the hierarchy.

`NumDefects` — Total number of defect groups for the last [run_diagnosis](#) run.

`NumFaults` — Total number of faults for the last [run_diagnosis](#) run.

DefectNum — Defect number reported in the current line.

FaultNum — Number of the fault reported in the current line; equivalent faults show the same number. This number helps identify the fault when there are several faults for the same defect.

FaultModel — Model used to identify the fault.

FaultType — Aligned with the **FaultModel** column. Entries can be SA0, SA1, STR, STF, and so forth.

Equiv — Reports a master fault or an equivalent fault.

Pin — Pin of the fault candidate (only the cell port, Instance/Pin=pinpath).

Cell — Library cell type; for example, FFD2I

SubNet — Subnet identifier if the fault is a subnet fault; empty if unknown

Score — Diagnosis match score based on the Tests Fail Simulation Fail (TFSF), Tests Fail Simulation Pass (TFSP), and Test Pass Simulation Fail (TPSF) results.

Chain — Chain name if a chain diagnosis is performed.

ScanCellPosition — Scan cell position if a chain diagnosis is performed.

TestPatternSet — Name of the test pattern set. In the case of split patterns, the name of the last split pattern file

NumPatternsPass — Total number of passing patterns as indicated by the failures log file

NumPatternsFail — Total number of failing patterns as indicated by the failures log file

NumTestsPassSimulationFail — TPSF score

NumTestsFailSimulationPass — TFSP score

NumTestsFailSimulationFail — TFSF score

CellFault — Indicates if the line reports a net or a cell fault. This column is empty if it cannot be determined (for example, a non-combinational logic cell).

Instance — Instance of the fault candidate.

Net — Net of the fault candidate if netlist data is available.

NumDefectFailPatterns — Total number of failing patterns for a given defect group. This column is empty when running chain diagnosis.

NumDefectPassPatterns — Total number of passing patterns for a given defect group. This column is empty when running chain diagnosis.

NumExplainPassPatterns — Total number of passing patterns that are explained by the fault.

NumFailures — Total number of failures in a data log.

NumFailuresDiagnosed — Number of failures used by TestMAX ATPG during the diagnosis.

NumFaultyChains — Number of faulty chains for a data log if a defect is in a chain.

`NumPartialExplainFailPatterns` — Total number of failing patterns that are partially explained by fault.

`NumPerfectExplainFailPatterns` — Total number of failing patterns that are perfectly explained by fault

`NumUnmappedPatterns` — Number of patterns that are unmapped and skipped for diagnosis. This is the same information that `#skipped_pats` prints in the report

`DiagTool` — Diagnosis tool name (TestMAX ATPG).

`DiagToolVersion` — Full name of TestMAX ATPG version.

`DiagnosticDate` — Date and time of the start of the TestMAX ATPG run: YYYY.MM.DD HH:MM:SS

`ShiftClock` — Clock name of the last faulty scan chain if the defect is in a chain.

`Truncate` — If the data log is truncated, this column will display a Y; If the data log is not truncated, this column will display an N or is empty.

`Lot` — Lot data from headers or another source; this column is empty by default.

`Wafer` — Wafer data from headers or another source; this column is empty by default.

`DieX` — DieX data from headers or another source; this column is empty by default.

`DieY` — DieY data from headers or another source; this column is empty by default.

If the diagnostics run fails, the `DFTCandidate` table contains only the `FileName`, `TestPatternSet`, `DiagTool`, `DiagToolVersion`, `DiagnosticDate`, `Lot`, `Wafer`, `DieX`, and `DieY` fields, and any other data from the header schema file specified by the [set_ydf](#) command.

Examples

```
write_ydf top_chip.all.ydf

write_ydf top_chip.ydf

write_ydf -candidates -cell top_chip.ydf

write_ydf top_chip.Cell.ydf -all -replace -cell

write_ydf top_chip.CellPin.ydf -all -replace -cell_pin

write_ydf top_chip.InstanceCell.ydf -all -replace -instance_cell

write_ydf top_chip.CellInstancePinNet.ydf -all -replace -cell_instance_pin_net

write_ydf top_chip.NetPath.ydf -all -replace -net_path

write_ydf top_chip.ViaPosition.ydf -all -replace -via_position

write_ydf top_chip.NetLayer.ydf -all -replace -net_layer
```

```
write_ydf top_chip.Via.ydf -all -replace -via
```

See Also[set_ydf](#)[report_physical](#)[report_layout](#)

Other Commands

This section describes the following commands:

- [alias](#)
- [build](#)
- [close_physical_db](#)
- [define_proc_attributes](#)
- [drc](#)
- [exit](#)
- [extract_nets](#)
- [extract_pins](#)
- [gsv_print](#)
- [help](#)
- [man](#)
- [match_names](#)
- [open_physical_db](#)
- [parse_proc_arguments](#)
- [Predefined Aliases](#)
- [quit](#)
- [refresh_schematic](#)
- [reorder_patterns](#)
- [reset_state](#)
- [test](#)

alias

Overview

This command defines an alternative symbolic name for a command and potential command arguments and switches.

Syntax

```
alias  
  [<alias_name> [<alias_text>]
```

Arguments

<alias_name>

The symbolic name of the alias to be defined or reported. If no <alias_name> is provided, then all currently defined aliases are reported.

<alias_text>

The replacement text to use when an alias is entered as a command. The first word of the <alias_text> must match a recognized command name. If no <alias_text> is provided, the alias corresponding to <alias_name> is reported. An alias can be defined as multiple command lines by separating the individual commands with a semicolon ';' surrounded by spaces. However, a multi line command cannot contain a source command.

Allowed Command Modes

All

Description

This command defines an alternative symbolic name for a command and potential command arguments and switches. This alias can then be used in place of the original command. A number of aliases are [predefined](#) upon tool startup. A few rules for defining and using aliases:

Aliases must be entered exactly as defined, including the same use of uppercase and lowercase characters. There is no abbreviation for aliases or case independence.

An alias can be defined with the same name as a command. In this case the alias has precedence over the command.

An alias cannot be defined with the same name as an existing alias. To modify an alias definition, first remove it with the `unalias` command.

More than one alias to the same command can be defined.

Expansion of aliases is done only once before performing command recognition. Use of alias definitions which reference other aliases will not work, given this restriction.

An alias can be defined to execute multiple commands by separating the commands with a semicolon ';' . A space must occur before and after the semicolon.

An alias that is defined using multiple commands cannot contain a reference to the `source` command.

Examples

```
BUILD> alias restart set pat -del ; build -force  
BUILD> alias  
alias name alias text
```

```
-----  
restart set pat -del ; build -force
```

```
DRC> alias
```

```
alias name alias text
```

```
-----  
readnet read_netlist -delete
```

```
restart build -force
```

```
compile run_build_model
```

```
exit exit -force
```

```
init_Faults add_faults -all
```

```
MAKE_PATTERNS run_atpg
```

```
DRC> alias exit
```

```
alias name alias text
```

```
-----  
exit exit -force
```

See Also

[unalias](#)

[Predefined Aliases](#)

build

Overview

Use this command to return a session to BUILD command mode to run commands that can only be applied while in BUILD command mode.

Syntax

```
build  
  [-force]
```

Arguments

-force

Used to force TestMAX ATPG into BUILD mode from any other mode, regardless of the presence of unsaved patterns or simulation models.

Allowed Command Modes

DRC,Test

Description

Use this command to return a session to BUILD command mode to run commands that can only be applied while in BUILD command mode. A simulation model that was previously generated is deleted.

Examples

```
DRC> BUILD  
Warning: Internal pattern set is now deleted.  
BUILD>  
  
TEST> build -force  
Warning: Internal pattern set is now deleted. (M133)  
Warning: Current simulation model is now deleted. (M39)  
BUILD>
```

See Also

[drc](#)
[test](#)

close_physical_db

Overview

This command closes the server process that accesses physical data from a PHDS database for physical diagnostics.

Syntax

```
close_physical_db  
    [-verbose]
```

Arguments

-verbose

Prints detailed information on the server process while closing the data access to the PHDS database.

Description

This command closes the server process that accesses physical data from a PHDS database for physical diagnostics. You can start the server process, called the DAPListener process, using the [open_physical_db](#) command.

For more information on using the `close_physical_db` command, see the "[Starting and Stopping a PHDS Server](#)" section in the *TestMAX ATPG User Guide*.

See Also

[set_physical_db](#)

[open_physical_db](#)

define_proc_attributes

Overview

The elements specify the following information:

arg_name is the name of the argument.

Syntax

```
string define_proc_attributes
    proc_name
    [-info info_text]
    [-define_args arg_defs]
    [-command_group group_name]
    [-hide_body]
    [-hidden]
    [-dont_abbrev]
    [-permanent]
```

Arguments

proc_name

Specifies the name of the existing procedure.

-info *info_text*

Provides a help string for the procedure. This is printed by the help command when you request help for the procedure. If you do not specify *info_text*, the default is "Procedure".

-define_args *arg_defs*

Defines each possible procedure argument for use with `help -verbose`. This is a list of lists where each list element defines one argument.

-command_group *group_name*

Defines the command group for the procedure. By default, procedures are placed in the "Procedures" command group.

-hide_body

Hides the body of the procedure from `info body`.

-hidden

Hides the procedure from `help` and `info proc`.

-dont_abbrev

Specifies that the procedure can never be abbreviated. By default, procedures can be abbreviated, subject to the value of the `sh_command_abbrev_mode` variable.

-permanent

Defines the procedure as permanent. You cannot modify permanent procedures in any way, so use this option carefully.

Description

This command associates attributes with a Tcl procedure. These attributes are used to define help for the procedure, locate it in a particular command group, and protect it.

When a procedure is created with the `proc` command, it is placed in the Procedures command group. There is no help text for its arguments. You can view the body of the procedure with `info body`, and you can modify the procedure and its attributes. The `define_proc_attributes` command allows you to change these aspects of a procedure.

Note that the arguments to Tcl procedures are all named, positional arguments. They can be programmed with default values, and there can be optional arguments by using the special argument name `args`. The `define_proc_attributes` command does not relate the information that you enter for argument definitions with `-define_args` to the actual argument names. If you are describing anything other than positional arguments, it is expected that you are also using `parse_proc_arguments` to validate and extract your arguments.

The `info_text` is displayed when you use the `help` command on the procedure.

Use `-define_args` to define help text and constraints for individual arguments. This makes the help text for the procedure look like the help for an application command. The value for `-define_args` is a list of lists. Each element has the following format:

```
arg_name option_help value_help data_type attributes
```

The elements specify the following information:

arg_name is the name of the argument.

option_help is a short description of the argument.

value_help is the argument name for positional arguments, or a one word description for dash options. It has no meaning for a Boolean option.

data_type is optional and is used for option validation. The *data_type* can be any of: string, list, boolean, int, float, or one_of_string. The default is string.

attributes is optional and is used for option validation. The *attributes* is a list that can have any of the following entries:

"required" - This argument must be specified. This attribute is mutually exclusive with optional.

"optional" - Specifying this argument is optional. This attribute is mutually exclusive with "required."

"value_help" - Indicates that the valid values for a one_of_string argument should be listed whenever argument help is shown.

"values {<list of allowable values>}" - If the argument type is one_of_string, you must specify the "values" attribute.

"merge_duplicates" - When this option appears more than once in a command, its values are concatenated into a list of values. The default behavior is that the right-most value for the option specified is used.

"remainder" - Specifies that any additional positional arguments should be returned in this option. This option is only valid for string option types, and by default the option is optional. You can require at least one item to be specified by also including the required option.

The default for *attributes* is "required."

Change the command group of the procedure using the `-command_group` command. Protect the contents of the procedure from being viewed by using `-hide_body`. Prevent further modifications to the procedure by using `-permanent`. Prevent abbreviation of the procedure by using `-dont_abbrev`.

Examples

The following procedure adds two numbers together and returns the sum. For demonstration purposes, unused arguments are defined.

```
prompt> proc plus {a b} { return [expr $a + $b]}

prompt> define_proc_attributes plus -info "Add two numbers" \
? -define_args {
  {a "first addend" a string required}
  {b "second addend" b string required}
  {"-verbose" "issue a message" "" boolean optional}}

prompt> help -verbose plus
Usage: plus # Add two numbers
[-verbose] (issue a message)
a (first addend)
b (second addend)

prompt> plus 5 6
11
```

In the following example, the `argHandler` procedure accepts an optional argument of each type supported by `define_proc_attributes`, then displays the options and values received:

```
proc argHandler {args} {
  parse_proc_arguments -args $args results
  foreach argname [array names results] {
    echo $argname = $results($argname)
  }
}

define_proc_attributes argHandler \
-info "Arguments processor" \
-define_args {
  {-Oos "oos help" AnOos one_of_string
    {required value_help {values {a b}}}}
  {-Int "int help" AnInt int optional}
```

```
{-Float "float help" AFloat float optional}
{-Bool "bool help" "" boolean optional}
{-String "string help" AString string optional}
{-List "list help" AList list optional}
{-IDup "int dup help" AIDup int {optional merge_duplicates}}
}
```

drc

command_summary

Syntax

```
drc  
  [-force]
```

Arguments

-force

Avoids the warning prompt that would otherwise occur when there are any unsaved patterns. All current patterns are deleted.

Allowed Command Modes

All

Description

Use this command to return a session to DRC command mode to run commands that can only be applied while in DRC command mode. Any patterns that were previously generated are deleted.

Note: You can also use the `run_build_model` command to transition to DRC command mode when you are in BUILD command mode.

Examples

```
TEST> drc -force  
Warning: Internal pattern set is now deleted.  
DRC>
```

See Also

[build](#)
[test](#)

exit

Overview

Use this command to terminate the current session and return to the windows or shell environment.

Syntax

```
exit  
    [-force]  
    [integer]
```

Arguments

-force

Avoids the warning prompt that would otherwise occur upon exit when there are any unsaved patterns. **Note:** In Tcl mode, the `-force` option is built in by default. Therefore, you should not use this option in Tcl mode.

integer

Returns the specified *integer* value to the exit calling process. By default, TestMAX ATPG returns 0 to the calling process when the `exit` command is specified.

Allowed Command Modes

All

Description

Use this command to terminate the current session and return to the windows or shell environment.

You can also use the `quit` command to terminate the current session.

Examples

```
DRC> exit
```

```
TEST> exit -force
```

```
TEST-T > exit
```

See Also

[quit](#)

extract_nets

Overview

This command extracts the subnet information for the nets of a design from a PHDS database.

Syntax

```
extract_nets
  [-all]
  [-keep_physical | -nokeep_physical]
  [-names {driver_pins}]
  [-verbose]
```

Arguments

-all

Extracts the subnet topology for all nets in the design.

-keep_physical | -nokeep_physical

The `-keep_physical` option (the default) extracts and saves the physical information for each net and every cell connected to the net. The `-nokeep_physical` option excludes this information during extraction.

-names {driver_pins}

Extracts subnet information for nets based on the specified name(s) of a driver pin or pins. You can print the extracted information using the [report_physical](#) `-net` command.

-verbose

Prints a verbose report of the subnet extraction.

Allowed Command Modes

All

Description

This command extracts the subnet information for the nets of a design from a PHDS database.

For extractions with a long runtime, this command reports incremental 15-minute status updates of the extraction process, which include the current percentage of extracted nets.

For details on using this command, see "[Static Subnet Extraction Using a PHDS Database](#)."

Examples

```
TEST-T> extract_nets -all
Connecting to physical database.
Successfully connected to physical database.
Continuing extraction, 9.40% of extractions completed
Continuing extraction, 18.79% of extractions completed
Continuing extraction, 28.62% of extractions completed
Continuing extraction, 38.01% of extractions completed
...
```

```
TEST-T> extract_nets -all -verbose
Connecting to physical database.
Successfully connected to physical database.
Setting top-level physical design name to 'RISC_CHIP'
Extracting all net geometries and subnets
Driver Pins: I_RISC_CORE/I_DATA_PATH/Oprnd_B_reg_15_/Q
Driver Pins: I_RISC_CORE/I_STACK_TOP/I2_STACK_MEM/PopDataOut_reg_2_/Q
...
Net Instance Pins: I_RISC_CORE/I_ALU/DP_OP_47_296_4134_U41/A1
Net Instance Pins: I_RISC_CORE/I_ALU/DP_OP_47_296_4134_U41/A2
...
Could not find valid module for net pin 'I_RISC_CORE/buffd7_G2B1I3/Z'.
Could not populate YDF data for pin 'I_RISC_CORE/buffd7_G2B1I3/Z'.
...
Successfully retrieved and populated net data.
Successfully annotated pin location data.
Writing subnet information for net I_RISC_CORE/Oprnd_B[15] (4)...
Writing subnet information for net I_RISC_CORE/Return_Addr[6] (3)...
...
Read_subnets completed: #nets(total/added/overwritten)=280/280/0, CPU_
time=0.00
Closing connection to physical database.
CPU_time = 1.28
```

See Also

[set_diagnosis](#)

[run_diagnosis](#)

[set_physical_db](#)

[open_physical_db](#)

[close_physical_db](#)

extract_pins

Overview

This command extracts physical information from a physical data store (PHDS) database for the pins of a design.

Syntax

```
extract_pins
  [-connected_pins | -noconnected_pins]
  [-names {pin_names}]
  [-scan_path all | scan_path]
  [-verbose]
```

Arguments

-connected_pins | -noconnected_pins

Extracts all pins connected to nets along the scan path specified by the `-scan_path` option. Use the `-connected_pins` option with the `-verbose` option of the [report_scan_path](#) command when you want to report the locations of additional pins connected to other fanouts on a scan path net. The default is `-noconnected_pins`.

-names {pin_names}

Extracts physical information for the specified pins. You can print the extracted information using the [report_physical](#) -pin command.

-scan_path all | scan_path

Extracts instance bounding box and pin location data for either all scan paths (the `all` argument) or for all pins along a specified scan path (the `scan_path` argument).

-verbose

Prints additional physical extraction information for pins.

Allowed Command Modes

TEST

Description

This command extracts physical information from a physical data store (PHDS) database for the pins of a design. For more information, see "[Using Physical Data for Diagnosis](#)."

See Also

[extract_nets](#)
[report_physical](#)
[report_scan_cells](#)
[set_physical_db](#)

gsv_print

Overview

This command creates a grayscale PostScript file, which captures the schematic displayed in the graphical schematic viewer (GSV).

Syntax

```
gsv_print  
-file filename -banner string
```

Arguments

-file *filename*

Specifies the name of output PostScript file.

-banner *string*

Prints a user-defined string.

Description

This command creates a grayscale PostScript file, which captures the schematic displayed in the graphical schematic viewer (GSV).

You can add the `gsv_print` command into the TestMAX ATPG scripts and capture schematic output automatically. The `computman_writing` host must have PostScript drivers installed (usually with `lpr/lp`). You can enclose the arguments in double quotation marks (“ ”).

gui_start

Overview

This command starts the TestMAX ATPG GUI.

Syntax

```
gui_start
```

Description

In the shell mode, enter the following command to display the TestMAX ATPG GUI in its current state:

```
gui_start
```

The TestMAX ATPG GUI appears. The context is switched to listen only to the GUI console. To exit the GUI, use the `gui_stop` command.

gui_stop

Overview

This command stops the TestMAX ATPG GUI session and reverts to the TestMAX ATPG shell command prompt.

Syntax

`gui_stop`

After you start the TestMAX ATPG GUI (using the `gui_start` command), enter the following command to exit the GUI:

`gui_stop`

Description

This command stops the TestMAX ATPG GUI session and reverts to the TestMAX ATPG shell command prompt. If you did not use the `gui_start` command to start the GUI, the `gui_stop` command exits the TestMAX ATPG application. You can also use the `gui_stop` command from the pulldown menu File > Exit GUI. If you use the `gui_stop` command before invoking TestMAX ATPG using the `gui_start` command, TestMAX ATPG displays an error message.

help

Overview

This command reports supported command switches and arguments for a specific command or a list of available commands if no arguments are provided.

Syntax

```
help
  [command_name]
  [-verbose]
  [-groups]
```

Arguments

command_name

Prints help information for the specified command. You can use any combination of the required characters. You can also use a single wildcard indicator '*' at the end of the command.

-verbose

Prints syntax information for all commands or the specified command.

-groups

Displays command groups only.

Allowed Command Modes

All

Description

This command reports supported command switches and arguments for a specific command or a list of available commands if no arguments are provided. For more detailed descriptions of commands, syntax, and examples, use the `man` command.

Every command supports a `-help` switch so in addition to using `help add_clocks` you can specify a command such as `add_clocks -help`. However, commands can be executed only in certain modes and are illegal in other modes. For example, you can specify `help write\_patterns` at any time but you can type `write_patterns -help` only when it is legal to use a `write_patterns` command.

Examples

```
DRC> help
DRC> help -verbose

BUILD-T> help report_primitives
DRC> help report_primitives -verbose
report_primitives      # REPort PRimitives
  [-ports]             ()
  [-pis]               ()
  [-pos]               ()
  [-pios]              ()
  [-type]              ()
```

```
[-summary]          ()
[-max]              ()
[-pins]             ()
[-all]              ()
[-name]             (primitive ID number | instance_name | net_name |
pin_pathname
```

```
BUILD> help add_c*
  add_capture_masks      # Add CApture Masks
  add_cell_constraints    # Add CEll Constraints
  add_clocks             # Add CLocks
  add_compressors        # Add COmpressors
```

See Also

[report_commands](#)

[man](#)

[List of Available Commands](#)

man

Overview

command_summary

Syntax

```
man  
[<command> | <rule_id> | <rule_violation_id> | <message_id>  
 | commands | getting_started | fault_classes | stil_examples  
 | ram_examples]
```

Arguments

command

Specifies a command for which online help information is displayed.

rule_id

Specifies the name of a rule for which online help information is displayed.

rule_violation_id

Specifies the name of a rule violation for which online help information for the associated rule is displayed.

message_id

Specifies the ID number of a warning or error message for which online help information is displayed.

commands | getting_started | fault_classes | stil_examples | ram_examples

Indicates a specific topic for which online help information is displayed.

Allowed Command Modes

All

Description

This command is the linkage with the online help utility. This is an independent help viewing mechanism, that can be run by itself or called up during a session. If the online help is opened, the use of a `man` command causes it to display a corresponding reference page. If the online help is not already opened, a `man` command automatically starts it.

Note the following:

- This command is no functional in the 64-bit shell mode.
- Using this command with no arguments displays a command summary without starting the online help and is equivalent to a `help` command with no arguments.
- Every command supports a `-man` switch, so in addition to using `man add_clock` you could also do `add_clock -man`. However, commands can be executed only in certain modes and are illegal in other modes. For example, you can type `man write_patterns` at any time but you can type [write_patterns](#) `-man` only when it is legal to use a [write_patterns](#) command.

Examples

```
BUILD> man  
DRC> man add_clocks  
TEST> man getting_started
```

See Also

[help](#)

[Report Commands](#)

[List of Available Commands](#)

Table Column Outside Table:

match_names

Overview

This command builds a lookup table and establishes a link between logical and physical instances or pin names for physical diagnostics.

Syntax

```
match_names
  [-all]
  [-file file_name]
  [-max_errors number]
  [-report <unmatched | matched | all>]
  [-sample [percentage]]
  [-verbose]
```

Arguments

-all

Prints a summary report of all matched logical and physical instances in the current session. This option is the default.

-file *file_name*

Specifies a file that contains logical names to match to the corresponding physical names. Each line in this file is an individual logical driver pin name.

-max_errors *number*

Prints only the specified number of either modules, instances, or pins.

-report <unmatched | matched | all>

Specifies the type of names to include in the name matching report. The `unmatched` argument prints only the unmatched names and the `matched` argument prints matched names. The `all` argument, the default, prints both unmatched and matched names.

-sample [*percentage*]

Checks a single name for each level of hierarchy in the design against the physical design store (PHDS) database. If you include the percentage parameter, TestMAX ATPG matches the specified percentage of logical names.

-verbose

Prints additional name matching information, including a list of the mismatched modules and instances, the CPU time, and the memory usage.

Allowed Command Modes

DRC, TEST

Description

This command builds a lookup table and establishes a link between logical and physical instances or pin

names for physical diagnostics. It also reports name mismatches between the logical and physical databases. The `match_names` command ensures that the physical pin names are properly associated with the logical pin names and the subnet information is annotated to the logical view.

To list the current name matching rules, specify the [report_settings](#) `match_names` command.

For more information on using the `match_names` command, see "[Name Matching Using a PHDS Database](#)."

Examples

```
TEST-T> match_names
Setting top-level physical design name to 'RISC_CHIP'
Performing Pin Level Analysis
    Matched 564 of 884 instance pins
Checking for logical wrapper
Checking for physical wrapper
Checking for differences in the lowest hierarchy levels
Performing Hierarchy Level Analysis
    Module Inst Count Matched Unmatched Unmatched Names
-----
    STACK_TOP                1 0                1        320
-----
Name Match Summary
-----
Number of instance names matched: 564
Number of mismatches found: 320
Percent Correct = 63.80%
CPU_time: 0.02 sec
Query_time: 2.61 sec
Total_time: 2.62 sec
Memory usage summary: 0MB
-----
Closing connection to physical database.
```

For a complete description of this report, see "[Understanding the Name Matching Coverage Report](#)."

See Also

[set_match_names](#)
[report_physical](#)

open_physical_db

Overview

This command starts a server process that queries a PHDS (Synopsys Physical Design Store) database used for physical diagnostics.

Syntax

```
open_physical_db
```

Description

This command starts a server process that queries a PHDS (Synopsys Physical Design Store) database used for physical diagnostics. Before using this command, make sure a connection is established with the PHDS database. For example:

```
set_physical_db -database ./phds
set_physical_db -port_number 9990
open_physical_db
```

You can confirm the server process started by viewing a list of active jobs on the machine and search for a process with the name "DAPListener."

When you have completed physical diagnostics, you should close this server process using the [close_physical_db](#) command.

The specification of the port number (using the [set_physical_db -port_number](#) command) is optional before opening the physical database because the default, 9998, is used.

For more information on using the `open_physical_db` command, see the "[Starting and Stopping a PHDS Server](#)" section in the *TestMAX ATPG User Guide*.

See Also

[set_physical_db](#)

[close_physical_db](#)

parse_proc_arguments

Overview

The `parse_proc_arguments` command is used within a Tcl procedure to enable use of the `-help` option, and to support argument validation.

Syntax

```
string parse_proc_arguments
    -args arg_list
    result_array
list    arg_list
string  result_array
```

Arguments

-args *arg_list*

Specifies the list of arguments passed into the Tcl procedure.

result_array

Specifies the name of the array into which the parsed arguments should be stored.

Description

The `parse_proc_arguments` command is used within a Tcl procedure to enable use of the `-help` option, and to support argument validation. It should typically be the first command called within a procedure.

Procedures that use `parse_proc_arguments` will validate the semantics of the procedure arguments and generate the same syntax and semantic error messages as any application command (see the examples that follow).

When a procedure that uses `parse_proc_arguments` is invoked with the `-help` option, `parse_proc_arguments` will print help information (in the same style as using `help -verbose`) and will then cause the calling procedure to return. Similarly, if there was any type of error with the arguments (missing required arguments, invalid value, and so on), `parse_proc_arguments` will return a Tcl error and the calling procedure will terminate and return.

If you didn't specify `-help`, and the specified arguments were valid, the array variable `result_array` will contain each of the argument values, subscripted with the argument name. Note that the argument name here is not the names of the arguments in the procedure definition, but rather the names of the arguments as defined using the `define_proc_attributes` command.

The `parse_proc_arguments` command cannot be used outside of a procedure.

Example

The following procedure shows how `parse_proc_arguments` is typically used. The `argHandler` procedure parses the arguments it receives. If the parse is successful, `argHandler` prints the options or values actually received.

```
proc argHandler args {
    parse_proc_arguments -args $args results
```

```

foreach argname [array names results] {
    echo " $argname = $results($argname)"
}
}
define_proc_attributes argHandler -info "argument processor" \
    -define_args \
    {{-Oos "oos help" AnOos one_of_string {required value_help {values {a
b}}}}}
    {-Int "int help" AnInt int optional}
    {-Float "float help" AFloat float optional}
    {-Bool "bool help" "" boolean optional}
    {-String "string help" AString string optional}
    {-List "list help" AList list optional}}
    {-IDup int dup AIDup int {optional merge_duplicates}}}

```

Invoking `argHandler` with the `-help` option generates the following:

```

prompt> argHandler -help
Usage: argHandler # argument processor
      -Oos      (oos help:
                  Values: a, b)
      [-Int AnInt]      (int help)
      [-Float AFloat]  (float help)
      [-Bool]          (bool help)
      [-String AString] (string help)
      [-List AList]    (list help)

```

Invoking `argHandler` with an invalid option causes the following output (and a Tcl error):

```

prompt> argHandler -Int z
Error: value 'z' for option '-Int' not of type 'integer' (CMD-009)
Error: Required argument '-Oos' was not found (CMD-007)
Invoking argHandler with valid arguments generates the following:
prompt> argHandler -Int 6 -Oos a
      -Oos = a
      -Int = 6

```

Predefined Aliases

Several predefined aliases are available when you invoke TestMAX ATPG using a Synopsys-supplied setup or startup command file at the following location:

```
$SYNOPSYS/admin/setup/tmaxtcl.rc
```

To prevent this command setup file (and all setup files) from being used, invoke TestMAX ATPG with the `-nostartup` option. For more information, see "[Setup Command Files](#)."

The following table is a summary of all predefined aliases:

Alias	Behavior
<code>read_verilog</code>	Same as the <code>read_netlist</code> command
<code>read_edif</code>	Same as the <code>read_netlist</code> command
<code>read_vhdl</code>	Same as the <code>read_netlist</code> command
<code>read_test_protocol</code>	Same as the <code>run_drc</code> command
<code>add_set</code>	Same as the <code>add_clocks</code> command
<code>add_reset</code>	Same as the add_clocks command
<code>report_alias</code>	Same as the alias command
<code>history</code>	Same as the report_commands <code>-history</code> command
<code>include</code>	Same as the <code>source</code> command
<code>link</code>	Same as the <code>run_build_model</code> command
<code>create_test_clock</code>	Same as the add_clocks command
<code>create_test_patterns</code>	Same as the run_atpg <code>-auto</code> command
<code>read_init_protocol</code>	Same as the set_drc command
<code>read_test_protocol</code>	Same as the run_drc command
<code>set_test_hold</code>	Same as the add_pi_constraints command
<code>write_test</code>	Same as the <code>write_patterns</code> command
<code>reset_build</code>	Resets to default BUILD state, deletes everything
<code>reset_drc</code>	Resets to default DRC state, deletes clocks, DRC file
<code>reset_test</code>	Resets to default TEST state, deletes patterns
<code>reset_all</code>	Resets to default invoke state, deletes everything

The following alias simplifies the generation of the template configuration file for the [stil2verilog](#) command:

```
alias write_testbench_config "write_testbench -parameters {-generate_config  
stil2verilog.config} -replace"
```

quit

Overview

Use this command to terminate the current session and return to the windows or shell environment.

Syntax

```
quit  
  [-force]
```

Arguments

-force

Avoids the prompt that occurs at exit if there are any current unsaved patterns.

Allowed Command Modes

All

Description

Use this command to terminate the current session and return to the windows or shell environment.

Note: You can also use the `exit` command to terminate the current session.

Examples

```
DRC> quit
```

```
TEST> quit -force
```

See Also

[exit](#)

refresh_schematic

Overview

Use this command to redraw the schematic.

Syntax

```
refresh_schematic
```

Arguments

None

Allowed Command Modes

All

Description

Use this command to redraw the schematic. Any changes to the pin data display format will appear in the redrawn image.

Examples

```
DRC> refresh_schematic
```

See Also

[set_pindata](#)

Table Column Outside Table:

reorder_patterns

Overview

This command enables you to remove and reorder individual patterns and blocks of patterns as part of the DFTMAX Ultra flow.

Syntax

```
reorder_patterns  
  [-remove {pattern_num pattern_num} | -insert {pattern_num pattern_num}]
```

Arguments

-remove {pattern_num pattern_num}

Removes individual patterns or blocks of patterns. To remove one or more patterns, specify a list of one or more pattern numbers to remove. The following command removes patterns 3 and 9:

```
reorder_patterns -remove {3 9}
```

You can specify blocks of patterns by providing the first and last pattern numbers of the block as a sublist inside the pattern removal list. You can mix individual pattern numbers and pattern blocks in the list.

The following command removes patterns 5 through 7:

```
reorder_patterns -remove {{5 7}}
```

The following command removes patterns 3, 5 through 7, and 9:

```
reorder_patterns -remove {3 {5 7} 9}
```

-insert {pattern_num pattern_num}

Use this option with the `-remove` option to remove patterns and reinsert them at a different location in the pattern set.

For each pattern number or block provided in the removal list, specify the pattern number where the removed patterns should be reinserted in the insertion list. For removed patterns that should not be reinserted, specify an insertion pattern value of X.

The following command removes pattern 3, and reinserts patterns 4 through 6 at pattern 0:

```
reorder_patterns -remove {3 {4 6}} -insert {X 0}
```

The `-remove` and `-insert` options reference the original pattern numbers that existed before pattern removal or insertion is performed by the `reorder_patterns` command.

You can issue multiple `reorder_patterns` commands as needed. Each command resequences the patterns and pattern numbers after performing the specified pattern manipulations. Subsequent `reorder_patterns` commands must refer to the resequenced pattern numbers.

You can reorder chain test patterns, stuck-at patterns, and transition fault patterns. However, you cannot perform reordering operations that mix these pattern types. Attempting to do so results in an error, as shown in the following example:

Error: Pattern 2 and 13 are not of same type. Reordering is possible
between patterns of same type only
Transition patterns are 6 to 99
Chain test patterns are 1 to 5

Allowed Command Modes

DRC, Test

Description

This command enables you to remove and reorder individual patterns and blocks of patterns as part of the DFTMAX Ultra flow. For more information, see "[Using TestMAX ATPG and DFTMAX Ultra Compression](#)" in the *TestMAX ATPG User Guide*.

See Also

[set_patterns](#)

[write_patterns](#)

reset_state

Overview

Use this command to reset the session state in preparation for restarting fault simulation or test generation using the current fault list.

Syntax

```
reset_state
```

Arguments

None

Allowed Command Modes

Test

Description

Use this command to reset the session state in preparation for restarting fault simulation or test generation using the current fault list. Existing internal patterns are deleted and the faults are reset to the status they had when initially added to the fault list, except those faults whose status was changed to UD or AU remain at that status.

Examples

```
TEST> report_summaries
      Uncollapsed Fault Summary Report
-----
fault class          code #faults
-----
Detected             DT   83321
Possibly detected    PT    324
Undetectable         UD   1071
ATPG untestable      AU   3453
Not detected         ND    239
-----
total faults 88408
test coverage 95.59%
-----
Pattern Summary Report
-----
#internal patterns 1605
-----
TEST> reset_state
Warning: Internal pattern set is now deleted. (M133)
TEST> report_summaries
      Uncollapsed Fault Summary Report
-----
fault class          code #faults
-----
Detected             DT   11816
```

```
Possibly detected  PT    56
Undetectable       UD   1071
ATPG untestable    AU      3721
Not detected       ND   71744
-----
total faults 88408
test coverage 13.56%
-----
                Pattern Summary Report
-----
#internal patterns 0
-----
```

See Also

- [add_faults](#)
- [remove_faults](#)
- [read_faults](#)
- [report_faults](#)
- [update_faults](#)

source

Overview

Use this command for executing commands contained in an external file.

Syntax

```
source  
  file_name [-continue_on_error] [-echo] [-verbose]
```

Arguments

file_name

Specifies the path name of a file containing commands to be executed.

-continue_on_error

Causes the script to continue running even if errors have occurred.

-echo

Used during Tcl Mode, echoes the commands in the *file_name* and displays them in the GUI transcript or shell xterm.

-verbose

Displays all results, such as variable assignments, as they appear.

Allowed Command Modes

All

Description

Use this command for executing commands contained in an external file. The command history list will only contain this command and not include commands inside the [command file](#).

Note: Use the `set_commands` command to control error handling while executing commands from the command file as well as whether the commands are to appear in the transcript.

Examples

```
DRC> source noabort  
DRC> source my_setup.cmd
```

See Also

[Setup Command Files](#)

[Set Commands](#)

test

Overview

This command changes the session to TEST command mode, where fault simulation and test generation can be performed.

Syntax

```
test
```

Arguments

None

Allowed Command Modes

DRC

Description

This command changes the session to TEST command mode, where fault simulation and test generation can be performed. To enter TEST command mode, the DRC process must be successfully completed.

Note: You can also use the `run_drc` command to transition to TEST mode.

Examples

```
TEST> drc
Warning: Internal pattern set is now deleted.
DRC> test
-----
Begin scan design rule checking...
Begin simulating sg0 load_unload proc...
Begin clock rules checking...
      .
      .
      .
-----
DRC Summary Report
-----
Warning: Rule V16 (miscalculated formal parameters) failed 2 times.
Warning: Rule S19 (nonscan cell disturb) failed 2 times.
There were 4 violations that occurred during DRC process.
Design rules checking was successful, CPU time=0.66 sec.
-----
---
```

```
TEST>
```

See Also

[build](#)
[drc](#)
[run build_model](#)
[run_drc](#)

unalias

Overview

This command is used to remove alias definitions.

Syntax

```
unalias  
  <alias_name | -all>
```

Arguments

alias_name

Deletes the alias definition `alias_name`. If no alias exists with this name, you get an error message.

-all

Deletes all defined aliases.

Allowed Command Modes

All

Description

This command is used to remove alias definitions.

Examples

```
BUILD> alias
alias name  alias text
-----
readnet    read_netlist -delete
restart    build -force
compile    run_build_model
status     report_summaries
BUILD> unalias compile
BUILD> alias
alias name  alias text
-----
readnet    read_netlist -delete
restart    build -force
compile    run_build_model
status     report_summaries
```

```
BUILD> alias
alias name  alias text
-----
readnet    read_netlist -delete
restart    build -force
status     report_summaries
TEST> unalias -all
TEST> alias
alias name  alias text
-----
Warning: Requested report contained no entries. (M13)
```

See Also

[alias](#)